

操作系统第一册

从内核对象、进程与内存到文件系统、网络 and 恢复

CPU Books

本册系统讲解操作系统的对象、机制与边界，从执行流、系统调用和地址空间出发，依次进入内存管理、设备 I/O、文件系统、网络、安全、持久化与系统观测。硬件原理仅在解释内核机制所依赖的条件时引入，不另设硬件导论。

前言

操作系统不是一组系统调用名，也不是“进程、内存、文件、网络”这些名词的并列清单。它是一套把硬件资源变成可控抽象的工程协议：CPU 时间被包装成可调度实体，物理内存被包装成带权限的地址空间，设备被包装成文件和事件，崩溃风险被包装成日志、提交点和恢复规则。

本册是一部操作系统原理教材。它从一台没有操作系统的机器开始，但不重复数字电路、处理器数据通路和总线结构；这些内容由独立硬件卷完整讲解。本册只取已经成立的硬件事实，逐步追问：一项程序怎样装入，协调代码怎样常驻，处理器怎样从不合作的程序手中重新取得控制，多项执行怎样拥有可保存的身份，以及受限程序怎样请求内核服务。

正文不在开篇预先展示完整操作系统。它先让旧机器暴露一项具体失败，再增加解决该失败所需的最小机制，由此依次建立任务与调度、系统调用、程序映像、同步、地址空间、页缓存、内核分配器、设备请求、块层、VFS、文件系统、网络端点、安全、恢复与观测。各章都围绕状态对象、转换条件、不变量和失败路径组织内容。

当这些对象都已经获得明确定义，书末再沿一次文件保存请求完成全书综合：执行流如何被调度，用户请求如何进入内核，地址如何验证，文件对象如何引用，设备请求如何异步完成，数据又如何跨越持久化边界。此时全景不是术语清单，而是读者能够逐段核对的因果链。

本册通常先给出具体问题或反例，再分析直接方案的局限，继而建立相应机制、不变量与观察证据。读者不应只记住“进程是资源分配单位，线程是调度单位”这样的概括，而应能够回答：旧机器为什么失败，新增对象保存哪些状态，谁有权改变状态，错误路径如何退出，哪些证据能够支持结论。

本册以原理分析为主。正文中的观察与验证用于帮助读者检查机制边界；完整实现和规模化实验由后续实践卷展开。

核心思想

操作系统的每个特性都应该能回答三个问题：它依赖哪个硬件事实，它修复了前一阶段的哪个失败，又引入了哪些新的状态、成本和测试责任。

缩写与术语表

本表只收录本册反复出现的缩写。缩写不是背诵目标；它们的价值是帮助读者把较长的机制名称与正文中的状态对象和协议边界对应起来。

缩写	全称或常见含义	本册语境
ABI	Application Binary Interface	系统调用参数、调用约定、用户栈、ELF 装载契约
ACPI	Advanced Configuration and Power Interface	固件向内核描述 CPU、NUMA、设备和电源信息
ALU	Arithmetic Logic Unit	CPU 数据通路中执行整数算术和逻辑运算的部件
APIC	Advanced Programmable Interrupt Controller	x86 中断投递、本地定时器、IPI 和多核启动
PCID	Process Context ID	x86-64 TLB 中区分地址空间的标签，减少切换时全局刷新
BIO	Block I/O	块层从文件系统接收的页片段读写意图
BPF	Berkeley Packet Filter	seccomp、eBPF tracing、网络和安全钩子的执行载体
CFS	Completely Fair Scheduler	Linux 经典公平调度模型，以 <code>vruntime</code> 解释按权重分配 CPU
CFI	Control Flow Integrity	控制流保护，限制间接跳转和返回路径被劫持
CPL	Current Privilege Level	x86-64 当前特权级，决定是否可执行特权操作
COW	Copy-on-Write	<code>fork</code> 、 <code>MAP_PRIVATE</code> 和 COW 文件系统的共享后复制语义
CR/MSR	Control Register / Model-Specific Register	x86-64 控制寄存器和模型专用寄存器，保存页表根、系统调用入口和特权控制状态
DMA	Direct Memory Access	设备直接读写内存，要求映射、缓存一致性和生命周期管理
EEVDF	Earliest Eligible Virtual Deadline First	Linux 自 6.6 起逐步采用的公平调度策略，结合 lag 与虚拟截止期选择任务
EOI	End Of Interrupt	中断处理后通知控制器可以投递后续中断
FUA	Force Unit Access	块设备写入要求到达稳定介质，而非仅进入设备缓存
GDT	Global Descriptor Table	x86-64 早期保护和段描述结构，长模式下仍参与入口设置
GFP	Get Free Page flags	内核分配上下文，决定是否可睡眠、可 I/O、可回收
IDT	Interrupt Descriptor Table	x86-64 中断、异常和陷入入口表

IOMMU	I/O Memory Management Unit	设备 DMA 地址翻译和隔离
IPI	Inter-Processor Interrupt	CPU 间重调度、TLB shutdown、多核控制消息
IRQ	Interrupt Request	外设中断请求及其内核处理路径
ISA	Instruction Set Architecture	软件可见的指令、寄存器、异常、内存和特权契约
LSM	Linux Security Module	SELinux、AppArmor、BPF LSM 等安全钩子框架
MMU	Memory Management Unit	执行虚拟地址翻译和页表权限检查的硬件单元
MMIO	Memory-Mapped I/O	CPU 通过地址空间读写设备寄存器
NAPI	New API	Linux 网络收包中断转轮询机制，控制中断风暴
NUMA	Non-Uniform Memory Access	多插槽内存局部性、调度和页分配策略
OOM	Out Of Memory	内存耗尽、回收失败和 OOM killer 决策
PCIe	Peripheral Component Interconnect Express	现代外设互联，承载配置空间、BAR、MSI/MSI-X 和 DMA
PTE/PMD/PUD/PGD	Page Table Entry 等页表层级	多级页表、权限位、缺页和 TLB 失效
RCU	Read-Copy-Update	读路径低成本、写路径延迟释放的同步机制
RSS	Resident Set Size	进程实际驻留物理页，不等于虚拟地址空间大小
skb	socket buffer	Linux 网络栈中承载包数据和协议元数据的核心对象
SMP	Symmetric Multiprocessing	多核并发、per-CPU 数据、runqueue 和 IPI
TLB	Translation Lookaside Buffer	地址翻译缓存，页表修改后需失效旧项
VFS	Virtual File System	统一 superblock、inode、dentry、file 的文件系统抽象
VMA	Virtual Memory Area	进程虚拟地址区间策略，用于缺页判断和权限控制
WAL	Write-Ahead Log	先写日志再提交数据的崩溃恢复协议

怎样阅读本册

本册按照“机器演化、对象建立、状态转换、边界与证据”的顺序组织，而不是把术语并列罗列。开篇只有硬件、固件和一项待运行程序；只有当当前机器无法继续承担新的工作时，正文才引入一项内核能力。全书综合路径安排在正文之后，用来检验前面建立的对象能否共同解释一次文件保存。涉及具体体系结构时统一采用 x86-64；汇编统一采用 Intel 语法，目的操作数在前，源操作数在后，例如 `mov rax, qword ptr [rbx]`、`add rax, 1`、`syscall`、`iretq`。

第一部分回答“没有操作系统的机器怎样逐步获得受控执行”。从人工装入和常驻协调代码出发，依次建立强制返回、task、调度、系统调用边界、程序映像、等待队列和同步规则。每一节都应能指出旧机器的失败和新增机制的最小职责。

第二部分回答“地址如何获得含义、权限和生命周期”。虚拟内存、文件映射、页缓存和内核分配器共同决定页面何时建立映射、何时写回、何时回收，以及内存压力下如何保持状态一致。

第三部分回答“异步设备怎样形成统一文件接口”。驱动、DMA、块层和 VFS 沿着 request 身份、buffer 所有权、完成记录和对象引用逐层展开。

第四部分回答“可寻址块怎样成为可恢复的持久对象，通信又怎样成为可等待的端点”。文件系统格式、身份与索引、缓存与恢复、现代文件系统和网络栈共享对象、队列与完成边界，但持久化提交和网络确认不能相互混同。

第五部分回答“系统在攻击、崩溃和性能异常下如何保持可解释性”。安全隔离、跨层持久化、错误恢复与系统观测共同要求结论能够指向具体状态、权限边界、失败路径和可观察证据。

阅读阶段	先问的问题	不合格读法
机器演化与执行	旧机器为何失败，状态怎样进入和离开 runqueue、wait queue 与内核边界	只背调度算法或系统调用名称
地址与内存	何时验证、映射、写回和回收	把虚拟地址直接等同于物理内存
设备与持久对象	请求身份、buffer 所有权、完成记录和提交边界在何处	把 <code>write</code> 成功直接等同于持久化
安全与观测	哪条证据支持契约成立或失败	把工具输出直接当作结论

学习每章时都要落到五个问题：

1. 状态在哪里：进程表、runqueue、锁等待队列、页表、VMA、fd 表、socket、inode、page cache、日志还是设备队列。
2. 硬件在哪里：RIP、RSP、RFLAGS、通用寄存器、cache、TLB、页表、MMIO 寄存器、DMA 描述符、APIC/MSI-X 还是定时器。
3. 权限在哪里：CPU 特权级、页表权限、文件权限、capability、namespace 还是对象引用。
4. 等待在哪里：runnable 等 CPU、blocked 等事件、mutex 等持有者、page fault 等页面、socket 等缓冲、I/O 等 completion。
5. 提交在哪里：`exec` 切换地址空间、状态更新完成、写入 page cache、块设备 flush、目录持久化还是 manifest 生效。

每章可以按固定节奏读：先找失败场景，再看直接补丁为什么不够，再看 OS 机制如何建立状态对象，最后用不变量和证据收尾。若一节讲 fd，就要画出 fd table、

open file description、inode 和 page cache 的关系；若一节讲调度，就要写出 runnable、running、blocked、zombie 和 reaped 的状态转换；若一节讲锁，就要画出锁顺序图和等待队列；若一节讲 `exec`，就要标出失败回滚边界和提交点；若一节讲持久化，就要写出每个崩溃点后恢复程序应该相信什么。

阅读实现小节时，不能只看伪代码能不能运行。要追问四件事：数据结构是什么，复杂度是多少，维护了哪些不变量，失败路径是否会留下半提交状态。一个教学模型可以比真实内核小，但不能把真实问题讲错。

常见误区

不要把工具输出当成结论。工具只是证据来源；结论必须说明哪条 OS 契约被验证，哪条仍然只是合理假设。

目录

前言	ii
缩写与术语表	iii
怎样阅读本册	v
第一部分 从裸机到受控执行	1
第 1 章 一台能运行程序的机器为什么仍需要操作系统	3
第一项程序怎样在最小裸机上运行	3
为什么人工装入需要变成机器自己的工作	44
第二项任务为什么需要常驻协调代码	67
任务不交还处理器时怎么办	70
第 2 章 为什么执行流需要 task 与调度状态	76
进程资源、线程现场与任务状态	76
从一条执行流推导 task 的具体结构	78
两类现场：用户返回帧与调度切换帧	81
一次上下文切换究竟改变了什么	83
运行队列保存候选关系，等待队列保存条件关系	84
跨 CPU 迁移改变的是队列所有权	85
CPU 时间、等待时间与调度指标	86
调度策略与评价	87
阻塞、唤醒与上下文切换	90
调度案例：从执行轨迹到策略选择	94
公平、优先级与期限的不同承诺	97
多核调度：处理器位置也是调度结果	99
进程与线程生命周期的完整边界	101
本章不变量与综合推演	103
从教学调度器到可验证的多核实现	109
第 3 章 为什么用户请求需要受控入口	136
从一个无权动作得到受控入口	136
请求首先需要有一个稳定的边界契约	140
用户控制流怎样变成可恢复的入口现场	143
入口帧之后为何还要分发与逐层验证	145
用户地址为何必须转换成内核拥有的状态	148
对象引用怎样封闭检查与使用之间的竞态	151
权限判断为何需要独立的凭据快照	153
阻塞以后如何保留同一项系统调用	157
进入内核以后为何仍要区分执行上下文	159
边界观察为何也必须服从状态形成顺序	163
处理函数结束以后为何还不能立即返回	165
正确路径建立后怎样判断入口成本是否值得优化	169
把各阶段合成一项完整而可核对的请求	172
把入口语义应用到具体请求	176
分发、用户内存与部分完成	176
信号、重启与错误恢复	179
凭据、能力与安全检查	180
入口契约的实现证据	188

第 4 章 为什么程序字节还不是可运行映像	197
程序、进程与程序映像	197
exec 引起的完整状态替换	197
ELF 校验、段映射与用户栈	199
提交点、失败回滚与身份变化	201
动态链接、解释器脚本与辅助向量	202
第 5 章 为什么共享状态需要同步与通信对象	210
共享状态、竞争条件与同步目标	210
原子性、内存顺序与睡眠唤醒	211
锁、等待队列与 IPC	212
死锁、活性与优先级反转	215
读多写少、无锁读取与锁依赖	216
把同步规则落实为状态机	220
 第二部分 地址空间与内存	 234
第 6 章 为什么地址不能直接等同于物理位置	236
虚拟地址为何需要独立解释	236
VMA、页表项与物理页的职责	237
页面建立、复制、回收与换出	238
内存压力、回收失败与 OOM	242
巨页、交换与地址翻译成本	243
从缺页现场到可测试地址空间	251
第 7 章 为什么文件内容需要可共享的内存身份	272
文件内容如何进入地址空间	272
文件缺页、共享写入与私有复制	273
页缓存索引、预读与写回	274
截断、映射失效与一致性	276
直接 I/O、DAX 与同步边界	277
第 8 章 为什么内核还需要自己的分配器	284
内核分配面对的对象与上下文	284
从物理页到内核对象的分配过程	284
页框区域、连续性与碎片	285
buddy、slab 与分配上下文	286
泄漏、越界与对象生命周期	288
SLUB、内存压缩与资源归属	288
 第三部分 异步设备与文件接口	 291
第 9 章 为什么设备操作需要长期请求对象	293
异步设备请求与完成证据	293
从发现设备到安全移除	294
队列所有权与中断分工	295
描述符环、DMA 与内存顺序	295
超时、复位与迟到完成	298
设备模型、IOMMU 与虚拟设备	299

第 10 章 为什么设备请求需要公共块层与文件接口	306
块请求从何而来	306
合并、拆分与 I/O 调度	307
多队列、flush 与 FUA	308
限流、统计与错误完成	309
fd、file、dentry 与 inode	311
缓冲 I/O、直接 I/O 与异步完成	314
一次文件访问的跨层路径	318
短读写、取消与引用生命周期	319
VFS 与文件系统的持久化边界	319
从文件对象到持久块的综合验证	320
 第四部分 持久对象与网络端点	 327
第 11 章 文件系统的形成：从可寻址空间到持久对象	329
从持久字节到固定大小的块	329
空闲空间为何需要统一记录	330
文件为何需要独立于数据块的身份	330
名称为何必须与对象身份分离	330
重新挂载为何需要超级块	331
从职责关系得到最小磁盘布局	332
布局字段如何编码为磁盘字节	334
第 12 章 文件身份、空间索引与路径命名	342
inode 记录与文件生命周期	342
文件偏移到数据块：从指针到 extent 树	349
空闲空间、块组与分配局部性	353
名称索引：线性目录、hash 与 B-tree	357
VFS 对象与统一分发	359
第 13 章 缓存、写回与崩溃一致性	364
create、unlink、rename 与 truncate	364
页缓存、脏页与写回	368
崩溃一致性、日志与检查点	380
写时复制、快照与根切换	388
闪存约束、FTL 与日志结构	394
第 14 章 现代文件系统的组织与取舍	403
小文件、校验和与空间效率	403
文件系统设计维度与取舍	413
结构不变量与崩溃恢复检验	414
关键机制的运行路径	416
第 15 章 为什么网络通信需要持久端点	423
网络端点为何需要内核对象	423
bind、listen、accept 与 connect	423
发送缓冲、接收缓冲与背压	424
连接关闭、错误与重试边界	427
数据包从设备队列到套接字	428
网络事件、流量控制与提交边界	432

第五部分 安全、恢复与证据	439
第 16 章 为什么已经隔离仍不等于最小权限	441
从受控进程到最小权限	441
主体、对象、操作与审计	443
DAC、能力、沙箱与资源限制	443
namespace、容器与逃逸边界	444
LSM、seccomp 与系统加固	445
策略命中以后还需要可核对证据	448
第 17 章 为什么写成功仍不等于可恢复	456
易失状态与持久状态	456
从一次写入逐步得到持久化协议	456
文件发布、目录同步与 manifest	458
WAL、检查点与幂等恢复	459
撕裂写、校验和与介质故障	460
COW、RAID 与多层存储映射	460
第 18 章 为什么症状不能直接说明原因	473
错误、oops 与 panic	473
从症状建立证据链	474
指标、事件、调用链与转储	475
故障注入与因果验证	479
调度、内存、I/O 与网络诊断	479
调试工具与性能边界	484
全书综合：一次保存怎样穿过完整系统	491
能力清单	529
延伸阅读说明	531
参考资料与版本基线	532

第一部分

从裸机到受控执行

本部分导读

本部分从一台只有硬件、固件和一项待运行程序的机器开始。人工装入能够完成一次计算，却不能自动接收下一项任务；装入器能够交出处理器，却不能保留两项任务的中间状态；常驻协调代码能够切换现场，却无法从不合作的任务手中重新取得控制。每一次失败只增加一项必要能力，直到机器具有可保存的执行身份、可调度状态、受控内核入口、可替换程序映像和可等待的同步关系。

第 1 章 一台能运行程序的机器为什么仍需要操作系统

逻辑起点。机器只有复位、时钟、处理器、易失内存、非易失启动存储、最小互连和一项待运行任务。当前还没有 kernel、task、scheduler、system call 或文件。第一项任务要求把四个整数相加，并把结果 21 写到约定地址；每个后续结构都必须由这台机器已经暴露的具体失败推导出来。

第一项程序怎样在最小裸机上运行

先做一件可以亲眼核对的事。机器断开运行，复位保持有效；操作人员把一段程序和四个整数写入指定的存储位置，然后释放复位。机器完成计算以后，另一个指定位置应出现整数 21。这一次运行不依赖文件、命令行、进程或系统调用，没有任何操作系统替人安排步骤。

真正的问题不是“求和怎么算”，而是桌面上的一组器件怎样共同兑现这项要求。接通电源不会自动产生第一条指令；一串存储字节也不会自己变成正在进行的计算。我们必须先认清当前机器有哪些部件、它们之间传递什么，再逐步建立地址、程序字节和处理器状态的含义。

先把一次运行落实为几项人工动作

桌面上只有一块接好电源的电路板和一台能够向存储单元写入字节的调试工具。操作人员希望运行一段求和程序，输入是

7, -2, 11, 5,

预期结果是 21。在没有操作系统的条件下，这项要求首先表现为四项具体动作：

1. 保持复位，使处理器暂时不能执行普通指令；
2. 把程序字节、四个输入整数和最初需要的栈内容写到约定位置；
3. 设置完成后释放复位，让处理器从固定起点开始；
4. 等待程序停止或返回，再读取结果位置中的四个字节。

这里的调试工具只在机器停止时帮助写入 RAM。它不参与程序运行，也不是程序可以调用的设备。采用这种安排不是因为它方便，而是因为它暴露了最朴素的方法：人在运行之前把每个必要字节摆好，机器只负责执行。第二章才会处理这种人工装入为何难以长期使用。

“程序”这个名称暂时只表示一段按顺序解释的指令字节。“一次运行”则表示处理器从约定起点开始，根据这些字节反复更新自身状态和存储内容，直到到达约定的结束位置。此时还没有程序身份、进程状态或资源管理对象。

为什么不能只有处理器和一片 RAM。假设机器只接处理器和 RAM。处理器内部的状态在刚上电时未必具有确定值，RAM 中的字节也会在断电后丢失。即使操作人员已经写好程序，仍有两个问题没有答案：处理器从哪个地址开始读取；写入工具撤走以后，谁保证每次释放复位都回到相同起点。

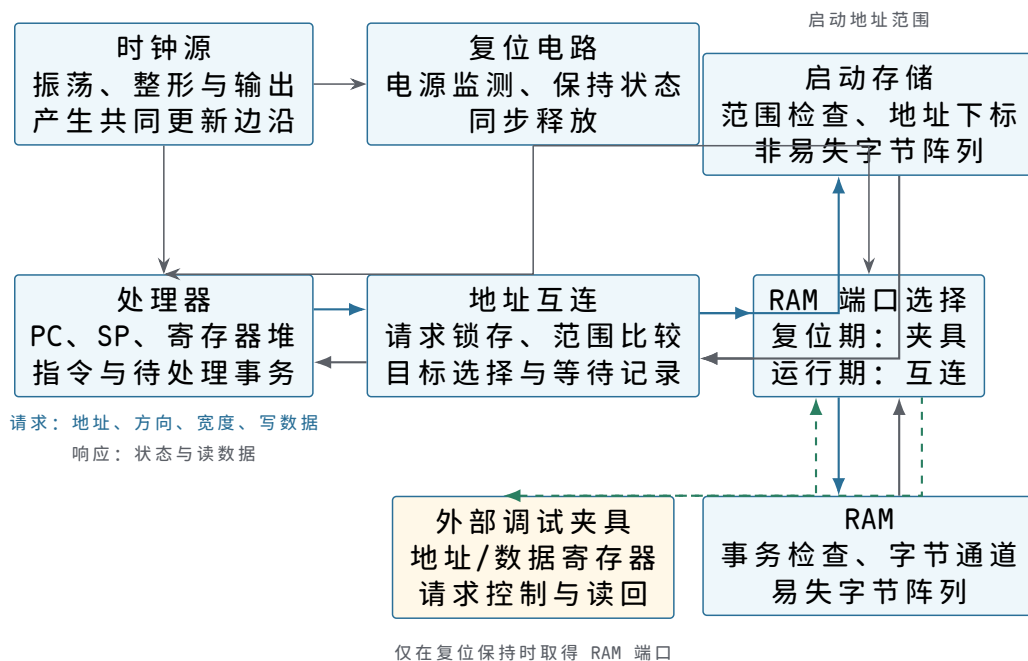
因此，运行机器还需要一小段断电后仍保留的启动字节。它们放在非易失启动存储中。复位释放时，处理器先从这段固定内容开始执行；固定内容检查本次人工装入的布局，准备寄存器和栈，最后才把指令位置交给 RAM 中的求和程序。后文把这段最先执行的软件称为启动程序或固件。

RAM 仍然不可替代。求和程序要保存输入、栈和最终结果，这些内容会在运行期间改变。启动存储负责“断电之后仍然存在”，RAM 负责“运行期间能够改写”，两者解决的是不同问题。

五类部件怎样形成一台可运行机器。处理器、启动存储和 RAM 还不能直接随意并联。处理器发出一个地址时，必须只有一个目标回答；目标返回较慢时，处理器还必须等待结果。机器因而使用一组选择和转发电路，把一次请求交给唯一器件，再把器件的响应送回处理器。后文在解释选择规则后，才把这组电路称为地址互连。

此外，寄存器更新需要共同的时序边界，刚上电的随机状态需要被收束到固定起点。时钟源提供周期性边沿，复位电路在电源尚未稳定时阻止普通执行，并在释放时给处理器规定初始状态。

至此，当前运行机器包含五类必要部件：



图示：蓝色箭头表示运行期请求，灰色箭头表示响应，虚线表示复位期间的人工准备通路。复位电路同时约束处理器与 RAM 端口选择器，因此调试夹具和处理器不会同时驱动 RAM。图中没有操作系统、调度器、系统调用或可编程计时器。

这幅图同时给出器件边界和端口关系，但每个框中的结构仍需分别验证。后文会依次展开处理器如何保存指令状态，启动存储与 RAM 的阵列怎样被地址选中，互连怎样保持尚未完成的请求，时钟与复位怎样产生确定边界，以及调试夹具怎样在人工准备结束后交还 RAM 端口。

连接不是一条没有含义的双向线。处理器读取时，地址和读请求从处理器流向目标，数据从目标返回。处理器写入时，地址、写数据和允许更新的字节位置流向目标，目标返回成功或失败状态。时钟与复位则沿另一组连接送到需要按边沿更新的部件。不同方向承载不同责任，不能用一根无说明的线概括。

连接方向	运行时携带的内容	接收方必须完成的判断
处理器到选择电路	地址、读取或写入、宽度、写数据	哪个器件拥有该地址
选择电路到目标	已选中的请求和器件内部位置	请求是否合法，何时可以完成
目标到处理器	成功或失败、读取数据	当前指令能否提交结果
时钟与复位到处理器	周期边沿、复位是否有效	保持旧状态还是进入规定起点

表中尚未给信号命名，因为读者首先需要理解信息方向。等到讨论快慢不同的器件时，再引入“有效”和“就绪”等握手信号，并说明它们在哪个边沿共同成立。

当前程序将占用哪些位置。整体方位明确后，才需要给本次运行分配具体位置。教学处理器能够表达 32 位地址；每个地址指出一个字节。启动存储位于低地址，RAM 位于另一段地址。求和程序、输入、结果和栈在 RAM 中彼此分开：

地址或范围	人在释放复位前放入什么	运行期间由谁改变
0x00000000--0x0000ffff	固定启动程序	普通运行不能改写
0x00100000--0x0010001f	八条求和指令	本次程序不改写
0x00102000--0x0010200f	四个输入整数	本次程序只读取
0x00102020--0x00102023	初始清零的结果槽	求和程序写入 21
0x001efffc--0x001effff	返回启动程序的地址	程序结束时读取

这些名称不是存储器内部的类型。RAM 只保存位；“指令”“输入”“结果”和“返回地址”来自处理器在不同阶段怎样使用同一组字节。地址表目前也只是本次人工约定，尚无硬件权限阻止程序写到别的区域。

把目标写成可以检查的启动契约。释放复位前，操作人员和机器之间需要一份最小约定。否则即使字节都已写入，启动程序也不知道入口、输入数量和结果位置。当前样机采用以下具体值：

约定项	本次取值	为什么必须明确
求和程序入口	0x00100000	决定第一次任务取指的位置
输入首地址	0x00102000	决定第一个整数从哪里读取
输入数量	4	决定循环何时结束
结果地址	0x00102020	让程序与观察者指向同一结果槽
初始栈位置	0x001efffc	保存程序结束后的控制去向
预期结果	21	提供可独立核对的外部事实

启动程序会把输入首地址、数量和结果地址放进处理器寄存器，并把栈指针指向预置返回地址。这里先说明约定的作用，寄存器和栈为何能够保存这些值将在下一节从计算中逐步得到。

什么时候才算完成了第一次运行。“机器开始动了”不是完成标准。对本章而言，成功至少要同时满足四项可检查事实：

1. 复位释放后，第一笔取指来自规定的启动位置；
2. 启动程序准备的入口、参数与栈恰好等于上表；
3. 求和程序逐条执行，结果槽最终保存字节 15 00 00 00；
4. 程序沿预置返回地址回到启动程序，启动程序停止继续改写本次结果。

第四项把“结果已经写入”和“本次控制流已经结束”分开。程序可能先写出 21，随后又因错误继续执行；只观察结果槽不足以证明整次运行正确。后文将分别给出结果写入和控制返回的状态快照。

本章尚未解决的问题。操作人员必须在每次运行前保持复位，逐区域写入程序、输入和栈，再检查地址是否一致。机器自身既不能从外部接收一份新程序，也

不能判断一串到达的字节是否完整。先把这次人工运行讲清楚，章末再用这些具体成本推动下一次演进。

从“求和”要求中辨认机器必须保留的事实

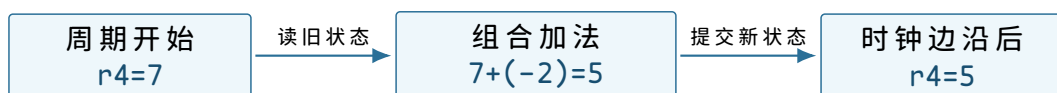
整体连接已经给出了机器的方位，现在沿着其中的处理器继续追问。设想求和程序刚读完第一个整数 7，下一步准备读取 -2。如果此刻把机器中的全部状态抹去，再告诉它“继续”，它无法判断下一条指令在哪里，也不知道已经累加出的部分和、尚未读取的数组位置以及剩余元素个数。因此，“运行”不能只表示某段指令存在；运行意味着若干事实在相邻动作之间得以保留。

对这个求和循环，至少有五项事实不能丢失：

1. 下一条机器指令的地址；
2. 当前部分和；
3. 下一个输入元素的地址；
4. 尚未处理的元素个数；
5. 函数返回时应当回到的位置。

这些事实的含义来自程序约定，电路只需要保存对应的位串。教学处理器把“下一条指令地址”单独保存在程序计数器 (program counter, PC) 中，把计算中的位串保存在通用寄存器中，把函数调用形成的返回地址保存在由栈指针 (stack pointer, SP) 定位的内存区域中。PC 和 SP 的名称描述用途，不表示里面的位具有特殊材料；它们仍是普通二进制状态。

状态与组合计算为什么必须分开。假设加法器的两个输入分别为 7 和 -2。只要输入稳定，加法器的输出就会稳定为 5。但加法器本身并不记得这个结果；输入变化后，输出随之变化。若下一条指令还要使用 5，就必须在规定时刻把它写入寄存器。寄存器在一个时钟边沿接收新值，在下一个边沿到来前保持该值，这使连续动作能够首尾相接。



图示：图中的加法结果在时钟边沿之前只是候选值；写入寄存器后，它才成为下一条指令可以读取的处理器状态。

“提交”在本章中表示一次处理器状态更新已经越过时钟边界。它不表示数据已经断电保存，也不表示整个任务完成。后续讨论存储和设备时，还会出现不同层次的完成边界。

给教学处理器规定一个足够小的软件视图。真实处理器的寄存器和指令很多，本章只保留运行该任务所需的部分。教学处理器有八个 32 位通用寄存器 `r0--r7`，另有 PC、SP 和零标志 `Z`。寄存器的用途由启动约定和任务代码共同决定：

状态	本次任务中的初始含义	执行期间怎样变化
PC	任务入口 <code>0x00100000</code>	顺序前进，分支和返回会改写它
SP	指向预置返回地址	调用时下降，返回时上升
r0	输入数组首地址 <code>0x00102000</code>	每轮增加 4，指向下一个整数
r1	元素个数 4	每轮减 1，为 0 时循环结束
r2	结果地址 <code>0x00102020</code>	本任务中保持不变
r4	部分和，初值 0	每轮加上一个数组元素

r5	当前读出的整数	每轮被下一次装入覆盖
Z	最近一次规定运算的结果是否为 0	供条件分支读取

这张表同时说明了一个重要边界：处理器不会识别“数组首地址”或“元素个数”。当启动程序把 `0x00102000` 写进 `r0` 时，硬件只得到一个 32 位位串。只有任务按照约定把它用作装入指令的基址，它才表现为数组地址。

指令也必须以字节形式存在。处理器不能执行写在纸上的“求和”二字。它读取固定格式的机器指令位串，根据其中的操作码和操作数字段打开不同的数据通路。本章采用固定 4 字节的教学指令。第一个字节是操作码，第二、第三个字节通常给出寄存器编号，最后一个字节可表示小范围立即数或地址偏移。分支指令把后两个字节解释为相对位移。

指令形式	状态变化	本章中的用途
<code>MOVI rd,imm</code>	<code>rd <- imm</code>	把部分和设为 0
<code>LOAD rd,[rb+d]</code>	<code>rd <- MEM32[rb+d]</code>	从数组读取一个整数
<code>STORE rs,[rb+d]</code>	<code>MEM32[rb+d] <- rs</code>	写回最终结果
<code>ADD rd,rs</code>	<code>rd <- rd+rs</code>	更新部分和
<code>ADDI rd,imm</code>	<code>rd <- rd+imm</code>	移动指针或减少计数
<code>BNEZ rs,target</code>	<code>rs!=0</code> 时改写 PC	控制循环
<code>CALL target</code>	压入返回地址并改写 PC	调用子程序
<code>RET</code>	从栈中取回 PC	返回调用者

这不是某个商业处理器的真实编码，而是一个字段完整、状态变化明确的教学接口。后文讨论的操作系统原理不依赖某个特定指令集；使用固定编码的意义在于，每次“取指”和“执行”都能对应到确切地址、确切字节和确切状态变化。

求和任务的核心指令如下。左侧地址按每条指令 4 字节递增：

```

0x00100000 MOVI  r4, 0
0x00100004 LOAD  r5, [r0+0]
0x00100008 ADD   r4, r5
0x0010000c ADDI  r0, 4
0x00100010 ADDI  r1, -1
0x00100014 BNEZ  r1, 0x00100004
0x00100018 STORE r4, [r2+0]
0x0010001c RET

```

第一条指令把 `r4` 置零。循环体每次读取一个 32 位整数，更新部分和，将输入指针移到下一个整数，再减少剩余数量。计数不为零时，PC 回到 `0x00100004`；第四轮后分支不成立，任务写出结果并执行 `RET`。

复算点。若 `r0=0x00102000`、`r1=4`，循环执行两轮后应有 `r0=0x00102008`、`r1=2`、`r4=5`。这里只计算程序语义；这些字节怎样进入内存、处理器怎样读到它们，仍未解决。

为什么既需要断电保存的位置，也需要运行期可写的位置。现在已有一段机器指令，但还要说明复位后最早执行的字节为什么可靠。若所有字节都只放在 RAM 中，断电后内容消失；下一次通电时，PC 即使得到一个固定初值，也读不到可靠指令。机器

因此需要一片非易失启动存储，保存最早执行的启动程序。

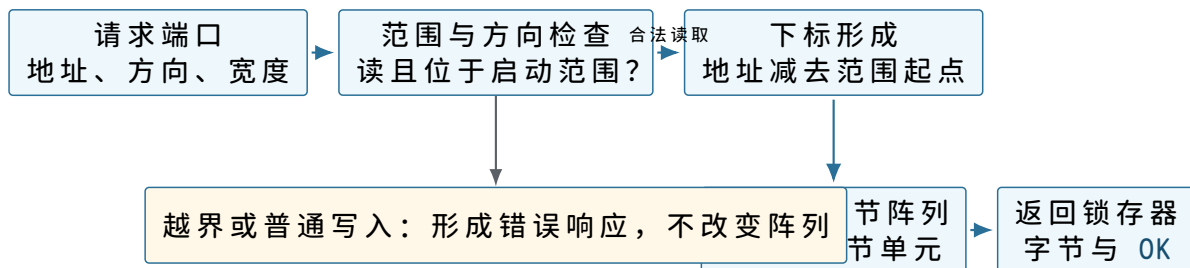
仅有启动存储仍不够。求和任务要更新栈、部分数据和装入后的任务区域，而本章把启动存储视为运行期只读。机器还需要 RAM，为会变化的字节提供位置。两类存储都能响应地址读取，但它们的保留时间和写入规则不同，不能用一句“内存”混在一起。

启动存储的三层含义。硬件模型。启动存储是一组按地址索引的非易失字节单元。输入是读地址和读请求，输出是相应字节及完成状态。上电期间的普通写请求不会改变它；如何烧写固件属于机器制造或维护阶段，本章不展开。

软件模型。启动存储占用物理地址 `0x00000000` 至 `0x0000ffff`。复位入口、人工装入检查程序和返回入口都位于其中。程序使用普通取指或读指令访问这段地址，但写入会得到错误响应。

抽象。启动软件可以依赖一段跨断电保留且复位后立即可读的字节序列。这个抽象只保证地址与字节，不提供文件名、目录或版本选择；这些含义若有需要，必须由固件格式另行建立。

“按地址索引”在器件内部至少要落实为一条可追踪的读取路径。请求先经过方向和范围检查，再把物理地址换算为阵列下标；字节阵列只接受这个内部下标，不理解 PC、固件入口或指令。阵列读出的字节经过返回锁存器形成稳定响应。写请求不进入阵列，而是转到只读错误发生器。



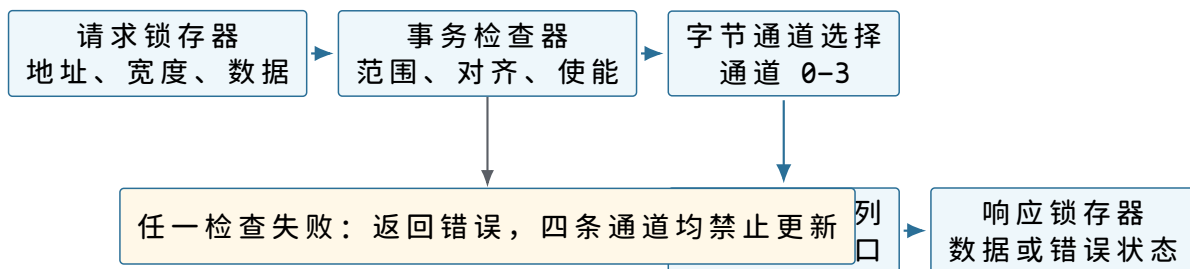
图示：启动存储内部真正保存的是非易失字节阵列。检查器、下标形成和响应锁存器负责把外部事务变成一次稳定读取；它们不为字节附加“指令”或“固件”标签。

RAM 的三层含义。硬件模型。RAM 是按地址索引的易失存储单元阵列。读请求返回旧值，写请求按照地址、写数据和字节使能更新选中单元。电源失效后，内容不再可靠。

软件模型。物理地址 `0x00100000--0x001ffffff` 可读写。任务代码、输入数据、栈和固件工作区都必须在这一范围内划出不重叠区域。RAM 不保存“代码”“整数”或“栈帧”标签，同一组字节的解释完全由访问它的指令和软件约定决定。

抽象。软件得到一片可变的、按字节寻址的工作空间。当前抽象不包含虚拟地址、访问权限和所有者；任何能够发出物理写请求的代码都可能改写其中任意位置。

RAM 的内部路径比启动存储多出写入选择。请求锁存器先把一笔事务的字段固定下来；检查器确认范围、宽度和对齐；通道选择器再根据地址低位与 `byte_enable` 决定四条八位通道中哪些可以更新。读操作沿同一组通道取出旧字节，经组合器形成返回值。



图示：写入的提交边界位于阵列端口：检查全部完成后，被使能字节在同一更新边界改变。响应锁存器中的 OK 表示这次阵列更新已经发生，而不是仅仅收到了写请求。

四个整数在内存中究竟是什么。输入数组从 `0x00102000` 开始，每个元素占 4 字节。按低有效字节在前的次序，整数 7 存成 `07 00 00 00`，整数 -2 的 32 位补码为 `fe ff ff ff`。四个元素的实际布局如下：

地址范围	字节内容	按 32 位有符号整数解释
<code>0x00102000--03</code>	<code>07 00 00 00</code>	7
<code>0x00102004--07</code>	<code>fe ff ff ff</code>	-2
<code>0x00102008--0b</code>	<code>0b 00 00 00</code>	11
<code>0x0010200c--0f</code>	<code>05 00 00 00</code>	5
<code>0x00102020--23</code>	装入时清零	任务完成后应为 <code>15 00 00 00</code>

`LOAD r5,[r0+0]` 并不是“读取数组元素”这一高级动作。处理器先计算 `r0+0` 得到物理地址，再请求从该地址开始的 4 个字节，最后按规定字节序把它们组合成 32 位值写入 `r5`。数组是连续元素的约定，装入指令只看到地址和宽度。

存储容量与地址范围不是同一个概念。本样机的 RAM 容量为 1MiB，需要 2^{20} 个字节位置。处理器却能表达 2^{32} 个不同的物理地址。没有必要让每个地址都对应 RAM；地址还要留给启动存储和设备。地址是请求中的编号，容量是某个器件实际实现的单元数量。二者通过后面引入的地址互连建立对应关系。

到这里，机器已经有了保存启动字节和运行字节的器件，但处理器还没有连接到它们。下一步必须说明一次读写请求携带什么，以及多个器件如何保证只有一个作出响应。

一个地址怎样在 RAM 中选中确切的字节

前文把 RAM 画成一个方框，并规定它占用一段物理地址。这个方框还需要展开到足以解释“为什么地址加一会得到下一个字节”。本节不重复存储电路的门级实现，只说明控制一次访问所必需的内部结构。

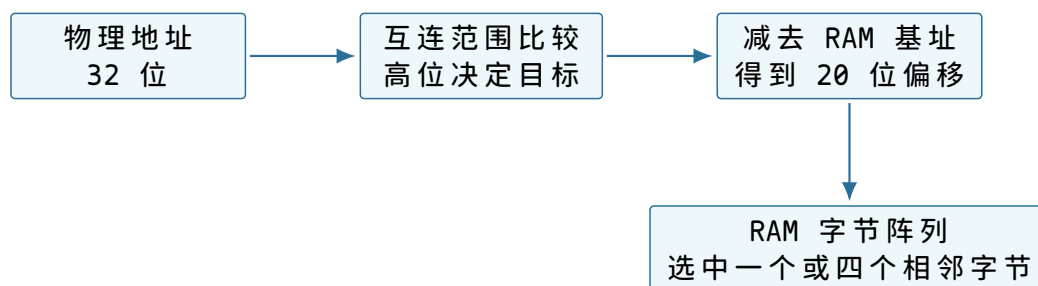
容量来自地址数量与每个位置的宽度。本章 RAM 范围为 `0x00100000--0x001ffffff`，共

$$0x001ffffff - 0x00100000 + 1 = 0x00100000$$

个字节，即 2^{20} 字节。RAM 内部不需要保存完整的高位地址。互连已经证明请求落在这个范围后，向 RAM 提供范围内偏移：

$$\text{offset} = \text{physical_address} - 0x00100000.$$

偏移的低 20 位足以选择 2^{20} 个字节位置。



图示：完整物理地址用于整机选择目标；目标内部只需使用足够区分自身位置的偏移位。

如果直接把 2^{20} 个字节画成一条线，硬件结构会显得不现实。实际阵列通常分成行、列和若干存储体。对本章的软件契约而言，这些组织方式可以不同，只要同一地址总返回同一组已提交字节，并遵守规定的响应次序。

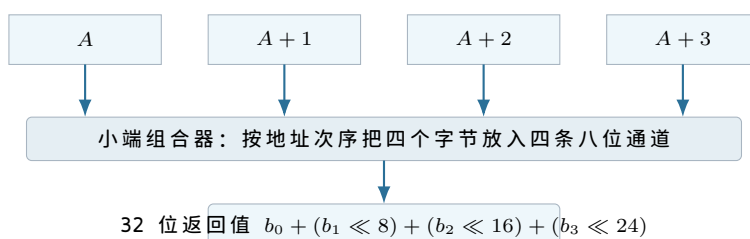
为何一次四字节读取仍以字节地址命名。处理器的地址单位是字节。`LOAD` 宽度为四时，地址 `A` 表示读取 `A, A+1, A+2, A+3` 四个相邻字节。地址没有自动除以四；只是 RAM 可以把低两位用来选择一个四字节组中的起点。

地址低两位	四字节访问是否对齐	本章处理方式
00	是	读取或写入同一个四字节组
01	否	返回 <code>ALIGNMENT</code> ，不改变任何字节
10	否	返回 <code>ALIGNMENT</code> ，不改变任何字节
11	否	返回 <code>ALIGNMENT</code> ，不改变任何字节

禁止未对齐访问不是数学必然。某些处理器会拆成两次阵列读取，再在内部拼接。本章选择拒绝，是为了让每条四字节事务只涉及一个对齐组，错误时也容易保证“全写或全不写”。程序因此必须让整数地址和 `SP` 都是四的倍数。

四条字节通道怎样组成一个 32 位值。可以把数据端口理解为四条八位通道。对齐地址 A 的低两位为零；第零通道连接 A ，第一通道连接 $A+1$ ，依此类推。读取时，小端组合器产生：

$$\text{value} = b_0 + (b_1 \ll 8) + (b_2 \ll 16) + (b_3 \ll 24).$$



图示：字节顺序是处理器与存储接口之间的解释规则。RAM 单元只保存八个位，并不保存“最低有效字节”这个语义标签。

写入时为什么需要字节使能。写请求除了 32 位数据，还带四位 `byte_enable`。四字节 `STORE` 使用 `1111`；若以后增加单字节存储，地址低位和字节使能可只选择一条通道。例如在地址 $A+2$ 写入 `0x7f`，可能使用：

```
aligned_address = A
write_data      = 0x007f0000
byte_enable     = 0100
```

RAM 必须先同时检查范围、对齐规则和字节使能是否合法，再更新被选择的通道。若先更新第零通道、随后才发现第四个字节越界，软件将面对部分写入。本章接口把检查与更新分开，成功边沿一次提交全部被使能字节。

读取端口和写入端口是否能同时工作。当前机器只有一个处理器请求发起者，互连一次只接受一笔未完成事务。RAM 因而使用一个共享端口就足够：某个周期接受读或写，经过固定或可变延迟后给出一次响应。在响应出现以前，请求字段必须由发起者或 RAM 内部锁存。

```
ram_pending:
  valid
  operation
  offset
  width
  write_data
  byte_enable
```

这些字段解释了“RAM 正在忙”究竟保存什么。若只保存地址而忘记操作方向，响应时就无法判断应返回数据还是提交写入；若不保存字节使能，等待期间输入线变化会改变原请求含义。

内部阶段	已锁存的状态	对外接口状态
------	--------	--------

IDLE	无待处理请求	<code>req_ready=1</code>
CHECK	请求全部字段	暂不接受第二笔请求
READ	偏移与宽度	阵列选择相应字节
WRITE	偏移、数据、字节使能	成功边沿提交所选字节
RESPOND	数据或错误码	<code>resp_valid=1</code>

阶段名只是本章的一种实现。软件不能读取这些名称，只能观察到请求是否被接受、响应何时有效以及成功响应对应什么效果。把内部状态列出，是为了证明接口承诺有足够的物理状态可以实现，而不是把特定状态机强加给所有 RAM。

启动存储与 RAM 为什么不能合成同一种可写阵列。如果复位入口也位于普通可写 RAM，操作者准备任务时的一次错址就可能覆盖下一次启动所需的第一条指令。机器随后连检查错误记录的代码都无法取得。将启动程序放在普通运行不能写的存储中，保留了一个比任务内容更稳定的起点。

两种存储仍可共享读取请求格式：

性质	启动存储	RAM
复位后内容	制造或维护阶段已确定	本章不规定
普通读取	支持	支持
普通写入	返回 <code>READ_ONLY</code>	合法范围内提交
本章用途	检查、交接、返回入口	任务代码、数据、栈与工作区

因此“都能返回字节”不等于“具有相同生命周期”。启动存储给出稳定起点，RAM 给出运行期可变状态；地址互连把同一种请求送到承担不同责任的目标。

物理地址表仍然不是内存保护表。范围比较只回答地址属于哪个器件。任务向 `0x001ff000` 写入时，互连会正确选择 RAM，RAM 也会正确更新准备记录；两个器件都履行了当前契约，却破坏了下一次启动依据。要拒绝这笔写入，判断还需要知道“当前请求者身份”和“该范围允许何种访问”。第一章尚未建立这两项输入，所以不能从地址表推断保护已经存在。

复位为什么不顺便清空全部 RAM。处理器复位只需设置少量控制寄存器，几个边沿内即可完成。RAM 有 2^{20} 个字节；若要求复位信号直接把每个存储单元同时改成零，就要为整个阵列增加清零通路，并规定清零期间普通访问如何处理。另一种实现是由启动程序逐地址写零，但这需要发出 2^{18} 笔四字节写，复位释放到首次检查之间会出现很长的执行阶段。

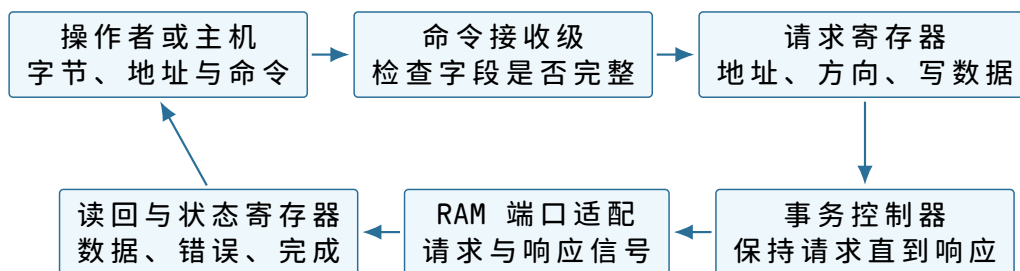
本章没有任何需求要求全部 RAM 为零。代码区、输入区、结果槽、返回栈字和准备记录都会由操作者明确写入；启动程序工作区只在首次写入后读取。其余字节保持未规定更简单，也迫使程序不依赖自己尚未建立的初值。

可选策略	获得的性质	付出的代价或仍有的限制
硬件全阵列清零	复位后每个 RAM 字节为零	阵列增加清零结构，复位时序更复杂
启动程序逐字清零	无需阵列专用清零线	启动时间随 RAM 容量增长
只初始化将要使用的区域	启动工作与本次需求成比例	未声明区域必须始终视为未规定

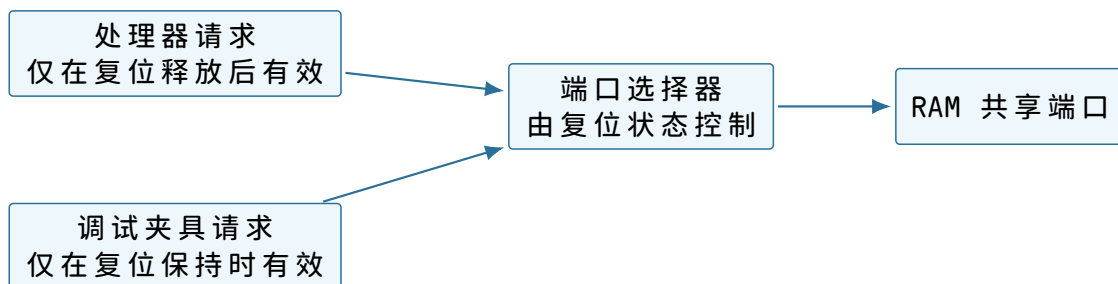
第三种策略形成一条重要纪律：任何代码在读取某个可写字节前，都要能指出本次运行中哪一步先写入了它。若回答只能追溯到“机器通常会给零”，该读取就没有建立在本章契约上。

调试夹具怎样避免端口争用。人工准备要求外部夹具能够访问 RAM，而正常运行时 RAM 端口属于处理器。两者若同时驱动地址线，电气上会冲突，协议上也无法判断哪笔请求先发生。本章利用复位状态设置一个简单选择器：

夹具内部至少需要保存当前地址、访问方向、一个字节的写数据以及尚未完成标志。操作者给出一项读写命令后，命令接收级把这些字段写入请求寄存器；事务控制器保持字段不变，直到 RAM 返回成功或错误；读操作得到的字节写入读回寄存器，随后才向操作者报告完成。若夹具只把按钮或主机信号直接接到 RAM 引脚，输入在事务期间改变时便无法证明实际访问了哪个地址。



图示：夹具的提交边界是 RAM 响应到达并写入状态寄存器。主机发出命令并不等于 RAM 已经改变；读回核对也必须在对应该事务完成以后进行。



图示：选择器不是运行中的第二个请求发起者。复位状态保证同一时刻只有一侧能够产生有效请求，因此第一章仍是单发起者机器。

夹具写入期间，处理器被复位阻止，不能观察半成品；复位释放以后，选择器固定选择处理器，夹具即使改变自己的地址输入也不会触及 RAM。操作者若要再次修改内容，必须重新保持复位，先让处理器停止普通事务，再取得端口。

复位状态	处理器端口	夹具端口	RAM 接受者
保持有效	不发普通请求	可读写	夹具
正在释放的规定边沿	仍不接受新请求	结束最后一笔请求	无；完成所有权转换
已经释放	可发普通请求	被隔离	处理器

中间一行防止夹具最后一笔写尚未响应，处理器就开始第一笔取指。复位控制器只有在夹具报告空闲后才允许释放；因此 `prepared=1` 的写响应先于第一条启动指令。这个硬件次序与前文的人工提交顺序共同保证，启动程序不会读到正在变化的准备记录。

把八条助记符落实为内存中的三十二个字节。汇编形式便于人阅读，但处理器取回的是字节。若不把两者对应起来，“操作者已经把代码写入 RAM”仍然缺少可核对对象。下面为本章用到的操作码规定确切数值：

操作码	指令	四个字节的解释
0x10	MOVI	操作码，目标寄存器，16 位有符号立即数低字节、高字节

0x11	ADD	操作码，目标兼第一源寄存器，第二源寄存器，保留字节
0x12	ADDI	操作码，目标寄存器，16 位有符号立即数低字节、高字节
0x20	LOAD	操作码，目标寄存器，基址寄存器，8 位有符号偏移
0x21	STORE	操作码，源寄存器，基址寄存器，8 位有符号偏移
0x31	BNEZ	操作码，测试寄存器，16 位相对指令位移低字节、高字节
0x40	CALL	操作码，24 位相对字节位移
0x41	RET	操作码，三个必须为零的保留字节

寄存器编号直接使用 0x00--0x07 表示 r0--r7。16 位字段也采用低有效字节在前的顺序。分支位移以“从顺序后继开始相差多少条 4 字节指令”计数，因此循环分支从 0x00100014 的顺序后继 0x00100018 回到 0x00100004，相差 -5 条指令。16 位补码 -5 为 0xffffb，在内存中写成 fb ff。

任务代码的地址、字节和含义逐行对应。

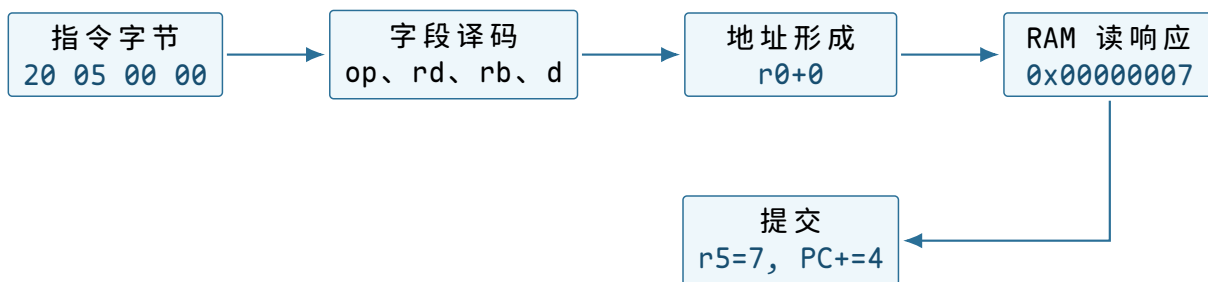
起始地址	四个实际字节	译码结果
0x00100000	10 04 00 00	MOVI r4,0
0x00100004	20 05 00 00	LOAD r5,[r0+0]
0x00100008	11 04 05 00	ADD r4,r4,r5
0x0010000c	12 00 04 00	ADDI r0,4
0x00100010	12 01 ff ff	ADDI r1,-1
0x00100014	31 01 fb ff	BNEZ r1,-5
0x00100018	21 04 02 00	STORE r4,[r2+0]
0x0010001c	41 00 00 00	RET

这张表让三个容易混淆的地址分开了。指令地址是 PC 的值；装入和存储指令形成的数据地址来自寄存器与偏移；编码中的寄存器号只是选择处理器内部状态，不是内存地址。例如地址 0x00100004 保存的第二个字节 05 表示目标寄存器 r5，并不表示要访问物理地址 5。

完整译码一条装入指令。当 PC 为 0x00100004 时，处理器取回 20 05 00 00。译码器按固定字段作出以下决定：

1. 第一个字节 0x20 选择 LOAD 语义；
2. 第二个字节 0x05 选择写回目标 r5；
3. 第三个字节 0x00 选择基址 r0；
4. 第四个字节 0x00 符号扩展为偏移 0；
5. 读取 r0=0x00102000，形成有效地址 0x00102000；
6. 发起 4 字节读事务；
7. 响应成功后把返回值写入 r5，PC 增加 4。

其中第 5 步只产生地址，第 6 步才与外部存储交互，第 7 步才提交寄存器。若事务返回错误，`r5` 必须保持旧值，普通顺序 PC 也不能提交。否则软件会看到一条“既失败又部分完成”的装入指令。



图示：译码、地址形成、外部读取和状态提交是连续但不同的阶段。任何阶段失败，都不能假装后继阶段已经发生。

负数为什么没有负号字节。 -2 的 32 位补码由 $2^{32} - 2$ 得到，即 `0xfffffffffe`。低有效字节先存，所以 RAM 中依次是 `fe ff ff ff`。装入指令不附带“有符号”标签；它把 32 个位原样放入 `r5`。加法器对补码位串执行模 2^{32} 加法，得到与有符号加法一致的低 32 位。

第一轮后 `r4=0x00000007`。第二轮执行加法：

$$0x00000007 + 0xfffffffffe = 0x00000005 \pmod{2^{32}}.$$

最高位之外的进位被丢弃，寄存器得到 5。只有当上层要判断溢出或比较大小时，才需要区分有符号解释与无符号解释。本次输入之和未超出 32 位有符号范围。

对齐要求来自一次取回的组织方式。 教学处理器要求 4 字节指令地址和 4 字节数据地址均为 4 的倍数。因此 `0x00100004` 和 `0x0010200c` 合法，而 `0x00102002` 上的 4 字节读取返回 `ALIGNMENT`。这个约束简化了取指和 RAM 字节通道的组合。

对齐不是“地址看起来整齐”的排版规则。若处理器允许非对齐读取，它可能要发出两笔事务并拼接结果；若中间一笔失败，还要规定整条指令是否提交。不同体系结构可以选择不同接口，但软件必须遵守当前机器的明确契约。

保留字段为什么也必须检查。 `RET` 的后三个字节目前没有语义，规范要求它们为零。译码器若忽略任意值，未来给这些字段增加新含义时，旧映像可能被解释成另一种指令。要求发送端置零、处理器拒绝非零保留字段，可以使格式扩展保持明确。

复算点。 若循环分支的两个位移字节误写为 `fc ff`，符号扩展结果为 -4 。顺序后继 `0x00100018` 加上 -4×4 得到 `0x00100008`，循环会跳过 `LOAD`，反复累加旧 `r5`。校验和能检测传输改变；若映像本来就这样编码，只有执行语义检查或测试才能发现逻辑错误。

同一片 RAM 为什么可以同时放指令、数据和栈。 RAM 只根据地址返回字节。处理器在取指阶段把返回的四个字节送到指令译码器；装入阶段把返回字节送到通用寄存器；`RET` 则把返回字节送到 PC。目标位置没有自带类型，访问路径决定解释方式。

访问时的 PC/指令	读取的 RAM 地址	返回字节的解释
PC 取指	<code>0x00100008</code>	<code>ADD r4,r5</code> 的编码
<code>LOAD</code>	<code>0x00102004</code>	32 位输入整数 -2
<code>RET</code>	<code>0x001efffc</code>	新 PC <code>0x00000300</code>

这也意味着错误地址可能改变解释。若 PC 被破坏为 `0x00102000`，处理器会尝试把 `07 00 00 00` 当作指令，而不会得到“这是数组”的提示。若存储指令写到代码区域，后续取指会看到修改后的字节。当前机器没有执行权限和只读页，所有保护都只是合作约定。

一个地址怎样到达唯一器件

若把处理器的地址线同时接到启动存储和 RAM，而不给出选择规则，两个器件可能同时驱动返回数据，处理器也无法判断结果来自哪里。机器需要一个位于请求发起者和目标器件之间的地址互连。它读取地址的高位，与预先接好的范围比较，只允许匹配的目标接收本次请求。

一次事务包含哪些信息。处理器访问外部状态时发出一份请求。教学互连中的请求至少含有以下字段：

```
request:
  address      32-bit physical address
  operation    READ or WRITE
  width        1, 2, or 4 bytes
  write_data   valid only for WRITE
  byte_enable  which byte lanes may change
```

目标完成后返回：

```
response:
  status       OK, UNMAPPED, ALIGNMENT, or READ_ONLY
  read_data    valid only when a READ completes with OK
```

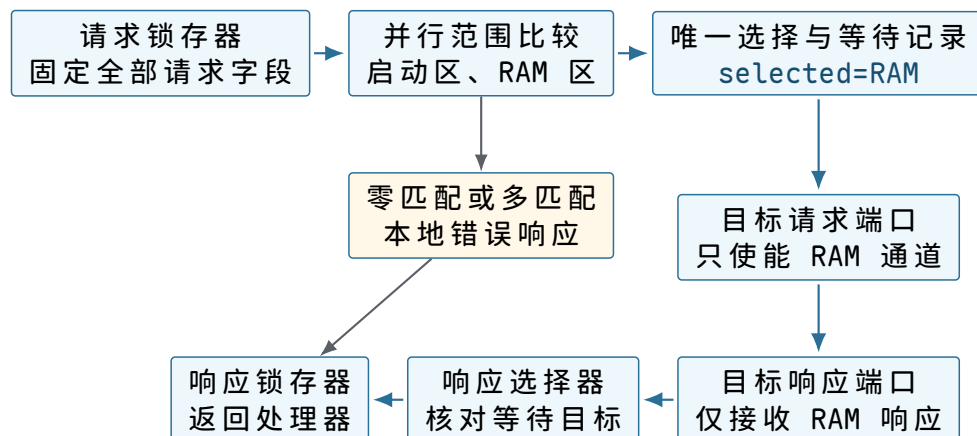
请求和响应可能由并行导线、片上网络分组或多周期握手承载。本书把从请求被接受到响应返回的这段交互称为总线事务（bus transaction）。这里的“总线”描述软件可观察的传送约定，不要求真实电路采用某个特定工业协议。

宽度字段不能省略。同一地址可用于读取一个字节或四个字节，目标必须知道哪些字节参与本次操作。写请求还需要字节使能，避免更新相邻位置。读请求中的写数据没有意义，写请求返回的读数据也没有意义；明确这些有效条件可以防止把任意电平误当成软件结果。

地址互连怎样进行选择。本样机采用一张固定物理地址表：

物理地址范围	被选中的目标	当前用途
0x00000000--0x0000ffff	启动存储	启动程序与复位入口
0x00100000--0x001fffff	RAM	人工装入的程序、数据、栈与结果
其余地址	无目标	返回 UNMAPPED

例如地址 0x00102004 落在 RAM 范围内。互连从 RAM 起始地址中减去 0x00100000，得到器件内部偏移 0x00002004，再把请求转发给 RAM。启动存储在这次事务中保持不响应。若地址为 0x20000000，没有范围匹配，互连直接返回 UNMAPPED，不能让处理器无限等待。



图示：地址 `0x00102004` 使 RAM 比较结果为 1。互连把这一位选择和请求字段一起保持到响应返回，因此较晚到达的响应仍能被送回正确事务；零匹配或多匹配由互连直接报错。

地址互连的三层含义。硬件模型。互连保存或固化一组地址范围，接收请求字段，产生目标选择信号，再把唯一目标的响应送回发起者。没有匹配或出现重叠匹配时，它产生错误响应。

软件模型。程序看到统一的物理地址空间。相同的 `LOAD/STORE` 指令访问不同范围时会到达不同器件；程序必须遵守每个范围允许的宽度、方向和对齐规则。

抽象。软件可以用一个物理地址唯一指出当前机器中的一个可寻址位置。这个能力不包含虚拟内存、名称、访问权限或并发仲裁；它只回答“这次请求交给谁”。

图中的“等待记录”不能由组合比较线替代。请求被接受以后，处理器可能撤掉原地址，RAM 也可能经过若干周期才响应；互连仍要记住本次选择的是 RAM，并在响应返回前拒绝另一笔会覆盖记录的请求。当前单发起者互连至少保存 `busy`、`selected_target` 和必要请求字段。响应提交后，这些字段才清除。

一次 RAM 读取的完整时序。以第一轮循环读取整数 7 为例。执行 `LOAD r5,[r0+0]` 时，`r0=0x00102000`。处理器先形成地址，再等待目标完成：

1. 处理器计算有效地址 `0x00102000+0`；
2. 它发出 4 字节读请求，并在请求未完成时保留该条指令的上下文；
3. 互连选择 RAM，把地址换算为器件内部偏移 `0x00002000`；
4. RAM 读取连续四个字节 `07 00 00 00`；
5. RAM 返回 `OK` 和读数据，互连把响应送回处理器；
6. 处理器把组合后的 32 位值 `0x00000007` 写入 `r5`；
7. 只有写回发生后，PC 才按该指令的完成规则进入下一位置。

若 RAM 需要多个时钟才能响应，处理器不能提前把 `r5` 改成未知值，也不能把这条装入当作已经完成。教学处理器在等待期间暂停该指令的提交。更复杂处理器可以执行其他独立工作，但最终仍必须使软件观察到与规定顺序一致的结果；本章不需要展开内部优化。

时钟与复位怎样把随机上电状态收束到第一条取指。存储和互连已经接好，处理器仍需要一个共同的状态更新节拍，并且需要在上电后得到确定的起点。电源刚稳定时，各寄存器中的位不应被假定为零。复位电路必须主动把少量关键状态设置为规定值，其中最重要的是 PC 和控制状态。

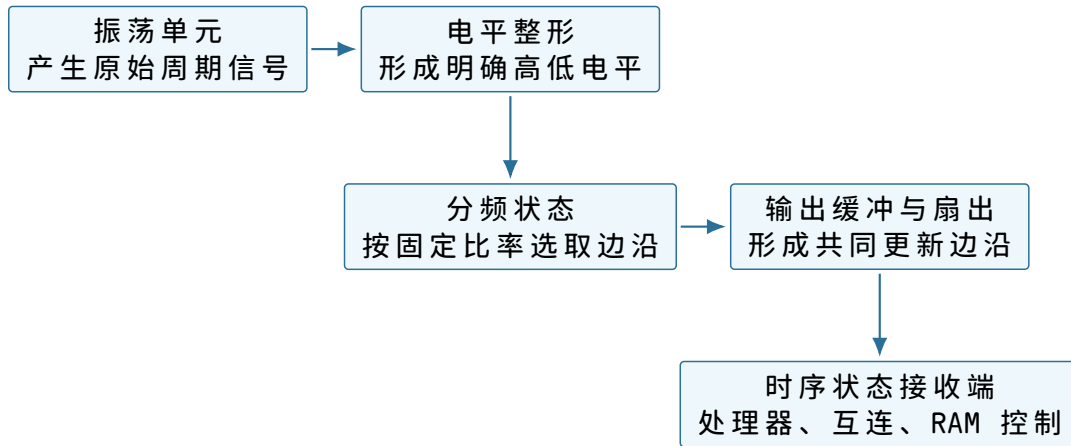
时钟不是软件计时器。时钟源周期性地产生边沿，时序电路在边沿处提交新状态。它使“先读取旧寄存器、再写入新寄存器”的顺序有了物理边界，但普通软件此时没有可读的时钟计数器。知道电路有时钟，不等于能够请求“十毫秒后通知我”。可编程计时器要在后续确有需求时作为新器件加入。

硬件模型。时钟源输出周期边沿；复位电路在电源和时钟尚未稳定时保持复位有效，稳定后按规定边沿释放。处理器在复位有效期间不提交普通指令状态。

软件模型。复位释放后，PC 被设为 `0x00000100`，SP 和通用寄存器的值除非另有规定，否则视为未定义。固件不能在初始化前读取这些寄存器并假定它们为零。

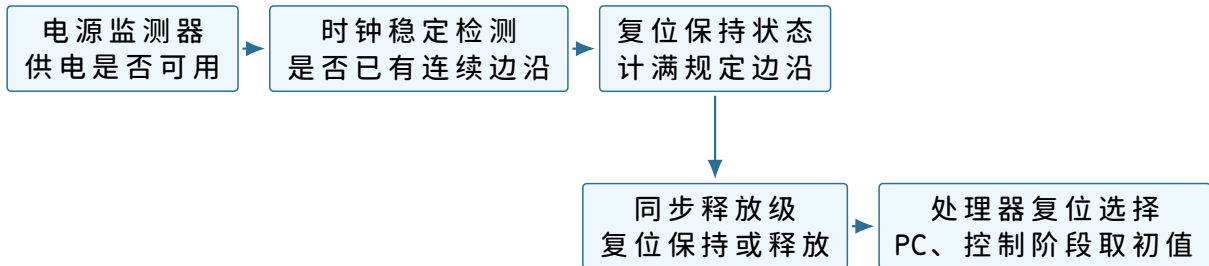
抽象。启动代码可以依赖每次复位都从同一入口开始，并且已提交的指令状态在时钟边界上遵守体系结构规则。它不能依赖真实频率恒定，也不能由边沿数推断日期或调度时刻。

时钟源内部也不是一个只写着“时钟”的方框。振荡单元先产生连续变化的原始信号；整形级把缓慢或带噪的过渡收束为数字高低电平；可选分频状态决定输出边沿相隔多少个原始周期；输出缓冲再把同一时钟送到处理器、互连的事务状态和 RAM 控制端。第一章只依赖这些部件看到同一规定边沿，不依赖振荡器的材料、真实频率或相位调节方法。



图示：分频状态和输出缓冲属于时钟路径本身；它们不构成软件可读计数器。处理器只能依赖状态在规定边沿更新，尚不能请求某个未来时刻的通知。

复位并不是一根从电源直接连到 PC 的线。电源监测器先给出“供电可用”，时钟稳定检测器再确认已有连续边沿；复位保持状态随后等待规定边沿数，并通过同步释放级只在时钟边界改变 `reset_active`。处理器中的复位选择器据此把 PC 和控制阶段写成规定起点，其他未承诺清零的寄存器不经过这条初始化通路。



图示：复位保持和同步释放把模拟上电过程收束为一个数字状态变化。处理器只对契约明确列出的状态采用复位初值；RAM 和普通通用寄存器不会因此自动全部清零。

从电源稳定到第一条指令完成。下面的时间线把“开机”拆成可观察事件。纵向虚线表示时钟边沿，横向各行表示不同部件：



图示：电源稳定并不立即开始取指。复位先保持若干边沿，释放后才把规定的 PC 值用于第一笔事务；第一条指令完成后，PC 才进入后继值。

第一条指令位于启动存储，而不是 RAM 中的任务入口。原因很直接：RAM 此时尚未装入任务，输入参数和栈也未建立。复位 PC 若直接指向 `0x00100000`，处理器会把上电后的不确定 RAM 字节当作指令，机器行为无法预测。

处理器怎样把指令字节变成状态变化。处理器是当前唯一能够主动发起取指和数据事务的部件。它内部至少要保留 PC、SP、通用寄存器、标志和当前指令的执行控制状态。这里不展开门级数据通路，只说明后续软件真正能依赖的边界。

硬件模型。处理器根据 PC 发出取指请求，译码返回的操作码与字段，读取旧寄存器，执行算术或形成访存地址，并在指令完成时提交目标寄存器、内存效果和下

一 PC。访存错误会阻止普通写回，并把失败位置与原因留在处理器内部的停止记录中。

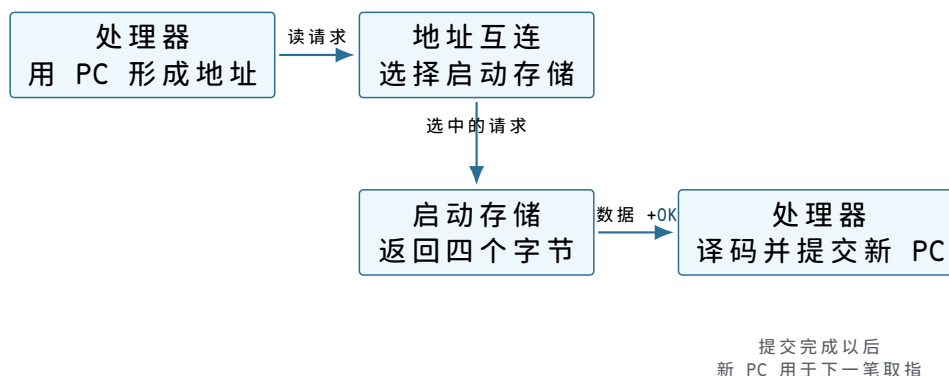
软件模型。软件使用本章定义的寄存器和指令。除分支、调用、返回和显式跳转外，PC 每完成一条固定长指令增加 4。调用约定规定哪些寄存器传参、SP 如何移动以及返回值放在哪里，这些规则不是电路自动理解的类型系统。

抽象。软件得到一条按指令集规则推进的执行流：每条完成的指令都有确定的前置状态和后置状态。当前处理器不隔离任务，不保存多个执行现场，也不会因为另一个任务等待而主动停止当前任务。

第一条取指逐字段走读。复位释放后，PC 为 `0x00000100`。处理器发出的请求为：

```
address      = 0x00000100
operation    = READ
width       = 4
write_data   = ignored
byte_enable  = 1111
```

互连发现地址属于启动存储。这四个字节就是启动程序的第一条普通指令；本章样机把它规定为读取人工准备记录中提交标志的 `LOAD`。启动存储返回数据及 `OK` 后，处理器完成译码并等待相应的数据读取；只有该装入指令成功，目标寄存器和顺序后继 `0x00000104` 才一同提交。这里不需要一条尚未解释的特殊“启动跳转”：复位 PC 本身就指向启动程序入口。



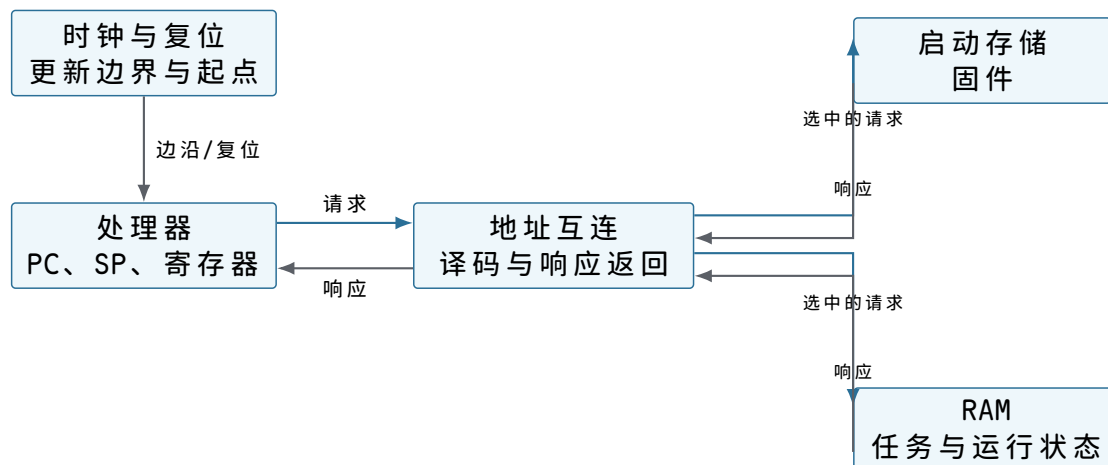
图示：取指不是处理器内部凭空取得指令。它是一笔带地址的读取事务；只有响应成功并完成译码后，处理器才提交新的 PC。

顺序 PC、分支 PC 与返回 PC。求和任务中出现三类下一 PC：

1. 普通指令完成后，下一 PC 为当前 PC 加 4；
2. `BNEZ` 根据 `r1` 的值，在循环入口和顺序后继之间选择；
3. `RET` 从 SP 指向的内存中读取返回地址，再把该值写入 PC。

三者最终都只是决定 PC 的新值，但所依赖的状态不同。普通前进只依赖当前 PC；条件分支还依赖寄存器或标志；返回还要成功完成一次栈内存读取。把它们都写成“跳到下一条”会隐藏错误来源。

把已经解释的连接重新合在一起。章首先给出的整体图用于确定方位。现在，每个方框和箭头都已有明确职责，可以用同一幅结构复核一次请求怎样往返。机器仍然只包含时钟与复位、处理器、地址互连、启动存储和 RAM。



图示：此时机器已经能从固定启动程序取指并访问 RAM。地址互连只负责按地址送达，不负责判断 RAM 中的字节是否构成一份完整程序。

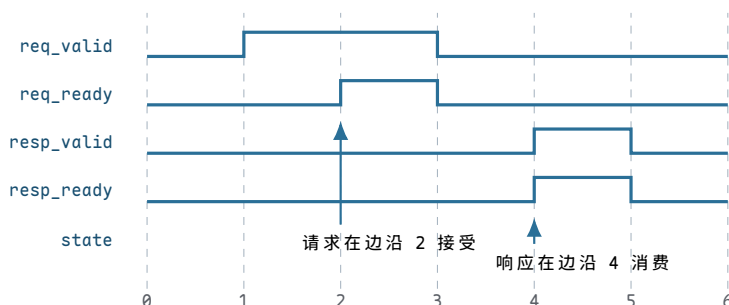
这幅图说明了当前能力的上限。复位能找到启动程序，启动程序能读写 RAM；但本章仍需由操作人员在复位期间预置求和程序。机器运行后没有接收新程序的通道，也没有谁替操作人员检查一串外部字节是否完整。

连接不仅传位，还必须约定何时有效。地址线、数据线和方向线即使全部接通，发起者仍要知道目标何时看到了请求，目标也要知道处理器何时接收了响应。若启动存储一周返回而 RAM 三周期返回，固定等待一个周期会让处理器过早采样 RAM。教学互连因此使用请求/响应握手。

请求通道的四个时序事实。处理器把请求字段放稳后置 `req_valid=1`。互连只有在能够接收时置 `req_ready=1`。某个时钟边沿同时看到二者为 1，才表示请求已被接受。在此之前，处理器必须保持地址、方向、宽度和写数据不变。

信号	驱动者	为 1 时的含义
<code>req_valid</code>	处理器	请求字段当前有效
<code>req_ready</code>	互连	本边沿可以接受这份请求
<code>resp_valid</code>	互连	响应状态和读数据当前有效
<code>resp_ready</code>	处理器	本边沿可以消费响应

响应通道采用同样规则。`resp_valid` 与 `resp_ready` 在同一边沿为 1，响应才被消费。目标可以在请求接受后的任意较晚周期给出响应，处理器无需预先知道固定延迟。



图示：处理器从周期 1 起保持请求，直到边沿 2 完成握手；目标稍后给出响应，并在边沿 4 被消费。有效信号单独为 1 都不表示传送已经发生。

为什么有效和就绪不能合成一根线。若只有一根“完成”线，无法区分“发起者现在提供有效请求”和“接收者现在有容量接受”。当互连忙于前一笔事务时，处理器必须保留请求；当处理器暂时不能接收响应时，互连也必须保留响应。两端各自表达有效与就绪，才能在不同速度部件之间建立无歧义边界。

当前处理器每次只允许一笔未完成事务，因此不需要请求编号。若以后允许多笔并发且响应可能乱序，就必须增加事务标识，并保存每笔请求对应的目标寄存器和 PC。那是性能优化引出的新状态，不属于最小机器。

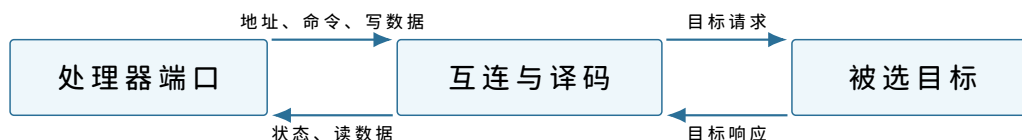
写请求中的数据何时可以撤掉。处理器从置 `req_valid` 开始，就要同时保持 `write_data` 和 `byte_enable`。只有请求握手边沿之后，才可撤掉这些字段。请求被互连接受并不等于目标写入已经完成；处理器还要等待响应握手，才能提交存储指令的顺序后继。

这形成两个边界：

1. 请求接受：互连已取得一份稳定请求，处理器可以释放请求通道；
2. 操作完成：目标已经执行或拒绝操作，响应被处理器消费。

在简单零等待器件上，两者可能落在相邻甚至同一周期，但软件模型仍不应把它们定义成同一个概念。

物理连接中的方向与扇出。教学图用箭头表达信息方向。地址、操作类型和写数据从处理器流向互连；读数据和响应状态从目标流回处理器；目标选择信号从互连流向某个器件。时钟可以扇出到多个时序部件，复位也可送达需要确定初态的控制状态。



图示：双向交互由两组单向信息构成。把一条线画成无说明的双向箭头，会隐藏谁必须保持字段以及哪个边沿完成传送。

为什么地址不能由所有目标共同解释。目标器件只需识别自己的内部偏移，不必都实现完整 32 位范围比较。互连先完成全局选择，再把低位偏移交给目标。这减少重复逻辑，也把地址冲突集中在一处检查。

例如 RAM 收到内部偏移 `0x00002004`，无需知道全局地址曾是 `0x00102004`。但软件错误报告应保留原始全局地址，因为程序使用的是物理地址表，不是器件内部偏移。

复位连接不应清空所有 RAM。处理器复位要确定 PC 和控制状态，互连复位要清除未完成事务。RAM 内容是否在复位时清零是另一件事。多数 RAM 不会因处理器复位自动擦除，启动程序必须检查或初始化依赖初值的区域。

这解释了为什么启动程序不能仅凭复位后看到的旧 RAM 字节就判断人工准备已经完成。复位只重建控制起点，不证明操作人员写完了全部区域。下一节会增加一份很小的启动说明，让启动程序在移交处理器前检查本次人工装入是否完整。

不同指令怎样越过各自的提交边界。“处理器执行一条指令”不能只画成一个方框。算术指令、装入、存储、分支和返回依赖的外部状态不同，失败时允许发生的效果也不同。教学处理器按指令类别规定提交规则。

指令类别	完成前读取	候选效果	提交条件
寄存器算术	源寄存器、立即数	目标寄存器与标志的新值	当前指令无译码错误
装入	基址寄存器、RAM 响应	目标寄存器的新值	读响应为 OK
存储	基址、源寄存器	RAM 中若干字节的新值	写响应为 OK

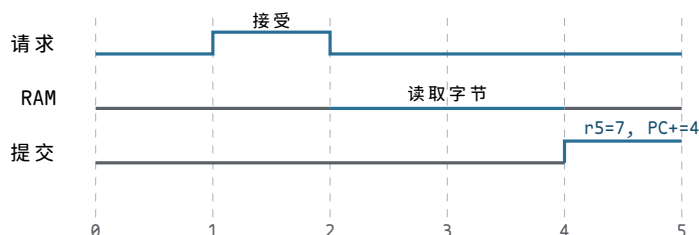
条件分支	测试寄存器、位移	顺序 PC 或目标 PC	目标满足对齐约束
RET	SP、栈读响应	新 PC 与增加后的 SP	栈读成功且返回 PC 合法

算术指令的结果在写回前只是处理器内部候选值。装入多了一笔外部读，不能在响应之前写目标寄存器。存储的效果发生在 RAM 一侧；处理器收到成功响应后才能继续，但不能先改 RAM、随后又向软件报告该指令未执行。教学模型要求目标对每笔写要么完整接受指定字节，要么返回错误且不改变任何字节。

装入等待期间保存什么。假设 RAM 需要三个周期完成读取。处理器至少要保留下列临时事实：

```
pending_load:
  instruction_pc = 0x00100004
  destination    = r5
  address        = 0x00102000
  width          = 4
  next_pc        = 0x00100008
```

它们不是软件可直接读取的通用寄存器，而是处理器内部执行状态。若等待时把 PC 提前提交到 `0x00100008`，又允许下一条指令读取旧 `r5`，程序会观察到错误顺序。最小处理器因此在响应返回前不开始下一条指令。



图示：请求被接受不等于装入已提交。RAM 完成读取后，处理器才同时更新目标寄存器和顺序 PC。

存储的提交点位于目标响应。`STORE r4,[r2+0]` 的处理器请求包含地址、宽度和数据。RAM 检查地址及字节使能，在接受写入的边界更新四个字节并返回 `OK`。若地址未对齐，它返回 `ALIGNMENT` 且四个字节全部保持旧值。

对软件而言，存储指令完成后，同一处理器随后读取该地址必须看到新值。当前机器只有一个请求发起者，没有缓存和其他处理器，所以不需要讨论多核可见顺序。这个简化条件应明确写出，不能把单核结论直接推广到并发机器。

分支先计算两个候选，再提交一个。`BNEZ` 的顺序后继为 `PC+4`，目标为

$$PC+4 + 4 \times \text{signextend}(\text{disp16}).$$

处理器读取测试寄存器，在两个候选中选择一个写入 PC。未被选择的地址不会产生取指事务。对循环分支，`disp16=0xffffb` 表示 `-5`，目标计算为

$$0x00100018 - 20 = 0x00100004.$$

若位移计算溢出 32 位地址或目标未对齐，教学处理器不提交普通 PC，而是记录失败原因并停止。真实体系结构可能规定模地址运算或不同的受控转移行为；本书此处只依赖“错误不会静默成为另一条合法顺序指令”这一教学契约。

CALL 与 RET 为什么同时改变内存和 PC。`CALL target` 需要保留顺序后继，以便子程序结束后继续。教学处理器按以下顺序定义调用：

1. 计算返回地址 `PC+4`；

2. 计算新栈指针 $SP-4$ ，检查是否对齐；
3. 向 $MEM32[SP-4]$ 写返回地址；
4. 写响应成功后提交 $SP \leftarrow SP-4$ 和 $PC \leftarrow target$ 。

若第 3 步失败，SP 和 PC 都保持调用前的体系结构值。RET 进行相反方向的动作：读取 $MEM32[SP]$ ，成功后提交 $PC \leftarrow value$ 和 $SP \leftarrow SP+4$ 。调用与返回因此不是纯粹的 PC 跳转，它们还维护一条位于 RAM 的控制流历史。

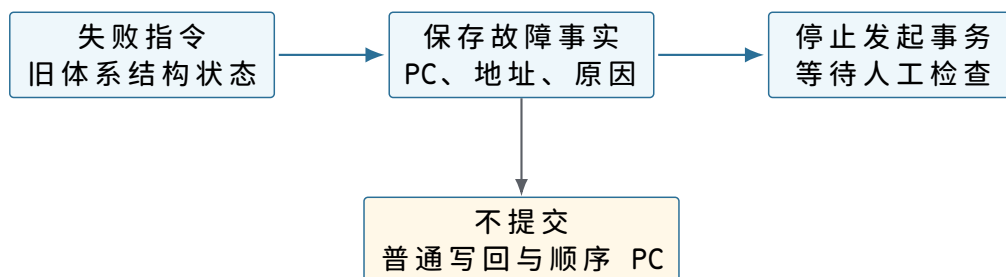
状态	调用前	CALL 成功后
PC	0x00100100	子程序入口 0x00100200
SP	0x001efffc	0x001efff8
栈字 0x001efff8	不属于有效栈	返回地址 0x00100104

当前求和核心没有调用子程序，但启动协议仍预置一个返回地址，使最外层 RET 能把控制权交回固件。最外层返回与普通函数返回使用相同处理器规则，区别只在栈中字节由谁建立。

错误响应怎样留下可检查的停止状态。如果处理器遇到未定义操作码后仍继续前进，后面的现象会掩盖最初原因；如果它只停住而不记录位置，操作者又无法区分“正在等待”和“已经失败”。因此当前样机给处理器增加四个仅供调试夹具读取的锁存字段：

fault_pc	address of the instruction that could not complete
fault_address	external address, if the instruction issued one
fault_cause	INVALID_OPCODE, UNMAPPED, ALIGNMENT, or READ_ONLY
halted	1 after a fault record has been committed

错误发生时，处理器先锁存前三项事实，再把 halted 置一。失败指令的普通目标寄存器、顺序 PC 和目标内存效果均不提交；处理器也不再发出新的普通取指请求。操作者可在保持复位前通过调试夹具读取这些字段，修正人工装入的内容，然后重新准备 RAM 并复位。



图示：停止记录只保存足以说明失败位置的最小事实。它不会保存一份可在稍后恢复的完整运行现场，也不会自动选择另一段程序继续执行。

无效操作码与未映射地址属于不同层次。若取回的第一个指令字节不是已定义操作码，错误在处理器译码阶段产生。此时 `fault_cause` 取值为 `INVALID_OPCODE`，没有数据地址。若合法 `LOAD` 形成的地址没有目标，处理器已经完成译码，互连返回 `UNMAPPED`，此时 `fault_address` 有效。区分二者可以判断是代码字节本身无法解释，还是合法指令访问了错误位置。

为什么停止记录还不是受控入口。当前硬件不会因为错误而把 PC 改到另一段程序，也没有保存全部通用寄存器，更没有从失败位置恢复的协议。它做的事情只有“冻结普通推进并留下原因”。后续确实需要机器自动转向一段处理程序时，才会进一步推导入入口地址、专用栈、现场保存和返回规则；不能把这里的停止锁存器提前称为完整的异常处理机制。

处理器内部必须保存哪些运行事实

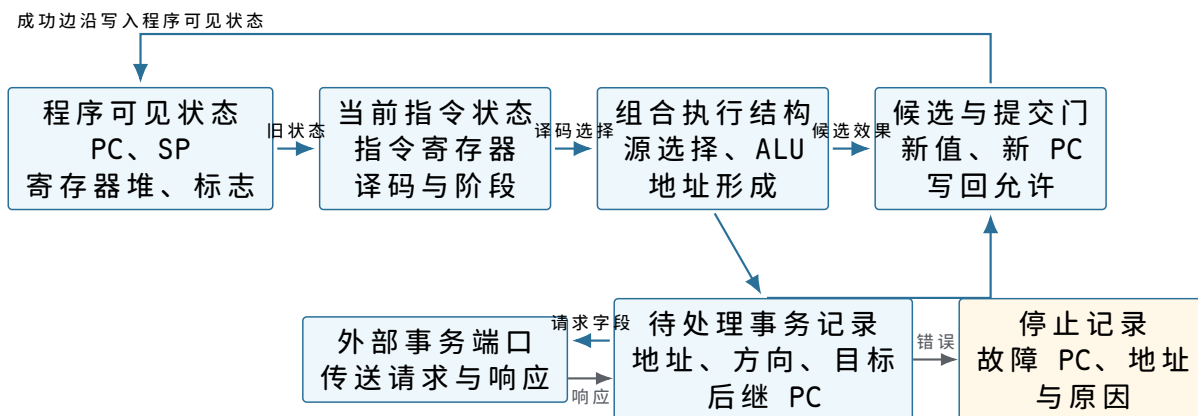
寄存器表只列出了程序能够直接命名的状态。处理器在一条指令尚未完成时，还要记住取到的指令、等待的事务和候选结果。否则外部响应经过数个周期回来时，它无法知道响应属于哪条指令，也无法在失败时保留旧状态。

可见状态与暂存状态先分开。

类别	具体字段	生命周期
程序可见	PC、SP、 <code>r0--r7</code> 、算术标志	从一条已提交指令延续到下一条
当前指令	<code>instruction_pc</code> 、四个编码字节、译码字段	从取指响应到提交或失败
访存等待	操作、地址、宽度、数据、目标寄存器、后继 PC	从数据请求接受到数据响应
候选效果	新寄存器值、新 PC、新 SP	只在当前指令成功时写入可见状态
停止记录	故障 PC、地址、原因、 <code>halted</code>	从失败提交保持到复位

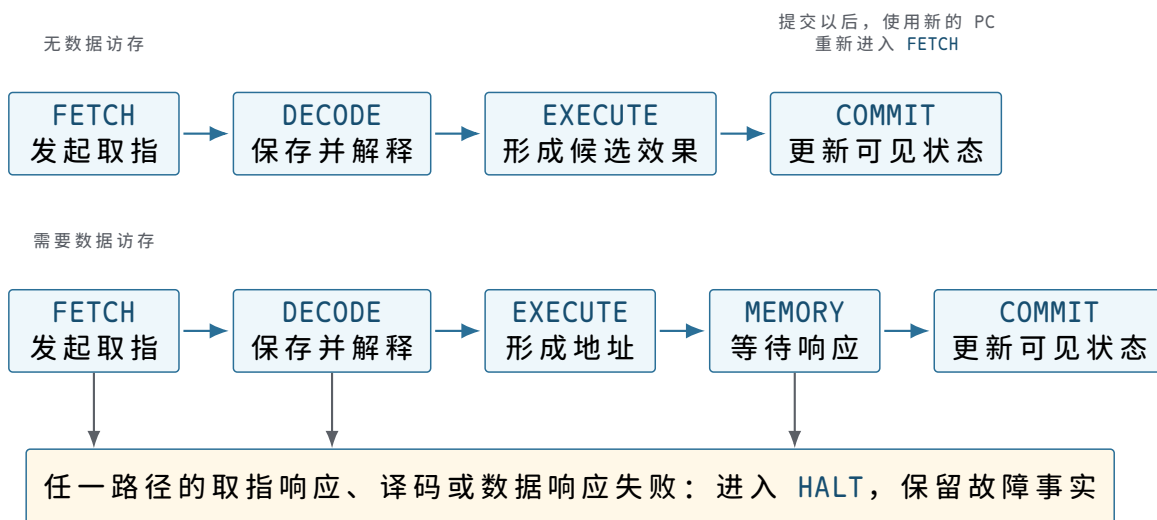
“切换到下一条指令”准确地说，是清除当前指令与候选字段，保留刚提交的程序可见状态，再用新的 PC 发起取指。它不是把处理器全部清零。

这些字段在处理器内分成几组真实的状态单元。PC、SP、寄存器堆和标志长期保存已提交状态；指令寄存器与控制阶段说明当前正在解释哪条指令；算术单元本身不保存结果，候选锁存器才把结果保持到提交边界；访存尚未完成时，待处理事务记录继续保存地址、方向、目标寄存器和后继 PC。外部端口只负责传送，不能代替任何一组跨周期状态。



图示：处理器内部的结构按生命周期分开：组合执行结构可以随输入变化，指令与待处理记录要保持到事务结束，候选效果只在成功边沿写入程序可见状态，错误则进入独立停止记录。

最小顺序处理器怎样分阶段推进。



图示：图中一次只有一条指令占据这些阶段。后续若讨论流水线，必须允许许多条指令同时处于不同阶段，并重新定义提交顺序；第一章不预设该结构。

FETCH 保存当前 PC 的副本。发起取指前，处理器执行：

```

instruction_pc = PC
request.address = PC
request.operation = READ
request.width = 4
  
```

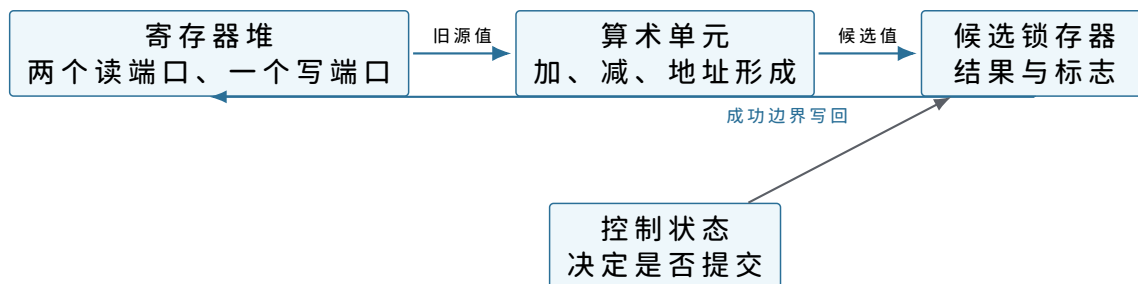
即使后续已有候选顺序 PC，故障报告也使用 `instruction_pc`。取指响应回来后，四个字节进入指令寄存器。只有此时译码器才稳定看到操作码和字段。若启动存储需要两个周期，处理器在等待期间不能用输入端口上暂时出现的其他数值进行译码。

DECODE 读取的是旧寄存器值。以 `ADD r4,r4,r5` 为例，译码字段选择两个读端口：

```

source_a = registers[4]
source_b = registers[5]
destination = 4
  
```

寄存器堆读出当前已提交的 `r4` 和 `r5`。运算结果先进入 `candidate_value`；在 **COMMIT** 以前，寄存器堆中的 `r4` 不变。这样，同一条指令的两个源操作数都来自统一的前置状态。



图示：候选锁存器把计算与提交隔开。发生错误时，控制状态可以丢弃候选值而不改动程序可见寄存器。

不同指令实际使用哪些内部字段。

指令	执行期间额外保存	成功时提交
MOVI	目标号、符号扩展后的立即数、顺序 PC	目标寄存器与 PC

ADD	两个旧源值、目标号、候选和	目标寄存器、标志与 PC
LOAD	有效地址、宽度、目标号、顺序 PC	返回值写目标寄存器，更新 PC
STORE	有效地址、宽度、源数据、字节使能	RAM 效果已经成功后更新 PC
BNEZ	测试值、顺序 PC、分支目标	二选一的新 PC
RET	旧 SP、栈地址、候选返回 PC	新 PC 与 SP+4

表中没有“当前任务编号”。第一章的处理器不知道任务身份；同一组字段先执行启动程序，后执行求和程序。软件约定改变了这些位的含义，硬件保存结构没有随之更换。

一次 LOAD 的内部状态逐周期变化。假设 RAM 三个周期后响应，且请求在第 2 个边沿被接受：

边沿后	控制阶段	保持的内部事实	程序可见状态
0	FETCH	instruction_pc=00100004	PC 仍为 00100004
1	DECODE	目标 r5、基址 r0	全部不变
2	MEMORY	地址 00102000、宽度 4、后继 PC	全部不变
3	MEMORY	同一待处理记录	全部不变
4	MEMORY	收到数据 7，形成候选	全部不变
5	COMMIT	待处理记录即将清除	r5=7, PC=00100008
6	FETCH	新 instruction_pc=00100008	保持边沿 5 的值

若边沿 4 收到 UNMAPPED，边沿 5 改为提交停止记录，r5 和 PC 保持边沿 0 以前的值。响应延迟只增加中间等待行，不改变成功提交后的状态。

一次 STORE 的候选效果为什么不在处理器里。装入的候选值最终写寄存器；存储的目标位于 RAM。处理器能保存地址和数据，却不能仅凭“请求已经送出”认定 RAM 已改变。RAM 的成功响应说明目标侧已经在自己的提交边界接受了全部字节，处理器随后只需更新 PC。

观察位置	请求接受后、响应前	成功响应后
处理器	保存地址、数据、字节使能与后继 PC	清除待处理记录，提交后继 PC
RAM	保存待检查的写请求	四字节已经作为一组更新
程序可见	当前 STORE 尚未完成	后续同地址读取必须看到新值

PC 为何不能在发请求时提前增加。若发起 LOAD 时就把 PC 改到下一条，等待中发生故障时会出现两个问题：停止记录不再自然指向失败指令；复位以外的恢复机制将不知道该重试哪一条。保留旧 PC，把 PC+4 只放在候选字段中，可以让“当前指令”在整个等待期保持稳定。

某些真实处理器会在内部预测和提前取后续指令，但仍需要一个按程序顺序提交的边界，使软件观察结果等价于规定模型。第一章直接采用顺序实现，不让优化

掩盖基本状态关系。

停止状态为何仍需要时钟边沿提交。组合电路检测到错误的瞬间，`fault_cause` 可能随输入变化。处理器在规定边沿同时锁存 PC、地址和原因，再置 `halted=1`。从下一周期开始，控制状态保持 `HALT`，不再发请求。这样操作者读到的四个字段来自同一次失败，而不是不同时刻的混合。

```
if response.error:
    next_fault_pc      = instruction_pc
    next_fault_address = pending.address
    next_fault_cause   = response.error
    next_halted        = 1
    commit no ordinary destination
```

交接时改变的是同一套寄存器，不是复制出第二套。启动程序进入任务时，处理器没有把旧 `r0--r7` 自动保存到隐藏区域。它们被任务入口值直接覆盖。启动程序能够在任务返回后继续，是因为返回入口不依赖交接前那些临时寄存器，长期需要的准备编号已经写在 RAM 工作区。

交接对象	本章实际动作	没有发生的动作
通用寄存器	写成任务参数和清零值	没有保存启动程序完整寄存器副本
SP	从启动栈值改为任务栈顶	没有同时保留两个活动 SP
PC	从交接指令改为任务入口	没有“任务模式”位
RAM	保留两段栈和工作区字节	没有自动禁止任务访问启动区

这一事实为后续多项程序问题留下准确起点：若程序 A 暂停后还要从原位置继续，就必须在覆盖寄存器前把它的 PC、SP 和仍有用的寄存器值保存到某处。第一章没有这种需要，所以也不提前引入保存现场结构。

机器开始运行以前，RAM 中究竟要有什么

到这里，处理器已经能够取指，启动存储和 RAM 也已经接入同一组地址。可是“接好了 RAM”并不等于“RAM 里已经有程序”。上电后的 RAM 字节没有本章可依赖的初值；即使某种器件偶然保留了上次断电前的电荷，也不能据此判断哪些字节属于这一次运行。

本章暂不增加任何自动输入设备。操作者使用机器外部的调试夹具，在复位保持有效时直接写 RAM。夹具可以选中一个物理地址并写入一个字节，也可以读回一个字节核对。它不属于正在运行的机器，处理器也不能通过指令调用它。这样的准备方式十分笨拙，却能把最基本的问题暴露得很清楚：在处理器取得第一条任务指令之前，必须先建立哪些事实？

只写入指令字节为什么还不够。这次要运行的程序把四个 32 位有符号整数相加。若只把八条静态指令写入 `0x00100000`，程序仍无法确定下列内容：

- 四个输入整数放在何处，按什么字节顺序解释；
- 应当处理几个元素，结果应写到哪个地址；
- 栈可以使用哪一段 RAM，最初的 SP 应取什么值；
- 最后一条 `RET` 应当把 PC 带回哪里；
- 这些区域是否已经完整写入，而不是只写完一半。

前四项决定程序如何运行，最后一项决定机器是否可以开始运行。把它们混成一句“程序已装入”会丢失最重要的边界。为此，我们先把一次运行所需的 RAM 分成五个区域。

准备记录 说明本次布局	任务代码 32 字节	输入与结果	任务栈 向低地址增长	启动程序工作区 任务不可使用
----------------	---------------	-------	---------------	-------------------

图示：图中横向宽度表示逻辑分区，不按真实容量比例绘制。每个区域仍由普通 RAM 字节组成；分区首先是一份准备约定，并不是 RAM 芯片自动提供的保护。

先把一次运行写成可核对的记录。操作者和启动程序需要对同一组地址达成一致。若这些数值只记在操作者的纸上，机器无法核对；若启动程序把数值全部写死，换一项程序就必须重做启动存储。于是我们在 RAM 中保留一小段固定位置，用字段描述本次准备结果。暂且称它为准备记录，地址从 `0x001ff000` 开始。

偏移与字段	本次取值	解释方式	启动前必须证明的条件
<code>+0x00 magic</code>	<code>RUN1</code>	四个固定字节	格式与启动程序约定一致
<code>+0x04 generation</code>	<code>17</code>	无符号整数	与操作者本次准备编号一致
<code>+0x08 code_base</code>	<code>0x00100000</code>	物理首地址	四字节对齐且位于 RAM
<code>+0x0c code_bytes</code>	<code>32</code>	字节数	非零、四的倍数、末地址不回绕
<code>+0x10 data_base</code>	<code>0x00102000</code>	输入首地址	四字节对齐且位于 RAM
<code>+0x14 data_bytes</code>	<code>16</code>	输入字节数	恰好容纳四个 32 位整数
<code>+0x18 result_base</code>	<code>0x00102020</code>	结果首地址	可写、对齐且不覆盖输入
<code>+0x1c stack_low</code>	<code>0x001e0000</code>	栈下界	低于 <code>stack_top</code>
<code>+0x20 stack_top</code>	<code>0x001f0000</code>	栈上界	对齐且不越过 RAM 末端
<code>+0x24 entry</code>	<code>0x00100000</code>	首条任务指令	落在代码区且四字节对齐
<code>+0x28 code_sum</code>	代码校验和	32 位无符号和	与实际 32 个代码字节相符
<code>+0x2c data_sum</code>	数据校验和	32 位无符号和	与实际 16 个输入字节相符
<code>+0x30 prepared</code>	<code>1</code>	提交标志	必须最后写入

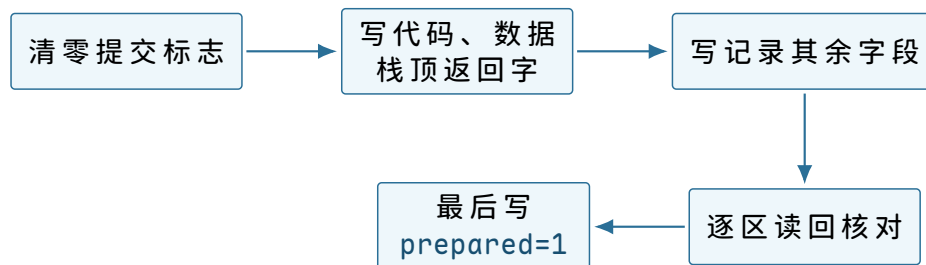
记录没有让程序自动进入 RAM。它只是把“操作者声称自己做完了什么”变成机器可读取的字节。启动程序仍要逐项验证这些声称。尤其不能看见 `prepared=1` 就直接跳转，因为旧 RAM 里可能恰好留着一个一。

为什么提交标志必须最后写。假设操作者先写 `prepared=1`，随后写到第 19 个代码字节时连接松动。启动程序若只看提交标志，就会把后半段旧字节当作新程序。相反，若提交标志最后写入，那么任意中断点都属于下面两种状态之一：

观察时刻	<code>prepared</code>	启动程序可以作出的判断
准备尚未完成	<code>0</code>	不得进入任务，不必猜测哪些区域已写完

全部写入并读回后	1	可以继续核对格式、边界与校验和
----------	---	-----------------

这里出现了本书第一次明确的提交顺序：先建立所有被依赖的内容，最后写一个代表“整组内容已经准备完成”的字段。这个字段不保证内容正确，却把“尚未完成”与“可以开始核对”分开了。



图示：顺序由操作者和调试夹具保证。机器运行以后才会读取提交标志，因此不存在处理器与夹具同时修改这些字节的情形。

代数检查为什么要在访问以前完成。判断一个区域是否落在 RAM 中，不能只检查首地址。若代码首地址为 `base`、长度为 `size`，最后一个有效字节是

$$\text{last} = \text{base} + \text{size} - 1.$$

直接用 32 位加法计算这个式子可能回绕。例如 `base=0xffffffff0`、`size=0x40` 会得到一个很小的结果，看起来反而位于低地址。启动程序因此采用不会先发生回绕的比较：

$$\text{size} > 0, \quad \text{base} \leq \text{RAM_END}, \quad \text{size} - 1 \leq \text{RAM_END} - \text{base}.$$

同一方法用于代码、输入和结果槽。两个半开区间 $[a, a+n)$ 与 $[b, b+m)$ 不重叠，当且仅当

$$a + n \leq b \quad \text{或} \quad b + m \leq a,$$

但端点加法仍需先做防回绕检查。对本次布局，启动程序应证明：

- 代码区 $[0x00100000, 0x00100020)$ 与输入区分离；
- 输入区 $[0x00102000, 0x00102010)$ 不覆盖结果槽；
- 栈区 $[0x001e0000, 0x001f0000)$ 不覆盖准备记录；
- 入口位于代码区，初始 SP 位于栈上界并满足四字节对齐。

这些检查并不创造内存保护。任务开始后仍能形成任意物理地址。它们只证明启动程序自己不会把一个显然自相矛盾的布局当作有效布局。

校验和能证明什么，不能证明什么。操作者把代码区每个字节按无符号数相加，只保留低 32 位，得到 `code_sum`；数据区同理。启动程序重新读取对应范围并复算。若结果不同，至少有一个字节、长度或地址与准备记录不一致。

简单加法校验和可能让两处相反方向的变化互相抵消，所以它不能抵抗蓄意修改，也不能证明程序含义正确。本章使用它只有一个有限目的：让最常见的漏写、错址和连接不稳在跳转前暴露。后续若需要对抗恶意内容，必须引入更强的完整性与身份机制，不能更换一个术语就假定安全性已经获得。

把具体的任务字节放入具体地址。现在可以给出这项任务的完整静态布局。指令均为四字节定长；表中“作用”是对字节编码的解释，真正存入 RAM 的仍是机器码。

物理地址	指令	成功提交后的作用
------	----	----------

0x00100000	MOVI r4,0	把累加器清零，PC 前进四字节
0x00100004	LOAD r5,[r0]	从当前输入地址读一个 32 位整数
0x00100008	ADD r4,r4,r5	把本轮整数加入累加器
0x0010000c	ADDI r0,r0,4	输入指针指向下一个整数
0x00100010	ADDI r1,r1,-1	剩余元素数减一
0x00100014	BNEZ r1,-5	若尚有元素，回到 0x00100004
0x00100018	STORE r4,[r2]	把最终和写入结果槽
0x0010001c	RET	从栈顶读取返回 PC

输入按小端字节序写入。数字 7, -2, 11, 5 的 32 位表示分别是 0x00000007、0xfffffffffe、0x0000000b 和 0x00000005。因此前八个输入字节是：

address	+0	+1	+2	+3
0x00102000	07	00	00	00
0x00102004	fe	ff	ff	ff

读取 0x00102004 的四个字节时，处理器按

$$fe + (ff \ll 8) + (ff \ll 16) + (ff \ll 24) = 0xfffffffffe$$

重组。加法单元只处理 32 位位型；把该位型解释为有符号数时才得到 -2。这一区分解释了为什么 RAM 不需要知道“这里存的是负整数”。

结果槽为何先写成已知的哨兵值。操作者在 0x00102020 写入 0xdeadbeef，而不是预先写零。运行后若仍读到哨兵值，说明结果存储没有成功提交；若预先写零而正确结果也恰好为零，就无法凭该地址区分“程序算出零”和“程序根本没有写”。

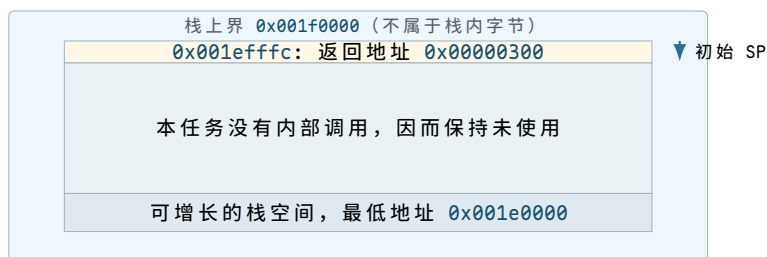
观察到的结果槽	可确定的事实	尚不能单独确定的事实
0xdeadbeef	预期结果写入尚未发生	程序未开始、运行中或已失败
0x00000015	某次写入留下十进制 21	是否由预期代码在本次运行写入
其他值	发生过非预期写入或计算	错误来自输入、代码、布局还是硬件

因此还需要准备编号和处理器停止状态共同判断。单个内存值是证据的一部分，不是完整运行历史。

最外层返回地址为何也要由操作者建立。任务最后执行 RET。按照已经定义的指令规则，它从 MEM32[SP] 读新 PC，随后把 SP 增加四。若初始 SP 直接等于栈上界 0x001f0000，第一次读取就落在栈区之外。启动前应当在最高的有效栈字写入返回地址：

MEM32[0x001efffc]	= 0x00000300
initial SP	= 0x001efffc

0x00000300 位于启动存储，其中的代码只把“任务已经合作返回”写入启动程序工作区，然后进入一个不再修改任务结果的停止循环。这里没有第二项任务，也没有调度决策；返回只把控制权交还给本次运行的固定启动程序。



图示：采用半开区间后，上界本身不是有效地址；把返回字放在 `stack_top-4`，恰好满足四字节读取。

启动程序如何判断“现在可以进入任务”。复位释放后，启动程序从 `0x00000100` 开始。它不负责把任务从外部搬进 RAM，因为这项工作已经由操作者完成；它只把不可信的 RAM 内容逐步缩小为一组可以采用的初值。

检查顺序不是任意排列。启动程序采用下列顺序：

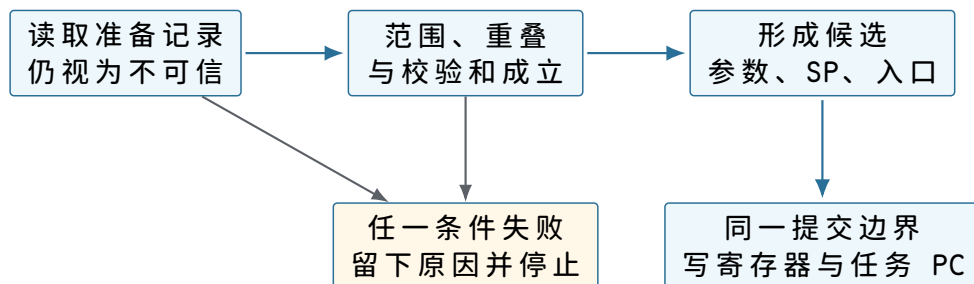
1. 读取 `prepared`。若不是一，进入“未准备”停止状态；
2. 核对 `magic` 和准备编号，排除旧格式与旧批次；
3. 检查所有地址的对齐、范围、长度与互不重叠关系；
4. 重新计算代码和输入校验和；
5. 检查返回地址位于启动存储规定的返回入口；
6. 把参数和栈值写成候选寄存器状态；
7. 最后一次重新读取 `prepared`，仍为一才提交任务入口 PC。

之所以先检查短字段，再扫描代码和输入，是因为明显无效的记录不值得触碰它声称的任意地址。之所以在跳转前重读提交标志，是为了明确提交所依据的仍是同一份准备结果。本章规定操作者在释放复位后不再使用夹具写 RAM，所以不会出现真正并发修改；第二次读取仍能让协议边界清晰，并为后续自动输入留下可比较的结构。

候选寄存器状态具体包含哪些值。

寄存器	任务入口值	该值从何而来
PC	0x00100000	准备记录的 <code>entry</code> ，已证明落在代码区
SP	0x001efffc	<code>stack_top-4</code> ，该处已有返回地址
r0	0x00102000	输入区首地址
r1	4	<code>data_bytes/4</code>
r2	0x00102020	结果槽首地址
r3	17	本次准备编号，供返回后核对
r4--r7	0	消除上电旧值对本任务的影响

启动程序自己的临时变量位于专用工作区，不能继续依赖将要交给任务的通用寄存器。建立入口状态时，它先在内部或工作区形成候选值；任何检查失败都不提交任务 PC。PC 是最后改变的状态，因为一旦 PC 指向任务入口，下一笔取指就不再属于启动程序。



图示：“形成候选值”和“让任务开始运行”是两个阶段。只有最后的 PC 提交才改变取指所属的代码区域。

六种准备错误分别停在哪里。

错误	最早能发现它的检查	为什么不能继续
提交标志仍为零	第一步	内容可能尚未全部写完
准备编号仍为 16	格式字段检查	RAM 中可能是上一次运行留下的记录
代码长度为 <code>0xffffffff0</code>	范围检查	末地址计算可能回绕，不能扫描
入口为 <code>0x00102000</code>	入口检查	它落在输入而不是代码区
代码校验和不符	完整性检查	至少一个被执行的字节与清单不一致
返回地址为奇数	栈字检查	<code>RET</code> 将产生未对齐取指地址

当前机器没有屏幕，也没有串行输出。启动程序把失败阶段和一个辅助数值写到工作区，然后执行停止指令。操作者通过夹具读取：

```

boot_status:
  phase   = PREPARED, FORMAT, RANGE, CHECKSUM, STACK, or ENTERED
  detail  = field offset or failing address
  run_id  = generation copied from the record
  
```

这仍是人工诊断。它的意义在于让每次失败对应一个已经说明的边界，而不是让机器“没反应”成为唯一现象。

这一阶段真正形成了什么。此时可以谨慎地把“可运行的一项程序”描述为四部分的组合：

指令字节 + 初始数据 + 初始处理器状态 + 开始条件。

缺少其中任何一项，物理 RAM 里即使存在某些正确字节，也不足以得到一次可重复运行。准备记录把前三项的位置写明，最后写入的提交标志给出开始核对的条件；启动程序完成验证并提交 PC，才给出真正的开始事件。

这一结构还不是文件，也不是进程映像。它没有名称查找、持久保存、权限、动态分配或多个运行实例。现在给它更大的概念名只会把尚未解决的问题藏起来。下一节将沿着这份具体布局，观察第一条任务指令怎样开始、最后一条又怎样返回。

一次人工准备的完整记录

现在把准备过程按地址和顺序展开。假设调试夹具每次最多写四个字节，并能在每笔写后读回。操作者为本次运行选择编号 17。

第一阶段：先宣告当前内容不可运行。处理器保持复位。夹具向 `0x001ff030` 写入四字节零，再读回确认。即使 RAM 中其余区域仍是上一次运行的完整内容，启动程序此刻也不得采用它。

```
write32 0x001ff030 0x00000000
```

```
read32 0x001ff030 -> 0x00000000
```

若这一步读回失败，不能继续写代码后再“看看是否能运行”。失败已经说明夹具、互连旁路或 RAM 写入路径不可靠，应在机器仍保持复位时诊断。

第二阶段：写入三十二个代码字节。

地址	写入的四个字节	读回后解释
0x00100000	10 04 00 00	清零 r4
0x00100004	20 05 00 00	从 r0 指向处装入 r5
0x00100008	11 04 05 00	$r4 \leftarrow r4 + r5$
0x0010000c	12 00 04 00	$r0 \leftarrow r0 + 4$
0x00100010	12 01 ff ff	$r1 \leftarrow r1 - 1$
0x00100014	31 01 fb ff	非零时回退五条指令
0x00100018	21 04 02 00	把 r4 写到 r2 指向处
0x0010001c	41 00 00 00	从任务栈返回

夹具按地址递增写入并读回。读回比较以字节为单位，而不是把四字节先解释成宿主机的整数，从而避免夹具所在计算机与教学处理器字节序不同造成误判。

第三阶段：写输入、结果哨兵和返回栈字。

地址	32 位写入值	用途
0x00102000	0x00000007	第一个输入 7
0x00102004	0xfffffffffe	第二个输入 -2
0x00102008	0x0000000b	第三个输入 11
0x0010200c	0x00000005	第四个输入 5
0x00102020	0xdeadbeef	尚未产生结果的哨兵
0x001efffc	0x00000300	最外层 RET 的返回地址

结果槽与输入末端之间保留十六字节空隙不是硬件要求，而是为了让本例中的错一位地址更容易观察。栈下方大部分空间暂未写入，因为任务没有内部调用；不能因此假定那些字节为零。

第四阶段：写准备记录但仍不提交。

```
0x001ff000 magic          = bytes "RUN1"
0x001ff004 generation     = 17
0x001ff008 code_base      = 0x00100000
0x001ff00c code_bytes     = 32
0x001ff010 data_base      = 0x00102000
0x001ff014 data_bytes     = 16
0x001ff018 result_base    = 0x00102020
0x001ff01c stack_low      = 0x001e0000
0x001ff020 stack_top      = 0x001f0000
0x001ff024 entry          = 0x00100000
0x001ff028 code_sum       = computed checksum
0x001ff02c data_sum       = computed checksum
0x001ff030 prepared       = 0
```

操作者重新从记录读取代码首地址与长度，再按记录指定范围读回代码并复算校验和，而不是只拿手边原始清单计算。这样可以同时发现“字节写对但记录地址写错”的情况。数据区同理。

第五阶段：最后四字节才改变准备结论。全部读回一致后，夹具执行唯一的提交写：

```
write32 0x001ff030 0x00000001
read32 0x001ff030 -> 0x00000001
```

随后操作者停止使用夹具写 RAM，再释放复位。若在提交后又修补一个代码字节，原校验和与“提交后内容不变”的前提同时失效，必须重新清零标志并完整核对。

启动检查怎样对应人工记录。启动程序的检查顺序可以用一张状态表精确表示。每个失败状态都保留准备编号、阶段和一个辅助值。

阶段	成功条件	失败时 detail 保存什么
READY	prepared=1	实际读到的标志
FORMAT	magic=RUN1	第一个不匹配的字节偏移
RANGE	所有区域范围合法且互不冲突	失败字段偏移或区域对
CODE	代码校验和相等	重新计算所得值
DATA	输入校验和相等	重新计算所得值
STACK	栈顶字为合法返回地址	实际返回字
ENTER	再次读取提交标志仍为一	第二次读到的值

成功检查的一组关键快照。

快照	当前 PC 范围	已证明的内容	尚未证明的内容
S_0	启动入口	复位起点确定	RAM 是否属于本批次
S_1	记录检查	标志、格式、编号正确	区域内容完整性
S_2	范围检查	地址和长度关系合法	实际字节是否匹配
S_3	校验循环	代码、输入校验和匹配	任务算法是否正确
S_4	交接准备	参数、栈和入口候选完整	任务是否最终返回
S_5	任务入口	入口状态已经提交	结果尚未产生

这组快照避免把不同层次的证明混合。校验和匹配只说明扫描到的字节与记录一致，不说明机器码一定实现求和；算法正确性要由译码表、指令 trace 和循环不变量说明。

一次校验失败的完整状态。假设地址 `0x0010000a` 的字节应为 `05`，实际读回为 `04`。准备记录中的 `code_sum` 仍按正确代码计算。启动程序扫描 32 字节后得到不同总和，于是写：

```
boot_status.phase = CODE
boot_status.detail = recomputed_sum
boot_status.run_id = 17
processor PC      = checksum_failure_stop
```

它不把 PC 改为 `0x00100000`，也不保留一部分任务参数作为有效入口状态。操作者保持或重新拉起复位，清零提交标志，修复字节，再重新扫描整个代码区。只重算出错位置之后的部分不足以证明此前字节未被同时改变。

人工准备为何无法成为长期办法。本例只有 32 个代码字节、16 个输入字节和一份短记录，操作者尚可逐项核对。若程序扩大到 64 KiB，以每笔四字节写入计算，需要 16384 笔写和同量级读回；任意一次地址抄错都可能迫使整段重新核对。更换输

入还要重复编辑记录和准备编号。

问题不在于操作者“不够仔细”，而在于机器没有承担可机械重复的搬运工作。已经存在的启动程序能读取 RAM、计算校验和、建立入口状态，却没有获得外部字节。下一章增加的第一个部件因此应当只解决“怎样让启动程序可靠地接收一串字节”，而不同时引入第二项任务或定时器。

从复位到任务入口：控制权怎样真正改变

RAM 已由操作者完整准备，但处理器尚处于复位状态。现在释放复位。为了避免把“启动”写成无法检查的瞬间，本节沿时间顺序记录谁发起事务、哪些状态只是候选、哪些状态已经提交。

复位只建立最少的处理器起点。 复位电路规定：

```
PC      = 0x00000100
halted  = 0
control = FETCH
SP, r0 ... r7 = unspecified
```

通用寄存器仍是不确定值，所以启动程序不能先使用旧 SP 压栈，也不能假定某个寄存器已经为零。启动程序开头使用处理器内部复位时可用的少量暂存状态，先把自己的工作栈设为 0x001fd000，随后才执行普通调用。这个栈属于启动程序工作区，与任务栈分离。

状态	复位刚释放	启动程序初始化后
PC	0x00000100	位于记录检查代码中
SP	未规定	0x001fd000
r0--r7	未规定	启动程序按自身约定使用
任务参数	尚不存在	仍未提交，只保存在候选区

这里的“未规定”不是随机数发生器，而是软件不得依赖的值。某次实验恰好观察到零，不会把它升级为体系结构承诺。

第一条装入为何分成两笔读取。 位于 0x00000100 的第一条指令要读取准备记录的提交标志。处理器先按 PC 取回指令字，再执行该指令形成的数据地址 0x001ff030。两笔事务不可混为一笔：

事务	地址	目标	返回内容
取指	0x00000100	启动存储	LOAD 的四个编码字节
数据读	0x001ff030	RAM	提交标志 1

第一笔读取回答“要执行什么动作”，第二笔读取才回答“提交标志是什么”。只有第二笔响应成功后，目标寄存器和顺序 PC 才提交。若准备记录地址未映射，处理器保留 fault_pc=0x00000100，而不是错误地报告下一条指令。



图示：PC 在数据读取等待期间仍指向产生该读取的指令。这使失败位置具有唯一含义。

验证完成以前，任务寄存器仍不能被看作有效。启动程序可能暂时把 `entry` 读入某个寄存器，但这不表示它已经接受入口。只有格式、范围、重叠、校验和和栈字全部通过后，候选区才包含一组相互一致的值。

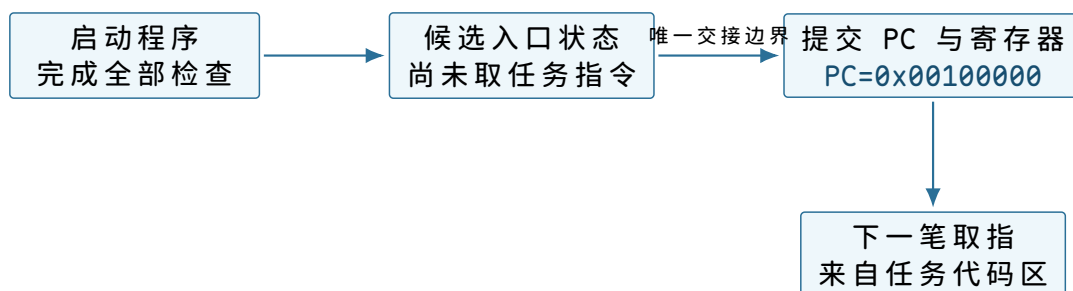
```
candidate:
  task_pc = 0x00100000
  task_sp = 0x001efffc
  r0      = 0x00102000
  r1      = 4
  r2      = 0x00102020
  r3      = 17
  r4..r7  = 0
```

若代码校验失败，启动程序丢弃整个候选区并停止，而不是保留已经读出的三个参数、只修补其中一个。这样，“任务入口状态”要么完整提交，要么完全没有出现。

交接边界上到底保存了什么、改变了什么。验证结束时，启动程序先把自己的继续地址 `0x00000300` 放入任务栈顶，再写任务通用寄存器和 SP，最后提交任务 PC。紧邻提交边界的两份快照如下。

状态	交接前最后一刻	交接后第一刻
PC	启动程序交接指令地址	<code>0x00100000</code>
SP	启动程序工作栈	<code>0x001efffc</code>
r0	启动程序临时值	<code>0x00102000</code>
r1	启动程序临时值	4
r2	启动程序临时值	<code>0x00102020</code>
r3	启动程序临时值	17
r4--r7	启动程序临时值	全部为零
启动程序工作栈	保留检查过程的帧	原样留在 RAM，但任务不应使用

处理器并没有一个名为“任务”的硬件开关。它只在同一时钟提交边界写入这些寄存器，并让下一笔取指使用新的 PC。我们把边界前后的两段执行分别称为启动程序与任务，是因为它们的代码地址、数据约定和责任不同。



图示：交接不是把代码“搬进处理器”，而是改变处理器下一次取指所使用的地址，并按约定建立这段代码将读取的寄存器和栈。

第一项任务怎样逐步得到 21。任务入口处的静态指令已经给出。现在不再用“循环执行四次”一句带过，而是从第一次取指开始，区分指令地址、数据地址和提交后的状态。

入口指令只清零累加器。第一笔任务取指访问 `0x00100000`。互连根据地址选择 RAM，返回 `MOVI r4,0` 的编码。处理器译码后形成两个候选：

```
candidate r4 = 0x00000000
candidate PC = 0x00100004
```

这条指令不访问数据 RAM。提交时同时写 `r4` 和 `PC`。其他寄存器保持入口值，尤其 `r0` 仍指向第一个输入，`r1` 仍为四。

第一轮装入为何同时出现两个地址。 `PC` 为 `0x00100004` 时，取指地址是 `0x00100004`；译码得到 `LOAD r5,[r0]` 后，读取旧 `r0=0x00102000`，形成数据地址 `0x00102000`。前者指向“装入动作的编码”，后者指向“被装入的整数”。

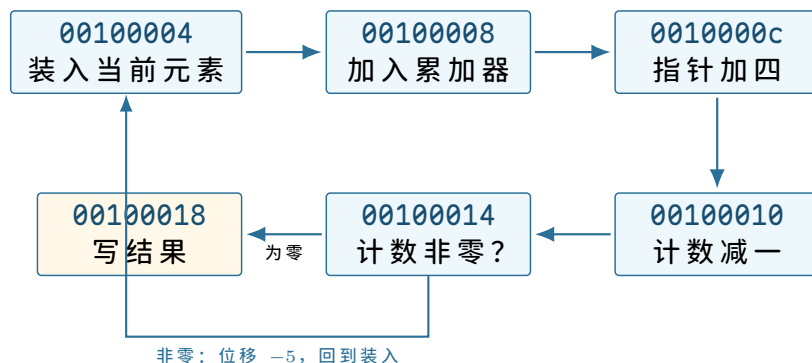
阶段	保存的事实	尚未允许发生的效果
取指完成	当前 <code>PC</code> 、操作码、目标寄存器号	不得写 <code>r5</code>
数据请求被接受	地址、宽度、目标 <code>r5</code> 、后继 <code>PC</code>	不得开始下一条指令
RAM 返回 <code>0x00000007</code>	候选 <code>r5=7</code>	响应成功前仍不提交
装入提交	<code>r5=7, PC=0x00100008</code>	无其他寄存器变化

随后 `ADD r4,r4,r5` 读取旧值 0 与 7，候选结果为 7，提交后 `r4=7`。再后两条指令分别令输入指针增加四、剩余计数减少一。

分支位移怎样回到正确的指令。 分支位于 `0x00100014`。它的顺序后继是 `0x00100018`，编码中的有符号位移为 -5 个指令字，即 -20 字节：

$$0x00100018 + (-20) = 0x00100004.$$

第一轮结束时 `r1=3`，所以采用目标地址。处理器不会重新执行清零指令；循环入口刻意放在装入处，累加器由此保留上一轮结果。



图示：回边指向装入指令而不是初始化指令。地址计算使“保留累加器”成为可以核对的控制流事实。

四轮以后哪些值应当精确相等。

轮次结束	本轮元素	<code>r4</code>	<code>r1</code>	<code>r0</code>
第一轮	7	7	3	<code>0x00102004</code>
第二轮	-2	5	2	<code>0x00102008</code>
第三轮	11	16	1	<code>0x0010200c</code>
第四轮	5	21	0	<code>0x00102010</code>

第四轮分支不采用回边，`PC` 顺序前进到 `0x00100018`。注意 `r0=0x00102010` 指向输入区末端的第一个字节，不再指向某个有效元素。这是常见的半开区间表示：已处理范围为 `[0x00102000, 0x00102010)`。

结果写入何时才算完成。 `STORE r4,[r2]` 形成地址 `0x00102020`、数据 `0x00000015` 和四个有效字节。小端写入后的 `RAM` 为：

`0x00102020 = 15`


```

0x00102021 = 00
0x00102022 = 00
0x00102023 = 00

```

RAM 在检查地址和字节使能后，于自己的写提交边界替换四个字节并返回 OK。处理器收到成功响应后才把 PC 改为 0x0010001c。若连接在响应前失败，处理器不能假定结果已写；本章 RAM 契约则保证目标要么接受全部四字节，要么一个也不改变。

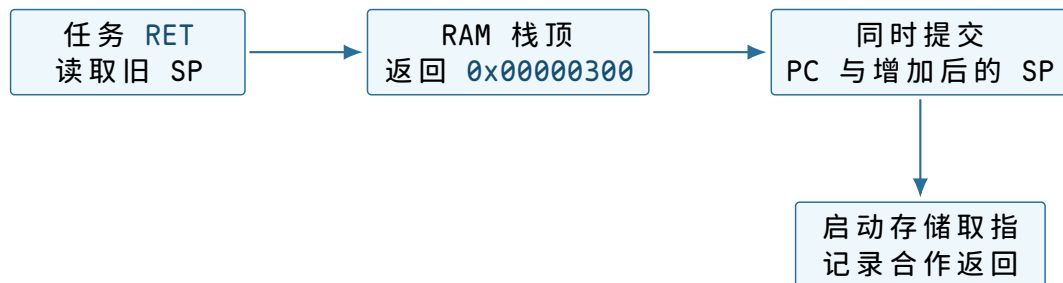
RET 怎样把控制权交回启动程序。执行 RET 前：

```

PC           = 0x0010001c
SP           = 0x001efffc
MEM32[0x001efffc] = 0x00000300

```

处理器读取栈顶，验证返回地址四字节对齐且位于可取指范围，然后形成 new_PC=0x00000300 与 new_SP=0x001f0000。两者在同一指令提交边界更新。下一笔取指重新落入启动存储。



图示：返回地址不是处理器猜出的“调用者”。它是启动前写入栈中的四个具体字节。

把整次运行放到同一条时间线上。现在可以从操作者开始准备一直排到结果被读回。表中的“可见状态”只列已经提交的事实。

阶段	主动者	关键动作	阶段结束时可依赖的事实
T ₀	操作者	保持复位，清零提交标志	处理器不发起普通事务
T ₁	调试夹具	写代码、输入、哨兵值和返回字	各区域存在，但尚未宣告完整
T ₂	调试夹具	写记录、读回、最后提交	prepared=1 且读回一致
T ₃	复位电路	在稳定边沿释放复位	PC=0x00000100
T ₄	启动程序	检查记录、范围、校验和与栈	候选入口状态完整
T ₅	启动程序	提交参数、SP 与任务 PC	下一笔取指属于任务
T ₆	任务	四轮装入与加法	r4=21, r1=0
T ₇	任务	写结果并执行 RET	结果槽为 21，PC 回到启动存储
T ₈	启动程序	写返回状态并停止修改结果	操作者可以稳定读回

“程序运行结束”在这台机器上不是一个物理引脚。我们把它定义为三个可共同核对的条件：

1. 启动程序工作区的 boot_status.phase 为 RETURNED；

2. 其中记录的准备编号仍为 17；
3. 结果槽不再是哨兵，并且本例值为 `0x00000015`。

仅满足第二项不能证明任务运行过；仅满足第三项不能排除旧 RAM；仅看到 PC 位于启动存储，也可能是验证尚未结束。组合条件把“本次运行、合作返回、结果可读”联系起来。

如果任务永远不执行 RET。把分支位移改成让程序永远回到自身。任务一旦进入该循环，时钟仍不断推动取指与分支，启动程序却没有机会继续执行。当前机器没有能在任务运行中主动改变 PC 的其他部件，也没有第二份处理器状态。

操作者只能重新拉起整机复位。复位会丢弃正在运行的寄存器值，并从 `0x00000100` 重新开始。若 RAM 仍保留原内容，启动程序还必须借助新的准备编号或人工清除提交标志，避免误把旧批次当作一次新运行。

如果任务写错了地址。任务中的 `STORE` 若把 `r2` 改为代码地址，RAM 会按普通写请求接受，因为当前互连只按地址选择目标，不检查“这段字节原来被当作代码”。本次最后一次取指已经完成时，错误写入甚至可能不影响本次结果，却会破坏下一次运行。

这说明启动前的区域不重叠检查只约束初始布局，不能约束运行中的每笔访问。要让某些地址在任务运行时不可写，后续必须增加会参与每笔事务判定的新机制，并说明判定依据从何而来。本章尚未获得这种能力。

如果处理器记录了故障。无效操作码、未映射地址、未对齐访问或向启动存储写入都会使普通效果不提交。处理器锁存 `fault_pc`、`fault_address` 与 `fault_cause`，再把 `halted` 置一。此后时钟仍存在，但取指状态机保持停止。

操作者读取记录后能定位最初失败的指令，却不能让任务从那里继续。复位清除 `halted` 的同时也重新建立 PC，所有未另行保存的寄存器进度都会丢失。故障记录因此属于整机诊断，不属于任务恢复。

到这里能够完整描述的运行。操作者在复位期间建立代码、数据、栈和准备记录；启动程序只验证这些既有字节并提交一组入口状态；处理器随后顺序执行一项任务，任务把 21 写入结果槽并合作返回。任何阶段失败都停止或依赖整机复位，没有后台程序、第二项任务、自动输入设备或运行中强制转移。

怎样证明描述的确是同一次运行

结果槽中出现 21 很有吸引力，但它不是充分证据。RAM 可能保留了上一次的 21，操作者也可能提前把该值写入。要判断这一次运行，必须把准备、交接、执行和返回四个阶段的证据连成一条不矛盾的链。

为每个阶段选择最小而不同的证据。

要证明的事件	使用的证据	单独使用时的不足
本批次已准备完整	<code>prepared=1</code> 、编号 17、两个校验和	只能说明启动前字节通过检查
任务入口已经提交	<code>phase=ENTERED</code> 、记录的入口 PC	不能说明任务走到循环末尾
结果存储已经发生	哨兵被替换为 <code>0x00000015</code>	不能单独排除旧结果或其他写入者
任务已经合作返回	<code>phase=RETURNED</code> 、编号 17	不能替代对结果数值的检查
没有处理器故障	<code>halted=0</code>	不能证明程序语义正确

启动程序在进入任务前把 `phase` 写为 `ENTERED`，返回入口把它改为 `RETURNED`。两次写入都同时复制准备编号。因为当前只有一个处理器和一个运行程序，不存在另一个合法写入者；即便如此，仍要把状态字和结果槽共同检查。

提交次序给证据建立先后关系。我们可以从已定义的提交边界推出：

1. `phase=ENTERED` 的写响应成功后，启动程序才提交任务入口 `PC`；
2. 任务四轮循环结束后，才可能到达结果 `STORE`；
3. 结果写响应成功后，任务才取到 `RET`；
4. `RET` 成功提交后，返回入口才可能写 `phase=RETURNED`。

于是同一编号下的 `RETURNED` 蕴含控制流已经越过结果存储的成功响应。但它仍不蕴含结果一定是 21，因为错误的算法也可以合作返回。最终数值要由指令 `trace` 和输入字节独立复算。



图示：箭头来自具体指令和提交规则，不是状态名称本身带来的保证。若将来出现并发写入者或缓存，这条证据链需要重新说明。

从循环不变量复算结果。逐条 `trace` 可以展示每条动态指令；另一个互补方法是找出每轮都保持的关系。设输入数组为 $a_0, \dots, a_3$ ，刚要开始第 k 轮时：

$$\begin{aligned} r0 &= \text{data_base} + 4k, \\ r1 &= 4 - k, \\ r4 &= \sum_{i=0}^{k-1} a_i. \end{aligned}$$

当 $k=0$ 时，入口初始化给出 $r0=\text{data_base}$ 、 $r1=4$ 和 $r4=0$ ，关系成立。

假设第 k 轮开始时关系成立。装入得到 a_k ，加法令累加器成为 $\sum_{i=0}^k a_i$ ，指针增加四，计数减少一。因此下一轮开始时：

$$r0 = \text{data_base} + 4(k+1), \quad r1 = 4 - (k+1),$$

不变量继续成立。当第四轮结束时 $r1=0$ ，分支退出，累加器为

$$7 + (-2) + 11 + 5 = 21.$$

这个推导依赖三项此前已具体建立的事实： $r1$ 初值确为四、每个元素恰为四字节、回边指向装入而不指向清零。若任何一项变化，证明必须随之修改。

32 位加法是否会溢出。本例的所有部分和都位于有符号 32 位范围，故位型加法与数学整数加法得到相同解释。一般输入可能溢出。本章处理器的 `ADD` 保留低 32 位，不自动停止：

$$0x7fffffff + 1 = 0x80000000.$$

若把结果解释为有符号数，它是 -2147483648 。因此“求和”在当前指令契约中准确含义是 32 位模加法；本例恰好无需区分。需要报告算术溢出的程序必须检查标志或使用更宽表示，不能从自然语言“相加”推断硬件会代劳。

用反事实检查准备协议的每一道边界。好的协议不仅要说明成功路径，还应能回答“少做一步会怎样”。下面每个实验只改变一个准备动作。

没有先清零旧的提交标志。RAM 中仍有编号 16 的完整记录，操作者直接开始覆盖代码，尚未写完就释放复位。若启动程序只检查 `prepared=1`，它可能扫描一半新、一半旧的代码。加入准备编号和校验和后，最迟在完整性检查处停止；而正确操作在第一步清零标志，可以更早明确“尚未完成”。

只检查代码首地址，不检查长度。令 `code_base=0x001ffff0`、`code_bytes=32`。首地址位于 RAM，但区域末端越过 `0x001fffff`。若启动程序逐字节复算校验和，第十七个字节已经没有目标，互连返回 `UNMAPPED`。预先做不回绕的范围检查能在任何扫描前拒绝该记录，并把原因定位到字段关系，而不是一次偶然的访存失败。

代码与输入区域重叠。若 `data_base=0x00100010`，后两个输入整数会覆盖计数更新和分支指令。两个区域各自都位于 RAM，两个校验和也可能分别按被覆盖后的内容计算成功；只有跨区域重叠检查能指出同一批字节被赋予了互不相容的角色。

先改变 PC，再写完任务寄存器。假设交接代码先提交 `PC=0x00100000`，下一周期才准备 `r0`。第一条任务指令清零 `r4` 后，第二条立即读取旧 `r0`，可能访问任意地址。入口 PC 与参数必须属于同一个交接提交边界，或者硬件必须保证提交后才开始取指。本章采用前者。

结果写入成功，但返回栈字损坏。四轮计算和结果 `STORE` 均可成功，随后 `RET` 读到未对齐地址并使处理器停止。此时结果槽为 21，`phase` 仍为 `ENTERED`，故障原因为返回目标非法。这个例子说明“结果已产生”与“任务已合作返回”是两个不同事件；状态设计需要同时容纳二者。

单点变化	预期停止位置	由哪项状态区分
提交标志未清零	编号或校验检查	<code>phase</code> 与 <code>detail</code>
代码区域越过 RAM	范围检查	失败字段偏移
代码与输入重叠	区域关系检查	两个区域编号
入口寄存器非原子建立	任务早期访存	<code>fault_pc</code> 与数据地址
返回栈字损坏	<code>RET</code>	结果已变、状态仍为 <code>ENTERED</code>

重复运行为什么不能只按一次复位。第一次任务返回后，RAM 至少有三处变化：结果槽已从哨兵变为 21，准备状态变为 `RETURNED`，任务也可能改写自己的栈。若操作者只复位处理器，启动程序看到的仍是编号 17 和旧结果。

本章规定每次运行采用新的准备编号，并重新执行以下动作：

1. 保持复位，先把 `prepared` 清零；
2. 写入新的编号，恢复输入、结果哨兵和初始栈字；
3. 必要时重写代码并读回全部声明区域；
4. 写入新的校验和，最后重新提交；
5. 释放复位。

即使代码没有变化，也不能省略恢复数据与结果槽，因为程序语义可能依赖初值。若希望快速重复运行而不重写不变代码，需要进一步区分只读模板与每次运行的可变状态，并由机器自己完成复制；这已经是后续演进，不应暗含在本章的人工步骤中。

复位、重新准备和重新运行是三个不同动作。

动作	确定改变的状态	不保证改变的状态
----	---------	----------

复位	PC、控制状态、halted	RAM 中的代码、输入、结果与记录
重新准备	声明的 RAM 区域与准备编号	处理器当前 PC 和内部执行状态
重新运行	从固定入口推进并产生新效果	断电后的保存、其他机器上的结果

所以正确顺序是先保持复位，再准备，最后释放复位。运行中直接让夹具改代码，会产生一份本章没有定义的交错历史。

当前机器的责任边界

经过完整走读，我们可以把每个部件承担的责任写得足够窄。

部件或参与者	它负责保证	它不负责保证
操作者与夹具	复位期间按顺序写入并读回声明字节	运行中自动提供新程序
启动存储	复位后提供不变的检查与返回指令	保存任务运行产生的结果
RAM	对成功事务保存和返回相应字节	理解代码、输入、栈等含义
地址互连	每个地址选择唯一目标并返回错误	阻止任务访问别的 RAM 区域
处理器	按指令提交规则更新 PC、寄存器与访存效果	保证任务返回或算法正确
启动程序	验证人工记录并建立一次入口状态	并发运行、自动装入或故障恢复
任务	按约定读取参数、写结果并合作返回	保护启动程序工作区

这张表没有额外创造一个新层次。它只是防止把某个部件的局部承诺扩大成整机承诺。例如 RAM 能够正确保存写入字节，不等于任务不能写错地址；启动程序检查入口合法，也不等于任务运行中只能在代码区取指。

首次运行已经得到的三个重要分界。第一，字节存在与可以开始执行不同。代码、数据、栈和入口状态共同通过检查以后，PC 的提交才建立开始事件。

第二，候选效果与已提交效果不同。装入等待响应时不能提前写目标寄存器，存储失败时不能留下半个新整数，交接失败时不能让任务看到半套参数。

第三，结果出现与控制权返回不同。结果写入发生在 STORE 提交，控制权返回发生在 RET 提交；状态记录让两者可以分别观察。

这些分界会在后面的机器演进中反复出现，但后续章节必须从新的具体问题重新推导它们的用途，而不能只复述三个口号。

当前机器仍然不能做的事情。

- 操作者必须逐字节或逐块写 RAM，程序越大，准备时间越长；
- 每次换程序都要人工计算地址、校验和、栈字和记录字段；
- 运行中的任务独占处理器，除故障停止和整机复位外，没有强制收回途径；
- 所有 RAM 地址对任务可见，初始分区没有运行期保护；
- RAM 断电后不提供本章可依赖的长期保存；

- 机器一次只有一组任务寄存器，没有第二个可暂停后继续的现场。

下一步最直接的困难不是同时运行很多任务，而是人工装入本身。假设程序由几万字组成，继续要求操作者借助夹具写入每个地址，既慢又容易把地址、长度或字节顺序抄错。要让机器从一条外部字节通道自动接收同样的代码与数据，就必须新增接收硬件，并让启动程序承担接收、校验和最后交接。这个问题将单独构成下一章。

逐条指令 trace：不省略循环中的状态。前面的四轮表只保留了每轮结果。下面把任务从入口到返回的 23 条动态指令全部列出。为了控制表格宽度，地址省略共同的前导 0x00；“执行后”表示该指令成功提交后的体系结构状态，未列出的寄存器和内存保持不变。

序号	指令地址	执行动作	提交后的关键状态
1	100000	MOVI r4,0	r4=0, PC=100004
2	100004	从 102000 装入 7	r5=7, PC=100008
3	100008	0 + 7	r4=7, PC=10000c
4	10000c	输入指针加 4	r0=102004
5	100010	计数减 1	r1=3
6	100014	r1!=0, 采用分支	PC=100004
7	100004	从 102004 装入 -2	r5=-2
8	100008	7 + (-2)	r4=5
9	10000c	输入指针加 4	r0=102008
10	100010	计数减 1	r1=2
11	100014	r1!=0, 采用分支	PC=100004
12	100004	从 102008 装入 11	r5=11
13	100008	5 + 11	r4=16
14	10000c	输入指针加 4	r0=10200c
15	100010	计数减 1	r1=1
16	100014	r1!=0, 采用分支	PC=100004
17	100004	从 10200c 装入 5	r5=5
18	100008	16 + 5	r4=21
19	10000c	输入指针加 4	r0=102010
20	100010	计数减 1	r1=0
21	100014	r1==0, 不分支	PC=100018
22	100018	向 102020 存储 21	MEM32[102020]=21
23	10001c	读取栈顶并返回	PC=000300, SP=1f0000

动态指令数可以由结构独立复算。初始化执行一次；循环体包含装入、加法、指针更新、计数更新和分支，共 5 条，执行四轮得到 20 条；最后存储和返回各一次，所以总数为 $1 + 20 + 2 = 23$ 。算式与逐行编号一致，说明循环次数和退出路径没有遗漏。

这类自我校验正是使用具体程序的价值。抽象文字很难暴露“循环到底有几条动态指令”这样的小错误，而带编号的 trace 会立即产生矛盾。

事务数与指令数不是同一个数量。每条指令至少需要一次取指事务，共 23 笔。四条动态 LOAD 再产生 4 笔数据读，STORE 产生 1 笔数据写，RET 产生 1 笔栈读。

因此不考虑启动前的人工准备，任务总共发出：

$$23 + 4 + 1 + 1 = 29$$

笔外部事务。

算术指令也会读取寄存器，但寄存器堆位于处理器内部，不经过地址互连。把所有“读”都称为内存事务会高估外部访问；反过来只数显式 **LOAD/STORE**，又会漏掉取指和返回栈读取。

若 RAM 响应时间不同，程序结果是否改变。设启动存储读取需 2 个周期，RAM 读取需 3 个周期，RAM 写入需 2 个周期。教学处理器在外部事务期间等待，因此总运行周期会增加，但只要每笔事务最终按同一顺序成功，23 条动态指令提交后的寄存器和内存结果不变。

这就是功能抽象与性能事实的分界：软件依赖读取返回相应地址的字节，不应依赖每笔读取恰好几个周期；若程序需要计时，必须另有可读计时器契约。

三组反例：改变一个初值会发生什么。

元素个数为零。当前代码先执行一次 **LOAD**，然后才减少计数并判断。因此若启动时 $r1=0$ ，它仍会读取一个元素，随后计数变为 **0xffffffff**，分支继续，最终很可能越过数组。这说明启动契约必须要求 $r1>0$ ，或者任务代码要在第一次装入前增加零长度检查。

硬件不会替程序选择哪一种语义。若任务格式允许空数组，正确代码应先判断计数；若格式规定至少一个元素，启动程序应验证输入。约束放在哪里，会决定错误由哪个层次报告。

结果地址与输入重叠。若启动程序把 $r2$ 错设为 **0x00102008**，最终存储会覆盖第三个输入整数 11。由于写发生在所有读取之后，本次结果仍是 21；但任务返回后输入数据已改变。若算法在写结果后还要重读数组，行为就会变化。

“这一次碰巧正确”不能证明布局合法。装入契约若要求输入保持不变，就应让结果槽与输入范围不重叠，并在建立参数时检查。

数组末尾越过 RAM。假设 $r0=0x001ffffc$ 、 $r1=2$ 。第一轮能读取 RAM 最后 4 字节；指针随后变为 **0x00200000**，第二轮地址不在任何目标范围，互连返回 **UNMAPPED**。检查数组范围时应验证：

$$\text{count} \leq \frac{\text{RAM_END} - \text{array_base}}{4},$$

并在乘法前避免 $\text{count} \times 4$ 溢出。

把器件承诺重新放回整机。现在所有交互已经走读完成，可以进行一次横向复核：

器件	保存或处理的事实	程序可依赖的接口	当前整机得到的承诺
时钟与复位	边沿、复位有效状态	固定复位 PC	确定起点与状态提交节拍
处理器	PC、SP、寄存器、译码状态	指令集与调用约定	可顺序推进的执行流
启动存储	非易失字节阵列	只读物理范围	复位后立即存在的固件字节
RAM	易失可写单元	可读写物理范围	运行期可变字节空间
地址互连	地址比较和响应路由	物理地址表与错误码	地址唯一选择目标

最右列刻意使用有限承诺。RAM 不承诺断电保存，互连不承诺权限，处理器也不承诺当前任务最终返回。准确列出这些限制，是为了让下一次增加硬件或程序时能够指出它究竟补上了哪一项缺口。

章末自检：能否独立重建这次运行。读完本章后，应当能够不借助“系统自动完成”这样的句子回答以下问题。

1. 复位释放后，为什么第一笔取指访问启动存储而不是任务 RAM？
2. `LOAD r5,[r0+0]` 的指令地址、数据地址和目标寄存器号分别是什么？
3. 为什么最后一个人工写入字节完成不等于任务已经可以执行？
4. 哪些字段共同证明入口地址合法？哪个算术检查必须防止 32 位回绕？
5. 启动程序为什么要分别建立任务 SP 和自己的 SP？
6. PC 在什么确切时刻从启动程序范围进入任务范围？此前谁控制取指次序？
7. 结果写入 RAM、任务返回、操作者稳定读回分别由什么事件完成？
8. 如果任务进入无限循环，当前机器中的哪个器件能够强制改变它的 PC？

前七问都能沿本章给出的地址、字段和事务找到具体答案。第八问的答案是“没有”。普通时钟只推动当前指令流，当前也没有能够请求控制转移的外部部件。这一空缺不是概念定义，而是机器在无限循环反例中的可观察事实。

第一章得到的机器。它能从确定复位入口启动，验证一项由操作者预先写入 RAM 的程序，为该程序建立参数和栈，逐条执行到结果写回，并在程序合作时返回启动代码。它尚不能自动接收程序。下一章只增加一条外部字节通道，由“人工准备为何不可持续”推导接收、校验与交接程序。

为什么人工装入需要变成机器自己的工作

继承的机器。操作人员能在复位期间把一项程序、输入和初始栈写入 RAM；释放复位后，启动程序建立寄存器并移交处理器。程序可以计算并合作返回，但更换程序仍要停机并借助外部写入工具。

现在只改变一个条件：机器已经装入机箱，运行期间不能再由操作人员逐地址改写 RAM。外部设备仍然能够按顺序送来字节，但机器必须自己接收、验证并放置这些字节。为此，本章才增加串行控制器和装入程序；它们不负责同时保留两项运行现场，也不能在程序失控时收回处理器。

从人工写入转向运行中的字节接收。上一章的操作人员可以在复位期间逐地址写 RAM。机器一旦离开实验台，这种方式就出现了具体成本：每次更换程序都要停机、连接调试工具、重新写入多个区域，并由人保证最后一个字节已经完成。机器自身无法重复这套过程。

一种选择是把永远不变的程序直接固化在启动存储中。这样不再需要每次装入，但更换程序要重新写非易失器件；输入参数和可写栈仍要另行准备。本章需要的是“保持固定启动程序不变，运行期间接收另一份可替换程序”，所以必须给运行中的处理器增加一个能够取得外部字节的接口。

为什么不能收到一条指令就立刻执行。外部连接按时间送来字节，处理器执行程序却会分支回到先前地址。若机器收到四个指令字节就立即执行，循环第二次取指时，先前字节已经从顺序数据流中消失。除非外部端同时承担一片可随机访问的指令存储，否则流式执行无法支持本书的求和循环。

另一个问题是完整性。前几条指令到达后便开始执行，后续传输若中断或校验失败，已经执行的指令可能改写 RAM。此时不能再把整次接收当作“从未发生”。因此，基本方案先把全部程序写入 RAM，验证通过以后才改变处理器入口。

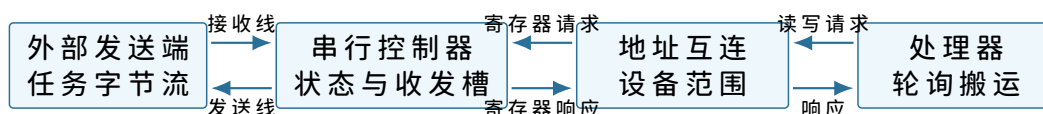
方案	省去的步骤	失去的能力或边界
程序固化在启动存储	每次传输与装入	不能方便替换程序
收到一条执行一条	完整 RAM 映像	不能可靠分支回旧指令
接收与执行重叠	等待完整传输的时间	不能在产生效果前完成整体验证
先接收、验证、再移交	无	需要接收状态、RAM 区域和明确提交点

最后一行没有宣称适用于所有机器。它只是给本章提供一条可检查的基本路径：外部字节先成为完整映像，处理器随后一次性从启动程序切到新入口。串行控制器只负责把线上位流整理成字节；映像格式、范围检查和最终移交仍由启动程序负责。

让外部任务进入机器：串行控制器

可以把求和任务永久写进启动存储，但那样每换一次任务都要重新烧写固件。本章为机器增加一个最小串行控制器。外部发送端在一根接收线上按时间发送位，控制器把位组合成字节，再把字节暴露为处理器可以寻址的状态。发送方向用于固件报告接受、拒绝和最终结果。

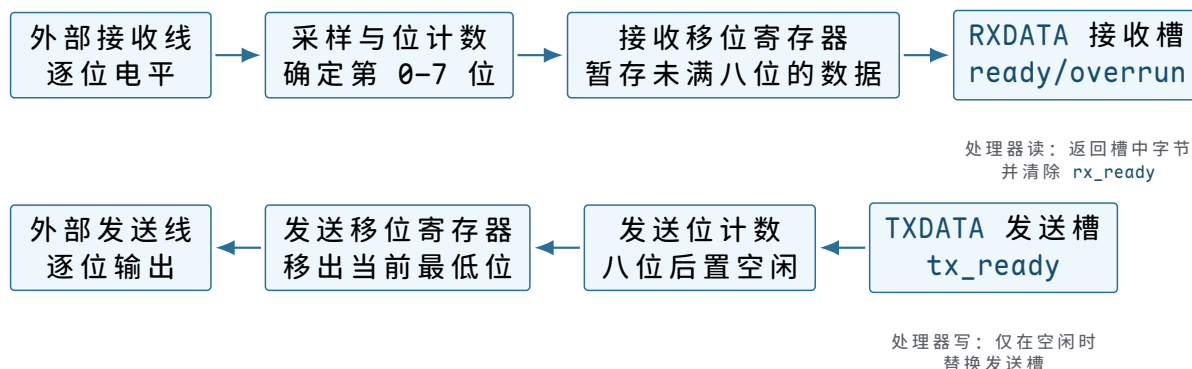
新器件接在哪里。 串行控制器有两类连接。第一类是机器外部的收发信号；第二类是接入现有地址互连的配置与数据端口。处理器不直接观察线上的每一位，而是通过内存映射寄存器读取已经组装好的字节。



图示：串行线与处理器请求不是同一种事件。控制器先在外围时间域接收位，再把完整字节保存在内部槽中；处理器随后通过互连读取这个槽。

串行控制器内部保存什么。硬件模型。 接收移位寄存器按串行时序收集位，形成一个字节后写入 `rx_data` 并置 `rx_ready=1`。处理器读取 `RXDATA` 后清除 `rx_ready`。若旧字节尚未读取而新字节到达，控制器置 `rx_overrun=1`。发送侧在 `tx_ready=1` 时接受一个字节，移出期间清零 `tx_ready`，发送完毕后重新置位。

接收线上的一个电平不能直接写进 `RXDATA`。采样状态先在约定时刻取得每一位，移位寄存器按位次保存它们，位计数器在收满八位时产生“字节完成”。只有这个事件才允许接收槽替换旧值并设置 `rx_ready`。发送方向按相反顺序工作：处理器写入发送槽，移位状态逐位取出，位计数归零后才重新设置 `tx_ready`。



图示：上排把外部位流提交为一个软件可读字节，下排把一个软件提交的字节展开成位流。接收槽和发送槽是两个独立所有权边界，不能用一根“串口缓冲”线概括。

软件模型。 地址互连为控制器分配三个 32 位寄存器：

地址与名称	访问方式	软件可见语义
0x40000000 STATUS	只读	bit0 为 RX_READY, bit1 为 RX_OVERRUN, bit2 为 TX_READY
0x40000004 RXDATA	只读	返回接收字节, 并消费当前接收槽
0x40000008 TXDATA	只写	在 TX_READY=1 时提交一个待发送字节

固件读取字节时必须先检查状态：

```
receive_byte:
    status = read32(0x40000000)
    if status has RX_OVERRUN: reject current transfer
    if status lacks RX_READY: repeat
    return read8(0x40000004)
```

抽象。上层软件得到一个按到达次序读取、按提交次序发送的字节通道。它不保证任务边界、长度、校验或重传；这些规则要由固件和外部发送端建立。它也不保证处理器在做其他计算时仍能及时取走字节，单槽接收的速率上限由轮询速度决定。

为什么不能只约定“发完为止”。串行字节本身没有结束标记。若发送端静默，固件无法区分“任务已经发完”和“下一个字节还没到”。因此，传输必须先提供长度。长度又可能因误码变成巨大数值，固件还需要固定标识和校验，才能避免把随机字节当作可执行映像。

本章使用 32 字节的教学头部：

偏移	字段	为什么必须存在
+0x00	magic	区分任务传输和随机输入，本章固定为 MTK1
+0x04	header_bytes	使接收者知道载荷从哪里开始
+0x08	image_bytes	决定接收终点并用于范围检查
+0x0c	load_address	指定载荷写入 RAM 的起始位置
+0x10	entry_offset	在载荷内部指出第一条任务指令
+0x14	zero_bytes	指定载荷后需清零的可写区域
+0x18	stack_bytes	说明任务要求的最小栈空间
+0x1c	checksum	检测头部之外的载荷是否损坏

这只是固件与发送端的装入协议，不是文件系统中的文件格式。它没有文件名、目录、权限、时间戳或持久对象身份。给它增加这些名称并不会让机器自动获得文件系统。

装入不是复制一遍就结束

固件收到头部后，不能立即按其中地址写 RAM。一个恶意或损坏的 `image_bytes` 可能在加法中溢出，使表面上的终点变成较小地址；载荷可能覆盖固件工作区或任务栈；入口偏移也可能指到载荷之外。所有会影响写入范围和最终 PC 的字段都必须先验证。

先确定本次内存所有权。本章采用以下 RAM 布局：

物理范围	启动期间的所有者	用途
0x00100000--0x0017ffff	待装入任务	映像、零填充区和任务数据

0x00180000--0x001effff	待装入任务	向下增长的任务栈候选区
0x001f0000--0x001fffff	固件	接收状态、校验状态和固件栈

“所有者”此时只是固件遵守的软件约定，并非硬件权限。固件在移交前不会主动把任务载荷写进自己的工作区；任务开始后却仍可用普通存储指令覆盖那里。正是这种缺乏强制边界的事实，决定了本章还不能称为有了操作系统保护。

范围检查必须考虑整数溢出。 设

$$L = \text{load_address}, \quad N = \text{image_bytes}, \quad Z = \text{zero_bytes}.$$

若直接计算 $L + N + Z$ ，32 位加法可能回绕。稳妥检查先确认每个长度不超过允许区间，再用减法形式判断：

$$N \leq U - L, \quad Z \leq U - (L + N),$$

其中 U 是任务载入区的开区间上界 0x00180000。在使用 $U - L$ 前，还要确认 $L < U$ 且 L 不低于 0x00100000。

入口地址

$$E = L + \text{entry_offset}$$

必须落在已接收的映像字节中，并满足 4 字节指令对齐。栈大小必须能在候选栈区内安排，且栈区不能与载荷及零填充区重叠。

```
validate_header(h):
    require h.magic == MTK1
    require h.header_bytes == 32
    require TASK_LOAD_LOW <= h.load_address < TASK_LOAD_HIGH
    require h.image_bytes <= TASK_LOAD_HIGH - h.load_address
    image_end = h.load_address + h.image_bytes
    require h.zero_bytes <= TASK_LOAD_HIGH - image_end
    require h.entry_offset < h.image_bytes
    require (h.load_address + h.entry_offset) is 4-byte aligned
    require MIN_STACK <= h.stack_bytes <= STACK_REGION_SIZE
```

验证顺序是协议的一部分。只有前一项证明相应运算安全，后一项才可以使用计算结果。把所有条件压成一个长布尔表达式，会掩盖哪一步可能先发生溢出。

逐字节接收时谁拥有哪些状态。 头部通过后，固件建立一个装入记录：

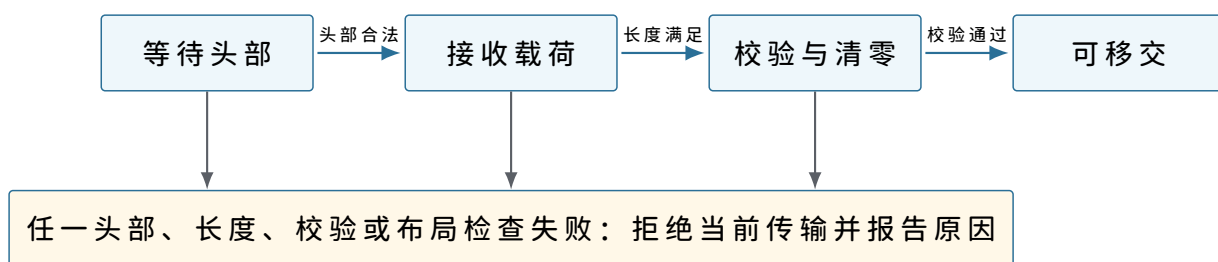
```
load_record:
    destination_start
    destination_next
    payload_remaining
    checksum_so_far
    expected_checksum
    entry_address
    zero_start
    zero_bytes
    stack_low
    stack_top
    state = RECEIVING
```

`destination_next` 指出下一个字节写到哪里；`payload_remaining` 防止接收循环依赖外部静默；`checksum_so_far` 只覆盖已经接受的载荷。每消费一个 `RXDATA` 字节，固件按固定顺序更新这些字段：

1. 从控制器消费一个字节；
2. 把字节写到 `destination_next`；
3. 将该字节并入校验状态；
4. 递增 `destination_next`；
5. 递减 `payload_remaining`。

若在第 2 步之后复位，RAM 中可能留下部分载荷，但复位会重新从启动存储执行，装入记录也会重建。固件不能仅凭 RAM 中“看起来像代码”的字节跳转，必须重新完成协议验证。

接收完成、验证完成与可执行不是一个时刻。最后一个字节写入 RAM 时，只能说明传输长度已经满足。固件还要比较最终校验值、清零 `zero_bytes` 指定的区域、建立输入数组和初始栈。为避免把半成品当作任务，装入记录按以下状态推进：



图示：“载荷到齐”只关闭接收阶段；只有校验、零填充和布局建立全部成功，装入记录才进入“可移交”。任何失败都不能改变 PC 到任务入口。

本章的提交点不是写入最后一个字节，而是稍后把处理器 PC 改为已经验证的入口地址。在此之前，即使 RAM 中已有完整任务，处理器仍执行固件。

用具体数字走完一次装入。求和任务头部给出：

```

magic          = MTK1
header_bytes   = 32
image_bytes    = 0x00000300
load_address   = 0x00100000
entry_offset   = 0x00000000
zero_bytes     = 0x00000100
stack_bytes    = 0x00004000
checksum       = expected value
  
```

载荷占用 `0x00100000--0x001002ff`，零填充区占用 `0x00100300--0x001003ff`，都低于任务载入区上界。任务栈使用 `0x001ec000--0x001effff`，与映像不重叠。入口地址就是 `0x00100000`。

输入数组是任务数据的一部分，位于 `0x00102000`，不在上述 `0x300` 字节载荷范围内会产生矛盾。为使布局一致，本次实际映像应覆盖到输入区之后，因此把 `image_bytes` 修正为 `0x00002100`。这项复算刻意展示了字段之间必须相互核对：若只逐项抄表，装入器会接受一个宣称包含输入数据、实际长度却到不了输入地址的映像。

修正后的范围为：

范围	装入后的内容	来源
<code>0x00100000--0x0010001f</code>	求和核心指令	载荷
<code>0x00100020--0x00101fff</code>	其他任务代码与常量	载荷

0x00102000--0x0010200f	四个输入整数	载荷
0x00102020--0x00102023	结果槽，初值为零	载荷
0x00102100--0x001021ff	未初始化区	固件清零
0x001ec000--0x001effff	任务栈	固件保留并建立初始内容

外部发送端不应发送含糊的“差不多大小”。装入协议中的长度必须覆盖实际传输的每个字节，固件也只能在校验过的范围内解释入口和数据。

从固件现场建立任务现场

任务字节可执行后，处理器仍保存固件的 PC、SP 和寄存器。直接把 PC 改为任务入口会让任务继承任意寄存器值，并继续使用固件栈。任务第一次访问参数或执行 RET 时就会得到错误结果。固件必须先构造一份明确的初始现场。

初始寄存器约定。本次启动约定如下：

状态	移交值	原因
r0	0x00102000	输入数组首地址
r1	4	数组元素个数
r2	0x00102020	结果槽地址
r3--r7	0	消除对固件残留值的依赖
SP	0x001efffc	指向预置的固件返回地址
PC	0x00100000	最后一步才写入的任务入口

SP 不是直接设为 0x001f0000。固件先在 0x001efffc--0x001effff 写入 32 位返回地址 0x00000300，再让 SP 指向这个字。任务最终执行 RET 时，处理器读取 MEM32[SP] 作为新 PC，并把 SP 增加 4，于是控制流回到固件返回入口。

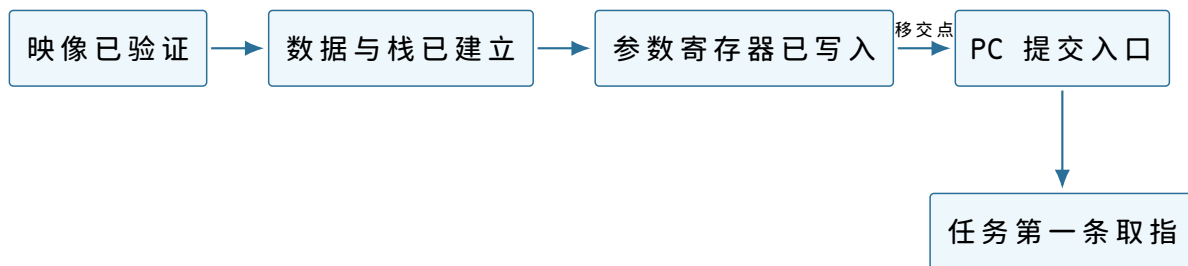


图示：向下增长表示压入新内容时 SP 减小。栈区上界本身是开区间；预置返回地址占用上界下方最后 4 字节。

为什么入口地址必须最后提交。建立初始现场的安全顺序是：

1. 确认装入记录处于“可移交”；
2. 停止接收本次映像，避免后续字节继续改写任务区；
3. 写入输入数据、结果初值和零填充区；
4. 建立任务栈并写入固件返回地址；
5. 设置通用寄存器和 SP；
6. 最后把 PC 改为任务入口。

前五步仍在固件指令流中执行。第六步一旦提交，下一次取指来自任务区域，固件就不能继续执行“还有一步初始化”。因此 PC 的改变是控制权移交的提交边界。



图示：“字节已经在 RAM 中”和“任务已经运行”之间隔着初始现场建立与 PC 提交。只有最右侧事件改变取指来源。

移交前后的处理器快照。移交前，固件可能有如下状态：

```

PC = 0x000002d8      SP = 0x001ffec0
r0 = load_record_ptr r1 = checksum_so_far
r2 = 0x40000000      r3 = temporary value
  
```

移交完成后的第一条任务指令开始前，状态必须变为：

```

PC = 0x00100000      SP = 0x001efffc
r0 = 0x00102000      r1 = 4
r2 = 0x00102020      r3 = 0
r4 = 0                r5 = 0
r6 = 0                r7 = 0
  
```

这两个快照说明“切换”不能只描述为跳转。至少有 PC、SP、参数寄存器和清理后的临时寄存器发生变化。当前固件不需要保存自己的全部现场，因为任务返回后固件进入一个固定返回入口，而不是回到 `0x000002d8` 继续原函数。下一章需要暂停并继续不同任务时，保存现场的要求会更严格。

把装入器写成可审计的状态机

“固件接收并装入任务”仍然包含多个可能暂停或失败的阶段。若所有局部变量只散落在寄存器中，错误入口无法判断已经接收多少字节，也无法给出稳定诊断。固件因此在自己的 RAM 工作区保存一份装入记录。它不是操作系统任务对象，只代表当前这一笔传输。

字段、写入者和有效期。

字段	谁在何时写入	后续读取目的
state	阶段转换代码	限制当前允许的操作
destination_next	每接受一个字节后递增	决定下一笔 RAM 写地址
payload_remaining	头部建立，逐字节递减	判断载荷何时到齐
checksum_so_far	每个已接受字节更新	与头部期望值比较
entry_address	头部验证后计算一次	移交时写入 PC
stack_low/top	布局验证后计算一次	清零栈并设置 SP
error_code	首个失败检测点	报告哪条契约被破坏

装入记录从收到头部开始存在，任务入口提交前仍由固件独占。任务运行后，记录不再描述一个“正在接收”的事务；固定返回入口可以把它改为 `FINISHED`，或直接重置为 `WAITING_HEADER`。若任务永不返回，记录也不会自行变化。

允许的状态转换。

当前状态	输入事件	必须完成的动作	后继状态
------	------	---------	------

WAITING	32 字节头到齐	解码并验证所有范围	RECEIVING 或 REJECTED
RECEIVING	一个载荷字节到达	写 RAM、更新校验和与计数	保持或 VERIFYING
RECEIVING	控制器溢出	记录传输错误	REJECTED
VERIFYING	校验匹配	清零、建栈、建立参数	READY
VERIFYING	校验不匹配	记录内容错误	REJECTED
READY	固件决定运行	设置现场并提交 PC	RUNNING
RUNNING	固定返回入口获得控制	读取并发送结果	FINISHED

状态表排除了若干非法动作：**WAITING** 状态不能消费载荷；**REJECTED** 状态不能跳任务入口；**RUNNING** 状态下固件也没有机会继续修改记录。把这些限制写入转换函数，比在各处检查零散布尔量更容易审计。

头部在串行线上也是字节。以修正后的映像为例，头部前 28 个字节可表示为：

offset	bytes	decoded value
00	4d 54 4b 31	magic "MTK1"
04	20 00 00 00	header_bytes = 0x20
08	00 21 00 00	image_bytes = 0x2100
0c	00 00 10 00	load_address = 0x00100000
10	00 00 00 00	entry_offset = 0
14	00 01 00 00	zero_bytes = 0x100
18	00 40 00 00	stack_bytes = 0x4000
1c		checksum

`load_address` 的四个字节为 `00 00 10 00`。按低有效字节在前组合后才是 `0x00100000`。若固件按相反字节序解码，会得到 `0x00001000`，落入未映射区域。传输协议必须规定字节序，不能依赖发送端和处理器“碰巧相同”。

校验和覆盖什么，不能证明什么。本章采用一个教学化的 32 位逐字节滚动校验：

```
checksum = 0
for each payload byte b:
    checksum = rotate_left(checksum, 5)
    checksum = checksum XOR b
```

发送端对全部载荷计算结果并放入头部，固件按实际收到的次序重新计算。它能够高概率检测常见误码、丢字节和次序变化，但不是密码学签名。知道算法的人可以构造另一个产生相同校验值的载荷；校验通过也不能证明代码可信、没有越界指令或实现了正确算法。

这里有三种不同判断：

1. 格式有效：头部字段可安全解释，范围没有越界；
2. 传输一致：载荷长度和校验符合发送端给出的值；
3. 程序可信：代码来源和行为满足安全策略。

本章只完成前两项。当前机器没有签名验证、权限或沙箱，不能从“校验通过”推出“任务可以任意访问机器而没有风险”。

用失败用例检验头部验证顺序。给装入器一组具体头部，比只读正常伪代码更容易发现边界错误。

长度恰好到达上界。若 `load_address=0x0017ff00`、`image_bytes=0x100`、`zero_bytes=0`，映像终点恰好为开区间上界 `0x00180000`，可以接受。最后一个有效字节地址为 `0x0017ffff`。

若 `image_bytes=0x101`，最后一个字节会落在为栈候选区保留的 `0x00180000`，必须拒绝。用开区间终点计算可以避免“最后一个地址加一”再次溢出。

32 位回绕伪装成小终点。设攻击头部为：

```
load_address = 0xffffffff
image_bytes = 0x00000040
```

32 位直接相加得到 `0x00000030`。若装入器只检查“终点小于 RAM 上界”，会错误接受。正确顺序先检查起点是否位于任务区；起点检查已经失败，根本不执行可能回绕的终点判断。

入口位于零填充区。若 `entry_offset=image_bytes+4`，入口落在固件清零的区域。虽然地址属于任务 RAM，那里并不是发送端提供且校验过的指令，所以必须拒绝。入口条件是

$$0 \leq \text{entry_offset} < \text{image_bytes},$$

而不是小于 `image_bytes + zero_bytes`。

入口未对齐。`entry_offset=2` 使 PC 成为 `0x00100002`。该地址位于载荷内部，却不满足教学处理器的 4 字节取指对齐，必须在移交前拒绝。若等到处理器取指才发现，错误仍可检测，但诊断会从“非法映像入口”退化成“处理器对齐故障”，丢失了协议层原因。

栈与映像不重叠仍可能过小。只检查栈区不重叠还不够。本章至少要预置 4 字节返回地址，因此 `stack_bytes<4` 必须拒绝。实际约定采用更大的 `MIN_STACK`，为任务的函数调用保留空间。最小值是启动协议的一部分，不由 RAM 自动推断。

复算点。对 `load_address=0x00170000`、`image_bytes=0x10000`，映像终点为 `0x00180000`，仍合法；若再要求 `zero_bytes=1`，则第一字节零区会进入栈候选范围，整个头部必须拒绝。

串行速度如何限制轮询装入器。采用常见的 8N1 串行帧时，一个数据字节在线上占 1 个起始位、8 个数据位和 1 个停止位，合计 10 位。若传输速率为 115200 位每秒，一个字节约每

$$\frac{10}{115200} \text{ s} \approx 86.8 \text{ } \mu\text{s}$$

到达一次。

假设处理器频率为 10MHz，两个字节到达之间约有 868 个处理器周期。固件读取状态、消费字节、写 RAM 和更新校验必须在这段预算内完成，或让发送端等待。若最坏路径需要 950 周期，平均速度看似接近，单槽控制器仍会周期性溢出。

平均成本不足以证明不会溢出。轮询循环平时可能只需 200 周期，但遇到错误报告、长 RAM 等待或其他固件工作时延迟会变大。不会溢出的条件约束的是两次消费之间的最长间隔，而不是平均每字节成本：

$$T_{\text{consume,max}} < T_{\text{arrival}}.$$

如果无法满足，可以选择让发送端依据硬件流量控制暂停、增加接收 FIFO，或以后引入中断和 DMA。每种方案都会新增状态和失败模式；当前样机选择最简单的发送端应答协议。

逐块应答建立背压。外部端每发送 64 字节后等待固件返回一个确认字节。固件只有在这 64 字节全部写入 RAM 且未发生溢出时才确认。若返回拒绝码，外部端从整

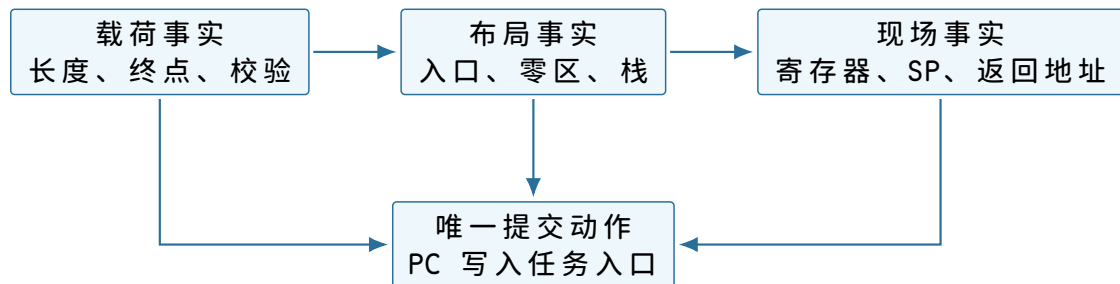
个映像开始重传。本章不实现分块恢复，因为那需要块编号、重复检测和部分校验等新状态。

这个协议把发送速率约束反馈给发送端，称为背压。背压不提高控制器内部容量，却阻止外部端无限快地提交数据。即便没有操作系统，异步生产者与有限缓冲之间也已经存在同样的资源问题。

一次成功移交的详细不变量。在提交任务 PC 之前，固件逐项确认：

1. 装入记录状态为 `READY`；
2. `destination_next=load_address+image_bytes`；
3. `payload_remaining=0`；
4. 实际校验值等于头部校验值；
5. 入口位于已校验映像且满足对齐；
6. 零填充区已经写零；
7. 输入数组和结果槽位于任务数据范围；
8. 栈范围与映像、固件工作区均不重叠；
9. `MEM32[0x001efffc]=0x00000300`；
10. 通用寄存器和 SP 已按启动契约设置。

这些条件称为移交不变量：只要固件准备改变 PC，它们就必须同时成立。若检查分散在不同函数中，最终移交函数仍应以断言或显式返回值确认它们，而不能依赖“前面大概检查过”。



图示：三组事实都成立，才允许执行控制权提交。PC 写入不是又一次“检查”，而是使任务开始取指的不可混淆边界。

用装入记录快照检查一次正常传输。下面选择四个时刻观察装入记录。设载荷长度为 `0x2100`，起点为 `0x00100000`。

观察时刻	下一写入地址	剩余字节	已经成立的事实
头部通过后	<code>0x00100000</code>	<code>0x2100</code>	范围与入口合法，尚无载荷字节
第 1 字节后	<code>0x00100001</code>	<code>0x20ff</code>	起始字节已写入并纳入校验
第 64 字节后	<code>0x00100040</code>	<code>0x20c0</code>	第一块可向发送端确认
最后字节后	<code>0x00102100</code>	<code>0</code>	长度满足，等待最终校验

两个数值始终满足不变量：

$$\text{destination_next} - \text{load_address} = \text{image_bytes} - \text{payload_remaining}.$$

左边是已经写入的地址跨度，右边是已经消费的字节数。若更新时漏掉递增或递减，其中一边会立刻失配。固件可在调试版本中每轮检查该式。

第 37 个字节损坏时哪些字节可见。假设传输没有丢失长度，但第 37 个字节从 `0x11` 变成 `0x10`。装入器会继续收满 `0x2100` 字节，最终校验不匹配，状态转为 `REJECTED`。此时 RAM 中确实有一份长度完整但内容错误的映像。

固件不必在发现校验失败时逐字节回滚，因为任务 PC 尚未提交。它必须保证两点：不把记录转为 `READY`；下一次装入重新覆盖可能使用的范围。若机器要在多个已装入映像之间保留旧版本，直接覆盖就不再足够，需要独立暂存区或双缓冲；本章只有一项任务，因此不引入该结构。

第 37 个字节之后串行溢出。若控制器置 `RX_OVERRUN`，固件知道至少一个字节丢失，却不知道具体有几个。此时 `destination_next` 只表示已经消费的字节数，不能对应发送端位置。固件立即进入 `REJECTED`，丢弃直到下一次明确的传输同步标记，再请求整包重发。

仅把 `payload_remaining` 继续减到零会造成错误对齐：后续字节向前填补缺口，头部长度的仍可能满足，但内容位置全部错位。校验大概率会失败，却要等整包结束才能发现。尽早利用硬件溢出状态可以缩短失败路径。

零填充区为什么不随载荷发送。任务常需要一片初始值全为零的可写区域，例如计数器数组或暂存缓冲。如果发送端把几千个零也逐字节发送，既增加传输时间，又没有增加信息。头部用 `zero_bytes` 表达“映像末尾之后这段范围必须被固件写零”。

这一做法建立了两类来源不同的任务字节：

区域	初值来源	校验与建立规则
映像区	外部载荷	每个字节参与传输校验
零填充区	固件写零	范围由已验证头部决定，不占载荷
栈区	固件初始化	预置返回地址，其余按启动策略清零

零填充必须发生在校验通过之后或之前均可，但在状态进入 `READY` 前必须完成。若清零中途发生复位，启动流程重新开始，不能沿用旧的 `READY` 标记。

为什么不能把整个 RAM 都清零。全清零看似简单，却会覆盖固件工作区，包括当前装入记录和固件栈。即使先避开工作区，清零整片 RAM 也会让启动时间随容量增长。按头部和固定布局只初始化任务真正依赖的区域，使写入范围与所有权一致。

任务栈如何支持一次真实函数调用。求和核心本身是叶函数，但正式任务通常会调用子程序。设任务在写结果前调用位于 `0x00100100` 的格式化例程，调用指令地址为 `0x00100018`，顺序后继为 `0x0010001c`。调用前：

```
PC = 0x00100018
SP = 0x001efffc
MEM32[0x001efffc] = 0x00000300 // outer return
```

`CALL` 先把 SP 降到 `0x001efff8`，写入内部返回地址 `0x0010001c`，再进入子程序：

```
PC = 0x00100100
SP = 0x001efff8
MEM32[0x001efff8] = 0x0010001c // return to task
MEM32[0x001efffc] = 0x00000300 // return to firmware
```

子程序的 `RET` 先消费 `0x0010001c`，恢复到任务内部，SP 回到 `0x001efffc`。任务最外层 `RET` 再消费 `0x00000300`，返回固件。两个返回地址按后进先出次序工作。

0x001f0000: 栈区上界	
0x001efffc: 返回固件 0x00000300	
0x001efff8: 返回任务 0x0010001c	← 子程序中的 SP
更深调用与局部保存继续向低地址扩展	

图示：返回地址的顺序来自 SP 的移动和 RAM 中的布局，不来自处理器对“函数层次”的理解。破坏任一栈字都会改变后续控制流。

谁负责保证栈不越界。当前处理器只检查地址是否位于整个 RAM，不知道任务栈下界 0x001ec000。若调用过深，SP 可以越过下界并覆盖其他任务数据，只要地址仍在 RAM，硬件就会接受。固件给出 16 KiB 空间并不能强制任务停在边界内。

可以让编写任务的人证明最大栈深，或在每次函数入口加入软件检查；两者仍依赖任务合作。真正的隔离需要地址访问权限，后续才会从任务相互覆盖的失败中推导。

移交失败时由谁收束状态。在 PC 提交之前，固件是唯一执行者，也是装入记录、任务目标区和串行协议的所有者。任何失败都由固件把记录转为 REJECTED，发送错误码，并回到等待新头部状态。

PC 提交之后，固件不再执行。任务拥有当前寄存器与任务栈，直到 RET。此后出现的无限循环、任意 RAM 写入和栈破坏，不能再由“装入器检查一次”解决。阶段不同，能采取行动的主体也不同。

阶段	当前执行者	可修改的关键状态	失败收束
接收头部	固件	装入记录、串行槽	拒绝传输
接收载荷	固件	任务 RAM、校验状态	拒绝并重传
建立现场	固件	任务栈、参数寄存器	不提交 PC
任务运行	任务	寄存器及全部可写物理 RAM	合作返回或整机错误入口
固定返回	固件	固件栈、发送状态	报告结果并等待

这张所有权表给出了本章最重要的时间边界：同一片 RAM 在不同阶段由不同代码解释和修改，但硬件尚未实施所有权。当前所有权是分析软件责任的工具，不是安全保证。

把固件主循环连成完整控制程序

前面已经逐个定义装入记录、串行寄存器和任务现场。现在可以给出不隐藏阶段的固件主循环。伪代码使用函数名表达已经解释过的局部动作，不涉及构建工具或源文件位置。

```

reset_entry:
    initialize firmware SP
    clear serial protocol state
    reset load_record to WAITING

wait_for_transfer:
    header = receive_exactly(32)
    if serial overrun: report SERIAL_OVERRUN; goto wait_for_transfer
    if not validate_header(header):
        report HEADER_INVALID
        goto wait_for_transfer

    initialize load_record from header

```

```

while payload_remaining != 0:
    b = receive_one_byte()
    if serial overrun:
        reject SERIAL_OVERRUN
        goto discard_and_resync
    store b at destination_next
    update checksum and counters
    acknowledge each completed 64-byte block

load_record.state = VERIFYING
if checksum_so_far != expected_checksum:
    report CHECKSUM_MISMATCH
    goto wait_for_transfer

zero requested region
build task stack and arguments
assert handoff invariants
load_record.state = READY
handoff_to_task()          // PC commit occurs here

```

`handoff_to_task` 正常情况下不返回到下一行。它写入任务寄存器和 SP，最后通过受控跳转提交 PC。任务执行最外层 `RET` 时进入另一个固定入口 `firmware_return`，而不是回到调用点。

为什么返回入口不能沿用装入函数的普通栈帧。装入函数的局部变量位于固件栈。移交任务时，SP 已切到任务栈；任务也可以改写所有通用寄存器。若返回入口试图像普通函数那样从旧 SP 取局部变量，它会读错位置。固定入口必须先把 SP 设为固件栈顶，再通过全局固定地址找到装入记录。

```

firmware_return:
    SP = 0x00200000
    load_record.state = FINISHED
    result = read32(0x00102020)
    send_result_frame(result)
    reset load_record to WAITING
    goto wait_for_transfer

```

这里 `0x00200000` 是 RAM 上界，固件实际第一项栈内容写在其下方。固件工作区位于 `0x001f0000--0x001ffffff`，任务栈止于 `0x001effff`，两者按软件布局分离。

结果帧为什么还要带状态。只发送四个结果字节，外部端无法区分正常结果、错误码和上一次传输残留。固件发送一个短帧：

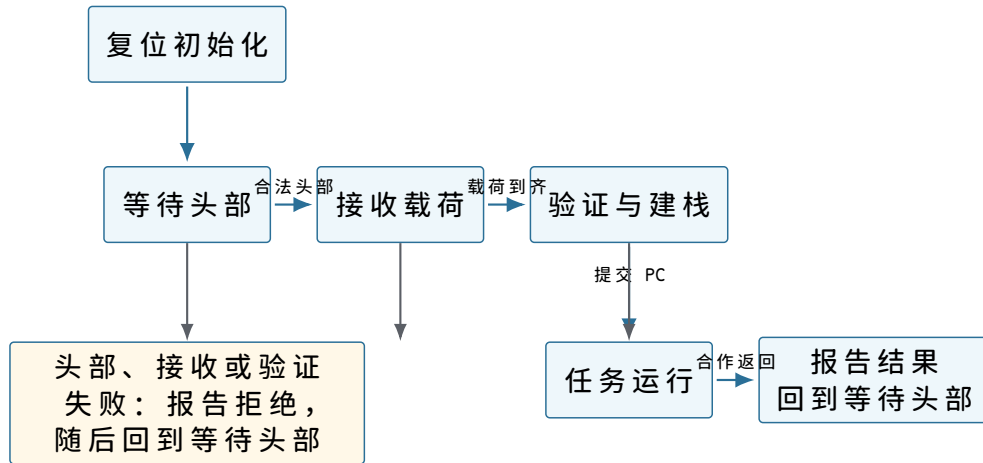
```

result_frame:
    magic      = "RES1"
    status     = OK or an error code
    value      = valid only when status == OK
    checksum   = checksum of preceding fields

```

本次成功帧中的 `value` 为 32 位整数 21，在线上按 `15 00 00 00` 发送。外部端只有在完整接收并验证结果帧后，才把任务标记为成功。固件写 `TXDATA` 只是提交一个待发送字节，不等于外部端已经收到整帧。

从复位到等待下一任务的状态图。



图示：上排是一次成功传输从左到右的状态变化，下排统一表示任务运行以前的拒绝路径。进入任务后，唯一正常出边仍是任务主动返回；当前机器没有从运行状态强制回到固件的事件。

状态图中的终止条件。等待头部没有时间上限；机器可以长期停在那里。接收载荷若外部端中途停止，当前协议也会无限等待，因为机器尚无可编程计时器。若希望超时拒绝，必须增加能产生时间事件的硬件和相应软件状态。本章不借用尚未加入的能力。

任务运行同样可能没有终点。状态图不是承诺所有路径最终回到等待，而是准确标出哪些转换需要外部合作。正式模型必须允许停留在 **RECEIVING** 或任务运行状态。

一次完整成功运行中的关键数值。

时刻	关键状态	可据此推出的事实
复位释放	PC=0x00000000	下一笔取指来自启动存储
头部通过	state=RECEIVING	地址与长度安全，内容尚未验证
载荷到齐	remaining=0	已写 0x2100 字节
校验通过	state=VERIFYING	传输内容与发送端一致
现场完成	SP=0x001efffc	栈顶含固件返回地址
移交提交	PC=0x00100000	任务开始取指
第四轮后	r4=21, r1=0	循环将退出
结果写回	MEM32[0x00102020]=21	处理器可读取结果
任务返回	PC=0x00000300	固件重新获得控制
结果帧到达	外部校验通过	本次任务对外完成

这十个时刻覆盖了硬件起点、协议接受、内容验证、执行移交、计算效果、控制返回和外部可见。任意相邻两行之间都存在不能省略的动作，因此不能把它们压缩成“开机后运行程序并输出结果”。

本章方案的成本。最小方案追求边界清楚，不追求吞吐最大。处理器轮询串行接口时不能做其他工作；每个任务都要完整传输；任务运行时固件完全停止；结果只存在 RAM；错误通常通过复位收束。这些成本不是待补的一串现代名词，而是当前路径中可以指认的位置。

当前成本	在路径中的原因	只有何时才值得改进
装入时处理器忙等	单槽串行接口只提供轮询状态	输入速度或并行工作成为问题

启动需完整传输	RAM 断电丢失且任务不在 ROM	需要持久保存多个任务
不能同时运行别的任务	只有一份当前处理器现场	第二任务需要取得处理器
无限循环无法收束	没有异步时间事件	必须限制任务运行区间
任意 RAM 写可破坏固件	物理地址没有权限	不可信任任务需要隔离

下一章首先处理第三行：在不要求任务重新从入口开始的条件下，让第二项任务获得处理器。其余问题继续保留，避免一次引入尚未由失败要求的结构。

沿着处理器状态走完求和任务

任务已经获得处理器。现在用具体地址和数值逐条检查执行结果。任务开始时输入内存为：

```
MEM32[0x00102000] = 7
MEM32[0x00102004] = -2
MEM32[0x00102008] = 11
MEM32[0x0010200c] = 5
MEM32[0x00102020] = 0
```

初始化指令。第一笔任务取指从 `0x00100000` 到达 RAM。互连不关心这是任务代码，只因地址匹配而选择 RAM。返回字节译码为 `MOVI r4,0`。指令提交后：

```
PC: 0x00100000 -> 0x00100004
r4: 0x00000000 -> 0x00000000
other architectural state: unchanged
```

虽然 `r4` 的数值碰巧没有变化，写入动作仍由指令语义决定。若固件没有按约定清零 `r4`，这条指令也会建立相同起点。

第一轮：从字节到部分和。在 `0x00100004`，`LOAD r5,[r0+0]` 读取 `0x00102000--03`。事务成功后 `r5=7`，PC 进入 `0x00100008`。`ADD r4,r5` 读取旧值 0 和 7，产生候选结果 7，提交后 `r4=7`。

接下来的两条 `ADDI` 分别改变指针和计数：

```
r0: 0x00102000 -> 0x00102004
r1: 4 -> 3
```

`BNEZ r1,0x00100004` 读取 `r1=3`，条件成立，因此下一 PC 不是顺序地址 `0x00100018`，而是循环入口 `0x00100004`。

四轮状态表。每轮在条件分支提交后，关键状态如下：

轮次	本轮读地址	读出值	部分和 r4	分支后的 r1
1	<code>0x00102000</code>	7	7	3
2	<code>0x00102004</code>	-2	5	2
3	<code>0x00102008</code>	11	16	1
4	<code>0x0010200c</code>	5	21	0

第四轮后，`BNEZ` 不采用分支目标，PC 顺序进入 `0x00100018`。此时 `r0=0x00102010`，正好指向数组末尾之后的位置。这个地址没有被解引用；它只表示所有元素已经消费。若循环多执行一轮，处理器会读取数组外字节，而当前机器没有边界检查替任务阻止这一错误。

结果写入的可见边界。STORE r4,[r2+0] 形成地址 0x00102020，写数据为 0x00000015，宽度为 4。RAM 在事务完成时把四个字节更新为 15 00 00 00。在写响应返回前，任务不能把该存储指令视为完成。

写入完成表示后续处理器读取能够取得新值。它不表示结果跨断电保存，因为目标是 RAM；也不表示外部发送端已经看到结果，因为串行发送尚未发生。可见、传达和持久是三个不同边界。



图示：前三个方框是不同层次的事实。最右侧使用虚线，因为 RAM 和串行发送都不能提供断电持久性。

任务怎样返回固件。执行到 0x0010001c 时，任务发出 RET。处理器必须先从 SP=0x001efffc 读取 4 字节返回地址。RAM 返回 0x00000300 后，处理器把 SP 增加到 0x001f0000，并把 PC 改为 0x00000300。这两个状态更新共同完成返回。

返回前后快照。

状态	RET 开始前	RET 完成后
PC	0x0010001c	0x00000300
SP	0x001efffc	0x001f0000
MEM32[0x001efffc]	0x00000300	内容仍在，但已不属于有效栈
MEM32[0x00102020]	21	21

SP 上升不会自动擦除旧返回地址。栈的有效范围由 SP 与约定共同决定，旧字节可以继续留在 RAM 中。后续压栈可能覆盖它。

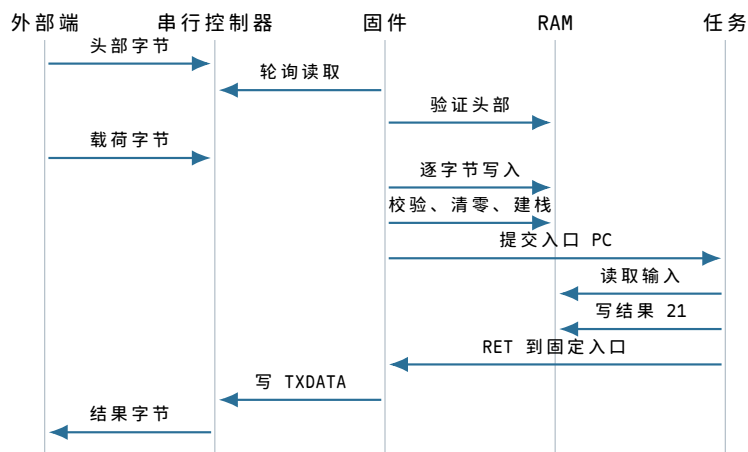
固件返回入口能依赖什么。固件不能假定任务保留了自己的临时寄存器，因为本章没有要求任务这样做。固定返回入口只依赖两项事实：PC 已指向 0x00000300，结果位于约定的 RAM 地址。它重新建立自己的 SP，读取结果，等待串行发送端可用，然后按四个字节发送结果或发送文本表示。

```

firmware_return:
    SP = FIRMWARE_STACK_TOP
    result = MEM32[0x00102020]
    for each byte in encode(result):
        repeat until STATUS has TX_READY
            write8(TXDATA, byte)
    enter firmware_wait_loop
  
```

固件必须先重建 SP，再进行函数调用或压栈。任务返回时的 SP 已在任务栈上界，不能当作固件栈继续使用。两片栈位于同一 RAM，却属于不同执行阶段的软件约定。

正常路径的端到端时间线。前面分别建立了器件和局部事务，现在可以把它们综合成一次完整运行。下图只使用已经解释过的对象：



图示：时间从上向下。载荷进入 RAM、任务获得 PC、结果进入 RAM、结果到达外部端分别是四个不同事件；前一事件不会自动包含后一事件。

把每个边界写成一句可检验的事实。

1. 头部接受：字段合法且后续计算不会越过任务区域；
2. 载荷接收完成：规定数量的字节已经写入 RAM；
3. 映像验证完成：校验通过、零区和栈均已建立；
4. 控制权移交：PC 已提交任务入口，下一次取指来自任务；
5. 计算完成：结果存储事务已得到 OK；
6. 任务返回：PC 和 SP 已按 RET 规则更新；
7. 外部可见：串行发送完成，外部端实际收到结果字节。

如果只说“任务运行成功”，就无法定位失败究竟发生在传输、装入、移交、计算、返回还是报告阶段。正式系统中的请求、完成和持久化边界更复杂，但分析方法从这台小机器开始已经相同：先命名对象状态，再指出哪个事件改变了它。

错误路径揭示了当前机器缺少什么。正常示例证明最小方案可行，错误路径则说明它的边界。下面每种错误都对应一个具体检测者，不能用“系统会处理”概括。

头部损坏：固件可以在移交前拒绝。若 magic 错误、长度越界或入口未对齐，固件仍掌握 PC，因此可以停止接收、通过串行发送错误码并回到等待头部状态。此时 RAM 中即使残留部分字节，也没有被执行。

错误处理的关键不是把 RAM 恢复成全零，而是不提交任务入口。下一次合法装入会覆盖规定范围；若固件需要避免旧数据泄漏，可以在重新使用前清零，但这属于额外策略。

串行溢出：控制器只能报告已经丢失。若发送端速度超过固件轮询速度，新字节会覆盖单槽接收寄存器，控制器随即置溢出位 `RX_OVERRUN`。固件无法由该位推断丢失字节的内容和位置，只能拒绝整次传输并要求外部端重发。校验和可以检测内容不一致，却不能凭空恢复缺失字节。

增加接收 FIFO、流量控制或 DMA 都能改变吞吐边界，但当前问题只需要一个最小可靠方案，所以采用“检测溢出并重传”的规则。后续若设备速率成为主要矛盾，再引入更复杂硬件。

未映射地址：互连报告错误，任务没有恢复机制。若任务错误地读取 `0x20000000`，互连返回 `UNMAPPED`。教学处理器把失败地址、访问方向和当时 PC 写入固定固件错误槽，然后跳到固件错误入口。这使机器至少不会把任意电平当作成功数据。

本章尚未为不同任务建立独立错误现场，也没有把错误转换成可返回给应用的信号。故障后的最小处理是报告并复位。错误可检测不等于错误可恢复。

任务写坏固件工作区：软件约定无法强制隔离。任务执行：

```
STORE r4, [0x001f0000]
```

时，地址合法地落在 RAM 范围。互连和 RAM 都会完成写入，因为它们不知道该范围按约定属于固件。任务随后返回时，固件的装入记录或栈可能已经损坏。这不是地址互连故障，而是当前机器缺少访问权限边界。

栈指针损坏：返回地址的存在也不足以保证返回。若任务把 SP 改成 0x00102000 后执行 RET，处理器会把第一个输入整数 0x00000007 当作返回 PC。该地址未按指令对齐或落在无效区域，机器随后进入错误入口。固件预置返回地址只能保证合作任务有一条返回路径，不能保证任意代码使用它。

无限循环：没有外部事件就无法取回处理器。任务可能执行：

```
repeat:
    ADDI r4, 1
    BNEZ r4, repeat
```

只要指令和 RAM 访问合法，处理器就会继续执行任务。串行控制器即使收到新命令，当前代码也没有读取它。时钟边沿只能推动指令前进，并不会把 PC 自动改回固件。机器此时既没有可编程时钟事件，也没有异步入口，固件无法重新获得控制。

这一失败是后续演进的关键证据。“任务最终会执行 RET”是合作约定，不是机器保证。若希望运行多个任务或限制单个任务占用处理器的时间，必须先回答怎样保存正在执行的现场，再回答怎样在任务不合作时进入受控代码。

这台机器的软件模型究竟是什么。到本章末，程序员面对的不是一堆孤立器件，而是一组边界清楚的契约。

地址契约。一个 32 位物理地址经过互连选择启动存储、RAM、串行控制器或错误响应。读写宽度和方向是事务的一部分。地址只标识位置，不表达对象身份、权限或所有权。

执行契约。PC 指出下一条指令，寄存器与 RAM 保存继续计算所需的状态。普通指令顺序推进，分支、调用和返回按明确规则改变 PC。只有已经提交的状态才属于下一条指令的输入。

启动契约。复位先进入固件。固件验证任务映像，建立内存布局、参数和栈，最后提交任务入口 PC。任务执行 RET 才会沿预置栈内容回到固定固件入口。

设备契约。串行控制器把异步到达的线信号变成带状态位的字节槽。固件通过轮询消费字节，并负责在溢出或校验失败时拒绝整次传输。控制器不理解任务格式。

为什么这仍然不是操作系统

固件已经做了不少软件工作：它初始化器件、接收字节、检查范围、装入任务、建立初始现场并报告结果。但“存在启动软件”不等于“已经有操作系统”。当前方案只服务于一条预先约定的运行路径，没有建立长期存在的任务身份和资源管理规则。

当前已经具备	依靠什么实现	当前仍未具备
确定的复位入口	复位状态与启动存储	多个任务的身份和生命周期
任务字节装入 RAM	串行协议与固件装入记录	文件、目录和持久命名
一次明确的控制权移交	初始寄存器、栈和 PC 提交	暂停后恢复任意任务现场

合作式返回	预置返回地址与 RET	强制收回处理器
物理地址访问	地址互连	地址隔离和访问权限
错误检测与复位	互连错误和固件入口	面向任务的独立错误恢复

表的左列是本章已经逐步建立并走读过的能力，右列则是具体失败留下的空缺。后续章节不会直接罗列现代操作系统模块，而会从这些空缺中一次选择一个进行演进。

本章复盘：从一个要求到一台可运行机器。最初的要求只有“计算四个整数之和”。为了让这句话在真实机器上成立，我们依次得到以下因果链：

1. 计算必须跨动作保留事实，因此需要寄存器、PC、SP 和可写状态；
2. 第一条指令必须跨断电存在，因此需要启动存储；
3. 运行数据必须可变，因此需要 RAM；
4. 多个器件不能同时响应同一请求，因此需要地址互连和明确事务；
5. 上电状态不可靠，因此需要时钟边界、复位状态和固定入口；
6. 外部任务不能凭空进入 RAM，因此需要串行控制器和传输协议；
7. 任意字节不能直接执行，因此需要范围、校验和入口验证；
8. 任务不能继承固件偶然状态，因此需要初始寄存器、独立栈和移交顺序；
9. 任务完成后必须有控制流去向，因此需要预置返回地址和固定固件入口。

这条链不是整机启动的口号，而是每一步都有对象、字段和状态变化的运行记录。求和任务的最终结果为 21，RAM 中的结果字节为 15 00 00 00，外部端只有在串行发送完成后才能看到它。

为什么四轮结束时一定得到 21。逐条 trace 能证明本次输入的实际运行结果，还可以从循环状态中提炼一个每轮都成立的关系。设原数组为 a_0, a_1, a_2, a_3 ，总元素数为 $n = 4$ ，已经处理的元素数为 k 。每次进入 LOAD 前，程序应满足：

$$\begin{aligned} r0 &= \text{array_base} + 4k, \\ r1 &= n - k, \\ r4 &= \sum_{i=0}^{k-1} a_i. \end{aligned}$$

这个关系称为循环不变量，因为初始化建立它，每一轮保持它，退出条件再利用它推出结果。

初始化为什么建立不变量。第一轮之前 $k = 0$ 。固件令 $r0=\text{array_base}$ 、 $r1=4$ ，第一条任务指令令 $r4=0$ 。空前缀的和定义为 0，因此三个等式全部成立。这里能看出任务代码和启动约定共同建立初态：若固件传错 $r0$ 或 $r1$ ，单靠 `MOVI r4,0` 不能修复。

一轮怎样保持不变量。假设进入第 k 轮时关系成立。LOAD 从 $\text{array_base} + 4k$ 读取 a_k ，ADD 把它加入 $r4$ ；`ADDI r0,4` 令指针对应 $k+1$ ，`ADDI r1,-1` 令剩余数变成 $n - (k+1)$ 。于是下一次进入循环时：

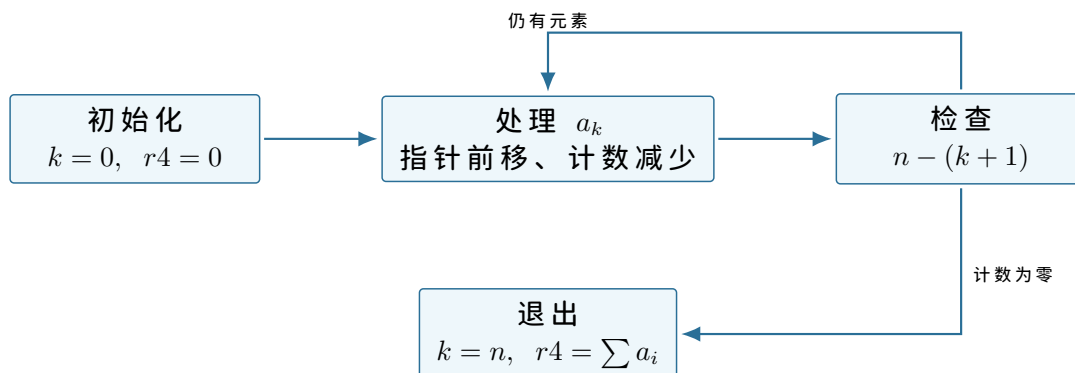
$$r4 = \sum_{i=0}^k a_i,$$

正好是把 k 替换为 $k+1$ 后的不变量。

退出条件怎样推出最终结果。分支不采用循环目标时 $r1=0$ 。由 $r1 = n - k$ 得 $k = n = 4$ ，因此

$$r4 = a_0 + a_1 + a_2 + a_3 = 7 - 2 + 11 + 5 = 21.$$

STORE 把该值写到结果槽。这个推导还揭示零长度问题：当前代码在第一次分支检查前就执行装入，只有 $n > 0$ 时初始化到第一次循环入口的路径才满足预期使用条件。



图示：机器 trace 给出每步具体值，不变量说明这些值为何对任意满足启动契约的非空数组都沿同一关系推进。

装入一项任务需要多少次数据移动。载荷长度 $0x2100$ 等于 8448 字节，也恰好是 132 个 64 字节块。对每个字节，串行控制器先把外部位收进 **RXDATA**；处理器读取一次 **RXDATA**；随后处理器向 RAM 写入一次。仅计算有效数据移动，就有 8448 笔设备读和 8448 笔 RAM 写。

状态轮询次数不固定。如果每个字节到达时固件恰好检查一次，需要至少 8448 笔 **STATUS** 读；若固件检查十次才等到一个字节，状态读会增加到约 84480 笔。轮询的成本来自“没有数据时仍要主动读取状态”。

动作	本次最少次数	次数由什么决定
读取 RXDATA	8448	载荷字节数
写入任务 RAM	8448	载荷字节数
读取 STATUS	不少于 8448	到达间隔与轮询频率
发送块确认	132 次确认帧	64 字节分块规则
零填充写入	256 字节	<code>zero_bytes=0x100</code>
建立返回地址	1 笔 4 字节写	启动栈约定

这个计数没有把取指事务算入，因为固件执行每条轮询和搬运指令还要取指。即使不建立精确周期模型，也能看出装入成本至少与载荷长度线性增长。DMA 可以减少逐字节处理器搬运，但要引入描述符、缓冲所有权和完成事件；当前规模没有要求它。

确认帧不是免费的。每 64 字节等待一次确认会限制连续突发长度，也增加往返时间。若确认帧占 8 个串行字节，132 次确认会额外发送 1056 字节。把块增大到 256 字节可减少确认次数，却会让出错后重传更多数据，并要求控制器或协议容纳更长未确认区间。

因此块大小不是任意常数。它在状态开销、错误恢复粒度和缓冲需求之间取舍。本章固定为 64 字节，只为使单槽轮询方案具有明确背压点。

三个快照足以区分“装入”“运行”和“返回”。

快照一：映像已经在 RAM，但固件仍运行。

```
load_record.state = VERIFYING
PC = 0x00000280
SP = 0x001ffec0
```



```
MEM[0x00100000..0x001020ff] = received image
task registers = not established
```

这时读取任务区域能看到完整字节，却不能说任务已运行。PC 仍在启动存储，寄存器和栈也仍属于固件阶段。

快照二：第一条任务指令尚未提交。

```
load_record.state = RUNNING
PC = 0x00100000
SP = 0x001efffc
r0 = 0x00102000
r1 = 4
r2 = 0x00102020
```

PC 已指向任务入口，下一笔取指将访问 RAM。第一条 **MOVI** 尚未提交，因此这个快照是控制权移交后的初始任务现场。

快照三：任务返回入口刚获得控制。

```
PC = 0x00000300
SP = 0x001f0000
r4 = 21
MEM32[0x00102020] = 21
firmware SP = not restored yet
```

处理器已经进入固件地址，但返回入口的第一条初始化指令尚未执行。SP 仍表示任务栈的空栈位置；固件必须先建立自己的 SP，之后才能安全调用发送例程。

快照	PC 所属范围	SP 所属范围	当前可执行者
映像已到齐	启动存储	固件工作区	固件装入器
任务初始现场	任务 RAM	任务栈	求和任务
返回入口刚进入	启动存储	任务栈上界	固件返回入口

第三行尤其说明 PC 和 SP 不必在同一条指令边界同时“属于同一软件”。受控入口的第一项职责常常就是从被打断或返回方的栈切换到入口代码自己的栈。当前固定返回协议很简单，但已经展示了这个状态过渡。

错误码怎样对应到最早能够判断的层次。

错误码	最早检测者	能证明的事实
SERIAL_OVERRUN	串行控制器/固件	至少一个外部字节未被消费
HEADER_INVALID	固件头部验证	字段不能形成安全布局
CHECKSUM_MISMATCH	固件载荷验证	实收载荷不同于发送端声明
INVALID_OPCODE	处理器译码	PC 指向的四字节不是合法指令
UNMAPPED	地址互连	一笔合法格式请求没有目标
ALIGNMENT	处理器或互连	地址违反当前宽度对齐规则

错误码不能相互替代。校验失败不能说明是哪条指令错误；无效操作码也不能证明串行传输损坏，因为发送端可能原本就提供了错误代码。诊断应报告最早被可靠观察到的契约破坏，同时保留 PC、地址和装入阶段等上下文。

经过这些计数、快照和不变量检查，本章不再依赖“任务应该可以运行”的直觉。机器的正确起点、数据移动量、执行结果和失败位置都能由具体状态复算。

同一个“完成”对三个观察者含义不同。求和任务、固件和外部发送端处在不同位置，能够观察的事件也不同。任务在存储指令收到成功响应后知道结果已经进入 RAM；固

件只有在任务返回后才开始读取结果；外部端则要等结果帧通过串行线到达并完成校验。

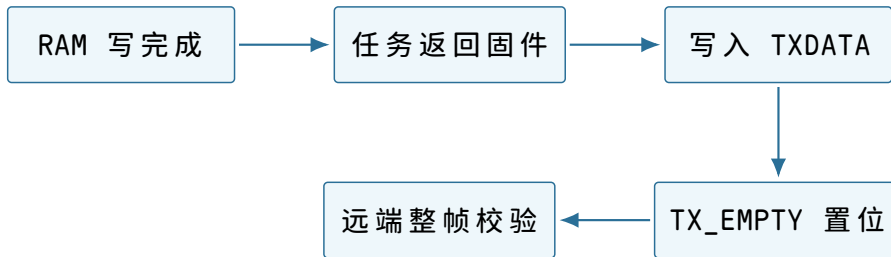
观察者	首次能够确认的事件	此时仍不能推出
求和任务	STORE 得到 OK	固件已经运行、外部端已经收到
固件返回入口	重新读取结果槽得到 21	发送线已经传完结果帧
外部发送端	完整结果帧到达且校验通过	结果断电后仍会保留

若固件在写入 TXDATA 后立即复位串行控制器，最后一个字节可能还在移位寄存器中，外部端收不到完整帧。因而“处理器完成设备寄存器写入”和“设备完成物理发送”也不是同一时刻。最小接口可以增加 TX_EMPTY 状态，表示发送槽和移位寄存器都为空；固件在进入等待或复位前检查它。

硬件模型。发送侧除 tx_ready 外保存移位寄存器占用状态，最后一位离开引脚后置 tx_empty=1。

软件模型。STATUS 的 bit3 暴露 TX_EMPTY。写 TXDATA 后该位清零，整字节发送结束后重新置位。

抽象。软件等待 TX_EMPTY=1，可以依赖此前提提交的字节已经离开控制器；这仍不保证远端正确采样或通过帧校验，端到端确认必须由远端协议提供。



图示：五个事件顺序发生，分别关闭计算、控制流、设备提交、物理发送和端到端接收边界。

重新运行任务与整机复位有什么不同。固件收到下一份合法头部后，可以覆盖任务区、重建栈并再次提交入口，而不必让电源复位。这种重新运行保留启动存储内容和固件初始化后的设备配置，只重建与本次任务相关的状态。

整机复位则把 PC 送回 0x00000000，清除处理器控制状态、串行协议状态和互连未完成事务。RAM 字节可能仍在，但不再被信任。两条路径的共同点是：在任务入口提交前都必须重新验证映像并建立现场。

状态	固件重新运行任务	整机复位
启动存储	不变	不变
固件设备配置	可以沿用	重新初始化
RAM 旧任务字节	按新映像覆盖或清零	可能仍在，但必须视为未验证
处理器 PC/SP	由固件直接建立新任务现场	先回复位入口，再执行固件
未完成串行事务	由协议明确丢弃和重同步	控制器复位后丢弃

“重启程序”在当前裸机上不是硬件指令，而是一段固件流程。硬件复位提供确定的最外层恢复手段，固件重新运行则是较小范围的软件动作。

自修改写入说明保护不能由布局表代替。设错误任务把结果地址 r2 改成 0x00100004，即循环中 LOAD 指令的位置。最终 STORE 会把整数 21 的字节 15 00 00 00

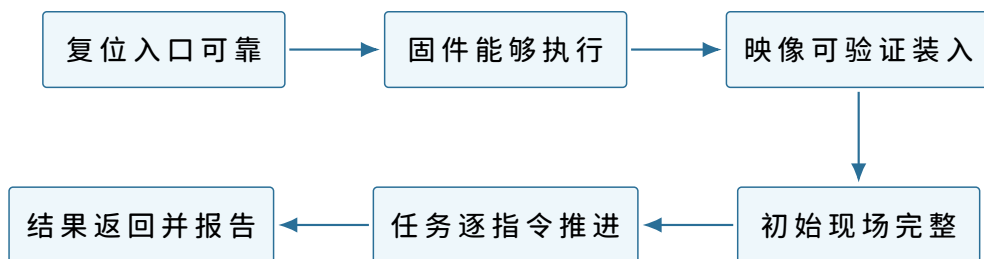
写入代码。任务随后立即执行位于 `0x0010001c` 的 `RET`，所以本次仍可能正常回到固件。

固件若不重新装入就再次从 `0x00100000` 运行任务，第二条指令字节已经不再是 `20 05 00 00`。操作码 `0x15` 未定义，处理器进入 `INVALID_OPCODE` 错误入口。第一次运行的结果正确，第二次才暴露代码破坏。

时刻	MEM[0x00100004..07]	后果
装入完成	20 05 00 00	译码为数组装入
错误存储后	15 00 00 00	当前 PC 已越过此处，暂未失败
再次从入口执行	同上	第二条取指得到无效操作码

地址布局告诉任务“哪里应当是代码”，RAM 和互连却没有只读规则。要阻止这种写入，机器需要一种把地址范围与允许操作绑定的强制机制。此处只记录失败，不提前展开地址保护；它会在后续机器真正需要隔离时成为新的演进问题。

本章事实之间的最终依赖关系。



图示：每个方框都依赖前一个方框已经建立的状态。缺少任意一环，都不能从“字节存在”直接推出“任务完成”。

至此，最小机器的器件、连接、软件接口、启动协议、执行 trace 和错误边界均已落到具体状态。后续章节可以把这台机器作为已知起点，而不必再次把“处理器能够执行一项任务”当作未经证明的前提。

从可观察现象反推连接错误。具体机器模型还应支持诊断。若上电后没有得到结果，不能只说“机器没跑起来”，而要利用 PC、事务和状态寄存器把故障范围逐步缩小。

复位后连第一笔请求都没有。若示波观察不到 `req_valid`，先检查时钟是否产生边沿、复位是否释放以及 PC 是否得到规定值。此时还没有理由检查任务映像，因为固件第一条取指尚未发生。

请求地址正确，但始终没有响应。观察到地址 `0x00000000` 和读请求，却没有 `resp_valid`，故障位于互连选择、启动存储连接或目标握手。若互连错误地把该地址送往 RAM，RAM 中的随机字节可能返回成功，随后表现为无效操作码。区分“无响应”和“错误目标响应”需要同时观察目标选择信号。

固件运行，但接收状态永远为零。若 PC 已在固件轮询循环，`STATUS` 读也持续成功，问题不在处理器取指和 RAM。应检查外部发送端是否产生串行位、控制器采样时序是否匹配以及接收移位寄存器是否能置 `RX_READY`。软件看到的零值只是控制器没有宣布完整字节，不足以判断线路哪一端失效。

任务结果正确，外部端却没有收到。先读 `MEM32[0x00102020]`。若值为 21，计算和 RAM 写入已经完成；再看 PC 是否到达固件返回入口，区分“任务未返回”和“返回后发送失败”；最后检查 `TX_READY/TX_EMPTY` 与远端帧校验。诊断顺序沿完成边界向外推进。

最后确认成功的事实	下一项观察	尚不应检查的远端事项
时钟与复位正常	第一笔取指请求	任务求和结果
固件取指成功	串行状态与接收字节	任务返回地址

映像校验成功	初始现场与入口 PC	外部结果帧
结果槽为 21	返回入口 PC	传输校验
TX_EMPTY=1	远端接收与帧校验	RAM 计算过程

这种从已知成功边界向下一边界推进的方法，与后续操作系统中的系统调用、设备请求和文件持久化诊断具有相同结构：先确定事实在哪一层成立，再检查下一层所需的新状态。

最小机器的最终接线清单。到本章末再列清单，作用是复核已经推导的连接，而不是用名词代替正文：

连接	承载的信息	接错后的直接后果
时钟到处理器、互连和控制器	共同状态更新边沿	握手双方不能在同一边界观察状态
复位到处理器与控制状态	PC 起点、协议初态	旧事务或任意 PC 被当作有效
处理器请求到互连	地址、读写、宽度、写数据	无法取指或访问错误目标
目标响应经互连回处理器	状态与读数据	指令永久等待或读取错误字节
互连到启动存储	固件读请求	复位后没有可靠指令
互连到 RAM	任务取指和数据读写	任务无法装入或保存运行状态
互连到串行控制器	状态、收发字节	外部任务无法进入或结果无法报告
串行控制器到外部端	按时间传送的位	软件寄存器与外部信息断开

每一行都能回到前文的一次具体事务。清单中没有中断控制器、计时器、地址转换器、磁盘或操作系统；这些结构只有在后续问题确实需要时才会加入，并重新说明硬件模型、软件接口和提供的抽象。

本章得到的机器。固定启动程序已经能够接收、验证和装入一项外部程序，并在程序合作返回后等待下一次传输。但 RAM 中只有一套当前程序布局 and 一份活动处理器状态。若希望保留 A 的中间结果，同时让 B 获得处理器，就必须回答暂停现场保存在哪里以及怎样恢复。

第二项任务为什么需要常驻协调代码

继承的机器。固件能装入一项任务，为它设置入口、栈和初始寄存器；任务完成时主动跳回固定入口。所有地址仍是物理地址，任务仍可访问整台机器。

直接从 A 跳到 B 会丢掉什么

设 A 正在累加一组数据。下一条指令地址保存在指令位置寄存器中，当前循环计数和部分和位于通用寄存器，函数调用关系位于 A 的栈。此时若把指令位置改成 B 的入口，处理器会立即开始取 B 的指令；但它仍带着 A 的栈顶与寄存器值。

给 B 设置自己的栈和寄存器可以让 B 正确开始，却会覆盖处理器中唯一的一份 A 状态。以后再把指令位置改回 A，并不能恢复 A，因为“不知道回到 A 的哪条指令”只是丢失状态的一部分。

必须保留的状态	丢失后的表现	为什么不能从代码映像重建
继续指令位置	A 从错误位置执行或重新开始	它取决于暂停发生的时刻
栈指针	返回地址与局部变量无法找到	栈深度随已经完成的调用变化
通用寄存器	循环变量、地址和中间结果改变	它们是运行结果，不是初始映像
状态标志	后续条件分支得到不同结果	标志来自暂停前最后一次运算
约定的扩展状态	浮点或向量计算被 B 覆盖	同样属于运行中的中间结果

这组状态可以称为任务的处理器现场。名称出现在问题之后：我们需要一份数据，能够让暂停的任务在未来像没有被中断一样继续，才有必要给这份数据命名。

现场保存在哪里

不能把 A 的现场继续留在处理器寄存器里，因为 B 就要使用这些寄存器。也不能把所有现场都压到同一栈上，再任意切换栈而不记位置。最小安排是为每项任务分配一块现场记录 and 一段独立栈：

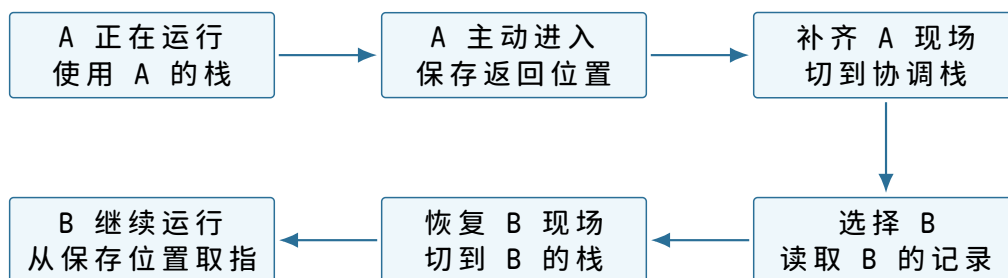
```
task_state A:
    status
    resume_instruction
    stack_pointer
    general_registers
    arithmetic_flags
    stack_low, stack_high

task_state B:
    same fields, different storage
```

status 至少区分“尚未开始”“可以继续”“正在运行”和“已经完成”。它不是为了美化结构，而是避免协调代码把一份只有初始入口、尚无暂停现场的记录当作可恢复记录。

谁有资格保存现场。若 A 自己把寄存器依次写入记录，会遇到一个细节：执行保存代码本身也需要寄存器，而且“保存后的继续地址”应当指向 A 交还处理器之后的位置。最小方案规定一个调用约定：A 像调用函数一样进入协调代码，调用约定先把一部分状态压到 A 的栈，协调代码再补存其余状态并记录当前栈指针。

协调代码必须常驻在不会被任务映像覆盖的内存中。它还需要一个只在选择任务和恢复现场时使用的协调栈，否则恢复 A 的栈之后再运行复杂选择代码，会误把协调者的局部数据写进 A。



图示：切换不是把 A 的代码换成 B 的代码，而是先把处理器唯一的活动现场写回 A 的记录，再把 B 的记录装入处理器。

第一次运行和恢复运行为何不同。B 第一次获得处理器时，没有“暂停位置”。它的记录由装入者人工构造成为一种可恢复形态：继续位置设为 B 的入口，栈指针设为 B 的初始栈顶，其余寄存器按启动约定给值。以后 B 主动交还处理器，记录才被真实运行状态覆盖。

B 的阶段	记录中的继续位置	栈中已经存在的内容
尚未第一次运行	任务入口	人工准备的初始参数或空栈
运行后主动交还	交还入口之后的继续点	B 已经形成的调用链与局部数据
已经完成	不再作为可恢复位置使用	可等待回收或检查结果

统一的恢复代码只读取记录，不必在最后一步区分“第一次”和“后来”。区别已经在记录如何构造、状态字段是否允许恢复中表达。

最小选择规则怎样形成

当前只有 A 和 B。协调代码可使用一个当前编号：A 交还时若 B 可继续，就选择 B；B 交还时再选择 A。这个规则尚不需要复杂队列，但仍要处理任务完成：

```
choose_next:
  inspect the other task
  if it is ready:
    return that task
  if current task only yielded and remains ready:
    return current task
  otherwise:
    wait for an external event
```

这里第一次出现“选择下一项可运行任务”的职责。由于选择只发生在任务主动进入协调代码后，它仍是合作式安排：协调者能决定交还之后运行谁，却不能决定任务何时交还。

一次切换的状态账本。

时刻	处理器活动现场	A 的内存记录	B 的内存记录
t_0	A 的状态	旧值，不可用于恢复 当前 A	B 的初始或暂停状态
t_1	A 正在执行保存入口	部分字段已更新	不变
t_2	协调代码使用协调栈	A 的完整暂停状态	不变
t_3	正在装入 B 的状态	完整且稳定	被读取，尚未运行
t_4	B 的活动现场	A 可恢复	B 记录成为旧值

任一时刻，正在运行任务的最新寄存器在处理器中，其内存记录只是上一次保存值；暂停任务的最新现场则必须完整位于内存记录和它自己的栈中。说“每项任务都有一份现场”时，需要同时说明活动现场究竟位于处理器还是内存。

这段常驻代码已经是什么

它不是完整操作系统，却已经具有三个此前不存在的职责：保存当前任务、选择下一任务、恢复所选任务。由于任务运行期间协调代码仍保留在内存中，可以把它称为常驻协调者。以后若加入更多任务，选择规则和记录集合会扩展，但本章不提前设计最终结构。

章末出现的问题。A 可以约定每处理一批数据就主动交还处理器；B 却可能进入无限循环，或者因为错误永远不调用交还入口。此时协调代码虽然常驻，却没有任何机会执行。下一章只处理“机器怎样在任务不合作时重新得到控制”。

任务不交还处理器时怎么办

继承的机器。常驻协调者能保存、选择和恢复 A/B；每项任务有独立现场记录与栈。切换入口只能由正在运行的任务主动调用。

常驻并不等于有机会执行

B 执行下面的指令序列：

```
repeat_forever:
  compute
  jump repeat_forever
```

处理器会不断从 B 的地址取指。协调代码虽然存在于另一段内存，但“存在”不会自动改变指令位置。外部观察者也不能靠在内存里写一个标志解决问题，因为 B 没有读取该标志。

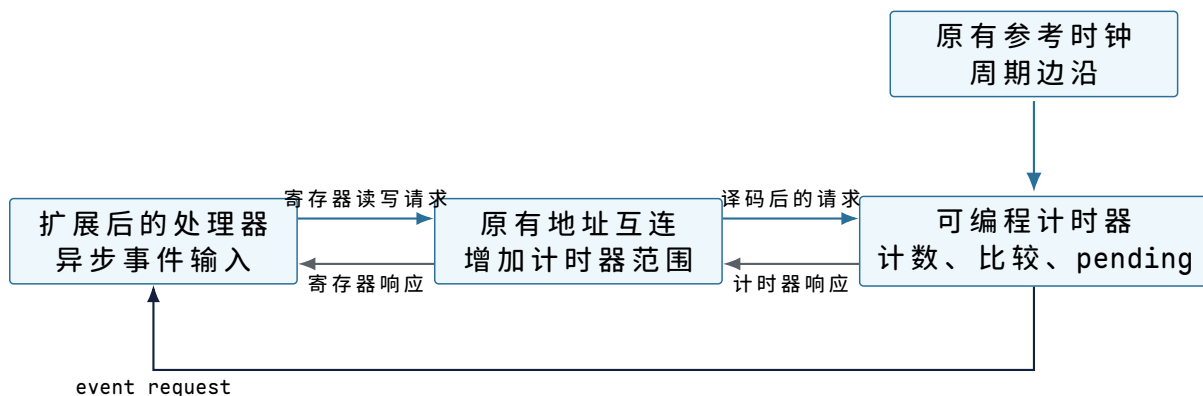
要在 B 不执行交还指令时重新获得控制，机器必须提供一种与当前指令流独立的事件：事件到达后，处理器先保存足以继续 B 的状态，再把指令位置改成预先登记的入口。

为什么选择可编程时钟事件

设备完成也能产生外部事件，但机器空闲时未必有设备活动，不能用它保证处理器一定被收回。一个可编程计时器可以在设定时间后发出事件，与 B 是否读内存、是否调用函数无关。

新器件接到现有机器的哪里。第一章的时钟源只能推动电路状态变化，软件不能要求它在未来某个计数值通知处理器。现在增加的计时器是一项独立器件：它接收同一参考时钟，通过地址互连暴露配置寄存器，并用一根事件请求线连接处理器的异步事件输入。

本样机只有一个事件源，所以计时器请求线直接连接处理器，不增加中断控制器。以后事件源增多，才会出现“多根请求线如何编号、屏蔽和路由”的新问题。



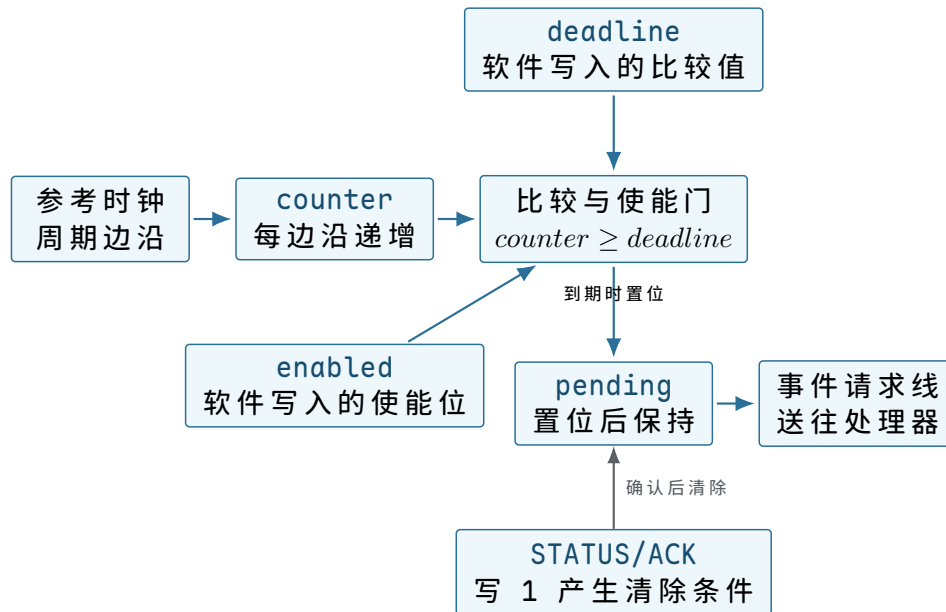
图示：总线方向用于配置和查询，事件请求线用于计时器主动通知处理器。软件写过寄存器并不等于事件已经发生；只有计数条件满足后，计时器才置 pending 并拉起请求线。

计时器的硬件模型。计时器内部保存四类最小状态：

内部状态	状态变化	对外作用
counter	每个参考时钟边沿递增	提供单调增加的比较基准

deadline	处理器通过写事务设置	指定下一次到期计数值
enabled	处理器启用或停止计时比较	决定是否检查到期条件
pending	到期时置 1，确认后清 0	为 1 时保持事件请求有效

这四类状态通过比较器和置位/清除逻辑连接，而不是由软件循环反复读取后自行判断。参考时钟只递增 `counter`；比较器同时观察 `counter`、`deadline` 与 `enabled`；比较成立时，它向 `pending` 发出置位条件。软件确认走独立清除输入，因此一次很短的“到期相等”可以转换为持续有效的事件请求。



图示：比较器只产生置位条件，`pending` 才保存尚未处理的事件。请求线直接反映 `pending`，所以处理器暂时未进入事件入口时，通知不会因比较条件只成立一个边沿而丢失。

每个时钟边沿后，若 `enabled=1` 且 `counter >= deadline`，硬件就设置 `pending=1`。请求线保持有效，直到软件确认事件。采用“保持”而不是一个极短脉冲，可以避免处理器暂时关闭事件接收时永久丢失通知。

这个模型没有自动周期。协调代码处理一次到期事件后，写入新的 `deadline`，才形成重复时间片。这样可以先看清“本次到期”和“安排下次到期”是两个动作。

计时器的软件模型。 软件不直接访问内部连线，而是通过地址互连看到一组内存映射寄存器：

物理地址与名称	访问方式	软件可见语义
0x40001000 COUNT	只读	返回当前 <code>counter</code> 快照
0x40001008 DEADLINE	读写	设置下一次到期比较值
0x40001010 CONTROL	读写	bit 0 控制 <code>enabled</code>
0x40001014 STATUS/ACK	读/写	读 bit 0 得 <code>pending</code> ；写 1 清除 <code>pending</code>

让计时器在 Q 个计数单位后产生一次事件，软件按以下顺序配置：

```

now = read64(COUNT)
write64(DEADLINE, now + Q)
write32(STATUS_ACK, 1)      // clear an earlier pending event
write32(CONTROL, ENABLE)

```

若先启用、后写 deadline，旧 deadline 可能立即满足并产生一次非预期事件。寄存器顺序因此是软件模型的一部分，不只是代码风格。

计时器提供的抽象。上层软件得到“在单调计数达到 deadline 后形成一个保持到确认的事件”。它没有得到以下保证：

- deadline 不是日期，也不自动等于毫秒；
- 到期表示请求已经提出，不表示处理器恰好在该计数值执行入口；
- 事件被屏蔽时，请求可以延迟交付；
- 计时器不会保存任务现场，也不会选择下一项任务。

因此，计时器抽象适合保证协调代码最终重新获得一次入口机会；任务选择仍属于软件。

处理器异步入口也需要明确三层模型。计时器有了请求线，原处理器仍只会顺序取指。机器还要扩展处理器的事件入口能力。

硬件模型。处理器在一条指令完成后检查事件输入。若事件允许且请求有效，处理器保存下一条指令位置、原栈指针和状态标志，切到固定入口栈顶，关闭同类事件嵌套，再把指令位置改为固定事件入口。事件返回指令执行相反的受控恢复。

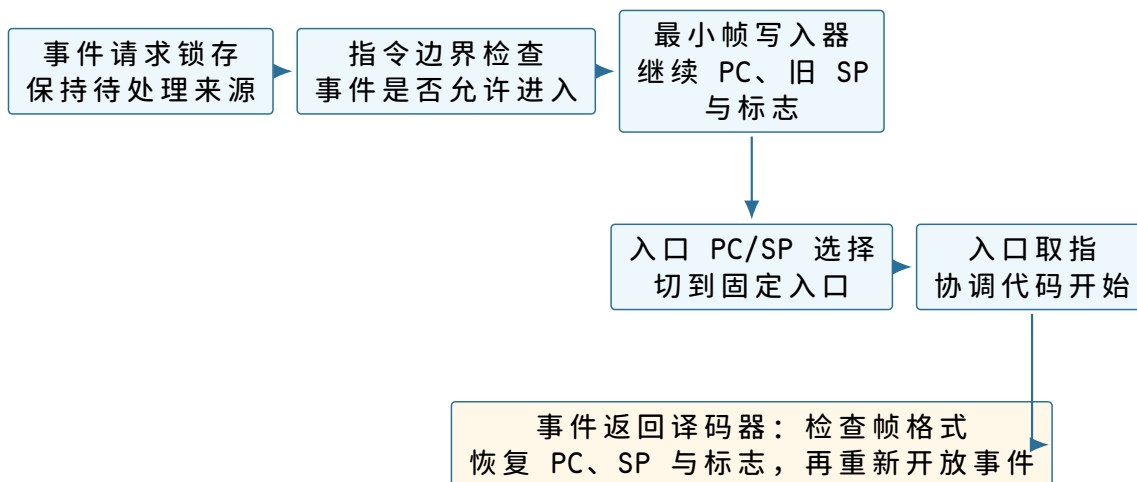
软件模型。固件把事件入口约定为 `0x00008000`，把入口栈顶约定为 `0x001f0000`。硬件建立的最小帧按顺序包含：

```
event_frame:
  resume_instruction
  interrupted_stack_pointer
  saved_flags
```

入口代码能读取这份帧，并通过确认寄存器清除计时器 pending。普通通用寄存器尚未由硬件保存，必须由入口软件补齐。

抽象。软件得到一个可在普通指令流之外发生的受控入口，以及一份足以返回被打断位置的最小继续状态。该抽象不等于完整上下文切换：它不保存全部寄存器，也不决定事件到达后继续原任务还是恢复另一个任务。

处理器扩展后的硬件路径可以分成五个相邻结构。请求锁存器记住计时器通知；边界检查器只在一条普通指令已经提交且事件允许时接受它；最小帧写入器把继续位置、旧 SP 和标志写到入口栈；入口选择器再把 PC 与 SP 改为固定入口值。事件返回译码器只接受符合约定的帧，并按相反方向恢复这些字段。任何一步失败都不能假装已经进入协调代码。



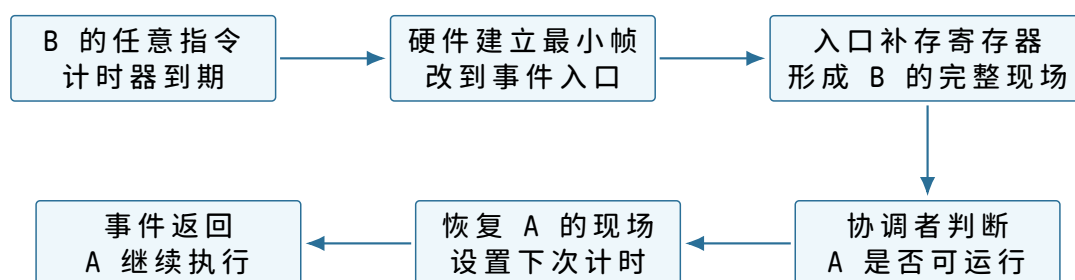
图示：异步入口不是把请求线直接接到 PC。处理器先等待精确指令边界，再建立可返回的最小帧，最后切换入口 PC/SP；事件返回只有在帧检查通过后才恢复被选中的执行流。

为支持这条路径，硬件至少要明确：

硬件能力	必须提供的事实	软件随后才能做的事
计时器	到期时产生一个可识别事件	规定最长连续运行区间
事件入口表	事件编号对应固定入口地址	让协调代码得到控制
最小现场保存	保存被打断继续位置与必要标志	以后从原指令流继续
事件屏蔽状态	防止入口最早阶段被同类事件反复覆盖	完成一次一致的现场建立
事件返回指令	从规定帧恢复状态与执行级别	回到所选任务

硬件不必替软件决定运行 A 还是 B。它只保证事件发生时能离开 B，并把控制交到一个软件已知入口。选择仍由协调代码完成。

硬件保存的状态为什么通常不够。事件到达的第一目标是安全进入入口，而不是一次完成所有任务切换。处理器通常只自动保存继续位置、状态标志以及进入和返回所必需的少量字段。通用寄存器若要保持不变，入口软件仍要尽早保存。



图示：时钟事件只强制打开入口机会。完整保存、任务选择和恢复仍是软件协议。

入口不能先使用尚未保存的通用寄存器计算，因为那会覆盖 B 的值。最早几条指令只能使用硬件约定允许破坏的状态，或者把寄存器按固定顺序压入已知安全的栈。

事件入口使用哪一份栈。B 的栈可能已接近边界，甚至因为错误把栈指针改成无效值。若硬件入口无条件继续使用 B 的栈，保存第一项状态就可能失败。机器需要一个不由普通任务控制的入口栈位置；单核样机可让硬件或最早入口从固定协调状态取得它。

栈	保存的调用关系	所有权边界
A 的任务栈	A 的函数返回地址与局部数据	A 运行或暂停时保留
B 的任务栈	B 的函数返回地址与局部数据	B 运行或暂停时保留
协调入口栈	事件最早保存与选择代码局部状态	普通任务不应改写

入口栈解决的是“如何安全开始保存”，不是自动隔离。由于当前机器仍允许 B 写任意物理地址，B 依然能够故意或错误地覆盖协调入口栈；这个缺陷留到章末。

怎样把事件帧并入 B 的任务记录。事件帧记录 B 被打断的位置，软件补存通用寄存器。协调者可以让 B 的任务记录指向这份完整帧，而不必把每个字段再复制一次。关键是记录必须说明帧位于哪一段栈、布局采用哪个入口约定。


```

timer_entry:
    hardware has saved resume position and flags
    save remaining registers in fixed order
    current_task.saved_stack = current stack pointer
    current_task.status = ready
    switch to coordinator stack
    next = choose_ready_task()
    restore next.saved_stack
    return_from_event

```

主动交还与时钟打断可以最终汇入相同的“已形成完整现场”边界，但两者最早入口不同。主动交还遵守普通调用约定；时钟可以发生在任意指令，必须按异步入口约定保存所有可能被后续代码破坏的状态。

第一次出现可运行集合

只有两项任务时也需要区分“暂停但能继续”和“已经完成”。协调者不应把完成的任务重新装入处理器。可以维护一个很小的可运行集合：

任务状态	是否参与选择	状态改变的入口
ready	是	被时钟打断或主动交还后
running	否，当前已占用处理器	协调者恢复该任务时
finished	否	任务进入完成入口时
not started	可先构造初始现场再加入	装入者完成代码、数据和栈准备时

选择规则可以是 A、B 轮流。每次恢复前重新设置计时器，给所选任务一个有限运行区间。时间区间结束并不表示任务完成，只表示协调者获得一次重新选择的机会。

“被唤醒”和“正在运行”尚未出现。本章任务只做计算，没有等待设备或条件；ready 已足够表达“具有运行资格”。以后加入等待后，某项任务即使没有完成也可能暂时不应进入选择集合，届时才需要额外状态和唤醒协议。现在提前画完整生命周期只会让尚无问题来源的概念进入样机。

一次强制切换的逐时刻状态。

时刻	控制流位置	B 的最新现场	A 的最新现场
t_0	B 的循环	处理器中	A 的内存记录与栈
t_1	硬件事件入口	最小字段在事件帧，其余仍在寄存器	不变
t_2	软件入口完成保存	B 的完整帧与栈	不变
t_3	协调者选择 A	稳定，可供以后恢复	正在被读取
t_4	事件返回到 A	B 的内存现场	处理器中

事件返回的目标不必是刚才被打断的 B。硬件要求返回帧格式正确；软件可以在返回前换成 A 的已保存帧。这个能力使时钟入口成为抢占切换点。

本章新增结构的准确边界

这一阶段只新增：

- 可编程计时器及其事件入口；
- 不依赖普通任务栈的最早入口状态；
- 能容纳异步打断的完整任务现场格式；

- ready/running/finished 三类选择状态；
- 每次恢复前重新设置时间预算的轮流选择规则。

它没有新增文件、用户态、系统调用、虚拟地址、权限或设备服务。所谓“最小抢占调度”在此只表示：协调者能在任务未主动交还时获得入口，并从可运行集合选择另一项任务。

章末出现的问题。 B 已经不能永久独占处理器，却仍能执行一条写指令覆盖 A 的栈、A 的代码、协调者记录或时钟入口。时间上的轮流使用已经建立，空间上的互不破坏仍未建立。下一次演进应从这次真实覆盖失败出发，再决定硬件与软件需要增加什么；本样稿在此停止。

本章得到的机器。 固件能够装入任务；常驻协调代码能够保存和恢复现场；计时器事件使协调代码从不合作的任務手中重新获得处理器。尚未解决的是：如何把每项执行流的长期状态组织成可扩展对象，如何表示可运行与等待关系，以及多核条件下由谁拥有这些队列。下一章只处理这组执行状态，不提前展开用户请求、地址空间或文件。

第 2 章 为什么执行流需要 task 与调度状态

继承的机器。处理器能够在精确指令边界进入计时器入口，入口软件能够把活动现场保存到内存，再恢复另一项任务。当前只有两份手工安排的记录；一旦任务增多、发生阻塞或跨处理器迁移，固定轮换就无法表达谁可以运行、谁在等待以及哪颗处理器拥有队列。

进程资源、线程现场与任务状态

处理器在一个时刻只能按照一套寄存器现场执行一条指令。即使机器有多个核心，可以同时执行的指令流数量仍然受核心数限制；磁盘中等待运行的程序、内存里已经启动的服务以及刚被唤醒的交互任务，通常远多于核心数。操作系统因此需要反复回答两个问题：哪些执行活动目前具备继续运行的条件，以及下一段处理器时间应当交给谁。

先区分“程序”和“程序的一次运行”。可执行文件存放在文件系统中，其中包含指令、初始数据和装载信息。它尚未运行时，没有当前指令位置，没有运行栈，也没有打开文件。要让处理器执行它，系统至少要完成以下工作：

- 1. 建立一套虚拟地址空间，将代码、数据、动态链接器和用户栈映射进去；
- 2. 确定初始指令位置和栈指针，为处理器准备第一份用户态寄存器现场；
- 3. 建立文件描述符表、当前目录、凭据、资源限制和信号处理状态；
- 4. 为这些状态设置归属和生命周期，使退出、等待和资源回收有明确对象。

完成这些步骤后，系统得到的已不再只是一个文件，而是一个具有独立地址视图、资源引用和安全身份的运行实例。这个资源与权限边界称为进程 (*process*)。同一个程序可以启动多次并形成多个进程；它们执行相同指令，却拥有不同的地址空间、文件描述符和运行状态。

为什么还需要线程？考虑一个服务器既要接收连接，又要处理已经到达的请求。若每项活动都使用独立进程，它们天然隔离，但共享连接表、缓存和统计数据时必须借助进程间通信；若所有活动只使用一条执行流，其中一次阻塞读取又会使其他工作停止。系统需要一种中间组织方式：保留一份进程资源，同时允许其中存在多套可独立暂停和恢复的处理器现场。这些执行现场称为 线程 (*thread*)。

状态类别	同一进程内的线程通常共享	每个线程必须分别保存
地址与数据	地址空间、代码、堆和共享映射	用户栈、内核栈、栈指针
文件资源	文件描述符表、当前目录	正在执行的系统调用及其局部现场
安全与限制	凭据、资源限制、命名空间	部分可独立设置的信号屏蔽状态
处理器执行	进程所获得的总体资源环境	指令位置、通用寄存器、浮点与向量状态
调度信息	进程级配额和分组关系	可运行状态、优先级、CPU 亲和性、等待原因

共享使线程之间传递数据成本较低，也意味着错误指针和未同步写入能够影响整个进程。进程隔离解决“一个运行实例不应直接改动另一个实例的资源”，线程同步解决“同一资源边界内的多条执行流怎样有序共享状态”。两者承担的责任不同。

先从一个很小的服务器看问题：

```
while true:
    c = accept(listen_fd)
    data = read(c)
    write(log_fd, data)
    write(c, "ok")
```

如果这段代码只有一个执行流，问题马上出现：没有客户端连接时，`accept` 会等待；客户端发数据慢时，`read` 会等待；磁盘慢时，写日志会等待。CPU 在这些等待期间其实可以运行别的请求，但硬件不会自动知道“这个程序在等网络，那个程序可以继续算”。内核必须把“能继续执行”和“正在等待某个条件”区分成状态，再让调度器只选择能继续执行的实体。

为了统一表示进程中的第一条线程和后来创建的线程，内核通常使用任务(task)表示一个可调度实体。task 保存线程的执行现场和调度状态，并引用所属进程的地址空间、文件表等资源。不同系统的术语和实现结构并不完全相同，本章使用“进程”表示资源边界，“线程或 task”表示调度器直接选择的执行实体。

若调度器完全忽略任务状态，可能写出下面的循环：

```
for task in all_tasks:
    run(task)
```

这个循环把“存在”和“可以运行”视为同一条件。等待磁盘的任务尚不具备继续执行的条件，已经退出的任务不应再次执行，刚被网卡唤醒的任务则需要重新进入候选集合。因此，系统必须将任务的状态与所在队列同时记录下来。

对象或集合	保存的事实	必须维持的关系
进程	地址空间、文件表、凭据、进程关系	资源必须在最后一个有效引用消失后回收
task	寄存器现场、栈、调度属性、等待原因	每个可恢复现场只能处于一个明确状态
runqueue	当前已经具备运行条件的 task	可运行但未占用 CPU 的 task 必须进入且只进入一个运行队列
wait queue	等待同一条件的 task 与等待规则	条件成立时，唤醒路径能够找到相应等待者
CPU current	某个核心当前执行的 task	同一 task 在同一时刻不能被两个核心同时执行

这些关系需要一套状态机。以下模型省略调试暂停、不可中断睡眠等细分状态，只保留建立调度语义所需的主干：

```
NEW      -> RUNNABLE
RUNNABLE -> RUNNING
RUNNING  -> RUNNABLE    (preempt or yield)
RUNNING  -> BLOCKED     (wait for a condition)
BLOCKED  -> RUNNABLE    (condition becomes true)
RUNNING  -> EXITED -> ZOMBIE -> REAPED
```

转换	触发事件	转换后必须成立的条件
----	------	------------

NEW → RUNNABLE	创建所需状态已经完整初始化	task 可见于调度器，且只位于一个 runqueue
RUNNABLE → RUNNING	调度器在某个 CPU 上选中 task	task 从 runqueue 移除，并成为该 CPU 的 current
RUNNING → RUNNABLE	时间片到期、主动让出或高优先级任务抢占	执行现场已保存，task 重新进入候选集合
RUNNING → BLOCKED	等待 I/O、锁、定时器或子进程事件	等待条件和等待队列已经登记，task 不再位于 runqueue
BLOCKED → RUNNABLE	条件成立、超时或可中断等待收到信号	唤醒原因已记录，task 重新进入一个 runqueue
RUNNING → EXITED	执行退出流程	不再被调度，运行资源按引用关系释放
EXITED → ZOMBIE	仍需保留退出状态供父进程读取	仅保留 PID、退出码和必要统计
ZOMBIE → REAPED	父进程或接管者完成等待与回收	最后的进程记录可以释放

状态与队列位置需要共同保持一致。BLOCKED task 不能被调度执行；RUNNABLE task 若没有进入任何 runqueue，将永远得不到 CPU；同一个 task 若被重复入队，可能在两个核心上同时恢复同一份栈。调度器的正确性首先取决于这些不变量，其次才是候选任务之间的排序策略。

上下文切换负责使状态机真正落到处理器上。内核保存当前线程的必要寄存器和栈指针，选择下一个线程，恢复其现场，并在地址空间发生变化时切换页表上下文。共享地址空间的线程之间切换通常不需要更换页表根；不同进程之间切换还会影响 TLB 与 cache 局部性。任务数量远高于核心数时，系统成本不仅包括保存和恢复寄存器，还包括运行队列等待、cache 重新填充和地址翻译缓存变化。

等待是调度的另一半。一个线程不运行，可能是被抢占，也可能是主动睡眠，可能等锁、等 I/O、等 page fault、等子进程、等信号、等定时器。分析性能和死锁时，必须区分 on-CPU 时间和 off-CPU 时间。只看 CPU 使用率会误导：CPU 低可能是线程都阻塞了，CPU 高可能是大量线程在抢锁或忙等。

后续章节的很多机制都会回到这套状态机。缺页时，线程可能睡在“等待磁盘把文件页读入”的队列上；socket 没数据时，线程睡在 socket receive wait queue 上；waitpid 睡在子进程退出事件上；fsync 睡在写回 completion 上。所谓操作系统“同时处理很多事”，本质是把不能前进的执行流从 CPU 上拿下来，并保证条件满足时能找回它。

从一条执行流推导 task 的具体结构

状态机说明了 task 可以在 RUNNING、RUNNABLE 和 BLOCKED 之间移动，但还没有回答一个更基础的问题：task 被移出 CPU 以后，系统究竟靠哪些数据把它恢复回来。不能只回答“保存上下文”，因为用户态返回现场、内核函数调用现场、地址空间身份和调度队列位置并不是同一类状态，也不是在同一时刻保存。

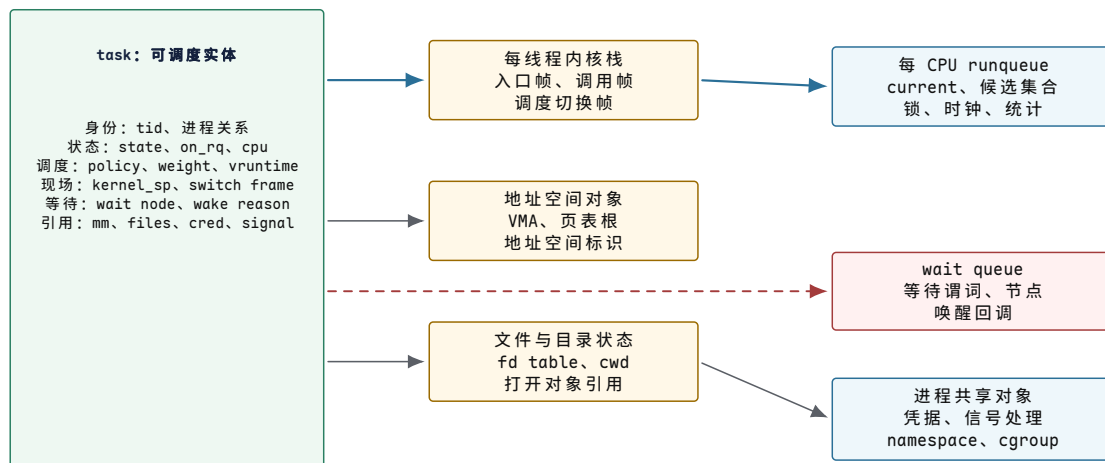
暂停一条执行流需要保留哪些事实

设线程 T 正在用户态执行，随后调用 read，因 socket 暂无数据而阻塞。若将来要让它像普通函数返回一样继续，系统至少要回答五个问题：

1. 返回用户态时，下一条指令在哪里，用户栈顶在哪里，条件码和通用寄存器应是什么值；
2. 阻塞发生在内核调用链的哪一层，恢复内核执行时应使用哪一片内核栈；
3. 用户虚拟地址应按哪套页表解释，线程又引用哪一组文件、凭据和信号状态；

4. 线程当前能否运行，若能运行位于哪个 CPU 的哪个运行队列，若不能运行正在等待什么；
5. 谁仍然引用这条执行流，它退出以后哪些结果要保留到父进程读取。

这五类事实变化频率不同。寄存器可能在每次入口与切换时变化，地址空间通常在 `exec` 或映射操作时变化，文件表可能在 `open/close` 时变化，调度字段会在每次入队、出队和运行核算时变化。把全部内容每次都复制一遍既昂贵又容易失去共享语义，因此 `task` 通常内嵌线程私有和调度所需的热状态，再通过引用连接体积更大、可能由多线程共享的资源对象。



图示: `task` 不是“进程全部资源的盒子”。它保存恢复执行和参加调度所需的私有状态，并引用地址空间、文件表和安全状态。运行时，`task` 由某个 CPU 的 `current` 到达；阻塞时，由等待队列节点到达；两种位置必须互斥。

下面给出一份教学化的最小结构。它不承诺某个内核版本的二进制布局，但字段分组对应真实实现必须承担的责任。省略某个字段并不表示责任消失，而是意味着它必须由另一个可到达对象保存。

```

struct Task {
    // Identity and lifetime
    TaskId tid;
    ProcessId tgid;
    RefCount refs;
    Task *parent;

    // Scheduler-owned state
    TaskState state;
    bool on_rq;
    CpuId cpu;
    SchedEntity se;          // weight, vruntime/deadline, queue node

    // Architecture-owned execution state
    Address kernel_sp;
    SwitchFrame switch_frame;
    ExtendedState *fp_vector;
    TrapFrame *user_return_frame;

    // Wait and wakeup linkage
    WaitNode wait_node;
    WakeReason wake_reason;

    // Shared process resources, reached by references
    AddressSpace *mm;
    FileTable *files;
    Credentials *cred;
  }

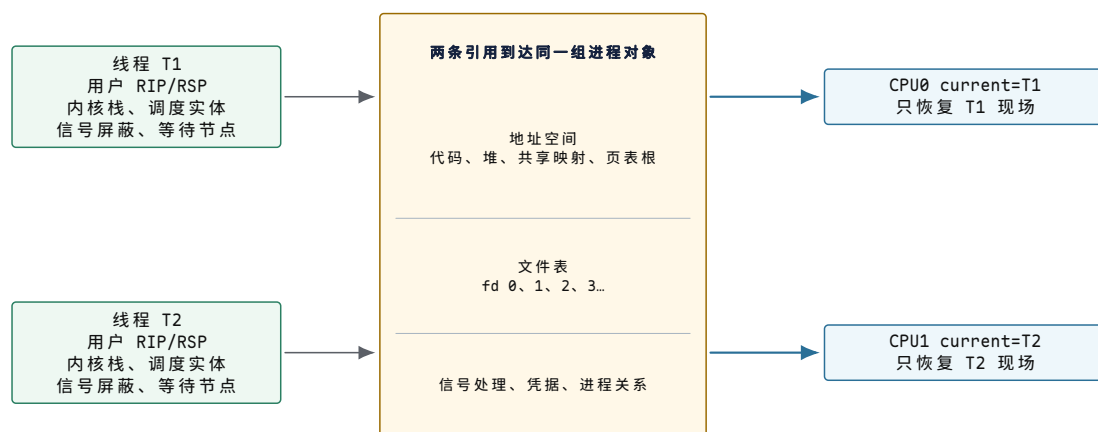
```

```
SignalState *signal;
};
```

字段组	回答的问题	主要修改路径	一致性要求
state/on_rq/cpu	能否运行，位于哪个 CPU 的何处	阻塞、唤醒、调度与迁移	状态、队列成员关系和 CPU 归属必须共同更新
kernel_sp/switch_frame	下次进入该 task 的内核执行流从哪里继续	架构切换例程	保存完成前不能让别的 CPU 恢复同一 task
user_return_frame	离开内核时恢复哪份用户态现场	系统调用、异常和中断入口/出口	必须属于该线程当前内核栈，且返回前经过权限检查
mm/files/cred	地址、句柄和权限按哪套资源解释	fork、exec、资源更新与退出	共享引用表达，最后引用消失前对象不能释放
wait_node/wait_reason	正在等待哪个条件，为何恢复	等待对象和唤醒路径	节点只能属于一个等待队列，醒后必须清理并复查谓词

线程私有状态与进程共享状态怎样分开

同一进程内的两个线程可以使用同一虚拟地址和同一 fd，却不能使用同一套栈指针和同一片内核栈。若 T1 在函数 A 中暂停，T2 在函数 B 中暂停，两者的返回地址、局部变量和寄存器不同；但 T1 调用 `close(3)` 后，T2 再访问 fd 3 会观察到文件表变化。前者要求分别保存，后者要求共享引用。



图示：两个线程各自保存可独立暂停和恢复的执行现场，却通过引用共享进程资源。共享不是把全部字段复制成相同值，而是让两个 task 到达同一对象；私有不是禁止访问同一内存，而是每个 task 有自己的恢复位置和调度归属。

这一区分也解释了线程创建为何比独立进程创建少做一部分工作：它可以增加共享对象引用，不必复制一套独立地址空间；但它仍需分配用户栈、内核栈、task、调度实体和初始寄存器现场。反过来，进程隔离更强，是因为地址空间、文件表和安全状态可以独立管理，而不是因为“进程拥有寄存器、线程没有”。

两类现场：用户返回帧与调度切换帧

“保存寄存器”最容易造成误解，因为入口路径和任务切换路径保存的集合不同。用户程序通过系统调用、异常或中断进入内核时，内核需要保留一份足以返回用户态的现场；调度器从 task A 切到 task B 时，只需保存内核调用约定规定的长期存活寄存器和内核栈位置。前者称为 用户返回帧 (trap frame)，后者称为调度切换帧 (switch frame)。

用户返回帧保存特权边界两侧的契约

以 x86-64 的常见组织为例，用户返回帧至少要表达用户指令位置、用户栈、标志、系统调用号或异常原因、参数寄存器和返回值位置。不同入口指令由硬件直接保存的字段并不完全相同；入口汇编必须把硬件已经提供的字段与软件补存的通用寄存器整理成内核能够统一处理的布局。因此，下面强调字段责任，不把一种具体指令的压栈顺序误写成所有平台的固定 ABI。

现场字段	保存的事实	为什么返回时需要	典型保存者
用户 RIP	被打断或系统调用之后的继续位置	离开内核后从正确指令边界继续	入口硬件状态与入口代码共同整理
用户 RSP	用户栈顶	恢复原调用栈和局部变量可达性	特权入口路径
RFLAGS	条件码、方向位及可返回的控制状态	保持用户程序分支与算术语义，并在返回前过滤特权位	入口路径保存，出口路径校验
通用寄存器	系统调用参数、临时值和 ABI 规定的存活值	内核不能无条件破坏用户可见寄存器	入口代码按统一帧布局补存
调用号/异常号/错误码	进入内核的原因	选择处理函数并形成错误结果	调用约定、硬件异常信息与入口代码
用户 CS/SS 或等价特权状态	返回后的代码与栈特权属性	防止构造越权返回环境	特权切换机制保存或由安全出口构造

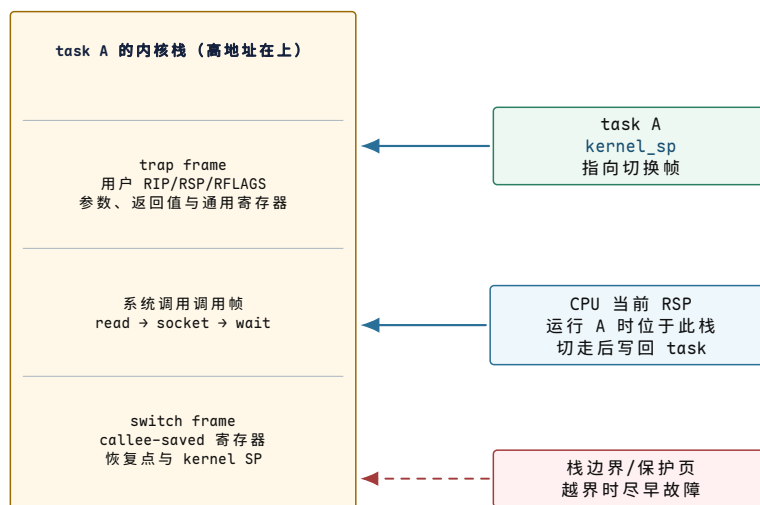


图示：用户态入口不是立即切换 task。处理器和入口代码先在当前 task 的内核栈上构造用户返回帧，内核处理函数仍代表同一 task 执行。只有处理过程中阻塞或出口发现需要重调度，才可能再发生任务上下文切换。

内核栈为何必须属于线程而不是 CPU

系统调用可能在内核深处阻塞。阻塞时，内核调用链中的返回地址、局部变量和已保存寄存器仍位于当前线程的内核栈上。CPU 随后运行另一个线程，如果两者复用同一片尚未保存的栈，后者会覆盖前者的恢复路径。因此，每个可同时处于暂

停状态的线程都需要独立内核栈；CPU 只在运行某个 task 时把当前栈指针指向该 task 的栈。



图示：同一片内核栈同时容纳用户返回帧、普通内核调用帧和调度切换帧。task A 阻塞后，这些内容保持不动；CPU 的 RSP 改指 task B 的内核栈。A 再次被选择时，先恢复底部切换帧，再沿调用链返回，最终使用顶部 trap frame 回到用户态。

内核栈通常较小，因此不能无限递归或放置巨大局部数组；栈底还会设置保护边界，以便溢出尽早变成可诊断故障。中断是否使用当前线程内核栈、每 CPU 中断栈或专用异常栈取决于体系结构与实现，但所有设计都必须回答相同问题：入口时选择哪片栈，嵌套事件的帧放在哪里，切换后谁仍然拥有原栈。

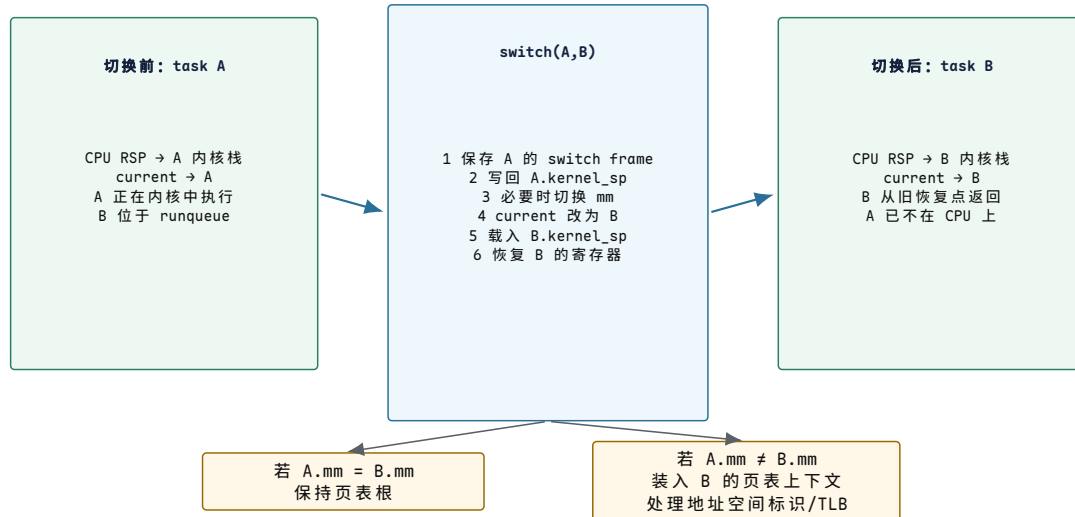
调度切换帧只保存继续内核控制流所需的最小集合

调度器调用切换例程时，本来就处在内核态。按照调用约定，调用者保存寄存器可以在普通函数调用中被覆盖，真正必须跨函数返回保持的是 callee-saved 寄存器、栈指针以及架构要求的控制状态。浮点和向量状态体积较大，内核可以按平台规则立即保存、延迟保存或在首次使用时管理，但不能让下一个 task 继承上一个 task 的用户可见值。

切换状态	是否总在 switch frame 内	作用	可省略或延迟的条件
内核栈指针	是	找到原 task 的调用链与恢复地址	不可省略
callee-saved 通用寄存器	通常是	保持内核调用约定承诺	只能按该架构调用约定精确决定
返回地址/恢复入口	是或由栈布局隐含	让原 task 将来从切换调用之后继续	不可失去
页表根与地址空间标识	由地址空间切换路径管理	让虚拟地址按 next 的空间解释	prev 与 next 共享同一地址空间时可保持
浮点/向量扩展状态	常在独立区域	保存用户可见的 SIMD、浮点与扩展寄存器	仅可依架构支持和所有权规则延迟
TLS、调试和特权控制状态	按平台分散保存	保持线程局部存储、断点和安全控制	值相同或硬件有独立标签时可避免重写

一次上下文切换究竟改变了什么

设 CPU0 当前运行 task A, A 已进入内核并准备阻塞; runqueue 中 task B 可运行。切换前, CPU 的通用寄存器和 RSP 属于 A, `current` 指向 A, 页表上下文为 A 的地址空间。切换后, RSP 和长期存活寄存器属于 B, `current` 指向 B; 只有 A 与 B 的 `mm` 不同时, 才需要改变地址空间上下文。



图示: 上下文切换的前后快照。被切换的是 CPU 当前归属的执行现场和 task 身份, 不是把 A 的全部进程资源复制到 B。共享地址空间的线程切换可以保持页表根; 不同进程切换才进入地址空间切换路径。

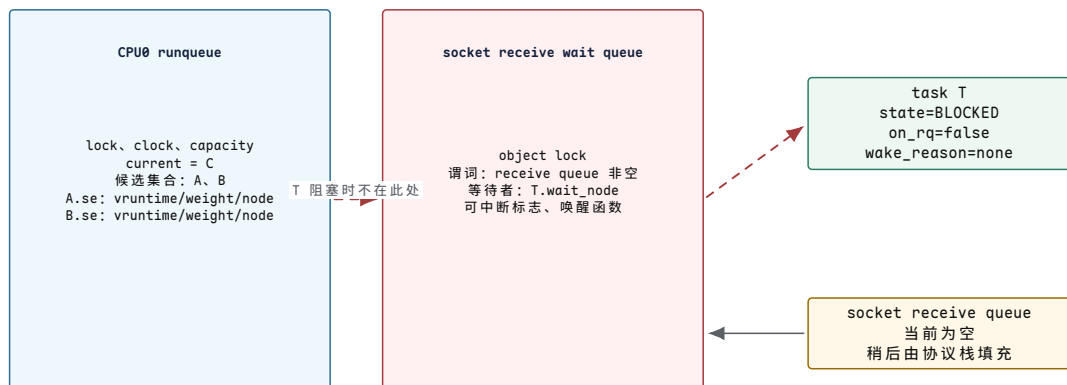
切换例程最反直觉的地方是“由 B 返回”。A 调用 `switch(A,B)` 后, 切换例程把 RSP 改为 B 以前保存的栈指针; 随后执行的返回指令从 B 的栈取出恢复地址, 所以控制流出现在 B 上次被切走的位置。A 没有等在某个隐藏的 CPU 槽位里, 它的等待位置就是 A 内核栈上的 switch frame。将来另一次 `switch(X,A)` 恢复 A 的栈, A 才从最初那次调用继续返回。

状态	切换前	切换动作	切换后
CPU RSP	指向 A 内核栈	保存到 A, 载入 B 的 <code>kernel_sp</code>	指向 B 内核栈
长期存活寄存器	保存 A 的内核计算状态	压入 A 栈, 从 B 栈恢复	包含 B 上次保存值
<code>current</code>	A	在受保护边界更新为 B	B
A 的位置	RUNNING/current	若阻塞则不入 runqueue; 若被抢占则重新入队	wait queue 或 runqueue
B 的位置	RUNNABLE/runqueue	从候选集合删除并选为 next	RUNNING/current
地址空间	A.mm	比较 A.mm 与 B.mm	相同则保持, 不同则使用 B.mm

地址空间切换也不是“清空全部 TLB”这一句就能概括。带地址空间标识的处理器可以让不同进程的翻译项同时留在 TLB 中, 以标识区分归属; 页表映射被修改时仍需按相应范围失效旧翻译。因此, 任务切换、页表根切换和 TLB 失效是相关但不同的事件, 必须分别记录。

运行队列保存候选关系，等待队列保存条件关系

task 只有一份，队列节点却可能服务于不同关系。runqueue 回答“哪些 task 已具备运行条件，按什么字段选择”；wait queue 回答“哪些 task 在等这个对象的哪个条件，条件变化时怎样找到它们”。阻塞过程不是把 task 从一个普通链表机械搬到另一个链表，而是先在对象锁保护下登记等待关系，再撤销运行资格。

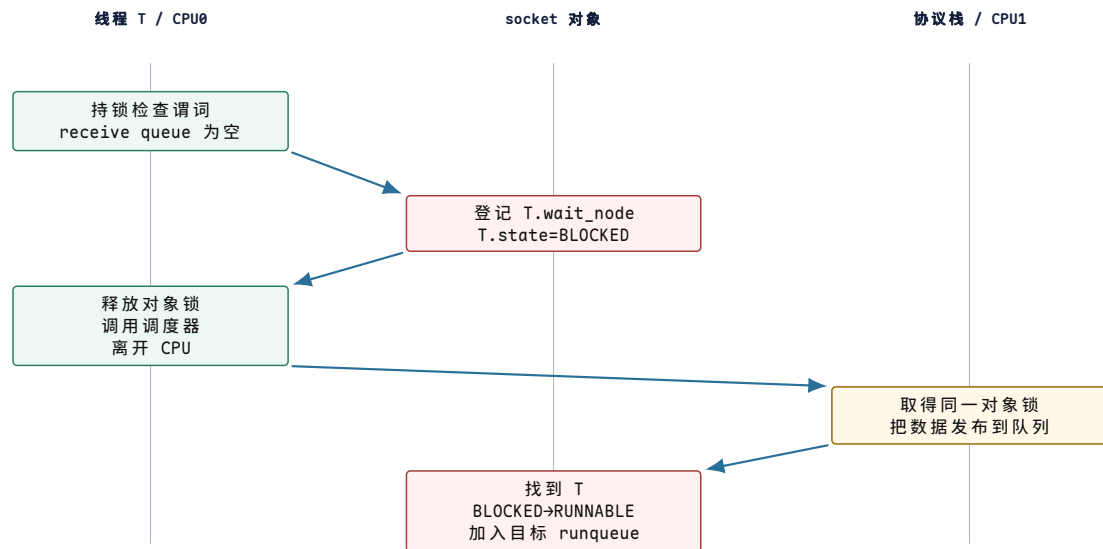


图示：runqueue 与 wait queue 保存不同关系。BLOCKED 的 T 由 socket 等待队列到达，不在任何 runqueue；唤醒后等待节点被清理，T 进入且只进入一个目标 runqueue，但尚未成为某个 CPU 的 current。

runqueue 字段	保存的事实	更新时机	错误后果
current	此 CPU 当前执行哪个 task	任务切换的受保护边界	CPU 记账和实际栈归属不一致
候选集合	具备运行条件但尚未占用 CPU 的 task	创建、唤醒、抢占、选择、迁移	task 丢失或重复执行
队列锁	谁可以同时改变候选关系与归属	每次入队、出队、选择和迁移	两个 CPU 同时选择同一 task
调度时钟与运行统计	当前运行段长度和候选者服务历史	tick、切换和显式核算点	公平、预算和期限判断使用旧值
容量与拓扑信息	此 CPU 能提供多少服务、与其他 CPU 距离如何	拓扑和频率/热状态更新	负载看似均衡，实际完成能力失衡

无丢失阻塞需要一个原子观察边界

仍以线程 T 等待 socket 数据为例。正确顺序不是“看见空，于是稍后睡眠”，而是在同一对象锁保护下完成三件事：检查 receive queue，登记 T 的等待节点，把 T 标为不可运行。生产者也在该同步关系下先把数据发布到 receive queue，再查找等待者并授予运行资格。这样事件要么发生在 T 检查前，T 看见数据而不睡；要么发生在登记后，生产者一定能找到 T。

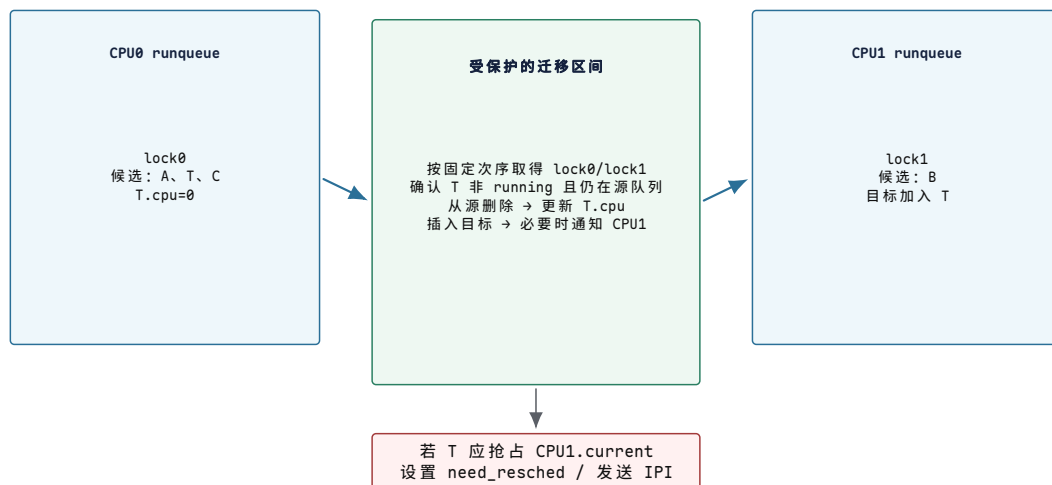


图示：一次无丢失阻塞与唤醒。对象锁把“条件仍为假”和“等待者已经可被找到”接成连续事实。wakeup 的提交结果只是 T 获得运行资格并进入目标 runqueue；T 何时执行、数据是否仍未被其他线程消费，必须在恢复后重新检查。

步骤	T 的状态与位置	socket/唤醒方状态	提交边界
1	RUNNING，是 CPU0 current	receive queue 为空	尚未决定阻塞
2	wait node 已登记，state 改为 BLOCKED	对象锁仍由 T 持有	等待关系已经可被唤醒方观察
3	释放锁并切走，不在 runqueue	条件仍可能为空	T 已撤销运行资格
4	仍为 BLOCKED	数据先进入 receive queue	谓词变真，但 T 尚未 runnable
5	RUNNABLE，只进入一个目标 runqueue	等待节点被移除，记录唤醒原因	唤醒资格完成，不等于已经运行
6	被选为 current 后继续 read	重新加锁检查 receive queue	谓词再次为真时才消费数据

跨 CPU 迁移改变的是队列所有权

多核系统的每 CPU runqueue 降低了共同队列锁竞争，却使 task 的 CPU 归属成为显式状态。task T 从 CPU0 迁移到 CPU1 时，源候选集合、T.cpu 和目标候选集合必须作为一个受保护转换完成。若先插入目标再删除源，两个 CPU 可能同时选择 T；若先删除后失去引用，T 仍为 RUNNABLE 却不在任何队列。



图示：跨 CPU 迁移是候选关系的所有权转移，不是复制 task。迁移区间内必须同时保护源队列、目标队列和 T.cpu；完成后 T 只在 CPU1 的候选集合中。IPI 只要求目标 CPU 尽快重新考虑调度，不直接让 T 在远端开始执行。

核心思想

上下文切换保存的是“怎样继续这条执行流”，运行队列保存的是“这条执行流是否有资格取得 CPU”，等待队列保存的是“哪个条件变化后应重新授予资格”，地址空间对象保存的是“这条执行流的地址按什么映射解释”。四类状态相互连接，但不能用一个含混的“进程上下文”替代。

CPU 时间、等待时间与调度指标

调度器改变的是任务获得处理器的先后顺序。要评价这种顺序，必须先区分任务自身需要的处理器时间和排队造成的时间。设任务到达系统的时刻为 A ，第一次取得 CPU 的时刻为 F ，完成时刻为 C ，实际占用 CPU 的服务时间为 S ，因 I/O、锁或事件而阻塞的累计时间为 B ，则有：

$$\text{响应时间} = F - A,$$

$$\text{周转时间} = C - A,$$

$$\text{可运行等待时间} = (C - A) - S - B.$$

响应时间描述请求多久开始得到处理，周转时间描述多久完成，可运行等待时间只计算任务已经具备运行条件却仍在队列中等待的部分。任务因 I/O 阻塞的时间不应计入可运行等待，否则会把存储或网络延迟误判成 CPU 调度拥塞。在本节随后使用的纯 CPU 算例中 $B = 0$ ，公式相应简化为 $(C - A) - S$ 。

分析一段调度过程时，可以并列记录三类状态。进程记录列出 PID、父 PID、退出状态、地址空间和 fd 表引用；线程记录列出 TID、所属 PID、等待原因、优先级、最近运行 CPU 和已消耗时间；runqueue 则列出当前可运行线程的顺序和调度统计。

tick	事件	runqueue	运行线程	状态变化
0	创建 A/B	A,B	A	A running
1	时间片结束	B,A	B	A runnable, B running
2	B 发起 I/O	A	A	B blocked(io)
3	I/O 完成	A,B	A	B runnable
4	A exit	B	B	A zombie

这张表同时呈现状态、队列和事件。后续策略可以在此基础上增加虚拟运行时间、deadline、优先级、CPU affinity、NUMA node 或 cgroup quota。

用三个任务比较两种顺序。A、B、C 分别在时刻 0、1、2 到达，需要的 CPU 服务时间分别为 5、2、1。先到先服务会让 A 从时刻 0 连续运行到 5，随后运行 B 和 C：

任务	首次运行	完成	响应时间	周转时间
A	0	5	0	5
B	5	7	4	6
C	7	8	5	6

若使用长度为 2 的时间片，并规定同一时刻新到达的任务先进入队列，再把时间片到期的任务放到队尾，执行顺序为 A、B、C、A、A；对应时间区间是 $[0, 2)$ 、 $[2, 4)$ 、 $[4, 5)$ 、 $[5, 7)$ 、 $[7, 8)$ 。

任务	首次运行	完成	响应时间	周转时间
A	0	8	0	8
B	2	4	1	3
C	4	5	2	3

第二种顺序显著缩短 B、C 的响应与周转时间，却使 A 更晚完成。与先到先服务相比，它增加了重新调度检查，并至少多发生一次切换到其他任务的动作。由此可以得到一个重要结论：调度没有脱离工作负载的单一最优顺序。批处理系统可能优先考虑吞吐和平均周转时间，交互系统更关注响应时间和尾延迟，实时系统则要求在截止时间前获得确定的服务量。后续算法的差异，本质上是选择不同的目标并承担相应代价。

调度策略与评价

在讨论排序算法之前，还要说明调度器何时有机会重新选择任务。当前任务主动阻塞、退出或让出处理器时，内核自然重新取得控制；但一个持续计算的任务可以长期不执行这些动作。若只依靠任务自愿让出，系统无法保证其他任务获得 CPU。

处理器定时器提供了外部控制点。内核设置定时器后，当前任务即使一直计算，定时器到期仍会触发中断。入口保存当前现场，内核累计运行时间，并在需要时设置重新调度标志。到达允许调度的安全点后，当前 task 从 RUNNING 转为 RUNNABLE，调度器选择下一项任务。由此形成 抢占式调度：任务不必主动合作，系统也能限制一次连续占用 CPU 的时长。

重新调度原因	当前任务的状态变化	选择下一任务前还要处理的事情
阻塞等待	RUNNING → BLOCKED	登记等待条件并从可运行集合移除
主动让出	RUNNING → RUNNABLE	保存现场并重新入队
时间片到期	RUNNING → RUNNABLE	结算运行时间，更新策略所需的排序字段
更高优先级任务被唤醒	当前任务可能保持 RUNNING 到安全点	标记需要抢占，处理目标 CPU 通知
任务退出	RUNNING → EXITED	记录退出状态，移除调度实体并启动资源释放

FCFS：最简单也最容易产生长等待

先来最简单的调度算法：first-come, first-served。任务按进入 runnable 队列的顺序执行，一个任务运行到阻塞、退出或主动让出 CPU 后，才轮到下一个任务。

```
while true:
    task = runqueue.pop_front()
    run_until_block_or_exit(task)
    if task.state == RUNNABLE:
        runqueue.push_back(task)
```

FCFS 的实现成本很低：入队 $O(1)$ ，出队 $O(1)$ 。它的问题是 convoy effect。一个长 CPU 任务排在前面，后面的短交互任务会长时间等待。若系统只跑批处理，FCFS 可以作为 reference；若系统需要交互响应，它通常不合格。

Round-robin：用时间片限制独占

Round-robin 在 FCFS 上增加时间片。每个 runnable 任务最多运行一个 quantum，时间片到期后被放回队尾。它解决了“一个任务长期霸占 CPU”的问题，但引入了时间片选择：时间片太大，交互延迟高；时间片太小，上下文切换成本高。

```
on timer_tick:
    current.used_ticks += 1
    if current.used_ticks == current.quantum:
        current.used_ticks = 0
        current.state = RUNNABLE
        runqueue.push_back(current)
        current = runqueue.pop_front()
```

假设上下文切换成本是 S ，时间片是 Q ，CPU 时间被切换消耗的比例大约是 $S / (S + Q)$ 。 Q 越小，响应越好，但浪费越多。这个公式不是精确模型，却能解释为什么时间片不能无限小。

优先级调度和饥饿

优先级调度把 runqueue 拆成多个队列，总是选择最高优先级的 runnable 任务。它适合表达“控制任务比日志任务重要”，但如果高优先级任务持续可运行，低优先级任务可能永远得不到 CPU。

```
for priority from HIGH to LOW:
    if !queue[priority].empty():
        return queue[priority].pop_front()
return idle_task
```

解决饥饿的常见方法是 aging：等待越久，优先级逐步提高。aging 的实现要小心，不能每个 tick 扫描所有任务，否则任务多时成本过高。可以在任务入队时记录等待起点，在选择或周期性维护时计算有效优先级。

MLFQ：用反馈区分交互和批处理

Multilevel feedback queue 使用多个优先级队列，新任务先进入高优先级；如果任务用完整个时间片，说明它偏 CPU 密集，下次降级；如果任务很快阻塞或让出 CPU，说明它可能是交互任务，保留较高优先级。MLFQ 不需要提前知道任务类型，而是从行为反馈中调整。

```
on_task_created(task):
    task.level = 0
    queues[0].push_back(task)

on_quantum_expired(task):
    task.level = min(task.level + 1, MAX_LEVEL)
    queues[task.level].push_back(task)
```



```
on_task_blocked_before_quantum(task):
    queues[task.level].push_back_when_woken(task)
```

MLFQ 的关键参数包括层数、每层时间片、降级规则、周期性 boost。boost 用来防止低层任务永久饥饿：每隔一段时间，把所有 runnable 任务提升回高层。实现上要记录任务当前层级、已用时间、最近 boost epoch。

公平调度：从 CFS 模型到 EEVDF

公平调度不只问“谁先来”，而是问每个 runnable 任务已经得到多少 CPU 服务。经典 CFS 模型给每个任务维护虚拟运行时间 `vruntime`：任务实际运行越久，`vruntime` 越大；权重越高，虚拟时间增长越慢。下面的伪代码是用于理解 CFS 核心思想的简化模型，不是当前 Linux 调度器的完整实现。

```
on_task_run(task, delta_exec):
    task.vruntime += delta_exec * NICE_0_WEIGHT / task.weight

pick_fair(rbtree):
    return leftmost_node_by_vruntime(rbtree)
```

若用红黑树保存 runnable 任务，插入、删除大约是 $O(\log n)$ ，取缓存的最左节点可接近 $O(1)$ 。这个模型解释了为什么 nice 值不是简单固定优先级：它影响获得 CPU 的比例，而不是让低优先级任务永远不运行。

现代 Linux 的公平调度逐步采用 EEVDF 决策。EEVDF 仍使用虚拟时间核算服务量，但先用 lag 判断任务是否有资格运行，再在有资格的任务中选择虚拟截止期最早者。因此，应把“按最小 `vruntime` 选择”的基础模型与“资格判断加虚拟截止期”的完整决策分开。

deadline 调度与准入控制

实时或准实时任务还会声明运行时间和截止时间。例如任务说“每 20ms 需要 3ms CPU”。调度器若盲目接受所有请求，总利用率超过 100 后必然 miss deadline。因此 deadline 调度需要准入控制。

```
admit(task):
    new_util = total_runtime_over_period + task.runtime / task.period
    if new_util > CPU_CAPACITY:
        return -EBUSY
    add_to_deadline_class(task)
    return OK

pick_deadline():
    return task with earliest absolute_deadline
```

EDF 在单 CPU 理想模型中有很强性质，但真实系统还有抢占成本、临界区、缓存失效、中断关闭和共享资源阻塞。理论可调度不等于工程一定按时，测试必须记录 deadline miss、最大关抢占时间和锁等待。

策略评价的运行证据

调度器优化必须有指标。常用指标包括平均响应时间、p95/p99 延迟、吞吐、上下文切换次数、公平误差、迁移次数、cache miss、deadline miss 和能耗。只报告“更快”没有意义。

指标	说明	常见误判
on-CPU time	真正执行指令的时间	把等待 I/O 误认为算法慢

runnable wait	可运行但没拿到 CPU 的时间	把调度拥塞误认为锁死
context switches	切换次数和成本	只看次数，不看每次成本
migration count	跨 CPU 迁移	均衡改善但缓存局部性变差
fairness error	实得 CPU 与权重目标差距	平均公平掩盖尾延迟

通用系统的附加约束

真实通用 OS 还要处理多核、NUMA、CPU affinity、实时任务、cgroup 配额、负载均衡和能耗。若多个 CPU 共用一个全局队列，频繁入队和出队会产生锁竞争；每 CPU runqueue 能降低竞争，但需要负载均衡。迁移任务可以平衡负载，却会丢失 cache/TLB 热状态。调度器必须在公平、吞吐、延迟、局部性和能耗之间折中。

阻塞、唤醒与上下文切换

task 保存的执行状态

硬件正在执行的是寄存器、RIP、RSP、RFLAGS 和页表上下文。内核把这组执行状态包装成 task，再让 task 指向地址空间、文件表、信号状态、凭据、namespace 和 cgroup。task 是可调度实体，不是进程所有资源的简单大盒子。

字段类别	保存什么	为什么重要
执行现场	内核栈、保存寄存器、返回路径	被切走后还能从同一点继续
调度状态	state、policy、prio、vruntime、on_rq	判断能否运行、怎样排序、是否已入队
地址空间	mm、active_mm	进程切换可能要切页表和 TLB
资源指针	files、fs、signal、sighand	fork、exec、exit 要复制、共享或释放
安全身份	cred、namespace、cgroup	权限、限额和可见性都依赖它
亲和性	allowed CPUs、last CPU、NUMA 统计	负载均衡不能只看队列长度

state 和 on_rq 要分开理解。state 说明任务是 runnable、sleeping、stopped 还是 exiting；on_rq 说明它是否已经挂到某个 runqueue。一个 runnable 任务没有入队，会永远等不到 CPU；一个 sleeping 任务留在 runqueue，会被错误调度。调度器使用锁和状态检查，使任务语义与队列位置保持一致。

特权级转换、调度决定与上下文切换

系统调用、定时器中断和上下文切换经常同时出现，但它们表示三类不同动作。用户线程执行系统调用时，处理器从用户态进入内核态，当前 task 通常没有改变；定时器中断到来时，内核可能只完成计时后返回原 task；只有调度器最终选择了另一个 task，才发生任务上下文切换。

动作	发生了什么	不必同时发生什么
进入内核	保存用户返回现场，切换到受保护的内核入口与内核栈	不一定更换当前 task，也不一定运行调度器

作出调度决定	比较可运行实体，确定是否继续当前 task	若仍选择当前 task，可以不切换执行现场
任务上下文切换	保存 prev 的内核执行现场，恢复 next 的现场	若二者共享地址空间，不一定切换页表根
地址空间切换	装入 next 的页表上下文并处理必要的 TLB 语义	内核线程切换或同进程线程切换未必需要该动作

这种区分有助于解释成本来源。一次系统调用必然承担进入和离开内核的成本，但不必引起任务切换；一次任务切换会改变寄存器和内核栈，还可能因地址空间变化带来 TLB 与 cache 局部性损失。统计“系统调用次数”和“上下文切换次数”得到的是不同现象，不能相互替代。

阻塞系统调用怎样离开 CPU

阻塞不是“函数停住”。内核要把当前任务从可运行集合移走，把它挂到等待队列，并保证事件到达时能找回它。以等待 pipe 或 socket 可读为例：

```
blocking_read(obj):
    lock(obj.lock)
    while !data_available(obj):
        prepare_to_wait(obj.read_waitq, current)
        current.state = TASK_INTERRUPTIBLE
        unlock(obj.lock)
        schedule()
        lock(obj.lock)
        finish_wait(obj.read_waitq, current)
    if signal_pending(current):
        unlock(obj.lock)
        return -EINTR
    copy_data_to_user()
    unlock(obj.lock)
    return bytes
```

检查谓词和入等待队列必须受同一把锁保护。若先检查发现没有数据，还没入队时生产者写入并唤醒，唤醒会丢失；随后当前任务睡下，就形成 lost wakeup。谓词循环也不是模板写法：任务可能被信号唤醒，可能被虚假唤醒，也可能多个等待者竞争同一份数据。

把这个错误具体走一遍，问题会更清楚。假设线程 A 正在 `read(pipefd)`，线程 B 稍后 `write(pipefd)`。错误实现若按“先看 pipe 为空，再准备睡眠”两步分开做，中间没有锁保护，就会出现下面的时序：

时刻	线程 A	线程 B 或内核状态
1	A 检查 pipe，发现没有数据	wait queue 仍为空
2	A 还没把自己挂入 wait queue	B 写入数据，调用 wakeup，但没有等待者可唤醒
3	A 把自己设为 sleeping 并调用 <code>schedule</code>	pipe 已经有数据，但没有新的 wakeup 会来
4	CPU 去运行别的任务	A 可能永久睡眠，除非信号、超时或其他写入再次触发事件

正确实现把“检查条件、登记等待者、改变 task 状态”放在同一个临界区里。这样生产者要么在 A 入队前看见锁被持有，等 A 完成睡眠准备后再写入并唤醒；要

么在 A 检查条件前已经写入数据，A 重新检查时发现已有数据而不会睡下。这里的锁保护一个跨 CPU 不变量：条件已经满足时，等待者不得再进入无人能够唤醒的睡眠；等待者已经睡下时，改变条件的一方必须能够找到它。

还要注意，`schedule()` 返回不表示数据一定已经可用。A 可能因为信号或超时返回，也可能被广播唤醒后发现另一个线程先取得了数据。因此睡眠后必须重新获得锁并检查谓词。`wake_up` 只修改任务状态并将其放回 `runqueue`；任务何时继续执行仍由调度器决定，恢复执行后还要重新证明条件仍然成立。

唤醒怎样进入 `runqueue`

唤醒不等于立即运行。唤醒方通常只把等待任务改回 `runnable`，放到目标 CPU 的 `runqueue`；是否抢占当前任务，要看调度类、优先级、抢占状态和 CPU 位置。

```
wake_up(waitq):
    lock(waitq.lock)
    for task in waitq:
        if task.wait_condition_is_satisfied:
            remove_from_waitq(task)
            task.state = TASK_RUNNING
            enqueue_task(select_runqueue(task), task)
            if task_should_preempt_current(task):
                set_need_resched(target_cpu)
    unlock(waitq.lock)
```

若目标 CPU 正在用户态运行，`need_resched` 常在返回用户态、定时器中断返回或显式调度点处理。若目标 CPU 是另一个核心，内核可能发 IPI 让它尽快重新调度。这样看，`wakeup` 的成本包括等待队列锁、`runqueue` 锁、跨 CPU 通知和 `cache line` 迁移。

再用一个具体的 `socket` 接收案例串起来。线程 A 在 CPU0 上调用 `recv`，接收队列为空，于是它把等待原因记成“等 `socket receive queue` 非空”，挂到 `socket` 的等待队列，然后从 CPU0 的 `runqueue` 中消失。此时 A 没有“暂停在 CPU 里”，它只是一个 `task` 对象，保存了内核栈、寄存器现场、等待队列链接和状态。CPU0 可以去运行线程 C。

随后网卡在 CPU1 上产生中断。驱动或 NAPI poll 把包交给协议栈，协议栈找到目标 `socket`，把数据放进 `receive queue`，然后调用 `wakeup`。该操作把 A 从 `socket wait queue` 移除，改成 `runnable`，并选择一个目标 `runqueue`。若 A 上次在 CPU0 运行且 `cache/TLB` 仍有局部性，调度器可能倾向放回 CPU0；若 CPU0 很忙而 CPU1 空闲，也可能迁移。迁移会使 A 在恢复时承担 `cache` 重新填充的成本。

阶段	状态变化	必须区分的边界
<code>recv</code> 阻塞	task 从 <code>runqueue</code> 移到 <code>socket wait queue</code>	它不再竞争 CPU，但内核必须记住等待条件
包到达	数据进入 <code>socket receive queue</code>	条件变为真，但等待线程还没有运行
<code>wakeup</code>	task 变成 <code>runnable</code> ，进入某个 <code>runqueue</code>	<code>runnable</code> 只表示有资格运行，不表示已经运行
调度选择	CPU 在安全点切换到该 task	可能立即抢占，也可能等当前任务返回或时间片结束
重新检查	<code>recv</code> 继续后再次检查 <code>receive queue</code>	多等待者、信号和竞态要求谓词循环

这个案例也给出了停滞请求的分析方法。若线程仍在 `wait queue`，原因可能是事件未发生、唤醒路径遗漏或等待条件错误；若线程已经 `runnable` 但长时间没有

运行，原因在调度拥塞；若线程运行后再次睡眠，则应检查虚假唤醒、多个等待者竞争以及协议状态。不同状态对应不同的诊断方向。

抢占点和不可抢占区

定时器中断可以打断用户态程序，但内核不能在任意位置无条件切走当前任务。持有自旋锁、关闭中断、正在修改 runqueue、正在操作每 CPU 数据时，随意抢占会破坏不变量。内核用 `preempt_count`、中断状态和锁规则标记这些区域。

```
timer_interrupt:
    account_runtime(current)
    if current.timeslice_expired:
        set_need_resched(current)
    if returning_to_user and need_resched:
        schedule()
    else if kernel_preemptible and preempt_count == 0:
        schedule()
```

实时系统关心的不是平均时间片，而是最长不可抢占区、最长关中断区和最高优先级任务被低优先级持锁路径阻塞的时间。一个系统平均响应很好，也可能因为某条持锁路径过长而出现不可接受的尾延迟。

fork、exec、wait 和调度器的连接

`fork` 创建的新任务不能在半初始化状态进入 runqueue。内核必须先准备内核栈、保存的用户寄存器、调度实体、地址空间引用、文件表引用、信号状态和 PID，再让调度器看到它。子进程第一次运行时不是从程序开头执行，而是从系统调用返回路径回到用户态，只是返回值寄存器被设置为 0。

```
fork scheduling boundary:
    allocate child task
    prepare child saved registers
    copy or share mm/files/signal/cred
    allocate pid and link parent-child relation
    enqueue child as runnable
    parent returns child pid
    child later returns 0 from copied frame
```

`exec` 成功后仍是同一个 task 被调度，但地址空间、用户栈、入口 RIP、部分信号处理和 close-on-exec fd 改变。`wait` 要求已退出子进程保留 zombie 壳，直到父进程读取退出码。调度器只负责不再运行 zombie；进程管理还要保证退出状态只被回收一次，最后一个引用释放 task。

Linux 实现中的关键对象

Linux 的进程创建以 `copy_process` 为中心，退出与回收由 `do_exit`、`release_task` 等过程协作完成；调度主干由 `__schedule` 和 `try_to_wake_up` 推进，公平调度类维护任务的入队、出队、虚拟运行时间和负载均衡。理解这些实现时，应把动作区分为四类：修改 task 状态、修改 runqueue、调整资源引用、通知等待者。

调度状态与并发控制的连接

调度过程还与共享状态的并发控制直接相连。只要两个线程可能同时访问同一份可变状态，就必须说明同步对象、等待对象和失败恢复。锁、futex、信号、IPC 和死锁将在后文继续展开，本章先建立统一问题：线程因何不能继续，它等待哪个条件，谁有资格改变条件，以及条件改变后谁负责唤醒。

机制	它保存的等待事实	常见错误
mutex	持有者是谁，等待者排在哪个队列	忘记在循环中复查条件，或异常路径漏解锁

futex	用户态字值和内核等待队列的绑定	只看内核队列, 不验证用户态谓词是否仍成立
condition variable	等待哪个共享状态变真	把 wakeup 当成状态已经满足
signal	异步事件和可中断睡眠的交汇点	忽略 EINTR, 把部分完成误当失败
pipe/socket	缓冲区从空到非空、从满到非满	读写端关闭后仍然等待不可能发生的事件

锁的规则也必须落到调度状态。自旋锁适合短临界区, 持锁时不能睡眠; mutex 可睡眠, 适合较长临界区; 读写锁提高读多写少场景的并发, 但写者饥饿和升级死锁要单独处理; RCU 让读侧几乎不阻塞, 但释放对象必须等待 grace period, 不能在最后一个指针消失时立即 free。区分这些机制, 需要考察持有期间能否调度、等待者位于何处以及对象何时可以真正释放。

```
safe_wait(lock, waitq, condition):
    lock(lock)
    while !condition():
        prepare_to_wait(waitq, current)
        unlock(lock)
        schedule()
        lock(lock)
    finish_wait(waitq, current)
    unlock(lock)
```

这段伪代码压缩了等待路径的核心不变量: 检查条件和入队必须受同一把锁保护; 睡眠前要释放保护业务状态的锁; 醒来后要重新加锁并复查条件; 退出等待时要清理 wait queue 链接。若先检查条件再入队, 中间事件可能已经发生并唤醒了空队列, 线程随后睡下就永远没人叫醒。若醒来后不复查, 多等待者会把同一个资源消费两次。

死锁分析也要用对象图, 不要只说“锁顺序错了”。把每个线程正在持有的锁、正在等待的锁、是否可被信号打断、是否持有不可睡眠锁都列出来。如果图中出现环, 而且环上没有超时、取消或外部释放路径, 系统就无法自行前进。lockdep 一类工具本质上是在运行时收集锁类顺序, 提前发现可能形成环的获取顺序。

验收标准

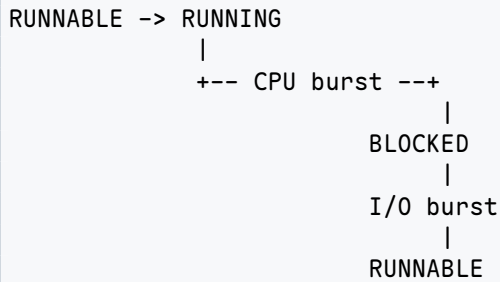
调度与并发章的最低验收: 给任一阻塞点, 都能写出等待谓词、等待队列、唤醒者、是否可中断、被唤醒后的复查动作、持锁规则和失败清理路径。缺一项, 代码即使在正常路径能跑, 也没有可证明的并发语义。

调度案例: 从执行轨迹到策略选择

前面已经说明了常见调度策略的基本规则。要进一步理解这些规则, 还需要把任务的到达、运行、阻塞和完成放在同一条时间线上。调度器观察不到“一个程序总共要运行多久”, 它只能看到已经到达的 task、过去消耗的时间以及当前是否可运行。这一限制决定了理论最优算法与可实现算法之间的差距。

CPU burst 与 I/O burst

线程从开始到结束通常不会连续占用 CPU。它先执行一段指令, 随后等待文件、网络、锁或定时器; 等待条件满足后, 再执行下一段指令。两次阻塞之间连续占用处理器的区间称为 CPU burst, 阻塞等待外部条件的区间称为 I/O burst。这里的 I/O 是广义说法, 也包括锁等待和定时等待。



这个划分解释了“CPU 密集型”和“I/O 密集型”的准确含义。CPU 密集型任务通常具有较长的 CPU burst，很少主动阻塞；I/O 密集型或交互任务的 CPU burst 较短，但会频繁等待外部事件。两者都可能运行很长时间，差异在于占用 CPU 与离开 CPU 的节奏。

工作负载	典型执行节奏	调度时需要关注的目标
批量计算	少量长 CPU burst	吞吐、平均周转时间、cache 局部性
交互请求	多次短 CPU burst 与网络等待交替	首次响应、p95/p99 延迟、唤醒抢占
后台写回	周期性被唤醒，处理一批脏页后再睡眠	对前台任务的干扰、I/O 与 CPU 节奏
实时控制	周期到达，每次需要有上界的 CPU 时间	截止期限、最坏阻塞时间、准入控制

调度器只应在 runnable task 之间分配 CPU。线程处于 I/O burst 时已经离开 runqueue，它的等待时间由设备、锁持有者或定时器决定。把 blocked task 也计入调度候选，会混淆“没有处理器时间”和“当前根本不能继续执行”这两种状态。

已知服务时间时的理论基准

先采用一个现实中通常无法满足、但便于建立基准的假设：系统预先知道每个任务还需要多少 CPU 时间。A、B、C 同时在时刻 0 到达，服务时间分别为 8、4、2。若按 A、B、C 的到达顺序执行，时间线为：

```

time: 0          8          12   14
      |----- A -----|--- B ---| C |
  
```

三个任务的等待时间分别为 0、8、12，平均等待时间为 $20/3$ ；周转时间分别为 8、12、14，平均周转时间为 $34/3$ 。长任务 A 位于队首，使两个短任务都要等待，这就是 convoy effect 在时间指标上的表现。

如果先运行服务时间最短的 C，再运行 B 和 A，时间线变为：

```

time: 0   2       6          14
      | C |--- B ---|----- A -----|
  
```

顺序	平均等待时间	平均周转时间	直接代价
A、B、C	$20/3$	$34/3$	短任务排在长任务之后
C、B、A	$8/3$	$22/3$	必须预先知道或准确估计服务时间

在所有任务同时到达、任务之间独立且服务时间已知的条件下，shortest job first (SJF) 能最小化平均等待时间。证明思路可以从交换相邻任务得到。若两个

相邻任务 X 、 Y 的服务时间满足 $S_X > S_Y$ ，先运行 X 会使 Y 多等待 S_X ；交换后， X 只多等待 S_Y 。由于 $S_Y < S_X$ ，交换能降低总等待时间。不断消除这样的逆序，最终得到按服务时间递增的顺序。

这个结论有严格的适用前提。它没有保证每个任务都公平，也没有处理任务陆续到达和服务时间未知的情况。一个持续到来的短任务流可能使长任务长期等待，因此真实系统还要引入 aging、公平核算或最大延迟约束。

任务陆续到达时为何需要抢占

现在令 A 在时刻 0 到达，需要 8 个单位； B 在时刻 1 到达，需要 4 个单位； C 在时刻 2 到达，需要 2 个单位。若 SJF 不允许抢占， A 一旦开始便运行到时刻 8， B 和 C 仍然要等待。若每当新任务到达时比较“剩余服务时间”，则得到 shortest remaining time first (SRTF)：

```
time: 0 1 2   4       7           14
      |A|B| C |-- B --|----- A -----|
```

任务	到达	完成	周转时间	等待时间
A	0	14	14	$14 - 8 = 6$
B	1	7	6	$6 - 4 = 2$
C	2	4	2	$2 - 2 = 0$

时刻 1， A 还剩 7 个单位，而新到达的 B 只需 4 个，因此 B 抢占 A ；时刻 2， B 还剩 3 个单位，而 C 只需 2 个，因此 C 再次抢占。每次决定都基于当前已知任务的剩余时间。

抢占并非没有成本。内核要进入调度路径、保存和恢复现场，工作集也可能离开 cache。若新任务只比当前任务短很少，抢占节省的等待时间可能小于切换造成的损失。工程实现通常会设置最小运行粒度、唤醒抢占阈值或延迟容忍度，避免在相近候选者之间频繁切换。

未来服务时间为何只能估计

普通程序不会在每个 CPU burst 开始前声明准确长度。服务时间还会受到输入数据、cache 命中、分支路径、锁竞争和缺页的影响。因此，SJF/SRTF 更适合作为评价基准，实际调度器只能根据过去行为预测未来。

一种基础预测方法是指数平滑。设第 n 次实际 CPU burst 为 t_n ，此前预测为 τ_n ，则下一次预测为

$$\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n, \quad 0 \leq \alpha \leq 1.$$

α 越大，最近一次 burst 的影响越强； α 越小，历史平均越稳定。假设初始预测为 6， $\alpha = 0.5$ ，接下来实际观察到 8 和 2，则预测依次变为

$$\tau_1 = 0.5 \times 8 + 0.5 \times 6 = 7, \quad \tau_2 = 0.5 \times 2 + 0.5 \times 7 = 4.5.$$

预测能识别长期偏交互或偏计算的行为，却无法保证下一次 burst 与过去相似。程序阶段变化时，旧历史还会造成滞后。MLFQ 采用的“用完整时间片就降低队列级别、很快阻塞则保留较高级别”，本质上也是一种基于近期行为的在线分类。

时间片同时控制响应与开销

Round-robin 不预测任务长度，而是为每个 runnable task 提供最长为 Q 的连续运行机会。若有 N 个始终可运行的同权任务，忽略中断和切换成本，某任务再次得到 CPU 的最长队列等待约为 $(N - 1)Q$ 。减小 Q 能缩短等待上界，但会增加调度频率。

设每次实际任务切换的平均直接成本为 S 。在任务每次都用满时间片的近似条件下，切换开销占比约为

$$\rho = \frac{S}{Q + S}.$$

若 $S = 5\mu s$ ，当 $Q = 10ms$ 时， $\rho \approx 0.05\%$ ；若把时间片缩短为 $50\mu s$ ， $\rho \approx 9.1\%$ 。后者还没有计入 cache 和 TLB 局部性损失。时间片不能仅根据“越短响应越快”来选择。

时间片范围	主要收益	主要风险
相对较长	切换较少，计算任务保持局部性	runnable task 的等待上界增大
相对较短	交互任务更快获得第一次服务	调度、中断和 cache 恢复成本上升
按任务动态调整	可结合延迟目标和任务权重	状态与策略更复杂，参数错误会产生抖动

队列边界也必须定义清楚。同一时刻可能同时发生时间片到期、新任务到达和阻塞任务被唤醒。实现需要规定这些事件的入队次序，否则同一算例可能得到不同结果。多核系统还要决定任务进入哪个 CPU 的 runqueue；因此，“轮转”不仅是一条 FIFO 规则，还包括计时、事件排序和放置策略。

公平、优先级与期限的不同承诺

调度策略经常同时使用“公平”“优先级”“期限”三个词。它们对应不同承诺：公平关心较长时间窗口内的服务比例，优先级关心当前谁应先获得 CPU，期限关心工作能否在指定时刻前完成。把三者混为一个数值，会使策略无法解释。

加权公平如何换算为服务比例

设两个始终可运行的任务 A、B 的权重分别为 1024 和 512。若一个 30ms 的观察窗口全部用于这两个任务，理想服务量分别为

$$30ms \times \frac{1024}{1024 + 512} = 20ms, \quad 30ms \times \frac{512}{1024 + 512} = 10ms.$$

高权重表示在竞争时获得更大比例，并不表示低权重任务永远不能运行。公平调度器需要记录“已经得到多少加权服务”，以便选择相对欠缺服务的任务。虚拟运行时间可以写成

$$\Delta v = \Delta t \times \frac{W_0}{w},$$

其中 Δt 是实际运行时间， w 是任务权重， W_0 是基准权重。A 运行 20ms、B 运行 10ms 后，若 $W_0 = 1024$ ，两者的虚拟增量都为 20ms。虚拟时间相同意味着二者按照各自权重获得了相称服务。

任务	权重	实际运行	虚拟运行增量
A	1024	20ms	$20 \times 1024 / 1024 = 20$
B	512	10ms	$10 \times 1024 / 512 = 20$

公平只在存在竞争时分配比例。如果 A 睡眠而 B 独自可运行，让 B 占满 CPU 不违反公平；空闲处理器没有必要为尚未就绪的 A 保留时间。A 被唤醒后如何放置虚拟时间则需要限制：若把它放到所有任务之前，频繁睡眠可以获得不受限的抢占

优势；若完全忽略睡眠期间的等待，交互任务又会出现明显延迟。调度器需要在长期份额与唤醒响应之间设置边界。

EEVDF 增加了资格与虚拟期限

只选择最小虚拟运行时间，主要表达“谁得到的加权服务较少”。当任务具有不同延迟需求时，还需要表达某项工作希望多快得到下一段服务。EEVDF 将决策分为两步：

1. 根据任务相对于理想服务的 lag，判断它当前是否有资格运行；
2. 在有资格的任务中，选择虚拟截止期最早的任务。

lag 描述任务实际获得的服务与理想份额之间的差额。获得服务不足的任务具有正的补偿需求，获得过多的任务需要暂时等待。虚拟截止期在公平份额之上表达延迟目标；较短的请求区间会产生较早的虚拟期限，从而更快得到处理。

资格检查很重要。若只比较 deadline，一个已经超额获得 CPU 的任务可能通过不断提交短请求长期占先。先限制 eligibility，再比较 deadline，使低延迟选择不能脱离公平核算。这里的 deadline 是调度器虚拟时间中的排序字段，不等同于实时任务必须满足的物理时刻。

固定优先级为何会产生反转

设高优先级任务 H 需要一把 mutex，低优先级任务 L 已经持有该 mutex，中优先级任务 M 与这把锁无关。若 H 到达后阻塞在 L 持有的锁上，而 M 持续抢占 L，就会出现以下关系：

```
L holds mutex
H waits for mutex held by L
M preempts L because M has higher priority than L
```

表面上是 M 的优先级低于 H，实际却因为 H 依赖 L，M 延迟了 L，也间接延迟了 H。高优先级任务的完成时间被低优先级持锁者控制，这称为优先级反转。

阶段	未采用继承时	采用优先级继承时
H 请求 mutex	H 阻塞，等待 L	H 阻塞，并把有效优先级传给 L
M 变为 runnable	M 抢占 L	L 以继承后的优先级继续运行
L 释放 mutex	释放时刻可能被 M 长期推迟	L 尽快完成临界区并唤醒 H
释放之后	H 才有机会继续	L 恢复原优先级，H 按高优先级运行

优先级继承不能消除临界区本身的执行时间，但能限制无关中优先级任务造成的额外阻塞。若锁形成嵌套依赖，继承还要沿等待链传播。实时分析因此必须同时知道调度优先级、锁依赖图和每个临界区的最坏执行时间。

实时期限需要准入与超支处理

周期任务通常用三项参数描述：周期 T 、每周期最多需要的运行时间 C 、相对截止期 D 。当 $D = T$ 时，一个任务的处理器利用率为 $U = C/T$ 。例如三项任务分别为

$$(C, T) = (2, 10), (1, 5), (4, 20),$$

总利用率为

$$U = \frac{2}{10} + \frac{1}{5} + \frac{4}{20} = 0.6.$$

利用率小于 1 只说明总服务需求没有超过单 CPU 的理论容量。共享锁、不可抢占区、中断和切换成本仍会占用时间；若截止期短于周期，还要使用更完整的可调度性分析。因此，准入控制必须保留安全余量，并计入系统开销与已知阻塞上界。

任务还可能超过声明的运行预算。若不限限制超支，一项异常任务可以破坏所有已准入任务的期限。系统通常需要在预算耗尽时进行限流、降低调度等级或报告 deadline miss。准入、运行时核算和超支处理共同构成期限承诺，仅在入队时按 deadline 排序并不足够。

多核调度：处理器位置也是调度结果

单核调度只选择“运行谁”；多核调度还要选择“在哪里运行”。同一时刻多个 CPU 会独立取任务、接收唤醒和处理定时器事件。若所有核心共同修改一个全局 runqueue，正确性较直观，但队列锁会成为频繁竞争的共享 cache line。

从全局队列到每 CPU runqueue

每 CPU runqueue 使多数选择操作只访问本地状态。CPU0 调度时无需取得 CPU1 的队列锁，不同核心可以并行选择任务。代价是负载可能不均：CPU0 的队列中有多个 runnable task，CPU1 却可能进入 idle。

组织方式	主要收益	必须额外解决的问题
单一全局 runqueue	所有 CPU 看到同一候选集合，天然均衡	锁竞争、cache line 迁移、选择结构扩展性
每 CPU runqueue	本地入队与选择可以并行，保持局部性	任务放置、负载均衡、跨 CPU 唤醒与迁移

由此可见，每 CPU 队列没有消除全局调度问题，而是把连续的全局竞争转化为周期性或按需的协调。系统需要在“少迁移以保持局部性”和“及时迁移以消除空闲”之间取得平衡。

新建与唤醒任务放到哪个 CPU

任务第一次进入 runnable 状态时，内核需要从允许集合中选择目标 CPU。允许集合首先受 affinity、cpuset、安全隔离和 CPU online 状态约束。在剩余候选中，还要考虑队列负载、处理器容量、cache 拓扑和任务上次运行位置。

被唤醒的任务通常在原 CPU 上保留热 cache、TLB 和 NUMA 内存局部性，因此优先返回原 CPU 可能降低恢复成本。但若原 CPU 正在运行重要任务而另一个 CPU 空闲，保留局部性会增加 runnable wait。唤醒放置需要比较两类成本，而不能只选择队列最短者。

放置依据	倾向保留原 CPU 的原因	倾向迁移的原因
运行队列	避免跨队列加锁和通知	原 CPU 的 runnable wait 明显更高
cache/TLB	上次工作集可能仍在本地	空闲 CPU 可立即执行任务
NUMA	内存页靠近原 CPU 所在节点	任务已经迁移内存或远端等待过长
CPU capacity	原核心可能性能更适合当前任务	高容量核心被更紧急任务占用
能耗	避免唤醒额外核心	集中负载可能触发降频或热限制

异构处理器进一步使“一个 runnable task 等于一份相同负载”的假设失效。高性能核心和节能核心单位时间能完成的工作不同，任务还可能受到热设计功耗和频率上限影响。负载比较需要按 CPU capacity 归一化，并结合任务利用率估计。

迁移必须是原子的队列转移

任务从 CPU0 迁移到 CPU1 时，不能先在目标队列插入、稍后再从源队列删除，否则两个 CPU 可能同时选择同一个 task；也不能先解除所有归属、再无保护地插入，否则并发唤醒可能使任务丢失。迁移过程至少要保持以下顺序：

1. 按固定次序取得源 runqueue 与目标 runqueue 的锁；
2. 确认 task 仍位于源队列，状态仍为 runnable，且没有正在某个 CPU 上执行；
3. 从源队列删除 task，更新 CPU 归属和迁移统计；
4. 将 task 插入目标队列，必要时通知目标 CPU 重新调度；
5. 释放两端队列锁。

若边界处理错误	可能出现的状态	外部表现
先入目标、后出源	task 同时位于两个 runqueue	同一内核栈被两个 CPU 恢复，状态遭到破坏
先出源、丢失引用	runnable task 不在任何队列	线程永久得不到 CPU
双队列锁顺序不一致	CPU0 等 CPU1 的锁，CPU1 等 CPU0 的锁	调度路径死锁
迁移 running task	task 仍在源 CPU 执行又被目标选择	两个 current 指向同一执行现场

这里的核心不变量仍然是“一个 runnable task 只属于一个 runqueue”，只是多核环境允许多个 CPU 同时修改相关状态，因此状态检查、队列转移和归属更新必须成为一个受保护的整体。

主动均衡与空闲拉取

负载均衡可以由繁忙 CPU 主动把任务推走，也可以由即将 idle 的 CPU 从其他队列拉取任务。主动推送能较早缓解长队列，但需要扫描其他 CPU 并承担跨核通知；空闲拉取只在确有空闲容量时发生，却可能让任务先经历一段不必要的等待。

触发方式	适用时机	需要避免的问题
周期均衡	定期检查相邻调度域的负载差	周期过短导致迁移抖动，过长导致空闲与排队并存
idle balance	CPU 即将无任务可运行	扫描成本超过即将获得的运行收益
wakeup balance	blocked task 重新变为 runnable	每次唤醒都跨核迁移，破坏 cache 局部性
overload push	实时或 deadline 队列出现过载	把不可迁移或亲和性受限任务错误移出

调度域按硬件拓扑分层。系统通常先在共享核心或共享 LLC 的近邻间均衡，再考虑同一 NUMA 节点，最后才跨节点或跨 socket。距离越远，可利用的空闲 CPU 越多，迁移带来的 cache 与内存局部性代价也越高。

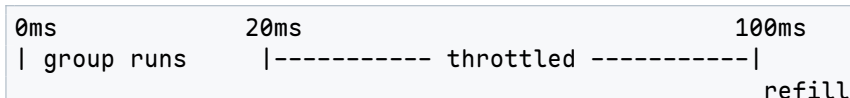
SMT 线程并不等同于完整核心

同时多线程(SMT)的两个逻辑 CPU 共享部分执行资源。若两个计算密集任务同时放在同一物理核心的 sibling 上，它们可能争用执行端口和 cache；把其中一个迁到独立物理核心通常能获得更高吞吐。另一方面，当系统只剩一个空闲 SMT sibling 时，让任务运行仍可能优于继续排队。

调度器因此需要理解拓扑层级，不能仅按逻辑 CPU 数量平均任务。安全隔离还可能要求互不信任的工作负载不共享 SMT 核心，这会进一步缩小允许放置集合。性能、公平和隔离在此处共同影响任务位置。

cgroup 配额为何会产生周期性延迟

设一个服务组的配额为每 100ms 可使用 20ms CPU。组内四个线程在周期开始时同时变为 runnable，并在 20ms 内耗尽共享预算。即使宿主机随后还有空闲 CPU，这四个线程仍可能被 throttle 到下一个周期：



这项限制针对整个 group 的累计运行时间，不是给每个线程各 20ms。若请求在 21ms 到达，它可能等待接近 79ms 才能继续，因此形成接近 quota period 的尾延迟尖峰。缩短 period 可以降低最长等待，却增加配额核算和补充频率；增加 quota 则会改变租户之间的资源隔离。

层级 cgroup 还要求同时满足祖先和当前组的预算。子组仍有余额时，若父组预算已经耗尽，子组也不能运行。解释容器中的 CPU 延迟时，需要沿层级检查配额、实际消耗和 throttled 时间，单看线程优先级无法得到完整原因。

进程与线程生命周期的完整边界

调度器选择的是 task，但 task 引用的进程资源有各自生命周期。创建、exec、线程退出和进程回收会改变这些引用关系。若只维护调度状态而忽略资源可见性，新 task 可能访问尚未初始化的对象，退出 task 也可能留下无法回收或过早释放的资源。

创建 task 时先构造，后发布

创建新 task 可以分为私有构造阶段和公开发布阶段。构造阶段分配 task 与内核栈，准备用户返回现场，复制或共享地址空间、文件表、凭据和信号状态；这些步骤尚未完成时，新对象只能由创建路径访问。发布阶段分配可见标识、连接父子关系，并把 task 置为 runnable。

阶段	可见范围	失败时的处理
分配 task 与栈	仅创建路径持有局部引用	释放已经分配的内存
建立资源引用	尚未进入 PID 查找和 runqueue	按逆序撤销 mm、files、cred 等引用
建立进程关系	父进程与管理结构开始可见	在锁保护下解除关系和标识
进入 runqueue	任意 CPU 可能立即选择该 task	此后不能再依赖“子任务尚未运行”的假设

“最后入队”是发布边界。若 task 提前进入 runqueue，另一个 CPU 可能在文件表、地址空间或返回寄存器尚未准备好时执行它。发布之后再补字段不能保证安全，因为调度器没有义务等待创建者完成剩余工作。

共享选择决定线程与进程的边界

创建接口可以针对不同资源选择“复制”或“共享”。共享地址空间使两个 task 成为同一进程中的线程；共享文件表会使一方关闭 fd 直接影响另一方；共享信号处理表则使处理动作属于共同资源。线程边界由一组共享关系共同形成，不能仅由一个名称标志决定。

资源	复制后的语义	共享后的语义
地址空间	修改私有映射通常不影响对方	任意线程的普通内存写对其他线程可见
文件描述符表	关闭本方 fd 槽不删除对方槽位	关闭共享槽位会影响所有引用该表的线程
文件对象	不同 fd 表仍可引用同一打开文件对象	offset、状态标志按对象规则共同变化
信号处理配置	后续修改各自独立	处理函数配置属于线程组共同状态
凭据	后续身份变化按各自对象处理	必须满足系统对线程组身份一致性的约束

即使共享地址空间，每个线程仍需要独立用户栈和内核栈。若两个线程使用同一栈，它们的函数调用、局部变量和返回地址会互相覆盖；若共享一套寄存器现场，调度器也无法分别暂停和恢复它们。因此，“共享进程资源”和“保留独立执行现场”必须同时成立。

多线程进程中的 exec

exec 的语义是用新程序替换当前进程的用户地址空间。单线程时，只需准备新映射和初始用户现场，再在提交点替换旧地址空间。多线程时，其他线程可能正在旧地址空间中执行；若它们在替换之后继续运行，保存的 RIP、RSP 和用户指针将指向已经失效的映射。

因此，多线程 exec 需要先使调用线程成为唯一能够继续的执行流，等待或终止同一线程组中的其他线程，再提交新地址空间。提交点之前失败，旧程序应尽量继续运行并收到错误；提交点之后，旧地址空间已经不可恢复，控制流只能进入新程序或终止。

阶段	仍可返回旧程序吗	需要保持的条件
读取并验证参数	可以	旧 mm、旧用户栈和调用线程仍有效
打开并检查可执行文件	可以	尚未改变线程组的公开执行状态
构造新 mm 与用户栈	通常可以	新对象尚未成为当前地址空间
处理其他线程	进入敏感过渡阶段	不能再允许其他线程恢复旧用户现场
提交新 mm	不再返回旧程序	当前 RIP、RSP、凭据和 fd 规则与新程序一致

exec 保留 PID 并不表示所有进程状态都保留。地址空间和用户栈被替换，设置 close-on-exec 的 fd 被关闭，信号处理配置按接口规则重置，凭据可能根据可执行文件发生受控转换。每类资源都有独立规则，不能用“进程没变”或“进程全换了”概括。

线程退出与进程退出

一个线程退出时，它自己的用户栈映射、内核栈、寄存器现场和调度实体可以进入释放流程，但同一进程中的其他线程仍可能引用共享地址空间和文件表。只有最后一个相关引用消失时，共享对象才可以释放。

```
thread exit:
```



```

stop scheduling this task
release per-thread state
drop references to shared process objects
if other threads remain:
    keep mm/files/process identity alive
else:
    begin process-wide exit and parent notification

```

进程级退出还要关闭文件引用、解除地址空间、记录退出码并通知父进程。这里存在两个不同的完成点：task 已经不再运行，以及父进程已经读取退出状态并完成回收。前者保证执行停止，后者决定最后的进程记录何时释放。

zombie 保存的是尚未交付的退出结果

子进程停止执行后，父进程仍可能需要退出码、终止原因和资源使用统计。若内核立即删除全部记录，wait 将无法区分“子进程尚未退出”“已经退出但结果丢失”和“从来不是自己的子进程”。因此系统保留一个不再可调度的 zombie 记录。

状态	仍然保留的内容	已经释放或禁止的内容
RUNNING/RUNNABLE	完整执行现场和资源引用	无
EXITING	退出码、清理进度、必要引用	不再接受新的普通工作
ZOMBIE	PID、父子关系、退出码、必要统计	用户地址空间、普通执行资源；不能再次调度
REAPED	不再保留可查询的子进程结果	最后进程记录与 PID 可进入复用流程

父进程调用 wait 后，退出结果完成一次性交付，zombie 才转为 reaped。若父进程提前退出，系统必须把仍需回收的子进程转交给指定的接管者，确保退出结果最终有人消费。zombie 的存在是父子进程协议的一部分，不是仍在运行的“空进程”。

PID、task 引用与编号复用

PID 是查找进程的标识，不等同于对象自身。进程被回收后，有限编号空间允许复用 PID。若内核或管理程序只保存一个整数，稍后再按该整数查找，可能得到新一代进程。需要跨时间稳定引用时，还应保存对象引用、创建代次或专用进程句柄。

这一问题与设备 request tag 的复用相似：编号只在特定生命周期内唯一。对象进入回收前，所有能够到达它的异步路径都必须完成或放弃引用；编号可以复用之后，迟到事件不能再作用于新对象。

本章不变量与综合推演

至此，可以用一个包含创建、调度、阻塞、迁移和退出的场景检查本章全部边界。父进程 P 在 CPU0 创建子进程 C；C 第一次进入 runqueue 后在 CPU1 运行，读取 socket 时阻塞；数据到达后 C 被唤醒到 CPU0，运行一个时间片后迁移回 CPU1，最终退出并由 P 回收。

阶段	主要状态变化	必须保持的不变量
创建	C 从私有构造对象变为 RUNNABLE	所有必要资源已初始化，C 只进入一个 runqueue
首次调度	C 从 RUNNABLE 变为 CPU1 的 current	C 已从 runqueue 移除，不在其他 CPU 执行
阻塞读取	C 从 RUNNING 变为 BLOCKED	等待谓词与 wait queue 已登记，C 不再占 CPU

网络唤醒	C 从 BLOCKED 变为 RUNNABLE	唤醒原因已记录，只进入一个目标 runqueue
抢占	C 的现场保存后重新入队	状态与 on_rq 一致，共享进程资源引用不变
迁移	C 从 CPU0 队列原子转移到 CPU1 队列	迁移期间不会同时属于两队列，也不会丢失
退出	C 停止调度并形成退出结果	地址空间与文件引用按引用计数释放，退出码仍可读取
回收	P 取得退出结果，C 进入 REAPED	最后引用消失后才释放记录并允许 PID 复用

本章的基础验收可以归纳为以下各项：

- 进程资源边界与线程执行现场能够分别说明，不能把程序文件、进程和 task 混为一体。
- RUNNABLE task 必须位于且只位于一个 runqueue，BLOCKED 和 ZOMBIE task 不得被选择。
- 阻塞前完成谓词检查和等待登记，唤醒只授予运行资格，恢复后仍要重新检查条件。
- 抢占、系统调用入口和上下文切换分别记录，不能用其中一个事件代替另一个事件的证据。
- 多核迁移保持源队列、目标队列和 task 归属的一致，不允许重复执行或丢失 runnable task。
- 公平份额、固定优先级和实时期限按各自承诺评价，锁阻塞与运行预算同时纳入分析。
- 新 task 完整初始化后才能发布；线程退出与进程回收按共享引用和父子协议分阶段完成。

这些不变量不依赖某一种具体调度算法。FCFS、round-robin、MLFQ、公平调度或 deadline 调度可以改变候选 task 的排序，却不能改变状态、队列、执行现场和资源生命周期的基本关系。

进程与调度的实现细节：从 task 到 switch

前面的章节已经讲了调度算法。本节把进程和调度器按内核实现拆开：进程对象有哪些字段，fork 如何复制资源，exec 如何替换地址空间，wait 如何回收子进程，调度器如何维护 runqueue，睡眠唤醒如何避免丢事件，上下文切换到底切了什么。

进程对象之间的引用关系。

Linux 中常用 `task_struct` 表示可调度实体。它不把所有资源都内嵌进去，而是引用地址空间、文件表、信号处理、凭据、命名空间、cgroup、调度实体等对象。这样线程可以共享一部分资源，fork 可以复制或共享不同对象，exec 可以替换地址空间但保留 PID 和部分文件描述符。

对象	回答的问题	生命周期事件
<code>task_struct</code>	这个可调度实体是谁、处于什么状态	fork 创建、exit 释放

<code>mm_struct</code>	用户地址空间是什么	fork COW、exec 替换、exit 释放
<code>files_struct</code>	fd 表指向哪些打开文件	fork 共享或复制、exec close-on-exec
<code>fs_struct</code>	cwd、root、umask 等进程文件系统上下文	chdir/chroot/fork
<code>sighand</code>	信号处理函数和 mask	fork/exec/signal
<code>cred</code>	uid、gid、capability、安全标签	setuid、exec、LSM 检查
<code>sched_entity</code>	调度时间与队列状态	enqueue、dequeue、account runtime

分析进程生命周期时要明确引用计数和共享边界：线程共享 `mm` 和 `files` 时，一个线程关闭 fd 会影响同进程其他线程；最后一个引用消失时，相应对象才进入释放阶段。

fork 的实现顺序。

fork 创建新的 task，并按照资源类型决定复制或共享，随后建立父子关系，把子任务放入 runnable 集合，最后向父子返回不同的值。若中间失败，必须按已经完成的步骤逆序回滚。

```
fork sketch:
  allocate task_struct and kernel stack
  copy scheduler attributes
  copy or share mm_struct with COW PTEs
  copy or share files_struct
  copy signal and credential state
  assign pid
  link into parent children list
  make child runnable
  return child pid to parent, 0 to child
```

这里的关键边界是新任务何时对调度器可见。所有必要状态初始化完成前，子任务不能进入 runqueue，否则另一个 CPU 可能立刻调度它并访问尚未就绪的对象。父子关系也要在锁保护下建立，否则 wait 路径可能遗漏子进程。

fork 的 COW 页表细节。

fork 复制页表时，匿名可写页通常不复制物理页，而是父子 PTE 都改成只读并标记 COW。物理页引用计数增加。任一方写入时，触发写保护缺页，内核复制新页，再把写入方 PTE 改为可写。

```
fork COW:
  for each writable anonymous PTE:
    clear write bit in parent PTE
    install read-only PTE in child
    mark both as COW
    increment page refcount
  flush parent TLB entries if permissions were tightened

write fault:
  if page refcount == 1:
    make PTE writable
  else:
    allocate new page
    copy old page contents
    decrement old page refcount
```

```
map new page writable
```

注意父进程 PTE 也要收紧权限，否则父进程写入不会 fault，就会直接修改共享页，破坏 fork 语义。权限收紧后还要处理 TLB，否则 CPU 可能继续使用旧的 writable 翻译。

exec 的关键承诺。

exec 成功后，当前进程身份大体保留，但用户地址空间被新程序替换。PID 不变，部分 fd 保留，设置了 close-on-exec 的 fd 关闭，信号处理按规则重置，凭据可能因 setuid 或文件 capability 改变。

```
execve sketch:
  copy argv/envp from user memory
  open executable and check permissions
  identify binary format
  create new mm_struct
  map program segments
  map interpreter if dynamically linked
  create user stack with argc/argv/envp/auxv
  apply credential transitions
  commit new mm atomically
  start at entry point
```

exec 失败要小心。理想情况下，在 commit 新地址空间前失败，旧程序仍能收到错误返回并继续运行；一旦已经不可逆地销毁旧地址空间，就只能终止进程。成熟内核会尽量把可能失败的检查和分配放在 commit 之前。

用户栈的构造。

新程序启动时不是 C 的 main 被内核直接调用。内核把初始用户栈构造成 ABI 规定的布局，包括 argc、argv 指针数组、envp 指针数组、auxiliary vector 和字符串数据。入口点通常是动态链接器或 _start。

```
initial user stack high to low:
  strings for argv and envp
  alignment padding
  auxiliary vector entries
  NULL
  envp pointers
  NULL
  argv pointers
  argc
```

auxv 给用户态传递页大小、随机数地址、程序头信息、vDSO 入口等。动态链接器用这些信息装载共享库、处理重定位和准备 libc。OS 的 exec 不是“把文件读到内存跳过去”这么简单，它要建立完整 ABI 环境。

wait 和 zombie 的意义。

子进程退出后不能立刻完全消失，因为父进程还可能调用 wait 获取退出状态和资源使用信息。于是退出进程进入 zombie 状态，保留 PID、退出码和少量统计；父进程 wait 后，内核释放最后对象。

```
exit:
  close files
  release mm
  record exit_code
  notify parent
  become ZOMBIE
  schedule away forever

wait:
```

```

find child in ZOMBIE state
copy exit status to parent
unlink child from parent list
free final task reference

```

若父进程不 wait, zombie 会留下。若父进程先退出, 子进程会被 reparent 给 init 或 subreaper。这里的设计保证退出状态不丢, 同时避免已退出任务再次运行。

runqueue 的最小不变量。

每 CPU runqueue 至少要维护当前任务、runnable 任务集合、锁、调度时钟和统计。一个任务在同一时刻只能处于一个位置: 某个 CPU 的 current, 某个 runqueue, 某个等待队列, 或者退出路径。

验收标准

- RUNNING 任务必须等于某个 CPU 的 current。
- RUNNABLE 任务必须在且只在一个 runqueue 中。
- SLEEPING 任务不能留在 runqueue 中。
- ZOMBIE 任务不能被调度器选中。
- 迁移任务时, 源 runqueue 和目标 runqueue 的锁顺序必须固定。
- 改变任务状态和队列位置必须在同一临界区内完成。

许多调度错误来自不变量遭到破坏, 而非候选选择算法本身: 任务重复入队, 睡眠任务未出队, 唤醒时状态已经变化, 或者迁移时两个 CPU 同时观察到同一任务。

CFS 的核心数据结构。

公平调度类用 sched_entity 记录虚拟运行时间、权重、统计和树节点。runqueue 中的 CFS 队列按 vruntime 排序, 最左边是当前最“亏”的任务, 优先运行。

```

CFS account:
delta_exec = now - curr.exec_start
weighted = delta_exec * NICE_0_LOAD / curr.weight
curr.vruntime += weighted
update rq.min_vruntime

CFS pick:
next = leftmost entity in rb_tree
return task_of(next)

```

min_vruntime 是队列的基准。新唤醒任务不能简单设置为 0, 否则会长期占便宜; 也不能设置得太大, 否则交互响应差。实际实现会把新实体放在相对 min_vruntime 合理的位置, 并配合唤醒抢占规则。

红黑树为什么合适。

公平调度需要频繁插入、删除和取最小 vruntime。普通链表插入简单但查找最小慢; 堆取最小快, 但删除任意 sleeping 任务不方便; 红黑树能在 $O(\log n)$ 完成插入、删除和按 key 排序。

结构	优点	缺点
FIFO 队列	$O(1)$, 简单	不能表达公平权重和 vruntime 排序
优先级数组	固定优先级快	动态公平排序不自然
堆	取最小快	删除任意实体复杂

红黑树

排序、插入、删除平衡

常数成本高于队列

数据结构的选择取决于所需操作。任务会频繁睡眠、唤醒、迁移和改变权重，调度器必须以可接受的成本支持这些变化。

睡眠唤醒的无丢失顺序。

等待队列正确性的核心是同一把锁保护“条件检查”和“入队睡眠”。否则唤醒可能发生在检查之后、入队之前，等待者永远睡下去。

```
correct wait:
    lock(obj.lock)
    while !condition:
        prepare_to_wait(waitq, current, SLEEPING)
        unlock(obj.lock)
        schedule()
        lock(obj.lock)
    finish_wait(waitq, current)
    unlock(obj.lock)

waker:
    lock(obj.lock)
    change condition
    wake_up(waitq)
    unlock(obj.lock)
```

很多真实内核代码会用更复杂的 helper，但它们都在维护这个原则。条件变量、pipe、socket、futex、completion、wait event 都逃不开同一条规则。

上下文切换切的是什么。

上下文切换分成调度器层和架构层。调度器选择 next，更新 current、统计、状态；架构层保存 callee-saved 寄存器、栈指针和必要控制状态，切到 next 的内核栈，再返回到 next 之前停下的位置。

```
schedule:
    prev = current
    next = pick_next_task(rq)
    if prev != next:
        rq.current = next
        context_switch(prev, next)

context_switch:
    switch address space if needed
    switch kernel stack
    save prev callee-saved registers
    restore next callee-saved registers
    return as next task
```

`context_switch` 的返回语义需要特别说明：函数返回时，当前执行流已经属于 next 任务。prev 将来再次被调度时，会从它原先离开的位置继续返回。因此，上下文切换在控制流表面仍表现为一次函数返回，返回前后的执行主体却已经改变。

地址空间切换和 lazy TLB。

若两个线程共享同一个 `mm_struct`，切换时不需要换页表根。若切到不同进程，通常要切换页表根或地址空间标识。切换页表会影响 TLB；x86-64 PCID 可以减少全量刷新。

```
switch_mm(prev_mm, next_mm):
    if prev_mm == next_mm:
        keep current page table root
```



```
else:
    load next page table root with PCID when enabled
    ensure stale translations cannot be used
```

内核线程可能没有自己的用户地址空间，常借用上一个进程的 `mm` 或使用特殊内核地址空间。`lazy TLB` 这类优化的目标，是减少无意义的页表切换和 `TLB` 刷新。

负载均衡和调度域。

多核系统具有层次结构。`SMT sibling`、同一核心、同一 `LLC`、同一 `NUMA node`、跨 `socket` 的迁移成本不同。调度器通常建立调度域，从近到远进行负载均衡。

```
load balance levels:
    SMT siblings
    cores sharing LLC
    CPUs in same NUMA node
    CPUs across NUMA nodes
```

迁移一个任务可能减少 `runnable wait`，却损失 `cache` 热状态并增加远端内存访问。`x86-64` 服务器和工作站上，调度器要估计 `SMT sibling`、共享 `LLC`、`NUMA node`、跨 `socket` 迁移、`turbo` 预算和功耗封顶。任务放错位置会让锁竞争、`cache miss`、远端内存访问和尾延迟一起变差。

cgroup CPU 配额。

容器和服务隔离需要限制一组任务的 `CPU` 使用。`cgroup CPU controller` 可以按周期给 `group` 分配 `runtime budget`。预算耗尽后，该组任务被 `throttled`，直到下个周期补充预算。

```
cgroup quota:
    period = 100ms
    quota = 20ms
    tasks in group may run total 20ms per period
    after budget exhausted:
        throttle group
    at next period:
        refill runtime and unthrottle
```

这会产生一种典型现象：容器内 `CPU` 使用率尚未达到一个完整核心，请求却出现周期性延迟尖峰，因为 `group` 已被配额限流。性能分析必须结合 `cgroup throttling` 统计，宿主机总体 `CPU` 使用率不足以解释该现象。

调度实现测试。

- `fork` 后子任务不能在初始化完成前进入 `runqueue`。
- `COW fork` 后父子任一方写入都不能修改另一方可见内容。
- `exec` 失败路径不得破坏旧程序继续运行的必要状态。
- `wait` 必须能回收 `zombie`，并且不能回收非子进程。
- 睡眠和唤醒并发时不能 `lost wakeup`。
- 迁移任务时不能同时出现在两个 `CPU` 的 `runqueue`。
- `cgroup quota` 耗尽时 `group` 被 `throttle`，周期补充后能恢复。
- 上下文切换后 `callee-saved` 寄存器、栈和地址空间必须正确。

从教学调度器到可验证的多核实现

调度与并发深讲：从“能跑”到“可解释地推进”

调度器和同步原语经常被分开讲，但真实系统中二者紧密绑定。线程等待锁会离开 CPU；I/O completion 会唤醒线程；定时器会打断当前线程；优先级和锁持有时间会相互影响；关闭路径要唤醒所有等待者。若只学调度算法，不学等待语义，就无法解释“线程为什么不运行”。

线程不运行的六种原因。

用户看到“程序卡住”，内核可能处在完全不同的状态。

原因	内核状态	诊断方向
等 CPU	RUNNABLE 但不在 CPU 上	runnable wait、runqueue 长度、优先级
等锁	SLEEPING 在 mutex/futex 等待队列	持锁者、锁顺序、临界区长度
等 I/O	SLEEPING 在 completion 等待队列	块层队列、设备延迟、writeback
等页面	page fault 或内存回收	major fault、swap、page cache miss
等事件	poll、epoll、timer、条件变量	等待谓词、关闭路径、超时
已退出	ZOMBIE 或 TERMINATED	父进程是否 wait、资源是否释放

这张表比“调度器选择下一个任务”更重要。调度器只能从 runnable 集合里选任务；一个 sleeping 任务不运行，通常不是调度策略问题，而是等待条件没有满足或唤醒丢失。

完整的阻塞读路径。

以 pipe 的阻塞读为例。读者发现缓冲区为空，不能忙等，否则浪费 CPU。它必须把自己放入等待队列，改变状态为 sleeping，然后调度走。写者写入数据后，改变条件并唤醒读者。

```

pipe_read(pipe, buf, len):
    lock(pipe.lock)
    while pipe.empty and not pipe.write_closed:
        current.state = SLEEPING
        enqueue(pipe.read_waitq, current)
        schedule_unlocking(pipe.lock)
        lock(pipe.lock)
    if pipe.empty and pipe.write_closed:
        unlock(pipe.lock)
        return 0
    n = copy_from_pipe(pipe, buf, len)
    wake_waiters(pipe.write_waitq)
    unlock(pipe.lock)
    return n

pipe_write(pipe, buf, len):
    lock(pipe.lock)
    while pipe.full and not pipe.read_closed:
        sleep_on(pipe.write_waitq, pipe.lock)
    if pipe.read_closed:
        unlock(pipe.lock)
        return -EPIPE
    n = copy_to_pipe(pipe, buf, len)
    wake_waiters(pipe.read_waitq)
    unlock(pipe.lock)

```

```
return n
```

这里多个边界：读端关闭要唤醒写者，写端关闭要唤醒读者；读者被信号打断时要返回 `EINTR` 或部分结果；非阻塞 `fd` 应返回 `EAGAIN` 而不是睡眠。一个合格 OS 模型必须把这些边界都写出来。

优先级反转。

优先级反转是并发与调度相互影响的典型问题。低优先级线程 `L` 持有锁，高优先级线程 `H` 等这个锁，中优先级线程 `M` 持续运行，导致 `L` 得不到 CPU 释放锁，`H` 虽然优先级高却无法推进。

```
L: lock(A), do work slowly
H: tries lock(A), blocks
M: CPU-bound, runnable

without inheritance:
  scheduler runs M before L
  H waits for L indirectly

with priority inheritance:
  L temporarily inherits H priority
  scheduler runs L so it can release A
```

priority inheritance 不是让锁更快，而是让等待关系反馈给调度器。实现要维护“谁在等谁”的依赖图，释放锁时撤销继承。若多把锁嵌套，继承链可能跨多个线程，测试必须覆盖链式等待。

死锁调试的步骤。

死锁不是“程序没输出”。它是等待图出现环。调试时先列出每个线程持有的资源和等待的资源。

```
Thread A: holds inode.lock, waits page.lock
Thread B: holds page.lock, waits inode.lock

wait-for graph:
  A -> page.lock -> B
  B -> inode.lock -> A
```

解决死锁的常用手段是全局锁顺序、trylock 回退、缩短临界区、拆开锁或使用 RCU。不要用“加更多锁”解决死锁，更多锁通常只会增加等待图边数。正确方法是减少共享状态、明确所有权和规定顺序。

调度和同步练习。

1. 给三个任务 `A/B/C`，分别是 CPU 密集、周期 I/O、交互输入，手算 FCFS、RR、MLFQ 的等待时间。
2. 构造一个 lost wakeup 的时序，并说明哪一把锁应同时保护等待谓词和等待队列。
3. 画出 priority inversion 的等待图，写出 priority inheritance 后每个线程的临时优先级。
4. 为一个有界队列写关闭协议：生产者关闭、消费者关闭、双端同时关闭分别怎样唤醒。
5. 设计一个测试，证明 blocked 线程永远不会被调度器选中。

调度算法对比：同一负载下的不同结论。

考虑三个任务：A 是长 CPU 任务，需要 30 个 tick；B 是交互任务，每运行 1 tick 就等待 I/O 5 tick；C 是短任务，只需要 3 tick。不同调度策略会给出完全不同的用户体验。

策略	表现	问题
FCFS	若 A 先到，C 和 B 长时间等待	convoy effect，交互体验差
RR	A 不能长期霸占 CPU，B/C 能穿插运行	时间片过小会增加切换成本
优先级	B 可以更快响应输入	低优先级任务可能饥饿
MLFQ	通过行为反馈照顾交互任务	参数复杂，可能被任务行为利用
公平调度	长期 CPU 份额接近权重	响应延迟需要额外 wakeup/preempt 规则
EDF	deadline 最近者先运行	需要准入控制，过载时仍会 miss

这说明没有“唯一正确”的调度算法。调度器服务于目标：批处理吞吐、桌面交互、实时 deadline、容器公平、能耗控制或尾延迟。教材中算法比较的目的，不是背哪个更先进，而是理解每个算法优化了什么、牺牲了什么。

关闭路径是并发程序的必修课。

很多同步教材只讲生产者消费者的正常路径，却不讲关闭。真实 OS 对象一定会关闭：pipe 端点关闭、socket 断开、进程退出、设备拔出、文件释放。关闭路径的核心规则是：改变关闭状态后，必须唤醒所有可能等待这个状态的线程。

```

queue_close(q):
    lock(q.lock)
    q.closed = true
    wake_all(q.not_empty)
    wake_all(q.not_full)
    unlock(q.lock)

queue_pop(q):
    lock(q.lock)
    while q.empty and not q.closed:
        sleep(q.not_empty, q.lock)
    if q.empty and q.closed:
        unlock(q.lock)
        return CLOSED
    item = pop(q)
    wake_one(q.not_full)
    unlock(q.lock)
    return item

```

若关闭只唤醒消费者，不唤醒生产者，正在等待空间的生产者可能永远睡眠。若关闭状态不受同一把锁保护，等待者可能错过关闭通知。关闭路径是检验并发模型是否完整的最好办法。

常见误区。

常见误区

- “线程没运行就是调度器不公平”：它可能根本不在 runnable 集合里。
- “加锁就安全”：锁必须保护正确对象，并且所有路径都遵守同一锁规则。

- “notify 等于发送消息”：notify 只唤醒等待者，条件仍要重新检查。
- “自旋锁比 mutex 快”：临界区短且不能睡眠时自旋才合适，长等待会浪费 CPU。
- “测试跑过一次没有死锁”：死锁是时序问题，需要锁顺序和等待图分析。

从时间片轮转到反馈队列。

Round Robin 的优点是简单：每个 runnable 任务轮流运行一个时间片。它解决了 FCFS 中长任务霸占 CPU 的问题，但它无法区分交互任务和批处理任务。交互任务经常运行很短时间就阻塞等待 I/O，如果它每次被唤醒后还要排在长 CPU 任务后面，用户会感觉系统迟钝。

多级反馈队列 (MLFQ) 的核心是用任务行为估计任务类型：刚创建或刚被唤醒的任务放在较高优先级；任务用完整个时间片，说明偏 CPU 密集，降低优先级；任务很快阻塞，说明偏交互或 I/O 密集，保持或提升优先级；周期性提升所有任务，防止饥饿。

```
on_tick(current):
    current.slice_used += 1
    if current.slice_used == quantum[current.level]:
        demote(current)
        enqueue(current)
        schedule()

on_sleep(current):
    current.sleep_start = now

on_wakeup(task):
    if now - task.sleep_start >= interactive_threshold:
        promote(task)
        enqueue(task)
```

MLFQ 的难点不是队列，而是参数。时间片太短，上下文切换成本高；时间片太长，交互延迟差；提升太频繁，CPU 任务占便宜；提升太少，低优先级任务饥饿。工业调度器通常不会直接照搬教科书 MLFQ，而是把类似思想融入更复杂的权重、虚拟运行时间和唤醒抢占规则。

公平调度的虚拟运行时间。

公平调度不承诺每次都平均，而承诺长期 CPU 份额接近权重。一个常见思路是维护每个任务的虚拟运行时间 `vruntime`。任务实际运行越久，`vruntime` 越大；权重越高，`vruntime` 增长越慢。调度器选择 `vruntime` 最小的任务，让落后的任务追上来。

```
account_runtime(task, delta_exec):
    task.vruntime += delta_exec * NICE_0_WEIGHT / task.weight

pick_next(rq):
    return leftmost_task_by_vruntime(rq.tree)
```

若用平衡树保存 runnable 任务，选择最小 `vruntime` 是 $O(\log n)$ 插入删除加 $O(1)$ 或 $O(\log n)$ 取最小，具体取决于实现是否缓存最左节点。这个算法牺牲了极简实现，换来权重公平和较平滑的延迟。它仍然需要补充唤醒抢占、最小时间粒度、CPU affinity 和负载均衡，否则单靠 `vruntime` 无法覆盖完整系统需求。

实时调度：可调度性比算法名字重要。

实时系统关注 deadline。固定优先级调度中，Rate Monotonic 把周期短的任务设为更高优先级；动态优先级调度中，EDF 选择绝对 deadline 最近的任务。它们

都不能解决过载：如果任务最坏执行时间总和超过 CPU 能力，deadline miss 只是时间问题。

算法	需要的输入	失败模式
RM	周期、最坏执行时间、固定优先级	高优先级任务过多时低优先级 miss
EDF	绝对 deadline、剩余执行时间	过载时大量任务连续 miss
CBS/预算调度	runtime budget、period、deadline	预算耗尽后任务被限速

实时任务必须接受准入控制。准入控制回答“把这个任务加入后，已有保证是否还能成立”。没有准入控制，所谓实时优先级只是让某些任务更早失败。测试也必须记录 deadline miss、最大响应时间和抢占延迟，而不是只证明任务最终运行过。

futex：用户态快路径和内核等待队列。

互斥锁若每次加锁解锁都进入内核，成本很高。futex 的思想是：无竞争时完全在用户态用原子操作完成；只有竞争时才进入内核，把线程挂到与某个用户地址关联的等待队列上。

```
mutex_lock(m):
    if atomic_compare_exchange(m.state, 0, 1):
        return
    while true:
        old = atomic_exchange(m.state, 2)
        if old == 0:
            return
        futex_wait(&m.state, expected=2)

mutex_unlock(m):
    old = atomic_exchange(m.state, 0)
    if old == 2:
        futex_wake(&m.state, 1)
```

关键是 `futex_wait` 必须原子地检查用户地址仍等于 `expected`，再入队睡眠。否则 `unlock` 可能在检查和入队之间发生，造成 `lost wakeup`。futex 展示了一个重要原则：OS 不需要接管所有同步操作，它只需要在用户态快路径失败时提供可靠睡眠和唤醒。

信号、取消和可中断睡眠。

阻塞调用不能只考虑“等到事件发生”。进程可能收到信号，线程可能被取消，文件描述符可能被关闭，设备可能拔出。内核等待点要定义是否可中断，以及中断后返回什么。

```
wait_event_interruptible(q, condition):
    lock(q.lock)
    while !condition:
        if signal_pending(current):
            unlock(q.lock)
            return -EINTR
        sleep(q, q.lock)
    unlock(q.lock)
    return 0
```

可中断等待提高响应性，但让调用者必须处理部分完成。例如 `read` 已复制 100 字节后收到信号，通常应返回 100，而不是丢弃已完成工作；若 0 字节完成才被信号打断，才返回 `EINTR`。这种细节决定系统调用是否可组合。

并发对象的状态机写法。

讲同步不能只写“加锁”。每个共享对象都应该有状态机。以有限队列为例，状态包括 open、closing、closed；计数包括 item 数和等待者；事件包括 push、pop、close producer、close consumer、timeout、signal。

当前状态	事件	动作
open, empty	pop blocking	消费者入 not_empty 等待队列
open, full	push blocking	生产者入 not_full 等待队列
open	close producer	标记 no_more_push, 唤醒消费者
open	close consumer	标记 no_more_pop, 唤醒生产者
closed, empty	pop	返回 EOF/CLOSED
closed/full	push	返回 EPIPE/CLOSED

状态机有两个好处。第一，它迫使设计者写出关闭路径和错误路径。第二，它直接变成测试矩阵：每个状态和事件组合至少有一个测试或证明。

同步算法复杂度和适用边界。

不同同步原语服务不同场景。自旋锁避免睡眠切换，适合极短临界区和中断上下文；mutex 允许睡眠，适合可能等待较久的临界区；读写锁适合读多写少，但写者饥饿要处理；RCU 让读者几乎无锁，代价是更新路径复杂且对象释放要等 grace period。

原语	适合	不适合
spinlock	短临界区、不能睡眠上下文	I/O、缺页、可能阻塞路径
mutex	普通内核对象互斥	中断上下文、自旋锁内层
rwlock	读多写少且临界区短	写频繁或读者长期持有
semaphore	资源计数与限流	表达复杂对象不变量
RCU	读多、遍历多、更新少	需要立即释放或强一致更新

原语选择错误会直接变成性能或正确性问题。把长 I/O 放在自旋锁内，会浪费 CPU 甚至死锁；用读写锁保护写多对象，会让写者互相等待且读者无收益；用 RCU 保护需要立即撤销权限的对象，可能留下过期访问窗口。

并发测试：从随机到可复现。

并发测试不能只靠压力跑。压力测试能暴露问题，但失败时如果没有事件日志和可控调度，很难复现。更可靠的方法是把关键同步点做成可插桩事件，枚举或随机探索不同交错。

```
test_lost_wakeup():
    pause_after(reader, "checked_empty")
    writer.write(byte)
    writer.wake()
    resume(reader)
    assert(reader.eventually_returns(byte))
```

高质量并发测试至少记录：线程 id、对象 id、锁 acquire/release、状态改变、入队/出队、睡眠/唤醒、错误返回和超时。测试失败时要能画出等待图或事件序列。

否则“偶现卡住”会变成无法定位的黑盒。

源码级补充：task_struct 与 Linux 调度类。

Linux 的调度对象不是一个抽象“线程”词条，而是 `task_struct` 加上调度类私有实体。读 `include/linux/sched.h`、`kernel/sched/core.c`、`kernel/sched/fair.c` 时，先找这些字段：`pid/tgid`、`state`、`flags`、`mm`、`active_mm`、`files`、`signal`、`sighand`、`ptrace`、`cgroups`、`cpus_mask`、`se`、`rt`、`dl`。

字段或对象	作用
<code>task_struct.state</code>	runnable、interruptible sleep、uninterruptible sleep 等状态
<code>mm/active_mm</code>	用户地址空间和内核线程借用地址空间
<code>files</code>	fd table, fork/clone 时按标志共享或复制
<code>signal/sighand</code>	进程级信号状态和 handler 表
<code>sched_entity se</code>	CFS 的 vruntime、权重、红黑树节点
<code>cpus_mask</code>	CPU affinity, 限制可运行 CPU 集合
<code>cgroups</code>	CPU、memory、io 等资源控制归属

进程创建读 `kernel/fork.c` 的 `copy_process`：它根据 `CLONE_*` 标志决定共享还是复制 `mm`、`files`、`sighand`、thread group。进程退出读 `kernel/exit.c` 的 `do_exit`、`exit_notify` 和 `wait` 路径：退出不是立即删除 task，而是留下 zombie 供父进程读取退出状态。

fork、clone、vfork 和线程。

`fork` 复制进程抽象，地址空间用 COW；`clone` 用标志精确控制共享哪些对象；`vfork` 暂时共享地址空间并挂起父进程直到子进程 `exec/exit`；NPTL 线程通常是共享 `mm`、`files`、`sighand` 的 `clone` 任务。TLS、`set_tid_address`、robust futex list 都和线程退出清理有关。

```
clone flags for pthread-like thread:
CLONE_VM      share address space
CLONE_FS      share fs context
CLONE_FILES   share fd table
CLONE_SIGHAND share signal handlers
CLONE_THREAD  same thread group
CLONE_SETTLS  set thread-local storage base
```

这解释了为什么“进程”和“线程”在内核里不是两种完全不同对象，而是同一种 task 在资源共享标志上的不同组合。

上下文切换：switch_mm、FPU 和 TLB。

`__schedule` 选择 next 后，底层切换要保存/恢复 CPU 上下文、切换内核栈、切换地址空间、处理 lazy FPU 状态。若 prev 和 next 属于同一 mm，可能避免完整地址空间切换；若不同 mm，需要更新页表基址并处理 TLB/PCID。

```
context_switch(prev, next):
    prepare_task_switch(prev, next)
    if prev.mm != next.mm:
        switch_mm(prev.mm, next.mm)
    switch_to(prev, next) // registers and stack
    finish_task_switch(prev)
```

上下文切换成本不只是保存寄存器。cache/TLB 热状态丢失、跨 CPU 迁移、FPU/向量寄存器保存和安全缓解措施都会影响成本。因此调度器要在公平、延迟和

局部性之间折中。

调度类顺序。

Linux 调度类有优先顺序：stop、deadline、realtime、fair、idle。核心调度路径从高到低询问各类是否有可运行任务。

```
pick_next_task(rq):
    for class in [stop, dl, rt, fair, idle]:
        p = class.pick_next_task(rq)
        if p != null:
            return p
```

stop class 用于极高优先级的 CPU 控制任务；deadline 处理 `SCHED_DEADLINE`；rt 处理 `SCHED_FIFO/RR`；fair 处理普通任务；idle 在无事可做时运行。若实时任务配置错误，普通 CFS 任务会长期得不到 CPU，所以实时调度必须配合权限和 runtime 限制。

CFS、红黑树和唤醒抢占。

CFS 用 `vruntime` 表示任务已获得的加权 CPU 服务，`runqueue` 用红黑树按 `vruntime` 排序，选择最左任务。关键参数包括 `sched_latency`、`min_granularity`、`wakeup_preempt` 规则和 `task weight`。

```
enqueue_task_fair(rq, p):
    update_curr(rq)
    place_entity(p.se)
    rb_insert_by_vruntime(rq.cfs_tasks, p.se)

pick_next_task_fair(rq):
    return rb_leftmost(rq.cfs_tasks)
```

唤醒抢占不是简单“新任务来了就抢”。它比较新任务的 `vruntime`、交互延迟、当前任务运行时间和最小粒度。过度抢占会提高响应但增加上下文切换；抢占太少会让交互任务尾延迟变差。

cgroup、NUMA balancing 与能源调度。

cgroup CPU controller 用权重、配额和周期限制任务组 CPU 时间。NUMA balancing 通过采样页访问和迁移任务/页面改善内存局部性。能源感知调度 EAS 和 `schedutil` 把任务放置、CPU capacity、频率调节和能耗模型联系起来。

机制	调度影响
CPU quota	组用完周期预算后 throttled，即使有 runnable 任务也不能运行
CPU weight	多组竞争时按权重分配长期份额
CPU affinity	限制任务可运行 CPU，可能改善局部性也可能造成热点
NUMA balancing	在任务迁移和页面迁移之间寻找更低远端访问成本
EAS/schedutil	结合 CPU capacity 和频率，平衡性能与能耗

诊断“调度不公平”时必须看 cgroup throttling、affinity、rt/deadline 任务、NUMA 迁移和 CPU 频率。只看 `runqueue` 长度会漏掉资源控制和硬件拓扑。

第二阶段深讲：进程生命周期不是 fork/exec/wait 三个词

进程章节容易被讲成系统调用列表：`fork` 创建，`exec` 装载，`wait` 回收，`exit` 退出。这样的讲法太薄。真实内核维护的是一组互相引用的对象：任务、地址空间、

文件表、信号状态、凭据、命名空间、调度实体、父子关系和资源统计。生命周期问题的核心是：什么时候对象对其他 CPU 可见，什么时候能失败，失败后怎样回滚，最后一个引用在哪里释放。

从硬件执行流到 task。

硬件只知道当前 CPU 正在执行一组寄存器、RIP、栈和页表根。操作系统把这组状态包装成 task。task 不等于进程的全部资源；它是可调度实体，指向更大的资源集合。

资源	为什么不直接内嵌进 task	生命周期问题
地址空间	线程可共享，exec 可替换	引用计数、COW、TLB shutdown
文件表	fork 可共享，dup 共享 open file	close-on-exec、并发 close/read
信号处理	进程级和线程级状态交织	handler、mask、pending queue
凭据	安全检查频繁读取，变更较少	RCU 发布、setuid/exec 转换
命名空间	容器内多个进程共享视图	引用计数、权限边界
调度实体	每个线程都要被调度	runqueue、vruntime、CPU affinity

这种“task 指向对象”的结构让共享和替换成为可能。同一进程内多个线程共享 mm 和 files；exec 替换 mm，但保留 PID 和部分 fd；fork 复制 task，同时复制或共享不同对象。若把所有资源平铺在一个大结构里，这些语义会变得混乱。

创建进程的真正顺序。

创建进程不能先把半成品放进 runqueue。只要任务进入 runnable 集合，另一个 CPU 就可能立刻运行它。因此 fork 的顺序必须从“不可见半成品”逐步推进到“对调度器可见”。

```
copy_process outline:
  allocate task and kernel stack
  initialize minimal scheduler state
  copy credentials or install new references
  copy/share files, fs, signal state
  copy mm with COW page tables if needed
  allocate pid
  link child into parent relationships
  publish task to pid tables
  make task runnable only after invariants hold
```

每一步都有失败可能：内存不足、pid 不足、文件表复制失败、页表复制失败、权限检查失败。错误路径必须按逆序释放已经拿到的引用。这里的质量不在正常路径跑通，而在第 5 步失败时前 4 步是否完整回滚。

阶段	成功后新增什么	失败时必须回滚什么
分配 task	task 内存和内核栈	释放栈和 task 对象
复制 files	fd 表引用增加	put 每个 open file 引用
复制 mm	页表页、PTE 引用、COW 标记	释放页表页、撤销引用
分配 pid	pid 映射占用	释放 pid，不能留下僵尸映射

链接父子	父进程 child list 可见	从列表摘除,避免 wait 看到半成品
入队运行	调度器可选择任务	之后失败通常不能简单回滚,要走退出路径

这张表解释了为什么内核代码里有大量 `goto bad_*`。这些标签表达的是阶段化回滚：从哪个阶段失败，就只回滚已经成功取得的资源。高质量内核代码宁愿显式写出阶段，也不能让半初始化对象泄漏到全局结构。

fork 后父子为什么返回不同值。

`fork` 的一个经典语义是父进程返回子 PID，子进程返回 0。硬件没有“两个返回值”的指令，内核通过修改子任务保存的寄存器帧实现这个效果。父进程系统调用正常返回时，返回值寄存器写入 child pid；子任务第一次被调度时，从复制出的内核返回路径继续，返回值寄存器已经被设置为 0。

```
parent path:
    ret = child.pid
    return_to_user(parent_regs with ret)

child prepared frame:
    child_regs = copy(parent_regs)
    child_regs.return_value = 0
    when scheduled, return_to_user(child_regs)
```

这个细节能帮助读者理解上下文切换：子进程不是从 `main` 开头重新执行，也不是内核调用了两次用户函数。它从和父进程相同的用户 PC 返回，只是寄存器状态略有不同。

exec 是不可逆提交点。

`execve` 成功后，旧程序映像消失，新程序从入口点运行。难点在于失败语义：在提交点之前失败，应返回错误给旧程序；提交点之后失败，旧程序已经没有完整地址空间可恢复，只能终止任务。

```
exec phases:
    copy argv/envp from old user memory
    open executable and check permissions
    parse binary format
    allocate new mm
    map text/data/interpreter/stack
    prepare credentials and auxv
    commit new mm and credentials
    jump to new entry
```

因此 `exec` 要尽量把可能失败的分配、权限检查和格式解析放在 `commit` 前。`commit` 动作要原子地切换 `mm`、设置新用户栈和入口、处理 `close-on-exec` fd、重置信号处理规则、应用 `setuid/capability` 变化。这个点之后，内核不能再假装旧程序还在。

线程创建和资源共享边界。

线程和进程的差别不是“更轻量”四个字。线程共享地址空间、文件表和许多进程级状态，但拥有独立的可调度上下文、用户栈、内核栈、TLS、信号 `mask` 和调度状态。共享使通信便宜，也让错误传播更广。

资源	进程间 fork 后	同进程线程间
地址空间	初始 COW，之后可分离	共享同一 <code>mm</code>

fd 表	可复制或共享，取决于语义	通常共享，close 影响其他线程
用户栈	子进程有复制视图	每线程独立栈
信号 mask	每线程独立 mask	进程 pending 与线程 pending 并存
PID/TID	父子不同 PID	同 TGID，不同 TID

线程库还要处理 TLS 和启动 trampoline。内核创建新线程时准备一个初始寄存器状态，使它第一次返回用户态时进入用户态线程入口，参数指向线程函数和参数对象。线程退出时要清理用户栈、TLS、robust futex、join 状态和内核 task 引用。

exit 不是立刻消失。

任务调用 `exit` 后，不能简单释放自己。它可能还在别的表里有引用，父进程还要读取退出码，`ptrace` 或 `wait` 还可能观察它。于是退出分成多个阶段：停止用户执行、释放大资源、记录退出状态、通知父进程、进入 zombie，最后由 `wait` 或 `reaper` 释放 task。

```
do_exit:
    mark task exiting
    close files and release mm
    release futexes and timers
    record exit_code and resource usage
    notify parent and reparent children
    become zombie
    schedule away forever

wait:
    find zombie child
    copy status to parent
    unlink child
    put final task reference
```

zombie 是一个必要状态：它保留父进程需要读取的结果，同时保证已退出任务不会再被调度。如果父进程不 `wait`，系统要通过 `init` 或 `subreaper` 接管，防止 zombie 永久积累。

父子关系和 reparent。

进程树不是装饰。父进程负责 `wait` 子进程；信号传播、作业控制、会话、进程组都依赖关系结构。若父进程先退出，子进程不能失去回收者，内核会把它们 `reparent` 给 `init` 或最近的 `subreaper`。

```
parent exits:
    for each child:
        choose new_parent = nearest subreaper or init
        move child to new_parent.children
        if child is zombie:
            notify new_parent
```

这里的并发点是父子列表、退出状态和 `wait` 扫描必须在锁保护下保持一致。否则 `wait` 可能漏掉刚退出的子进程，或者子进程被两个父对象同时看到。

进程生命周期的测试矩阵。

进程生命周期测试应该覆盖正常路径、失败路径和并发路径。

测试	关键断言	风险
----	------	----

fork 基本路径	父返回 child pid, 子返回 0	寄存器帧设置错误
fork 失败注入	每个阶段失败后引用数归零	资源泄漏或半成品可见
fork COW	父子写入互不污染	PTE/TLB 权限错误
exec 成功	PID 保留, 地址空间替换	fd/signal/cred 规则错误
exec 失败	commit 前失败旧程序继续	旧 mm 被提前破坏
exit/wait	zombie 被 wait 回收一次	重复回收或泄漏
多线程 close/read	共享 fd 表语义一致	use-after-close 或引用泄漏
reparent	父退出后子由 reaper 回收	zombie 泄漏

如果一本书只讲 fork/exec/wait 的定义, 不讲这些断言, 读者就无法写出可靠内核, 也无法读懂真实源码里的复杂错误路径。

第二阶段深讲：调度器工程化，从算法到多核策略

调度算法课常用 FCFS、RR、优先级、MLFQ、CFS、EDF 做比较, 但内核调度器不是算法博物馆。真实调度器要在中断、锁、cache、NUMA、cgroup、实时任务、能耗和安全边界之间做决策。本节关注算法落地后的工程问题: 运行队列怎么组织, 唤醒如何抢占, 负载如何迁移, 配额如何限流, 证据如何解释。

调度器只从 `runnable` 集合里选。

很多性能分析误把“线程没运行”归因于调度器。调度器只负责选择 `runnable` 任务; 如果任务 `sleeping`, 它等待的是锁、I/O、page fault、timer、信号或条件变量。正确诊断必须先问任务在哪个集合中。

任务位置	含义	调度器能否直接运行
CPU current	正在运行或在内核入口中	已经在运行
runqueue	<code>runnable</code> , 等待 CPU	可以被选择
wait queue	条件不满足, 睡眠中	不能, 必须被唤醒
完成请求	等设备或异步结果	不能, <code>completion</code> 路径唤醒
zombie list	已退出, 等父进程 <code>wait</code>	不能

这个区分决定了工具选择。`runnable` `wait` 高, 查 `runqueue` 和 CPU 配额; `sleep wait` 高, 查锁、I/O 或等待对象; `zombie` 多, 查父进程是否 `wait`。没有这个分类, 调度分析会变成猜测。

每 CPU `runqueue` 为什么必要。

单一全局 `runqueue` 实现简单, 但多核上所有 CPU 都要竞争一把锁, 且每次调度都会移动同一组 `cache line`。每 CPU `runqueue` 把局部调度变成局部锁, 降低争用; 代价是负载不均时需要跨 CPU 迁移。

```
per-cpu scheduling:
local timer tick updates current CPU's rq
local wakeup usually enqueues on target CPU rq
idle CPU tries to pull tasks from busy CPU
periodic balance compares load across domains
```

这就产生了调度域：SMT sibling、同一 LLC、同一 NUMA node、跨 node。近距离迁移成本低，远距离迁移成本高。调度器不能只看任务数量，还要看任务负载、cache 热度、内存位置和 CPU 能力。

唤醒路径比定时器路径更影响交互延迟。

交互任务多数时间在睡眠，事件到来后被唤醒。如果唤醒后还要等很久才运行，用户会感到卡顿。因此调度器不仅要在时间片耗尽时选择任务，还要在 wakeup 时判断是否应抢占当前任务。

```
wakeup(task):
    task.state = RUNNABLE
    place task on target runqueue
    if task should preempt current:
        set reschedule flag
        if remote CPU:
            send reschedule IPI
```

这里有两个难点。第一，唤醒任务放到哪个 CPU：放回上次 CPU 有 cache 局部性，放到空闲 CPU 有低延迟，放到数据所在 NUMA node 有内存局部性。第二，是否抢占当前任务：过度抢占会增加切换成本，抢占不足会增加交互延迟。

CFS 的 min_vruntime 是队列坐标系。

只讲 vruntime 还不够。每个 CFS runqueue 还维护 min_vruntime，表示队列已经推进到的公平时间基准。新任务和唤醒任务不能简单设为 0，否则它们会在树最左边占便宜；也不能放得太靠后，否则唤醒延迟差。

```
enqueue_fair(task, rq):
    if task is new:
        task.vruntime = rq.min_vruntime
    if task wakes from sleep:
        task.vruntime = max(task.vruntime, rq.min_vruntime - wakeup_granularity)
    insert task into rb_tree ordered by vruntime
```

min_vruntime 让每个 runqueue 有自己的坐标系。迁移任务时，需要把任务的 vruntime 从源 runqueue 坐标转换到目标 runqueue 坐标，否则任务可能因为跨 CPU 迁移得到不公平优势或惩罚。

EEVDF 思路：从公平份额到最早合格截止。

现代公平调度进一步引入虚拟 deadline 思路。直观地说，任务不仅有“已经用了多少 CPU”的账本，还可以有“按权重和 slice 计算出的虚拟截止时间”。调度器倾向选择已经 eligible 且虚拟 deadline 最早的实体，以改善延迟和公平之间的折中。

```
entity fields:
    vruntime
    weight
    slice
    virtual_deadline = vruntime + slice / weight_scaled

pick:
    among eligible entities
    choose earliest virtual_deadline
```

这里不需要把工业实现细节全部搬进原理卷，但要说明动机：单纯最小 vruntime 关注长期公平，交互延迟还要通过 slice、wakeup 和 eligibility 调节。调度器演进的方向，是把“公平”和“延迟目标”放进同一个可维护账本。

实时调度和普通调度的边界。

实时任务不能和普通 CFS 任务只靠 nice 值竞争。实时调度类通常优先于普通调度类，FIFO/RR 实时任务能长时间压制普通任务。deadline 调度还要处理 runtime、period、deadline 和准入控制。

调度类	目标	风险
stop/deadline	CPU 停止、deadline 保证	最高优先级路径错误会冻结系统
real-time FIFO/RR	固定优先级实时响应	饥饿普通任务，需要限额
fair	权重公平与通用吞吐	不能保证硬实时
idle	没有其他任务时运行	不应影响正常调度

实时任务需要限额和监控，否则一个死循环实时线程可以让系统失去响应。工业内核通常提供 RT throttling，防止实时任务占满全部 CPU 时间。

cgroup 配额造成的周期性延迟。

容器或服务组常被配置 CPU quota。假设周期 100ms，配额 20ms。组内任务在本周期累计运行 20ms 后会被 throttle，即使机器还有空闲 CPU，也要等下个周期补充预算。

```
cgroup runtime:
  on task execution:
    group.runtime_remaining -= delta
  if group.runtime_remaining <= 0:
    throttle all runnable tasks in group
  at period timer:
    refill group.runtime_remaining
    unthrottle group
```

这种机制会形成很典型的尾延迟：请求前 20ms 很快，预算耗尽后突然等待到周期刷新。排查容器内延迟时，必须看 throttled time、throttled periods 和 quota 配置，而不是只看宿主 CPU 利用率。

优先级反转是调度器和锁的交叉问题。

低优先级任务持锁，高优先级任务等锁，中优先级任务占 CPU，会导致高优先级任务间接被中优先级任务阻塞。调度器单独看 runnable 集合时看不出问题，因为真正持有资源的是低优先级任务。

```
priority inheritance:
  H waits on lock owned by L
  boost L effective priority to at least H
  if L waits on another owner X:
    propagate boost to X
  when lock released:
    recompute effective priority
```

实现上要维护 owner、waiters、effective priority 和继承链。释放锁时不能简单恢复原优先级，因为任务可能还持有另一个被高优先级任务等待的锁。这个例子说明同步原语不能和调度器完全隔离。

调度器证据：如何解释一次延迟。

一次请求慢，要按时间分段：

```
request latency =
  runnable wait
  + on-cpu execution
  + sleeping on I/O or lock
```



```
+ page fault/reclaim
+ scheduler and interrupt overhead
```

常用证据包括 `perf sched`、`/proc/<pid>/sched`、`tracepoint`、`off-CPU profile`、`/proc/schedstat`、`cgroup CPU 统计`、`pidstat -w`。不同证据回答不同问题。

证据	回答的问题	层次
<code>/proc/<pid>/sched</code>	任务运行、等待、迁移等统计	单任务
<code>perf sched</code>	调度事件时间线	系统级
<code>cgroup cpu.stat</code>	是否被 <code>quota throttle</code>	服务组
<code>tracepoint sched_*</code>	<code>wakeup</code> 、 <code>switch</code> 、 <code>migrate</code> 事件	内核事件
<code>off-CPU profile</code>	睡眠等待在哪里发生	锁/I/O

调度器优化要先定位是哪一段时间异常。若大部分时间在 `off-CPU` 等锁，改 `CFS` 参数不会解决；若大部分时间 `runnable wait`，才应看 `CPU 配额`、`优先级`、`runqueue` 和负载均衡。

调度器工程验收。

- 能解释 `task` 为什么不在 `CPU: runnable`、`sleeping`、`zombie`、`throttled` 要分清。
- 能说明每 `CPU runqueue` 与负载均衡的取舍。
- 能推导唤醒抢占对交互延迟和上下文切换成本的影响。
- 能解释 `vruntime`、`min_vruntime` 和迁移坐标转换的关系。
- 能说明 `cgroup quota` 为什么会制造周期性尾延迟。
- 能用调度事件证据区分 `on-CPU 慢` 和 `off-CPU 等待`。

第二阶段证据与验收：把卡顿、死锁和饥饿拆开

第二阶段的目标不是背更多调度算法和同步原语，而是能解释一个系统为什么没有推进。用户说“卡住了”，内核可能在等 `CPU`、等锁、等 `I/O`、等内存回收、等定时器、等信号，也可能已经退出但没人回收。本节把证据、问题类型和验收方法放在一起。

先分类：没有推进的六种状态。

现象	内核解释	主要证据
<code>CPU 忙但请求慢</code>	<code>runnable</code> 竞争、锁自旋、软中断占用	<code>perf top</code> 、 <code>perf sched</code> 、 <code>runqueue</code>
<code>CPU 不忙但请求慢</code>	线程 <code>sleeping</code> 等事件	<code>off-CPU profile</code> 、等待队列、 <code>I/O 统计</code>
周期性尖峰	<code>cgroup throttle</code> 、定时批处理、 <code>GC/回收</code>	<code>cgroup c p u . s t a t</code> 、 <code>tracepoint</code>
永久等待	<code>lost wakeup</code> 、死锁、关闭未唤醒	等待图、锁依赖、线程栈
低优先级长期得不到运行机会	高优先级或实时任务长期占用	调度类、优先级、 <code>runtime 统计</code>

zombie 增多	父进程未 wait 或 reaper 异常	ps、进程树、wait 路径
-----------	-----------------------	----------------

分类的价值是避免乱调参数。若线程在 I/O wait，修改时间片无效；若被 cgroup 限流，优化业务代码可能只降低配额内 CPU 时间，仍有周期性等待；若是 lost wakeup，增加 CPU 只会让 bug 更随机。

线程栈是等待原因的第一证据。

卡住时，先看线程栈。用户态栈能显示线程停在哪个库调用；内核栈能显示它睡在 futex、poll、read、writeback、page fault、mutex、waitpid 还是其他路径。

```
questions for a blocked thread:
Is it in futex_wait?
Is it in epoll_wait or poll?
Is it in read/write/fsync?
Is it in page fault or reclaim?
Is it waiting for child exit?
Is it runnable but not scheduled?
```

只有 runnable 但没被调度，才优先看调度器。若栈显示在 futex_wait，要找持锁者；若在 epoll_wait，要找 fd readiness 和关闭路径；若在 waitpid，要找子进程状态。

等待图：死锁和优先级反转的共同语言。

等待图把“谁等谁”显式化。线程等待锁，锁由另一个线程持有；线程等待 I/O，请求由设备 completion 完成；线程等待子进程，子进程由调度器和 exit 路径推进。死锁是等待图成环，优先级反转是等待图上的 owner 得不到 CPU。

```
wait graph example:
T1 waits lock A, owned by T2
T2 waits lock B, owned by T3
T3 waits lock C, owned by T1
=> cycle, deadlock

priority inversion:
High waits lock A, owned by Low
Medium runnable and preempts Low
=> no cycle, but scheduler must boost Low
```

这个区分很重要。死锁需要打破环；优先级反转需要让持锁者运行；lost wakeup 则可能没有 owner，因为等待者睡在一个再也不会被通知的队列上。

调度证据的最小集合。

一次调度问题至少收集：

```
minimum scheduling evidence:
number of runnable tasks
context switch rate
per-task runtime and wait time
migrations
cgroup throttled time
interrupt and softirq distribution
```

指标	解释	风险
nr_running	可运行任务数量	高表示 CPU 竞争

voluntary cs	主动阻塞切换	I/O/锁/等待事件多
involuntary cs	被抢占切换	时间片、优先级、竞争
migrations	CPU 迁移次数	可能损失 cache/NUMA 局部性
throttled time	cgroup 配额等待	容器周期性延迟
softirq time	网络/块等延后处理	可能抢占业务线程

这些指标要放到同一条时间线上看。平均值很容易掩盖尾延迟：一个 100ms 周期里前 20ms 很快，后 80ms 被 throttle，平均 CPU 使用并不能解释 p99。

同步证据的最小集合。

同步问题至少收集对象状态、等待谓词、持有者、等待者和关闭状态。

```
minimum synchronization evidence:
  object id and state fields
  lock owner
  waiter list
  wait predicate
  last state transition
  close/cancel/timeout flag
```

如果一个队列卡住，要能回答：队列是空还是满，closed 是否为真，谁应该唤醒 not_empty，谁应该唤醒 not_full，等待者的 timeout 是否已经过期。没有这些字段，日志再多也只是噪声。

第二阶段负面测试。

注入点	目的	期望
fork 第 N 步失败	验证资源回滚	无泄漏、无半成品可见
exec commit 前失败	验证旧程序保留	返回错误，旧地址空间仍可用
wait 与 exit 并发	验证 zombie 回收	只回收一次
唤醒发生在入队前	验证无 lost wakeup	等待者不永久睡眠
锁顺序反转	验证 deadlock 检测	报告等待环
高优先级等低优先级锁	验证优先级继承	持锁者被提升并释放锁
cgroup budget 耗尽	验证 throttle/unthrottle	周期刷新后恢复运行
关闭有等待者的队列	验证广播唤醒	所有等待者退出

这些测试比“创建几个线程跑一跑”更接近内核质量要求。它们检查的是状态机边界，而不是碰巧跑通的正常路径。

第二阶段完成标准。

- 能从 fork 的每个阶段说出可见性、失败点和回滚对象。

- 能解释 exec 的提交点，以及 commit 前后失败语义为什么不同。
- 能区分 runnable wait、on-CPU、off-CPU sleep 和 zombie。
- 能说明每 CPU runqueue、唤醒抢占、负载均衡和 cgroup 配额的关系。
- 能为一个同步对象写出字段、不变量、等待谓词和关闭路径。
- 能画出死锁等待图、优先级反转等待图和 lost wakeup 时序。
- 能用证据判断一个卡顿属于调度问题、锁问题、I/O 问题还是退出回收问题。

到这个标准，进程、调度和同步才不再是 API 名称，而是可证明推进的状态机。

第二阶段源码与实验：把任务、调度、同步落到可验证对象

前面的章节已经把进程生命周期、调度策略和同步状态机讲成了模型。模型还不够。读者必须能把模型落到三类材料上：真实内核源码里的对象和函数、可运行的小实验、出错时能区分原因的观测命令。本节不追求把 Linux 源码逐行翻译，而是给出一条能读通的路线：先找对象，再找状态转移，再找失败路径，最后用实验验证自己的理解。

源码阅读地图：先读结构，再读路径。

操作系统源码很大，直接从入口函数一路跳会很快迷路。更可靠的读法是先确定“哪些结构保存状态”，再读“哪些函数改变状态”。以 Linux 为参照，第二阶段最相关的路径如下。

主题	典型源码位置	读的时候要抓住什么
任务结构	<code>include/linux/sched.h</code>	<code>task_struct</code> 的状态、调度实体、父子关系、引用对象
fork 路径	<code>kernel/fork.c</code>	<code>kernel_clone</code> 、 <code>copy_process</code> 、 <code>copy_mm</code> 、 <code>copy_files</code> 、 <code>copy_thread</code>
exec 路径	<code>fs/exec.c</code>	二进制格式检查、参数复制、新 mm 提交点、凭据转换
exit/wait	<code>kernel/exit.c</code>	<code>do_exit</code> 、zombie、 <code>reparent</code> 、wait 扫描和最终释放
核心调度	<code>kernel/sched/core.c</code>	<code>enqueue/dequeue</code> 、 <code>wakeup</code> 、 <code>schedule</code> 、 <code>context switch</code> 调用边界
公平调度	<code>kernel/sched/fair.c</code>	<code>vruntime</code> 、 <code>min_vruntime</code> 、唤醒放置、负载均衡
实时调度	<code>kernel/sched/rt.c</code> 、 <code>kernel/sched/deadline.c</code>	固定优先级、预算、deadline 和准入控制
futex	<code>kernel/futex/core.c</code>	用户态原子值和内核等待队列如何连接
mutex/rtmutex	<code>kernel/locking/mutex.c</code> 、 <code>kernel/locking/rtmutex.c</code>	owner、waiter、优先级继承、慢路径

pipe 等待	fs/pipe.c	条件谓词、等待队列、关闭唤醒、非阻塞语义
RCU	kernel/rcu/tree.c	读侧临界区、宽限期、延迟释放

读源码时不要先追所有宏。先写出每个对象的字段表：哪些字段是身份，哪些字段是状态，哪些字段是链接关系，哪些字段是引用计数，哪些字段只在持锁时可改。然后再读改变这些字段的路径。这样能避免把源码读成函数调用列表。

fork 的源码读法：每一步都问可见性。

fork 或 clone 的阅读重点不是“它创建了进程”，而是每一步创建出的对象什么时候对其他 CPU 可见。一个半初始化任务若提前进入 PID 表、父子列表或 runqueue，后续调度、信号、wait 都可能观察到破碎状态。

```
source reading checklist for fork:
1. task object and kernel stack allocated?
2. scheduler fields initialized but not runnable?
3. credentials, files, fs, signal, mm copied or shared?
4. child register frame prepared?
5. pid allocated and published?
6. parent-child relation linked?
7. task finally made runnable?
8. every failure label releases exactly the acquired references?
```

这条路线能解释源码里的错误标签。若 copy_files 成功、copy_mm 失败，错误路径必须释放文件表引用；若 PID 已经发布，回滚就要同时撤销 PID 映射；若任务已经 runnable，很多失败不能再简单 free，而要走退出路径。源码中的阶段边界，就是原理章节里的生命周期边界。

上下文切换：保存的不是“整个进程”。

上下文切换常被抽象成“保存 A，恢复 B”。这句话容易误导。内核并不会每次复制整个进程资源；它保存的是继续执行所需的 CPU 上下文，并切换当前 CPU 指向的任务、栈、地址空间和调度状态。地址空间、文件表、凭据这些对象通过引用挂在 task 上，不会在每次切换时复制。

项目	切换时发生什么	为什么重要
内核栈	从 prev 的内核栈切到 next 的内核栈	每个线程有自己的内核执行上下文
通用寄存器	保存调用约定要求保存的部分，并恢复 next 的部分	保证内核函数返回后能继续执行
RIP/返回地址	由切换汇编和栈帧间接恢复	next 像是从上次 schedule 后继续
页表根	进程地址空间不同才需要切换或标记切换	影响 TLB、PCID 和内存隔离
FPU/SIMD 状态	通常惰性或按需保存恢复	AVX 状态很大，不能无脑复制
TLS/段寄存器	用户线程切换时要保持线程局部状态正确	影响 thread_local 和 libc 线程数据
调度统计	记录运行时间、等待时间和切换事件	后续诊断依赖这些统计

在 x86-64 Linux 中，可以沿着 schedule()、context_switch()、switch_to 和

架构相关切换汇编去读。读的时候重点看三件事：prev 的栈指针保存在哪里，next 的栈指针从哪里恢复，地址空间切换在哪个条件下发生。明白这三点，才算真正理解“线程切换比进程切换轻”的具体含义。

一个可检查的调度器数据结构。

为了把调度算法写得可测，先不要急着实现所有策略。可以先写一个最小模型：任务状态、runqueue、等待原因和调度统计必须显式存在。

```
#include <cstdint>
#include <deque>
#include <limits>
#include <stdexcept>
#include <vector>

using TaskId = std::size_t;
using ProcessId = std::uint32_t;

constexpr TaskId SCHED_IDLE_TASK = std::numeric_limits<TaskId>::max();
constexpr ProcessId SCHED_INVALID_PID = 0;

enum class TaskState {
    Runnable,
    Running,
    Sleeping,
    Zombie,
};

enum class WaitReason {
    None,
    Mutex,
    Io,
    Timer,
    ChildExit,
    PageFault,
};

struct Task {
    TaskId tid = SCHED_IDLE_TASK;
    ProcessId pid = SCHED_INVALID_PID;
    int priority = 0;
    TaskState state = TaskState::Runnable;
    WaitReason wait_reason = WaitReason::None;
    std::uint64_t vruntime = 0;
    std::uint64_t runnable_since = 0;
    std::uint64_t total_runtime = 0;
    std::uint64_t total_wait = 0;
};

struct RunQueue {
    std::deque<TaskId> rr;
    std::uint64_t clock = 0;
};
```

这段结构虽然小，却已经能写出硬断言：只有 **Runnable** 能入队；调度选中的任务必须从 **Runnable** 变成 **Running**；进入 **Sleeping** 时必须记录 **wait_reason**；进入 **Zombie** 后不能再回到 runqueue；每次 tick 要更新当前任务的 runtime，而不是误把 sleeping 时间算成运行时间。

```
void enqueue(RunQueue& rq, std::vector<Task>& tasks, TaskId tid) {
    auto& task = tasks.at(tid);
```

```

    if (task.state != TaskState::Runnable) {
        throw std::logic_error("only runnable task may enter runqueue");
    }
    task.runnable_since = rq.clock;
    rq.rr.push_back(tid);
}

TaskId pick_next(RunQueue& rq, std::vector<Task>& tasks) {
    while (!rq.rr.empty()) {
        const TaskId tid = rq.rr.front();
        rq.rr.pop_front();
        auto& task = tasks.at(tid);
        if (task.state == TaskState::Runnable) {
            task.total_wait += rq.clock - task.runnable_since;
            task.state = TaskState::Running;
            return tid;
        }
    }
    return SCHED_IDLE_TASK;
}

```

很多教学代码的调度器 bug 来自“队列里只有 tid，没有状态断言”。任务睡眠后忘记出队、退出后仍被选中、唤醒时重复入队，都会在压力测试下变成随机错误。把状态检查写进数据结构入口，能让错误尽早暴露。

futex 实验：用户态快路径为什么还需要内核。

futex 的实验目标不是背 futex(2) 参数，而是观察“无竞争不进内核，有竞争才睡眠”。可以写一个用户态 mutex：先用 CAS 从 0 改到 1；失败后把状态改成 2，再调用 futex wait；解锁时把状态置 0，若旧状态是 2，则 futex wake。

```

observe futex:
strace -f -e futex ./futex_mutex_demo uncontended
strace -f -e futex ./futex_mutex_demo contended
perf stat -e context-switches,cpu-migrations ./futex_mutex_demo contended

```

无竞争版本中，绝大多数 lock/unlock 不应产生 futex 系统调用；竞争版本中，应该能看到 wait/wake。若每次加锁都进内核，说明 fast path 失败；若竞争版本 CPU 占满但 futex 调用很少，可能在忙等；若 wait 后没人 wake，要检查 unlock 是否根据旧状态唤醒。

lost wakeup 的可复现实验。

lost wakeup 最难的地方是它看起来像偶发卡住。要让它稳定复现，可以故意写一个错误版本：等待者先检查谓词，释放锁，然后再把自己加入等待队列。唤醒者在这个窗口里改变条件并 notify，等待者随后入队睡眠，就会错过唯一一次唤醒。

```

wrong wait sequence:
lock(m)
if (!ready) {
    unlock(m)           // window begins
    enqueue(waiters, self)
    sleep()
}

waker:
lock(m)
ready = true
notify_one()
unlock(m)

```

正确实验要用 barrier 固定时序：让等待线程停在“释放锁之后、入队之前”，再让唤醒线程完成 notify，最后放行等待线程。期望结果是错误实现永久睡眠，正确实现不会睡眠或会立即重新检查谓词。这个测试比随机跑一万次更有价值，因为它直接覆盖了危险窗口。

condition variable 的三个负面案例。

条件变量的常见错误可以拆成三个小实验，每个实验都应该有明确期望。

错误	构造方法	期望现象
if 代替 while	两个消费者等待一个 item, notify all 后同时醒来	一个消费者拿到 item, 另一个必须重新睡眠；错误实现会读空队列
修改状态前 notify	producer 先 notify, 再 push item	consumer 醒来看到旧状态，可能继续睡或错误返回
关闭不广播	多个 producer/consumer 睡在不同队列，close 只唤醒一边	另一边永久等待

这三个案例对应三条规则：等待必须循环检查谓词；状态改变和 notify 的顺序必须让等待者能看到新状态；关闭是一种状态变化，必须唤醒所有可能等待这个状态的线程。

same-pool future deadlock。

线程池和 future 是现代 C++ 程序里很常见的同步组合。一个典型死锁是：线程池只有 N 个 worker，N 个任务都在等待同一个线程池中尚未运行的子任务。所有 worker 都被等待占住，子任务无人执行。

```
bad pattern:
worker task A:
    future = pool.submit(B)
    return future.get()    // blocks a worker

if all workers do this:
    no worker remains to run B
```

修复方向有三种。第一，把等待改成 continuation：B 完成后触发 A 的后续逻辑，不占住 worker。第二，允许 worker 在等待时帮助执行队列中的其他任务，但要小心重入和优先级。第三，把阻塞等待放到专门的 blocking pool，计算 pool 不做无限期等待。这个例子说明同步问题不只在内核；用户态运行时也必须有调度意识。

调度观测命令：每个命令回答一个问题。

观测命令不是越多越好。关键是每个命令要回答一个具体问题。长命令不要塞进表格；实验报告里应把命令、问题和结论分开写。

```
cat /proc/<pid>/sched
```

它回答单个任务运行了多久、等待了多久、迁移了几次。若 runnable wait 高，说明任务已经可运行但长期拿不到 CPU。

```
perf stat -e context-switches,cpu-migrations,page-faults ./workload
```

它回答切换、迁移和缺页是否异常。上下文切换暴涨常见于线程过多、锁竞争或频繁阻塞；迁移过多可能破坏 cache 和 NUMA 局部性；缺页多时要进一步区分 minor fault、major fault 和回收。

```
perf sched record ./workload
perf sched latency
```

这组命令回答“任务被唤醒后等了多久才真正运行”。若唤醒延迟集中在某些线程或某些 CPU 上，要继续检查 runqueue、CPU affinity、优先级和 cgroup quota。

```
pidstat -w -p <pid> 1
```

它区分自愿上下文切换和非自愿上下文切换。自愿切换多，通常说明线程主动阻塞在 I/O、锁或等待事件；非自愿切换多，通常说明被抢占或 CPU 竞争强。

```
cat /sys/fs/cgroup/<group>/cpu.stat
```

它回答服务组是否被 CPU 配额限流。若 throttled time 和 throttled periods 增长，p99 延迟很可能来自配额周期，而不是业务函数突然变慢。

```
strace -f -e futex,clone,execve,wait4 ./workload
```

它回答线程、进程和 futex 系统调用行为。若大量线程停在 futex wait，就要寻找锁 owner、等待谓词和唤醒路径；若 wait4 长期不返回，要检查子进程是否退出以及父子关系是否正确。

```
perf top
perf record ./workload
```

它们回答 CPU 真正在执行哪里。若热点在自旋、内存拷贝、哈希查找或内核软中断，优化方向完全不同。

做报告时不要只贴命令输出。要写清楚：实验负载是什么，预期瓶颈是什么，命令观察到的现象是否支持预期，若不支持下一步查什么。例如 p99 延迟高，同时 `cpu.stat` 显示大量 throttled time，说明要先看 CPU quota；若 `perf sched latency` 显示唤醒后等待长，但 cgroup 没限流，要看 runqueue、优先级和 CPU affinity。

从源码到实验的闭环。

第二阶段的最低闭环可以按下面顺序完成。

1. 读 `copy_process`，画出 fork 的阶段图和失败回滚表。
2. 写一个 fork/exec/wait 小程序，用 `strace -f` 验证父子返回值、exec 成功路径和 wait 回收。
3. 读 `schedule` 到架构切换路径，说明一次线程切换保存哪些状态。
4. 写一个 Round Robin 或公平调度模型，断言 sleeping/zombie 任务不能入队。
5. 写一个错误 condition variable 示例，用 barrier 稳定制造 lost wakeup。
6. 写一个 futex mutex 或阅读 pthread mutex 的 futex 行为，用 `strace -e futex` 区分快路径和慢路径。
7. 用 `perf sched` 或 `/proc/<pid>/sched` 分析一次人为制造的 runnable wait。

完成这些实验后，读者不再只是知道概念名称，而是能把一个现象定位到对象、状态、源码路径和观测证据。到这一步，任务、调度与同步这一章才真正具备工程教材的质量。

调度器深入：时间、等待与多核

原理

上一章讲了基本调度算法，本章把调度器当成内核子系统看。调度器不是一个简单的 `pick_next()` 函数，它维护可运行队列、睡眠队列、时间片账本、优先级、CPU 亲和性、负载均衡、抢占标志和统计数据。它还必须和锁、定时器、中断、系统调用返回路径配合。

调度器的核心循环是：当前任务运行；某个事件改变任务状态；如果当前任务不能继续运行或应该让出 CPU，内核选择下一个 `runnable` 任务；保存当前上下文，恢复下一个上下文。事件可能来自时间片耗尽、任务阻塞、任务退出、任务被唤醒、高优先级任务到达或 CPU 负载均衡。

调度器维护两个不同问题。第一是选择：在 `runnable` 集合里选谁。第二是状态转换：任务如何进入 `runnable`、`running`、`sleeping`、`stopped`、`zombie`。很多 bug 不在选择算法，而在状态转换，例如任务已经 `sleeping` 却仍留在 `runqueue`，或者唤醒丢失导致任务永远睡眠。

实践

实践报告要把调度器拆成四张表：

表	内容
任务表	pid/tid、状态、优先级、时间片、等待原因
<code>runqueue</code>	每个 CPU 的 <code>runnable</code> 任务和顺序
<code>sleep queue</code>	等待对象、超时时间、唤醒条件
统计表	on-CPU 时间、 <code>runnable wait</code> 、 <code>context switch</code> 、迁移次数

当报告说“调度不公平”时，必须给出 `runnable wait` 分布；当报告说“线程没运行”时，必须说明它是 `runnable` 但没拿到 CPU，还是 `sleeping` 等事件。

算法与实现分析

睡眠和唤醒。

阻塞系统调用和条件变量最终都要把任务放入某个等待队列。正确顺序是：持有保护等待条件的锁，检查条件，不满足则把当前任务加入等待队列并把状态改为 `sleeping`，然后原子地释放锁并调度。唤醒方改变条件后，从等待队列取任务，改为 `runnable`，放入 `runqueue`。

```
sleep(wait_queue, lock):
    current.state = SLEEPING
    wait_queue.push(current)
    unlock(lock)
    schedule()
    lock(lock)

wake_one(wait_queue):
    task = wait_queue.pop()
    if task != null:
        task.state = RUNNABLE
        enqueue(runqueue_for(task), task)
```

这里最危险的是 `lost wakeup`。如果等待者检查条件后、真正入队前，唤醒者改变条件并发出唤醒，那么等待者可能永远睡眠。解决方法是用同一把锁保护“检查条件”和“加入等待队列”。

抢占点。

内核不能在任意机器指令之间随便切换。抢占点通常出现在中断返回、系统调用返回、显式阻塞、时间片耗尽或内核允许抢占的位置。如果当前代码持有自旋锁或处于中断上下文，就不能睡眠调度。


```
maybe_preempt():
    if need_reschedule
        and preempt_count == 0
        and !in_interrupt():
            schedule()
```

`preempt_count` 是实现边界的典型例子。它不是业务概念，而是内核用来防止在不可抢占区域切走任务的计数。进入自旋锁或中断上下文时增加，退出时减少。若计数错误，可能在持锁时调度，导致死锁。

每 CPU runqueue。

多核系统若使用一个全局 runqueue，所有 CPU 都要争同一把锁。每 CPU runqueue 降低锁竞争：每个 CPU 主要从自己的队列取任务；周期性负载均衡把任务从繁忙 CPU 迁到空闲 CPU。

```
schedule_on_cpu(cpu):
    rq = cpu.runqueue
    lock(rq.lock)
    next = pick_next(rq)
    unlock(rq.lock)
    switch_to(next)

load_balance():
    busiest = find_busiest_cpu()
    idle = find_idle_cpu()
    migrate_some_tasks(busiest.rq, idle.rq)
```

多核调度的取舍是局部性和均衡。任务留在原 CPU，cache/TLB 更热；迁移到空闲 CPU，等待时间更低。调度器不能只看队列长度，还要看任务亲和性、NUMA 节点、迁移成本和负载类型。

实时任务。

实时调度关注 deadline，而不只是平均公平。Rate Monotonic 适合周期任务，周期越短优先级越高；Earliest Deadline First 总是选择绝对 deadline 最近的任务。单处理器上，在理想条件下 EDF 可以达到更高利用率，但实现要维护按 deadline 排序的队列。

```
pick_edf(runqueue):
    return min_by(runqueue, task.absolute_deadline)
```

实时系统必须做可调度性分析。若任务集合的最坏执行时间总和超过 CPU 能力，任何调度算法都无法保证 deadline。操作系统只能执行策略，不能创造时间。

唤醒抢占。

一个高优先级或交互任务被 I/O 唤醒时，如果只是放到 runqueue 尾部，可能仍要等当前 CPU 密集任务用完整个时间片。唤醒抢占的思想是：唤醒路径比较新任务和当前任务，若新任务更应该运行，就设置 `need_reschedule`，让返回路径尽快切换。

```
try_wake_task(task):
    rq = select_runqueue(task)
    lock(rq.lock)
    task.state = RUNNABLE
    enqueue(rq, task)
    if should_preempt(task, rq.current):
        rq.current.need_reschedule = true
        send_reschedule_ipi_if_remote(rq.cpu)
    unlock(rq.lock)
```

这里有两个风险。第一，过度抢占会增加上下文切换，降低吞吐。第二，跨 CPU 唤醒需要 IPI，若频率过高，会把多核机器变成互相打断的系统。调度器必须在响应延迟和扰动成本之间折中。

负载均衡的粒度。

负载均衡不是简单平均任务数量。一个 CPU 上有 1 个 CPU 密集任务，另一个 CPU 上有 20 个大部分睡眠的任务，按数量看不平衡，按实际负载可能合理。更好的指标是 runnable load、最近 CPU 使用、任务权重和亲和性。

```
load_balance_domain(domain):
    busiest = cpu with highest runnable_load
    idle = cpu with lowest runnable_load
    if busiest.load <= idle.load + imbalance_threshold:
        return
    task = choose_migratable_task(busiest, idle)
    if task != null:
        migrate(task, busiest, idle)
```

迁移必须持有源和目标 runqueue 锁，并规定锁顺序，例如按 CPU id 从小到大加锁。否则两个 CPU 同时互相迁移任务，会在 runqueue 锁上形成死锁。

调度器不变量。

调度器测试要检查不变量，而不是只看任务有没有输出。

验收标准

- 每个 RUNNABLE 线程恰好在一个 runqueue 中。
- RUNNING 线程只能是某个 CPU 的 current。
- SLEEPING 线程必须在一个等待队列中，或带有可解释超时。
- ZOMBIE 进程不能再获得 CPU，但退出状态必须可被父进程读取。
- 同一线程不能同时出现在迁移源队列和目标队列。
- 持有自旋锁、关中断或 `preempt_count > 0` 时不能调度到会睡眠的路径。

测试

- lost wakeup 测试：唤醒和睡眠并发发生时，等待者不会永久睡眠。
- 持有自旋锁时不能进入会睡眠的路径。
- 每 CPU runqueue 中，同一任务不能同时出现在两个队列。
- 任务迁移后，旧 CPU 不再能调度该任务。
- 时间片耗尽、阻塞、退出和唤醒都能触发正确重调度。
- 实时任务 deadline miss 必须被记录，而不是静默忽略。

尚未解决的问题。 task 已能运行、阻塞、唤醒和退出，但普通程序仍可直接请求任何动作，也没有稳定办法把参数、错误和返回值交给常驻内核。下一章建立受控入口，并把“进入内核”“调度”“切换现场”和“返回用户态”分成不同事件。

第 3 章 为什么用户请求需要受控入口

继承的机器。内核已经能保存 task 现场并在运行队列与等待队列之间移动任务。普通任务仍不能安全执行设备配置、页表修改或对象管理；直接跳入内核函数既不能建立可信入口帧，也不能证明参数地址、对象引用和调用者权限在使用期间保持有效。

从一个无权动作得到受控入口

先考虑一个尚未提供系统调用的最小系统。用户程序能够执行普通计算，也能够调用同一地址空间里的函数。现在程序要把系统时钟向前调整一秒。若时钟控制状态可以由任意程序直接写，一个出错的程序就能改变所有进程共同观察的时间；日志顺序、超时判断和文件时间戳都会随之失去共同基准。若完全禁止用户程序接触时钟控制状态，合法的时间同步程序也无法工作。

这个矛盾只要求先增加一个最小机制：用户程序不能直接写受保护状态，但可以把“希望执行哪项操作”以及“操作需要哪些参数”交给一段具有更高执行权限的代码。该代码先判断请求是否合法，再决定是否修改状态。这个受控请求入口称为**系统调用**（system call）。名称出现在这里，是因为我们已经知道它必须同时承担两项职责：跨越执行权限边界，以及在边界内验证请求。普通函数调用只改变当前程序自己的控制流，不能获得这两项能力。

后续讨论不会立即展示完整入口链路。我们先确定究竟保护什么，再分别解决请求怎样编码、用户执行现场保存在哪里、用户地址怎样转为稳定输入、权限依据来自哪里以及阻塞后怎样返回。这些局部结构全部建立之后，本章末尾才把它们连接成完整系统调用。

特权到底保护了什么

特权级保护的不是一个抽象名分，而是一组具体硬件状态。用户态不能改页表根，不能关中断，不能改中断向量，不能写任意控制寄存器，不能访问只映射给内核的 MMIO，不能把自己的代码标成内核入口。CPU 在执行敏感指令、地址翻译和异常返回时都会检查当前模式。

受保护对象	为什么危险	合法路径
页表根/地址空间控制	可映射任意物理页，读取内核或其他进程	内核内存管理代码
中断使能和向量表	可阻止调度或劫持异常入口	内核入口和中断子系统
设备 MMIO 寄存器	可重置设备、窃取数据、破坏存储	驱动通过 fd/ioctl/sysfs 等受控接口
特权返回状态	可伪造内核态返回或跳转地址	低级 entry/return 代码检查
I/O 权限和端口	可直接访问平台设备	内核 I/O 权限管理或驱动

x86-64 用 Ring 0 和 Ring 3 表达常见用户/内核边界。Ring 3 运行普通用户程序，Ring 0 运行内核；CPU 在执行敏感指令、访问 supervisor-only 页、装载控制寄存器、返回低特权级时检查 CPL 和相关权限位。

层级	角色	系统调用入口
Ring 3	用户程序	执行 <code>syscall</code> 请求进入内核
Ring 0	内核	由 <code>IA32_LSTAR</code> MSR 指向的入口接管
VMX root	hypervisor	可选择拦截 <code>guest</code> 的敏感事件

特权边界的验收问题是：用户程序是否能绕过内核直接改变受保护状态？若答案是能，系统调用实现得再漂亮也没有隔离。

为什么进入内核要换栈

用户栈由用户程序控制。它可能没有映射，可能只读，可能指向恶意构造的数据，也可能在系统调用前被故意放到临界边界。内核不能在用户栈上保存自己的返回地址、锁状态和局部变量，否则用户就能破坏内核控制流。因此跨特权级入口必须切到可信内核栈或受控异常栈。

```
on syscall from user:
  hardware saves minimal return state
  entry code switches from user rsp to current.kernel_stack
  entry code saves registers into pt_regs
  C syscall code runs on kernel stack
```

每个线程通常有自己的内核栈。线程在用户态运行时使用用户栈；进入内核后使用内核栈；若阻塞在系统调用中，内核栈保存它在内核里的等待位置。调度器切换线程时，既要能恢复用户态 `trap frame`，也要能恢复睡眠中的内核调用链。

函数调用、库调用和系统调用的三层区别

很多初学者把 `printf`、`write`、系统调用混在一起。实际上它们处在三层边界上。普通函数调用只改变当前进程内部的 `RIP` 和栈；库调用仍在用户态，只是封装了更复杂逻辑；系统调用才跨越 CPU 特权边界进入内核。

调用类型	发生在哪里	边界含义
普通函数调用	同一进程、同一特权级	调用者和被调者互相信任同一地址空间
libc/API 调用	用户态库代码	可做缓冲、格式化、兼容处理，但不能直接改内核对象
系统调用	用户态陷入内核态	CPU 切换特权，内核检查权限和对象状态

`printf("x")` 通常先把格式化结果写到用户态 `stdio buffer`；`buffer` 满、遇到换行或显式 `flush` 时，`libc` 可能调用 `write`；`write wrapper` 把系统调用号和参数放进寄存器，执行 `syscall`。内核并不知道 `printf` 的格式化细节，它只看见 `fd`、用户 `buffer` 和长度。

```
printf("hello\n")
-> libc formats into FILE buffer
-> libc write wrapper
-> CPU syscall instruction
-> kernel sys_write
-> fd table, file object, device/file path
```

这层区分能解释许多现象：程序崩溃时 `stdio buffer` 可能没写出；`write` 返回成功不等于数据落盘；系统调用开销来自特权切换、参数检查和内核路径，不是 C

函数调用本身。

系统调用入口的硬件动作逐步展开

以 x86-64 的 `syscall` 为例，用户态执行指令后，CPU 不会查 C 函数表。它读取预先由内核配置的 MSR，切换到内核入口地址，保存返回地址和标志的一部分，改变特权相关状态。随后汇编入口完成 `swapgs`、切换内核栈、构造 `pt_regs`。

```
user mode before syscall:
    rax = syscall number
    rdi, rsi, rdx, r10, r8, r9 = args
    rip = address after syscall
    rsp = user stack

hardware/syscall entry:
    save return rip into rcx
    save flags into r11
    load kernel entry rip from MSR
    enter privileged execution path

low-level kernel entry:
    swapgs to kernel per-cpu base
    load current thread kernel stack
    save user registers into pt_regs
    call C syscall dispatcher
```

不同架构寄存器名和指令不同，但结构类似：约定参数位置，触发陷入，保存返回状态，切换到可信上下文，分发表项，返回前处理信号/抢占/审计。把这条路径记清，后面看 `entry` 汇编就不会把它当成神秘代码。

返回用户态比进入内核更危险

进入内核时，内核获得控制权；返回用户态时，内核要把控制权交还给不可信程序。返回前必须保证：目标 RIP 是用户地址，返回 CPL 是 Ring 3，用户栈指针不会让内核继续使用，挂起信号和抢占已处理，调试/审计状态一致，寄存器中不能泄露内核敏感数据。

```
return_to_user(regs):
    if need_resched: schedule()
    if signal_pending: deliver_signal(regs)
    if tracing_or_audit: syscall_exit_work(regs)
    sanitize_user_return_state(regs)
    restore_user_registers(regs)
    sysretq or iretq
```

许多安全漏洞发生在返回路径：错误使用快速返回指令、没有正确验证用户返回地址、泄露内核寄存器内容、返回前忘记处理单步/信号状态。系统调用不是“进入内核执行函数再回来”这么简单，入口和出口都是安全边界。

参数复制是系统调用安全性的核心。用户态传入的指针不能被内核直接相信，因为它可能无效、不可读、不可写、跨页、在检查后被另一个线程改掉，或指向内核不应访问的区域。内核必须使用 `copy_from_user/copy_to_user` 一类机制，把用户内存当作不可信输入处理。

用户指针为什么不能直接解引用

用户指针看起来只是一个地址，但它属于用户地址空间的契约，不属于内核可信内存。直接解引用会产生四类问题。第一，地址可能无效，内核访问会 `fault`。第二，地址可能指向用户不可读/不可写区域。第三，地址所在页可能跨页，前半可读后半不可读。第四，另一个线程可能在检查之后修改数据，造成 TOCTOU。

```
bad:
```



```
// directly trusts user pointer
header = *(struct Header*)user_ptr
use(header.length)

better:
header = copy_struct_from_user(user_ptr)
validate(header.length)
data = copy_bytes_from_user(user_ptr + sizeof(header), header.length)
```

copy 不是低效保守，而是把不可信输入转成内核拥有的稳定快照。对于大 I/O，内核可以 pin 用户页或使用零拷贝，但那会引入页生命周期、DMA 映射和 completion 责任，不能简单理解成“直接用用户指针”。

EFAULT 是如何产生的

当内核复制用户数据时，用户页可能不存在或权限不允许。若普通内核代码访问坏地址直接 fault，通常是内核 bug；但 `copy_from_user` 的 fault 是预期错误路径。内核通过异常表或等价机制告诉 page fault handler：如果 fault 发生在这段用户复制指令上，不要 panic，而是跳到 fixup 路径返回失败。

```
copy loop instruction at address A:
  load byte from user pointer
  store byte to kernel buffer

exception table:
  faulting instruction A -> fixup address F

page fault handler:
  if fault_pc found in exception table:
    regs.rip = fixup_address
    return
  else:
    handle normal kernel/user fault
```

这解释了为什么用户指针复制需要专用 API。它不仅做范围检查，还和低级异常处理配合，把硬件 fault 转换成系统调用错误码。教学 OS 也应该明确区分“内核自己访问坏地址”和“内核按用户请求复制时遇到坏用户页”。

权限检查不是一个 if

一次系统调用经常要穿过多层权限。以 `openat` 为例，路径解析要检查每级目录搜索权限；mount namespace 决定看到哪棵树；符号链接可能改变目标；LSM 可在路径或 inode 层拒绝；文件 mode、ACL、capability 决定读写执行；打开 flags 决定是否创建、截断、追加；资源限制决定还能否分配 fd。

```
openat(dirfd, path, flags):
  resolve dirfd in current fd table
  walk path under mount namespace
  check execute/search permission on directories
  handle symlink policy and flags
  get stable inode/dentry reference
  check DAC/ACL/capability
  call LSM hooks
  create file object with open flags
  allocate fd in current process fd table
```

这也是为什么错误码能提供信息。`ENOENT`、`ENOTDIR`、`EACCES`、`EPERM`、`ELOOP`、`EMFILE` 分别表示不同层次失败。把所有失败都写成“权限不够”会掩盖状态机。

root、capability 和 namespace 的演进

早期 Unix 常把 uid 0 作为全能 root。现代系统把 root 权限拆成 capability，例如 CAP_NET_ADMIN、CAP_SYS_ADMIN、CAP_DAC_OVERRIDE。namespace 又让“这个 capability 对哪个对象集合有效”成为问题。容器内 root 不应自动拥有宿主全部权限。

机制	解决的问题	新复杂性
uid/gid/mode	简单主体和文件权限	粒度粗，root 过大
capability	把 root 权限拆小	哪个路径检查哪个 capability 要准确
namespace	隔离进程看到的资源视图	capability 必须相对 user namespace 解释
LSM	策略化强制访问控制	hook 覆盖面和策略复杂度
seccomp	收缩系统调用面	只看 syscall/参数，难懂对象语义

权限系统的本质是最小授权。内核应让调用者只获得完成任务所需能力；任何跳过对象所属 namespace 的 capability 检查，都可能把容器内权限放大成宿主权限。

权限不只有文件 mode 位。一个请求是否允许，可能取决于用户 ID、组 ID、capability、namespace、cgroup、seccomp、LSM、安全标签、mount 选项、fd 打开模式、对象引用和当前进程状态。OS 学习中的常见错误，是把“我是这个进程”误以为“我拥有所有资源”。

错误码也是契约。EACCES 表示权限不足，ENOENT 表示对象不存在，EBADF 表示 fd 无效，EFAULT 表示用户指针不可访问，EINTR 表示系统调用被信号打断，EAGAIN 表示非阻塞对象暂时不能推进。高质量 OS 程序必须按错误码分支，而不是把所有失败都写成 fatal。

请求首先需要是一个稳定的边界契约

只有入口地址还不能表达请求

假设处理器已经提供一条指令，能把执行权从用户程序转交给内核的固定入口。此时仍然缺少两个事实：用户究竟请求哪项服务，以及服务所需的参数在哪里。让每项服务占用一个不同入口地址看似直接，却会让入口数量随着服务增加而增长；内核还要为每个入口单独配置权限和返回规则。更重要的是，程序无法只凭一个稳定编号描述请求，兼容层、审计器和过滤器也难以统一观察所有入口。

因此，最小设计只保留一个受控入口，并给每项服务分配一个整数编号。用户在进入前把编号和参数放进约定位置；内核先读取编号，再选择处理函数。这个“编号放在哪里、参数怎样排列、结果怎样返回”的约定属于应用二进制接口（application binary interface, ABI）。这里先引入 ABI，是因为跨边界的两段代码可能由不同编译器、不同语言甚至不同年代生成，它们不能依赖 C 函数名、类型系统或源代码声明来彼此理解。

一个教学用的最小请求可以表示为：

```
boundary request:
  operation_number
  argument_0 ... argument_5
  user_continuation
  result_slot
```

operation_number 回答“请求什么”；参数回答“对哪个对象、使用哪些数值”；用户继续位置回答“处理完回到哪里”；结果槽回答“内核如何把成功范围或失败原因交

还给调用者”。这四类状态不会因为采用寄存器或内存传递而消失。真实 ABI 只是为它们选择了具体载体。

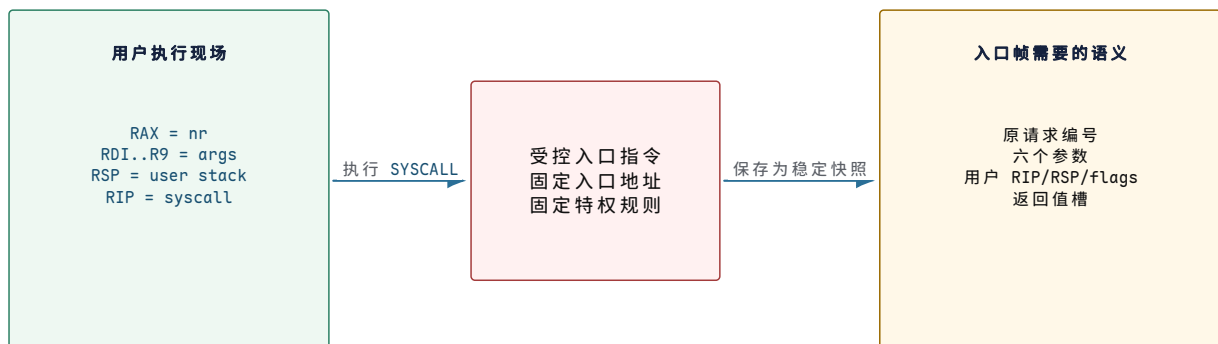
边界信息	最小作用	若缺失会发生什么
操作编号	从统一入口选择服务	内核不知道调用者希望修改哪类状态
参数集合	描述对象、地址、长度和选项	处理函数只能执行固定动作，无法服务不同对象
继续位置	保存用户程序下一步	内核完成后无法恢复原控制流
结果或错误	说明已经完成的范围	调用者无法区分成功、短完成和失败

x86-64 为什么选用这些寄存器

以常见的 x86-64 Linux ABI 为具体实例。进入前，RAX 保存系统调用号，六个整数或指针参数依次放在 RDI、RSI、RDX、R10、R8 和 R9。返回值仍放在 RAX。第四个参数使用 R10 而不是普通函数 ABI 常用的 RCX，原因不是任意偏好：SYSCALL 指令会把用户返回地址写入 RCX，原有 RCX 内容不能继续承担第四参数。

寄存器	边界含义	进入后为什么仍要保存
RAX	系统调用号，返回时改为结果	原编号要保留在 orig_ax 一类字段中，供审计、重启和跟踪使用
RDI--R9	最多六个直接参数	处理函数、过滤器或跟踪器可能在不同阶段读取它们
RCX	SYSCALL 自动写入用户返回 RIP	返回前要恢复用户下一条指令地址
R11	SYSCALL 自动写入用户 RFLAGS	返回时要恢复允许返回的用户标志
RSP	进入前仍指向用户栈	SYSCALL 不自动保存或切换它，入口代码必须先保护其值

这是一项体系结构相关的实例，不是所有系统的唯一布局。AArch64 常用 x8 保存系统调用号、x0--x5 保存参数，RISC-V 常用 a7 保存编号、a0--a5 保存参数。稳定的操作系统语义是“编号、参数、继续状态和结果必须有约定位置”；具体寄存器由体系结构 ABI 决定。



图示：边界契约先于具体处理函数。寄存器是用户提交请求时的临时载体；入口帧把这些值转成内核可以在检查、阻塞和调度期间持续引用的状态。

返回值为为什么要同时表达数量和错误

仅用“成功/失败”一位无法描述许多真实结果。请求读取 4096 字节时，文件末尾可能只剩 600 字节；向管道写入 4096 字节时，信号到达前可能已经提交 1024 字节。若内核只返回“失败”，调用者会重写已经完成的前缀；若只返回“成功”，调用

者又会误以为全部完成。因此，字节流接口通常用非负数表达已完成数量，用负的错误编号表达“没有可优先报告的完成前缀”。

x86-64 Linux 的原始系统调用约定把 `-EACCES`、`-EFAULT` 等负值放入 `RAX`。用户态 C 库包装层通常把它转换为返回 `-1`，并把正的错误编号写入线程局部的 `errno`。这两个边界要分开理解：内核 ABI 返回负错误编号，C 库 API 再提供符合 C 语言习惯的表示。

原始结果	调用者应如何解释	状态含义
4096	全部推进	本次请求的 4096 字节已达到该接口的完成边界
600	短完成	前 600 字节已提交；剩余部分要由调用者决定是否继续
0	依接口解释	<code>read</code> 可表示 EOF；其他接口可能表示成功但无数量
<code>-EINTR</code>	可按接口规则重试	信号在没有更优先的部分完成结果时打断等待
<code>-EFAULT</code>	修正用户地址	参数复制没有形成可报告的成功前缀

结果槽因而也是提交账本。处理函数不能先修改对象偏移 4096 字节，再在后半段故障时只返回 `-EFAULT`，否则调用者不知道哪些字节已经生效。后文讨论用户内存时会继续沿这条不变量分析部分复制。

结构体参数为什么需要长度与版本

六个寄存器足以传递少量整数，却不适合直接承载可扩展配置。常见做法是让一个参数指向用户结构体。结构体一旦成为 ABI，字段偏移就可能被旧程序长期依赖；在中间插入字段会改变后续偏移，扩大指针或时间类型也会改变布局。边界结构因此常带 `size`、`version`、`flags` 和保留字段。

```
struct request_v1 {
    uint32_t size;
    uint32_t flags;
    uint64_t object_id;
    uint64_t user_buffer;
    uint64_t length;
    uint64_t reserved[3];
};
```

`size` 让内核知道用户实际提供到哪个字段；保留字段要求用户清零，给未来扩展留下可验证空间；未知 `flags` 必须拒绝，而不是静默忽略可能改变安全语义的请求。内核把已知前缀复制到清零后的内部结构，再验证范围和组合。这样，新内核能识别旧结构，旧内核也能明确拒绝自己不理解的新语义。

```
copy_versioned_request(user_ptr):
    copy fixed header containing size and flags
    reject size smaller than mandatory prefix
    reject unsupported flag bits
    zero initialize kernel request
    copy min(user_size, known_size) bytes
    require unknown tail or reserved fields to be zero
    validate every length, offset and object reference
```

这里的内部结构不是用户结构的永久别名。复制完成后，内核拥有一份不随用户线程并发修改的快照；内部字段还可以使用内核指针、引用对象和已检查长度，而不受 ABI 对齐规则限制。

用户控制流怎样变成可恢复的入口现场

处理器自动完成的动作其实很少

现在已有编号和参数，但入口仍不能安全调用普通内核函数。进入前的 `RSP` 指向用户栈，`RCX` 和 `R11` 即将被覆盖，其他通用寄存器仍含用户值。若入口代码立即 `push`，它会把内核状态写到用户可控制的地址；该页可能未映射，也可能被另一线程改写。系统必须先区分处理器自动建立的最小状态与入口软件随后建立的完整状态。

在典型 x86-64 配置下，`SYSCALL` 的关键动作是：

1. 把下一条用户指令地址写入 `RCX`。
2. 把用户 `RFLAGS` 写入 `R11`，并按 `IA32_FMASK` 清除指定标志。
3. 从预先配置的 MSR 取得内核入口 `RIP` 和代码段相关属性。
4. 进入内核权限级并从入口 `RIP` 继续。

它不会自动把全部寄存器压入某个帧，也不会像某些异常门那样自动把用户 `RSP` 换成任务内核栈。用户 `RSP` 仍然只存在于寄存器里。入口最早的几条指令必须在不依赖用户栈的条件下暂存它，再取得当前任务的内核栈顶。

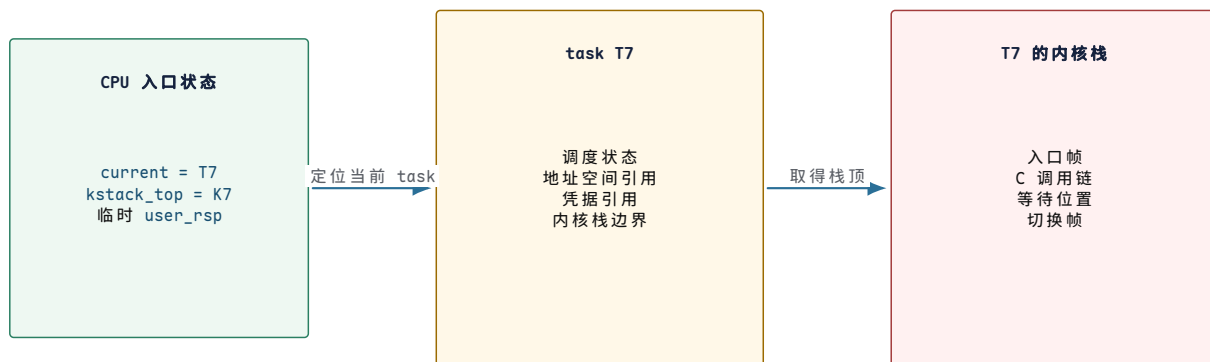
状态	谁首先保存	为什么不能省略
用户 <code>RIP</code>	处理器写入 <code>RCX</code>	决定系统调用返回后的下一条用户指令
用户 <code>RFLAGS</code>	处理器写入 <code>R11</code>	保留用户可恢复标志，同时阻止危险标志影响入口
用户 <code>RSP</code>	入口软件写入每 CPU 暂存区或等价位置	<code>SYSCALL</code> 不自动保存，换栈前又不能使用用户栈
参数与通用寄存器	入口软件写入内核栈帧	C 处理函数、信号与重启路径需要稳定快照
内核继续位置	普通调用链与调度切换帧保存	任务在内核中阻塞后必须从原等待点继续

最早入口为何依赖每 CPU 状态

换栈前不能从当前用户栈读取内核对象。入口却必须找到“当前 CPU 正在运行哪个 task”以及“这个 task 的内核栈顶在哪里”。一种常见组织是在每个 CPU 的受保护区域保存当前 task 指针、入口临时栈指针和用户 `RSP` 暂存槽。x86-64 内核还可能使用 `SWAPGS` 在用户与内核的 `GS` 基址之间切换，使固定偏移能够访问这组每 CPU 数据。

```
per_cpu_entry_state:
    current_task
    kernel_stack_top
    saved_user_rsp
    entry_scratch
    cpu_id
```

这些字段属于当前处理器的入口执行环境，不等于 task 自身。CPU A 的 `current_task` 会随着调度改变；task T 的内核栈则随 T 的生命周期存在，T 迁移到 CPU B 后仍使用自己的内核栈。把两者混为一体会产生一个常见误解：好像内核栈属于 CPU。实际上，每 CPU 状态只负责在入口最早阶段找到当前 task，长期保存阻塞调用链的是线程私有内核栈。



图示：每 CPU 入口状态只解决“此刻是谁在这颗 CPU 上进入内核”；task 和它的内核栈解决“这条执行流阻塞或迁移后怎样继续”。

为什么入口帧不能只保存返回地址

若处理函数永不阻塞、没有信号、没有调试、没有审计，也不调用会改寄存器的函数，保存 `RCX` 和 `R11` 似乎足够。真实内核不具备这些前提。处理函数遵循内核函数调用约定，会覆盖调用者保存寄存器；page fault 或中断可能嵌套；任务可能在等待数据时被调度出去；信号处理需要修改用户返回现场；调试器甚至可能观察或修改参数与结果。

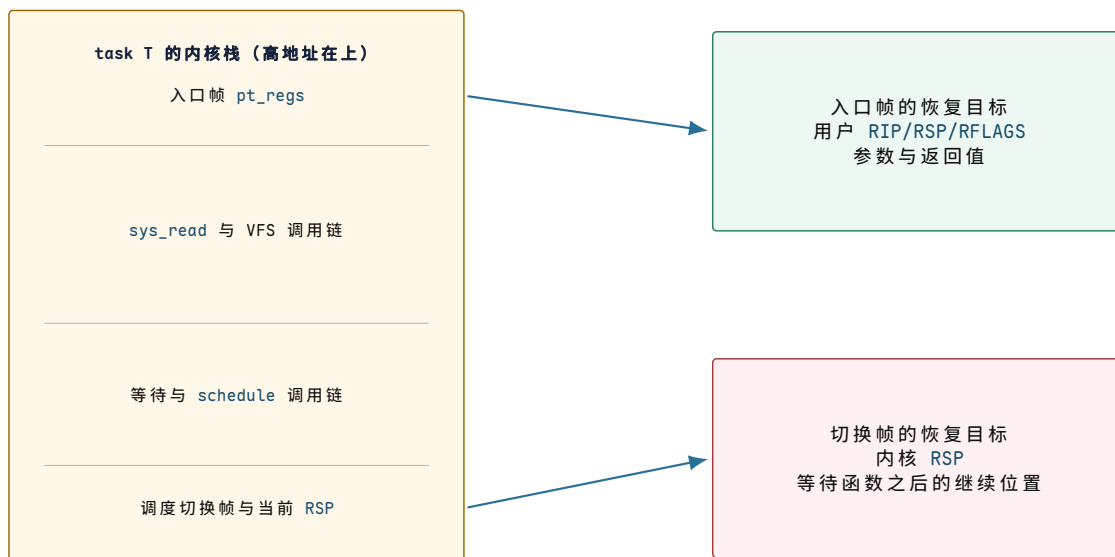
入口因此在内核栈上建立一个稳定的用户寄存器快照。Linux 常用 `struct pt_regs` 一类结构承载它。下面是按语义分组的教学布局，不承诺某个内核版本的精确字节偏移：

字段组	代表性字段	使用者与必要性
请求身份	<code>orig_ax</code>	保留原系统调用号，重启、 <code>seccomp</code> 、审计和 <code>ptrace</code> 需要它
直接参数	<code>di, si, dx, r10, r8, r9</code>	分发包装层从稳定帧读取，而不是依赖稍后已变化的物理寄存器
通用现场	<code>r15...rbx, bp</code>	信号、调试、异常嵌套和返回路径需要完整用户可见状态
返回结果	<code>ax</code>	处理函数写入成功数量或负错误编号
控制现场	<code>ip, cs, flags, sp, ss</code>	验证并恢复用户指令、栈和权限相关状态

`orig_ax` 与 `ax` 必须分开。进入时两者都可源于用户 `RAX`；分发后 `ax` 被改成结果，`orig_ax` 仍保存原编号。若信号路径决定重启调用，它需要知道原请求而不是把返回值误当成编号。

入口帧和调度切换帧为何是两类结构

入口帧回答“怎样回到用户态”；调度切换帧回答“怎样继续一条内核控制流”。任务 `T` 在 `read` 中睡眠时，入口帧位于 `T` 的内核栈较高位置，保存最初用户状态；更靠近当前栈顶的位置保存内核函数调用链和调度器切换所需的被调用者保存寄存器。调度器切到任务 `U` 时，不需要把 `T` 的整个 `pt_regs` 复制到 `U`；它只保存足以恢复 `T` 内核栈指针和内核继续地址的切换现场。



图示：一次阻塞系统调用同时保留两种继续关系。调度先用切换帧恢复内核控制流；处理函数结束后，返回路径才使用入口帧恢复用户控制流。

这一区分也解释了为什么“系统调用导致上下文切换”不是必然事实。进入内核只建立入口现场；若处理函数立即完成，仍由同一 task 在同一 CPU 上返回，不发生任务调度切换。只有调用阻塞、时间片耗尽或更高优先级任务需要运行时，调度器才另行保存内核切换现场并选择其他 task。

从用户指令到完整入口帧的逐步状态

把前述局部结构合在一起，可以得到入口的第一条完整但尚未分发的 trace。假设 task T7 请求 `getpid`，系统调用号已放入 `RAX`，用户下一条指令地址为 `0x40102a`，用户栈顶为 `0x7fff...e800`。

阶段	当前栈	新形成的状态	尚未发生的动作
执行入口前	用户栈	<code>RAX=nr</code> ，参数寄存器有效	未进入内核，未检查请求
<code>SYSCALL</code> 后	<code>RSP</code> 仍是用户值	<code>RCX=0x40102a</code> ， <code>R11=user flags</code>	未自动保存其余寄存器，未自动换栈
入口最早阶段	不使用用户栈	暂存用户 <code>RSP</code> ，取得 T7 与 K7 栈顶	尚未调用 C 代码
切到 K7 后	T7 内核栈	按约定压入完整入口帧，写入 <code>orig_ax</code>	尚未根据编号选择服务
入口帧完成	T7 内核栈	内核已有稳定用户状态快照	尚未证明参数、对象或权限合法

入口帧完成只是把不稳定寄存器转为稳定的内核记录，不代表请求已经获准。下一步才是从 `orig_ax` 找到候选处理函数，并让过滤、参数复制和权限规则逐层缩小请求的可执行范围。

核心思想

受控入口首先是一项状态保存协议。处理器只提供最小权限转换和两个返回寄存器；入口软件必须在不信任用户栈的前提下找到当前 task、切到线程内核栈，并构造能够经历函数调用、阻塞、调度、信号和调试的入口帧。入口成功不等于请求获准，更不等于操作完成。

入口帧之后为何还要分发与逐层验证

服务增多以后需要从编号找到实现

最初只有“读取进程编号”和“调整系统时钟”两项服务时，入口可以用两个条件分支选择实现。当服务增长到数百项，连续条件分支既难维护，也无法直接验证编号是否落在合法范围。编号已经是一个稠密或近似稠密的整数空间，因此可以把“编号到处理函数”的关系保存成数组。这张数组称为系统调用分发表。

```
syscall_table:
[NR_read]      -> sys_read
[NR_write]     -> sys_write
[NR_getpid]    -> sys_getpid
[NR_clock_settime] -> sys_clock_settime
```

分发表保存的是候选入口，不是授权清单。编号合法只说明内核认识这项操作；当前主体是否有权执行、参数是否有效、目标对象是否处于允许状态，都要由后续路径判断。若把“表项存在”误当成“请求获准”，任何进程都能调用修改时钟、挂载文件系统或控制网络配置的服务。

分发阶段检查	能证明什么	不能证明什么
编号非负且小于表长	访问分发表不会越界	该编号在当前 ABI 中一定实现
表项不是空缺处理函数	内核有候选实现	调用者具备对象权限
ABI 类型匹配	参数能按正确位宽和顺序解释	参数值、用户地址和长度合法
入口来源是用户态	当前帧可按用户返回规则处理	用户提交的内容可信

越界或未实现编号通常返回 `-ENOSYS`。内核不能让用户编号直接变成未检查的函数指针或数组越界索引。投机执行缓解还可能在范围检查后增加索引约束，防止处理器短暂沿错误路径读取越界表项；这是在基本范围检查正确之后，为微体系结构信息泄露增加的防护。

为什么分发包装层要从入口帧取参数

入口汇编已经把参数保存进 `pt_regs`。分发层可以按 ABI 从帧中取出六个值，再调用类型化包装函数。这样，体系结构相关的寄存器排列集中在边界层，主体实现只面对普通整数、句柄和用户地址标记。

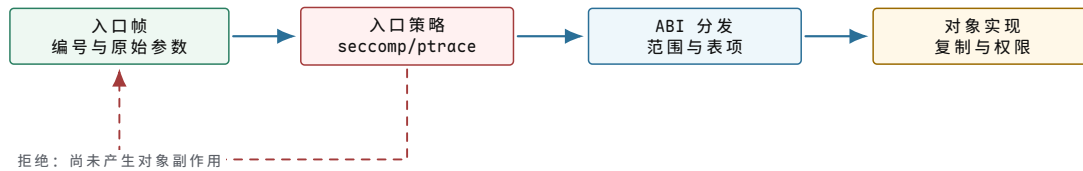
```
dispatch(regs):
    nr = regs.orig_ax
    if nr outside implemented range:
        regs.ax = -ENOSYS
        return
    args = decode_abi_arguments(regs)
    regs.ax = syscall_table[nr](args)
```

这里的 `args` 仍然只是未经信任的数值。若某个值表示长度，处理函数要检查上限和加法溢出；若表示 `fd`，要取得稳定文件引用；若表示用户指针，要复制或固定页面；若表示 `flags`，要拒绝未知位。包装层完成“按 ABI 解码”，具体实现完成“按对象语义验证”，两者不能由一次统一检查替代。

入口过滤为什么位于实际处理函数之前

有些进程只应使用系统调用集合中的一小部分。例如，负责解析不可信图片的沙箱进程可能只需读写已有 `fd`、分配内存和退出，不需要创建网络连接或加载内核模块。若等到处理函数已经取得对象锁、分配资源之后再拒绝，撤销成本高，也容易遗漏副作用。因此内核可以在分发前后设置一组入口工作项：

1. 根据 task 的跟踪状态决定是否通知调试器。
2. 让 seccomp 过滤器观察系统调用号和原始参数，决定允许、返回错误、通知监管者或终止。
3. 建立审计上下文，记录主体、编号和稍后形成的结果。
4. 在检查通过后调用实际处理函数。



图示：入口过滤处理原始请求面，对象权限处理已解析的目标。前者适合收缩可调用操作集合，后者才能判断“这个主体是否能操作这个对象”。

seccomp 看到的指针通常只是一个数值，不能安全遍历任意复杂对象语义；LSM 钩子则位于目标对象已解析的位置，可以检查 inode、socket、task 等内核对象。把两者放在不同阶段不是重复，而是因为它们拥有的稳定信息不同。

最小 getpid 如何完成而不触碰用户内存

getpid 适合作为第一条完整分发例子，因为它没有用户指针，也通常不会阻塞。进入前，用户包装函数只提交系统调用号。入口建立帧后，分发层找到处理函数；处理函数从当前 task 的进程标识关系中取得对调用者可见的 PID；结果写入 regs.ax，随后进入返回检查。

步骤	读取状态	写入状态	边界含义
1	用户 RAX=NR_getpid	RCX/R11 与入口临时状态	权限转换完成，请求尚未分发
2	每 CPU current	T7 内核栈上的入口帧	用户现场成为稳定快照
3	orig_ax	选择 sys_getpid	编号已解析，但结果尚未形成
4	T7 的 PID namespace 关系	regs.ax=visible_pid	服务结果形成
5	返回工作标志与入口帧	用户 RAX=visible_pid	用户重新获得控制权

即便这条路径不复制用户内存，返回值仍可能相对 PID namespace 解释。处理函数不能简单返回一个不考虑调用者视图的全局整数。这个例子提前暴露一项通用规则：请求结果经常依赖当前 task 所在的命名空间或凭据视图，入口帧本身不会替处理函数完成这类对象语义。

检查顺序为什么必须服从信息依赖

系统调用常有多项检查，但顺序不能只按代码整洁程度排列。某项检查若需要稳定对象，就必须在取得对象引用之后；某项检查若能在分配资源前拒绝，就应尽量提前；某项检查若会泄露对象是否存在，还要遵守接口规定的错误优先级。

以“通过 fd 写数据”为例，可以先形成如下依赖，而不急于展开文件系统：

1. 验证长度能否在内部类型中表示，避免后续计算溢出。
2. 用 fd 槽位取得稳定的打开对象引用；失败时返回 -EBADF。
3. 检查该打开对象是否允许写以及对象当前状态是否接受写入。
4. 把用户数据转为内核可控制的字节或页面引用。

5. 让对象实现提交尽可能长的前缀，并返回实际数量。
6. 释放临时页面、缓冲区和对象引用。

有些实现会为性能调整复制与对象检查的具体先后，但必须保持两个不变量：在对象使用期间持有稳定引用；在报告错误时，已经提交的前缀不能从结果中消失。后续小节分别推导这两项结构。

用户地址为何必须转换成内核拥有的状态

一个地址数值同时缺少三类保证

用户把 `0x7fff...1000` 放入参数寄存器时，内核得到的只是一个整数。这个整数没有附带三项保证：

1. 当前地址空间是否存在覆盖它的合法区间。
2. 对应页面此刻是否已建立、是否允许所需方向的访问。
3. 从检查到使用期间，另一个线程是否会改写映射或内容。

第一项属于地址范围与 VMA 策略，第二项属于当前页表和缺页处理，第三项属于并发快照与对象生命周期。仅检查“地址小于用户空间上界”最多排除明显的内核地址，不能证明页面已经可读，更不能固定内容。

动作	得到的保证	仍然缺少的保证
<code>access_ok(ptr, n)</code>	范围形式上位于允许的用户地址域且不溢出	页面存在、权限成立、内容稳定
逐页访问或复制	每个实际访问字节要么成功，要么进入可修复 fault 路径	复制后的用户原位置不再变化
复制到内核缓冲区	内核拥有当时的内容快照	原用户内容随后是否改变；通常已不再重要
固定用户页	页面在异步期间不会被回收或换成另一页	用户是否还能改页内容、设备访问何时结束

先检查再直接解引用为何仍然有竞态

假设线程 A 先验证用户页可读，随后在解析结构时反复从用户地址取字段；线程 B 可以在两次读取之间改写长度或指针。A 第一次看见长度 16，分配 16 字节；第二次又从用户地址读到长度 4096，再复制 4096 字节，就会越过已分配缓冲区。这类“检查时与使用时不一致”称为 **检查-使用时间竞争**（time of check to time of use, TOCTOU）。

```
unsafe:
if user_header.length <= 16:
    kbuf = allocate(user_header.length)
    copy(kbuf, user_header.data, user_header.length)
```

修复的关键不是增加一次相同检查，而是改变状态所有权：先把固定大小的头复制到内核局部变量，所有后续判断都读取这份快照；长度通过检查后，再复制对应数据。若协议允许用户在操作期间并发更新共享内存，就要使用明确的序列号、原子字段或锁协议，而不是假设普通读取构成快照。

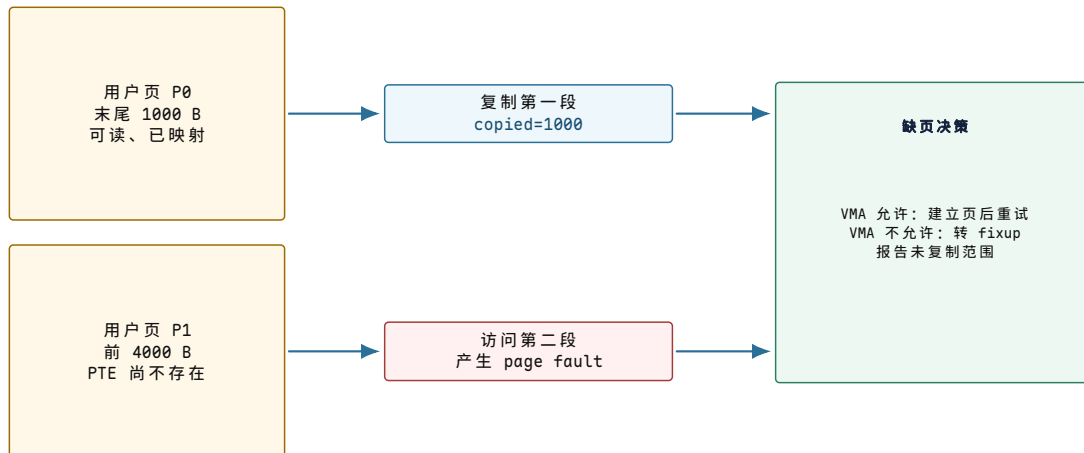
```
safer:
khdr = zero_initialized_header()
copy_fixed_header_from_user(&khdr, user_header)
validate(khdr.length, khdr.flags)
kbuf = allocate(khdr.length)
```



```
copy_bytes_from_user(kbuf, khdr.data, khdr.length)
```

跨页复制为什么可能在中间故障

请求复制 5000 字节时，起始地址可能落在某页最后 1000 字节，剩余 4000 字节位于下一页。第一页可读并不能证明第二页可读。第二页可能尚未建立 PTE，但 VMA 允许按需分配；此时 page fault 可以建立页面后重试。也可能根本没有覆盖第二页的 VMA；此时复制必须收束为错误或部分完成。



图示：用户复制以实际访问为准逐页推进。范围检查不是访问保证；可修复缺页与不可访问地址必须在 fault 路径中分开。

异常表怎样把预期 fault 变成 EFAULT

普通内核指针访问坏地址通常说明内核错误。用户复制却有意访问不可信地址，fault 是接口必须处理的输入结果。内核需要一种方法区分这两类内核态 fault。常见方案是在异常表中记录“复制指令地址 → 修复地址”的映射。

```

copy instruction at A:
  load from user address

exception table entry:
  fault_ip A -> fixup_ip F

page fault handler:
  if fault cannot be resolved and current ip has fixup:
    set instruction pointer to F
    continue with "bytes not copied" result
  else:
    follow ordinary kernel fault policy
  
```

异常表不让错误页面 magically 变得可读。它只提供一个受控退出点，使复制 helper 能报告剩余字节，而不是把预期的坏用户参数升级成内核崩溃。上层系统调用再根据自己的提交语义，把 helper 结果转换为 `-EFAULT`、短完成数量或其他接口结果。

部分复制和部分系统调用完成不是同一层结果

许多底层复制 helper 返回“还有多少字节未复制”，而系统调用返回“已完成多少字节”或负错误。两者方向不同，且中间还隔着对象提交。例如，内核成功从用户地址复制 1000 字节，不代表文件对象已经接受这 1000 字节；相反，对某些分段写路径，前一批字节可能已经提交，后一批复制才遇到 fault。

时刻	用户复制进度	对象提交进度	可返回结果
----	--------	--------	-------

尚未复制	0/5000	0/5000	若立即失败，返回负错误
内核快照完成	5000/5000	0/5000	仍可能因对象状态失败
第一段提交	5000/5000	1000/5000	后续失败时通常优先返回 1000
全部提交	5000/5000	5000/5000	返回 5000

设计处理函数时，应分别维护 `copied_from_user` 与 `committed_to_object`，不能用一个计数器同时代表两种进度。最终返回值通常由对象提交进度决定，因为它告诉调用者重试时应从哪里继续。

输出参数为什么要先在内核中完整构造

有些系统调用把结构体写回用户地址。若内核逐字段直接写用户结构，写到一半遇到坏页，用户可能观察到新旧字段混合；若结构体含未初始化 padding，还可能泄露内核栈内容。稳妥路径是先在清零的内核对象中构造完整结果，设置明确版本和长度，再进行一次边界复制。

```
produce_user_result(user_out):
    result = zero_initialized()
    fill documented fields
    result.size = documented_size
    validate output range
    if copy_to_user(user_out, &result, result.size) fails:
        return -EFAULT
    return 0
```

如果接口规定输出可以部分形成，就必须明确每个字段的有效条件或返回实际数量。默认情况下，让用户看到一个完整旧结果或完整新结果，比暴露半个结构更容易形成稳定契约。

嵌套指针为什么要逐层形成快照

`execve` 的 `argv`、`sendmsg` 的 `iovec`、若干 `ioctl` 结构都含用户指针数组。复制外层结构只冻结了指针数值，没有冻结每个指针指向的内容。内核要先限制数组个数和总长度，复制指针数组，再逐项复制或固定实际数据；每次加法和乘法都要检查溢出。

```
copy_iovecs(user_iov, count):
    reject count above interface limit
    allocate zeroed kernel iovec array
    copy outer array once
    total = 0
    for each copied entry:
        reject length or total + length overflow
        validate address range for requested direction
        total += length
    return stable kernel array and checked total
```

外层数组、内层数据和总长度是三个不同状态。异步接口若在系统调用返回后仍使用内层页面，还必须把页面固定和解除固定的责任交给长期请求对象；单纯保存用户指针不延长页面生命周期。

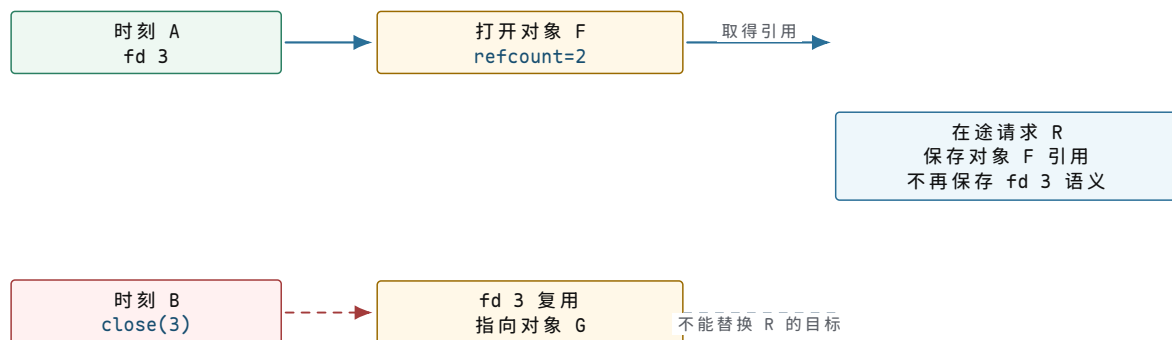
核心思想

用户指针不是内核对象引用。范围检查只排除明显非法数值，实际复制还要处理逐页 `fault`；复制到内核缓冲区才能形成内容快照，固定页面才能延长页面身份，但仍不自动禁止用户修改。系统调用返回值必须依据对象已提交范围，而不是把底层复制计数直接照搬给用户。

对象引用怎样封闭检查与使用之间的竞态

fd 整数为何不能作为长期对象身份

用户传入 `fd 3` 时，内核不能把整数 3 保存到异步请求里，稍后再查一次。线程 B 可能在此期间关闭 `fd 3`，另一次 `open` 又复用同一槽位。稍后查询得到的可能是完全不同的对象。因此，系统调用在开始使用对象时要从 `fd table` 取得指向打开文件对象的稳定引用，并增加引用计数；后续路径持有对象引用，而不是反复解释整数。



图示：fd 是可复用槽位，不是永久身份。请求 R 在时刻 A 取得 F 的引用后，关闭与槽位复用只改变 `fd table`；F 要等 R 也释放引用后才能销毁。

取得引用和删除槽位为什么需要同步

`fget(fd)` 与 `close(fd)` 可以并发。若读取槽位指针和增加对象引用之间没有保护，`close` 可能删除最后一个槽位引用并释放对象，读取方随后对已释放内存加引用。实现可以用锁保护这两个动作，也可以使用 RCU、原子引用和重试协议；无论采用何种技术，都必须形成同一语义：

1. 查询要么在槽位仍有效时取得一个真实对象引用。
2. 要么观察到槽位已经关闭并返回 `-EBADF`。
3. 不允许返回一个已开始销毁且无法安全使用的裸指针。

```

lookup_fd_with_reference(fd):
    enter fd-table read-side protection
    file = read slot fd
    if file exists and reference can be acquired:
        leave protection
        return file
    leave protection
    return EBADF
  
```

引用计数解决“对象何时可以释放”，不自动解决“对象内部字段怎样并发修改”。取得 `file` 引用后，偏移、`flags`、队列或文件系统状态仍按各自锁和原子规则保护。生命周期同步与内容同步是两项不同职责。

打开对象为什么还要与 inode 分开

两个进程可以分别打开同一路径。它们需要共享文件身份、长度和权限元数据，却可以拥有不同当前偏移和打开 `flags`。若把所有状态放在 `inode` 中，独立打开者

会相互推进偏移；若只保存打开对象，又无法让多个打开者共享同一文件身份。因此，fd 槽位引用一次打开实例，打开实例再引用持久文件身份。

层次	本章需要的边界状态	为什么不能由相邻层替代
fd 槽位	槽位引用、close-on-exec 等 fd 级标志	整数会复用，不适合保存共享偏移或文件身份
打开对象	当前偏移、打开模式、引用计数、操作方法	每次独立打开需要自己的可变访问状态
inode 等对象身份	所有者、mode、长度和内容索引入口	多次打开和多个名字可以共享同一身份

这里不展开文件系统布局，只建立权限检查所需的对象边界。完整 inode 为什么存在、磁盘上保存哪些字段，将由文件系统部分从块分配问题重新推导。

路径字符串为什么要转换成稳定路径对象

字符串 `"/tmp/report"` 也不是永久对象身份。另一个线程可以在检查和打开之间 rename 路径组件，挂载关系也可能改变。内核的路径遍历因此逐级取得 dentry、mount 和 inode 等稳定引用，并在需要的位置检查目录搜索权限、符号链接策略和最终对象权限。检查必须绑定实际解析到的对象，而不是先检查一个字符串，稍后重新解析同一字符串使用。

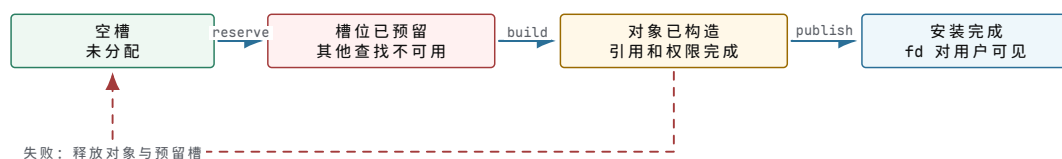
不安全步骤	并发变化	结果
检查路径字符串对应普通文件	攻击者把末级名字替换成符号链接	使用阶段可能到达检查时未授权的目标
检查父目录后释放引用	父目录被移动到另一挂载关系	后续创建发生在不同可见树中
凭名字再次查找已检查对象	原名字删除并复用到新 inode	检查与使用作用于两个身份

`openat` 允许调用者用目录 fd 作为起点，减少依赖易变的当前工作目录；`openat2` 一类接口还能明确限制符号链接、挂载穿越和解析范围。这些接口不是随意增加 flags，而是在基本路径解析遇到具体竞态和隔离需求后形成的约束。

对象发布为何要区分保留槽位与安装引用

创建新 fd 的系统调用既要建立打开对象，也要在当前 fd table 中选择空槽。若先把半初始化对象发布到槽位，另一个线程可能立即使用未完成对象；若所有工作完成后才查找槽位，又可能因 fd 上限失败并浪费已经取得的外部资源。常见协议把过程拆成：

1. 预留一个尚不可使用的 fd 编号。
2. 构造并验证打开对象，取得所需引用。
3. 只有对象完全可用时，原子地把对象安装到预留槽位。
4. 任一步失败都释放预留槽和临时对象。



图示：“找到空编号”和“让编号引用有效对象”是两个边界。预留防止竞争分配，发布点保证其他线程只看到完整对象。

引用取得之后权限仍可能随状态变化

稳定引用保证对象身份不被替换，却不保证操作永远允许。文件系统可能被重新挂成只读，对象可能进入关闭或撤销状态，凭据可能在调用前已经改变，强制访问控制策略也可能拒绝操作。处理函数需要在与当前操作相符的位置检查动态状态，并用对象锁、序列号或重试规则保证检查和提交作用于同一状态版本。

这项原则可写成一个三段账本：

边界	必须稳定的事实	典型保护
身份边界	使用期间仍是同一个打开对象、inode 或 socket	引用计数、RCU、对象生命周期锁
授权边界	权限判断读取一致的主体和对象安全状态	不可变凭据快照、对象锁、LSM hook 位置
提交边界	状态更新与返回数量一致	操作锁、事务、短完成计数、错误回滚

只有三项都成立，系统调用才不会出现“检查了 A、使用了 B”或“返回失败但已不可见地修改”的边界错误。下一节继续推导授权边界中的主体状态。

权限判断为何需要独立的凭据快照

只在 task 中保存一个 uid 为什么不够

先设想最简单的文件权限：每个 task 只有一个用户编号，每个对象保存所有者编号和“所有者可读”位。读取时比较两个编号即可。这个方案遇到第一个实际需求就不够了：一个登录用户执行某个受控管理程序时，程序可能需要暂时使用文件所有者权限，完成后再恢复原身份。系统既要记住“是谁发起”，又要记住“本次访问按谁的权限判断”。

因此，Unix 风格凭据至少区分真实身份与有效身份。真实用户 ID(real UID)描述进程来源；有效用户 ID(effective UID)通常参与访问检查；保存的 set-user-ID 让受控程序能够在规则允许时切换有效身份并恢复。文件访问还有 filesystem UID 一类实现字段，用于把文件系统检查与某些临时身份变化分开。

凭据字段	回答的问题	为什么不能只保留一个 uid
uid	谁启动或拥有这个进程来源	审计来源不应随临时授权一起消失
euid	当前访问通常按谁判断	受控程序需要暂时获得或放权限
suid	允许恢复到哪个受控身份	若只覆盖原值，就无法安全恢复
fsuid	文件系统检查采用哪个身份	让特定服务缩小文件访问身份而不改变其他语义
gid/egid/groups	当前属于哪些组	一个用户可能通过多个组获得对象权限

字段增多不是历史装饰，而是不同语义不能安全复用一个变量。若临时提高 euid 时也覆盖真实来源，审计记录会把请求错误归因给特权身份；若没有可恢复字段，程序只能永久放弃或永久保留权限。

为什么凭据要成为 task 引用的独立对象

同一进程中的多个线程通常共享身份语义。若每个 task 都逐字段保存一份凭据，改变身份时必须原子更新多个线程；读取方可能看见一半旧字段、一半新字段。内核更容易维护的组织是：task 持有指向凭据对象的引用，凭据对象发布后保持不

可变；身份改变时先复制旧凭据、修改新副本、完成全部检查，再一次性替换 task 或线程组的凭据指针。

```
credential update:
  old = current.cred
  new = copy(old)
  apply requested uid/gid/capability changes to new
  validate transition using old authority
  publish current.cred = new
  release old reference after readers leave
```

这是一种写时复制和发布协议。读取权限的热路径只需取得一个稳定凭据指针，便能在整个检查期间看到自洽字段集合；写路径承担复制和发布成本。RCU 或等价读侧保护可延迟旧对象回收，避免并发权限检查使用已释放凭据。



图示：权限读取使用不可变快照。身份变化不是原地逐字段修改 C0，而是构造 C1 并发布新引用；旧读者离开后 C0 才能回收。

当前凭据与打开时凭据为何可能不同

处理当前系统调用时，`current_cred` 描述调用者此刻的主体。某些长期对象还要记住创建或打开时的安全上下文。例如，文件被打开后通过 `fd` 传给另一个进程，后者使用的打开模式来自原打开对象，但每次操作是否还需按当前凭据检查取决于接口与安全策略。异步工作若在内核工作线程中执行，也不能误用工作线程自身的管理身份替代原请求者。

因此，要明确两种凭据角色：

- 主体凭据：当前发起检查的 task 具有谁的权限。
- 对象或请求凭据：对象创建、文件打开或异步请求提交时捕获的安全上下文。

异步请求若需要延后执行权限相关操作，应显式持有提交者凭据引用或已经授权的对象引用。不能在工作线程运行时重新取 `current`，因为那时 `current` 已是内核工作线程，它的权限通常比用户请求者更高。

mode 位如何从一个具体访问问题得到

已有稳定主体身份和对象身份后，最小文件检查可以把访问者分为所有者、所属组和其他主体三类。每类分别保存读、写、执行位。执行位对目录通常表示搜索或穿越能力，而不表示读取目录项内容；因此目录的读权限与搜索权限要分开。

请求	所需对象状态	不能替代它的检查
读取普通文件内容	匹配主体类别的读位或等价授权	父目录可搜索不代表文件可读
写普通文件内容	写位、打开模式以及非只读文件系统状态	拥有目录写权限不代表可改已有文件内容
查找目录中的名字	目录搜索/执行权限	能列出目录名不一定能穿越到目标

在目录中创建名字	目录写与搜索权限、挂载和策略约束	新文件 mode 尚未存在，不能检查它来授权创建
----------	------------------	--------------------------

这张表说明权限必须在拥有相应对象语义的层检查。路径遍历检查每级目录，最终打开检查目标对象，写路径还检查打开实例和文件系统当前状态。把这些条件压成一个早期 `if` 会缺少尚未解析出来的对象信息。

ACL 为什么在三类 mode 不够时出现

若项目文件要允许用户 Alice 读、用户 Bob 写、组 review 只读，而其他人无权访问，仅靠所有者/组/其他三类位无法表达。改变文件所属组会影响其他组员，为每种组合创建新组也会迅速失控。访问控制列表（access control list, ACL）为指定用户和组增加条目，并用 mask 约束一组条目的最大有效权限。

ACL 没有取消 mode 位。系统需要规定所有者条目、命名用户条目、组条目、mask 与 other 条目的求值顺序，并把 mode 与 ACL 的可见表示保持一致。权限检查因此先确定主体与对象，再按对象是否有 ACL 选择求值规则。

为何不能让 root 永远绕过所有检查

单一 uid 0 能简化早期管理，却把挂载、网络配置、DAC 绕过、进程控制、模块加载等无关能力集中到一个主体。一个只需绑定低端口的服务一旦被攻破，就可能同时获得修改文件和控制内核的能力。由此产生的改进是把全能身份拆为若干能力（capability）。

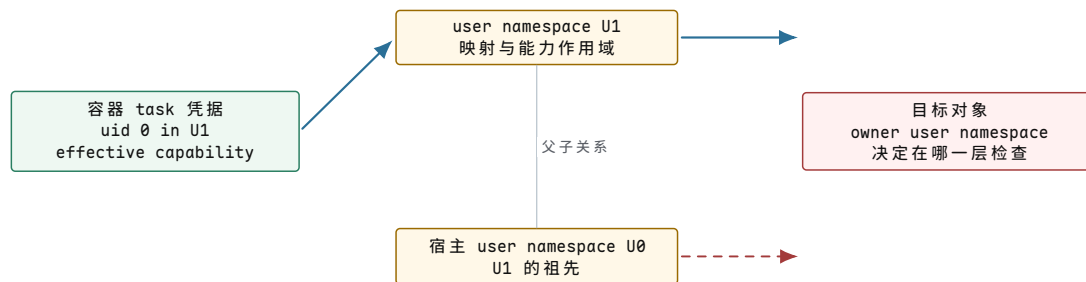
凭据对象通常保存多组能力位，而不只是一组“当前能力”：

能力集合	约束的状态	设计作用
permitted	进程可使其生效的上界	阻止任意获得不在授权上界内的能力
effective	当前权限检查实际使用的能力	允许暂时关闭已获准能力
inheritable	可参与 exec 继承计算的能力	控制普通执行链上的传递
bounding	后续 exec 也不能越过的系统性上界	让服务永久删去不需要的能力
ambient	满足条件时跨非特权 exec 保留的能力	支持不依赖 setuid 的有限授权传递

能力检查必须针对具体操作选择具体 bit。`CAP_DAC_OVERRIDE` 与 `CAP_NET_ADMIN` 的职责不同；使用范围过大的能力会重新形成单一 root 的风险。`CAP_SYS_ADMIN` 覆盖面很广，因此尤其需要避免把不相关操作都归入它。

能力为什么必须相对 user namespace 解释

容器内 uid 0 若等同于宿主 uid 0，容器就没有形成安全边界。user namespace 保存容器内外 uid/gid 映射，并成为能力解释的作用域。一个凭据可以在自己的 user namespace 中拥有某项能力，却不自动在祖先 namespace 或宿主对象上拥有同等权限。



图示：能力不是脱离对象的全局布尔值。检查要把调用者凭据中的能力集合放到目标对象所属 user namespace 关系中解释。

`ns_capable(target_user_ns, cap)` 一类语义同时回答“具有什么能力”和“相对哪个命名空间有效”。只调用全局 `capable` 而忽略对象归属，可能把容器内权限扩大到宿主。

LSM 为什么在对象已稳定后再判断

`mode`、`ACL` 和 `capability` 仍主要表达 Unix 自主访问控制。系统还可能要求“Web 服务只能读取标为 `public` 的对象，即使 `uid/mode` 原本允许”，或者“某域不能向另一域建立 `socket`”。Linux Security Module (LSM) 在对象语义已经明确的位置调用安全钩子，让 SELinux、AppArmor 或 BPF LSM 等策略检查主体标签、对象标签和操作。

```
authorize_open:
  resolve path to stable object
  check mount and object state
  evaluate mode / ACL / applicable capability
  invoke security policy hook with subject and object
  only then publish usable file reference
```

钩子过早时只有路径字符串或 `fd` 整数，无法绑定稳定对象；钩子过晚时对象可能已经发布或产生副作用。合适位置是“对象已解析且引用稳定，但操作尚未提交”。不同操作的提交点不同，所以安全钩子分布在 `inode`、`file`、`task`、`socket`、`BPF` 等路径，而不是只有一个总开关。

一次授权判断的主体-对象-操作账本

面对复杂权限问题，可以先写三项输入，再追踪决策：

输入	需要冻结的状态	代表性内容
主体	一次检查期间一致的凭据快照	<code>uid/gid/groups</code> 、能力集合、 <code>user namespace</code> 、安全标签
对象	使用期间不被替换的对象引用	<code>inode/socket/task</code> 、所有者、 <code>mode/ACL</code> 、对象 <code>namespace</code> 、标签
操作	明确的请求语义与提交点	读、写、执行、创建、连接、发送信号、改变配置

授权结果只对这组三元关系成立。“主体能读文件 A”不能推出它能写 A，也不能推出它能读同名但已被替换的文件 B。对象状态若在提交前发生需要重新判断的变化，操作必须在锁内检查或使用版本号重试。

核心思想

凭据之所以成为独立对象，是为了让一次权限检查读取自治且可共享的主体快照。`uid`、有效身份、组和多组 `capability` 分别表达不同授权边界；`user namespace` 决定能力作用域；`mode`、`ACL` 与 LSM 按已稳定对象的语义继续收

缩权限。授权从来不是入口处一个孤立的 `if`，而是主体、对象和操作在提交前形成的关系。

阻塞以后如何保留同一项系统调用

没有数据时不能靠入口帧一直占用 CPU

现在考虑 `read` 一个空管道。对象引用有效，用户缓冲区范围也可写，但管道缓冲区暂时没有数据。让内核循环检查会持续占用 CPU；立即返回错误又会迫使用户程序频繁重试。系统需要把“当前条件不满足”保存到等待关系中，让其他 `task` 运行；数据到达时再使读者具备运行资格。

入口帧只能保存用户返回现场，不能回答“等待哪个条件”。管道对象因此维护等待队列；读 `task` 在栈上的等待记录或长期等待对象中保存队列链接、`task` 引用、唤醒规则和等待原因。

等待状态	保存在哪里	作用
用户返回现场	<code>task</code> 内核栈入口帧	系统调用最终完成后回到用户程序
内核继续位置	内核调用链与调度切换帧	<code>task</code> 再次运行时从条件复查处继续
等待条件关系	管道等待队列条目	数据到达路径知道应唤醒哪些 <code>task</code>
任务可运行状态	<code>task</code> 与 <code>runqueue</code>	调度器知道它当前是否有资格竞争 CPU
已完成前缀	处理函数局部或请求对象	信号、错误发生时决定返回数量还是负错误

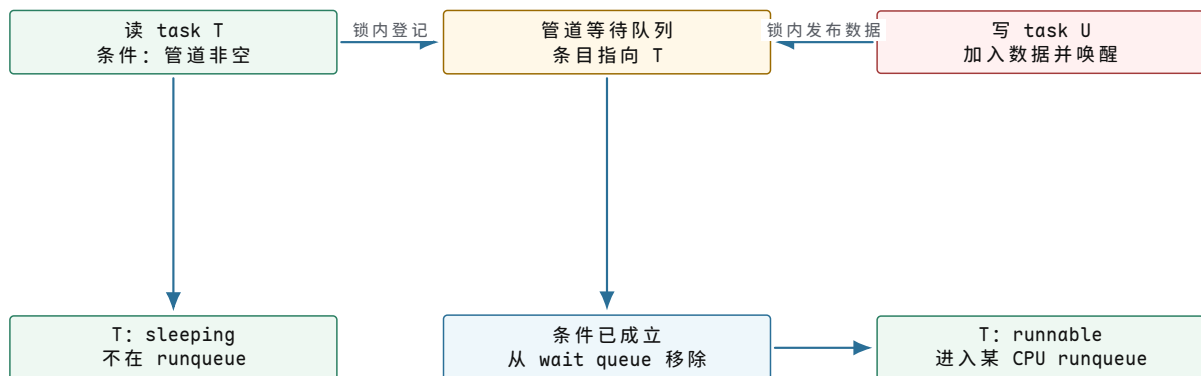
为什么入队和检查条件必须形成原子观察边界

若 `task` 先检查“管道为空”，尚未加入等待队列时写者放入数据并执行唤醒，唤醒看不到任何等待者；读者随后入队并睡眠，可能永远等不到已经发生的事件。这是丢失唤醒。

正确协议在对象锁或等价同步下安排状态：

```
read_from_pipe:
    lock pipe
    while pipe is empty:
        add current task to pipe.read_waiters
        set task state to interruptible sleep
        unlock pipe
        schedule
        lock pipe
        remove wait entry if still linked
    copy available data
    set task state to running
    unlock pipe
```

具体内核通常把这些步骤封装在等待原语中，并仔细安排内存顺序。核心不变量是：写者要么观察到等待者并唤醒，要么读者在决定睡眠前观察到数据已经到达。不存在“事件发生了，但两方都没观察到”的间隙。



图示：等待队列保存条件关系，runqueue 保存运行资格。唤醒只把 T 变为 runnable，不保证 T 立即获得 CPU；恢复后 T 仍要在锁内复查条件。

唤醒为什么不等于系统调用已经完成

数据到达时，读者只能证明“值得让读者重新检查”。数据可能被另一个读者先取走，信号也可能先到达；因此 T 获得 CPU 后必须重新检查管道状态。等待循环使用条件而不是单次 `if`，既处理竞争，也处理无条件或虚假唤醒。

唤醒路径不直接改写 T 的用户返回值。它改变 task 的运行资格；T 恢复内核调用链后，自己复制数据、推进对象状态并写入入口帧的结果槽。这样，完成数量仍由持有操作上下文的执行流决定。

信号为什么能打断可中断等待

用户可能要求终止或处理某个事件。若所有阻塞调用都不可中断，task 可能长期无法响应。可中断等待把“数据到达”和“有待处理信号”都作为离开睡眠的条件。task 被信号唤醒后，处理函数先看是否已有提交前缀：

调用进度	信号到达后的优先结果	原因
尚未提交任何数据	返回中断语义或准备重启	调用者尚不需要处理成功前缀
已经读取 600 字节	返回 600	完成数量比中断错误更重要，避免重复消费
操作不可安全重启	返回 <code>-EINTR</code> 等接口结果	从头执行可能重复产生副作用
操作可重启且策略允许	内部标记重启，信号处理后恢复请求	向用户隐藏无副作用的中断窗口

这里再次出现“结果优先级”。同一次调用内部可能同时发生信号和部分完成，ABI 只能返回一个主结果。接口必须选择能让调用者正确继续的那个结果，并把信号保留到返回工作路径处理。

系统调用重启为什么不只是把 RIP 后退

无参数、无副作用的简单等待或许能重新执行入口指令。带超时、临时内核状态或已规范化参数的调用还要保存剩余时间和重启函数。restart block 一类结构可以记录：

```

restart state:
  restart_function
  normalized arguments
  remaining timeout or absolute deadline
  object identity or restart cookie
  
```

采用绝对 deadline 比反复使用原相对超时更可靠。若每次信号后都重新等待完整 10 秒，多次信号会无限延长总等待；保存绝对截止时间后，重启只计算剩余

预算。

```
restart_timed_wait:
    remaining = deadline - current_time
    if remaining <= 0:
        return timeout
    resume wait with remaining budget
```

并非所有调用都允许自动重启。创建对象、发送消息或设备控制若已产生不可辨认的副作用，重新执行可能重复操作。重启规则属于每项系统调用的语义，不由入口层统一猜测。

调度切换后按什么顺序继续

读 task T 睡眠后，调度器保存 T 的内核切换现场并运行 U。数据到达使 T runnable；当调度器后来选择 T 时，恢复顺序是：

1. 恢复 T 的地址空间与体系结构线程状态（若此前运行的是另一进程）。
2. 恢复 T 的内核栈指针和被调用者保存寄存器。
3. 从 `schedule` 之后的内核位置继续，而不是直接回用户态。
4. 重新取得对象锁并复查等待条件。
5. 形成数据或错误结果，逐层返回到系统调用出口。
6. 出口处理信号、调度请求和用户返回状态，最后恢复入口帧。

入口帧一直位于 T 的内核栈中，但调度器首次恢复的不是该帧，而是更靠近当前栈顶的切换现场。只有系统调用 C 调用链完成后，控制流才逐步退回入口帧。

核心思想

阻塞不会把系统调用变成另一项请求。入口帧保存用户继续状态，内核调用链保存处理函数继续位置，等待队列保存条件关系，runqueue 保存运行资格，局部进度保存已提交前缀。唤醒只改变运行资格；task 恢复后必须重新检查条件并亲自形成返回结果。

进入内核以后为何仍要区分执行上下文

系统调用执行期间设备事件不会停止

task 进入内核后，网卡仍可能收到数据，定时器仍会到期，其他 CPU 也仍在修改共享对象。若整个系统调用期间都禁止中断，一次缓慢的路径解析或页面回收就会让设备完成长时间得不到处理，调度时钟也无法推进。若始终允许中断，又会出现新的问题：同一 CPU 正在使用 task 的内核栈和每 CPU 临时状态时，中断入口会再次保存一份现场。

于是不能只说“当前在内核态”。至少要知道控制流由哪一种事件进入、能否阻塞、能否被抢占，以及它正在使用哪一份栈。系统调用上下文与中断上下文都运行内核代码，但它们拥有的继续条件不同：

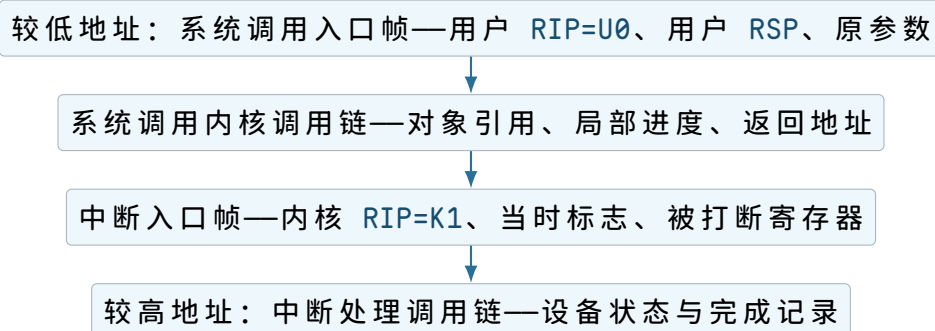
执行来源	当前控制流依附的状态	能否主动等待
系统调用	当前 task、其内核栈和入口帧	在不持有禁止睡眠资源时可以
用户异常	当前 task、异常帧和故障地址	可修复故障路径可能等待页面或 I/O
设备中断	CPU 上被打断的现场与中断入口帧	不能把当前 task 当成中断工作的请求者去睡眠

不可屏蔽事件	最小独立入口状态与严格受限的代码	不能依赖普通锁或普通调度
--------	------------------	--------------

设备中断恰好发生在 task T 运行时，不表示中断工作属于 T。T 只是被借用了 CPU；中断完成后通常回到被打断位置。若中断处理因等待而调用调度器，就没有一项独立 task 可保存“中断处理到哪里”，也会把 T 的内核调用链夹在一个无法解释的等待关系中。

嵌套入口为什么需要另一份入口帧

设 T 正在执行系统调用处理函数，处理器在位置 K_1 收到网卡中断。原系统调用入口帧保存的是“将来怎样回到用户位置 U_0 ”；中断入口新保存的则是“怎样回到内核位置 K_1 ”。两份帧不能合并：



图示：中断返回只弹出中断调用链和中断帧，恢复到 K_1 。系统调用稍后完成时才使用更早的系统调用入口帧回到 U_0 。

若中断发生在用户态，中断帧保存用户继续状态；若发生在内核态，它保存内核继续状态。入口代码必须依据被打断特权级和入口类型决定需要切换哪一份栈、需要补齐哪些字段，而不能假设所有事件都来自用户态。

为什么中断入口常使用每 CPU 或专用栈状态

中断在任意 task 上都可能发生。若它先查找一个需要取得普通锁的全局结构来决定栈位置，而中断恰好打断了持有该锁的代码，入口尚未建立安全现场就会自锁。最早阶段更适合使用当前 CPU 可直接定位的状态，包括入口栈顶、嵌套深度、当前 task 指针和紧急事件专用栈。

专用栈还处理两类风险。第一，当前 task 的内核栈可能接近耗尽，严重故障若继续压栈会覆盖邻接内存；第二，某些异常可能在普通入口本身故障时到达，需要与已经损坏的栈状态隔离。专用栈不是为了让所有中断拥有独立线程，而是确保最早保存与诊断不依赖可能失效的普通栈。

状态位置	适合保存的内容	不应混入的内容
task 内核栈	本次系统调用调用链、局部进度、入口帧	其他 task 的入口现场
每 CPU 入口区	最早栈指针、当前 task、入口嵌套标志	会跨 CPU 迁移的长期请求状态
普通中断栈	可屏蔽中断的短调用链与临时设备状态	需要睡眠等待的操作进度
紧急专用栈	严重异常的最小保存与诊断状态	常规业务缓冲区和大对象

CPU 迁移进一步限制了每 CPU 状态的使用。T 读取“当前 CPU 的临时槽”后若允许抢占并迁移，后续写回可能落到另一 CPU。访问方要么在短临界区禁止迁移，要么保存并使用显式 CPU 身份，不能把一次读取的 CPU 编号当作永久 task 属性。

为何需要记录当前上下文允许哪些动作

一段公共辅助函数可能从系统调用路径调用，也可能从设备中断调用。仅凭代码地址无法判断它能否睡眠。内核需要维护足够的上下文状态，例如中断嵌套、抢占禁止深度、持有的自旋类资源以及中断屏蔽状态，使调度与诊断路径能够拒绝不合法的等待。

```
may_block_here:
    require running on behalf of a schedulable task
    require not inside hard interrupt or emergency entry
    require preemption/atomic nesting permits scheduling
    require no held resource whose protocol forbids sleep
```

“原子上下文”在这里不是说整段代码由一条硬件原子指令完成，而是说当前执行不能把 CPU 让给另一个 task 后再继续。原因可能是处于中断调用链，也可能是持有自旋锁或显式禁止抢占。把这两个含义混为“执行很快”会造成错误：一段代码即使只有几十条指令，只要调用了可能缺页或等待内存的函数，就可能睡眠。

用户复制为何会影响锁的选择

`copy_from_user` 看似只是复制字节，但目标用户页可能尚未建立 PTE，需要进入缺页路径；缺页处理又可能分配页面或等待存储 I/O。因此用户复制在一般情形下是潜在睡眠点。

若对象实现持有禁止睡眠的自旋锁再复制用户内存，可能出现：

1. 复制访问用户页并产生 page fault；
2. fault 路径需要等待页面；
3. 调度器发现当前处于禁止调度的上下文；
4. 操作无法安全继续，也不能保留自旋锁让其他 CPU 无限等待。

正确安排要由对象协议决定。常见方向包括先把小输入复制到内核快照，再取得对象自旋锁；或者在可睡眠互斥边界内复制；或者为大 I/O 预先固定并验证页面，然后在不可睡眠阶段只访问已保证存在的映射。三者的代价和生命期不同，但都必须明确“缺页可能发生在哪个边界”。

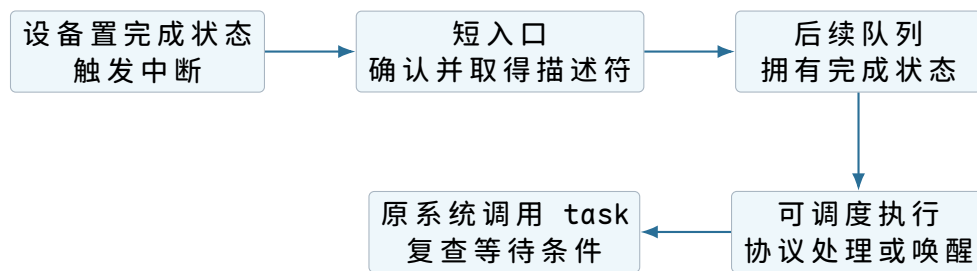
安排方式	获得的保证	仍需处理的问题
先复制后加锁	持锁阶段不再访问原用户输入	加锁前对象状态可能变化，锁内必须重验
可睡眠锁内复制	对象条件在复制期间保持稳定	长缺页会延长锁占用，影响其他 task
预先固定用户页	后续阶段可引用稳定页面集合	固定成本、解除固定和写回责任
分段保留并复制	限制单次锁占用与故障范围	必须精确推进已提交前缀

这也解释了为何“检查参数-加锁-复制-提交”不是所有调用都能照搬的固定顺序。顺序取决于每一步可能睡眠、依赖哪些对象状态，以及失败时能否撤销。

中断工作为什么要拆成短入口和可调度后续

设备完成可能需要解析大量描述符、分配缓冲区、唤醒 task 或继续网络协议处理。全部放在硬中断上下文会延长同 CPU 上其他中断的响应时间，也不能使用可睡眠操作。于是先在严格入口中完成不可延迟的最小工作：确认设备事件、取得完成描述符的所有权、屏蔽或应答必要状态，并登记后续工作。

后续可在受限制的延后上下文或内核线程中执行。若工作最终可能等待，就需要一项可调度的 task 作为载体。拆分后的所有权边界必须明确：入口把哪些描述符移交给后续队列，设备何时可以复用槽位，后续失败由谁释放缓冲区。



图示：设备中断不直接替等待中的 task 完成系统调用。它取得硬件完成状态并安排后续；后续更新对象条件、唤醒 task，task 恢复后再形成自己的系统调用结果。

内核抢占为何不同于中断嵌套

中断是外部或处理器事件插入当前控制流；内核抢占则是调度器在安全点暂停当前 task，选择另一个 task。两者都会让当前代码暂时不执行，但保存对象与恢复位置不同。

暂停原因	主要保存位置	恢复关系
设备中断插入	当前栈上的中断帧或专用中断栈	中断返回到被打断指令
内核抢占 T	T 的内核栈与调度切换帧	调度器将来选择 T 后继续
系统调用主动等待	T 的调用链、等待队列与切换帧	条件唤醒后由调度器恢复
用户态信号交付	用户信号帧与改写后的入口帧	先运行处理函数，信号返回后恢复原现场

代码在修改每 CPU 数据或持有某些低层锁时，可以短暂禁止内核抢占，保证 task 不会在临界区被迁移或切换。禁止抢占不一定屏蔽设备中断；如果中断处理也访问同一状态，还需要相应的中断同步协议。反过来，屏蔽本地中断也不自动构成跨 CPU 互斥，其他 CPU 仍可并发访问。

怎样核对一段入口代码的上下文闭合

对入口到处理函数之间的每段代码，可按以下状态逐项核对，而不是只看函数是否“很短”：

1. 当前使用 task 栈、中断栈还是紧急栈？最大嵌套深度怎样受限？
2. 当前代码代表哪个 task，还是不代表任何可调度 task？
3. 本地中断是否允许，内核抢占是否允许，task 是否可能迁移 CPU？
4. 持有哪些锁或引用；其中哪些要求释放前不能 fault、不能调度？
5. 访问用户地址是否可能缺页；内存分配是否可能回收或等待？
6. 事件再次嵌套时，会覆盖每 CPU 临时槽还是获得独立槽位？
7. 离开时哪一份帧被弹出，返回内核位置还是用户位置？

这些问题将“上下文错误”还原为具体状态冲突。例如，“在中断中睡眠”实质是没有任何一项可独立调度的 task 来拥有等待中的调用链；“持自旋锁访问用户页”实质是故障路径可能要求调度，而当前锁协议禁止让出 CPU；“每 CPU 指针失效”实质是读取与使用之间发生了迁移。

核心思想

内核态不是单一执行环境。系统调用、用户异常、设备中断、紧急事件和调度恢复拥有不同的栈、继续位置与阻塞能力。每段代码只有在说明了当前现场由谁拥有、允许哪些嵌套以及能否调度后，才能判断锁、用户复制和内存分配是否安全。

边界观察为何也必须服从状态形成顺序

只记录入口参数为何无法说明实际操作

诊断程序看到 `openat(dirfd, user_path, flags)` 的入口时，只能得到整数、标志位和一个用户地址。路径内容尚未形成内核快照，`dirfd` 尚未转换成稳定目录对象，符号链接和挂载关系也尚未解析。若日志此时写下“打开了某文件”，它把请求意图误写成已经发生的事实。

反过来，只在出口记录返回值也不够。一个 `-EACCES` 不能说明拒绝的是哪个对象、使用哪份凭据、在哪一层策略中发生。可核对的边界记录应随着信息稳定逐步补全：

观察时刻	此时稳定的信息	不能提前声称的事实
入口建立后	ABI、调用编号、原始标量参数	用户指针内容真实且不变
参数快照后	内核拥有的字符串或结构字段	路径已对应最终对象
对象解析后	稳定对象身份、挂载与命名空间关系	权限已经允许、操作已经提交
授权完成后	主体、对象、操作与策略判断	后续对象状态仍允许提交
提交边界后	实际改变、完成前缀与错误	数据已经持久化或被外部系统接收
出口工作后	最终返回值、信号或重启决定	用户程序已经处理该结果

同一条审计事件可以在入口创建上下文，随后逐步填充字段，最后在出口发布。若调用阻塞，上下文必须跟随 `task` 或独立请求状态保存，不能留在某个 CPU 的临时槽中；`task` 恢复时可能已经迁移到另一 CPU。

原始参数与规范化参数为什么都可能需要保留

原始参数回答“调用者提出了什么”，规范化参数回答“内核按哪种含义执行”。两者可能不同。例如旧 ABI 的窄时间字段会转换为统一宽类型；路径中的相对起点会结合目录对象；标志组合会被检查并分解成内部操作选项。

安全日志不能随意重新读取用户指针来补字段，因为用户已经可以改写原内存。若确实要记录字符串，应使用本次操作已经取得的内核快照，并设置长度上限与转义规则。否则日志本身会产生第二次 TOCTOU：系统实际检查的是字符串 A，日志稍后重新读取成字符串 B。

记录形式	适合回答的问题	必须注明的限制
原始寄存器值	调用者如何编码请求	指针只是地址，尚无对象语义
内核参数快照	实现实际解析了哪些字段	可能已经过兼容转换和规范化
稳定对象标识	操作最终指向哪个内核对象	路径名称后来可以改变

提交计数与终态	实际产生多少效果	完成不一定代表持久化
---------	----------	------------

跟踪器能够改写入口帧时谁拥有最终请求

某些跟踪机制允许监管者在入口暂停 task，查看甚至修改系统调用号、参数寄存器或返回值。暂停发生时，被跟踪 task 的入口帧已经存在，但对象操作尚未开始。监管者完成修改后，内核必须重新读取入口帧并按新值执行分发与验证，不能继续使用暂停前缓存的编号。

这形成明确的修改边界：

1. task 建立入口帧，产生原始请求状态；
2. 跟踪入口事件把 task 停在对象副作用之前；
3. 监管者通过受控接口修改允许字段；
4. task 恢复，边界层把修改后的帧视为最终原始请求；
5. 参数快照、对象解析和授权从该请求重新开始。

监管者不能因为有能力观察入口，就直接获得任意内核指针或绕过用户地址检查。它改的是被跟踪 task 的用户可见请求状态；修改后的指针仍然属于该 task 的地址空间，修改后的操作仍要经过对象权限与安全策略。

出口跟踪同样位于对象结果已经形成之后、处理器返回用户态之前。监管者可以观察规定的结果槽；若接口允许改写返回值，审计记录应能区分“对象实现结果”和“最终暴露结果”。改变返回数字并不会自动撤销已经提交的对象副作用。

入口策略拒绝为什么要发生在对象副作用之前

系统调用集合过滤器适合根据编号、体系结构和原始标量参数快速收缩请求面。若它允许通知一个用户态监管者再决定，原 task 可能等待较长时间。等待期间用户内存、fd 槽和进程关系都可能改变，所以监管者不能仅凭入口时看到的 fd 数字替内核完成最终对象授权。

安全分工是：入口策略决定“这类请求是否可以继续进入内核实现”；对象实现随后取得稳定引用并判断“当前主体能否对当前对象执行该操作”。若监管协议要代替某项操作，它必须定义参数快照、返回值注入、fd 等对象如何安全移交，以及原 task 退出时怎样取消通知。否则一次入口批准会被错误地当作对未来任意 fd 重用的批准。

阻塞调用的时长为何要拆成几个区间

从入口到出口的总时长包含不同原因。只报告一个总数，无法区分边界开销、锁竞争、条件等待和设备服务。可以使用已经得到的状态边界划分时间：

时间区间	起止边界	主要解释对象
入口固定时间	入口指令到对象实现开始	现场保存、过滤、分发与参数转换
对象活动时间	获得对象到首次等待或完成	锁、查找、复制与提交算法
非运行等待时间	离开 runqueue 到重新获得 CPU	条件等待与调度延迟
唤醒后排队时间	变为 runnable 到实际恢复	运行队列竞争与优先级
出口固定时间	结果确定到用户态恢复	信号、跟踪、调度检查与返回验证

时间源本身也要注明。墙上时间可能被管理员调整，单调时钟适合计算持续时间，CPU 运行时间只累计 task 真正执行的区间。把不同来源的时间戳直接相减，可能得到负值或把睡眠误算为处理器工作。

观察代码为何不能破坏原路径

跟踪点位于锁内、入口最早阶段或中断上下文时，记录动作受同样的上下文限制。它不能无界分配内存、等待用户态消费者或复制任意长字符串。常见安排是写入预分配的每 CPU 环形缓冲，记录固定大小的标量和稳定标识，随后由可调度读取者解释。

即使这样仍要定义缓冲区满时的策略：覆盖旧记录、丢弃新记录或增加丢失计数。不能让关键中断路径为保住日志而无限等待。日志记录还要避免泄露未初始化填充字节、内核地址和不属于观察者权限范围的数据。

观察设施关闭时，热路径应尽量只付出一次可预测的关闭检查；启用后增加的成本要能被测量。这属于正确路径建立后的优化。不能为减少跟踪开销而让入口记录与出口记录失去配对，也不能因审计分配失败就悄悄执行一项本应被强制审计策略拒绝的敏感操作。

核心思想

观察不是站在系统调用之外读取一条现成事实。请求、参数快照、对象身份、授权结果、提交前缀和最终用户结果在不同时刻形成。日志、跟踪和性能计时只有绑定这些边界，才能说明记录的是意图、稳定输入、实际对象还是已经提交的效果。

处理函数结束以后为何还不能立即返回

一个返回值不足以描述返回时刻的全部状态

设管道读取已经得到 600 字节，处理函数把 600 写入入口帧的结果槽。若内核此时只恢复 RIP、RSP 和通用寄存器，表面上已经完成了调用。然而在 task 阻塞期间，另外几类状态可能同时发生变化：

- 时间片已经用完，task 应先让出处理器；
- 一个信号已经挂起，需要在用户态继续执行前交付；
- 跟踪器要求观察或改写本次调用的退出状态；
- 审计设施需要把最终结果与入口记录配对；
- 调用被内部标记为可重启，必须决定返回错误还是安排重试；
- 入口帧中的用户返回地址或标志不再满足快速返回指令的约束。

这些状态不属于管道读取算法，却决定 task 下一步实际执行什么。若每项系统调用都自行处理，不同调用会产生不同的信号、调度和跟踪语义。于是需要一段共享的返回工作路径：对象实现只提交操作结果；返回路径在结果已经确定后，统一结清 task 级的未决工作。

未决状态	返回前要做的动作	不能交给对象实现的原因
需要重新调度	调用调度器，之后重新检查标志	时间片属于 task 与 CPU，不属于文件或管道
挂起信号	选择信号、建立用户信号帧	信号会改写用户继续位置，与具体对象无关

调用重启	改写结果与继续状态，保留重启参数	是否重启取决于信号处置和已提交前缀
跟踪或审计	生成退出事件，允许受控观察	入口与退出必须共享统一边界
快速返回不安全	改走能完整恢复状态的慢路径	返回指令受体系结构条件限制

返回工作不是一次性检查。处理信号时可能唤醒调试器，恢复后时间片又可能耗尽；调度回来后还可能收到新信号。合理结构是“观察工作标志-完成一种工作-重新观察”，直到没有必须在用户态之前完成的事项。

```

leave_system_call:
repeat:
    flags = task.return_work
    if flags says reschedule:
        schedule()
        continue
    if flags says signal or restart:
        prepare_user_delivery()
        continue
    if flags says trace or audit:
        finish_boundary_observation()
        continue
until no return work remains
validate_user_return_state()
restore_and_return()

```

这里的循环不是无条件忙等。每个分支都消费一个状态或发生一次有意义的切换；重新读取标志是为了覆盖并发到达的新事件。

为什么调度检查位于用户返回之前

系统调用处理函数结束时，task 正在内核态运行。若时间片已经耗尽但仍直接返回用户态，task 会继续运行，直到下一次中断或入口才可能让出 CPU。负载很高时，这会破坏调度延迟的上界。

返回边界已经具备三个条件：当前 task 有完整可恢复现场；内核栈和地址空间仍处于可控状态；处理函数不再持有要求立即释放的局部锁。因此它是安全调度点。调度器保存的仍是内核切换现场。将来恢复该 task 时，它从返回工作循环继续，重新读取标志，而不是绕过未处理的信号直接进入用户态。

还要区分“需要调度”和“已经选择了下一个 task”。前者只是 task 或 CPU 上的请求标志；只有调用调度器并取得运行队列锁后，才会做出选择。若此时没有更合适的可运行任务，当前 task 仍可能继续执行。

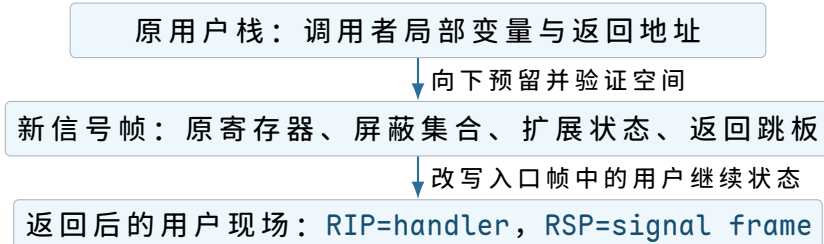
信号交付为何要在用户栈上建立一份新现场

信号处理函数是用户代码。内核不能在自己的栈上调用它，也不能丢失原来的用户执行状态。要让处理函数结束后回到原位置，必须在用户地址空间中建立一份可由信号返回协议解释的现场。它通常包含：

信号帧内容	来源	恢复时的用途
被中断的通用寄存器	系统调用入口帧或用户异常帧	让原用户控制流继续
原用户 RIP/RSP/flags	已验证的入口状态	恢复指令位置、栈和允许的标志

信号编号与扩展信息	内核挂起信号对象	作为处理函数参数
信号屏蔽集合	task 的信号状态	处理函数结束后恢复屏蔽关系
浮点与向量状态	task 的体系结构扩展状态	避免处理函数破坏原计算
用户返回跳板地址	受信任的用户映射或 ABI 约定	发起受控的信号返回请求

这份帧写在用户栈或预先登记的备用信号栈上，所以仍要走用户内存验证和受控复制。普通用户栈可能已经接近边界，备用栈可能未映射，处理器扩展状态也可能使帧比预期更大。构造失败不能退回到一个半写的帧上继续执行；内核需要把失败转换为规定的致命信号或终止结果。



图示：内核并不是在用户栈上保存自己的调用链。信号帧只保存用户可恢复现场；系统调用内核栈仍按正常出口退栈。

信号处理函数最终执行的“返回”也不能只是普通 `ret`。它需要请求内核读取信号帧，恢复屏蔽集合和扩展状态，并逐字段验证用户可控制的数据。信号帧位于用户内存，用户可能在处理期间修改它；恢复路径必须把它当作不可信输入，不能因为它最初由内核写入就跳过检查。

重启决定为什么必须晚于信号选择

内核在等待中断时可能暂存“这项调用可以重启”。这还不是最终决定。若用户安装了自定义处理函数，内核需要先构造信号帧；若信号被忽略，可能不必改变用户控制流；若调用已经提交部分结果，则应优先返回完成数量。

最终重启至少要同时满足四项条件：

1. 当前调用的语义允许从保存的状态继续或重新进入；
2. 尚无必须先向用户报告的提交前缀；
3. 所选信号的处置方式允许自动重启；
4. 重启参数、剩余期限与对象身份仍然有效。

满足条件时，返回路径可以把入口帧改造成“信号处理结束后再次提出请求”的状态。这里要恢复原系统调用编号，而不是把当前结果寄存器中的内部重启码当成新编号。若使用 `restart block`，则信号返回后进入专门的重启入口，再由该入口使用保存的规范化参数。

为什么返回地址和标志必须再次验证

入口时保存的用户 `RIP` 与 `RSP` 可能被信号、跟踪器或系统调用语义受控修改。它们也可能来自恶意用户构造的信号返回帧。内核即将把执行权交还给较低特权级，此时必须确认：

- 指令地址和栈地址具有体系结构规定的用户地址形式；
- 返回目标位于允许的用户地址范围，而不是内核映射；
- 段选择子、特权级和地址宽度与当前 ABI 一致；

- 标志寄存器中的特权控制位已清除或恢复为规定值；
- 调试、单步、地址空间和控制流保护相关状态走了相应慢路径；
- 不会把仅用于内核的临时寄存器内容泄露给用户态。

“入口时验证过”不能代替此处验证，因为中间路径可能合法地改变返回状态。返回验证保护的并不是系统调用参数，而是下一条用户指令的执行边界。

快速返回为何只能是一项经过证明的优化

x86-64 提供快速系统调用返回指令，但它对返回地址、标志和段状态有特定约束。完整的中断式返回路径能从显式帧中恢复更多体系结构状态，成本也更高。两者的合理关系是：

1. 先建立一条能验证并完整恢复状态的通用返回路径；
2. 证明当前入口类型、用户地址、标志和跟踪状态满足快速路径条件；
3. 只有证明成立时才选择快速返回；
4. 任一条件不确定，都退回通用路径，而不是补猜缺失状态。

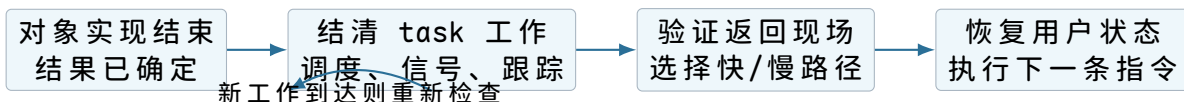
返回情形	快速路径是否合适	理由
普通 64 位调用，无信号和跟踪	可能合适	返回地址和标志保持在快速指令约束内
信号改写了用户继续状态	需要重新证明	新 <code>RIP/RSP</code> 来自刚构造的用户现场
调试器要求单步	通常走慢路径	调试标志和退出观察需要完整处理
兼容 ABI 调用	使用对应 ABI 的路径	地址宽度、段状态和参数布局不同
信号返回恢复扩展状态	通用验证路径	用户帧包含大量不可信恢复字段

快速路径的收益来自缩短已经正确的公共路径，而不是省略语义。若优化改变了信号、跟踪或错误结果，它就不再是同一接口的实现。

返回前如何避免泄露内核临时状态

系统调用 ABI 规定哪些寄存器保存、哪些可破坏，但“可破坏”不等于可以携带内核秘密。入口代码、C 调用约定和体系结构代码会使用临时寄存器；如果返回指令恢复的集合少于内核使用的集合，未覆盖寄存器可能暴露指针、栈内容或地址布局。

安全出口需要为每个返回寄存器给出明确来源：从用户入口帧恢复、写入规定结果、写入 ABI 允许的固定值，或在返回前清零。浮点、向量、调试和控制寄存器也要遵守 task 所有权切换规则。不能用“用户大概不会读取”代替状态定义。



图示：“处理函数返回”与“处理器回到用户态”是两个时刻。中间的返回工作路径结清 task 级状态，并证明最终用户现场可安全恢复。

核心思想

系统调用出口不是入口的机械逆过程。入口把不可信请求转换成内核拥有的状态；出口要先结清并发到达的 task 级工作，再把受控结果转换成一个可安全恢复的用户现场。快速返回只能建立在完整语义已经可用、且本次状态满足额外约束的证明之上。

正确路径建立后怎样判断入口成本是否值得优化

先把一项只读请求的固定成本逐项记账

读取时间、查询 CPU 编号或获得进程标识一类请求可能只需要几个字段。若每次都经历特权切换，固定边界成本可能大于真正的数据读取成本。要讨论优化，先把基础方案的工作拆开：

基础步骤	即使结果很小仍需完成的工作	保护的语义
建立入口现场	保存继续位置并切到内核拥有的栈	可恢复性与隔离
入口过滤与分发	识别 ABI、编号和边界策略	稳定契约与安全策略
取得共享状态	读取时间源或 task 字段	一致性
返回工作检查	检查信号、调度和跟踪	task 级语义
恢复特权级	验证并恢复用户现场	安全交还执行权

这些工作对修改全局状态的调用是必要代价。对于“频繁读取、极少修改、允许只在用户态获得快照”的信息，重复跨越边界就值得重新审视。优化问题由此变得具体：能否把一份经过约束的只读状态映射给用户，让用户在不进入内核的情况下得到与系统调用相同的结果？

共享只读数据为何仍需要一致性协议

直接映射一页时间数据并不充分。内核可能正好在用户读取一半时更新基准时间、周期换算系数和时钟源编号；把新旧字段拼在一起会得到不存在的时间点。读者又不能取得会阻塞内核更新的普通锁。

一种适合“单个写者协议、读者短暂读取”的安排是版本序列。写者更新前把序号改成奇数，更新完再改成下一个偶数；读者只有在前后读到同一个偶数时才接受快照：

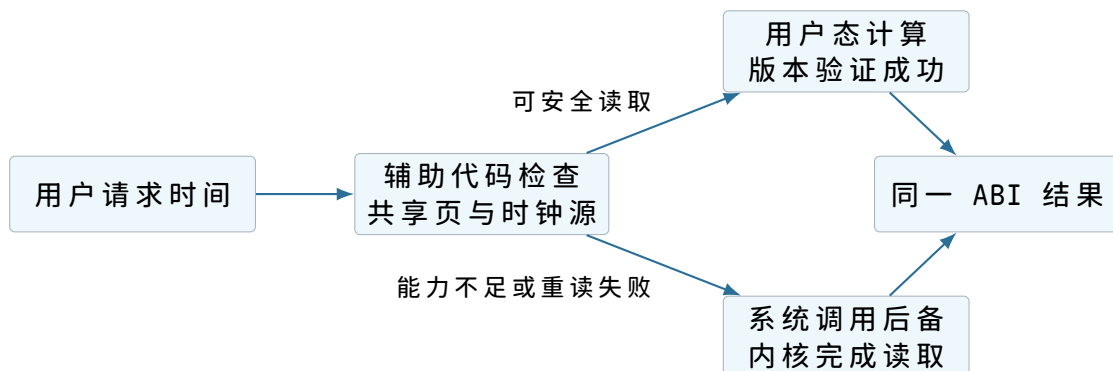
```
read_shared_time:
  repeat:
    begin = shared.sequence
    if begin is odd:
      continue
    base      = shared.base_time
    cycle_base = shared.cycle_base
    multiplier = shared.multiplier
    shift     = shared.shift
    now_cycle  = read_hardware_counter()
    end = shared.sequence
  until begin == end and end is even
  return base + scale(now_cycle - cycle_base, multiplier, shift)
```

读者没有取得所有权；它靠验证“读取期间没有写者完成或正在写”来接受快照。若验证失败，只是重读，不带着混合状态计算结果。共享页还必须只向用户暴露接

口所需字段，不能顺便映射内核内部对象。

用户态辅助入口如何保持系统调用的后备语义

仅有数据页还不够。不同处理器的计数器读取方式、时钟源能力和换算规则可能变化，需要一段由内核按 ABI 提供、但在用户态执行的辅助代码。这就是从上述限制得到的 vDSO 类接口：调用形式看起来像普通库函数；适合时在用户态读共享状态，不适合时退回真正系统调用。



图示：用户态快速读取不是另一套时间语义。它必须与系统调用共享结果约定，并保留无法安全完成时的内核后备路径。

这种方法只适合不需要执行权限检查、修改对象或产生不可逆副作用的请求。若把“打开文件”或“改变系统时间”也做成用户态共享写入，内核就失去了验证与提交边界。

为什么减少调用次数和缩短单次入口是两种优化

设一次边界往返固定成本为 C_e ，每项请求的实际处理成本为 C_o ，连续提出 n 项请求的基础成本近似为：

$$T_{\text{separate}} = n(C_e + C_o)$$

若协议允许一次提交 n 项独立描述符，边界成本可以摊薄：

$$T_{\text{batch}} = C_e + nC_o + C_q$$

其中 C_q 是队列解析、描述符验证和完成记录成本。批处理优化的是入口次数；vDSO 类方案优化的是某一类只读请求的整个入口。二者不能互相替代。

批处理也不是把多个原调用简单拼接。接口必须说明：

- 描述符由谁拥有，提交后用户能否改写；
- 一项失败是否阻止后续项，还是每项独立产生结果；
- 完成顺序是否等于提交顺序；
- 部分完成时怎样报告已提交前缀；
- task 退出、对象关闭或信号到达时怎样取消；
- 完成记录何时对用户可见，是否可能覆盖未消费结果。

如果这些问题没有定义，减少入口次数只会把同步错误从单个调用移到共享队列。

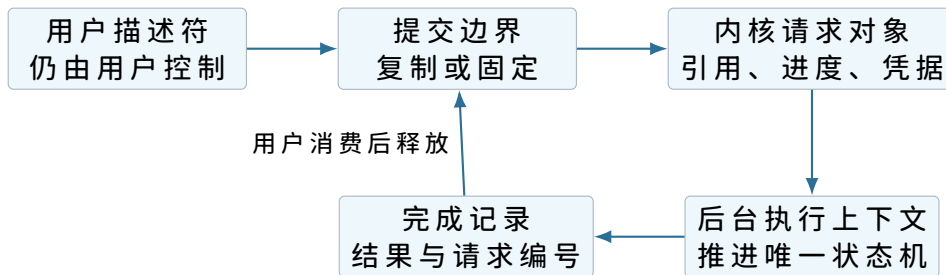
异步提交为什么需要把调用栈中的状态搬到独立对象

同步调用期间，参数快照、对象引用和进度可以位于当前 task 的内核栈上，因为完成发生在该调用返回之前。异步提交允许 task 先返回，真正操作稍后由另一

个执行上下文完成。原内核栈会被复用，局部变量不再存在；于是操作状态必须成为独立、可引用的请求对象。

请求对象字段组	保存内容	存在理由
身份与关联	请求编号、提交者、完成队列	让完成结果找到原请求
规范化参数	长度、偏移、操作码、固定缓冲描述	用户描述符提交后可能改变
对象引用	文件、套接字、内存注册对象	task 返回或关闭槽位后仍要保证生命期
进度与结果	已提交字节、错误、完成状态	支持部分完成和唯一终态
取消状态	取消请求、不可取消点、竞争结果	定义取消与完成同时发生时的唯一答案
凭据与策略上下文	规定时刻捕获的身份或引用	后台执行不能随意使用工作线程身份

这一步揭示一个重要差异：异步接口不是“系统调用返回得更快”，而是改变了请求状态的所有权和生命期。只把用户指针存入队列、稍后直接访问，会重新引入地址变化、页面回收和 TOCTOU 问题。实现要么在提交时复制小参数，要么注册并固定受约束的缓冲区，并规定解除注册与在途请求的同步关系。



图示：同步调用依赖当前内核栈保存进度；异步提交必须把仍需存活的状态迁移到独立请求对象。描述符被接受、操作提交和完成可见是三个不同时间点。

接受、提交和完成为什么必须分开

异步接口返回“接受成功”只说明描述符已通过初步验证并进入内核拥有的队列。设备或对象可能尚未看到请求，更没有产生结果。至少要区分：

时刻	已经成立的事实	尚不能推断的事实
接受	内核已取得请求状态的所有权	操作已到达对象或设备
提交	对象或下层队列已经接收工作	数据已复制、落盘或被对端接收
完成	接口定义的结果已经确定并发布	若接口只保证缓存完成，则不代表持久化
用户消费	用户已读到完成记录	对象的外部副作用已经被撤销

取消请求也只能相对于这些状态定义。尚未提交时通常可从队列删除；已经提交后可能只能标记“不再需要结果”，不能保证设备停止；完成与取消并发时，状态机必须以原子转换决定究竟发布成功、取消还是已完成。不能同时产生两个终态。

兼容 ABI 为什么不能只截断寄存器

同一内核可能服务不同地址宽度或历史布局的用户程序。32 位调用者传入的指针、`long` 和结构体对齐方式与 64 位内核不同。若只把寄存器高位清零然后调用同一实现，嵌套结构体仍会按错误偏移解释。

差异来源	直接复用的风险	兼容层的职责
指针宽度	高位扩展或范围判断错误	明确转成内核地址表示并验证用户范围
<code>long</code> /时间字段宽度	溢出、符号扩展和时间范围变化	按旧布局取值，再转为统一内核类型
结构体填充与对齐	从错误字节读取字段	使用对应 ABI 布局逐字段转换
系统调用编号表	同一编号可能表示不同操作	先识别 ABI，再进入相应分发表
返回值宽度	部分结果被静默截断	按接口规定检测溢出并返回兼容错误

合理的共享边界位于“已经转换成内核统一表示”之后：各 ABI 包装层负责读取旧格式、检查范围并形成规范化参数；对象实现只处理统一类型。这样既避免复制业务逻辑，也不会让核心实现到处猜测调用者位宽。

安全缓解措施为什么也属于边界成本

现代处理器可能在架构检查完成前推测执行后续指令。系统调用入口跨越信任边界，返回路径又可能改变预测上下文；某些平台需要序列化、预测器状态管理或间接分支约束。这些措施增加固定入口成本，但它们处理的是微体系结构可观察性，不能从 C 级权限检查中推导出来。

书写和优化时应保持两个层次：

- 架构语义说明寄存器、页表、特权级和提交状态必须满足什么条件；
- 平台缓解说明为了让推测执行不跨越该条件，还需执行哪些附加步骤。

缓解策略会随处理器能力和系统配置变化，因此不应把某一组指令写成系统调用概念本身。无论启用何种缓解，参数快照、对象引用、授权与返回验证仍是基础正确性；关闭缓解也不能省略它们。

核心思想

边界优化必须从一条已正确、可计量的基础路径出发。只读共享快照减少某类入口，批处理摊薄多项请求的固定成本，异步提交改变请求状态的所有权与生命期，兼容层把不同用户布局转换成统一内核表示。每项优化都要保留原接口的权限、提交、失败和返回语义。

把各阶段合成一项完整而可核对的请求

先跟踪一次会阻塞的 `pipe read`

现在才把前面逐项得到的结构连成完整路径。设 task T 调用 `read(fd, user_buf, 4096)`，管道开始为空，随后写者写入 600 字节，同时 T 收到一个待处理信号。为了避免“调用成功”和“信号到达”混成一句话，逐时刻记录所有权与提交状态。

时刻	T 或 CPU 状态	关键对象状态	已经成立的接口事实
t_0	T 在用户态	fd 槽引用管道读端	尚未提出本次请求

t_1	入口帧已建立	参数仍只是整数与用户地址	可恢复，但尚未取得对象
t_2	处理函数运行	已取得读端引用，长度已验证	fd 重用不再改变本次对象
t_3	T 在等待队列并睡眠	管道为空，未提交字节	调用仍在内核中进行
t_4	写者唤醒 T	管道有 600 字节，T runnable	只获得运行资格，尚未完成读取
t_5	T 恢复并持有管道锁	600 字节可消费，信号挂起	条件成立，信号尚未决定结果
t_6	T 复制并推进读位置	600 字节已交付给用户缓冲区	完成前缀为 600，不能从头重启
t_7	返回工作选择信号	信号帧构造在用户栈	主返回值保持 600
t_8	处理器回到用户态	RIP 指向信号处理函数	处理函数之后可读到 600 的调用结果

在 t_2 之后关闭同一 fd 槽位，不会改变 T 已取得的对象引用。在 t_4 唤醒后，另一读者可能先消费数据，所以 T 到 t_5 仍要复查。到 t_6 才首次出现接口可见提交；从这一刻起，信号不能把主结果改成“完全未发生”的中断错误。

600 字节究竟在哪一个时刻算作已读取

“从管道移除”和“写入用户缓冲区”必须安排成不会无故丢失数据的协议。若先推进管道读位置，随后用户页故障且复制无法完成，数据已经从管道消失，调用者却没有得到它。若先复制又允许其他读者同时消费同一区间，则会重复交付。

实现需要在对象同步边界内为当前读者保留区间，按可处理的片段受控复制，并只对成功复制的字节推进提交位置。具体锁粒度可以优化，但必须维持：

$$0 \leq \text{committed} \leq \text{reserved} \leq \min(\text{requested}, \text{available})$$

用户复制在第 300 字节失败时，接口可返回 300，而对象只消费对应的 300 字节；尚未成功复制的保留区间要释放给后续读取。底层复制函数报告的“剩余 300”要先换算成“成功 300”，再由系统调用层根据是否存在成功前缀选择主结果。

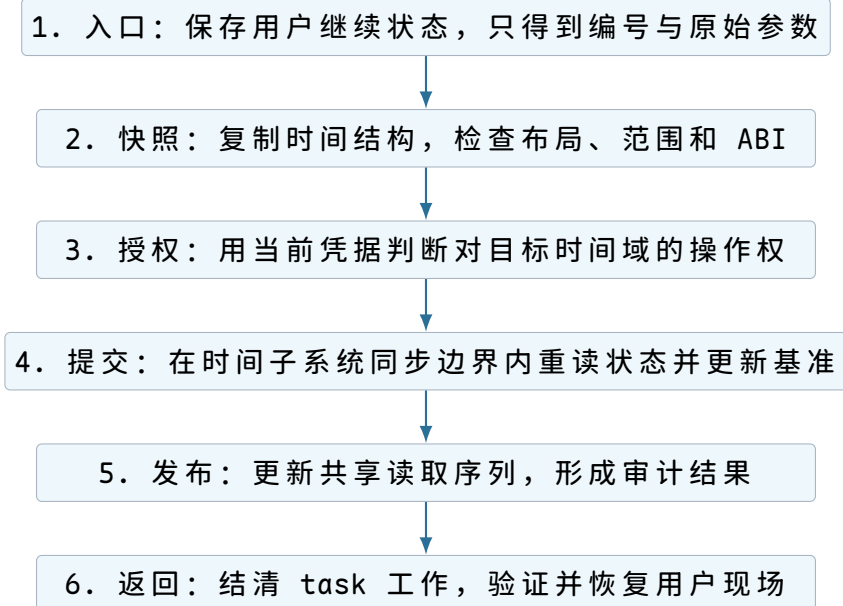
再跟踪一次改变全局时钟的请求

回到本章开头的问题。管理员程序请求把系统实时时钟改到新值。与读取管道不同，这项操作没有可自然返回的字节前缀，而且会影响全局观察者。完整路径应当说明：

1. 用户库把操作编号、时钟类型和用户结构地址放入 ABI 规定位置。
2. 入口代码保存用户继续状态，切换到 task 的内核栈，建立统一入口帧。
3. 分发层先完成入口过滤，再由包装层从用户结构取得时间值并转换成统一内核表示。
4. 转换过程检查字段范围，例如纳秒必须位于接口允许区间；用户随后改写原结构不再影响已取得的内核快照。
5. 授权层使用当前 task 的凭据，检查相对于正确 user namespace 的时间管理能力。
6. 时间子系统在自己的写侧同步边界内重新读取相关状态，计算并提交新基准；授权成功并不预留未来提交权。

7. 提交成功后更新保证用户只读快照一致性的版本序列，并通知需要观察时间变化的子系统。
8. 对象实现写入零结果；返回工作处理调度、信号、审计和跟踪，最后验证用户现场。

如果权限检查失败，第 6 步不能发生；如果用户结构无效，甚至不应进入授权；如果提交前参数范围错误，时钟保持原值。审计记录要区分“请求过什么值”“以哪份凭据判断”“是否真正提交”与“最终向用户返回什么结果”。



图示：这条链路直到本章末才完整出现，因为编号、入口帧、用户快照、凭据、提交与返回边界已经分别由具体问题推导出来。

失败注入怎样验证边界真的存在

正常执行只能证明某条路径可用，不能证明状态在失败时仍闭合。可以在不同边界人为制造失败，逐项检查“谁拥有状态、是否已经提交、应返回什么”。

注入位置	预期保持的状态	应观察的结果
入口编号不存在	不取得对象、不复制用户缓冲区	返回未实现类错误，用户继续状态完整
用户缓冲区第二页缺失	只提交成功复制的前缀	返回前缀数量；若前缀为零则返回地址错误
fd 在取得引用前被关闭	不得使用已释放对象	取得槽位同步决定旧对象或无效 fd
fd 在取得引用后被关闭	当前操作仍持有对象生命期	槽位可重用，本次请求继续引用原对象
权限在提交前变化	按接口规定时刻重新判断或用快照	不把曾经检查过等同于永久授权
等待时收到信号	保留已提交前缀和对象一致性	无前缀时中断或重启，有前缀时返回数量
信号帧用户栈不可写	不恢复到半构造现场	进入规定的失败或终止路径
快速返回条件被破坏	完整状态仍可恢复	退回通用返回路径，结果语义不变

每个测试都应同时观察接口结果和内核对象状态。只断言“返回了 `-EFAULT`”不够；还要确认没有泄漏对象引用、没有重复消费数据、没有发布半初始化 `fd`，也没有把错误的用户现场交给处理器。

用五个问题审查任意新系统调用

面对一个新的系统调用，不必先背诵完整内核链路。按状态形成顺序提出五个问题即可定位主要结构：

1. 请求如何成为稳定输入？哪些标量可直接取值，哪些用户结构必须复制，嵌套指针和尺寸乘法怎样验证？
2. 操作对象怎样获得稳定身份？是从 `fd`、路径还是其他句柄取得；引用何时增加，槽位关闭或重用是否会影响本次操作？
3. 谁在什么时刻授权哪项操作？使用当前、捕获还是对象凭据；检查依赖哪些对象状态，提交前是否必须重读？
4. 哪一步首次产生不可忽略的外部效果？部分完成怎样计数；失败和信号能否重启；发布、完成和持久化是否为不同边界？
5. 怎样形成安全的用户继续状态？返回前有哪些 `task` 工作；输出是否完整初始化；用户地址、标志和扩展状态由谁验证？

这五问不是固定写作模板，而是一组状态完整性检查。对于 `getpid`，第二问几乎为空；对于异步 I/O，第二和第四问会展开成独立请求对象与状态机；对于信号返回，第五问本身就是主要实现。问题的篇幅由机制决定，而不是强行平均。

本章最终得到的状态分层

从“用户不能直接改全局时钟”出发，本章没有立即假设一个完整操作系统入口。现在可以把已经逐步得到的状态按所有者归纳：

所有者	代表状态	负责回答的问题
处理器与每 CPU 入口区	入口地址、最早栈信息、CPU 局部状态	怎样安全离开用户执行环境
<code>task</code> 与内核栈	入口帧、调用链、切换帧、返回工作标志	本次控制流在哪里继续
系统调用边界层	ABI、编号、包装与过滤状态	用户究竟请求了什么
用户快照或固定映射	规范化参数、输出暂存、页引用	用户内存变化是否还能影响操作
对象与引用	打开对象、路径对象、请求对象	操作作用于谁，生命期到何时
凭据与安全策略	身份、能力集合、命名空间、LSM 上下文	谁能对该对象做哪项操作
等待与调度结构	等待队列、 <code>runqueue</code> 、已提交前缀	条件未满足时怎样停下并继续
返回协议	信号帧、重启状态、快慢路径条件	怎样把结果安全交还用户

这些结构并不都在每次调用中出现。它们分别处理不同的不稳定性：控制流会被打断，用户内存会变化，`fd` 槽会重用，凭据与对象状态会并发改变，等待会跨越调度，返回现场会被信号重写。结构存在的理由就是封闭相应变化；若某项调用不接触那类变化，就不应为了形式完整而制造无用状态。

核心思想

一次系统调用可以概括为四次所有权变化：处理器把用户继续状态交给 task 的入口帧；边界层把用户控制的参数变成内核拥有的快照；对象层取得稳定引用并在明确时刻提交效果；返回层把内核结果变成经过验证的用户继续状态。阻塞、信号、跟踪和优化都不能破坏这四次转换。

把入口语义应用到具体请求

分析一个系统调用时，按下面顺序写报告：

1. 用户态传入什么：整数、fd、地址、长度、flags、路径字符串。
2. 内核查找什么对象：进程表、fd 表、路径、inode、socket、VMA。
3. 检查哪些权限：对象存在、打开模式、读写执行、用户指针、资源限制。
4. 改变哪些状态：offset、buffer、引用计数、队列、页表、信号状态。
5. 返回什么：字节数、对象句柄、0/-1、errno、部分完成数量。
6. 哪些错误可重试，哪些错误必须失败，哪些错误表示调用者 bug。

以 `read(fd, buf, len)` 为例。内核先用 fd 查进程 fd 表，确认对象可读；再检查用户 buffer 是否可写；然后根据对象类型走普通文件、pipe、socket 或设备路径。普通文件可能从 page cache 复制，pipe 可能等待缓冲区，socket 可能返回当前已有字节。返回值为正时表示读取字节数，返回 0 可能是 EOF，返回 -1 时 errno 才解释失败原因。

以 `execve(path, argv, envp)` 为例。它不是创建新进程，而是在当前进程中替换用户态程序映像。PID 通常保持不变，地址空间被替换，用户栈重新构造，信号处理和 fd 状态按规则继承或关闭。若把 exec 理解成“启动新进程”，就无法解释 shell 中 fork+exec 的组合。

分发、用户内存与部分完成

系统调用分发

系统调用入口通常把系统调用号和参数放在约定寄存器中。内核入口保存用户上下文，检查系统调用号范围，从系统调用表中找到处理函数，再把参数交给对应实现。返回时把结果或错误码放回约定位置。

```
syscall_entry(trap_frame):
    nr = trap_frame.regs[SYSCALL_NR]
    if nr >= syscall_table_size:
        return -ENOSYS
    current.user_context = trap_frame
    result = syscall_table[nr](trap_frame.args)
    trap_frame.return_value = result
    return_to_user(trap_frame)
```

这段算法的关键不是函数指针表，而是入口边界。内核必须确认当前线程确实来自用户态，不能让用户随便伪造内核上下文；必须处理信号、抢占、审计、seccomp 和 tracing；必须保证错误路径也能恢复到一致状态。

用户指针复制

用户指针复制要处理跨页、部分可访问和竞态。简单版本可以逐字节或逐页检查；高性能版本会利用硬件异常处理快速路径。下面定义的是系统调用层的教学包装函数 `copy_user_bytes`，不是 Linux `copy_from_user` 的真实函数签名。包装层把底层的部分复制结果转换成成功字节数或 `-EFAULT`，以便先讲清提交语义。

```

copy_user_bytes(dst, user_ptr, len):
    copied = 0
    while copied < len:
        page = translate_user_page(user_ptr + copied, READ)
        if page == fault:
            return -EFAULT
        n = min(bytes_until_page_end(user_ptr + copied), len - copied)
        memcpy(dst + copied, kernel_map(page), n)
        copied += n
    return copied

```

真实内核还要处理 TOCTOU：检查之后用户线程可能修改内存。因此安全策略是把用户数据复制到内核缓冲区后再解析，不在解析过程中反复信任用户地址。路径字符串、数组参数和结构体参数都应遵守这个原则。

用 `write(fd, user_buf, 8192)` 看一次跨页复制。假设 `user_buf` 从页 A 的最后 100 字节开始，后面 8092 字节落在页 B 和页 C。页 A 可读，页 B 暂时没有 PTE 但 VMA 允许按需调入，页 C 不在任何 VMA 中。内核不能只检查起始地址，也不能在持有文件锁并已经修改文件状态后才发现页 C 不合法。稳妥路径通常是先把用户数据复制或 pin 到内核可控制的状态，再提交给文件或设备层。

片段	复制时发生什么	结果
页 A 尾部 100 字节	页表权限允许读，直接复制	<code>copied = 100</code>
页 B	PTE 不 present，缺页处理按 VMA 调入或分配页	成功后继续复制
页 C	找不到 VMA 或权限不允许读	教学包装层返回 <code>-EFAULT</code> ；真实 helper 报告未复制字节数

错误处理取决于提交点。如果系统调用还没有把任何字节交给文件对象，返回 `-EFAULT` 比较干净；如果某些接口允许部分完成，必须返回已经完成的字节数，并让调用者继续处理。关键是不能让内核对象处在“文件 offset 已经推进、page cache 半更新、但调用者只看到 `EFAULT`”的混乱状态。用户指针复制看似是内存问题，本质上是在保护系统调用的提交边界。

权限检查的顺序

权限检查要先定位对象，再检查主体是否有权操作对象，最后检查操作是否符合对象当前状态。以 `write(fd)` 为例：`fd` 必须存在；open file description 必须允许写；文件系统不能只读；用户 buffer 必须可读；长度不能超过限制；对象当前不能处于不允许写入的状态。

```

sys_write(fd, user_buf, len):
    file = fd_table_lookup(current, fd)
    if file == null: return -EBADF
    if !file.flags.can_write: return -EBADF
    if len > MAX_RW_COUNT: len = MAX_RW_COUNT
    kbuf = copy_user_bytes(user_buf, len)
    if kbuf.error: return -EFAULT
    return file.ops.write(file, kbuf, len)

```

不同错误码表达不同层次。`fd` 不存在和 `fd` 不可写都可能返回 `EBADF`；路径权限不足可能是 `EACCES`；文件系统只读可能是 `EROFS`；用户指针错误是 `EFAULT`。错误码越准确，调用者越能做正确恢复。

fork、exec、wait 的组合

Unix 风格进程创建不是一个巨大 `spawn`，而是 `fork`、`exec`、`wait` 的组合。`fork` 复制当前进程抽象；`exec` 替换当前进程映像；`wait` 回收子进程退出状态。这个设计让 `shell` 能在 `fork` 后、`exec` 前修改子进程 `fd`，例如重定向 `stdin/stdout`。

```
pid = fork()
if pid == 0:
    dup2(output_fd, STDOUT_FILENO)
    execve(program, argv, envp)
    _exit(127)
else:
    status = waitpid(pid)
```

算法不变量是：父进程和子进程在 `fork` 后分离推进；`exec` 成功后不会返回原用户程序；子进程退出后先变成 `zombie`，直到父进程 `wait` 取走退出状态。若没有 `zombie` 状态，父进程可能永远无法知道子进程怎样退出；若没有 `reaped`，进程表项会泄漏。

系统调用重启与信号

阻塞系统调用可能被信号打断。内核要决定返回 `EINTR`，还是在信号处理结束后自动重启系统调用。这个规则影响用户程序正确性：某些调用可安全重启，某些调用已经部分完成，不能假装从未发生。

```
blocking_read(fd, buf, len):
    while no_data_available(fd):
        ret = sleep_interruptible(fd.waitq)
        if ret == INTERRUPTED:
            if bytes_copied > 0:
                return bytes_copied
            if syscall_restartable(current):
                return -ERESTARTSYS
            return -EINTR
    return copy_available_data()
```

不变量是：已经对用户可见的部分完成不能被自动撤销。若 `read` 已经复制了 10 个字节，再收到信号，返回 10 比返回 `EINTR` 更能表达真实状态。调用者必须循环处理短读短写。

TOCTOU 与路径权限

`time-of-check to time-of-use` 是系统调用常见漏洞。比如先检查路径 `/tmp/a` 权限，再打开这个路径；中间攻击者把路径替换成符号链接，最终操作对象已经不是被检查对象。正确做法是把权限检查绑定到已经解析并持有引用的对象。

```
unsafe_open(path):
    if may_access_path(path):
        return open_path(path)

safe_open(path):
    inode = resolve_and_get_inode(path)
    if inode == null:
        return -ENOENT
    if !may_access_inode(current, inode):
        put_inode(inode)
        return -EACCES
    return make_file_from_inode(inode)
```

路径名是查找输入，不是稳定对象。`fd`、`inode` 引用和 `file` 引用才是内核能持有的对象身份。安全系统调用设计要尽量让检查和使用落在同一个对象引用上。

再看一个真实感更强的攻击时序。受害程序想安全打开用户指定目录下的 `data.txt`，先调用 `stat(path)` 检查它不是符号链接、权限也允许，然后再调用 `open(path)`。攻击者在两次调用之间把 `data.txt` 替换成指向 `/etc/shadow` 的符号链接。若内核或程序把“刚才检查过的路径字符串”当作对象身份，第二次打开的就可能是另一个对象。

时刻	动作	对象身份
1	受害者 <code>stat("/tmp/x/data.txt")</code>	检查的是旧 dentry/inode
2	攻击者 <code>rename</code> 替换路径分量	路径字符串不变，指向对象已变
3	受害者 <code>open("/tmp/x/data.txt")</code>	解析得到新 dentry/inode
4	程序继续写入	权限检查和使用对象不一致

更可靠的接口会缩短或消除这个窗口。例如先打开目录 `fd`，再用 `openat` 相对目录解析；使用 `O_NOFOLLOW` 拒绝末尾符号链接；在打开后对拿到的 `fd` 做 `fstat`，因为 `fd` 绑定的是已经打开的 `file` 对象；需要创建时用 `O_CREAT|O_EXCL` 把“检查不存在”和“创建”合成一个原子操作。原则不变：路径是名字，`fd/inode/file` 引用才是稳定对象。

信号、重启与错误恢复

系统调用测试要验证边界，而不只是验证 happy path。

- 非法 `fd` 返回 `EBADF`，不应修改其他状态。
- 只读 `fd` 写入失败，错误应表达权限问题。
- 用户 `buffer` 不可访问时返回指针错误，不应让内核崩溃。
- 短读短写被调用者正确循环处理。
- `EINTR` 和 `EAGAIN` 被区分处理。
- `fork` 后父子进程共享 `open file description` 的 `offset`，独立 `open` 则 `offset` 分离。
- `exec` 后用户地址空间替换，但按规则保留或关闭 `fd`。
- `seccomp`、资源限制或 `capability` 缺失时，请求被拒绝且有可解释错误。

测试结论要写清“内核没有做什么”。例如权限失败时没有写入数据，非法指针没有部分污染内核对象，系统调用被信号打断后调用者能判断是否需要重试。负面保证是 OS 边界的核心。

一次受控调用涉及的边界对象

系统调用深讲材料可以展开很多低级细节：`entry` 汇编、异常修复表、`capability`、`LSM`、`seccomp`、`vDSO`、审计、`ptrace`、`restart block` 和资源限制。主线阅读时先把它们压成一张账本。任一系统调用都要能说清：入口寄存器保存在哪里，用户指针何时复制成内核快照，权限在哪个稳定对象上检查，失败是否已经改变状态，返回前是否处理信号、审计和抢占。

边界	必须回答的问题	错误后果
----	---------	------

entry/return	用户 RIP/RSP/flags 和参数寄存器怎样保存与恢复	返回到错误特权级、泄露寄存器或跳过信号处理
用户内存	外层结构、内层指针和长度何时复制、何时 pin 页	E F A U L T 变 panic, TOCTOU, 整数溢出
对象解析	fd、path、inode、socket、namespace 是否已变成稳定引用	检查和使用不是同一对象
权限策略	DAC、capability、LSM、seccomp、rlimit 谁先失败	错误码混乱或绕过最小权限
提交点	哪一行之后状态对用户可见，失败如何回滚	fd 泄漏、引用计数泄漏、半初始化对象可见

这张账本能把看似分散的实现放回同一条路径。例如 `openat` 的提交点不是解析路径，也不是创建 `file` 对象，而是 `fd` 安装到进程 `fd` 表；在这之前失败必须释放 `file`、`dentry`、`inode` 和路径引用。在 `write` 中，返回正数可能表示部分完成；调用者不能把它和完整业务提交等同。在 `ioctl` 中，命令结构体版本、长度和 `padding` 都属于 ABI，不能因为“这是私有命令”就直接信任用户传入的内存布局。

凭据、能力与安全检查

x86-64 syscall 入口

x86-64 快速系统调用通常使用 `syscall/sysret`，入口地址和相关段选择子由 MSR 配置。进入内核后，低级入口要处理 `swaps`、保存用户栈、切到内核栈、保存寄存器、检查 `tracing/seccomp`，再进入 C 层分发。

```
entry_SYSCALL_64:
    swaps
    save user rsp into per-cpu area
    load kernel rsp
    push pt_regs
    call do_syscall_64
    check signals, reschedule, tracing
    restore regs
    sysretq or iretq
```

系统调用号表把 ABI 固定下来，定义宏再把具体内核实现接入相应表项。系统调用入口不能理解成普通函数指针调用：在进入分发之前，处理器和入口代码必须先完成特权转换、现场保存和寄存器协议的整理。

access_ok 与 copy 用户

`access_ok` 只做范围和用户地址边界的快速判断，不保证后续复制一定成功。真正复制仍可能 `fault`，所以 `copy_from_user` 必须有异常修复表或等价机制。真实内核会为用户访问指令建立 `fixup`：访问失败时跳到错误处理地址，而不是让内核普通 `page fault` 变成 `panic`。

```
copy_from_user(dst, src, len):
    if !access_ok(src, len):
        return bytes_not_copied
    instrument_usercopy_before()
    n = raw_copy_from_user(dst, src, len)
    instrument_usercopy_after()
    return n
```

这里返回值语义也很重要：很多内核 `helper` 返回“未复制字节数”，上层再转换成 `EFAULT` 或短完成。写教学代码时可以返回错误码，但必须明确是否允许部分复

制。

capable、ns_capable 和 user namespace

capability 判断不再只是“uid 0”。`capable(CAP_NET_ADMIN)` 检查当前主体是否有网络管理能力；`ns_capable(user_ns, cap)` 把能力放进某个 user namespace 里解释。容器依赖 user namespace 把容器内 root 映射成宿主上非特权身份。

```
if (!ns_capable(net->user_ns, CAP_NET_ADMIN))
    return -EPERM;
```

安全边界的常见错误是检查了全局 capability，却忘了对象所属 namespace；或者在 user namespace 中给了过宽能力，导致容器内操作影响宿主对象。权限判断必须跟随对象：network namespace、mount namespace、inode 或 cgroup 分别属于哪个 user namespace，决定能力应在哪个范围内解释。

LSM hook 和 seccomp

LSM 在 VFS、进程、socket、BPF 等路径插入 `security_*` hook。SELinux、AppArmor、BPF LSM 都可在这些点拒绝操作。seccomp 则在系统调用入口附近按 BPF filter 检查系统调用号和参数，返回 allow、errno、trap、trace、kill、user notify 等动作。

```
sys_openat:
    file = path_openat(...)
    ret = security_file_open(file)
    if ret:
        fput(file)
        return ret
    install_fd(file)
```

安全检查要放在对象已解析且引用稳定之后，否则路径 TOCTOU 仍可能存在。seccomp 适合收缩系统调用面，但它看不到所有内核对象语义；LSM 能看对象语义，但策略复杂。二者互补，不是替代关系。

vDSO 为什么能加速时间调用

有些“系统调用”只是读取内核维护的只读数据，例如当前时间。频繁进入内核会有上下文切换开销。vDSO 把一小段内核提供的用户态代码和只读数据映射到进程地址空间，使 `clock_gettime` 等调用能在用户态完成快路径。

```
clock_gettime():
    if vDSO available and clock mode stable:
        read seqlock-protected time data in user mapping
        return computed time
    else:
        syscall clock_gettime
```

vDSO 依赖 seqlock 一类协议：内核更新时改变序列号，用户态读取若发现并发更新就重试。它展示了一个重要设计：不是所有内核服务都必须每次陷入内核，稳定只读数据可通过受控共享映射给用户态。

系统调用表、ABI 和兼容层

系统调用 ABI 一旦发布，就很难改变。系统调用号、参数寄存器、返回值约定、错误码范围和结构体布局都属于用户态契约。32 位兼容进程还可能经过兼容系统调用层，把 32 位结构体转换为内核内部格式；这种转换本身也属于边界验证的一部分。

```
syscall ABI:
    rax = syscall number
```

```
rdi, rsi, rdx, r10, r8, r9 = arguments
rax = return value or negative errno
```

这解释了为什么内核常新增系统调用而不是随意改变旧系统调用语义。用户态程序、libc、容器 seccomp profile、审计工具和 trace 工具都依赖 ABI 稳定。教学 OS 也应把 syscall 编号集中管理，并写兼容性测试。

审计、ptrace 和 restart block

系统调用入口不只分发表项。它还可能经过 audit、ptrace、seccomp、ftrace 和 signal 检查。调试器通过 ptrace 可在 syscall enter/exit 停住进程，修改寄存器或观察参数；安全审计记录主体、对象和结果；信号可能让阻塞系统调用返回 `EINTR` 或进入 restart。

```
do_syscall_64(regs):
    nr = regs.orig_ax
    if ptrace_or_seccomp_active:
        nr = syscall_trace_enter(regs, nr)
    ret = dispatch_syscall(nr, regs)
    regs.ax = ret
    syscall_exit_work(regs)
```

restart block 用于保存重启系统调用所需状态。比如带 timeout 的阻塞调用被信号打断后，剩余 timeout 需要重新计算，而不能简单从头开始等待。系统调用重启是用户可见语义和内核内部状态的交叉点。

资源限制：rlimit 与 errno

权限检查之外，系统调用还要检查资源限制。进程可打开 fd 数、文件大小、地址空间、栈、锁定内存、进程数都可能受 rlimit 或 cgroup 限制。错误码要表达限制来源。

限制	触发路径	常见错误
open files	open/dup/socket/accept	EMFILE/ENFILE
file size	write/truncate	EFBIG 或信号
address space	mmap/brk	ENOMEM
locked memory	mlock	ENOMEM/EPERM
process count	fork/clone	EAGAIN

这类错误通常不是权限不足，而是资源边界。高质量程序应把 `EMFILE`、`ENOMEM`、`EAGAIN` 分开处理：释放资源、限流、等待或明确失败。

用户内存数组和二次复制

`execve`、`sendmsg`、`ioctl` 这类系统调用会传入指针数组或嵌套结构。内核不能只复制外层结构后继续信任内层用户指针。正确做法是逐层验证长度上限，把需要长期使用的数据复制到内核内存或 pin 住用户页。

```
copy_iovec(user_iov, count):
    if count > UIO_MAXIOV:
        return -EINVAL
    k_iov = copy_array_from_user(user_iov, count)
    for each entry in k_iov:
        if entry.len overflows total:
            return -EINVAL
        if !access_ok(entry.base, entry.len):
            return -EFAULT
    return k_iov
```

二次复制和整数溢出是系统调用漏洞高发区。长度相加要检查溢出，数组个数要有上限，结构体 padding 不能泄露内核栈数据，输出结构体应先清零再填字段。

系统调用、缺页和文件 I/O 实现深讲

本节把系统调用、虚拟内存、fd、VFS 和 page cache 连成一条实现路径。用户看到的是 `open/read/write/mmap/fork/exec`，内核看到的是 fd 表、file、inode、VMA、PTE、folio、bio、request 和设备 completion。深入理解 OS，必须能把这些对象按生命周期串起来。

系统调用入口的最小状态。

CPU 进入系统调用入口后，内核必须拿到系统调用号、参数寄存器、用户返回地址、用户栈、状态寄存器和当前线程。入口代码先建立可信执行环境，再调用 C 层分发。

```
syscall entry state:
  user RIP and RFLAGS
  user stack pointer
  syscall number
  argument registers
  current task pointer
  kernel stack pointer
  page table context
```

入口路径不能直接相信用户栈。用户栈可能不可访问、被另一个线程修改，甚至故意指向内核地址。参数寄存器只是整数，指针参数要在具体系统调用里通过 `copy_from_user` 或 `iov` 机制处理。

fd 表、file 和 inode。

文件描述符不是文件本身。fd 是进程 fd 表中的整数索引；索引指向 open file description，也就是内核的 `struct file`；file 再指向 inode 或 socket 等具体对象。多个 fd 可以指向同一个 file，共享文件 offset；多个 file 可以指向同一个 inode，各自有不同 offset 和 flags。

```
process files_struct:
  fd 0 -> file A -> inode terminal
  fd 1 -> file B -> inode output.txt
  fd 2 -> file B -> inode output.txt

dup(1):
  new fd points to same file B
  file offset is shared

open same path again:
  new file C points to same inode
  file offset is independent
```

这解释了 shell 重定向、dup2、fork 后 fd 共享、close-on-exec 的行为。若父子进程 fork 后共享同一个 file，它们写同一 fd 时共享 offset；若各自重新 open，同一 inode 也有不同 offset。

open 的路径解析。

`open("/a/b/c")` 不是一次查表。内核从当前 root 或 cwd 出发，逐级解析 dentry，处理挂载点、符号链接、权限、缓存命中和可能的文件系统 I/O。路径解析必须防止 race 和越权。

```
open path:
  choose starting point: root or cwd
  split path components
  for each component:
```



```

lookup dentry in dcache
if miss, ask filesystem lookup
follow mount point if needed
handle symlink subject to flags and limits
check execute/search permission on directory
check final file permissions and flags
allocate struct file
install into fd table

```

路径解析有大量安全边界。攻击者可能在检查后替换路径组件；符号链接可能指向意外位置；目录权限和文件权限不同；`O_NOFOLLOW`、`O_CLOEXEC`、`openat`、`openat2`都是在修复具体 `race` 或权限边界。

read 的三条路径：缓存命中、缓存未命中、直接 I/O。

普通文件 `read` 首先查 `page cache`。若命中，内核把缓存页复制到用户 `buffer`；若未命中，文件系统和块层发起 I/O，设备完成后再复制；若使用 `O_DIRECT`，则尽量绕过 `page cache`，把用户页或内核 `bounce buffer` 直接交给块层。

路径	数据流	代价
page cache hit	cache page 到用户 buffer	copy 成本，低延迟
page cache miss	storage 到 page cache，再 copy	I/O 等待，后续可复用
direct I/O	用户页和块设备之间 DMA 或接近 DMA	对齐、pin 页、生命周期复杂

直接 I/O 不是“一定更快”。它减少 `page cache` 污染和复制，但增加对齐、页 pin、设备队列和应用缓存管理成本。数据库常用它，是因为数据库自己管理缓存和写入顺序；普通应用通常受益于 `page cache`。

page cache 的 xarray 思想。

文件的 `page cache` 需要按文件 `offset` 快速找到页。Linux 使用 `address space` 和 `xarray` 一类结构管理文件页。概念上可以理解为：

```

file address_space:
    key = page index within file
    value = folio containing file data

read page:
    index = file_offset / PAGE_SIZE
    folio = xarray_lookup(mapping, index)
    if folio exists and uptodate:
        copy from folio
    else:
        allocate folio and start read I/O

```

`folio/page` 的状态位很重要：`uptodate` 表示内容有效，`dirty` 表示需要写回，`writeback` 表示正在写，`locked` 表示某个路径正在操作。缺少这些状态，多个线程会重复读同一页或同时写回同一页。

write 的脏页路径。

普通 `buffered write` 通常先把用户数据复制到 `page cache`，把页标记 `dirty`，然后返回。后台 `writeback` 或 `fsync` 再把 `dirty` 页写到存储设备。

```

buffered write:
    lookup or create page cache folio
    copy_from_user into folio
    mark folio dirty
    update file size if extending

```

```

return bytes copied

later writeback:
  pick dirty folios
  filesystem maps file blocks
  submit bios to block layer
  on completion clear writeback

```

这解释了为什么 `write` 可能很快而 `fsync` 很慢。前者只把数据推进到内核内存和文件系统状态，后者要等设备日志顺序满足持久化要求。

`mmap` 读文件：不经过 `read` 也用 `page cache`。

`mmap` 文件后，用户第一次访问对应地址会触发缺页。内核通过 `VMA` 找到文件和 `offset`，再从 `page cache` 找或读对应页，安装 `PTE`。之后用户态 `load/store` 直接访问映射页。

```

file mmap fault:
  CPU raises page fault at virtual address
  kernel finds VMA
  offset = vma.file_offset + (addr - vma.start)
  folio = page_cache_get_or_read(file.mapping, offset)
  install PTE mapping folio
  return to faulting instruction

```

所以 `read` 和 `mmap` 不是两套完全分离的缓存。它们常共享 `page cache`。一个进程 `read` 过的文件页，另一个进程 `mmap` 访问可能命中；反过来也成立。区别在于数据进入用户态的方式：`read` 复制，`mmap` 建 `PTE`。

`COW fault` 的实现分支。

`MAP_PRIVATE` 文件映射或 `fork` 后匿名页共享，都可能使用 `COW`。写入时，硬件发现 `PTE` 不可写，触发保护异常。内核检查 `VMA` 允许写且 `PTE` 是 `COW`，然后复制页。

```

COW write fault:
  locate VMA and PTE
  verify write is allowed by VMA
  if PTE is COW:
    old = pte.page
    new = allocate_page()
    copy_page(new, old)
    install writable PTE to new page
    decrement old refcount
    flush TLB for address
  else:
    signal permission fault

```

`COW` 的难点在并发。两个线程可能同时 `fault` 同一页；`fork`、`unmap`、`reclaim` 也可能并发修改页表。内核必须用页表锁、页引用计数和重试逻辑保证不会复制错误页或泄漏引用。

缺页错误如何变成信号或 `EFAULT`。

同一个坏地址，在不同上下文下结果不同。用户态直接访问坏地址，内核通常发送 `SIGSEGV`。系统调用 `copy_from_user` 过程中访问坏地址，系统调用应返回 `EFAULT`。内核自己访问坏内核地址，则是内核 `bug`，可能 `oops` 或 `panic`。

上下文	fault 结果	原因
用户指令 load/store	signal	用户程序违反地址空间规则

内核 copy user	-EFAULT	系统调用参数错误，可返回给调用者
内核普通指针	oops/panic	内核自身错误或硬件故障
缺页可修复	返回重试指令	懒分配、COW、文件页读入

这就是 exception table 的价值：内核某些 copy 用户路径可以声明“若这里 fault，跳到修复位置返回错误”，而不是让内核崩溃。

epoll 的 readiness 不是 completion。

文件和 socket 的 readiness 只表示“现在尝试某类操作可能不阻塞”，不是保证一定读到固定字节，也不是 I/O completion。多线程或边缘触发下，readiness 状态可能在应用处理前就变化。

```
epoll wait:
  if socket receive queue not empty:
    report readable
  user thread wakes
  another thread may consume data first
  read may still return EAGAIN on nonblocking fd
```

所以非阻塞 I/O 必须在循环中处理 EAGAIN。epoll 降低“等待很多 fd”的成本，但不取消对象状态变化和并发竞争。readiness API 和 io_uring completion API 的语义不同，不能混用理解。

io_uring 的提交和完成环。

io_uring 用共享 ring 减少系统调用次数。用户态写 submission queue entry，内核消费 SQE 并执行请求，完成后写 completion queue entry。关键仍是所有权、顺序和生命周期。

```
io_uring lifecycle:
  userspace fills SQE
  userspace advances SQ tail with store-release
  kernel observes SQ tail with load-acquire
  kernel validates fd, buffer, opcode
  request is issued to file/socket/block layer
  completion CQE is written
  userspace reads CQE and recycles resources
```

若 buffer 在 completion 前释放，异步路径可能使用悬空内存；若 fd 在请求执行前关闭，内核必须通过引用计数保证对象要么仍有效，要么请求明确失败；若应用不消费 CQE，完成队列会背压。

fd 生命周期和引用计数。

并发 close 和 I/O 是系统调用实现的经典难点。用户关闭 fd 后，fd 表项删除，新的 open 可能复用同一整数。但已经拿到 struct file 引用的内核 I/O 可以继续完成。

```
sys_read(fd):
  file = fget(fd)           // increments file refcount if fd valid
  if file == null:
    return -EBADF
  ret = vfs_read(file)
  fput(file)                // may release after last ref
  return ret

sys_close(fd):
  file = remove fd table entry
```

```
fput(file)
```

这解释了为什么 fd 整数不能长期当对象身份。close 后同一个数字可能马上指向另一个文件。内核内部必须使用对象引用，用户态高层库也要小心 fd reuse 带来的取消和多线程 race。

文件系统日志和 page cache 的关系。

page cache 保存文件数据，文件系统日志通常保护元数据或特定模式下的数据顺序。dirty page 写回和日志提交是相关但不同的状态机。fsync 必须把两者推进到一致点。

```
fsync(file):
    write dirty file data pages
    wait data writeback completion
    commit filesystem metadata transaction
    issue flush/FUA as needed
    return only after required durability point
```

不同文件系统和挂载选项会改变数据与元数据顺序。例如 ordered 模式要求相关数据块在提交元数据前写出；writeback 模式可能只保证元数据一致；journal data 模式连数据也进日志。应用不能凭感觉推断崩溃语义，必须理解目标文件系统契约。

把一次请求完整编号。

分析复杂 I/O 时，给每一层对象编号能避免混乱。

编号	对象	状态问题
1	用户 fd 整数	是否仍指向预期 file
2	struct file	引用计数、offset、flags
3	inode/address space	文件大小、page cache、权限
4	folio/page	uptodate、dirty、locked、writeback
5	bio/request	块映射、队列、超时
6	DMA buffer/descriptor	map/unmap、所有权、completion
7	device command	status、错误码、reset 影响

任何一层状态没写清楚，都会产生 bug。例如 fd 已 close 但 file 引用还在，page dirty 但 writeback 未完成，request 已超时但设备稍后仍 completion，reset 后旧 DMA 仍可能写内存。

实现级测试矩阵。

验收标准

- dup 后两个 fd 共享 offset；重新 open 的 fd 不共享 offset。
- O_CLOEXEC fd 在 exec 后关闭，普通 fd 按规则保留。
- 坏用户指针在 read/write 中返回 EFAULT，不导致内核崩溃。
- MAP_PRIVATE 写入后文件内容不变，MAP_SHARED 按规则变 dirty。
- fork 后 COW 写入不影响父子另一方。
- close 与并发 read/write 不造成 use-after-free，fd reuse 不混淆对象。
- page cache 命中路径不发块 I/O，冷缓存路径能观察到块 I/O。

- `fsync` 后再模拟崩溃，文件数据和目录项语义符合预期。
- 非阻塞 `fd` 在未 `ready` 时返回 `EAGAIN`，`epoll` 事件后仍能处理竞态。
- `io_uring` 请求在 `completion` 前不能释放 `buffer` 或丢失对象引用。

入口契约的实现证据

第三阶段深讲：特权边界、装载和内存提交点

系统调用、`exec` 和缺页处理都在改变“谁能访问什么”。这类代码最怕半提交：权限已经放开但对象还没准备好，地址空间已经切换但旧程序还要返回错误，页表已经失效但物理页提前释放。学习这一章时，应把每条路径拆成准备、验证、提交、回滚四个阶段，而不是只记住 API 名字。

边界代码的共同结构。

边界代码有三个共同问题。第一，输入来自不可信主体：系统调用参数来自用户寄存器和用户内存，ELF 文件可能被攻击者构造，`page fault` 地址可能来自错误或恶意访问。第二，执行路径会改变全局可见状态：`fd` 表、地址空间、页表、文件页、凭据和调度状态。第三，失败必须可解释：有些失败返回错误码，有些发送信号，有些说明内核自身已经损坏。

```
boundary operation:
  snapshot untrusted input
  validate object identity and permissions
  allocate temporary resources
  build new state off to the side
  commit with a short ordered sequence
  publish visibility only after invariants hold
  roll back temporary resources on every pre-commit failure
```

这个结构适用于 `openat`、`execve`、`mmap`、`mprotect`、`COW fault` 和文件写回。源码阅读时，只要找到“提交点”在哪里，前后语义就会清楚很多：提交点之前失败要回到旧状态，提交点之后失败只能按新状态处理或终止任务。

系统调用返回值也是 ABI。

系统调用的结果不只是一个整数。ABI 规定返回值寄存器、错误码编码、哪些寄存器被破坏、哪些状态在返回前必须处理。`libc wrapper` 再把内核返回值转换成 `-1` 和 `errno`。如果教学 OS 忽略这些细节，用户程序就只能靠猜。

返回类型	例子	调用者应该怎样理解
非负数量	<code>read/write</code> 返回字节数	可能短读/短写，不等于请求全部完成
新句柄	<code>open/socket/dup</code> 返回 <code>fd</code>	<code>fd</code> 是进程表索引，不是对象永久身份
0 表示成功	<code>close/mprotect/munmap</code>	只说明接口语义完成，不说明持久化
<code>-errno</code>	内核内部常见约定	<code>wrapper</code> 转成 <code>errno</code> 或直接传递
信号	用户访存错误、非法指令	错误发生在用户执行流，不一定有 <code>syscall</code> 返回

`close` 的错误尤其容易被误解。`fd` 表项可能已经释放并可被复用，即使底层写回错误在 `close` 期间才报告。用户态不能在 `close` 返回错误后再重试同一个 `fd`，

因为这个整数可能已经指向别的文件。正确设计要把持久化错误尽量通过 `fsync` 这类明确边界暴露。

restartable syscall 和 EINTR。

阻塞系统调用可能在等待期间收到信号。内核有时返回 `EINTR`，有时把系统调用改写成可重启状态，让信号处理返回后重新执行。重启不是简单“再调一次函数”，因为部分 I/O、超时参数和用户可见副作用必须精确处理。

```
blocking read:
  copy no bytes yet
  sleep waiting for data
  signal arrives
  if handler installed:
    either return -EINTR
    or arrange restart after signal frame

partial read:
  copied some bytes
  signal arrives
  return copied bytes, not -EINTR
```

这个规则解释了为什么系统调用实现要记录“是否已经产生用户可见进展”。已经复制给用户的字节、已经消耗的 `socket` 数据、已经创建的 `fd`，不能因为信号到来就假装从未发生。测试时应同时覆盖“阻塞前被打断”和“部分完成后被打断”。

copy_from_user 的分段提交。

大 `read/write` 不应一次性申请巨大内核缓冲。常见实现会分段处理用户 `iovec` 或页片段。分段处理带来短完成语义：前几段成功，后面遇到坏页或信号，返回值可能是已经完成的字节数。

```
write_iter(file, iov):
  done = 0
  for each segment in iov:
    while segment has bytes:
      chunk = copy_chunk_from_user(segment)
      if chunk fault:
        return done > 0 ? done : -EFAULT
      ret = file_write_chunk(file, chunk)
      if ret < 0:
        return done > 0 ? done : ret
      done += ret
  return done
```

这里的提交粒度是 `chunk`。若 `chunk` 已写入 `pipe` 或 `socket queue`，就不能在后续 `fault` 时撤销。文件系统写入还要区分 `page cache dirty`、`journal commit` 和设备持久化。一本教材若只说“复制用户 `buffer` 再写文件”，读者会漏掉短写、`EFAULT` 和持久化边界。

exec 的三道权限门。

`execve` 至少经过三道权限门。第一道是路径和文件权限：调用者是否能走到该文件，文件是否可执行，挂载是否 `noexec`。第二道是二进制格式和装载权限：ELF 段权限是否合理，解释器是否允许执行，栈是否可执行。第三道是凭据转换：`setuid`、`file capability`、`LSM`、`no_new_privs` 和 `ptrace` 规则共同决定新程序的身份。

```
exec permission gates:
  path permission and executable inode
  binary format and memory layout policy
  credential transition and security hooks
```

这三道门的错误码和失败时机不同。路径不存在是 `ENOENT`；不是可执行格式可能是 `ENOEXEC`；参数过大是 `E2BIG`；权限不足是 `EACCES` 或 `EPERM`；解释器不存在也会让整个 `exec` 失败，但旧程序必须还能拿到错误。把这些都归为“启动失败”，无法写出 `shell`、容器运行时或服务管理器。

ELF 段权限不是 section 权限。

运行时装载看 `program header`，链接和调试看 `section header`。很多 ELF 被 `strip` 后没有完整 `section` 信息，仍然能运行。教学装载器如果读取 `.text/.data/.bss` 名字来建立映射，就是把链接器视角误当成内核视角。

视角	使用对象	典型消费者
运行时装载	<code>PT_LOAD/PT_INTERP/PT_PHDR</code>	内核 <code>exec</code> 、动态链接器
链接和重定位	<code>section</code> 、 <code>symbol</code> 、 <code>relocation</code>	链接器、 <code>objdump</code> 、调试器
调试展开	<code>DWARF</code> 、 <code>CFI</code> 、 <code>eh_frame</code>	<code>gdb</code> 、异常展开、 <code>profiler</code>

段映射还要处理文件尾页。若 `p_filesz` 结束在页中间，而 `p_memsz` 更大，同一页中从文件末尾到页末的部分必须清零；后续整页 BSS 也要清零或懒分配零页。这个细节若错，未初始化全局变量会读到旧文件或未清理内存。

地址空间切换的提交顺序。

`exec` 提交时，最危险的是先激活新页表再访问旧地址空间数据，或先释放旧 `mm` 再需要从旧用户栈取参数。安全做法是：提交前把 `path`、`argv`、`envp`、解释器路径和 `auxv` 所需数据复制到内核或新栈页；新 `mm` 完整可用后再短序列切换。

```
safe exec commit:
  prepare new_mm, new_stack, new_regs
  copy all old user data before old_mm is dropped
  stop sibling threads if multithreaded
  install credentials that belong to new image
  install current.mm = new_mm
  switch hardware page table
  make old_mm unreachable from current
  drop old_mm references
  return to new user entry
```

顺序里有两个可见性点：`current.mm` 对内核其他路径可见，硬件页表根对 CPU 访存可见。多核系统还要考虑同一地址空间是否仍在其他 CPU 活跃。若旧 `mm` 的页表页在远端 CPU TLB 失效前被释放，后果和普通 `munmap` 的 `shutdown` 错误一样严重。

mprotect 的权限收缩和扩大。

`mprotect` 改权限时，扩大权限和收缩权限的风险不同。扩大权限要检查 `VMA` 的 `may` 权限和文件打开模式；收缩权限要立刻让硬件旧权限失效。比如把 `RW` 改成 `R`，如果不 `flush TLB`，CPU 可能继续使用缓存的 `writable` 翻译。

```
mprotect(range, new_prot):
  split vmas at range boundaries
  for each vma in range:
    if new_prot exceeds vma.may_prot:
      rollback splits if needed
      return -EACCES
  update vma.prot
  update present PTE permission bits
  flush TLB for affected range
```

VMA 切分是另一个回滚点。若先切分出三个 VMA，后续某段权限检查失败，需要合并回原状态或保证中间状态不会被观察到。真实内核会通过锁保护 mm 结构，让其他线程不能在半修改区间中处理 fault。

munmap 和延迟释放。

munmap 删除映射后，用户不能再访问这段虚拟地址。但对应物理页不一定立刻释放：其他进程可能映射同一文件页，I/O 可能持有页引用，RCU 或 TLB gather 可能延迟释放页表页。语义上的不可访问和资源上的最终释放是两件事。

```
munmap(range):
    remove or split VMAs
    clear PTEs in range
    collect old PTE pages and mapped pages into batch
    flush TLB for range on all active CPUs
    drop page references after stale translations are gone
    free page table pages when no walker can see them
```

这能解释为什么内存统计不是瞬时下降，也解释为什么页表释放要和 TLB shutdown、页表遍历锁、RCU 等机制配合。教学 OS 可简化，但必须保留顺序原则：先撤销可达性，再失效旧翻译，最后释放底层资源。

page fault 的锁顺序。

缺页路径会碰到 mm 锁、VMA 锁、页表锁、page cache 锁、folio 锁、文件系统锁和 I/O 等待。锁顺序错了会出现非常隐蔽的死锁。一个基本原则是：先稳定地址空间和 VMA，再稳定页表项，再稳定物理页或文件页；等待 I/O 时尽量不持有会阻塞其他 fault 的大锁。

```
file page fault:
    take mmap read lock or VMA lock
    find and validate VMA
    locate page cache folio
    if folio locked or needs I/O:
        drop/reduce locks that block unrelated faults
        wait for folio
        retry validation
    take page table lock
    install PTE if still valid
    drop locks
```

这里的 retry 很重要。等待 I/O 期间，另一个线程可能 munmap 或 mprotect 了同一区间。醒来后不能直接安装 PTE，必须重新验证 VMA 和权限。许多内存管理 bug 都来自“睡眠前检查过，醒来后不再检查”。

源码阅读路线。

这一阶段读 Linux 源码可以按路径而不是文件顺序推进。

路径	关键文件或函数	读法
系统调用入口	arch/x86/entry/、kernel/seccomp.c	参数、seccomp、返回工作
用户复制	arch/x86/lib/usercopy*、lib/iov_iter.c	fault fixup、短拷贝、iov
exec/ELF	fs/exec.c、fs/binfmt_elf.c	准备新镜像和提交点
VMA 管理	mm/mmap.c、include/linux/mm_types.h	maple tree、切分、合并

缺页	mm/memory.c、mm/filemap.c	anon/file/COW fault 分支
页表失效	mm/tlb.c、架构 TLB 代码	batch、shutdown、 延迟释放
回收/OOM	mm/vmscan.c、mm/oom_kill.c	reclaim 顺序和 victim 选择

每条路径都按同一张表做笔记：输入对象、持有锁、可睡眠点、提交点、回滚对象、用户可见结果。这样能把几万行源码压成可理解的状态机。

边界反例集。

高质量测试要故意打破边界。

反例	构造方法	必须观察到
用户指针半页坏	iovec 前半有效后半 unmapped	短完成或 EFAULT， 内核不崩
exec 坏解释器	ELF 的 PT_INTERP 指向不存在 路径	exec 失败，旧程序 继续
ELF 溢出段	p_vaddr + p_memsz 整数溢出	拒绝装载
mprotect 后旧权限	RW 改 R 后立即写	必须 fault，不能 被 TLB 旧权限放行
fault 等 I/O 时 munmap	线程 A fault 等页，线程 B unmap	A 醒来后重新验证 并失败
close 与 read 并发	一个线程 read 已 fget，另一个 close 并复用 fd	read 用旧 file 引用，fd 数字可 复用
memcg OOM	cgroup 内触发限制	只在 cgroup 内选 择 victim

这些反例比正常路径更能检验实现质量。正常路径只证明代码能跑，反例才能证明边界没有漏。

第一阶段源码走读：x86-64 系统调用入口

第一阶段不能只说“系统调用进入内核”。真正要过关，必须能把一条 `syscall` 指令映射到 x86-64 寄存器约定、低级入口汇编、`pt_regs`、C 层分发、用户指针复制、错误码和返回路径。这样后面讲 `open`、`read`、`mmap`、`exec`、信号和 `seccomp` 时，读者知道每个对象为什么能被内核检查，而不是把系统调用当成神秘函数表。

核心理想

x86-64 `syscall` 入口是 OS 最核心的安全边界之一。它不是“跳到一个 C 函数”，而是先由 CPU 和入口汇编建立可信执行环境，再由 C 层根据系统调用号、寄存器参数、当前任务、`seccomp`、`tracing`、权限和用户指针决定能否推进。

先固定源码入口和读法。

读 Linux 时先看这些路径。不同内核版本函数名会有细微变化，但对象和边界基本稳定。

源码路径	先看什么	读法
arch/x86/entry/entry_64.S	entry_SYSCALL_64、返回用户态路径	看寄存器怎样保存、栈怎样切换、何时走慢路径
arch/x86/entry/common.c	do_syscall_64、exit work	看系统调用号、seccomp、tracing、信号、调度检查
arch/x86/entry/syscalls/syscall_64.tbl	系统调用号表	看 ABI 如何把数字绑定到实现名
include/linux/syscalls.h	SYSCALL_DEFINE* 声明	看 C 实现怎样接入统一表项
arch/x86/include/asm/ptrace.h	struct pt_regs	看 trap frame 在 C 层怎样表示
lib/usercopy.c、arch/x86/lib/usercopy_64.c	usercopy	看用户指针复制怎样和异常修复配合

读入口代码时不要从第一行汇编硬啃。先带着四个问题读：用户态把什么放进寄存器，CPU 自动改了什么，入口汇编补保存了什么，C 层分发前还有哪些安全工作。

用户态 ABI：参数不是压到栈上。

x86-64 Linux 系统调用 ABI 把系统调用号放在 `rax`，最多六个参数放在 `rdi/rsi/rdx/r10/r8/r9`。注意第四个参数用 `r10`，不是 C 函数调用 ABI 的 `rcx`。原因是 `syscall` 指令会用 `rcx` 保存用户返回 RIP，用 `r11` 保存用户 RFLAGS。

寄存器	syscall ABI 含义	为什么重要
<code>rax</code>	系统调用号，返回时放返回值	调度到哪个 <code>sys_*</code> 实现
<code>rdi</code>	第 1 参数	例如 <code>write(fd,...)</code> 的 <code>fd</code>
<code>rsi</code>	第 2 参数	例如用户 <code>buffer</code> 指针
<code>rdx</code>	第 3 参数	例如长度
<code>r10</code>	第 4 参数	<code>rcx</code> 被硬件占用，不能放参数
<code>r8/r9</code>	第 5/6 参数	六参数系统调用使用
<code>rcx</code>	用户返回 RIP	<code>sysretq</code> 快速返回需要
<code>r11</code>	用户 RFLAGS	返回时恢复用户可见标志

这张表解释了为什么系统调用 `wrapper` 不是普通 C 调用。`libc` 或内联汇编必须把参数移动到这些寄存器，再执行 `syscall`。如果你在调试器里看系统调用入口，先看寄存器，而不是看用户栈。

CPU 做了什么，入口汇编又补了什么。

`syscall` 指令本身只做一部分特权转移：保存返回 RIP 到 `rcx`，保存 RFLAGS 到 `r11`，把 RIP 改成内核配置的入口地址，按 MSR 配置更新特权相关状态。它不会自动保存所有通用寄存器，也不会替你验证用户参数。

```
before syscall:
  rax = syscall number
  rdi/rsi/rdx/r10/r8/r9 = arguments
  rip = address of syscall instruction
  rsp = user stack
```



```
after CPU syscall transition:
    rcx = user return rip
    r11 = user rflags
    rip = IA32_LSTAR entry address
    execution enters ring 0 entry path
```

随后 `entry_64.S` 的入口代码补齐内核需要的上下文：处理 `swaps`，找到当前 CPU 的内核数据，切换到当前线程的内核栈，按约定把寄存器保存成 `pt_regs` 形状，再调用 C 层分发函数。

```
entry_SYSCALL_64 outline:
    swaps or equivalent per-cpu base transition
    save user rsp
    load current task kernel stack
    push user ss, user rsp, rflags, user cs, user rip style frame
    save general-purpose registers into pt_regs
    call do_syscall_64(regs, nr)
```

这里的关键不是背汇编每一行，而是理解入口汇编在构造一个 C 层可读的事实包：这个任务从哪里来，用户 RIP/RSP/RFLAGS 是什么，参数寄存器是什么，返回时要恢复到哪里。

pt_regs：低级入口交给 C 代码的证据包。

`pt_regs` 可以理解为“这次陷入的证据包”。系统调用、page fault、中断和调试异常都会需要类似对象，只是字段含义和错误码不同。系统调用路径尤其关心参数寄存器、系统调用号、用户返回 RIP、用户栈和返回值位置。

pt_regs 类字段	来源	用途
通用寄存器	用户态执行 <code>syscall</code> 前的寄存器	提取参数、保存用户可见状态
<code>orig_ax</code>	原始系统调用号	tracing/seccomp/restart <code>syscall</code> 需要原号
<code>ip</code>	用户返回 RIP	信号、ptrace、返回路径验证
<code>sp</code>	用户 RSP	信号帧构造和调试输出
<code>flags</code>	用户 RFLAGS	单步、方向标志、中断相关状态
<code>ax</code>	返回值位置	C 层结果写回给用户态

入口代码漏保存字段，后面不是“少打印一个值”，而是可能破坏调试、信号、系统调用重启、seccomp、ptrace 和返回用户态。第一阶段读源码时要形成这个判断：低级结构体字段是安全边界的一部分。

C 层分发：不只是查表调用。

C 层分发通常可以抽象成下面的流程，但真实路径还要处理 tracing、audit、seccomp、ptrace、系统调用重启、信号和返回用户态前的 work。

```
do_syscall_64(regs, nr):
    if nr is outside syscall table:
        regs.ax = -ENOSYS
        return
    if syscall_enter_work is pending:
        run tracing/audit/seccomp hooks
        nr may be changed or rejected
    result = syscall_table[nr](regs arguments)
    regs.ax = result
    if syscall_exit_work is pending:
```

```
run tracing/audit/signal/resched checks
```

这说明系统调用号表只是中间一步。比如 `seccomp` 可能在真正执行前拒绝某个系统调用；`ptrace` 可能观察或改写参数；`audit` 可能记录事件；信号可能导致阻塞系统调用返回 `-EINTR` 或被重启。把系统调用理解成“数组下标调函数”会漏掉安全和可观测性。

以 `write(fd, buf, len)` 追一条完整路径。

用户态调用 `write(1, buf, len)` 时，硬件入口只知道三个整数：`fd`、用户地址、长度。它不知道这是终端、文件、`pipe` 还是 `socket`。内核必须逐层把整数还原成受保护对象。

```
write(fd, user_buf, len):
    fd table lookup -> struct file
    check file was opened for writing
    copy or pin user buffer under user access rules
    dispatch file->f_op->write_iter
    update file offset or stream queue
    return bytes written or negative errno
```

如果 `fd` 无效，返回 `-EBADF`。如果用户 `buffer` 不可读，返回 `-EFAULT`。如果对象暂时不能写且是非阻塞 `fd`，返回 `-EAGAIN`。如果信号打断等待，可能返回 `-EINTR`。这些错误不是装饰，它们分别来自 `fd` 表、用户地址空间、对象等待语义和信号状态。

用户指针复制：`fault` 可以是正常错误路径。

用户指针的特殊性在于：它来自不可信地址空间，但内核必须按请求访问它。直接解引用用户指针会把用户错误变成内核错误。正确路径是 `usercopy`：访问前后有范围、权限、异常修复和对对象生命周期约束。

```
copy_from_user kernel idea:
    while bytes remain:
        try load from user address
        if page fault happens at a usercopy instruction:
            jump to exception-table fixup
            return bytes not copied or -EFAULT
        store into kernel-owned buffer
```

异常表的价值在这里体现出来：同样是 `page fault`，如果 `faulting RIP` 在 `usercopy` 的异常表范围内，它是用户输入错误；如果发生在普通内核指针解引用上，它更可能是内核 `bug`。第一阶段讲 `fault` 时必须把这两类区分开。

返回路径：快返回和慢返回。

系统调用返回不是简单 `return result`。内核要决定能否走快速 `sysretq` 风格路径，还是必须走更通用的 `iretq` 风格路径。若用户返回 `RIP/RFLAGS`、`tracing`、信号、单步调试、抢占或兼容状态需要特殊处理，就可能走慢路径。

返回条件	快路径倾向	慢路径原因
普通系统调用，无 pending work	可快速恢复寄存器并返回	无
有信号待投递	不适合直接快返	需要构造用户信号帧
需要调度	不适合直接快返	返回前要切换任务
<code>ptrace</code> 、 <code>seccomp</code> 、 <code>audit</code>	不适合直接快返	需要通知观察者或改写结果

返回状态不满足快速指令限制	走通用返回	<code>iretq</code> 能表达更完整状态恢复
---------------	-------	-------------------------------

这也是为什么返回路径经常比入口更容易出安全问题。入口时内核取得控制权；返回时内核把控制权交回给不可信程序，必须保证所有用户可见状态都被正确恢复，内核私密状态没有泄漏。

观察证据：用真实系统校准理解。

原理卷不要求读者修改内核，但要求能把系统调用入口的解释和系统证据对上。

命令或文件	能观察什么	如何判读
<code>strace -e trace=write ./a.out</code>	用户态发起的系统调用和 <code>errno</code>	只能看到边界结果，看不到内核内部等待
<code>perf trace -e syscalls:sys_enter_*</code>	<code>syscall</code> enter/exit tracepoint	能看到系统调用号、参数和返回时序
<code>grep syscall /proc/kallsyms</code>	内核符号是否可见	受权限和 <code>kallsyms</code> 设置影响
<code>cat /proc/<pid>/syscall</code>	线程当前 <code>syscall</code> 和参数	只在任务正在系统调用中时有意义
<code>bpftrace tracepoint:syscalls:*</code>	<code>syscall</code> tracepoint 采样	可按 <code>pid</code> 、 <code>comm</code> 、 <code>errno</code> 过滤

用这些证据时要保持边界感。`strace` 看到 `write(1,...)=5`，只能证明用户/内核边界返回了 5；它不能证明数据已经落盘，也不能说明底层是否发生 `page fault`、锁等待、设备 I/O 或信号处理。要解释更深路径，需要结合 `perf`、`ftrace`、`eBPF`、`/proc/<pid>/stack` 和具体子系统日志。

第一阶段验收题。

读完本节，应能不看书回答这些问题。

1. 为什么 x86-64 Linux 第四个系统调用参数放在 `r10` 而不是 `rcx`？
2. `rcx` 和 `r11` 在 `syscall` 入口中分别保存什么？
3. 为什么入口汇编要构造 `pt_regs`，而不是直接调用 C 函数？
4. `do_syscall_64` 查表前后为什么还有 `seccomp`、`tracing`、`audit` 和 `signal work`？
5. `copy_from_user` 遇到 `page fault` 为什么通常返回 `EFAULT` 而不是让内核崩溃？
6. `sysretq` 和 `iretq` 都能回用户态，为什么还要区分快路径和慢路径？
7. `strace` 能证明什么，不能证明什么？

这些问题能答清，系统调用就不再是“用户调用内核函数”，而是一条从 x86-64 硬件入口、低级汇编、C 层分发、对象权限、用户内存和返回路径共同组成的受控边界。

尚未解决的问题。受控入口可以接收编号、验证参数并返回错误，但磁盘中的机器码文件仍不是一份可运行的地址空间。下一章从格式识别和段权限开始，建立新映像并寻找不可回退的提交点。

第 4 章 为什么程序字节还不是可运行映像

继承的机器。 `task` 可以通过受控入口请求内核服务，内核能够稳定引用对象并在阻塞后继续同一项请求。当前问题是把一个已有 `task` 的旧映像替换为文件中的新程序，同时保证校验失败时旧映像仍可继续，提交以后所有返回状态都属于新程序。

程序、进程与程序映像

`fork` 复制了一个进程的执行环境，而 `exec` 把当前进程的用户态程序替换成另一个程序。这个替换不是“创建新进程”，因为 `pid`、父子关系、部分资源限制和很多内核对象仍然属于同一个进程；它也不是普通文件读取，因为内核必须验证可执行文件格式、建立新的地址空间、设置入口点、构造用户栈、处理解释器、重置部分信号语义并关闭带 `close-on-exec` 标志的 `fd`。

ELF 装载器是系统调用边界、文件系统、虚拟内存和权限模型的交汇点。用户给内核一个路径和参数数组，内核必须在不信任用户内存的前提下读取这些字符串；路径解析得到 `inode` 后，还要检查执行权限、文件类型、挂载选项和安全策略；读取 ELF 头后，装载器根据 `program header` 而不是 `section header` 建立映射。`section` 是链接和调试视角，`program segment` 才是运行时装载视角。

核心思想

`exec` 的核心契约是原子替换用户态映像：成功后旧程序不能继续执行；失败时旧程序必须仍然可运行，除非失败发生在明确不可恢复的提交点之后。

真实系统还要处理动态链接。若 ELF 带 `PT_INTERP`，内核并不直接跳到主程序入口，而是同时映射解释器，把控制权交给动态链接器。动态链接器再加载共享库、重定位符号，最后跳到用户程序的入口。内核只负责把 `aux vector`、入口、`program header` 信息和随机数等必要元数据放到用户栈。

`exec` 引起的完整状态替换

一个教学内核中的 `execve(path, argv, envp)` 可以拆成九个阶段：

1. 从用户内存复制 `path`、`argv`、`envp`，设置长度上限。
2. 路径解析并打开可执行 `inode`。
3. 检查文件类型、执行权限、`noexec` 挂载选项和安全策略。
4. 读取并验证 ELF header。
5. 扫描 `program header`，计算新地址空间布局。
6. 创建临时地址空间，映射 `PT_LOAD` 段和用户栈。
7. 若存在 `PT_INTERP`，装载解释器并设置实际入口。
8. 构造用户栈：`argc`、`argv` 指针、`envp` 指针、`auxv`、字符串和对齐填充。
9. 提交：切换进程 `mm`、寄存器入口和栈指针，关闭 `CLOEXEC` `fd`。

这里最容易讲错的是失败回滚。前七步应尽量在临时对象中完成：临时 `mm`、临时 `VMA`、临时页、临时 `fd` 引用。只有全部验证成功后，才进入提交阶段并销毁旧地址空间。否则一个坏 ELF 就可能把调用者进程破坏到无法返回错误码。

对象	成功后状态	失败回滚
旧 mm	被新 mm 替换并释放引用	保持可运行
新 mm	成为 current 的地址空间	释放所有 VMA 和页
文件引用	可执行文件可关闭或保留映射引用	put inode/file 引用
fd table	关闭 FD_CLOEXEC 条目	不应改变
信号处理	捕获处理器重置，忽略语义按规则保留	不应改变
寄存器	RIP 指向入口，RSP 指向新用户栈	不应改变

把这条路径放进一个具体命令会更清楚。假设 shell 进程 pid 为 3182，执行：

```
execve("/usr/bin/grep",
      ["grep", "-n", "main", "a.txt"],
      envp);
```

进入内核时，`path`、`argv` 和 `envp` 都还在旧地址空间里。这里的旧地址空间不是“快要没用的垃圾”，而是内核目前唯一能读到这些字符串的地方。内核不能先释放旧 mm 再慢慢读参数；一旦旧页表被拆掉，`argv[2]` 指向的 “main” 可能就再也翻译不出来。因此第一轮工作是复制输入：先复制路径字符串，再按指针数组逐个复制 `argv/envp` 字符串，并统计总字节数。若 `argv[3]` 恰好跨到一个不可读页，正确结果是 `-EFAULT`，旧 shell 继续运行，fd table、signal handler 和旧地址空间都不能被改坏。

阶段	内核手里的稳定证据	此时失败应怎样收场
复制用户参数	内核缓冲区里的 <code>path</code> 、 <code>argv</code> 、 <code>envp</code> 副本	返回 <code>EFAULT/E2BIG</code> ，旧程序继续
路径解析	指向 <code>/usr/bin/grep</code> 的 inode/file 引用	返回 <code>ENOENT/EACCES</code> ，释放引用
ELF 验证	ELF header 和 program header 副本	返回 <code>ENOEXEC</code> ，旧 mm 不变
临时装载	新 mm、VMA、栈页、解释器信息	释放临时 mm，旧程序继续
提交	新入口、新栈、新凭据、CLOEXEC 关闭集合	成功后不再回到旧用户代码

接着内核解析路径，得到稳定的 inode 和 file 引用。权限检查必须绑定这个对象，而不是只检查字符串 “/usr/bin/grep”。检查通过后，内核读取 ELF header。若这是动态链接的程序，program header 里通常有 `PT_INTERP`，例如指向 `/lib64/ld-linux-x86-64.so.2`。这意味着“用户写的是执行 grep”，但第一条用户态指令很可能属于动态链接器；动态链接器根据 `auxv` 找到主程序的 program header，完成共享库装载和重定位，最后再跳到 grep 的入口。这个过程解释了为什么 `exec` 栈上要有 `AT_PHDR`、`AT_PHNUM`、`AT_ENTRY`、`AT_BASE` 和 `AT_RANDOM`，它们不是装饰字段，而是用户态运行时启动所需的证据。

再看文件描述符。shell 可能提前打开了管道：

```
grep -n main a.txt | less
```


这时 `grep` 进程继承的 `STDOUT_FILENO` 可能不是终端，而是管道写端。`exec` 成功后，普通 `fd` 继续指向同一个 open file description；带 `FD_CLOEXEC` 的 `fd` 则在提交时关闭。这个规则必须在提交点附近执行：太早关闭会导致 `exec` 失败后旧程序少了 `fd`；太晚关闭会让新程序短暂看到本不该继承的能力。于是 `FD_CLOEXEC` 不是“关闭一些数字”这么简单，它是在程序映像替换边界上裁剪继承能力。

最后把新用户栈具体展开。假设栈顶从高地址向低地址增长，内核通常先把字符串放到栈下方，再压入指针表和 `auxv`。进入动态链接器或程序入口时，用户态看到的是：

```
low address
  argc = 4
  argv[0] -> "grep"
  argv[1] -> "-n"
  argv[2] -> "main"
  argv[3] -> "a.txt"
  argv[4] = NULL
  envp[0] -> ...
  ...
  NULL
  auxv entries: AT_PHDR, AT_PHNUM, AT_ENTRY, AT_RANDOM, ...
  padding for alignment
  strings: "grep\0-n\0main\0a.txt\0..."
high address
```

如果这里指针顺序错了，`main(int argc, char** argv)` 看到的参数就会错；如果 16 字节对齐错了，某些 ABI 约定和运行时启动代码会在很早的位置崩溃；如果 `AT_RANDOM` 指向了旧地址空间或未映射地址，`libc` 初始化 `canary` 时会 `fault`。用户栈构造不是格式化输出，而是内核和用户态启动代码之间的二进制契约。

ELF 校验、段映射与用户栈

ELF header 验证

ELF 验证要防止把任意文件当代码执行。最小检查包括 `magic`、`class`、`endianness`、`machine`、`type`、`program header` 范围和整数溢出。

```
validate_elf(file):
  eh = read_at(file, 0, sizeof(Elf64_Ehdr))
  require eh.magic == 0x7f 'E' 'L' 'F'
  require eh.class == ELFCLASS64
  require eh.machine == EM_X86_64
  require eh.phentsize == sizeof(Elf64_Phdr)
  require eh.phnum <= MAX_PHDRS
  require checked_mul(eh.phnum, eh.phentsize, &ph_size)
  require checked_add(eh.phoff, ph_size, &ph_end)
  require ph_end <= file.size or file supports sparse read policy
```

复杂度是 $O(\text{phnum})$ 。真正关键的是边界检查：`p_offset + p_filesz`、`p_vaddr + p_memsz`、页对齐和权限组合都可能溢出。装载器不能相信编译器或链接器一定生成合法文件，因为攻击者可以手工构造 ELF。

段映射

`PT_LOAD` 段描述文件区间到虚拟地址区间的映射。文件大小 `p_filesz` 是需要从文件读出的内容，内存大小 `p_memsz` 包括 BSS 零填充。

```
map_load_segment(mm, file, ph):
  require ph.memsz >= ph.filesz
  require same_page_offset(ph.vaddr, ph.offset)
  start = page_down(ph.vaddr)
```

```

end    = page_up(ph.vaddr + ph.memsz)
file_start = page_down(ph.offset)
prot = prot_from_flags(ph.flags)
map_file(mm, start, end, file, file_start, prot)
zero_range(mm, ph.vaddr + ph.filesz, ph.vaddr + ph.memsz)

```

若采用 demand paging, `map_file` 不必立即读入所有页面, 只需建立 VMA, 等 page fault 时按文件偏移填充页面。这样 `exec` 延迟低, 内存占用也低。若采用 eager loading, 实现简单但启动大程序会有明显 I/O 峰值。

用 `/usr/bin/grep` 的第一条指令看 demand paging。exec 提交完成后, 寄存器 RIP 已经指向解释器或主程序入口, RSP 指向新用户栈; 但入口所在的代码页可能还没有真正读入物理内存。CPU 返回用户态后开始取指, MMU 查入口虚拟地址的 PTE, 发现 `present=0`, 于是触发 instruction page fault。内核重新进入缺页处理, 先用 `fault` 地址找到对应 VMA, 再发现它来自可执行文件的 `PT_LOAD` 段, 权限是 `read+exec`, 文件偏移由 `vaddr - segment.vaddr + segment.offset` 算出。若 page cache 里没有这一页, 内核提交文件读; 读完后安装只读可执行 PTE, 返回用户态重试同一条取指。

时刻	状态	关键边界
exec 提交	新 mm 已安装, 代码段只有 VMA 或部分 PTE	旧程序不能继续运行
第一次取指	入口页 PTE 不 present, CPU 产生缺页异常	faulting 指令还没执行
缺页处理	根据 VMA 找到 ELF 文件和页偏移	必须检查 VMA 可执行权限
读入页面	page cache miss 时等待存储 I/O	当前 task 可睡眠, 调度器运行别人
安装 PTE	PTE 标 成 user、present、read、exec、not writable	权限不能超过 ELF 段权限
重试取指	同一 RIP 再次执行, 这次翻译成功	demand paging 对用户程序透明

这条路径解释了两个现象。第一, 大程序 `exec` 可以很快返回到用户态, 因为只物化了马上需要的页; 后面真正访问到的代码和数据才陆续缺页。第二, `exec` 已经成功不代表后续每次取指和读数据都不会失败: 底层文件系统 I/O 错误、映射被异常截断、权限标志错误, 都可能在第一次访问某页时暴露。教材若只说“`exec` 装载 ELF”, 读者会误以为装载一定是一次性读完整个文件; 真实边界是“建立地址空间契约”和“按需把页物化”两步。

用户栈构造

用户栈是 ABI 契约的一部分。教学系统可以简化 `auxv`, 但不能把 `argv` 字符串和 `argv` 指针混为一谈。

```

build_user_stack(argv, envp, auxv):
    sp = USER_STACK_TOP
    for s in reverse(strings(argv, envp)):
        sp -= len(s) + 1
        copy_to_new_user(sp, s)
        remember_pointer(s, sp)
    sp = align_down(sp, 16)
    push_null()
    push_auxv(auxv)
    push_null()

```

```
push_pointers(envp)
push_null()
push_pointers(argv)
push(argc)
return sp
```

栈构造要设置总大小上限，防止用户传入巨量参数耗尽内核内存。所有用户字符串必须先复制到内核或临时页中，因为在 `exec` 期间旧地址空间即将被替换；若边复制边销毁旧 `mm`，就可能读到无效地址。

提交点

提交阶段要尽量短，并且顺序清晰：

```
commit_exec(new_image):
    old_mm = current.mm
    current.mm = new_image.mm
    current.regs.ip = new_image.entry
    current.regs.sp = new_image.stack_pointer
    reset_caught_signal_handlers(current)
    close_cloexec_fds(current.files)
    activate_mm(new_image.mm)
    drop_mm(old_mm)
    return_to_user()
```

提交点之后不应再执行可能失败且需要返回旧程序的操作。比如关闭 `CLOEXEC` `fd` 一般可以在提交逻辑中完成，因为关闭失败不应阻止 `exec` 成功；但读取解释器、分配栈页和权限检查必须放在提交点之前。

提交点、失败回滚与身份变化

1. 正常 ELF：静态程序能打印 `argc/argv/envp`，入口地址和栈对齐正确。
2. 坏 magic：返回 `ENOEXEC`，旧程序继续运行。
3. 无执行权限：返回 `EACCES`，`fd table` 不变化。
4. 段溢出：构造 `p_vaddr + p_memsz` 溢出的 ELF，装载器必须拒绝。
5. BSS 清零：全局未初始化数组初值必须为零。
6. `FD_CLOEXEC`：带标志 `fd` 在新程序中不可见，普通 `fd` 仍可用。
7. 大 `argv`：超过限制返回 `E2BIG`，不破坏旧地址空间。
8. 动态链接入口：带 `PT_INTERP` 时入口应进入解释器而不是主程序 `e_entry`。

验收标准

掌握本章内容后，应当能够区分 `fork` 与 `exec`，解释 ELF program header 的运行时意义，画出初始用户栈布局，并指出 `exec` 的失败回滚边界和提交点。

映像替换涉及的完整状态集合

`exec` 很容易被误讲成“把一个 ELF 读进内存”。真正的主线更宽：它同时替换地址空间、重建用户栈、改变入口寄存器、裁剪 `fd` 继承、可能改变凭据、处理多线程、交给解释器或动态链接器，并把旧程序的错误恢复边界切断。把这些细节归入同一张状态账本，才能避免只把装载理解为几次内存映射。

账本	exec 要修改什么	不能出错的边界

地址空间	新 mm、VMA、栈、brk、mmap base、VDSO	提交前失败必须保留旧 mm，提交后不能再回旧代码
文件与路径	可执行文件、解释器、脚本原文件、mount flags	权限检查必须绑定稳定 inode/file，而不是路径字符串
fd 继承	普通 fd 继承，FD_CLOEXEC fd 关闭	不能在 exec 失败时提前关闭旧程序 fd
凭据与安全	setuid、file capability、LSM、no_new_privs、dumpable	提权 exec 要清理危险环境和调试关系
线程组	成功后只保留调用 exec 的线程	旧用户态锁和其他线程栈不能进入新映像
用户态启动	argc/argv/envp/auxv、入口 RIP、栈对齐	动态链接器和 libc 在 main 前就依赖这些字段

这张账本能解释很多看似零散的规则。脚本 `#!/bin/sh` 需要重新构造 argv，是因为真正执行的是解释器；动态 ELF 入口先到 `ld-linux`，是因为重定位和共享库加载属于用户态运行时；`no_new_privs` 会压制提权，是因为 `seccomp` 和沙箱不能允许“先收紧入口、再 exec 一个 `setuid` 程序拿回更大权限”；多线程 exec 要清掉其他线程，是因为旧程序里的用户态 mutex、TLS、栈和信号现场对新程序没有可解释语义。

再用一个带提权和动态链接的例子收束。进程执行一个 `setuid root` 的动态链接程序时，内核不能只检查 ELF magic。它要先解析可执行 inode，计算新凭据，判断 `no_new_privs`、文件 capability 和 LSM 策略；若发生权限边界变化，运行时还要进入 `secure mode`，忽略或清理会影响动态链接器的环境变量，例如 `LD_PRELOAD`。否则攻击者可以让高权限程序在启动前加载自己控制的共享库。

```
privileged_exec_boundary(file):
    old_cred = current_cred
    new_cred = compute_exec_cred(file, old_cred)
    if privilege_will_change(old_cred, new_cred):
        enable_secureexec()
        suppress_unsafe_loader_environment()
        restrict_dump_and_ptrace()
        install_new_image_and_cred_at_commit()
```

这里的提交点必须把“新程序身份”和“新程序映像”作为一组状态安装。若凭据已经提升但地址空间仍是旧程序，旧代码就拿到了不该有的身份；若地址空间已经换成新程序但 fd 和环境还没裁剪，新程序又可能短暂看到不该继承的能力。好的教学实现即使简化 `setuid` 和动态链接，也要保留这条不变量：exec 成功后看到的是一组自治的新运行身份，exec 失败后旧程序仍保持原身份和原资源。

验收标准

分析 `exec` 时，至少要区分四个点：仍可回滚到旧程序的点、开始销毁旧用户映像的点、新凭据和新地址空间一起生效的点、返回用户态且永不返回旧代码的点。缺少这四个点，就很难判断错误路径是否安全。

动态链接、解释器脚本与辅助向量

ELF 结构字段

分析 ELF 装载时要区分三类头：`Elf64_Ehdr` 描述整个文件，`Elf64_Phdr` 描述运行时段，`Elf64_Shdr` 描述链接或调试所用的节。内核运行装载主要依据 `program`

header; section header 即使被移除，也不影响程序运行。

字段或类型	含义	装载用途
PT_LOAD	可映射段	建立 VMA，设置 R/W/X 权限
PT_INTERP	动态解释器路径	装载 <code>ld-linux.so</code> 并把入口交给它
PT_PHDR	program header 自身位置	传给用户态动态链接器
PT_GNU_STACK	栈权限提示	决定用户栈是否可执行
e_entry	ELF 入口虚拟地址	静态程序入口或解释器最终跳转参考

Linux 的 ELF 装载路径同样遵循清晰阶段：读取文件头和程序头，检查解释器，准备新地址空间，映射可装载段，建立堆边界，构造用户栈和辅助向量，最后设置返回用户态所需的寄存器现场。

aux vector 布局

auxv 是内核放到用户栈上的键值数组，给动态链接器和 libc 传递信息。常见项包括 AT_PHDR、AT_PHENT、AT_PHNUM、AT_ENTRY、AT_BASE、AT_PAGESZ、AT_RANDOM、AT_EXECFN、AT_NULL。

```

user stack top:
strings for argv/envp
random bytes
auxv: AT_PHDR, value
      AT_PHENT, value
      AT_PHNUM, value
      AT_ENTRY, value
      AT_RANDOM, pointer
      AT_NULL, 0
envp pointers, NULL
argv pointers, NULL
argc

```

AT_RANDOM 为 stack canary 和 ASLR 提供随机种子；AT_BASE 指向动态链接器基址；AT_PHDR 让动态链接器找到主程序段表。若 auxv 构造错误，程序可能在 main 前就崩溃。

解释器脚本和 binfmt

`#!/bin/sh` 脚本不是 ELF。内核识别 shebang 后，把解释器路径和脚本路径重写成新的 argv，再递归执行解释器。Linux 用 binfmt 框架支持多种二进制格式：ELF、脚本、misc 注册格式等。

```

execve("script.sh", ["script.sh", "a"]):
  read first line "#!/bin/sh"
  new argv = ["/bin/sh", "script.sh", "a"]
  execve("/bin/sh", new argv)

```

这里要限制递归深度，否则脚本解释器互相指向会无限递归。binfmt_misc 允许用户注册自定义格式，例如通过魔数把某类文件交给解释器。安全检查必须覆盖最终解释器和原脚本，不能只检查第一层路径。

vfork + exec 为什么高效但危险

vfork 让子进程在 exec 或 exit 前与父进程共享地址空间，父进程通常被挂起。它避免了复制页表和 COW 设置，适合立刻 exec 的路径；但子进程若在 exec

前修改内存或调用复杂函数，会破坏父进程状态。

```
child = vfork()
if child == 0:
    // only async-signal-safe, minimal setup
    dup2(fd, STDOUT_FILENO)
    execve(path, argv, envp)
    _exit(127)
```

现代系统常用 `posix_spawn` 在库和内核之间选择更安全高效的实现。理解 `vfork` 的意义，不是鼓励随便使用，而是理解 `exec` 前后地址空间提交点的重要性。

setuid、凭据变化与 no_new_privs

`exec` 不只是换代码。执行 `setuid/setgid` 文件时，进程凭据可能变化；`capability` 也会根据文件能力、`bounding set`、`ambient set` 和 `no_new_privs` 重新计算。动态链接器和环境变量处理必须考虑提权执行，避免用户用 `LD_PRELOAD` 一类环境影响高权限程序。

```
exec_security_transition(file):
    read inode mode, owner, file capabilities
    if no_new_privs:
        suppress privilege gain
        compute new effective/permitted/inheritable caps
        clear dangerous personality and env effects
        install new credentials at commit
```

这解释了为什么 `exec` 路径会调用 LSM hook 和凭据提交逻辑。安全检查不能只看 ELF 格式合法，还要看执行后主体身份是否改变、哪些 fd 继承、`dumpable` 标志如何更新、`ptrace` 是否允许继续。

ASLR、PIE 与栈保护输入

`exec` 建立新地址空间时，会选择随机化的 `mmap base`、`stack base`、`brk` 和 `VDSO` 位置。PIE 主程序也可随机基址加载。内核还向 `auxv` 提供 `AT_RANDOM`，供 `libc` 初始化 `stack canary`。

机制	exec 阶段输入
ASLR	随机选择 <code>mmap</code> 、 <code>stack</code> 、 <code>brk</code> 、 <code>VDSO</code> 、PIE 基址
stack canary	<code>AT_RANDOM</code> 提供随机字节
NX stack	<code>PT_GNU_STACK</code> 和默认策略决定栈可执行性
RELRO/动态链接	program header 和 dynamic section 给链接器信息

这些保护不是 `exec` 独立完成的，但 `exec` 负责把初始地址布局和 `auxv` 种子交给用户态运行时。若地址布局固定或随机数可预测，后续保护会明显削弱。

exec 与多线程

多线程进程调用 `exec` 时，成功后只留下调用 `exec` 的线程，其他线程被清理。因为新程序映像不可能继承旧程序的线程栈、锁状态和用户态同步对象。内核必须停止同一线程组中的其他线程，并让共享资源进入一致状态。

```
de_thread_for_exec(current):
    signal other threads to exit
    wait until thread group is single-threaded
    take over thread group leader identity if needed
```

```
proceed to install new mm
```

这也是为什么 `exec` 提交点必须非常谨慎：旧线程可能持有用户态锁，但 `exec` 后这些锁没有意义；内核对象如 `fd table`、`credentials`、`signal struct` 还要按规则继承或重置。

binfmt_misc 与解释器安全边界

`binfmt` 框架允许注册自定义格式，把某些 `magic` 或扩展名交给解释器。它方便运行脚本、模拟器格式或语言运行时，但也扩大了 `exec` 策略面。解释器路径、参数重写、递归深度、权限继承和 `mount noexec` 都必须纳入检查。

```
registered format:
  magic = "\xCA\xFE"
  interpreter = "/usr/bin/custom-runner"
exec file:
  kernel rewrites argv:
    custom-runner original-file original-args...
```

安全模型必须说明权限检查作用于原文件、解释器，还是二者都检查。只检查脚本可执行而不检查解释器，或绕过 `noexec` 挂载，都会形成边界漏洞。

一次可复算的映像替换：从路径到新入口

前文已经分别说明 ELF 段、用户栈、解释器和提交点。下面把这些结构放进一次带具体初值的替换过程，用同一份对象账本同时验证成功路径和失败路径。设进程 `P` 的标识为 `420`，当前只有线程 `T0`，旧映像入口为 `0x00401000`，旧地址空间对象记为 `M0`。`T0` 执行：

```
// 路径、参数和环境都位于旧地址空间 M0；内核必须先取得稳定副本。
execve("/opt/demo/bin/calc",
      ["calc", "17", "25"],
      ["LANG=zh_CN.UTF-8", "MODE=safe"])
```

目标文件是 64 位位置无关可执行文件。ELF program header 给出三个需要装入的区段：

区段	文件范围	目标虚拟范围	权限与对齐	需要验证的事实
PT_LOAD 0	偏移 <code>0x0000</code> ， 长度 <code>0x1a30</code>	<code>0x555555554000</code> 起	读、执行；页 对齐	文件范围未 越界，尾页 补零不覆盖 下一段
PT_LOAD 1	偏移 <code>0x2000</code> ， 长度 <code>0x0830</code>	<code>0x555555557000</code> 起	只读；页对齐	重定位后仍 不可写
PT_LOAD 2	偏移 <code>0x3000</code> ， 文件长 <code>0x0410</code> 、 内存长 <code>0x0a00</code>	<code>0x555555558000</code> 起	读、写；页对 齐	<code>memsz >= filesz</code> ，BSS 区域清零

这一组数字让“装入程序”不再等于复制整个文件。第一段的文件偏移和虚拟地址页内偏移必须相容；第二段不能因为重定位方便而永久保持可写；第三段后半没有对应文件字节，却必须在新程序第一次读取之前表现为零。内核还要为解释器、用户栈、随机化区域和辅助页面选择互不重叠的地址。

准备阶段为何仍然依赖旧地址空间。

系统调用入口建立在 `T0` 的内核栈上，但 `path`、`argv` 和 `envp` 指针仍按 `M0` 解释。如果内核先切换到新页表，再读取参数，原指针可能已经无效；如果只保存指针而不复制字符串，另一个线程还能在校验过程中改变内容。即使本例只有一个线

程，接口也应维持同一条稳定规则：先限制总字节数和指针数，再把参数复制到内核拥有的临时对象。

```
prepare_exec(user_path, user_argv, user_envp):
    // 复制以后不再信任用户指针；长度上限同时限制内存消耗和扫描时间。
    args = copy_argument_block(user_argv, user_envp, ARG_MAX)
    path = copy_path(user_path, PATH_MAX)

    // 打开对象保存稳定的文件身份；之后的路径改名不会替换这个引用。
    image_file = open_for_exec(path)
    deny_write_if_required(image_file)

    // 新地址空间尚未公开，任何失败都只释放准备对象。
    candidate_mm = allocate_empty_address_space()
    return ExecCandidate(args, image_file, candidate_mm)
```

对象	代表字段	当前所有者与寿命	失败时的责任
参数块	字符串字节、指针个数、总长度	exec 准备对象；提交后复制进用户栈	释放内核页，不改变 M0
可执行文件引用	inode/file 引用、打开方式、写入排斥状态	exec 准备对象；提交后转入新映像记录	解除写入排斥并减少引用
候选地址空间 M1	VMA 集合、页表根、ASLR 选择、入口	准备路径独占；提交后由进程 P 持有	撤销映射并释放页表和页面
旧地址空间 M0	旧 VMA、页表、用户栈和代码	提交前始终由 P 持有	准备失败时继续运行，不能提前释放

先验证结构，再产生可见映射。

ELF 解析器不能把文件头中的长度和计数直接用于内存访问。设 program header 表从文件偏移 0x40 开始，每项 56 字节，共 9 项。解析器至少要用不溢出的算术验证 $0x40 + 9 * 56$ 位于文件范围内，并逐项检查文件偏移、虚拟范围、对齐关系和权限组合。这些检查发生在候选对象中，不把“看起来像 ELF”误当成“已经可以执行”。

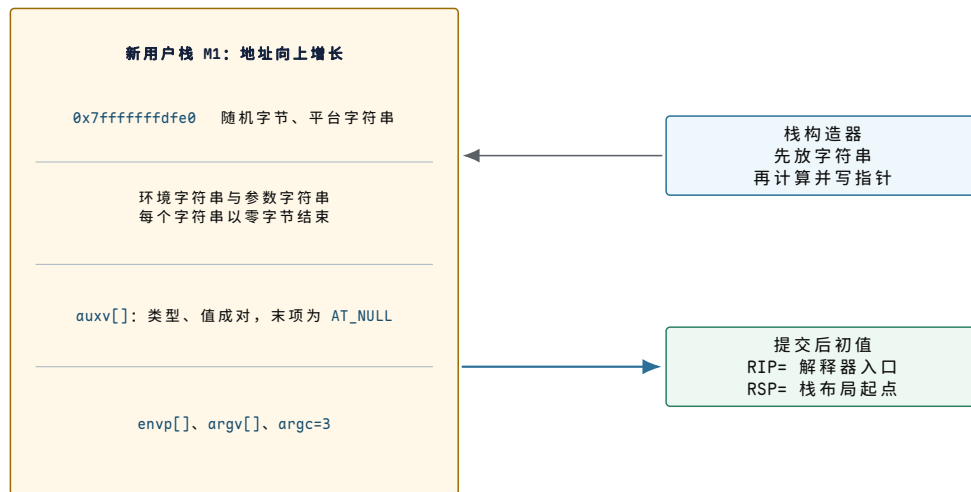
```
validate_load_segment(ph, file_size):
    // 先检查加法和区间，再读取或建立任何映射。
    require checked_add(ph.offset, ph.filesz) <= file_size
    require ph.memsz >= ph.filesz
    require same_page_offset(ph.offset, ph.vaddr)

    // 地址上界和区间重叠由候选地址空间统一判定。
    range = checked_user_range(ph.vaddr, ph.memsz)
    require !candidate_mm.overlaps(range)
    require !(ph.writable && ph.executable) unless policy_allows_wx
```

映射阶段只建立 M1。文件支持的页面可以按需缺页装入；BSS 尾部由匿名零页或新分配页支持；暂时需要重定位的页面必须在动态链接器完成后收紧权限。此时处理器仍按 M0 取指，父进程、/proc 观察者和同一进程的其他内核路径也不应把 M1 当成已经生效的映像。

用户栈是一份具有内部引用的序列化对象。

设内核为新栈选择顶端 0x00007ffffffe000。字符串从高地址向低地址复制，随后按 ABI 要求对齐，再写入辅助向量、环境指针、参数指针和 argc。指针值必须指向 M1 中最终地址，不能指向准备阶段的内核参数块。

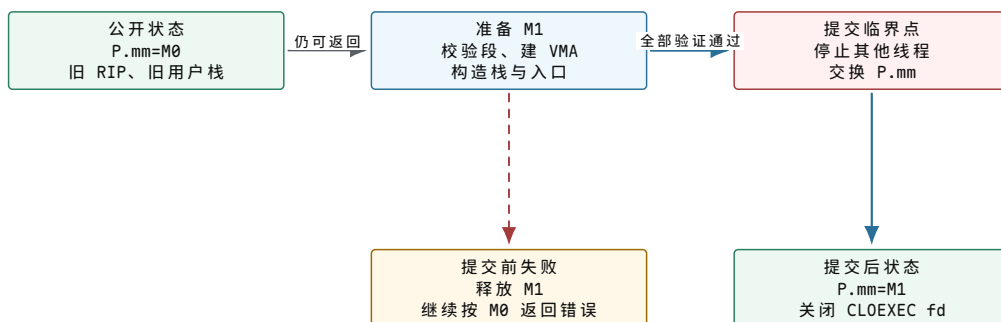


图示：栈中的指针引用同一栈内更高地址的字符串。构造顺序可以从高地址向低地址进行，但提交后的读取顺序由 ABI 规定；两种顺序不能混为一谈。

动态链接程序的初始 RIP 通常不是主程序 ELF 头中的入口，而是 PT_INTERP 指定解释器的入口。主程序入口、program header 地址、页大小、安全随机数和执行文件描述等信息通过辅助向量交给解释器。解释器完成依赖库映射和重定位后，才转到主程序入口。这项用户态工作发生在 exec 已经提交之后；若解释器随后缺库退出，内核不能恢复旧映像。

提交点只交换已经验证完成的根引用。

到提交前，M0 和 M1 同时存在。M0 是公开状态，M1 是准备状态。提交不能逐个把旧 VMA 替换成新 VMA，否则观察者可能看见代码来自 M1、栈仍来自 M0 的混合状态。教学模型把提交表达为根引用交换，并在交换前完成所有仍可回滚的工作。



图示：提交前的失败路径回收候选对象并保留 M0；提交后的错误只能终止或在新映像内报告。判断一道错误路径是否可回滚，关键是它发生在根引用交换之前还是之后。

```
commit_exec(candidate):
    // 所有可能返回普通错误的校验必须已经完成。
    require candidate.state == READY
    stop_or_remove_sibling_threads()
    lock(process.exec_lock)

    // 根引用交换是用户可见身份变化的中心提交点。
    old_mm = process.mm
    process.mm = candidate.mm
    install_new_credentials(candidate.cred)
    close_cloexec_descriptors(process.files)
    install_user_entry(candidate.entry, candidate.stack_pointer)

    unlock(process.exec_lock)
    // 旧地址空间在不可达且引用归零后回收；不能在交换前释放。
    release_address_space(old_mm)
```

这里还要区分三个相邻边界。文件格式通过验证，只说明候选字节可解释；M1 准备完成，只说明存在一个可以提交的地址空间；根引用交换完成，才说明进程身份已经切换。动态链接成功和主程序开始执行则是提交后的后续结果。

失败位置决定错误返回还是进程终止。

失败位置	已改变的状态	收束方式	用户可见结果
复制参数时遇到坏指针	只有临时参数块	释放临时页	旧程序得到 EFAULT
ELF 段越出文件	文件引用与部分 M1 映射	撤销候选映射并关闭引用	旧程序得到 ENOEXEC
构造栈时内存不足	M1 已基本形成	释放 M1，保持 M0	旧程序得到 ENOMEM
根引用交换后解释器缺少依赖	M0 已不可恢复，M1 已公开	新映像按装载器规则退出	进程终止并留下新身份的退出状态
提交后第一次取指缺页失败	新凭据和新地址空间可能已经生效	形成致命信号或异常退出	不能回到旧程序继续

本章复核：一次 exec 必须对清的六本账

完整复核不再从“exec 会加载程序”开始，而是逐项核对：输入字节属于 M0，稳定副本属于准备对象；文件引用固定映像身份；M1 独立保存候选映射；栈指针引用 M1 内的最终字符串；提交只交换已完成的根对象；提交前失败回滚，提交后失败由新映像承担。只有六项同时成立，进程标识保持不变与程序映像被完全替换才不矛盾。

练习 4.1 判断三个不同的成功边界

某次 `execve` 已通过 ELF 校验并完成 M1 映射，但尚未交换 `process.mm`。分别说明“格式有效”“候选映像完成”“exec 已成功”是否成立，并指出此时发生内存不足应由哪个映像继续。

解答 4.1 判断三个不同的成功边界

格式有效和候选映像完成已经成立，exec 成功尚未成立，因为进程公开根引用仍指向 M0。内存不足发生在提交前，应释放 M1 和准备对象，按 M0 返回 ENOMEM；不能留下部分新映像。

练习 4.2 复算用户栈中的引用

若三个参数字符串最终位于 `0x7fffffff900`、`0x7fffffff905` 和 `0x7fffffff908`，说明 `argv[0..2]` 应保存什么，为什么不能保存内核参数块地址。

解答 4.2 复算用户栈中的引用

`argv` 三项依次保存这三个用户虚拟地址，末项再保存空指针。内核参数块只在准备阶段可达，提交后会释放，而且用户态不能按内核地址访问它；因此栈内引用必须指向 M1 中的最终字符串。

练习 4.3 定位不可回滚的失败

比较“解释器 ELF 自身校验失败”和“解释器已经开始运行但找不到共享库”。哪一种通常可以向旧程序返回错误，哪一种只能在新映像中终止？用提交点解释，而不要只依据错误名称。

解答 4.3 定位不可回滚的失败

内核在提交前校验解释器 ELF 时失败，仍可释放候选映像并让旧程序得到错误。解释器开始运行意味着根引用已经交换，旧地址空间和旧凭据变化不能作为普通调用回滚；缺库只能由新映像报告并退出。

尚未解决的问题。新映像可以建立并提交，但多个执行流共享数据时仍可能丢失更新；一次条件等待也可能在检查条件与进入睡眠之间错过唤醒。下一章从具体交错推导原子操作、临界区、等待队列和进程间通信对象。

第 5 章 为什么共享状态需要同步与通信对象

继承的机器。内核已经有多个可抢占 task、受控系统调用入口和可替换程序映像。只要两个执行流都能修改同一状态，“各自执行正确”就不再保证组合结果正确；只禁止并发又会让等待设备或事件的任务占住处理器。

多个执行流共享 CPU 和内存，需要同步原语防止互相破坏。本章从 `futex` 到 RCU，从 `pipe` 到信号，把等待关系和锁顺序讲完整。

共享状态、竞争条件与同步目标

多线程程序的核心风险不是“同时执行”这个事实，而是共享状态缺少清晰的不变量。两个线程同时修改一个队列，硬件只能执行 `load`、`store`、`CAS`、`fence` 等局部操作；它不知道 `head`、`tail`、`size`、`closed` 之间应该满足什么关系。操作系统和运行库提供 `mutex`、`condition variable`、`semaphore`、`futex`、`pipe`、`signal` 等机制，帮助程序把共享状态、等待和唤醒写成可检查的协议。

操作系统进入并发世界以后，错误不再只来自单个函数写错变量，而来自两个执行流同时触碰同一份状态。中断处理程序可能和普通内核线程同时访问设备队列；两个 CPU 可能同时把任务放入 `runqueue`；一个进程关闭 `fd` 时，另一个线程可能正在通过同一个 `open file description` 发起 I/O。锁的目的不是让代码“看起来串行”，而是给共享对象规定一个可证明的临界区：谁能读，谁能写，写入完成前谁必须等待，等待时是否允许睡眠。

从裸机单片机演进过来时，最早的“锁”通常是关中断。它简单有效，因为只有一个 CPU，且并发主要来自中断打断主循环。进入多核后，关本地中断不能阻止另一个 CPU 访问同一对象，于是需要自旋锁；进入可能长时间等待的路径后，自旋浪费 CPU，于是需要 `mutex`、`semaphore` 和等待队列。每一种同步原语都绑定一个上下文限制：中断上下文不能睡眠；持有自旋锁时不能调用可能阻塞的函数；持有文件系统锁时不能随意回调用户内存复制路径；调度器自己的锁必须在切换前后保持严格顺序。

核心思想

锁不是性能技巧，而是状态所有权的边界。设计锁时先写对象不变量，再决定哪些操作必须在同一把锁下完成。

内核锁设计要同时回答四个问题：保护哪个对象，持锁期间允许做什么，等待者在哪里排队，失败路径如何释放。很多死锁来自第四个问题被忽略：正常路径释放锁，错误路径提前返回却忘了释放；或者关闭路径拿到 A 锁后等待 B，而另一个路径拿到 B 后等待 A。

互斥锁保护的不是某个变量，而是一组字段之间的不变量。条件变量也不是消息队列，它表达的是“某个谓词暂时不满足，线程可以睡眠，等状态可能改变后再重新检查”。正确等待必须同时包含谓词、锁和退出路径。只写 `wait()` 而不写完整谓词，会遇到虚假唤醒、丢失唤醒和关闭卡死。

一个有界队列足以说明同步原理。队列状态包括 `buffer`、`capacity`、`head`、`tail`、`size`、`closed`。生产者等待 `size < capacity || closed`，消费者等待 `size > 0 || closed`。`push` 改变 `buffer`、`tail`、`size`；`pop` 改变 `buffer`、`head`、`size`；`close` 改变 `closed` 并唤醒所有等待者。任何一个字段在锁外修改，都会破坏状态机。

IPC 是跨进程同步和数据交换。`pipe` 把字节流连接到 `fd`，`socket` 把通信扩展

到本机或网络，shared memory 共享页但不自动提供同步，message queue 提供消息边界，eventfd 把计数型事件暴露成 fd readiness。IPC 的关键不是哪个接口高级，而是数据在哪里、所有权如何转移、背压如何发生、关闭如何传播。

信号是异步控制流。它可以通知进程发生了外部事件，例如终端中断、子进程退出、非法内存访问、定时器到期。信号难学，是因为它打断正常执行路径，而且信号处理函数只能安全调用很少一部分函数。工程上常把信号转成更普通的事件，例如写 pipe、设置原子标志或使用 signalfd，再交给主循环处理。

原子性、内存顺序与睡眠唤醒

同步实践先写不变量表。

字段	含义	受谁保护
capacity	最大槽数，固定大于 0	构造后只读
head	下一个可读槽	mutex
tail	下一个可写槽	mutex
size	当前元素数量	mutex
closed	是否拒绝新写入	mutex

然后写等待谓词：

- 生产者等待：`size < capacity || closed`
- 消费者等待：`size > 0 || closed`
- 关闭等待：`running_workers == 0` 或 `queue_empty && all_workers_stopped`

下面是常见内核锁原语的实践边界：

原语	适用场景	禁止事项	典型对象
关中断	单核短临界区，中断共享变量	多核共享保护，长时间计算	tick 计数、单核设备寄存器影子状态
自旋锁	多核短临界区，不能睡眠的上下文	持锁 I/O、持锁 page fault、持锁等待用户事件	runqueue、描述符环、引用计数
mutex	进程上下文中可能等待的互斥	中断上下文获取，持有后进入不可重入路径	inode 元数据、文件系统目录更新
读写锁	读多写少且临界区短	长读者导致写者饥饿，升级锁顺序混乱	mount 表、路由表快照
RCU	读路径极热，读者不阻塞写者	立即释放旧对象，读区内睡眠	fdtable、路径缓存、配置表
semaphore	计数资源，允许多个持有者	用它表达复杂对象不变量	buffer 数量、并发 I/O 限额

一个合理的锁设计文档至少包含下面字段：

```
Object:
    runqueue[cpu]

Invariant:
    task.state == RUNNABLE implies task is on exactly one runqueue
```

```

rq.nr_running == number of tasks linked from rq.head

Lock:
    rq.lock protects rq.head, rq.nr_running, task.rq_link

Allowed while holding:
    enqueue/dequeue, choose_next, update vruntime

Forbidden while holding:
    page allocation with reclaim, disk I/O, copy_from_user, waiting on mutex

```

等待队列必须把“检查条件”和“入队睡眠”放在同一把锁保护下，否则会出现 lost wakeup。正确模型不是“先看条件，不满足就睡”，而是“在保护条件的锁内，把自己放到等待集合中，然后原子地释放锁并切换状态”。唤醒方也在同一把锁下改变条件并唤醒等待者。

IPC 实践也按同样方式写。pipe 的状态包括读端引用数、写端引用数、内核缓冲区、读写阻塞队列。写端关闭后，读端读空返回 EOF；读端关闭后，写端写入会失败或收到信号；缓冲区满时写阻塞或返回 EAGAIN；缓冲区空时读阻塞或返回 EAGAIN。

信号实践要避免在 handler 里做复杂逻辑。推荐的纸上模型是：handler 只记录最小事实，例如设置原子标志；主循环在安全位置检查标志并执行真正清理。若系统使用 signalfd，则信号也进入 fd/event loop 模型，和其他 I/O 事件统一处理。

锁、等待队列与 IPC

环形缓冲区

环形缓冲区是设备驱动、pipe、socket buffer 和日志队列的基础结构。它用固定数组保存数据，用 head 指向下一个可读位置，用 tail 指向下一个可写位置。若额外维护 size，判断空满很直接；若不维护 size，则常留一个空槽，用 head == tail 表示空，用 next(tail) == head 表示满。

```

bool push(Ring* r, byte b) {
    if (r->size == r->capacity) return false;
    r->buf[r->tail] = b;
    r->tail = (r->tail + 1) % r->capacity;
    r->size += 1;
    return true;
}

bool pop(Ring* r, byte* out) {
    if (r->size == 0) return false;
    *out = r->buf[r->head];
    r->head = (r->head + 1) % r->capacity;
    r->size -= 1;
    return true;
}

```

push/pop 都是 $O(1)$ 。难点不在算法复杂度，而在并发所有权。单生产者单消费者可以用两个原子索引实现无锁队列；多生产者多消费者通常先用 mutex 保护不变量，等有证据证明锁是瓶颈，再考虑更复杂结构。

mutex 和等待队列

mutex 的关键不是把自旋改成睡眠，而是在失败时把当前线程从 runnable 集合移到等待队列：

```

mutex_lock(m):
    disable_preempt()

```

```

spin_lock(m.wait_lock)
if m.owner == null:
    m.owner = current
    spin_unlock(m.wait_lock)
    enable_preempt()
    return
enqueue(m.waiters, current)
current.state = BLOCKED
schedule_unlocking(m.wait_lock)
enable_preempt()
retry mutex_lock(m)

mutex_unlock(m):
    spin_lock(m.wait_lock)
    if empty(m.waiters):
        m.owner = null
    else:
        next = dequeue(m.waiters)
        m.owner = next
        next.state = RUNNABLE
        enqueue_runqueue(next)
    spin_unlock(m.wait_lock)

```

这里有三个不变量：owner 要么为空要么指向持有线程；等待队列里的线程必须是 BLOCKED；被唤醒线程必须重新进入 runnable 集合。若 unlock 先释放 owner 再唤醒，另一个新线程可能插队，造成等待者饥饿。若唤醒时没有持有等待队列锁，等待节点可能和 timeout/cancel 路径并发释放。

条件变量

条件变量的算法核心是“检查谓词、睡眠、醒来后重新检查”。虚假唤醒不是异常，而是接口允许的行为；因此 wait 必须写在 while 循环中。

```

lock(m)
while !predicate():
    wait(cv, m)
// predicate is true while holding m
do_state_transition()
unlock(m)

```

notify 的语义也要讲清：通知不是把数据交给等待者，只是说明谓词可能变真。若生产者先通知再修改状态，消费者醒来仍看不到新状态；若修改状态后忘记通知，等待者可能永远睡眠；若关闭时只通知一个等待队列，其他等待者可能卡住。

信号量

信号量表达可用资源数量。P/acquire 操作在计数大于 0 时减一，否则等待；V/release 操作加一并唤醒等待者。它适合连接池许可、队列空槽、I/O in-flight 限额，不适合保护多字段复杂不变量。

```

acquire(sem):
    lock(sem.lock)
    while sem.count == 0:
        wait(sem.cv, sem.lock)
    sem.count -= 1
    unlock(sem.lock)

release(sem):
    lock(sem.lock)
    sem.count += 1
    notify_one(sem.cv)

```



```
unlock(sem.lock)
```

信号量的常见 bug 是许可泄漏：acquire 成功后，错误路径忘记 release。工程上要用作用域对象或 finally 路径保证释放。另一个 bug 是把信号量当成关闭广播；关闭是额外状态，必须单独表达。

自旋锁

最小自旋锁可以用原子交换实现：

```
lock(spinlock *l):
    while atomic_exchange(l->held, true) == true:
        cpu_relax()

unlock(spinlock *l):
    atomic_store_release(l->held, false)
```

这个算法的互斥性来自原子交换：同一时刻只有一个 CPU 能看到旧值为 false。复杂度不是固定常数，因为竞争时等待时间取决于持锁者运行时间和 CPU 调度。实现上必须使用 acquire/release 语义，否则 CPU 或编译器可能把临界区内存访问重排到锁外。真实内核还会加入持有者记录、递归检查、抢占关闭、IRQ 保存和等待统计。

普通 test-and-set 自旋锁在高竞争下会让所有 CPU 反复写同一个 cache line。ticket lock 用取号保证公平：

```
lock(ticket_lock *l):
    my = fetch_add(l->next, 1)
    while load_acquire(l->owner) != my:
        cpu_relax()

unlock(ticket_lock *l):
    store_release(l->owner, l->owner + 1)
```

ticket lock 的优点是 FIFO 公平，缺点是所有等待者仍然轮询 owner cache line。队列锁进一步让每个等待者轮询自己的节点，降低缓存抖动。教学内核可以先实现 test-and-set，再用压力测试观察竞争成本，最后引入 ticket 或 MCS 解释公平性和 cache line 迁移。

RCU 的读写分离

RCU 解决的是读路径极热、写路径较少的场景。读者进入 RCU 临界区时不获取传统互斥锁，只保证自己不会在临界区内被回收对象悬空；写者发布新对象后，必须等所有旧读者经过 quiescent state 才释放旧对象。

```
reader():
    rcu_read_lock()
    p = rcu_dereference(global_ptr)
    use(p)
    rcu_read_unlock()

writer():
    old = global_ptr
    new = clone_and_update(old)
    rcu_assign_pointer(global_ptr, new)
    call_rcu(old, free_after_grace_period)
```

RCU 的不变量是：任何读者看到的对象在读侧临界区结束前仍然有效。它不保证读者看到最新状态，也不适合读区内睡眠的普通实现。把 RCU 当成“无锁万能替代品”会导致非常隐蔽的 use-after-free。

IPC 的背压算法

pipe 和 socket 都需要背压。缓冲区满时，写入者要么阻塞，要么在非阻塞模式返回 `EAGAIN`。背压算法的本质是有界队列：

```
write(pipe, bytes):
    while bytes not empty:
        if pipe.buffer_full():
            if nonblocking: return bytes_written_or_EAGAIN
            sleep_until_not_full_or_closed()
        copy_some_bytes_into_buffer()
    wake_readers()
```

这里的 `copy_some_bytes` 说明短写是正常路径。只要缓冲区剩余空间小于请求长度，写入就可能部分完成。调用者必须把“已完成多少字节”作为状态保存，而不是假设一次 `write` 完成整个消息。

死锁条件与锁顺序

经典死锁需要四个条件同时成立：互斥、持有并等待、不可抢占、循环等待。内核无法完全消除前三个条件，因为对象确实需要互斥，很多资源不能强制抢占。因此工程上主要打破循环等待：给锁分层，规定全局顺序。

```
Allowed lock order:
tasklist_lock
-> mm.lock
    -> vma.lock
        -> page.lock
superblock.lock
-> inode.lock
    -> pagecache.lock
        -> buffer.lock
netns.lock
-> socket.lock
    -> skb_queue.lock
```

锁顺序不是文档装饰，而要能被测试和运行时检查。简化 lockdep 可以在每个线程记录当前持有锁栈，每次获取新锁时加入一条“已持有锁 -> 新锁”的有向边；若图出现环，就报告潜在死锁。

```
on_lock_acquire(new_lock):
    for held in current.held_locks:
        graph_add_edge(held.class, new_lock.class)
    if graph_has_cycle():
        report_deadlock_risk(current, held, new_lock)
    push(current.held_locks, new_lock)
```

这个检查的成本取决于锁类数量和边数量，不能在极热路径无节制开启，但作为调试构建非常有价值。更重要的是，它把“某次测试没有卡死”升级成“锁顺序图没有出现已知环”。

死锁、活性与优先级反转

并发测试要覆盖正常路径、等待路径和关闭路径。只测单线程 `push/pop` 没有意义，因为它没有触发同步问题。

- 多生产者多消费者下，元素不丢、不重、不乱破坏队列不变量。
- 队列空时消费者等待，生产者 `push` 后消费者能醒来。
- 队列满时生产者等待，消费者 `pop` 后生产者能醒来。

- close 能唤醒所有生产者和消费者，不留下永久睡眠线程。
- timeout 或 cancel 路径不会破坏 size/head/tail。
- pipe 读写能处理 EOF、EPIPE、EAGAIN 和短读短写。
- 信号只改变允许改变的最小状态，不在 handler 中执行不安全操作。

测试报告必须写出等待谓词和唤醒来源。若测试只说“不会死锁”，但没有说明哪个线程等什么、谁负责唤醒、关闭时怎样退出，这个测试不可维护。

锁测试不能只跑正常路径。最低测试包括：

1. 互斥测试：多个 CPU 同时递增共享计数器，最终值必须等于操作次数。
2. lost wakeup 测试：让唤醒和入睡交错，等待者不能永久睡眠。
3. 中断上下文测试：持有自旋锁时触发同类中断，检查是否需要 irqsave。
4. 死锁检测测试：故意构造 A 后 B 与 B 后 A，lockdep 必须报告环。
5. 饥饿测试：大量短任务竞争 mutex，等待时间不能无限增长。
6. 退出路径测试：加锁后在每个错误分支注入失败，所有锁计数必须归零。

核心思想

对同步机制的理解应达到以下程度：能够说明每把锁保护的對象，判断自旋锁、mutex 与 RCU 的上下文限制，写出 lost wakeup 的反例及修复，并用锁顺序图解释死锁风险。

读多写少、无锁读取与锁依赖

raw spinlock、ticket lock 与 queued spinlock

简单 test-and-set 锁在竞争下会让所有 CPU 反复写同一 cache line。ticket lock 提供 FIFO 公平，但所有等待者仍然轮询同一个 owner。queued spinlock 把等待者组织成队列，思想接近 MCS：每个 CPU 主要等待自己的节点，减少 cache line 抖动。

```
mcs_lock(lock, node):
    node.locked = true
    pred = xchg(lock.tail, node)
    if pred != null:
        pred.next = node
        while node.locked:
            cpu_relax()

mcs_unlock(lock, node):
    if node.next == null:
        if cmpxchg(lock.tail, node, null):
            return
        wait until node.next != null
    node.next.locked = false
```

Linux 的 qspinlock 比这段伪代码更紧凑：pending bit 处理短竞争，MCS queue 承载较长竞争，锁路径还要配合抢占控制和虚拟化环境。queued lock 并不因为结构更复杂就必然更快；它是为高竞争多核场景降低共享缓存行争用而增加的机制。

rwlock、rwsem 和乐观自旋

读写锁允许多个读者并行，一个写者独占。rwlock 通常用于短临界区，不能睡眠；rwsem 是可睡眠读写信号量，适合 VMA、文件系统等较长路径。rwsem 还可能

做 optimistic spinning: 如果持有者正在 CPU 上运行, 等待者短暂自旋, 期望持有者很快释放, 避免睡眠/唤醒成本。

原语	优点	风险
rwlock	读者并行, 路径短	长读者或写频繁时收益差, 不能睡眠
rwsem	可睡眠, 适合较长临界区	饥饿、公平性、唤醒策略复杂
optimistic spin	避免短等待睡眠成本	持有者不运行时浪费 CPU

读写信号量必须同时维护等待链表、持有者、读者计数和写者交接。若一条路径持有 rwsem 后, 又等待另一项同样需要该 rwsem 的工作完成, 就会形成隐藏的等待环。

seqlock: 写者优势的读一致性

seqlock 用一个递增序列号表达写入是否发生。读者开始时读取 seq, 复制数据, 结束时再读 seq; 如果 seq 变化或为奇数, 重试。写者持锁并把 seq 变奇数, 写完后变偶数。

```
read:
do {
    seq = read_seqbegin(&s)
    local = shared_data
} while (read_seqretry(&s, seq))

write:
write_seqlock(&s)
shared_data = new_value
write_sequnlock(&s)
```

seqlock 适合读多写少、读者可重试、数据可复制的场景, 例如时间数据。它不适合读者不能重试、数据包含指针且对象生命周期复杂、写者频繁导致读者长期重试的场景。

per-CPU 与 preempt/migrate 控制

per-CPU 变量让每个 CPU 有自己的副本, 减少锁和 cache line bouncing。但访问 per-CPU 数据时, 要确保当前执行不会迁移到另一个 CPU, 否则先取到 CPU0 的指针, 随后迁移到 CPU1 继续使用, 会破坏语义。

```
this_cpu_counter_add(delta):
    preempt_disable()
    per_cpu(counter, smp_processor_id()) += delta
    preempt_enable()
```

有些路径用 `migrate_disable` 而不是完全禁止抢占, 表示任务可以被抢占但不能迁移 CPU。读调度和网络栈源码时, 这类边界经常出现。per-CPU 不是无锁魔法, 它把共享变成分片, 同时引入汇总、CPU hotplug 和迁移限制。

内存屏障原语

原语	语义
<code>smp_rmb</code>	约束读-读顺序
<code>smp_wmb</code>	约束写-写顺序
<code>smp_mb</code>	完整内存屏障, 约束读写重排

<code>smp_store_release</code>	发布数据前的写入对获取方可见
<code>smp_load_acquire</code>	看到发布指针后能看到发布前初始化

屏障必须服务于具体协议。若无法说出“谁发布、谁获取、保护哪组字段”，就不应随便加 `smp_mb`。错误屏障可能既没修 bug，又增加维护负担。

pipefs 与 pipe_buffer

Linux 管道不是用户态数组，而是内核 pipe inode 和一组 `pipe_buffer`。每个 buffer 可以引用匿名页、page cache 页或 splice 传递的页片段。管道容量有默认值和上限，常见默认容量是 65536 字节量级，但会受内核版本和配置影响。

```
pipe_write(file, iov):
    pipe = file.private_data
    lock(pipe.mutex)
    while pipe_full(pipe):
        if nonblocking: return -EAGAIN
        wait(pipe.wr_wait, pipe.mutex)
    copy_or_splice_pages_into_pipe_buffers()
    wake_readers(pipe.rd_wait)
    unlock(pipe.mutex)
```

管道实现的关键不在函数名称，而在关闭语义：写端全部关闭且缓冲为空时，读操作返回 EOF；读端全部关闭时，写操作失败并可能触发 `SIGPIPE`；非阻塞模式在暂时不能推进时返回 `EAGAIN`。这些结果都来自端点引用和缓冲区状态，而不是单纯来自一次读写调用。

System V 与 POSIX IPC

System V IPC 提供 semaphore、message queue、shared memory，典型接口是 `semget/semop`、`msgget/msgsnd/msgrcv`、`shmget/shmat`。POSIX IPC 提供命名 semaphore、POSIX message queue 和共享内存对象，接口如 `sem_open`、`mq_open`、`shm_open`。

机制	特点	适用边界
System V semaphore	内核维护计数和 undo 语义	传统系统和跨进程资源计数
POSIX semaphore	named/unnamed 两类	进程间或线程间许可控制
System V msgqueue	类型化消息，内核队列	需要消息边界但吞吐不极致
POSIX mq	描述符化，可通知	与 fd/event loop 集成更方便
shared memory	共享页，不自带同步	大数据交换，需 futex/锁配合

共享内存常和 futex 一起使用：数据放在共享页，锁字也放在共享页；无竞争时用户态原子操作完成，有竞争时 futex 让内核按这个用户地址建立等待队列。这样数据路径不必每次进入内核。

eventfd、timerfd、signalfd 和 memfd

Linux 把很多事件转成 fd，是为了让 epoll 能统一等待。

接口	内核语义	常见用途
eventfd	64 位计数器，read 消费，write 增加	线程/进程通知、io_uring、KVM kick
timerfd	定时器到期次数可读	event loop 中处理超时
signalfd	把被 mask 的信号作为 fd 读取	避免复杂 async signal handler
memfd	匿名内存文件，可 seal	共享内存、沙箱传递只读 blob

eventfd 的 EFD_SEMAPHORE 模式每次 read 只消耗 1，否则 read 返回当前计数并清零。这个小差异决定它表达的是“累计事件”还是“许可”。memfd sealing 可以禁止继续写、增长或收缩，让发送方把一段内存变成稳定对象再交给接收方。

信号投递路径

信号不是立即调用用户函数。内核先把信号加入 pending 集合；线程返回用户态前检查 pending signal；若未被屏蔽且有 handler，内核在用户栈上构造 signal frame，把返回地址改成 handler；handler 执行后通过 sigreturn 恢复原上下文。

```
send_signal(task, sig, info):
    add_to_pending(task.signal_pending, sig, info)
    if task sleeping interruptibly:
        wake_up(task)

return_to_user(task):
    if has_unblocked_pending_signal(task):
        build_signal_frame_on_user_stack()
        task.regs.ip = handler_address
        task.regs.sp = signal_frame_sp
```

这解释了为什么 handler 可用函数受限：它运行在异步插入的用户控制流里，可能打断 malloc、printf、锁持有路径。signalfd 的工程价值在于把异步信号改造成普通 fd 事件，降低重入风险。

IPC 性能决策树

需求	推荐思路
少量控制事件	eventfd、pipe、socketpair
需要消息边界	Unix datagram socket、POSIX mq、自定义帧协议
大块数据同机传输	shared memory + futex/eventfd 通知
要跨机器扩展	TCP/QUIC/RPC，明确超时、重试和幂等
要把文件数据转发给 socket	sendfile、splice、zero-copy 路径
要统一事件循环	fd 化 接口：epoll + eventfd/timerfd/signalfd

性能不是只看平均延迟。管道有内核拷贝和容量上限；共享内存吞吐高但同步复杂；socket 语义完整但协议开销高；message queue 保留边界但容量和优先级策略会影响尾延迟。选择 IPC 时要写出数据大小、频率、是否需要消息边界、是否跨主机、背压和关闭语义。

lockdep 报告怎么读

lockdep 报告通常会给出两个锁类、已观察到的获取顺序、当前试图形成的新顺序和调用栈。读报告时先找锁类名，再画边：A 后 B，B 后 A 是否形成环；然后检查是否真实同一实例、同一上下文，还是锁类标注过粗。

```
Observed order:
inode_lock -> page_lock
New order:
page_lock -> inode_lock
Result:
possible circular locking dependency
```

不要把 lockdep 当成“测试失败噪声”。它发现的是潜在等待环，可能在当前负载下未死锁，但在错误时序下会卡死。高质量内核代码要能解释每条锁边的必要性，或调整锁类/顺序消除环。

把同步规则落实为状态机

第二阶段深讲：同步原语要写成状态机

同步章节若只讲 mutex、condition variable、semaphore 的 API，很容易产生错觉：只要调用了锁，就安全。真实系统的同步安全来自对象状态机。每个共享对象都应该明确：有哪些字段，字段之间的不变量是什么，哪些事件改变状态，等待者等的谓词是什么，关闭和取消怎样唤醒所有路径。

有界队列：最小但完整的同步对象。

一个有界队列包含 buffer、capacity、head、tail、size、closed、读等待队列和写等待队列。它的核心不变量是：

```
0 <= size <= capacity
head and tail are always in [0, capacity)
size == number of occupied slots
closed implies no new successful push begins
waiters sleep only while their predicate is false
```

生产者等待 `size < capacity || closed`，消费者等待 `size > 0 || closed`。注意关闭状态必须写进谓词，否则关闭后空队列上的消费者可能永远睡眠，满队列上的生产者也可能永远睡眠。

正确 wait 的四步。

正确等待不是一个 `wait()` 调用，而是四个动作的组合：持锁检查谓词，把自己加入等待集合，释放锁并睡眠，醒来后重新持锁检查谓词。

```
wait_until(q, predicate):
    lock(q.lock)
    while !predicate():
        enqueue(q.waiters, current)
        current.state = SLEEPING
        schedule_unlocking(q.lock)
    lock(q.lock)
    unlock(q.lock)
```

这段伪代码中的 `schedule_unlocking` 表达一个关键原子性：任务状态变成 sleeping、等待节点进入队列、锁释放、调度走，必须对唤醒方形成一致顺序。若先释放锁再入队，唤醒可能发生在空窗口里，造成 lost wakeup。

notify 不是消息。

条件变量和等待队列的 `notify` 或 `wake` 只是告诉等待者“相关状态可能变了”。它不携带数据，也不保证谓词已经为真，更不保证只有一个等待者会醒。醒来后必须重新检查谓词。

```
consumer:
    lock(m)
    while queue.empty() and not queue.closed:
        wait(cv, m)
    if queue.empty() and queue.closed:
        return EOF
    item = queue.pop()
    unlock(m)

producer:
    lock(m)
    queue.push(item)
    notify_one(cv)
    unlock(m)
```

若消费者用 `if` 而不是 `while`，虚假唤醒或多个消费者竞争同一个 `item` 都可能破坏逻辑。若 `producer` 在修改状态前 `notify`，消费者醒来仍可能看到旧状态。若关闭时忘记 `notify all`，等待者可能永久挂起。

信号量和条件变量的边界。

信号量表达“许可数量”，条件变量表达“对象状态变化”。它们能互相模拟一部分场景，但语义侧重点不同。

问题	适合的原语	原因
连接池最多 N 个并发连接	semaphore	许可数量就是核心状态
队列不空/不满	condition variable	谓词依赖多个字段
等待某个请求完成一次	completion/future	一次性状态转移
等待多个 fd readiness	epoll/wait queue	等待集合是 fd 对象
读多写少配置表	RCU/seqlock	读路径要极轻

把 semaphore 用来表达复杂队列状态，会让关闭、取消和错误路径变得隐晦；把 condition variable 用来表达简单许可，也会多出不必要的 mutex 和谓词。选择原语前先写状态机。

futex 的状态机。

用户态 mutex 常用一个整数表示状态：0 表示 unlocked，1 表示 locked 无已知等待者，2 表示 locked 且可能有等待者。futex 只在竞争路径进入内核。

```
lock state:
    0 unlocked
    1 locked, no known waiter
    2 locked, contended

lock:
    CAS 0 -> 1 succeeds: acquired
    otherwise set/observe 2 and futex_wait(state, 2)

unlock:
    exchange state -> 0
    if old == 2: futex_wake(state, 1)
```

内核 `futex wait` 的关键动作是检查用户地址仍等于 `expected`。若用户态状态已经变了，内核不能睡眠，而要返回让用户态重试。这个比较把用户态原子状态和内核等待队列连接起来。

读多写少：seqlock 和 RCU。

读多写少对象不一定适合普通 `mutex`。`seqlock` 让读者无锁复制数据，若写者更新期间序列号变化，读者重试。它适合时间戳、统计快照这类可重试读取，不适合包含指针且对象可能被释放的复杂结构。

```
read seqlock:
do:
    seq = read_begin()
    local = copy(shared)
    while read_retry(seq)

write seqlock:
    write_lock()
    update shared
    write_unlock()
```

RCU 则适合指针发布：读者拿到旧对象也没关系，只要旧对象在读侧临界区结束前不释放。写者创建新对象、原子发布指针、等待 `grace period` 后释放旧对象。RCU 的难点不是读，而是更新和回收。

关闭、取消和超时。

完整同步对象必须处理关闭、取消和超时。否则测试只覆盖正常路径，系统一退出就卡死。

```
pop_with_timeout(q, deadline):
    lock(q.lock)
    while q.empty and not q.closed:
        remaining = deadline - now
        if remaining <= 0:
            unlock(q.lock)
            return TIMEOUT
        timed_wait(q.not_empty, q.lock, remaining)
    if q.empty and q.closed:
        unlock(q.lock)
        return CLOSED
    item = pop(q)
    notify_one(q.not_full)
    unlock(q.lock)
    return item
```

`timeout` 路径要把等待节点从队列移除，不能留下悬挂指针。取消路径也一样。关闭路径要唤醒所有等待者，让它们重新检查 `closed`。这些路径是同步代码最容易漏测的部分。

状态机测试矩阵。

有界队列的测试不应只验证 `push/pop`。应按状态和事件组合设计：

状态	事件	期望
empty open	blocking pop	消费者睡眠
empty open	producer push	消费者被唤醒并取得 item
full open	blocking push	生产者睡眠

full open	consumer pop	生产者被唤醒并写入
empty closed	pop	返回 CLOSED/EOF
full closed	push	返回 CLOSED/EPIPE
waiting	timeout	等待节点移除，返回 TIMEOUT
waiting	cancel	等待节点移除，不破坏队列

这类测试能暴露三种经典 bug: lost wakeup、关闭不唤醒、timeout 后等待节点悬挂。压力测试只能偶然撞到这些问题，状态机测试能稳定覆盖。

同步状态机验收。

- 每个共享对象都有字段表和不变量。
- 每个等待都有明确谓词，且谓词包含关闭或取消条件。
- 每个状态改变都说明唤醒哪个等待队列。
- 每个 timeout/cancel 路径都移除等待节点。
- 每个关闭路径都唤醒所有可能等待者。
- 每个测试都能说明覆盖了哪个状态和事件。

并发、同步、IPC 与信号

原理

多线程程序的核心风险不是“同时执行”这个事实，而是共享状态缺少清晰的不变量。两个线程同时修改一个队列，硬件只能执行 load、store、CAS、fence 等局部操作；它不知道 head、tail、size、closed 之间应该满足什么关系。操作系统和运行库提供 mutex、condition variable、semaphore、futex、pipe、signal 等机制，帮助程序把共享状态、等待和唤醒写成可检查的协议。

互斥锁保护的不是某个变量，而是一组字段之间的不变量。条件变量也不是消息队列，它表达的是“某个谓词暂时不满足，线程可以睡眠，等状态可能改变后再重新检查”。正确等待必须同时包含谓词、锁和退出路径。只写 `wait()` 而不写完整谓词，会遇到虚假唤醒、丢失唤醒和关闭卡死。

一个有界队列足以说明同步原理。队列状态包括 buffer、capacity、head、tail、size、closed。生产者等待 `size < capacity || closed`，消费者等待 `size > 0 || closed`。push 改变 buffer、tail、size；pop 改变 buffer、head、size；close 改变 closed 并唤醒所有等待者。任何一个字段在锁外修改，都会破坏状态机。

IPC 是跨进程同步和数据交换。pipe 把字节流连接到 fd，socket 把通信扩展到本机或网络，shared memory 共享页但不自动提供同步，message queue 提供消息边界，eventfd 把计数型事件暴露成 fd readiness。IPC 的关键不是哪个接口高级，而是数据在哪里、所有权如何转移、背压如何发生、关闭如何传播。

信号是异步控制流。它可以通知进程发生了外部事件，例如终端中断、子进程退出、非法内存访问、定时器到期。信号难学，是因为它打断正常执行路径，而且信号处理函数只能安全调用很少一部分函数。工程上常把信号转成更普通的事件，例如写 pipe、设置原子标志或使用 signalfd，再交给主循环处理。

实践

同步实践先写不变量表。

字段	含义	受谁保护
capacity	最大槽数，固定大于 0	构造后只读
head	下一个可读槽	mutex
tail	下一个可写槽	mutex
size	当前元素数量	mutex
closed	是否拒绝新写入	mutex

然后写等待谓词：

- 生产者等待：`size < capacity || closed`
- 消费者等待：`size > 0 || closed`
- 关闭等待：`running_workers == 0` 或 `queue_empty && all_workers_stopped`

IPC 实践也按同样方式写。pipe 的状态包括读端引用数、写端引用数、内核缓冲区、读写阻塞队列。写端关闭后，读端读空返回 EOF；读端关闭后，写端写入会失败或收到信号；缓冲区满时写阻塞或返回 EAGAIN；缓冲区空时读阻塞或返回 EAGAIN。

信号实践要避免在 handler 里做复杂逻辑。推荐的纸上模型是：handler 只记录最小事实，例如设置原子标志；主循环在安全位置检查标志并执行真正清理。若系统使用 signalfd，则信号也进入 fd/event loop 模型，和其他 I/O 事件统一处理。

算法与实现分析

环形缓冲区。

环形缓冲区是设备驱动、pipe、socket buffer 和日志队列的基础结构。它用固定数组保存数据，用 head 指向下一个可读位置，用 tail 指向下一个可写位置。若额外维护 size，判断空满很直接；若不维护 size，则常留一个空槽，用 `head == tail` 表示空，用 `next(tail) == head` 表示满。

```
bool push(Ring* r, byte b) {
    if (r->size == r->capacity) return false;
    r->buf[r->tail] = b;
    r->tail = (r->tail + 1) % r->capacity;
    r->size += 1;
    return true;
}

bool pop(Ring* r, byte* out) {
    if (r->size == 0) return false;
    *out = r->buf[r->head];
    r->head = (r->head + 1) % r->capacity;
    r->size -= 1;
    return true;
}
```

push/pop 都是 $O(1)$ 。难点不在算法复杂度，而在并发所有权。单生产者单消费者可以用两个原子索引实现无锁队列；多生产者多消费者通常先用 mutex 保护不变量，等有证据证明锁是瓶颈，再考虑更复杂结构。

信号量。

信号量表达可用资源数量。P/acquire 操作在计数大于 0 时减一，否则等待；V/release 操作加一并唤醒等待者。它适合连接池许可、队列空槽、I/O in-flight 限额，不适合保护多字段复杂不变量。

```

acquire(sem):
    lock(sem.lock)
    while sem.count == 0:
        wait(sem.cv, sem.lock)
    sem.count -= 1
    unlock(sem.lock)

release(sem):
    lock(sem.lock)
    sem.count += 1
    notify_one(sem.cv)
    unlock(sem.lock)

```

信号量的常见 bug 是许可泄漏：acquire 成功后，错误路径忘记 release。工程上要用作用域对象或 finally 路径保证释放。另一个 bug 是把信号量当成关闭广播；关闭是额外状态，必须单独表达。

条件变量。

条件变量的算法核心是“检查谓词、睡眠、醒来后重新检查”。虚假唤醒不是异常，而是接口允许的行为；因此 wait 必须写在 while 循环中。

```

lock(m)
while !predicate():
    wait(cv, m)
// predicate is true while holding m
do_state_transition()
unlock(m)

```

notify 的语义也要讲清：通知不是把数据交给等待者，只是说明谓词可能变真。若生产者先通知再修改状态，消费者醒来仍看不到新状态；若修改状态后忘记通知，等待者可能永远睡眠；若关闭时只通知一个等待队列，其他等待者可能卡住。

IPC 的背压算法。

pipe 和 socket 都需要背压。缓冲区满时，写入者要么阻塞，要么在非阻塞模式返回 EAGAIN。背压算法的本质是有界队列：

```

write(pipe, bytes):
    while bytes not empty:
        if pipe.buffer_full():
            if nonblocking: return bytes_written_or_EAGAIN
            sleep_until_not_full_or_closed()
        copy_some_bytes_into_buffer()
        wake_readers()

```

这里的 copy_some_bytes 说明短写是正常路径。只要缓冲区剩余空间小于请求长度，写入就可能部分完成。调用者必须把“已完成多少字节”作为状态保存，而不是假设一次 write 完成整个消息。

测试

并发测试要覆盖正常路径、等待路径和关闭路径。只测单线程 push/pop 没有意义，因为它没有触发同步问题。

- 多生产者多消费者下，元素不丢、不重、不乱破坏队列不变量。
- 队列空时消费者等待，生产者 push 后消费者能醒来。
- 队列满时生产者等待，消费者 pop 后生产者能醒来。
- close 能唤醒所有生产者和消费者，不留下永久睡眠线程。

- timeout 或 cancel 路径不会破坏 size/head/tail。
- pipe 读写能处理 EOF、EPIPE、EAGAIN 和短读短写。
- 信号只改变允许改变的最小状态，不在 handler 中执行不安全操作。

测试报告必须写出等待谓词和唤醒来源。若测试只说“不会死锁”，但没有说明哪个线程等什么、谁负责唤醒、关闭时怎样退出，这个测试不可维护。

源码级补充：Linux IPC、信号与事件 fd

pipefs 与 pipe_buffer。

Linux 管道不是用户态数组，而是内核 pipe inode 和一组 pipe_buffer。每个 buffer 可以引用匿名页、page cache 页或 splice 传递的页片段。管道容量有默认值和上限，常见默认容量是 65536 字节量级，但会受内核版本和配置影响。

```
pipe_write(file, iov):
    pipe = file.private_data
    lock(pipe.mutex)
    while pipe_full(pipe):
        if nonblocking: return -EAGAIN
        wait(pipe.wr_wait, pipe.mutex)
    copy_or_splice_pages_into_pipe_buffers()
    wake_readers(pipe.rd_wait)
    unlock(pipe.mutex)
```

读源码时看 fs/pipe.c: pipe_write、pipe_read、pipe_wait、pipe_buffer。关注点不是函数名，而是关闭语义：写端全关且缓冲为空时读返回 EOF；读端全关时写失败并可能触发 SIGPIPE；非阻塞模式返回 EAGAIN。

System V 与 POSIX IPC。

System V IPC 提供 semaphore、message queue、shared memory，典型接口是 semget/semop、msgget/msgsnd/msgrcv、shmget/shmat。POSIX IPC 提供命名 semaphore、POSIX message queue 和共享内存对象，接口如 sem_open、mq_open、shm_open。

机制	特点	适用边界
System V semaphore	内核维护计数和 undo 语义	传统系统和跨进程资源计数
POSIX semaphore	named/unnamed 两类	进程间或线程间许可控制
System V msgqueue	类型化消息，内核队列	需要消息边界但吞吐不极致
POSIX mq	描述符化，可通知	与 fd/event loop 集成更方便
shared memory	共享页，不自带同步	大数据交换，需 futex/锁配合

共享内存常和 futex 一起使用：数据放在共享页，锁字也放在共享页；无竞争时用户态原子操作完成，有竞争时 futex 让内核按这个用户地址建立等待队列。这样数据路径不必每次进入内核。

eventfd、timerfd、signalfd 和 memfd。

Linux 把很多事件转成 fd，是为了让 epoll 能统一等待。

接口	内核语义	常见用途
----	------	------

eventfd	64 位计数器，read 消费，write 增加	线程/进程通知、io_uring、KVM kick
timerfd	定时器到期次数可读	event loop 中处理超时
signalfd	把被 mask 的信号作为 fd 读取	避免复杂 async signal handler
memfd	匿名内存文件，可 seal	共享内存、沙箱传递只读 blob

eventfd 的 EFD_SEMAPHORE 模式每次 read 只消耗 1，否则 read 返回当前计数并清零。这个小差异决定它表达的是“累计事件”还是“许可”。memfd sealing 可以禁止继续写、增长或收缩，让发送方把一段内存变成稳定对象再交给接收方。

信号投递路径。

信号不是立即调用用户函数。内核先把信号加入 pending 集合；线程返回用户态前检查 pending signal；若未被屏蔽且有 handler，内核在用户栈上构造 signal frame，把返回地址改成 handler；handler 执行后通过 sigreturn 恢复原上下文。

```
send_signal(task, sig, info):
    add_to_pending(task.signal_pending, sig, info)
    if task sleeping interruptibly:
        wake_up(task)

return_to_user(task):
    if has_unblocked_pending_signal(task):
        build_signal_frame_on_user_stack()
        task.regs.ip = handler_address
        task.regs.sp = signal_frame_sp
```

这解释了为什么 handler 可用函数受限：它运行在异步插入的用户控制流里，可能打断 malloc、printf、锁持有路径。signalfd 的工程价值在于把异步信号改造成普通 fd 事件，降低重入风险。

IPC 性能决策树。

需求	推荐思路
少量控制事件	eventfd、pipe、socketpair
需要消息边界	Unix datagram socket、POSIX mq、自定义帧协议
大块数据同机传输	shared memory + futex/eventfd 通知
要跨机器扩展	TCP/QUIC/RPC，明确超时、重试和幂等
要把文件数据转发给 socket	sendfile、splice、zero-copy 路径
要统一事件循环	fd 化 接口：epoll + eventfd/timerfd/signalfd

性能不是只看平均延迟。管道有内核拷贝和容量上限；共享内存吞吐高但同步复杂；socket 语义完整但协议开销高；message queue 保留边界但容量和优先级策略会影响尾延迟。选择 IPC 时要写出数据大小、频率、是否需要消息边界、是否跨主机、背压和关闭语义。

内核锁、死锁与可证明的等待

原理

操作系统进入并发世界以后，错误不再只来自单个函数写错变量，而来自两个执行流同时触碰同一份状态。中断处理程序可能和普通内核线程同时访问设备队列；两个 CPU 可能同时把任务放入 runqueue；一个进程关闭 fd 时，另一个线程可能正在通过同一个 open file description 发起 I/O。锁的目的不是让代码“看起来串行”，而是给共享对象规定一个可证明的临界区：谁能读，谁能写，写入完成前谁必须等待，等待时是否允许睡眠。

从裸机单片机演进过来时，最早的“锁”通常是关中断。它简单有效，因为只有一个 CPU，且并发主要来自中断打断主循环。进入多核后，关本地中断不能阻止另一个 CPU 访问同一对象，于是需要自旋锁；进入可能长时间等待的路径后，自旋浪费 CPU，于是需要 mutex、semaphore 和等待队列。每一种同步原语都绑定一个上下文限制：中断上下文不能睡眠；持有自旋锁时不能调用可能阻塞的函数；持有文件系统锁时不能随意回调用用户内存复制路径；调度器自己的锁必须在切换前后保持严格顺序。

核心理念

锁不是性能技巧，而是状态所有权的边界。设计锁时先写对象不变量，再决定哪些操作必须在同一把锁下完成。

内核锁设计要同时回答四个问题：保护哪个对象，持锁期间允许做什么，等待者在哪儿排队，失败路径如何释放。很多死锁来自第四个问题被忽略：正常路径释放锁，错误路径提前返回却忘了释放；或者关闭路径拿到 A 锁后等待 B，而另一个路径拿到 B 后等待 A。

实践

下面是常见内核锁原语的实践边界：

原语	适用场景	禁止事项	典型对象
关中断	单核短临界区，中断共享变量	多核共享保护，长时间计算	tick 计数、单核设备寄存器影子状态
自旋锁	多核短临界区，不能睡眠的上下文	持锁 I/O、持锁 page fault、持锁等待用户事件	runqueue、描述符环、引用计数
mutex	进程上下文中可能等待的互斥	中断上下文获取，持有后进入不可重入路径	inode 元数据、文件系统目录更新
读写锁	读多写少且临界区短	长读者导致写者饥饿，升级锁顺序混乱	mount 表、路由表快照
RCU	读路径极热，读者不阻塞写者	立即释放旧对象，读区内睡眠	fdtable、路径缓存、配置表
semaphore	计数资源，允许多个持有者	用它表达复杂对象不变量	buffer 数量、并发 I/O 限额

一个合理的锁设计文档至少包含下面字段：

Object:
runqueue[cpu]

Invariant:
task.state == RUNNABLE implies task is on exactly one runqueue
rq.nr_running == number of tasks linked from rq.head


```

Lock:
    rq.lock protects rq.head, rq.nr_running, task.rq_link

Allowed while holding:
    enqueue/dequeue, choose_next, update vruntime

Forbidden while holding:
    page allocation with reclaim, disk I/O, copy_from_user, waiting on mutex

```

等待队列必须把“检查条件”和“入队睡眠”放在同一把锁保护下，否则会出现 lost wakeup。正确模型不是“先看条件，不满足就睡”，而是“在保护条件的锁内，把自己放到等待集合中，然后原子地释放锁并切换状态”。唤醒方也在同一把锁下改变条件并唤醒等待者。

算法与实现分析

自旋锁。

最小自旋锁可以用原子交换实现：

```

lock(spinlock *l):
    while atomic_exchange(l->held, true) == true:
        cpu_relax()

unlock(spinlock *l):
    atomic_store_release(l->held, false)

```

这个算法的互斥性来自原子交换：同一时刻只有一个 CPU 能看到旧值为 false。复杂度不是固定常数，因为竞争时等待时间取决于持锁者运行时间和 CPU 调度。实现上必须使用 acquire/release 语义，否则 CPU 或编译器可能把临界区内存访问重排到锁外。真实内核还会加入持有者记录、递归检查、抢占关闭、IRQ 保存和等待统计。

普通 test-and-set 自旋锁在高竞争下会让所有 CPU 反复写同一个 cache line。ticket lock 用取号保证公平：

```

lock(ticket_lock *l):
    my = fetch_add(l->next, 1)
    while load_acquire(l->owner) != my:
        cpu_relax()

unlock(ticket_lock *l):
    store_release(l->owner, l->owner + 1)

```

ticket lock 的优点是 FIFO 公平，缺点是所有等待者仍然轮询 owner cache line。队列锁进一步让每个等待者轮询自己的节点，降低缓存抖动。教学内核可以先实现 test-and-set，再用压力测试观察竞争成本，最后引入 ticket 或 MCS 解释公平性和 cache line 迁移。

mutex 和等待队列。

mutex 的关键不是把自旋改成睡眠，而是在失败时把当前线程从 runnable 集合移到等待队列：

```

mutex_lock(m):
    disable_preempt()
    spin_lock(m.wait_lock)
    if m.owner == null:
        m.owner = current
        spin_unlock(m.wait_lock)
    enable_preempt()

```

```

        return
    enqueue(m.waiters, current)
    current.state = BLOCKED
    schedule_unlocking(m.wait_lock)
    enable_preempt()
    retry mutex_lock(m)

mutex_unlock(m):
    spin_lock(m.wait_lock)
    if empty(m.waiters):
        m.owner = null
    else:
        next = dequeue(m.waiters)
        m.owner = next
        next.state = RUNNABLE
        enqueue_runqueue(next)
    spin_unlock(m.wait_lock)

```

这里三个不变量：`owner` 要么为空要么指向持有线程；等待队列里的线程必须是 `BLOCKED`；被唤醒线程必须重新进入 `runnable` 集合。若 `unlock` 先释放 `owner` 再唤醒，另一个新线程可能插队，造成等待者饥饿。若唤醒时没有持有等待队列锁，等待节点可能和 `timeout/cancel` 路径并发释放。

死锁条件与锁顺序。

经典死锁需要四个条件同时成立：互斥、持有并等待、不可抢占、循环等待。内核无法完全消除前三个条件，因为对象确实需要互斥，很多资源不能强制抢占。因此工程上主要打破循环等待：给锁分层，规定全局顺序。

```

Allowed lock order:
tasklist_lock
-> mm.lock
   -> vma.lock
      -> page.lock
superblock.lock
-> inode.lock
   -> pagecache.lock
      -> buffer.lock
netns.lock
-> socket.lock
   -> skb_queue.lock

```

锁顺序不是文档装饰，而要能被测试和运行时检查。简化 `lockdep` 可以在每个线程记录当前持有锁栈，每次获取新锁时加入一条“已持有锁 -> 新锁”的有向边；若图出现环，就报告潜在死锁。

```

on_lock_acquire(new_lock):
    for held in current.held_locks:
        graph_add_edge(held.class, new_lock.class)
        if graph_has_cycle():
            report_deadlock_risk(current, held, new_lock)
    push(current.held_locks, new_lock)

```

这个检查的成本取决于锁类数量和边数量，不能在极热路径无节制开启，但作为调试构建非常有价值。更重要的是，它把“某次测试没有卡死”升级成“锁顺序图没有出现已知环”。

RCU 的读写分离。

RCU 解决的是读路径极热、写路径较少的场景。读者进入 RCU 临界区时不获取传统互斥锁，只保证自己不会在临界区内被回收对象悬空；写者发布新对象后，必

须等所有旧读者经过 quiescent state 才释放旧对象。

```
reader():
    rcu_read_lock()
    p = rcu_dereference(global_ptr)
    use(p)
    rcu_read_unlock()

writer():
    old = global_ptr
    new = clone_and_update(old)
    rcu_assign_pointer(global_ptr, new)
    call_rcu(old, free_after_grace_period)
```

RCU 的不变量是：任何读者看到的对象在读侧临界区结束前仍然有效。它不保证读者看到最新状态，也不适合读区内睡眠的普通实现。把 RCU 当成“无锁万能替代品”会导致非常隐蔽的 use-after-free。

测试

锁测试不能只跑正常路径。最低测试包括：

1. 互斥测试：多个 CPU 同时递增共享计数器，最终值必须等于操作次数。
2. lost wakeup 测试：让唤醒和入睡交错，等待者不能永久睡眠。
3. 中断上下文测试：持有自旋锁时触发同类中断，检查是否需要 irqsave。
4. 死锁检测测试：故意构造 A 后 B 与 B 后 A，lockdep 必须报告环。
5. 饥饿测试：大量短任务竞争 mutex，等待时间不能无限增长。
6. 退出路径测试：加锁后在每个错误分支注入失败，所有锁计数必须归零。

验收标准

本章合格标准：能说明每把锁保护哪个对象；能判断自旋锁、mutex、RCU 的上下文限制；能写出 lost wakeup 的反例和修复；能用锁顺序图解释死锁风险。

源码级补充：qspinlock、rwsem、seqlock 与 lockdep

raw spinlock、ticket lock 与 queued spinlock。

简单 test-and-set 锁在竞争下会让所有 CPU 反复写同一 cache line。ticket lock 提供 FIFO 公平，但所有等待者仍然轮询同一个 owner。queued spinlock 把等待者组织成队列，思想接近 MCS：每个 CPU 主要等待自己的节点，减少 cache line 抖动。

```
mcs_lock(lock, node):
    node.locked = true
    pred = xchg(lock.tail, node)
    if pred != null:
        pred.next = node
        while node.locked:
            cpu_relax()

mcs_unlock(lock, node):
    if node.next == null:
        if cmpxchg(lock.tail, node, null):
            return
        wait until node.next != null
    node.next.locked = false
```

Linux 的 `qspinlock` 比这段伪代码更紧凑，读 `kernel/locking/qspinlock.c` 时重点看三件事：pending bit 如何处理短竞争，MCS queue 如何承载长竞争，锁路径如何配合 preempt 和 paravirt。不要把 queued lock 理解成“更复杂所以一定更快”，它是为高竞争多核场景付出的复杂度。

rwlock、rwsem 和乐观自旋。

读写锁允许多个读者并行，一个写者独占。rwlock 通常用于短临界区，不能睡眠；rwsem 是可睡眠读写信号量，适合 VMA、文件系统等较长路径。rwsem 还可能做 optimistic spinning：如果持有者正在 CPU 上运行，等待者短暂自旋，期望持有者很快释放，避免睡眠/唤醒成本。

原语	优点	风险
rwlock	读者并行，路径短	长读者或写频繁时收益差，不能睡眠
rwsem	可睡眠，适合较长临界区	饥饿、公平性、唤醒策略复杂
optimistic spin	避免短等待睡眠成本	持有者不运行时浪费 CPU

读源码可看 `kernel/locking/rwsem.c`。关注 wait list、owner、reader count、writer handoff。若一个路径持有 rwsem 后等待另一个也需要此 rwsem 的工作完成，就容易形成隐藏死锁。

seqlock：写者优势的读一致性。

seqlock 用一个递增序列号表达写入是否发生。读者开始时读取 seq，复制数据，结束时再读 seq；如果 seq 变化或为奇数，重试。写者持锁并把 seq 变奇数，写完后变偶数。

```
read:
do {
    seq = read_seqbegin(&s)
    local = shared_data
} while (read_seqretry(&s, seq))

write:
write_seqlock(&s)
shared_data = new_value
write_sequnlock(&s)
```

seqlock 适合读多写少、读者可重试、数据可复制的场景，例如时间数据。它不适合读者不能重试、数据包含指针且对象生命周期复杂、写者频繁导致读者长期重试的场景。

per-CPU 与 preempt/migrate 控制。

per-CPU 变量让每个 CPU 有自己的副本，减少锁和 cache line bouncing。但访问 per-CPU 数据时，要确保当前执行不会迁移到另一个 CPU，否则先取到 CPU0 的指针，随后迁移到 CPU1 继续使用，会破坏语义。

```
this_cpu_counter_add(delta):
    preempt_disable()
    per_cpu(counter, smp_processor_id()) += delta
    preempt_enable()
```

有些路径用 `migrate_disable` 而不是完全禁止抢占，表示任务可以被抢占但不能迁移 CPU。读调度和网络栈源码时，这类边界经常出现。per-CPU 不是无锁魔法，它把共享变成分片，同时引入汇总、CPU hotplug 和迁移限制。

内存屏障原语。

原语	语义
<code>smp_rmb</code>	约束读-读顺序
<code>smp_wmb</code>	约束写-写顺序
<code>smp_mb</code>	完整内存屏障，约束读写重排
<code>smp_store_release</code>	发布数据前的写入对获取方可见
<code>smp_load_acquire</code>	看到发布指针后能看到发布前初始化

屏障必须服务于具体协议。若无法说出“谁发布、谁获取、保护哪组字段”，就不应随便加 `smp_mb`。错误屏障可能既没修 bug，又增加维护负担。

lockdep 报告怎么读。

lockdep 报告通常会给出两个锁类、已观察到的获取顺序、当前试图形成的新顺序和调用栈。读报告时先找锁类名，再画边：A 后 B，B 后 A 是否形成环；然后检查是否真实同一实例、同一上下文，还是锁类标注过粗。

```
Observed order:
inode_lock -> page_lock
New order:
page_lock -> inode_lock
Result:
possible circular locking dependency
```

不要把 lockdep 当成“测试失败噪声”。它发现的是潜在等待环，可能在当前负载下未死锁，但在错误时序下会卡死。高质量内核代码要能解释每条锁边的必要性，或调整锁类/顺序消除环。

尚未解决的问题。执行与并发边界已经形成，但所有程序仍把地址直接解释为同一物理位置。下一章从两个程序使用同一数值地址的冲突开始，把地址含义、访问权限和物理归属分离。

第二部分

地址空间与内存

本部分导读

受控入口解决了“谁可以执行内核动作”，却没有解决两个程序为何可以使用相同虚拟地址、文件页为何能被多处共享，以及内核临时对象从哪里获得存储。本部分从地址冲突推导地址空间，从重复读取推导页缓存与文件映射，再从内核对象的不同大小、寿命和上下文限制推导分配器。读者需要始终区分策略区间、当前页表映射、物理页归属和持久文件偏移。

第 6 章 为什么地址不能直接等同于物理位置

继承的机器。内核能调度执行流、处理受控请求并保护共享状态，但程序地址仍缺少独立解释。若 A 与 B 都使用地址 `0x400000`，直接物理寻址会让两项程序覆盖同一位置，也无法给代码、数据、栈和内核设置不同权限。

虚拟地址为何需要独立解释

先看一个容易误判的程序：

```
char* p = malloc(1 << 30); // request 1 GiB virtual range
p[0] = 1;                  // touch first page
p[4096] = 2;                // touch second page
```

如果 `malloc` 成功就必须立即占用 1 GiB 连续物理内存，那么程序只触碰两个页面也会长期占住其余空间；多个进程还会因为各自期望使用相似地址而互相冲突。系统需要先给程序一个可使用的地址范围，在真正访问其中某页时再决定该页对应哪一个物理页。与此同时，程序不能借此访问内核或其他进程，系统还必须为每个范围保存读、写和执行权限。

由这两个要求才得到虚拟内存：进程使用的是自身地址空间中的虚拟地址，页表在需要时把虚拟页映射到物理页，并附带 `readable`、`writable`、`executable`、`present` 等状态。一次访问是否合法，不只取决于地址数值，也取决于该范围是否已经获得承诺、访问类型是否被允许，以及尚未准备物理页时能否通过缺页处理补齐。

把这条路径拆开，会看到三个层次：

1. 用户代码只拿到虚拟地址，例如 `p + 4096`。
2. 内核用 VMA 记录“这段地址理论上属于堆，允许读写，可以按需分配”。
3. CPU/MMU 用页表项记录“这一页当前是否有物理页，权限是什么”。

因此“没有 PTE”和“非法地址”不是同一件事。一个地址如果不在任何 VMA 中，访问就是非法；一个地址在 VMA 中但还没有 PTE，可能只是第一次访问，需要缺页处理分配或调入页面。理解这层区别，后面的 `mmap`、COW、swap、page cache 才不会混在一起。

隔离来自默认不可见。一个进程不能随便读写另一个进程的页；没有映射的地址会触发缺页；只读页不能写；不可执行页不能取指。操作系统通过这些规则同时提供保护、共享和按需分配。

虚拟地址空间不仅是页表。内核还维护 VMA 或等价区域结构，用来描述一段虚拟地址的来源和权限：匿名内存、栈、堆、共享库、文件映射、设备映射。页表项是某一页当前是否能翻译，VMA 是这一段地址理论上是否合法、缺页时应该怎样处理。没有页表项不一定是错误，可能只是尚未按需分配。

一次访问的大致路径是：CPU 发出虚拟地址；TLB 命中则直接得到物理地址和权限；TLB 未命中则硬件或内核进行页表遍历；页表项不存在或权限不符时触发异常；内核根据 VMA 判断是合法缺页、权限错误还是非法访问；合法缺页可能分配物理页、从文件读页、建立 COW 私有页或映射零页。

这条路径可以用一个具体地址走一遍。假设页大小是 4096，进程访问 `0x401234`：

```
virtual address = 0x401234
virtual page    = 0x401
offset in page  = 0x234
```

```

find VMA containing 0x401234
yes: [0x400000, 0x500000) readable executable file mapping

look up PTE for virtual page 0x401
present: physical page 0x83000, readable executable

physical address = 0x83000 * 4096 + 0x234

```

若 PTE 不存在，内核不会立刻把程序杀掉，而是先问 VMA：这是不是文件映射的合法页？是不是栈增长？是不是匿名页按需分配？若 VMA 也找不到，才是典型的非法访问。若 VMA 找到了但访问类型不允许，例如往只读代码页写，就是权限错误。缺页异常只是入口，真正结论由 VMA、PTE、访问类型和错误码共同决定。

copy-on-write 是虚拟内存最有代表性的协议。`fork` 时，父子进程可以先共享物理页，并把页表标记为只读和 COW。任何一方写入时触发保护异常，内核复制物理页，更新写入方页表，再继续执行。这样 `fork` 不需要立刻复制整个地址空间，但必须正确维护引用计数和权限。

核心理念

虚拟内存不是“把地址翻译成另一个地址”这么简单。它同时维护三件事：哪些虚拟区间被承诺，当前哪些页已经物化成物理页，每次读写执行是否被授权。

VMA、页表项与物理页的职责

实验的 `PageTable` 支持按页映射。读访问要求 `present` 和 `readable`；写访问要求 `writable`；执行访问要求 `executable`。地址翻译保留页内 `offset`，帮助读者理解“页号转换，页内偏移不变”。

输入	计算
page size	4096
virtual address	0x401234
virtual page number	0x401
offset	0x234
physical page number	查页表得到
physical address	<code>physical_page * 4096 + offset</code>

权限矩阵要单独写：

访问	present	readable	writable	executable	结果
load	yes	yes	no	no	成功
store	yes	yes	no	no	权限错误
fetch	yes	yes	no	no	NX 错误
fetch	yes	yes	no	yes	成功
load	no	-	-	-	缺页或非法

观察真实系统时，要区分虚拟大小、RSS、page cache、minor fault 和 major fault。虚拟地址空间很大，不代表物理内存已经占用；RSS 增加，可能是匿名页，也可能是文件页；minor fault 可能只是建立映射，major fault 更可能等待存储设备。

页面建立、复制、回收与换出

物理页分配：free list、bitmap 和 buddy

最简单的物理页分配器是 free list。每个空闲页框放进链表，分配时弹出一个，释放时放回去。它的分配和释放都是 $O(1)$ ，适合固定大小页框；缺点是难以快速找到连续页，也难以检测重复释放和碎片分布。

```
page_alloc():
    if free_list.empty(): return null
    page = free_list.pop()
    page.refcount = 1
    return page

page_free(page):
    assert(page.refcount == 0)
    free_list.push(page)
```

bitmap 分配器用一个 bit 表示一个页框是否空闲。它节省元数据，容易扫描连续区域，但朴素扫描是 $O(n)$ 。可以通过分层 bitmap 或缓存上次搜索位置降低平均成本。

buddy allocator 用 2 的幂大小管理连续页块。分配 8 页时，寻找 order=3 的块；如果只有更大的块，就不断拆分；释放时，如果相邻 buddy 也空闲，就合并成更大块。buddy 的分配和释放大约是 $O(\log N)$ ，能较好处理连续页需求，但仍会产生内部碎片。

虚拟地址查找：VMA 和页表

页表回答“这一页当前映射到哪里”，VMA 回答“这一段地址理论上是否合法”。缺页时，内核先查 VMA：若地址不属于任何 VMA，就是非法访问；若属于可按需分配的匿名 VMA，就分配零页；若属于文件映射 VMA，就从 page cache 找页或发起 I/O。

VMA 可以用有序区间结构保存。查找地址属于哪个区间，朴素线性扫描是 $O(n)$ ；平衡树查找是 $O(\log n)$ ；现代内核还会使用更适合区间查询和缓存局部性的结构。教学模型可以先用有序数组，因为它能把语义讲清楚。

```
handle_page_fault(addr, access):
    vma = find_vma(addr)
    if vma == null: signal(SIGSEGV)
    if !vma.allows(access): signal(SIGSEGV)
    page = materialize_page(vma, addr)
    install_pte(addr, page, vma.permissions)
```

COW 的引用计数

COW 的不变量是：共享页必须只读，物理页有引用计数；写入共享页时，写入方得到新副本。若 refcount 为 1，可以直接把页改成可写；若 refcount 大于 1，必须复制。

```
handle_write_fault(vaddr):
    pte = lookup_pte(vaddr)
    page = pte.page
    if !pte.cow: signal(SIGSEGV)
    if page.refcount == 1:
        pte.writable = true
        pte.cow = false
    else:
        new_page = alloc_page()
        copy_page(new_page, page)
        page.refcount -= 1
        install_pte(vaddr, new_page, writable=true, cow=false)
```


这段算法最容易错在引用计数和权限更新顺序。若忘记把共享页设为只读，写入不会触发 fault，父子进程会互相污染；若复制后忘记减少旧页引用计数，会泄漏；若 TLB 中仍缓存旧权限，必须刷新或失效对应条目。

用一个父子进程的实际页来走一遍。父进程有一页虚拟地址 `0x7000`，物理页是 `P42`，内容是字符串 `"abc"`。`fork` 前，父进程 PTE 指向 `P42`，权限可读可写，`P42.refcount = 1`。`fork` 时，内核不复制 `P42` 的 4096 字节，而是让父子页表都指向同一个 `P42`，同时把两个 PTE 都改成只读，并在软件元数据里标记 COW，`P42.refcount = 2`。

时刻	父进程 PTE	子进程 PTE
fork 前	<code>0x7000 -> P42, writable</code>	不存在
fork 后	<code>0x7000 -> P42, read-only, COW</code>	<code>0x7000 -> P42, read-only, COW</code>
子进程写入前	<code>P42.refcount = 2</code>	store 触发写保护 fault
子进程 fault 后	仍指向 <code>P42, read-only, COW</code>	指向新页 <code>P99, writable</code>
后续父进程写入	若 <code>P42.refcount = 1</code> 可直接恢复 <code>writable</code>	已经和父进程分离

关键点是“写入必须在 store 提交前被拦住”。CPU 执行子进程的 `*p = 'x'` 时，地址翻译发现 PTE present 但不可写，于是保存 faulting address、错误码和 RIP 进入内核。因为 store 还没有修改 `P42`，内核才能安全复制旧页到 `P99`，把子进程 PTE 改为 `P99, writable`，再返回原指令重试。重试后写入落在 `P99`，父进程继续看到旧的 `"abc"`。

失败路径也要讲清楚。若 fault 地址不在允许写的 VMA 内，不能执行 COW，而应给进程发段错误。若分配 `P99` 失败，可能触发回收或 OOM。若更新 PTE 后没有失效旧 TLB，CPU 可能继续使用旧只读翻译，导致重复 fault；更危险的是撤销可写权限时没有失效旧可写翻译，会让某个 CPU 绕过新的只读保护继续写共享页。COW 的正确性不是“复制一页”这么简单，而是 VMA 权限、PTE 权限、物理页引用计数、TLB 失效和 fault 可重试性共同成立。

页面置换

当物理内存不足时，内核要回收页面。理想算法 OPT 总是淘汰未来最晚再访问的页面，但它需要知道未来，不能实现。LRU 淘汰最近最久未使用的页，更接近实际局部性，但精确 LRU 维护成本高。Clock 算法用访问位近似 LRU：扫描环形列表，访问位为 1 就清零并跳过，访问位为 0 就淘汰。

```
clock_evict():
    while true:
        page = clock_hand.page
        if page.referenced:
            page.referenced = false
            advance(clock_hand)
        else if page.dirty:
            schedule_writeback(page)
            advance(clock_hand)
        else:
            remove_and_return(page)
```

置换策略必须和文件系统、匿名页、swap、dirty writeback 结合。干净文件页可以直接丢弃，未来再从文件读；脏文件页要先写回；匿名页若要回收，需要 swap 或压缩；被锁定或正在 I/O 的页不能随便回收。

多级页表

64 位地址空间太大，不能为每个虚拟页都预留一个线性页表。多级页表把虚拟页号拆成多段索引，只为实际使用的区域分配中间页表页。以四级页表为例，虚拟地址被拆成 L4、L3、L2、L1 和页内 offset。

```
walk_page_table(root, vaddr, create):
    table = root
    for level in [L4, L3, L2]:
        index = index_for(level, vaddr)
        if table[index] not present:
            if not create:
                return null
            table[index] = allocate_page_table()
            table = table[index].next_table
    return &table[index_for(L1, vaddr)]
```

查找复杂度按层数是常数，但每层都可能带来内存访问。TLB 的价值就在于缓存最近翻译，避免每次访问都走多级页表。页表设计还影响内核内存占用：稀疏地址空间只分配用到的页表页，密集映射则消耗更多元数据。

TLB shutdown

改变页表后，CPU TLB 中可能仍缓存旧翻译。单核系统可以本地失效；多核系统中，同一地址空间可能正在其他 CPU 运行，需要发送 shutdown IPI 让它们失效对应条目。

```
unmap_page(mm, vaddr):
    pte = walk_page_table(mm.root, vaddr, false)
    old = clear_pte(pte)
    cpus = mm.active_cpus
    for cpu in cpus except current_cpu:
        send_tlb_shutdown(cpu, mm, vaddr)
    local_invlpg(vaddr)
    wait_for_shutdown_ack(cpus)
    put_page(old.page)
```

顺序很重要。若在其他 CPU 失效 TLB 前就释放物理页，该页可能被重新分配给别的对象，而旧 CPU 仍通过过期 TLB 写入它，形成严重内存破坏。TLB shutdown 是虚拟内存和多核同步的交叉点。

再把这个危险展开成具体事故。进程 X 有虚拟页 0x9000 -> P10，它的两个线程分别在 CPU0 和 CPU1 上运行。CPU1 的 TLB 里已经缓存了这条翻译。CPU0 执行 munmap(0x9000)，把页表项清掉，然后若立刻把 P10 释放给物理页分配器，分配器可能马上把 P10 交给内核网络栈当作 packet buffer。此时 CPU1 如果还没收到 shutdown，仍可用旧 TLB 把用户态 0x9000 翻译到 P10，一次普通 store 就会写坏网络 buffer。这个 bug 表面可能表现成网络包校验失败、随机内核崩溃或安全越权，根因却是页表撤销和 TLB 旧副本之间的生命周期错误。

因此 shutdown 的完成点必须在“物理页可重用”之前。教学模型可以记成三步：先让新页表状态不可再产生新翻译；再让所有可能持有旧翻译的 CPU 丢掉旧 TLB；最后释放旧物理页或改变其用途。

阶段	必须完成的事	如果提前进入下一步
清 PTE	后续 page walk 不能再得到旧映射	新 TLB 项仍可能被硬件装入
本地失效	当前 CPU 不再使用旧翻译	当前线程可能继续访问已撤销页

远端 IPI	其他 active CPU 执行 <code>invlpg</code> 或等价操作	远端 CPU 可能写入已释放物理页
等待 ack	确认旧翻译不可再被使用	释放页会造成 use-after-free 到物理内存
释放旧页	页框回到分配器或引用计数下降	到这一步才安全改变页用途

这也是为什么 `mprotect`、`munmap`、COW 和进程退出在多核上不是“改一张表”就结束。内核还要知道哪些 CPU 可能正在运行这个地址空间，哪些映射范围需要 flush，能否批量合并 shutdown，是否允许延迟释放。优化可以减少 IPI 次数，但不能破坏上面的安全边界。

overcommit 与 OOM

虚拟内存允许系统承诺比物理内存更多的虚拟地址。例如 `mmap` 或 `malloc` 成功不代表物理页已经分配，首次写入才可能触发分配。overcommit 提高灵活性，但内存真正不足时必须选择失败策略。

```
handle_anon_fault(vma, addr):
    page = alloc_page()
    if page == null:
        victim = oom_select_process()
        if victim == null:
            return SIGBUS_OR_ENOMEM
        kill(victim)
        retry allocation
    zero_page(page)
    install_pte(addr, page, WRITE | USER)
```

OOM killer 不是随意杀进程。它通常综合内存占用、权限、oom score、是否关键服务、cgroup 限制和可回收性。容器场景下，cgroup 内 OOM 应优先限制在该 cgroup 内，而不是扩大到整个宿主。

把 overcommit 放进一个常见服务事故。机器有 8GiB 内存，服务 A 启动时 `malloc(6GiB)` 做缓存索引，服务 B 也 `malloc(6GiB)` 准备批处理。两个 `malloc` 都可能成功，因为内核只是给它们各自建立虚拟区间和 VMA，并没有立即分配 12GiB 物理页。监控里 VSZ 很大，但 RSS 还不高。事故发生在两者开始真正写入页面时：

```
for each 4096-byte page in allocated range:
    p[i] = 0;
```

每写一个尚未物化的匿名页，CPU 都会触发缺页；内核确认 VMA 允许写，分配物理页，清零，安装 PTE，然后重试 store。刚开始这很顺利；随着空闲页减少，内核先回收干净文件页，再尝试写回脏页，再考虑 swap 或压缩；若仍不能满足分配，就进入 OOM 决策。此时失败点不是 `malloc`，而是某一次普通 store 的缺页处理。

阶段	内核状态	应用看到什么
<code>malloc</code> 成功	VMA/堆区扩展，物理页可能未分配	返回非空指针
首次写前几页	匿名页缺页，分配物理页并清零	store 正常完成，只是可能变慢
内存压力上升	回收 page cache、写回脏页、触发 swap	延迟抖动，RSS 上升

无法回收	OOM 在进程或 cgroup 内选择受害者	某进程被杀，或 fault 返回失败语义
cgroup 限制命中	只在该资源组内回收和 OOM	宿主其他服务不应被拖下水

这解释了为什么服务不能只用“malloc 成功”作为容量保证。真正可靠的设计要设置 cgroup memory.max、应用层缓存上限、分批预热、失败回退和监控 RSS/PSI。overcommit 的价值是让稀疏使用的大地址空间可行；代价是失败可能延迟到第一次触碰具体页面时才出现。

内存压力、回收失败与 OOM

测试要覆盖成功翻译、写只读页失败、访问未映射页失败、unmap 后访问失败、页内 offset 保留。错误结果必须携带可读诊断，不能只返回一个空值。

- fork 后父子共享只读 COW 页，任一方写入后得到私有副本。
- mmap 文件页首次访问触发填页，第二次访问走热路径。
- 栈增长必须受上限约束，不能无限扩张。
- mprotect 改变权限后，旧访问策略失效。
- munmap 后再次访问必须失败。
- 用户态指针传入系统调用时，内核必须检查可读/可写范围。

权限测试尤其重要。没有权限测试的虚拟内存模型，会把地址转换误写成普通哈希表查找，丢掉操作系统最重要的保护语义。

地址策略、当前映射与页面归属

虚拟内存的细节很多：page fault、VMA 切分、PTE 标志、TLB shutdown、COW、swap、THP、mlock、PSI、OOM 和 cgroup。主线里先把它压成一张“缺页账本”。每次 CPU 访问地址失败，内核都要回答：地址属于哪个 VMA，访问类型是否被允许，页面内容从哪里来，PTE 怎样安装，旧 TLB 是否要失效，失败时返回信号还是 errno，线程是否需要睡眠等待 I/O 或回收。

场景	内核主要动作	关键风险
匿名首次写	分配物理页、清零、安装 writable PTE	overcommit 后在首次触碰时 OOM
COW 写 fault	检查引用、复制页面、替换 PTE、flush TLB	父子进程互相污染或复制后权限错误
文件映射 fault	查 page cache、必要时发起读 I/O、安装映射	把 I/O 等待误判成 CPU 慢
mprotect	VMA 切分、改权限、更新页表、失效 TLB	旧 TLB 保留过宽权限
内存回收	扫 LRU、写回脏页、swap out、唤醒等待者	回收不可回收页或引发严重抖动

虚拟内存可以按对象追踪：mm_struct 是进程地址空间账本，VMA 是区间权限和来源账本，PTE 是处理器当前可执行的翻译账本，struct page/folio 是物理页生命周期账本，rmap 则把物理页反查到映射它的地址空间。任何优化都不能破坏这些账本的一致性。THP 减少 TLB 压力，但拆分时仍要恢复普通页语义；swap 降低内存占用，但 fault 延迟会进入请求尾部；mlock 保护实时路径或密钥页，但会减少可回收集合；PSI 不消除内存压力，只报告线程因内存等待而失去前进能力。

内核自己的内存分配也属于这张账本。页分配器负责按页框分配和回收，buddy 系统处理连续页块；slab/SLUB 把页切成固定大小对象，服务 `task_struct`、`inode`、`dentry`、`socket` 等高频对象；`vmalloc` 提供虚拟连续但物理不连续的内存映射。教学实现可以先只有 page allocator 和几个对象池，但必须保留三个边界：中断上下文不能随意睡眠等待内存，DMA buffer 可能要求物理连续或满足设备地址限制，对象缓存释放后不能仍被 RCU 读者或异步 completion 引用。

分配器	适合的对象	验收问题
页分配器	页表页、大块缓冲、匿名页和 page cache 页	压力下是否能回收、写回或明确失败
SLUB/slab	大量同类型小对象	析构、引用计数、越界和 use-after-free 是否可检测
vmalloc	大型内核虚拟连续缓冲	不能交给要求物理连续 DMA 的设备
per-CPU allocator	每 CPU 统计、队列和热路径缓存	CPU 热插拔和迁移时账本是否一致

验收标准

本章验收问题：给一个虚拟地址和一次读/写/取指访问，能否说明它命中哪个 VMA、PTE 应有哪组权限、缺页后页面从哪里来、是否可能睡眠、失败给用户什么信号或错误、哪些观测指标能证明判断。能回答这些，才算把虚拟内存读成 OS 机制，而不是地址转换表。

巨页、交换与地址翻译成本

页表项标志

x86-64 PTE 典型标志包括 `present`、`writable`、`user`、`accessed`、`dirty`、`global`、`NX` 等。操作系统把 VMA 权限、文件打开模式、COW 状态和硬件标志合成 PTE。注意 COW 不是单纯硬件位，通常由软件在只读 PTE 和附加元数据中表达。

标志	语义
present	PTE 可用于地址翻译，否则触发缺页
writable	store 是否允许
user	ring 3 是否可访问
accessed	CPU 访问后置位，用于回收近似 LRU
dirty	CPU 写入后置位，用于写回和 COW 判断
NX	禁止取指执行

权限收缩必须刷新 TLB。例如把页从可写改成只读，如果旧 TLB 仍保留可写权限，写入可能绕过新的 COW 或 `mprotect` 规则。

mprotect、munmap 和 VMA 切分

`mprotect` 改变区间权限，`munmap` 删除映射。二者都可能把一个 VMA 切成三段：前半保持原权限，中间修改或删除，后半保持原权限。

```
before:
  [0x1000, 0x9000) RW
mprotect [0x3000, 0x5000) R
after:
```



```
[0x1000, 0x3000) RW
[0x3000, 0x5000) R
[0x5000, 0x9000) RW
```

VMA 切分后，要更新页表权限、失效 TLB、处理锁定页和文件映射引用。教学实现若只改一个 flags 字段，会在部分区间修改时出错。

Transparent Huge Page

普通页常为 4KiB，大页可为 2MiB 或更大。THP 尝试把连续匿名页合并成 PMD 级大页，减少 TLB miss。代价是分配连续内存更难，拆分和 COW 成本更高，延迟可能出现尖峰。

```
khugepaged:
  scan VMAs marked hugepage-friendly
  find 512 contiguous 4KiB pages
  allocate 2MiB huge page
  copy or remap contents
  replace PTE table with PMD huge mapping
```

THP 是性能优化，不应改变用户语义。发生 mprotect、munmap、COW 或内存压力时，大页可能被拆回普通页。性能报告要说明 THP 是否开启，否则 TLB 和内存延迟数据难以比较。

swap cache 和 swappiness

匿名页回收需要把内容写到 swap。swap slot 是匿名页在交换区的位置；swap cache 避免同一 swap 页被重复读写。swappiness 影响内核在文件页和匿名页之间的回收倾向。

```
swap_out(page):
  slot = allocate_swap_slot()
  write_page_to_swap(slot, page)
  replace_pte_with_swap_entry(vaddr, slot)
  free_physical_page(page)

swap_in(fault):
  slot = fault.swap_entry
  page = swap_cache_lookup_or_read(slot)
  install_pte(fault.addr, page)
```

swap 让系统在内存压力下继续运行，但延迟可能变得非常高。服务端系统常用 PSI、cgroup memory.high 和限流来避免进入严重 swap 抖动。

mlock 和不可回收页

mlock 把页锁在内存中，常用于实时路径、密码材料或避免 page fault 的场景。锁定页不能被普通回收换出，因此必须受权限和 rlimit 限制。

```
mlock(addr, len):
  check RLIMIT_MEMLOCK
  for each VMA page in range:
    fault page in if needed
    mark unevictable
```

滥用 mlock 会减少可回收内存，影响整机。教学 OS 要把“不可回收”作为页面状态加入回收测试，证明回收器不会错误丢弃 locked page。

系统调用与内存深讲：一次请求如何安全穿过边界

系统调用和内存管理是 OS 隔离能力的核心。系统调用控制“用户能请求什么”，页表控制“用户能访问什么”。二者常常一起出现：用户把指针传给系统调用，内核

必须检查这段用户内存；缺页可能在系统调用复制参数时发生；`exec` 会替换地址空间并重建用户栈。

为什么用户指针不能直接解引用。

用户传入的地址可能不存在、只读、跨页、被另一个线程并发修改，甚至在检查后被 `munmap`。内核若直接解引用，轻则崩溃，重则形成安全漏洞。因此内核要用专门的 `copy` 接口，并准备处理 `copy` 期间的 `page fault`。

```
sys_write(fd, user_buf, len):
    file = lookup_fd(fd)
    if file == null:
        return -EBADF
    if not file.can_write:
        return -EBADF
    kbuf = allocate_temp(min(len, MAX_RW_COUNT))
    ret = copy_from_user(kbuf, user_buf, len)
    if ret < 0:
        return -EFAULT
    return vfs_write(file, kbuf, len)
```

注意顺序：先查 `fd` 和写权限，再复制用户数据；复制失败不能留下半提交文件状态。对很大的写入，真实系统不会一次复制全部，而是分段处理；每段成功后返回值可能是短写，调用者必须循环。

`page fault` 的三种结论。

同样是 `page fault`，内核结论可能完全不同。

结论	条件	处理
合法缺页	地址属于 VMA，权限允许，只是 PTE 不存在	分配页、读文件页或建立 COW 页
权限错误	地址属于 VMA，但访问类型不允许	发送 SIGSEGV 或返回 <code>EFAULT</code>
非法地址	不属于任何 VMA	发送 SIGSEGV 或返回 <code>EFAULT</code>

区别取决于上下文。用户态普通 `load/store` 错误通常给进程发信号；内核在 `copy_from_user` 中遇到用户地址错误，通常让系统调用返回 `EFAULT`。内核访问自己错误地址则是内核 bug，不能当普通用户错误处理。

`fork`、COW 与 `exec` 的组合。

shell 启动程序常见流程是 `fork` 后在子进程中设置 `fd`，再 `exec`。若 `fork` 立刻复制整个地址空间，启动大进程会很慢。COW 让父子先共享只读页，只有写入时才复制。`exec` 成功后，子进程旧地址空间被替换，很多 COW 页根本不用复制。

```
parent shell:
    pid = fork()

child after fork:
    dup2(file, STDOUT)
    execve(program)

kernel optimization:
    fork shares pages as read-only COW
    exec discards old user mappings
    only pages written before exec are copied
```

这说明机制之间是组合出来的。fork 的语义简单，COW 让它便宜，exec 让它变成启动新程序，fd 继承让 shell 能做重定向。很多 OS 设计的美感来自小机制组合，而不是一个巨大的 spawn 黑盒。

内存回收为什么会影响系统调用延迟。

系统调用可能看起来只是写 socket 或打开文件，却在中途分配内存。若内存不足，分配器可能触发回收；回收可能写回脏页；写回可能等待块设备；块设备完成再唤醒当前线程。于是一个普通请求的尾延迟可能来自内存和 I/O，而不是业务代码。

```
sys_send()
-> allocate socket buffer
-> memory pressure
-> reclaim pages
-> dirty page writeback
-> block I/O wait
-> scheduler wakeup
-> return to send path
```

这也是为什么真实性能分析要看 off-CPU 时间、page fault、writeback 和 block I/O。只看 CPU profiler 会漏掉大部分等待。

系统调用与内存练习。

1. 为什么 copy_from_user 失败应该返回 EFAULT，而不是让内核 panic？
2. fork 后父子进程共享 COW 页，父进程写入一页后，页表和物理页引用计数怎样变化？
3. 为什么 exec 失败前应尽量保留旧地址空间可运行？
4. mmap(MAP_PRIVATE) 写入文件映射页后，文件内容为什么不变？
5. 为什么内存压力会让看似无关的系统调用变慢？

系统调用错误矩阵。

高质量系统调用实现要能区分不同失败原因。把所有失败都返回一个通用错误，会让调用者无法恢复。

错误	典型原因	调用者策略
EBADF	fd 不存在或模式不允许	程序 bug 或句柄生命周期错误
EFAULT	用户指针不可访问	程序 bug，不能重试同一指针
EINTR	阻塞调用被信号打断	根据已完成字节数决定是否重试
EAGAIN	非阻塞对象暂时不能推进	等 readiness 后重试
ENOMEM	内存不足或限制触发	降级、释放资源或失败返回
EIO	设备或底层 I/O 错误	上报、切换副本、进入恢复流程
EACCES	权限不足	请求授权或拒绝操作

错误码本身就是 API 的一部分。比如 EAGAIN 表示状态可能稍后改变，EFAULT 表示调用者传了坏地址，二者的恢复策略完全不同。

页表权限和文件权限的区别。

初学者容易混淆“我能打开文件”和“我能访问内存”。文件权限是文件系统对象上的访问控制；页表权限是 CPU 执行每次内存访问时检查的硬件权限。二者可以关联，但不是同一个东西。

```
open file read-only:
    file.can_write = false

mmap file private writable:
    VMA may allow write
    first write creates anonymous COW page
    file content is not modified

mmap file shared writable:
    fd must allow write
    dirty page belongs to file mapping
    msync/fsync affects persistence
```

这就是为什么 `MAP_PRIVATE` 可以让只读文件映射在进程内“可写”：写入的是私有匿名副本，不是文件本体。反过来，`MAP_SHARED` 可写映射必须受文件打开模式和文件系统权限限制。

本章小结。

- 系统调用跨越信任边界，必须检查 fd、权限、用户指针和对象状态。
- page fault 不是单一错误；它可能是合法懒加载、权限错误或非法地址。
- fork、COW、exec、fd 继承共同组成高效进程启动模型。
- 内存压力可能把普通系统调用拖入回收和 I/O 等待。
- 错误码要表达恢复策略，而不是只表达“失败了”。

系统调用入口的生命周期。

系统调用入口不是简单跳到一个 C 函数。CPU 从用户态进入内核后，内核要切换到受信任内核栈，保存用户寄存器，建立当前线程的系统调用上下文，验证系统调用号和参数，再调用具体实现。返回前还要处理信号、抢占、审计、ptrace/seccomp 等边界。

```
syscall_entry:
    switch_to_kernel_stack()
    save_user_registers()
    nr = user_registers.syscall_number
    if seccomp_denies(nr):
        return_error_or_kill()
    ret = syscall_table[nr](arg0, arg1, ...)
    if signal_pending(current):
        prepare_signal_frame_or_restart()
    if need_reschedule:
        schedule()
    restore_user_registers(ret)
    return_to_user()
```

这条路径解释了几个现象。系统调用可能被信号打断，也可能在返回用户态前被抢占；同一个系统调用在 tracing 或 seccomp 下成本不同；错误返回不是异常抛出，而是 ABI 规定的返回值和错误码。系统调用接口必须稳定，因为它是用户程序和内核之间的长期二进制契约。

用户指针的 TOCTOU 问题。

即使内核先检查用户地址范围，再复制数据，也不能假设这段内存存在检查后不变。另一个线程可能修改 buffer、调用 `munmap`，或改变内存内容。正确模型是：检

查只能证明“此刻看起来可访问”，真正复制必须能处理 `fault`；若需要稳定数据，内核要复制到内核缓冲区后再验证和使用。

```
bad_path(user_header):
    if validate_user_header(user_header):
        // user thread may change fields here
        use_header_fields_directly(user_header)

better_path(user_header):
    hdr = copy_from_user(user_header, sizeof(Header))
    if !validate_header(hdr):
        return -EINVAL
    use_private_copy(hdr)
```

不是所有用户数据都必须一次复制完。大 I/O 通常分段复制或使用 `pinned pages`、`iov iterator`、`zero-copy` 机制。但无论哪种方式，所有权和生命周期都要写清楚：谁能修改这段内存，设备或内核何时完成访问，失败时如何解除 `pin` 或引用。

VMA、PTE 和 page cache 的三层关系。

很多虚拟内存错误来自混淆三层对象。VMA 是虚拟地址区间的策略；PTE 是某个虚拟页当前的硬件映射；page cache 是文件内容在内存中的缓存页。文件 `mmap` 首次访问时，VMA 说明这段地址来自哪个文件和偏移；page cache 提供或创建对应页；PTE 将该页映射进进程地址空间。

对象	回答的问题	典型变化
VMA	这段地址是否合法，权限和来源是什么	<code>mmap/munmap/mprotect</code> 改变
PTE	这一页现在是否可由硬件访问	缺页、换出、COW、 <code>unmap</code> 改变
page cache page	文件某偏移的内容是否在内存	<code>read</code> 、 <code>mmap fault</code> 、 <code>writeback</code> 、 <code>reclaim</code> 改变

因此，没有 PTE 不一定是错误；可能是合法 VMA 的懒加载。没有 VMA 通常才是非法地址。page cache 中有页，也不代表某进程已经有 PTE；多个进程可以通过不同 PTE 映射同一个 page cache 页。

缺页处理的完整分支。

缺页处理要先分类，再处理。分类至少包括：地址是否在用户范围，是否属于 VMA，访问权限是否允许，PTE 是否存在但权限不符，是否 COW，是否文件页，是否栈增长，是否内核 `copy` 用户数据时发生。

```
handle_fault(mm, addr, access, context):
    if !is_user_range(addr):
        return fault_result(context, BAD_ADDRESS)
    vma = find_vma(mm, addr)
    if vma == null:
        return fault_result(context, NO_VMA)
    if is_stack_growth(vma, addr):
        expand_stack_or_fail(vma, addr)
    if !vma.allows(access):
        return fault_result(context, PERMISSION)
    pte = walk_pte(mm, addr, create=true)
    if pte.present and access.write and pte.cow:
        return handle_cow_fault(pte)
    if !pte.present:
        return materialize_page(vma, addr, access)
```



```
return fault_result(context, PERMISSION)
```

`fault_result` 取决于上下文。用户态访问失败通常发信号；系统调用复制用户内存失败通常返回 `EFAULT`；内核访问内核坏地址通常 `panic`。把这些路径混成一个错误码，会掩盖边界。

页面回收的实际顺序。

内存回收不是随机找页释放。回收器通常先尝试容易回收的页：干净文件页可以直接丢；脏文件页要写回；匿名页需要 `swap` 或压缩；被 `mlock`、`DMA pin`、正在 `I/O` 或引用计数异常的页不能回收。

```
reclaim_scan():
    for page in inactive_list:
        if page.pinned or page.under_writeback:
            skip(page)
        else if page.clean and page.file_backed:
            remove_from_page_cache(page)
            free(page)
        else if page.dirty and page.file_backed:
            start_writeback(page)
        else if page.anonymous and swap_available:
            write_to_swap(page)
        else:
            keep_or_escalate_pressure()
```

这个顺序解释了为什么内存压力会传导到存储 `I/O`。系统缺内存时，回收器可能把脏页写回设备；设备慢时，回收也慢；回收慢又会让分配路径阻塞，最终表现为系统调用尾延迟升高。

TLB、PCID 和 `shutdown` 成本。

TLB 缓存虚拟地址翻译。切换地址空间时，若没有 `x86-64 PCID` 这样的标签，CPU 需要清空大量 TLB 项；有 `PCID` 时，可以保留不同地址空间的翻译，但页表修改仍要失效相关条目。多核下，其他 CPU 可能正在用同一地址空间运行，必须通过 `IPI` 通知它们失效。

操作	必要性	成本来源
local invalidate	当前 CPU 旧翻译不能继续用	pipeline/TLB 扰动
remote shutdown	其他 CPU 旧翻译不能继续用	IPI、等待 <code>ack</code> 、跨核同步
batch unmap	合并多个失效请求	延迟释放与复杂生命周期
PCID	减少切换地址空间的全局 flush	标识管理、复用和一致性规则

这也是为什么高频 `mmap/munmap/mprotect` 可能拖慢多线程程序。它们不是只改一张表，还可能引发跨 CPU 协议。优化思路通常是批量映射、复用区域、减少权限翻转和避免在热路径频繁改变页表。

分配器层次：页、`slab` 和用户堆。

内核内存分配有层次。页分配器管理物理页框；`slab/slub` 等对象分配器管理固定大小内核对象；用户态 `malloc` 在进程地址空间内管理堆，并通过 `brk/mmap` 向内核申请更大区域。

分配层	管理对象	典型问题
-----	------	------

page allocator	物理页和连续页块	外部碎片、NUMA、本地性
slab allocator	inode、task、dentry 等固定对象	对象复用、构造析构、缓存热度
vm allocator	内核虚拟连续区域	页表开销、TLB、地址空间碎片
user malloc	用户堆小对象	arena、碎片、RSS 与虚拟大小差异

理解层次能避免错误诊断。用户进程 RSS 增长不一定是内核泄漏；slab 增长可能来自 dentry/inode cache；虚拟地址空间变大不一定占用物理页；页分配失败可能是连续块碎片而不是总内存为零。

exec 的失败原子性。

execve 成功后旧程序消失；失败时通常应让调用进程还能继续运行并看到错误码。这要求内核在替换地址空间时谨慎排序。若过早销毁旧地址空间，然后发现 ELF 无效或无法分配栈，就可能让进程处于不可恢复半状态。

```
execve(path, argv, envp):
    file = open_executable(path)
    image = validate_elf(file)
    new_mm = build_new_address_space(image, argv, envp)
    if any_step_failed:
        destroy_partial_new_mm()
        return -errno
    old_mm = current.mm
    current.mm = new_mm
    switch_page_table(new_mm)
    destroy_old_mm(old_mm)
    start_at_entry(image.entry)
```

真实实现还要处理解释器脚本、动态链接器、setuid、安全标签、fd 的 CLOEXEC、信号处理器重置和多线程 exec 语义。核心原则仍然是：新镜像完全准备好之前，不要破坏失败后必须保留的旧执行环境。

内存与系统调用测试矩阵。

测试	触发方式	期望
坏用户读指针	write(fd, bad, len)	返回 EFAULT，内核不崩溃
坏用户写指针	read(fd, bad, len)	返回 EFAULT 或短读语义清楚
COW 写入	fork 后父子分别写同页	内容分离，引用计数正确
权限缺页	写只读映射或执行 NX 页	信号或错误，不能绕过权限
munmap 竞态	复制期间另线程 unmap	返回错误或短完成，不 use-after-free
TLB 失效	mprotect 后旧权限访问	旧权限不能继续生效
内存压力	限制内存后触发分配	有 OOM/ENOMEM 证据和恢复路径

这些测试共同证明：边界错误不会变成内核错误，权限变化能真正影响硬件访问，资源不足有可解释失败，部分完成语义被调用者看见。

从缺页现场到可测试地址空间

第三阶段 C++20 模型：从 VMA 到 PTE 的可测内存管理器

虚拟内存不能只靠文字理解。读者至少要实现一个小模型，能把 VMA、PTE、物理页、COW 和缺页处理连起来。这个模型不追求模拟所有硬件细节，而是追求状态可检查：非法地址不能访问，权限收缩必须生效，COW 写入必须复制，引用计数必须闭合。

模型边界。

教学模型保留四类对象。

对象	责任	不做什么
PhysicalMemory	分配页框、维护引用计数、复制页	不模拟 NUMA、cache、DMA
AddressSpace	管理 VMA 和页表	不模拟多级页表的每级页表页
PageTableEntry	保存页框、权限、COW 标志	不模拟 accessed/dirty 的硬件置位细节
FaultHandler	按 VMA/PTE/访问类型处理缺页	不模拟真实信号栈和架构异常帧

先省略一部分硬件细节，是为了让核心不变量清晰。等这些不变量稳定后，再加入 accessed/dirty、swap、file mapping、TLB shutdown 和 memcg，才不会把系统写成一团不可测代码。

类型和常量。

先用强类型别名和命名常量写清地址单位。所有有语义的数字都要命名。

```
#include <algorithm>
#include <cstdint>
#include <deque>
#include <limits>
#include <stdexcept>
#include <unordered_map>
#include <utility>
#include <vector>

using VirtualAddress = std::uint64_t;
using PageNumber = std::uint64_t;
using FrameNumber = std::uint64_t;

constexpr std::uint64_t VM_PAGE_SIZE = 4096;
constexpr VirtualAddress VM_USER_TOP = 0x0000800000000000ULL;
constexpr FrameNumber VM_INVALID_FRAME =
    std::numeric_limits<FrameNumber>::max();

enum class AccessKind {
    Read,
    Write,
    Execute,
};
```

地址计算要集中放在小函数里，避免到处手写除法和取模。

```
constexpr PageNumber page_number(VirtualAddress address) {
    return address / VM_PAGE_SIZE;
```

```

}

constexpr std::uint64_t page_offset(VirtualAddress address) {
    return address % VM_PAGE_SIZE;
}

constexpr VirtualAddress page_down(VirtualAddress address) {
    return address - page_offset(address);
}

constexpr VirtualAddress page_up(VirtualAddress address) {
    const auto offset = page_offset(address);
    if (offset == 0) {
        return address;
    }
    return address + (VM_PAGE_SIZE - offset);
}

```

权限和 VMA。

VMA 是区间策略，不是当前硬件映射。它说明地址是否属于进程、理论上允许什么访问、缺页时应该怎样补页。

```

struct VmPermissions {
    bool readable = false;
    bool writable = false;
    bool executable = false;
};

bool allows(const VmPermissions& permissions, AccessKind access) {
    switch (access) {
        case AccessKind::Read:
            return permissions.readable;
        case AccessKind::Write:
            return permissions.writable;
        case AccessKind::Execute:
            return permissions.executable;
    }
    throw std::logic_error("unknown access kind");
}

enum class VmaKind {
    Anonymous,
    FilePrivate,
    FileShared,
};

struct Vma {
    VirtualAddress start = 0;
    VirtualAddress end = 0;
    VmPermissions permissions;
    VmaKind kind = VmaKind::Anonymous;
    std::uint64_t file_offset = 0;
};

```

VMA 插入时必须检查页对齐、非空区间、用户地址范围和不重叠。教学模型可用有序 vector；查找是线性扫描，语义最直观。

```

class VmaTable {
public:

```

```

void insert(Vma vma) {
    this->validate_new_vma(vma);
    auto position = std::ranges::lower_bound(
        this->vmass_, vma.start, {}, &Vma::start);
    this->vmass_.insert(position, std::move(vma));
}

[[nodiscard]] const Vma* find(VirtualAddress address) const {
    for (const auto& vma : this->vmass_) {
        if (address < vma.start) {
            return nullptr;
        }
        if (vma.start <= address && address < vma.end) {
            return &vma;
        }
    }
    return nullptr;
}

private:
void validate_new_vma(const Vma& vma) const {
    if (vma.start >= vma.end) {
        throw std::invalid_argument("empty VMA");
    }
    if (page_offset(vma.start) != 0 || page_offset(vma.end) != 0) {
        throw std::invalid_argument("VMA must be page aligned");
    }
    if (vma.end > VM_USER_TOP) {
        throw std::invalid_argument("VMA outside user range");
    }
    for (const auto& existing : this->vmass_) {
        const bool disjoint = vma.end <= existing.start || existing.end <= vma.start;
        if (!disjoint) {
            throw std::invalid_argument("overlapping VMA");
        }
    }
}

std::vector<Vma> vmass_;
};

```

这段代码故意没有实现自动合并。教学阶段先让每次插入的边界完全显式；等测试覆盖充分后，再把相邻 VMA 合并作为优化加入。

物理页和引用计数。

物理内存模型用 frame number 标识页框，页内容用固定大小数组表示。为了节省书中篇幅，可以把内容简化为 byte vector；真实实现中应避免在热路径频繁分配。

```

struct PhysicalPage {
    std::vector<std::uint8_t> bytes = std::vector<std::uint8_t>(VM_PAGE_SIZE);
    std::uint32_t refcount = 0;
};

class PhysicalMemory {
public:
    FrameNumber allocate_zeroed() {
        FrameNumber frame = VM_INVALID_FRAME;
        if (!this->free_list.empty()) {

```



```

        frame = this->free_list_.front();
        this->free_list_.pop_front();
        auto& page = this->pages_.at(frame);
        std::ranges::fill(page.bytes, 0);
        page.refcount = 1;
        return frame;
    }

    frame = static_cast<FrameNumber>(this->pages_.size());
    PhysicalPage page;
    page.refcount = 1;
    this->pages_.push_back(std::move(page));
    return frame;
}

FrameNumber clone(FrameNumber source) {
    const FrameNumber target = this->allocate_zeroed();
    this->pages_.at(target).bytes = this->pages_.at(source).bytes;
    return target;
}

void retain(FrameNumber frame) {
    this->pages_.at(frame).refcount += 1;
}

void release(FrameNumber frame) {
    auto& page = this->pages_.at(frame);
    if (page.refcount == 0) {
        throw std::logic_error("double release of physical page");
    }
    page.refcount -= 1;
    if (page.refcount == 0) {
        this->free_list_.push_back(frame);
    }
}

[[nodiscard]] std::uint32_t refcount(FrameNumber frame) const {
    return this->pages_.at(frame).refcount;
}

private:
    std::vector<PhysicalPage> pages_;
    std::deque<FrameNumber> free_list_;
};

```

引用计数是 COW 的核心证据。每个测试结束时，物理页活跃数、引用计数和页表映射数要能对上，否则模型可能泄漏或重复释放。

PTE 和地址空间。

PTE 保存当前硬件可见映射。VMA 允许写不等于 PTE 当前可写；COW 页的 VMA 允许写，但 PTE 要暂时只读，让第一次写触发 fault。

```

struct PageTableEntry {
    FrameNumber frame = VM_INVALID_FRAME;
    VmPermissions permissions;
    bool present = false;
    bool cow = false;
};

class AddressSpace {

```

```

public:
    void map_vma(Vma vma) {
        this->vmass_.insert(vma);
    }

    [[nodiscard]] const Vma* find_vma(VirtualAddress address) const {
        return this->vmass_.find(address);
    }

    [[nodiscard]] PageTableEntry* find_pte(VirtualAddress address) {
        const auto iterator = this->page_table_.find(page_number(address));
        if (iterator == this->page_table_.end()) {
            return nullptr;
        }
        return &iterator->second;
    }

    void install_pte(VirtualAddress address, PageTableEntry pte) {
        this->page_table_[page_number(address)] = pte;
    }

private:
    VmaTable vmass_;
    std::unordered_map<PageNumber, PageTableEntry> page_table_;
};

```

这个页表是哈希表，不是多级页表。它足以表达“某个虚拟页当前是否 present、权限是什么、是否 COW”。后续实践卷再把它替换成四级页表或模拟 TLB。

缺页处理。

缺页处理先查 VMA，再查权限，再决定是新分配、COW 还是权限错误。返回结果要区分用户可见错误。

```

enum class FaultResult {
    Resolved,
    SegmentationFault,
    ProtectionFault,
};

class FaultHandler {
public:
    explicit FaultHandler(PhysicalMemory& memory) : memory_(memory) {}

    FaultResult handle(AddressSpace& space, VirtualAddress address, AccessKind
        access) {
        const Vma* vma = space.find_vma(address);
        if (vma == nullptr) {
            return FaultResult::SegmentationFault;
        }
        if (!allows(vma->permissions, access)) {
            return FaultResult::ProtectionFault;
        }

        PageTableEntry* pte = space.find_pte(address);
        if (pte != nullptr && pte->present) {
            return this->handle_present_pte(*pte, access);
        }

        PageTableEntry new_pte;
        new_pte.frame = this->memory_.allocate_zeroed();
    }
};

```

```

    new_pte.present = true;
    new_pte.permissions = vma->permissions;
    space.install_pte(address, new_pte);
    return FaultResult::Resolved;
}

private:
FaultResult handle_present_pte(PageTableEntry& pte, AccessKind access) {
    if (allows(pte.permissions, access)) {
        return FaultResult::Resolved;
    }
    if (access == AccessKind::Write && pte.cow) {
        return this->resolve_cow(pte);
    }
    return FaultResult::ProtectionFault;
}

FaultResult resolve_cow(PageTableEntry& pte) {
    if (this->memory_.refcount(pte.frame) == 1) {
        pte.permissions.writable = true;
        pte.cow = false;
        return FaultResult::Resolved;
    }

    const FrameNumber old_frame = pte.frame;
    const FrameNumber new_frame = this->memory_.clone(old_frame);
    this->memory_.release(old_frame);
    pte.frame = new_frame;
    pte.permissions.writable = true;
    pte.cow = false;
    return FaultResult::Resolved;
}

PhysicalMemory& memory_;
};

```

这段实现有一个清晰的不变量：COW 处理前，PTE 不可写；处理后，当前地址空间获得可写私有页；旧 frame 引用计数减少。若 refcount 为 1，不复制，只把权限改回可写。

fork 的 COW 准备。

fork 时复制页表，但共享物理页，把双方 PTE 改为只读 COW。VMA 不需要复制物理页，只要复制区间策略。

```

fork address space:
copy VMA table
for each present writable anonymous PTE:
    clear writable in parent PTE
    set cow in parent PTE
    create child PTE pointing to same frame
    clear writable in child PTE
    set cow in child PTE
    retain physical frame

```

教学代码实现 fork 时要特别注意父进程 PTE 也要改成只读。如果只改子进程，父进程写入不会 fault，会直接污染共享页。真实系统还要 flush 父进程对应 TLB，否则父 CPU 可能继续使用旧 writable 权限。

mprotect 的模型。

`mprotect` 至少要改 VMA 权限和已有 PTE 权限。教学模型可以先拒绝跨 VMA 的复杂修改，只允许整段 VMA 修改；实践卷再实现切分。

```
mprotect simplified:
  find exact VMA [start, end)
  verify new permissions do not exceed may permissions
  update VMA permissions
  for each PTE in range:
    pte.permissions = intersect(pte.permissions, new_permissions)
    if new permissions remove write:
      record need_tlb_flush
```

注意“扩大权限”不能只改 PTE，还要检查原始 `may` 权限；“缩小权限”不能只改 VMA，还要改已经 `present` 的 PTE。否则已有映射会继续按旧权限访问。

测试用例。

这个模型至少要有以下测试。

测试	步骤	期望
未映射地址	不建 VMA，访问地址	返回段错误结果
懒分配	建匿名 VMA，首次读	分配零页并 <code>resolved</code>
写只读 VMA	VMA 只读，执行写 <code>fault</code>	返回权限错误结果
COW refcount>1	父子共享页，子写	子获得新 frame，旧 frame refcount 减一
COW refcount=1	单独持有 COW 页后写	不复制，只恢复 <code>writable</code>
<code>mprotect</code> 收缩	RW 改 R 后写	写失败，旧权限不能生效
double release	对同一 frame release 两次	抛出逻辑错误或测试失败

测试要断言内部状态，而不是只断言返回值。例如 COW 测试必须检查父子 frame 是否不同、内容是否正确、旧 frame 引用计数是否符合预期。只有这样才能发现“看似成功但泄漏引用”的错误。

从教学模型到真实内核。

这个模型和真实内核差距很大，但差距是有方向的。

- 把哈希页表替换为多级页表，可学习页表页分配和页表遍历。
- 给 PTE 加 `accessed/dirty`，可学习回收和写回。
- 给 VMA 加文件对象和 `offset`，可学习 `mmap` 与 `page cache`。
- 给地址空间加 `active CPU` 集合，可学习 `TLB shutdown`。
- 给物理页加 `memcg`、`zone`、`order` 和 `migratetype`，可学习 `buddy`、`NUMA`、`compaction`。
- 给 `fault handler` 加可睡眠点和 `retry`，可学习真实缺页路径的并发。

不要一开始就实现所有功能。先让最小模型通过严格测试，再逐步加入复杂字段。操作系统工程的质量来自可维护的不变量，而不是一次性写出大量不可验证的机制。

第一阶段源码走读：x86-64 #PF 到 VMA、PTE 与 COW

前面已经讲过虚拟地址、VMA、PTE、TLB 和 COW，但只讲概念还不够。真正读懂操作系统，必须能把一次用户态 load/store 失败拆成硬件异常、入口现场、VMA 判定、PTE 状态、缺页修复、TLB 失效和返回重试。x86-64 上这条路径的核心异常是 #PF，Linux 里它从 `arch/x86/mm/fault.c` 进入，再落到 `mm/memory.c` 的通用内存管理逻辑。

核心思想

#PF 不是“程序访问错了内存”的同义词。它只是 CPU 告诉内核：“这次地址翻译或权限检查没有按当前页表完成。”内核必须结合 CR2、error code、当前上下文、VMA 和 PTE，才能决定是分配零页、读文件页、执行 COW、返回 `-EFAULT`，还是给进程发送信号。

硬件交给内核的最小证据。

x86-64 上发生 page fault 时，CPU 至少提供三类证据：faulting virtual address、错误码和被打断现场。faulting address 由 CR2 保存；错误码说明这是不是 present 页上的保护错误、是不是写访问、是不是用户态访问、是不是取指访问等；入口保存的寄存器现场说明 fault 发生在什么 RIP、什么 RSP、什么 CPL。

证据	典型来源	OS 需要回答的问题
fault address	CR2	这个地址属于哪个 VMA，是否 canonical
error code.P	page fault error code	是 PTE 不存在，还是 present 页权限拒绝
error code.W/R	page fault error code	是读、写还是取指触发
error code.U/S	page fault error code	是 Ring 3 访问，还是内核路径访问
trap frame	异常入口保存	返回后重试哪条指令，还是转成信号/错误

这张表解释了为什么“缺页处理”不能只写成 `if page missing then allocate`。同样是 #PF，缺少 PTE 可能是匿名懒分配，present 页写保护可能是 COW，用户态访问无 VMA 是 `SIGSEGV`，内核 `copy_from_user` 访问坏地址应该返回 `-EFAULT`，内核普通指针 fault 则可能是 oops。

源码地图：从 x86-64 异常到通用 mm。

阅读 Linux 时，先不要在整个源码树里搜索“page fault”到处跳。应该按层次看：

路径	角色	阅读重点
<code>arch/x86/mm/fault.c</code>	x86-64 page fault 入口	读取 CR2、解释 error code、区分用户/内核上下文
<code>arch/x86/entry/entry_64.S</code>	异常入口汇编	trap frame、寄存器保存、进入 C 函数前的状态
<code>include/linux/mm_types.h</code>	mm 核心结构	<code>mm_struct</code> 、 <code>vm_area_struct</code> 、页表相关对象
<code>mm/mmap.c</code>	VMA 创建与查找	<code>mmap</code> 、 <code>mprotect</code> 、VMA 合并拆分

mm/memory.c	通用 fault 主干	handle_mm_fault、匿名 fault、文件 fault、COW
mm/rmap.c	反向映射	页被哪些映射引用，回收和 COW 需要的关系
mm/filemap.c	文件页 fault	page cache、文件映射、读入文件页
arch/x86/mm/tlb.c	TLB 失效	本地 flush、远端 shutdown、PCID 相关路径

这条地图的分层很重要：`arch/x86/mm/fault.c` 负责架构相关入口，`mm/memory.c` 负责大部分架构无关语义。不要把二者混成一个函数理解。x86-64 入口知道 CR2 和 error code；通用 mm 知道 VMA、folio、匿名页、文件页、COW、回收和锁。

第一步：地址形式和 VMA 查找不是同一件事。

用户态传来的值先要满足 x86-64 canonical address 规则。非 canonical 地址不是“找不到 VMA”那么简单，它在页表遍历前就可以被硬件拒绝。通过这一关后，内核才用 fault address 查找当前进程的 VMA。

```
page fault classification:
address is not canonical:
    reject before treating it as a normal VMA miss
no VMA covers address:
    user instruction -> SIGSEGV
    kernel usercopy -> -EFAULT
VMA covers address but permissions reject access:
    protection failure, not demand allocation
VMA covers address and permissions allow access:
    inspect PTE and repair if possible
```

VMA 是虚拟地址区间的语义说明。它回答“这段地址是否应该存在，允许读写执行吗，来自匿名内存还是文件，私有还是共享”。PTE 是当前硬件翻译状态。一个合法 VMA 允许暂时没有 PTE；一个没有 VMA 的地址不能靠分配页修复。

第二步：error code 决定走缺失路径还是保护路径。

page fault error code 的 present/protection 位把路径分成两类。若不是 present 页，常见情况是懒分配、文件页未读入或 swap 页未换入。若是 present 页上的保护 fault，常见情况是 COW、写只读映射、用户态访问 supervisor 页或取指违反 NX。

硬件现象	内核可能动作	不能误判成什么
not-present 读 匿名页	分配零页，安装 PTE，重试指令	程序错误
not-present 读 文件页	查 page cache，必要时发起 I/O	简单 malloc
present 写只读 PTE	若 VMA 允许写且 PTE 是 COW，则复制或提升	普通权限失败
present 写只读 VMA	发送信号或返回错误	COW
present 取指 NX 页	发送信号或安全违规处理	数据页缺失

这里的关键是同时看 VMA 和 PTE。VMA 允许写、PTE 不可写，可能是 COW；VMA 本身不允许写，PTE 不可写就是权限拒绝。只看 PTE 会把只读文件映射误当成 COW；只看 VMA 会看不见硬件当前状态。

匿名懒分配：合法地址为什么第一次访问才有物理页。

`mmap` 或堆增长可以先建立 VMA，不立刻分配物理页。第一次读写触发 not-present #PF，内核分配一个清零页，安装 PTE，返回用户态重试 faulting instruction。用户程序感觉这条 load/store 成功了，但中间发生过一次异常和内核修复。

```
anonymous demand fault:
  find VMA covering fault address
  verify VMA permits the access
  allocate zero-filled physical page
  install user PTE with VMA-derived permissions
  return to the same user RIP
CPU retries the faulting load/store
```

这就是为什么 VIRT 可以很大而 RSS 很小。VMA 代表承诺，RSS 代表当前真的驻留的物理页。OS 用这个策略避免提前为大量可能永远不会访问的地址分配内存。

文件映射缺页：read 和 mmap 在 page cache 汇合。

文件 `mmap` 后，用户第一次读某个页面也会触发 not-present #PF。内核根据 VMA 找到文件和偏移，再到 page cache 查对应页。若 page cache 命中，这是 minor fault；若需要从存储设备读入，这是 major fault。读入后安装 PTE，用户指令重试。

```
file-backed fault:
  fault address -> VMA
  VMA file + page offset -> page-cache index
  if page cache hit:
    map cached folio and return
  else:
    submit storage read, wait for completion
    mark folio uptodate
    map folio and return
```

因此 `read` 和 `mmap` 共享的不只是“文件内容”，而是内核里的 page cache 对象。前者通常把 page cache 内容复制到用户 buffer；后者把 page cache 页映射进用户地址空间。理解这一点，后面讲 page cache 回收、dirty、writeback 和文件系统日志才不会断开。

COW 的完整条件：VMA 允许写，PTE 暂时禁止写。

fork 后父子共享匿名物理页。为了发现第一次写，父子 PTE 都要清掉 writable，并标记 COW。用户写入时，CPU 看到 present PTE 但不可写，于是产生保护型 #PF。内核确认 VMA 允许写、PTE 是 COW，再决定复制还是提升。

```
COW write fault:
  write to present but read-only PTE
  VMA says private mapping is writable
  PTE carries COW state
  if physical page refcount > 1:
    allocate new page and copy old content
    decrement old page refcount
    install writable PTE to new page
  else:
    only restore writable bit
    invalidate stale TLB translation
    return and retry the store
```

复制和提升的差别体现了 COW 的性能设计。若物理页仍被父子或其他映射共享，必须复制；若只剩当前映射，直接把 PTE 改回 writable 即可。无论哪种，都要处理 TLB，因为 CPU 可能还缓存着旧的只读或旧的可写翻译。

为什么 fork 时父进程也必须写保护。

常见误解是“fork 只要把子进程页表设成只读”。这是错的。父进程如果保留 writable PTE，它可以继续写共享物理页，子进程会直接看到父进程的新内容，COW 就失效了。正确做法是父子双方都写保护，并清理父进程可能已有的 writable TLB 项。

```
fork private pages:
  for each writable private PTE:
    increment physical page refcount
    clear parent writable bit
    mark parent PTE as COW
    install child read-only COW PTE
    invalidate parent stale writable TLB entry
```

这一步把“fork 很快”变成可理解的状态机：fork 不复制所有页，只复制页表项和引用计数；第一次写才复制物理页。代价是 fork 时要批量改 PTE 权限，并承担 TLB 失效成本。

TLB shutdown: COW 的隐藏正确性条件。

如果父进程在 fork 前刚写过某页，CPU TLB 可能缓存了 writable 翻译。fork 清掉父 PTE writable 后，若没有失效旧 TLB，父进程下一次写可能不触发 #PF，而是直接修改共享物理页。这不是性能问题，而是隔离正确性问题。

```
stale TLB failure:
  parent writes page, TLB caches writable PTE
  fork clears parent PTE writable but forgets TLB invalidate
  parent writes again through stale TLB
  child observes modified shared frame
  COW handler never runs
```

多核时问题更明显：同一个地址空间的线程可能在多个 CPU 上运行，远端 CPU 也可能缓存旧翻译。内核需要本地 TLB invalidate，也可能需要 IPI 触发远端 CPU 执行 shutdown。后面看到 mprotect、munmap、COW、页迁移性能问题时，要把这笔成本算进去。

用户指针 fault: 为什么有时是 -EFAULT 而不是信号。

同一个坏地址，在用户指令里访问和在系统调用参数复制时访问，结果不同。用户程序自己执行 `mov rax, qword ptr [rdi]` 访问坏地址，通常应得到信号。若用户把坏指针传给 `read` 或 `write`，内核在 `copy_to_user` 或 `copy_from_user` 中 fault，则系统调用应返回 `-EFAULT`。

```
; Intel syntax, user instruction fault example
mov rax, qword ptr [rdi]

; syscall usercopy fault example, conceptual
copy_from_user(kernel_buffer, user_pointer, length)
if page fault is not repairable:
  return -EFAULT
```

Linux 用异常表一类机制标记某些内核访问用户内存的指令：这里 fault 不代表内核坏了，而是用户参数不可访问，应跳到修复位置返回错误。没有这个区分，坏用户指针就会把内核打崩。

可观察证据：不要只看 maps。

调试缺页路径时，`/proc/<pid>/maps` 只能告诉你 VMA，不告诉你每页是否已经有 PTE、是否驻留、是否 dirty。需要把多个证据合起来看：

命令或文件	能看到什么	不能单独推出什么
<code>/proc/<pid>/maps</code>	VMA 区间、权限、文件映射	物理页是否已分配
<code>/proc/<pid>/smaps</code>	RSS、dirty、shared/private 统计	每一次 fault 的源码分支
<code>perf stat -e page-faults</code>	page fault 总量	major/minor 和 fault 原因细节
<code>perf stat -e minor-faults,</code> <code>major-faults</code>	缺页是否涉及 I/O	COW 与匿名零页的区别
<code>strace</code>	系统调用返回 <code>EFAULT</code> 等错误	用户指令自己的 page fault

真正的分析要能把现象落回路径。例如 VMA 存在但 RSS 没涨，说明还没实际触页；minor fault 增加但 major fault 不增，通常没有存储 I/O；fork 后父子写同一页 RSS 增加，可能是 COW 分裂；系统调用返回 `EFAULT`，说明坏用户指针被 `usercopy` 路径捕获。

配套 C++20 模型覆盖哪些失败。

下面的模型不是完整 Linux，也不模拟真实指令译码。它只把第一阶段最容易讲空的断言变成测试：

- 合法匿名 VMA 第一次访问会 minor fault，分配零页后重试成功。
- 只读 VMA 的写入不能被当成 COW，必须失败且不能偷偷分配页。
- fork 后父子共享页，第一次写触发 COW，父子内容分离。
- 如果 fork 时忘记失效父进程 writable TLB，父进程会绕过 COW 污染子进程。
- 文件映射第一次 fault 通过 page cache 建立映射，并记录 major fault。
- `syscall usercopy` 遇到坏用户指针返回失败，而不是模拟成内核崩溃。
- 非 canonical 地址不能被误讲成普通 VMA miss。

验收标准

第一阶段关于虚拟内存的最低验收：能解释一条用户态写指令为什么可能正常完成、minor fault 后完成、major fault 后完成、COW 后完成、返回 `-EFAULT`，或变成信号；并能指出每种结果依赖哪个 VMA/PTE/TLB 条件。

完整 C++20 模型源码。

```
#include <array>
#include <cstdint>
#include <cstdint>
#include <iostream>
#include <map>
#include <optional>
#include <stdexcept>
#include <string>
#include <string_view>
#include <unordered_map>
#include <vector>

using Byte = std::uint8_t;
using Word = std::uint64_t;
using VirtualAddress = std::uint64_t;
```

```

using PageNumber = std::uint64_t;
using Pcid = std::uint16_t;
using FrameId = std::uint32_t;

constexpr std::uint64_t X86_VIRTUAL_ADDRESS_BITS = 48U;
constexpr std::uint64_t X86_CANONICAL_SIGN_BIT =
    1ULL << (X86_VIRTUAL_ADDRESS_BITS - 1U);
constexpr VirtualAddress X86_CANONICAL_LOW_MASK =
    (1ULL << X86_VIRTUAL_ADDRESS_BITS) - 1ULL;
constexpr VirtualAddress X86_CANONICAL_HIGH_MASK = ~X86_CANONICAL_LOW_MASK;
constexpr std::uint64_t X86_PAGE_SHIFT = 12U;
constexpr std::size_t X86_PAGE_SIZE = 1ULL << X86_PAGE_SHIFT;
constexpr VirtualAddress X86_PAGE_OFFSET_MASK =
    static_cast<VirtualAddress>(X86_PAGE_SIZE - 1U);
constexpr Word X86_PF_PROTECTION_BIT = 1ULL << 0U;
constexpr Word X86_PF_WRITE_BIT = 1ULL << 1U;
constexpr Word X86_PF_USER_BIT = 1ULL << 2U;
constexpr std::size_t X86_FAULT_RETRY_LIMIT = 3U;
constexpr std::size_t HASH_COMBINE_SHIFT = 1U;

constexpr Pcid TEST_PARENT_PCID = 41U;
constexpr Pcid TEST_CHILD_PCID = 42U;
constexpr Pcid TEST_FILE_PCID = 43U;
constexpr VirtualAddress TEST_ANON_PAGE = 0x0000'0000'0040'0000ULL;
constexpr VirtualAddress TEST_ANON_VA = TEST_ANON_PAGE + 0x30ULL;
constexpr VirtualAddress TEST_READ_ONLY_PAGE = 0x0000'0000'0050'0000ULL;
constexpr VirtualAddress TEST_READ_ONLY_VA = TEST_READ_ONLY_PAGE + 0x20ULL;
constexpr VirtualAddress TEST_COW_PAGE = 0x0000'0000'0060'0000ULL;
constexpr VirtualAddress TEST_COW_VA = TEST_COW_PAGE + 0x10ULL;
constexpr VirtualAddress TEST_FILE_PAGE = 0x0000'0000'0070'0000ULL;
constexpr VirtualAddress TEST_FILE_VA = TEST_FILE_PAGE + 0x80ULL;
constexpr VirtualAddress TEST_BAD_USER_VA = 0x0000'0000'0080'0000ULL;
constexpr VirtualAddress TEST_NON_CANONICAL_VA = 0x0000'8000'0000'0000ULL;
constexpr std::uint64_t TEST_FILE_PAGE_INDEX = 9U;
constexpr std::size_t TEST_FILE_BYTE_OFFSET =
    static_cast<std::size_t>(TEST_FILE_VA & X86_PAGE_OFFSET_MASK);
constexpr Byte TEST_ZERO_BYTE = 0x00U;
constexpr Byte TEST_ANON_BYTE = 0x5AU;
constexpr Byte TEST_ORIGINAL_BYTE = 0x11U;
constexpr Byte TEST_PARENT_BYTE = 0x22U;
constexpr Byte TEST_CHILD_BYTE = 0x33U;
constexpr Byte TEST_FILE_BYTE = 0x7EU;

enum class AccessKind {
    Read,
    Write,
    Execute,
};

enum class VmaKind {
    Anonymous,
    FileBacked,
};

enum class FaultContext {
    UserInstruction,
    UserCopy,
};

enum class FaultDisposition {
    Handled,
    Signal,
    BadUserPointer,
};

enum class AccessFailureKind {
    SegmentationViolation,
    BadUserPointer,
    RetryLimitExceeded,
};

struct PageTableEntry {
    FrameId frame = 0;
    bool present = false;
    bool writable = false;
    bool user = true;
    bool executable = false;
    bool cow = false;
};

struct Translation {
    FrameId frame = 0;
    std::size_t offset = 0;
};

struct Vma {
    VirtualAddress start = 0;
    VirtualAddress end = 0;
    bool readable = false;
    bool writable = false;
    bool executable = false;
    bool private_mapping = true;
    VmaKind kind = VmaKind::Anonymous;
    std::uint64_t file_page_index = 0;
};

struct FaultStats {

```



```

    std::size_t minor_faults = 0;
    std::size_t major_faults = 0;
    std::size_t cow_copies = 0;
    std::size_t cow_promotions = 0;
    std::size_t signals = 0;
    std::size_t bad_user_pointers = 0;
};

struct PageCacheResult {
    FrameId frame = 0;
    bool hit = false;
};

struct TlbKey {
    Pcid pcid = 0;
    PageNumber vpn = 0;

    bool operator==(const TlbKey& other) const {
        return this->pcid == other.pcid && this->vpn == other.vpn;
    }
};

struct TlbKeyHash {
    std::size_t operator()(const TlbKey& key) const {
        const std::size_t pcid_hash = std::hash<Pcid>{}(key.pcid);
        const std::size_t vpn_hash = std::hash<PageNumber>{}(key.vpn);
        return pcid_hash ^ (vpn_hash << HASH_COMBINE_SHIFT);
    }
};

bool is_canonical(VirtualAddress address) {
    const bool sign_bit_set = (address & X86_CANONICAL_SIGN_BIT) != 0;
    const VirtualAddress high_bits = address & X86_CANONICAL_HIGH_MASK;
    if (sign_bit_set) {
        return high_bits == X86_CANONICAL_HIGH_MASK;
    }
    return high_bits == 0;
}

VirtualAddress page_base(VirtualAddress address) {
    return address & ~X86_PAGE_OFFSET_MASK;
}

PageNumber virtual_page_number(VirtualAddress address) {
    return address >> X86_PAGE_SHIFT;
}

std::size_t page_offset(VirtualAddress address) {
    return static_cast<std::size_t>(address & X86_PAGE_OFFSET_MASK);
}

void require(bool condition, std::string_view message) {
    if (!condition) {
        throw std::runtime_error(std::string(message));
    }
}

class PageFault : public std::runtime_error {
public:
    PageFault(VirtualAddress fault_address, AccessKind fault_access,
              Word fault_error_code, std::string_view message)
        : std::runtime_error(std::string(message)),
          address_(fault_address),
          access_(fault_access),
          error_code_(fault_error_code) {}

    VirtualAddress address() const {
        return this->address_;
    }

    AccessKind access() const {
        return this->access_;
    }

    Word error_code() const {
        return this->error_code_;
    }

private:
    VirtualAddress address_ = 0;
    AccessKind access_ = AccessKind::Read;
    Word error_code_ = 0;
};

class AccessFailure : public std::runtime_error {
public:
    AccessFailure(AccessFailureKind failure_kind, std::string_view message)
        : std::runtime_error(std::string(message)), kind_(failure_kind) {}

    AccessFailureKind kind() const {
        return this->kind_;
    }

private:
    AccessFailureKind kind_ = AccessFailureKind::SegmentationViolation;
};

```

```

template <typename Callable>
void require_access_failure(AccessFailureKind expected, Callable action) {
    try {
        action();
    } catch (const AccessFailure& failure) {
        require(failure.kind() == expected, "unexpected access failure kind");
        return;
    }
    throw std::runtime_error("expected access failure was not raised");
}

class PhysicalMemory {
public:
    FrameId allocate_zero_frame() {
        const FrameId frame = static_cast<FrameId>(this->frames_.size());
        this->frames_.push_back(Frame{});
        this->refcounts_.push_back(1U);
        return frame;
    }

    FrameId allocate_copy_frame(FrameId source) {
        const FrameId frame = this->allocate_zero_frame();
        this->frames_.at(frame) = this->frames_.at(source);
        return frame;
    }

    void increment_ref(FrameId frame) {
        ++this->refcounts_.at(frame);
    }

    void decrement_ref(FrameId frame) {
        std::size_t& refcount = this->refcounts_.at(frame);
        require(refcount > 0, "physical frame refcount underflow");
        --refcount;
    }

    std::size_t refcount(FrameId frame) const {
        return this->refcounts_.at(frame);
    }

    Byte load8(FrameId frame, std::size_t offset) const {
        require(offset < X86_PAGE_SIZE, "load offset must stay inside page");
        return this->frames_.at(frame).bytes.at(offset);
    }

    void store8(FrameId frame, std::size_t offset, Byte value) {
        require(offset < X86_PAGE_SIZE, "store offset must stay inside page");
        this->frames_.at(frame).bytes.at(offset) = value;
    }

    std::size_t frame_count() const {
        return this->frames_.size();
    }

private:
    struct Frame {
        std::array<Byte, X86_PAGE_SIZE> bytes{};
    };

    std::vector<Frame> frames_;
    std::vector<std::size_t> refcounts_;
};

class PageCache {
public:
    PageCacheResult get_or_read_page(PhysicalMemory& memory,
                                     std::uint64_t file_page_index) {
        const auto iterator = this->pages_.find(file_page_index);
        if (iterator != this->pages_.end()) {
            return PageCacheResult{.frame = iterator->second, .hit = true};
        }

        const FrameId frame = memory.allocate_zero_frame();
        memory.store8(frame, TEST_FILE_BYTE_OFFSET, TEST_FILE_BYTE);
        this->pages_[file_page_index] = frame;
        return PageCacheResult{.frame = frame, .hit = false};
    }

    std::size_t size() const {
        return this->pages_.size();
    }

private:
    std::map<std::uint64_t, FrameId> pages_;
};

class Tlb {
public:
    std::optional<PageTableEntry> lookup(Pcid pcid, PageNumber vpn) const {
        const auto iterator = this->entries_.find(TlbKey{.pcid = pcid, .vpn = vpn});
        if (iterator == this->entries_.end()) {
            return std::nullopt;
        }
        return iterator->second;
    }

    void insert(Pcid pcid, PageNumber vpn, const PageTableEntry& entry) {

```

```

    this->entries_[TlbKey{.pcid = pcid, .vpn = vpn}] = entry;
}

void invalidate(Pcid pcid, PageNumber vpn) {
    this->entries_.erase(TlbKey{.pcid = pcid, .vpn = vpn});
}

std::size_t size() const {
    return this->entries_.size();
}

private:
    std::unordered_map<TlbKey, PageTableEntry, TlbKeyHash> entries_;
};

class AddressSpace {
public:
    explicit AddressSpace(Pcid pcid) : pcid_(pcid) {}

    Pcid pcid() const {
        return this->pcid_;
    }

    void add_vma(const Vma& vma) {
        require(is_canonical(vma.start), "VMA start must be canonical");
        require(is_canonical(vma.end - 1U), "VMA end must be canonical");
        require(vma.start < vma.end, "VMA must not be empty");
        this->vmass_.push_back(vma);
    }

    const Vma* find_vma(VirtualAddress address) const {
        for (const Vma& vma : this->vmass_) {
            if (address >= vma.start && address < vma.end) {
                return &vma;
            }
        }
        return nullptr;
    }

    void map_page(VirtualAddress virtual_page, const PageTableEntry& entry) {
        require((virtual_page & X86_PAGE_OFFSET_MASK) == 0,
            "mapped virtual address must be page aligned");
        this->entries_[virtual_page_number(virtual_page)] = entry;
    }

    const PageTableEntry* find_pte(PageNumber vpn) const {
        const auto iterator = this->entries_.find(vpn);
        if (iterator == this->entries_.end()) {
            return nullptr;
        }
        return &iterator->second;
    }

    PageTableEntry* find_pte_for_mutation(PageNumber vpn) {
        const auto iterator = this->entries_.find(vpn);
        if (iterator == this->entries_.end()) {
            return nullptr;
        }
        return &iterator->second;
    }

    const std::vector<Vma>& vmass() const {
        return this->vmass_;
    }

    const std::map<PageNumber, PageTableEntry>& entries() const {
        return this->entries_;
    }

    std::map<PageNumber, PageTableEntry>& entries_for_mutation() {
        return this->entries_;
    }

    std::size_t mapped_page_count() const {
        return this->entries_.size();
    }

private:
    Pcid pcid_ = 0;
    std::vector<Vma> vmass_;
    std::map<PageNumber, PageTableEntry> entries_;
};

class Mmu {
public:
    Translation translate(const AddressSpace& address_space, Tlb& tlb,
        VirtualAddress virtual_address, AccessKind access) const {
        if (!is_canonical(virtual_address)) {
            throw PageFault(
                virtual_address, access, this->error_code(access, false),
                "x86-64 rejected non-canonical virtual address before VMA lookup");
        }

        const PageNumber vpn = virtual_page_number(virtual_address);
        const std::optional<PageTableEntry> cached = tlb.lookup(address_space.pcid(), vpn);
        if (cached.has_value()) {
            this->check_entry(virtual_address, access, cached.value());
        }
    }

```

```

    return Translation{.frame = cached->frame, .offset = page_offset(virtual_address));
}

const PageTableEntry* walked = address_space.find_pte(vpn);
if (walked == nullptr || !walked->present) {
    throw PageFault(virtual_address, access, this->error_code(access, false),
        "page walk found no present PTE");
}
this->check_entry(virtual_address, access, *walked);
tlb.insert(address_space.pcid(), vpn, *walked);
return Translation{.frame = walked->frame, .offset = page_offset(virtual_address));
}

private:
void check_entry(VirtualAddress address, AccessKind access,
    const PageTableEntry& entry) const {
    if (!entry.user) {
        throw PageFault(address, access, this->error_code(access, true),
            "ring 3 access reached supervisor PTE");
    }
    if (access == AccessKind::Write && !entry.writable) {
        throw PageFault(address, access, this->error_code(access, true),
            "write access rejected by PTE");
    }
    if (access == AccessKind::Execute && !entry.executable) {
        throw PageFault(address, access, this->error_code(access, true),
            "execute access rejected by PTE");
    }
}

Word error_code(AccessKind access, bool protection) const {
    Word code = X86_PF_USER_BIT;
    if (protection) {
        code |= X86_PF_PROTECTION_BIT;
    }
    if (access == AccessKind::Write) {
        code |= X86_PF_WRITE_BIT;
    }
    return code;
}
};

class KernelMemoryManager {
public:
    KernelMemoryManager(PhysicalMemory& memory, PageCache& page_cache, Tlb& tlb)
        : memory_(memory), page_cache_(page_cache), tlb_(tlb) {}

    const FaultStats& stats() const {
        return this->stats_;
    }

    FaultDisposition handle_page_fault(AddressSpace& address_space,
        const PageFault& fault,
        FaultContext context) {
        const Vma* vma = address_space.find_vma(fault.address());
        if (vma == nullptr || !this->vma_allows(*vma, fault.access())) {
            return this->reject_fault(context);
        }

        const PageNumber vpn = virtual_page_number(fault.address());
        PageTableEntry* pte = address_space.find_pte_for_mutation(vpn);
        const bool protection_fault =
            (fault.error_code() & X86_PF_PROTECTION_BIT) != 0;
        if (!protection_fault) {
            return this->handle_missing_pte(address_space, *vma, vpn);
        }
        if (fault.access() == AccessKind::Write && pte != nullptr && pte->cow) {
            return this->handle_cow_fault(address_space, vpn, *pte);
        }
        return this->reject_fault(context);
    }

    void fork_private(AddressSpace& parent, AddressSpace& child,
        bool invalidate_parent_tlb) {
        for (const Vma& vma : parent.vmas()) {
            child.add_vma(vma);
        }

        for (auto& [vpn, entry] : parent.entries_for_mutation()) {
            PageTableEntry child_entry = entry;
            const VirtualAddress virtual_page = vpn << X86_PAGE_SHIFT;
            const Vma* vma = parent.find_vma(virtual_page);
            if (entry.present && vma != nullptr) {
                this->memory_.increment_ref(entry.frame);
                if (vma->private_mapping && vma->writable) {
                    entry.writable = false;
                    entry.cow = true;
                    child_entry.writable = false;
                    child_entry.cow = true;
                    if (invalidate_parent_tlb) {
                        this->tlb_.invalidate(parent.pcid(), vpn);
                    }
                }
            }
        }
        child.map_page(virtual_page, child_entry);
    }
}

```

```

private:
bool vma_allows(const Vma& vma, AccessKind access) const {
    if (access == AccessKind::Read) {
        return vma.readable;
    }
    if (access == AccessKind::Write) {
        return vma.writable;
    }
    return vma.executable;
}

FaultDisposition reject_fault(FaultContext context) {
    if (context == FaultContext::UserCopy) {
        ++this->stats_.bad_user_pointers;
        return FaultDisposition::BadUserPointer;
    }
    ++this->stats_.signals;
    return FaultDisposition::Signal;
}

FaultDisposition handle_missing_pte(AddressSpace& address_space, const Vma& vma,
                                   PageNumber vpn) {
    if (vma.kind == VmaKind::Anonymous) {
        const FrameId frame = this->memory_.allocate_zero_frame();
        address_space.map_page(
            vpn << X86_PAGE_SHIFT,
            PageTableEntry{
                .frame = frame,
                .present = true,
                .writable = vma.writable,
                .user = true,
                .executable = vma.executable,
                .cow = false,
            });
        ++this->stats_.minor_faults;
        return FaultDisposition::Handled;
    }

    PageCacheResult result =
        this->page_cache_.get_or_read_page(this->memory_, vma.file_page_index);
    this->memory_.increment_ref(result.frame);
    address_space.map_page(
        vpn << X86_PAGE_SHIFT,
        PageTableEntry{
            .frame = result.frame,
            .present = true,
            .writable = vma.writable && !vma.private_mapping,
            .user = true,
            .executable = vma.executable,
            .cow = false,
        });
    if (result.hit) {
        ++this->stats_.minor_faults;
    } else {
        ++this->stats_.major_faults;
    }
    return FaultDisposition::Handled;
}

FaultDisposition handle_cow_fault(AddressSpace& address_space, PageNumber vpn,
                                  PageTableEntry& pte) {
    if (this->memory_.refcount(pte.frame) == 1U) {
        pte.writable = true;
        pte.cow = false;
        this->tlb_.invalidate(address_space.pcid(), vpn);
        ++this->stats_.cow_promotions;
        return FaultDisposition::Handled;
    }

    const FrameId old_frame = pte.frame;
    const FrameId new_frame = this->memory_.allocate_copy_frame(old_frame);
    this->memory_.decrement_ref(old_frame);
    pte.frame = new_frame;
    pte.writable = true;
    pte.cow = false;
    this->tlb_.invalidate(address_space.pcid(), vpn);
    ++this->stats_.cow_copies;
    return FaultDisposition::Handled;
}

PhysicalMemory& memory_;
PageCache& page_cache_;
Tlb& tlb_;
FaultStats stats_;
};

class UserCpu {
public:
    UserCpu(const Mmu& mmu, PhysicalMemory& memory, Tlb& tlb,
            KernelMemoryManager& kernel)
        : mmu_(mmu), memory_(memory), tlb_(tlb), kernel_(kernel) {}

    Byte load8(AddressSpace& address_space, VirtualAddress virtual_address) {
        for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
            try {
                const Translation translation =

```



```

        this->mmu_.translate(address_space, this->tlb_, virtual_address,
                             AccessKind::Read);
    } return this->memory_.load8(translation.frame, translation.offset);
} catch (const PageFault& fault) {
    this->handle_or_throw(address_space, fault, FaultContext::UserInstruction);
}
}
throw AccessFailure(AccessFailureKind::RetryLimitExceeded,
                    "load exceeded page fault retry limit");
}

void store8(AddressSpace& address_space, VirtualAddress virtual_address,
            Byte value) {
    for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
        try {
            const Translation translation =
                this->mmu_.translate(address_space, this->tlb_, virtual_address,
                                     AccessKind::Write);
            this->memory_.store8(translation.frame, translation.offset, value);
            return;
        } catch (const PageFault& fault) {
            this->handle_or_throw(address_space, fault, FaultContext::UserInstruction);
        }
    }
    throw AccessFailure(AccessFailureKind::RetryLimitExceeded,
                        "store exceeded page fault retry limit");
}

bool copy_from_user(AddressSpace& address_space, VirtualAddress virtual_address,
                   Byte& output) {
    for (std::size_t attempt = 0; attempt < X86_FAULT_RETRY_LIMIT; ++attempt) {
        try {
            const Translation translation =
                this->mmu_.translate(address_space, this->tlb_, virtual_address,
                                     AccessKind::Read);
            output = this->memory_.load8(translation.frame, translation.offset);
            return true;
        } catch (const PageFault& fault) {
            const FaultDisposition disposition =
                this->kernel_.handle_page_fault(address_space, fault,
                                                FaultContext::UserCopy);
            if (disposition == FaultDisposition::Handled) {
                continue;
            }
            return false;
        }
    }
    return false;
}

private:
void handle_or_throw(AddressSpace& address_space, const PageFault& fault,
                    FaultContext context) {
    const FaultDisposition disposition =
        this->kernel_.handle_page_fault(address_space, fault, context);
    if (disposition == FaultDisposition::Signal) {
        throw AccessFailure(AccessFailureKind::SegmentationViolation,
                            "page fault became a user signal");
    }
    if (disposition == FaultDisposition::BadUserPointer) {
        throw AccessFailure(AccessFailureKind::BadUserPointer,
                            "page fault became a bad user pointer");
    }
}

const Mmu& mmu_;
PhysicalMemory& memory_;
Tlb& tlb_;
KernelMemoryManager& kernel_;
};

Vma anonymous_vma(VirtualAddress page, bool writable) {
    return Vma{
        .start = page,
        .end = page + X86_PAGE_SIZE,
        .readable = true,
        .writable = writable,
        .executable = false,
        .private_mapping = true,
        .kind = VmaKind::Anonymous,
        .file_page_index = 0,
    };
}

Vma file_vma(VirtualAddress page) {
    return Vma{
        .start = page,
        .end = page + X86_PAGE_SIZE,
        .readable = true,
        .writable = false,
        .executable = false,
        .private_mapping = true,
        .kind = VmaKind::FileBacked,
        .file_page_index = TEST_FILE_PAGE_INDEX,
    };
}

```

```

struct TestMachine {
    PhysicalMemory memory;
    PageCache page_cache;
    Tlb tlb;
    Mmu mmu;
    KernelMemoryManager kernel;
    UserCpu cpu;

    TestMachine()
        : memory(),
          page_cache(),
          tlb(),
          mmu(),
          kernel(memory, page_cache, tlb),
          cpu(mmu, memory, tlb, kernel) {}
};

void test_anonymous_fault_allocates_and_retries() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_ANON_PAGE, true));

    require(process.mapped_page_count() == 0U,
        "anonymous VMA should not allocate a PTE before access");
    require(machine.cpu.load8(process, TEST_ANON_VA) == TEST_ZERO_BYTE,
        "anonymous demand fault should return a zero-filled byte");
    machine.cpu.store8(process, TEST_ANON_VA, TEST_ANON_BYTE);
    require(machine.cpu.load8(process, TEST_ANON_VA) == TEST_ANON_BYTE,
        "anonymous page did not preserve user store");
    require(machine.kernel.stats().minor_faults == 1U,
        "anonymous demand paging should be a minor fault in this model");
}

void test_vma_permission_rejects_write() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_READ_ONLY_PAGE, false));

    require_access_failure(AccessFailureKind::SegmentationViolation, [&machine,
        &process] {
        machine.cpu.store8(process, TEST_READ_ONLY_VA, TEST_ANON_BYTE);
    });
    require(machine.kernel.stats().signals == 1U,
        "write to read-only VMA should become a user-visible failure");
    require(process.mapped_page_count() == 0U,
        "permission failure must not allocate a page");
}

void test_cow_split_preserves_parent_child_isolation() {
    TestMachine machine;
    AddressSpace parent(TEST_PARENT_PCID);
    parent.add_vma(anonymous_vma(TEST_COW_PAGE, true));

    machine.cpu.store8(parent, TEST_COW_VA, TEST_ORIGINAL_BYTE);
    const FrameId shared_frame =
        parent.find_pte(virtual_page_number(TEST_COW_VA))>->frame;
    AddressSpace child(TEST_CHILD_PCID);
    machine.kernel.fork_private(parent, child, true);

    require(machine.memory.refcount(shared_frame) == 2U,
        "fork should share the original frame before either process writes");
    machine.cpu.store8(parent, TEST_COW_VA, TEST_PARENT_BYTE);
    require(machine.cpu.load8(parent, TEST_COW_VA) == TEST_PARENT_BYTE,
        "parent COW write did not install private data");
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_ORIGINAL_BYTE,
        "child should still see the pre-COW byte");
    require(machine.kernel.stats().cow_copies == 1U,
        "shared COW page should require one copy");

    machine.cpu.store8(child, TEST_COW_VA, TEST_CHILD_BYTE);
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_CHILD_BYTE,
        "child COW promotion did not make the child page writable");
    require(machine.kernel.stats().cow_promotions == 1U,
        "last private COW reference should be promoted without copying");
}

void test_missing_tlb_shutdown_breaks_cow() {
    TestMachine machine;
    AddressSpace parent(TEST_PARENT_PCID);
    parent.add_vma(anonymous_vma(TEST_COW_PAGE, true));

    machine.cpu.store8(parent, TEST_COW_VA, TEST_ORIGINAL_BYTE);
    AddressSpace child(TEST_CHILD_PCID);
    machine.kernel.fork_private(parent, child, false);

    machine.cpu.store8(parent, TEST_COW_VA, TEST_PARENT_BYTE);
    require(machine.cpu.load8(child, TEST_COW_VA) == TEST_PARENT_BYTE,
        "without shutdown, stale writable TLB should corrupt the shared page");
    require(machine.kernel.stats().cow_copies == 0U,
        "stale TLB write should bypass the COW handler entirely");
}

void test_file_fault_and_bad_user_pointer() {
    TestMachine machine;
    AddressSpace process(TEST_FILE_PCID);
    process.add_vma(file_vma(TEST_FILE_PAGE));
}

```

```

require(machine.cpu.load8(process, TEST_FILE_VA) == TEST_FILE_BYTE,
    "file-backed fault did not load page-cache content");
require(machine.kernel.stats().major_faults == 1U,
    "first file-backed page should model a major fault");
require(machine.page_cache.size() == 1U,
    "file fault should populate exactly one page-cache entry");

Byte copied = TEST_ZERO_BYTE;
require(!machine.cpu.copy_from_user(process, TEST_BAD_USER_VA, copied),
    "bad user pointer should be reported to the syscall layer");
require(machine.kernel.stats().bad_user_pointers == 1U,
    "bad user pointer statistic was not updated");
}

void test_non_canonical_address_is_not_a_vma_miss() {
    TestMachine machine;
    AddressSpace process(TEST_PARENT_PCID);
    process.add_vma(anonymous_vma(TEST_ANON_PAGE, true));

    require(!is_canonical(TEST_NON_CANONICAL_VA),
        "test address must be non-canonical");
    require_access_failure(AccessFailureKind::SegmentationViolation, [&machine,
                                                                    &process] {
        static_cast<void*>(machine.cpu.load8(process, TEST_NON_CANONICAL_VA));
    });
}

int main() {
    test_anonymous_fault_allocates_and_retries();
    test_vma_permission_rejects_write();
    test_cow_split_preserves_parent_child_isolation();
    test_missing_tlb_shutdown_breaks_cow();
    test_file_fault_and_bad_user_pointer();
    test_non_canonical_address_is_not_a_vma_miss();
    std::cout << "x86-64 page fault and COW model checks passed\n";
}

```

尚未解决的问题。匿名页已经有独立映射、复制和回收规则，但同一文件区间若被 `read`、`write` 与 `mmap` 分别读入，会出现重复页面和相互矛盾的可见值。下一章为文件内容建立可共享的内存身份。

第 7 章 为什么文件内容需要可共享的内存身份

继承的机器。 VMA 表达地址区间策略，PTE 表达当前映射，物理页保存实际字节。文件内容还没有统一的缓存身份：按描述符读取与按地址缺页可能为同一文件偏移建立两份页面，共享写、私有复制、截断和回写因而无法遵循同一套失效规则。

文件内容如何进入地址空间

`read` 和 `write` 把文件内容显式复制到用户缓冲区，而 `mmap` 把文件区间映射到进程地址空间，让普通 `load/store` 触发文件页面的调入和回写。它看起来像内存功能，实际是虚拟内存、文件系统和 `page cache` 的共同契约：VMA 记录虚拟地址区间；`page cache` 记录文件偏移到物理页的缓存；缺页处理把两者连接起来；写回系统决定脏页何时落盘。

先写一个常见误解：

```
addr = mmap(file, length)
# wrong mental model:
# kernel reads the whole file into memory now
# addr points to that private copy
```

这个模型会立刻解释不了三个现象。第一，映射一个很大的文件可以很快返回，因为内核通常只建立 VMA，不会立刻把所有页面读入内存。第二，多个进程映射同一个文件区间，读到的可能是同一份 `page cache` 页，而不是每个进程一份私有拷贝。第三，另一个进程把文件 `truncate` 后，当前进程访问原来映射的越界部分可能得到 `SIGBUS`，因为虚拟地址仍在 VMA 里，但背后的文件页已经不存在。

正确过渡是：

```
mmap:
  create VMA [addr, addr + length)
  remember file and file offset
  do not install every PTE yet

first load from addr + x:
  page fault
  find VMA
  compute file page index
  find or load page cache page
  install PTE
  resume user instruction
```

所以 `mmap` 的核心不是“少一次拷贝”这一句话，而是把文件偏移延迟绑定到虚拟地址访问上。这个延迟绑定带来性能，也带来错误边界：缺页可能等待 I/O，写入可能变成 COW 或 `dirty page`，持久化要靠 `msync/fsync` 推进，文件大小变化可能让合法 VMA 里的访问失败。

用一个小文件把状态走完整。文件 `data.bin` 有 9000 字节，页大小 4096。进程执行：

```
fd = open("data.bin", O_RDWR);
p = mmap(NULL, 8192, PROT_READ | PROT_WRITE,
          MAP_SHARED, fd, 0);
x = p[5000];
```

`mmap` 返回时，常见状态并不是“两个页都已经在内存里”。更准确的账本是：

对象	刚 <code>mmap</code> 后	第一次读 <code>p[5000]</code> 后
VMA	记录虚拟区间、文件、offset 0、共享可写	不变
PTE	两个虚拟页通常都还没有 present 映射	第二个虚拟页 present，指向 page cache 页
page cache	可能没有对应页，也可能已有别的进程留下的页	(inode, index=1) 有缓存页
文件系统 I/O	通常没有立即读 8192 字节	若 cache miss，提交读第 1 个文件页
用户指令	还没执行 load	fault 返回后重新执行 load，读到字节

这里的 `p[5000]` 落在映射的第二个虚拟页，因为 $5000 / 4096 = 1$ 。缺页处理先用 fault 地址找到 VMA，再把 `addr - vma.start` 换算成文件页索引 1，然后到 inode 的 page cache 中查 `index=1`。若缓存命中，内核直接安装 PTE；若缓存未命中，内核分配一个 locked page，把它先放入 page cache，再发起磁盘读。读完成前 faulting 线程睡眠；完成后解锁页面，安装 PTE，回到用户态重新执行那条 load。注意是重新执行 faulting 指令，不是从下一行继续，因为第一次 load 还没有真正完成。

再看边界。映射长度是 8192，只覆盖文件前两个页；文件实际有 9000 字节，所以这两个页都在文件范围内。若改成映射 16384 字节，再访问第三页的一部分，VMA 可能仍然覆盖该地址，但文件没有对应内容，内核不能凭空制造稳定文件页，应该按系统语义触发 `SIGBUS` 或等价错误。这也是 `SIGSEGV` 和 `SIGBUS` 的区别：前者常表示地址不属于合法映射或权限不对；后者常表示地址形式上属于映射，但背后的文件对象不能提供该页。

为什么需要 `mmap`？第一，它减少系统调用和复制，适合随机访问大文件。第二，它让多个进程共享同一份文件页，天然支持共享库和共享内存。第三，它把文件 I/O 纳入页表权限和缺页机制，统一了“按需加载”。但它也带来更复杂的一致性：一个进程通过 `write` 修改文件，另一个进程通过 `mmap` 读同一区间，看到的应是同一份 page cache 语义，而不是两个互相独立的缓存。

核心思想

`mmap` 不是把文件一次性读进内存，而是建立“虚拟地址 -> 文件偏移 -> page cache 页面”的延迟绑定。

`MAP_SHARED` 和 `MAP_PRIVATE` 是理解文件映射的分水岭。共享映射的写入会把 page cache 页面标脏，之后由 `msync`、`fsync` 或后台写回落盘；私有映射初始可共享文件页，但第一次写入触发 COW，产生匿名页，文件本身不被修改。共享库代码段通常是只读文件映射；进程堆和栈通常是匿名映射；数据库会谨慎选择 `mmap` 或 direct I/O，因为错误的写回假设会破坏延迟和持久化边界。

文件缺页、共享写入与私有复制

一个教学 `mmap` 需要以下对象：

对象	保存什么	不变量
VMA	虚拟区间、权限、flags、file、offset	VMA 不重叠，权限不超过文件打开模式

page table entry	虚拟页到物理页、权限、dirty/accessed	PTE 权限不能超过 VMA 权限
page cache page	inode、文件页索引、物理页、dirty、refcount	同一 inode+index 对应一个缓存页
anon page	私有 COW 后的页面	不再回写到文件
writeback item	脏页、目标块、提交状态	写回完成前页面不能被当成干净页回收

文件映射缺页路径如下：

```
handle_page_fault(addr, access):
    vma = find_vma(current.mm, addr)
    if vma == null or access not allowed by vma:
        send_sigsegv()
    page_index = vma.file_offset_pages + ((addr - vma.start) / PAGE_SIZE)
    page = page_cache_get_or_load(vma.file, page_index)
    if access == WRITE and vma.private:
        page = copy_page(page)
        mark_anon(page)
    install_pte(addr, page, pte_prot(vma, access))
```

`mmap` 实现中最重要的错误路径是长度、偏移和权限检查。映射长度为零、地址未对齐、偏移未按页对齐、写共享映射但 `fd` 没有写权限、文件系统不支持映射、区间越过用户地址空间，都必须明确返回错误。不要把所有失败都压成一个 `EINVAL`，否则测试和调试会失去边界信息。

页缓存索引、预读与写回

VMA 查找

最简单的 VMA 管理可以用按起始地址排序的数组或链表，查找复杂度为 $O(n)$ 。这对教学模型足够，但进程有大量映射时会拖慢 page fault。真实内核会使用树结构，Linux 近年使用 `maple tree` 管理 VMA，以降低查找、插入和遍历成本。

```
find_vma(mm, addr):
    for vma in mm.vmas_by_start:
        if vma.start <= addr and addr < vma.end:
            return vma
        if addr < vma.start:
            return null
    return null
```

插入 VMA 时必须检查相邻区间是否可合并。若权限、`flags`、`file` 和连续 `offset` 一致，就可以合并，减少 VMA 数量。若频繁切分和合并出错，会导致 `munmap` 后仍然能访问、或合法区间被错误 `SIGSEGV`。

page cache 索引

page cache 的键通常是 `(inode, page_index)`。这个键保证 `read/write` 与 `mmap` 共享同一页：

```
page_cache_get_or_load(file, index):
    key = (file.inode, index)
    lock(page_cache.lock)
    page = radix_tree_lookup(page_cache, key)
    if page != null:
        page.refcount++
    unlock(page_cache.lock)
```

```

    return page
page = allocate_page_locked()
radix_tree_insert(page_cache, key, page)
unlock(page_cache.lock)
read_file_page(file, index, page)
unlock_page(page)
return page

```

这里先插入 locked page，再执行 I/O，是为了阻止多个线程同时读取同一文件页。后来的缺页者发现页面存在但 locked，就等待页面解锁。否则并发 page fault 会造成重复 I/O，甚至最后一个完成者覆盖前一个完成者的状态。

shared 与 private 写故障

写故障要区分三种情况：VMA 不允许写，私有映射需要 COW，共享映射需要让 PTE 可写并跟踪脏页。

```

handle_write_fault(vma, addr, old_page):
    if not vma.writable:
        send_sigsegv()
    if vma.private:
        new_page = allocate_page()
        copy(new_page, old_page)
        install_pte(addr, new_page, WRITE | USER)
        put_page(old_page)
        return
    mark_page_dirty(old_page)
    install_pte(addr, old_page, WRITE | USER | SHARED)

```

COW 的复杂度是一次页面复制 $O(\text{PAGE_SIZE})$ 。它用写时成本换取 fork/exec 和私有映射的低启动成本。共享映射的复杂性则在持久化：PTE dirty、page dirty、文件系统 journal dirty 和块设备队列不是同一件事。用户调用 `msync(MS_SYNC)` 期望脏页写回并等待完成，但这仍不必然等价于包含目录项的业务提交，后者需要文件系统和上层协议共同保证。

继续用 `data.bin`。两个进程 A 和 B 都用 `MAP_SHARED` 映射文件第一页。A 执行：

```
p[10] = 'X';
```

如果 A 的 PTE 当前是只读但 VMA 允许写，CPU 会产生写权限 fault。内核确认这是共享可写映射后，可以把同一份 page cache 页装成可写 PTE，并在合适的时机把页面标脏。此后 B 通过 `read(fd, buf, ...)` 或自己的共享映射读同一文件页，看到的可能就是 page cache 中已修改的内容。此时状态应分层看：

层次	p[10]='X' 之后发生了什么
CPU/PTE	A 的页表项允许写；硬件 dirty 位可能被置位
page cache	(inode,0) 对应页内容已变，页面需要写回
文件系统	还未必分配或提交所有元数据事务
块层/设备	还未必收到写请求，更未必完成持久化
应用语义	写了内存，不等于业务事务已经提交

这就是很多 mmap 程序出错的地方：它们看到另一个进程能读到新字节，就以为数据已经“保存到磁盘”。实际上那只是 page cache 可见性，不是掉电后的持久性。若随后机器掉电，而 dirty page 还没写回，磁盘上的文件可能仍是旧内容。若应用需要“写完记录后断电也能恢复”，至少要推进到 `msync(MS_SYNC)` 或 `fsync`，并且还要考虑文件长度、目录项、日志或 manifest 的提交顺序。

`MAP_PRIVATE` 的同一行代码完全不同。A 第一次写时，内核复制一页，把 A 的 PTE 指向匿名页；page cache 仍保存文件原始内容，B 也仍看到原始内容。此时“写入成功”只对 A 的进程地址空间成立，不对文件成立。私有映射不是“共享映射但晚点落盘”，而是写入后脱离文件页的匿名内存。

回收与写回

内存压力下，干净文件页可以直接回收，因为它们能从文件重新读入；脏文件页必须先写回；匿名页如果没有 swap，就不能随意丢弃。

```
reclaim_page(page):
    if page.refcount != 0 or page.locked:
        return BUSY
    if page.file_backed and not page.dirty:
        remove_from_page_cache(page)
        free(page)
        return OK
    if page.file_backed and page.dirty:
        enqueue_writeback(page)
        return RETRY_LATER
    if page.anon and swap_available:
        write_to_swap(page)
        free(page)
        return OK
    return BUSY
```

这个算法说明 page cache 不只是性能缓存，也是内存回收策略的一部分。错误地回收 dirty page 会丢数据；错误地保留所有文件页会让匿名内存被挤爆。

截断、映射失效与一致性

1. 基本映射：映射文件第一页，读取字节应等于文件内容。
2. 懒加载：`mmap` 后不访问不应触发磁盘读，第一次访问才缺页。
3. 权限：只读 fd 不能建立可写 `MAP_SHARED`。
4. 私有写：`MAP_PRIVATE` 写入后文件内容不变。
5. 共享写：`MAP_SHARED` 写入后 `read` 同区间能看到 page cache 更新。
6. `munmap`：解除映射后访问同地址触发 SIGSEGV 或等价错误。
7. 截断：文件被 `truncate` 后访问越界映射要触发 SIGBUS 或操作系统定义的文件映射错误。
8. 回写：脏页经过 `msync` 后，模拟块设备能观察到写请求。

验收标准

完成本章后，读者应能画出 VMA、PTE、page cache 与 inode 的关系，解释 private/shared 映射的写故障差异，并区分 `msync`、`fsync` 和业务提交的边界。

文件映射与持久化语义的衔接

文件映射、page cache 和写回最终都要服务恢复语义。普通应用常把“别的进程现在能读到新字节”误认为“断电后仍能读到新字节”；存储系统则必须把每一步写成可恢复协议。主线里先抓住四层提交：内存可见、page cache 变脏、块设备完成、文件系统和目录项持久。业务提交通常还要在这些层之上，再写 manifest、日志或版本号。

边界	已经保证什么	仍未保证什么
<code>memcpy</code> 到映射区	当前进程地址空间已改变	文件页未必写回，私有映射不改文件
其他进程读到新字节	共享 page cache 可见	掉电后未必存在
<code>msync(MS_SYNC)</code>	指定映射脏页写回并等待结果	目录项、rename、其他文件未必提交
<code>fsync(file)</code>	文件数据和必要元数据推进到持久层	多文件事务顺序未必正确
<code>rename + fsync(dir)</code>	目录项切换可恢复	应用协议仍要能识别旧版/新版

可靠更新通常需要单调证据。先写新数据并同步，再写临时 manifest 并同步，随后 `rename` 替换正式 manifest，最后同步目录。崩溃恢复时，程序只相信通过校验和、长度、版本号或事务记录证明完整的版本。若先更新 manifest 再同步数据，恢复程序可能看到“版本 42 已提交”，但数据页还停在旧内容。持久化错误最危险的地方是正常测试很难暴露，只有把崩溃点插在每个提交边界之后，才能证明恢复算法不会相信半成品。

直接 I/O、DAX 与同步边界

do_mmap 到 mmap_region

Linux 的 `mmap` 路径会从系统调用参数进入 `do_mmap`，经过权限、地址选择、长度和 flags 校验后，调用 `mmap_region` 建立或合并 VMA。文件映射还会调用 `file->f_op->mmap`，让具体文件系统或设备设置 `vm_ops`。

```
sys_mmap
-> ksys_mmap_pgoff
-> vm_mmap_pgoff
-> do_mmap
-> get_unmapped_area
-> mmap_region
-> file->f_op->mmap
```

VMA 的权限与行为由一组标志共同表达。`VM_READ/WRITE/EXEC/SHARED/MAYWRITE/LOCKED/HUGEPAGE` 等状态共同决定缺页处理、权限修改、锁页、回收和写回行为，不能只根据映射时传入的一个保护参数推断后续结果。

address_space 与 XArray

page cache 不挂在 fd 上，而挂在文件的 `address_space` 上。一个 inode 的 `i_mapping` 用 XArray 以页索引为 key 保存缓存页或 folio。这样不同进程、不同 fd、read/write/mmap 都能找到同一份文件页。

```
filemap_fault(vmf):
mapping = vmf.vma.file.f_mapping
index = offset_to_page_index(vmf.address)
folio = xa_load(mapping.i_pages, index)
if folio missing:
    folio = page_cache_sync_readahead(...)
    read_folio_from_filesystem(...)
install_pte(vmf.address, folio)
```

XArray 替代较早的 radix tree，支持稀疏索引、并发查找和特殊 entry。理解

page cache 时不必先掌握 XArray 的全部内部细节，但必须知道它的索引来自文件偏移，而不是进程虚拟地址。

readahead 与访问模式

顺序读时，内核会预读后续页；随机读时，过度预读会浪费 I/O 和内存。应用可用 `madvise` 给出提示：`MADV_SEQUENTIAL`、`MADV_RANDOM`、`MADV_DONTNEED`、`MADV_HUGEPAGE`。

```
on_page_cache_miss(file, index):
    if sequential_pattern_detected(file):
        submit_readahead(index + 1, window)
    else:
        read_single_page(index)
```

预读失败案例：数据库随机读大文件，错误地被当成顺序流，page cache 被无用页污染；或者媒体顺序读取未触发预读，设备队列深度上不去。性能报告要注明访问模式和缓存状态。

writeback 与错误范围

脏页写回由 `flusher` 线程、内存压力、`fsync/msync` 等触发。Linux 用 `writeback_control` 描述写回目标，用 `address_space` 记录写回错误。一个 `write` 成功后发生的异步写回错误，可能在之后的 `fsync` 才报告。

```
fsync(file):
    filemap_fdatawrite(mapping)
    filemap_fdatawait(mapping)
    err = file_check_and_advance_wb_err(file)
    if err:
        return err
    return filesystem_fsync_metadata(file)
```

错误范围跟踪很重要。否则一个进程造成的写回错误可能被另一个不相关进程消费，或者永远没人知道。可靠程序必须检查 `fsync` 返回值。

O_DIRECT 和 DAX

`O_DIRECT` 尝试绕过 page cache，把用户页直接用于块 I/O。它通常要求地址、长度和文件偏移对齐设备或文件系统要求。绕过 page cache 不等于绕过一致性：若同一文件同时被 buffered I/O 和 direct I/O 访问，内核必须同步或失效相关缓存页。

DAX 面向持久内存，允许文件直接映射到持久介质地址，跳过 page cache。这样 load/store 可直接访问持久介质，但持久化顺序、cache flush 和失败原子性更需要应用和文件系统谨慎处理。

模式	优点	风险
buffered I/O	page cache、预读、合并、易用	双重缓存、fsync 边界易误解
<code>O_DIRECT</code>	减少缓存污染，数据库可控	对齐、短 I/O、混用一致性复杂
DAX	跳过 page cache，低延迟持久内存	cache flush、持久顺序、硬件依赖强

截断、SIGBUS 与映射失效

文件映射后，文件可能被另一个进程 truncate。若映射区间超过新的文件大小，访问超出部分通常触发 `SIGBUS`，因为虚拟地址属于 VMA，但背后的文件页不再存在。这和访问完全未映射地址的 `SIGSEGV` 不同。

```

file_truncate(inode, new_size):
    invalidate page cache pages beyond new_size
    unmap mappings beyond new_size or mark invalid
    wake waiters

fault_mapped_file(addr):
    if page_index beyond inode.size:
        signal(SIGBUS)

```

这个边界常被忽略。mmap 程序不能假设映射后文件大小永远不变，尤其是共享文件、日志文件和被管理进程替换的文件。

msync、fsync 和 fdatasync

msync 面向映射区间，把 dirty mapped pages 写回；**fsync** 面向文件，等待文件数据和必要元数据持久；**fdatasync** 通常只保证读取文件数据所需的元数据。它们重叠但不等价。

接口	语义重点
msync	映射页写回，可按地址范围控制
fsync	文件数据和必要元数据持久化
fdatasync	文件数据和读取必需元数据，不一定同步所有时间戳
syncfs	某挂载文件系统的脏数据同步

数据库和存储服务必须明确使用哪个边界。只调用 **msync** 某段映射，不一定同步目录项或业务 manifest；只调用 **fsync** 文件，不一定说明应用的多文件事务已提交。

用一个索引文件更新看清楚持久化边界。程序维护两个文件：**data.dat** 保存记录，**manifest** 保存“当前有效数据文件和版本”。它用 mmap 修改 **data.dat** 的某一页，然后更新 **manifest** 指向新版本：

```

memcpy(mapped_data + off, record, len);
msync(mapped_data + page_start, PAGE_SIZE, MS_SYNC);
write(manifest_fd, "version=42\n", 11);
fsync(manifest_fd);

```

这段看起来已经很谨慎，但还要继续问：manifest 文件是覆盖写还是 rename 新文件？目录项是否 fsync？data 文件长度是否已经扩展并持久？如果 **record** 跨两页，只 msync 了一页会怎样？如果 **msync** 成功但 **fsync(manifest)** 失败，恢复时应该相信哪个版本？持久化不是“调用一个同步函数”就完了，而是要把恢复算法需要的证据按顺序写到介质上。

动作	它推进了哪层状态	仍未保证什么
写 mmap 内存	改了 page cache 或私有匿名页	未必到块设备
msync(MS_SYNC) 数据页	等待指定映射脏页写回	未必同步目录项或其他文件
fsync(data_fd)	推进 data 文件数据和必要元数据	不代表 manifest 已提交
rename(tmp, manifest)	原子切换目录项视图	目录项持久化仍要看目录 fsync

<code>fsync(dirfd)</code>	推进目录项更新	不修复上层协议顺序错误
---------------------------	---------	-------------

可靠更新通常要让恢复路径有单调证据。例如先把新数据页写入并同步，再写临时 manifest，`fsync` 临时文件，`rename` 替换正式 manifest，最后 `fsync` 目录。崩溃后要么看到旧 manifest 指向旧数据，要么看到新 manifest 指向已经同步过的新数据。若顺序反过来，先提交 manifest 再写 data，恢复时可能读到“版本 42 已提交”但数据页仍是旧内容。这里的核心不是某个固定配方，而是每一步都要说明：崩溃发生在这一行之后，恢复程序会看到哪些对象状态。

userfaultfd 和用户态缺页处理

`userfaultfd` 允许用户态接管某些缺页事件，用于虚拟机迁移、按需加载和用户态分页。内核把 fault 事件交给用户态管理进程，后者填充页面并唤醒 faulting 线程。

```
fault on registered range:
  kernel queues userfault event
  faulting thread sleeps
  pager reads event
  pager copies page data into address
  pager wakes faulting thread
```

这个机制说明 page fault 不只属于内核内部，也可以成为用户态系统软件的接口。但它要求严格权限和资源控制，否则恶意 pager 可以让线程永久睡眠或制造内存压力。

一次共享页实验：同一文件偏移为何只能有一个缓存身份

设文件 F 的 inode 编号为 901，页大小为 4096 字节。进程 A 通过 `read` 读取偏移 8192，进程 B 把同一文件映射到 `0x70000000`，随后读取地址 `0x70002064`。两个操作都需要文件页索引 2 中偏移 100 的字节。如果两条路径分别分配页面，A 可能看到旧内容，B 却看到新内容；回写和截断也不知道该处理哪一份副本。

```
// A 的描述符路径和 B 的地址路径必须汇合到同一缓存键。
cache_key = (inode 901, page_index 2)
A: pread(fd, buffer, 200, 8192)
B: load byte at 0x70002064
```

因此，缓存身份不能属于 fd，也不能属于 VMA。fd 是一次打开关系，VMA 是一段地址策略；两者寿命都可能短于缓存页。稳定键应由文件身份和文件内页索引组成，代表性实现把它保存在 inode 对应的 `address_space` 索引中。

对象	代表字段	责任与非责任	主要保护方式
<code>address_space</code>	host inode、页索引结构、回写操作、错误序号	确定同一文件偏移的缓存身份；不保存某进程的虚拟地址	索引锁、页锁和引用
缓存页或 folio	index、uptodate、dirty、writeback、mapping、引用计数	保存当前缓存字节和状态；不决定映射到哪个用户地址	页锁、状态位与引用
VMA	起止地址、文件引用、文件偏移、权限、共享/私有策略	把地址故障翻译成文件页索引；不拥有公共缓存页内容	地址空间锁与 VMA 引用

PTE	物理页号、读写权限、present、dirty 等硬件可见状态	保存当前地址翻译；不替代文件缓存索引和持久化状态	页表锁与失效协议
-----	---------------------------------	--------------------------	----------

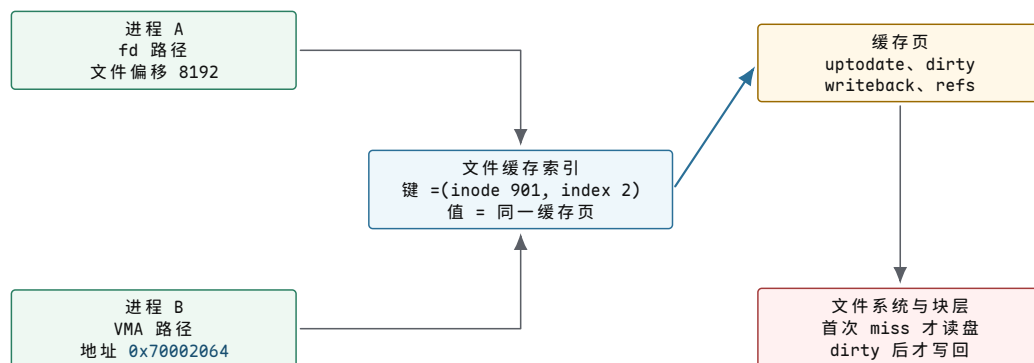
第一次缺页怎样发布缓存页。

B 的地址减去 VMA 起点得到映射内偏移，再加 VMA 保存的文件起始偏移，得到文件偏移 8292。向下取整得到页索引 2。若索引中没有页面，缺页处理程序先分配一个尚未公开的页面，以“锁定、内容未就绪”的状态插入索引，然后发起读取。其他执行流查到同一页面时等待它，而不是再次建立副本。

```
file_fault(vma, address):
    // 地址策略先给出文件页索引；此时还没有当前 PTE。
    offset = vma.file_offset + (address - vma.start)
    index = offset / PAGE_SIZE

    // 插入者发布唯一身份，未完成读取前保持页面 locked。
    page, created = lookup_or_create_locked(vma.file.mapping, index)
    if created:
        submit_read(page)
        wait_until_unlocked(page)

    // uptodate 是建立用户映射的前置条件，不能把 I/O 错误当成零页。
    require page.uptodate
    install_pte(address, page, permissions_from(vma))
```

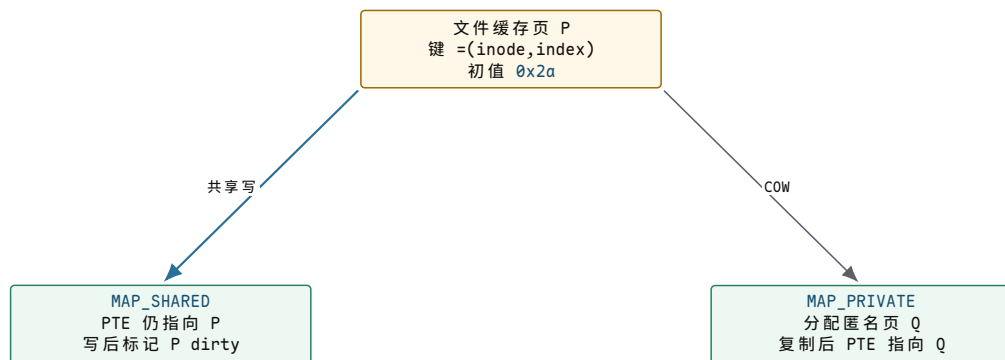


图示：描述符读取和映射缺页采用不同入口，却用相同的文件身份与页索引查找缓存。VMA 和 PTE 可以有多个，公共缓存身份仍只有一份。

共享写和私有写在首次写故障处分开。

MAP_SHARED 的写入目标仍是文件缓存页。写权限建立前，内核要确认映射允许写，对页面加锁，等待可能存在的写回或建立适当并发协议，随后标脏。PTE 的 dirty 位说明处理器发生过写入，缓存页的 dirty 状态说明文件系统需要安排回写；两者位于不同层，不能互相替代。

MAP_PRIVATE 则只共享最初读取的字节。第一次写故障分配匿名页，把旧内容复制过去，再让当前 PTE 指向匿名页。文件缓存页不因此变脏，其他进程也不会看到这次私有修改。



图示：两种映射可以从同一只读缓存页开始。共享写改变公共文件页并进入回写协议；私有写建立匿名身份，之后不再由文件回写负责。

回写完成不等于应用事务完成。

缓存页从 clean 变为 dirty，表示内存中的文件版本领先于已知持久版本；进入 writeback 后，设备请求拥有正在传输的字节范围；完成回调清除 writeback，并根据结果更新错误序号。普通后台回写完成不自动向某个调用者形成持久化承诺。只有同步接口圈定范围、等待相关写回并取得错误结果，调用者才能判断本次要求是否满足。

```
sync_file_range(mapping, first, last):
    // 先把范围内的 dirty 页转交给写回请求，避免只等待旧请求。
    submit_dirty_pages(mapping, first, last)
    wait_for_writeback(mapping, first, last)

    // 错误序号让每个打开对象判断自己尚未观察过的新错误。
    error = sample_writeback_error(mapping)
    return error
```

这里仍应区分“缓存页不再 dirty”“设备完成写请求”和“介质在故障模型下保持结果”。前两项属于本章可追踪状态，最后一项还需要第十七章的刷新、顺序和恢复协议。

truncate 与缺页并发为何需要失效协议。

若另一个进程把 F 截断到 4096 字节，页索引 2 已越出新文件长度。仅更新 inode 的长度不够：旧缓存页、已有 PTE 和正在进行的缺页都可能继续暴露被删除范围。截断路径必须先发布新长度，阻止或序列化新的越界映射，再撤销 PTE、移除缓存索引并等待已有引用退出。

```
truncate_file(inode, new_size):
    // 新长度先成为边界判定依据，后续 fault 不得重新发布越界页。
    lock(inode.mapping_invalidation)
    inode.size = new_size
    unmap_user_ptes_beyond(new_size)
    remove_cache_pages_beyond(new_size)
    unlock(inode.mapping_invalidation)
```

交错位置	若没有协调会发生什么	正确收束	用户结果
fault 已算出 index 2，尚未插入页	truncate 后又重新发布越界缓存页	插入前复查长度或持有失效序列	映射访问得到 SIGBUS
页已映射，truncate 更新长度	处理器继续通过旧 PTE 读取删除内容	先撤销越界 PTE，再移除缓存页	后续访问重新故障并失败

页正在 writeback	删除范围与设备写请求 寿命冲突	等待、取消或让完成回 调识别失效代次	不让迟到完 成恢复已删 除身份
------------------	--------------------	-----------------------	-----------------------

本章复核：地址、缓存身份与持久身份

一个用户地址首先由 VMA 解释成文件偏移，再由 inode 和页索引找到缓存身份，最后由 PTE 建立当前物理映射。地址可以变化，PTE 可以失效，多个进程可以同时引用；公共缓存键仍然稳定。共享写、私有写、回写和截断之所以能够放进同一章，是因为它们都在改变或终止这条身份关系。

练习 7.1 追踪同一文件页的两个入口

A 使用 `pread` 读取偏移 12288，B 的 VMA 从文件偏移 8192 开始，B 访问映射内偏移 4096。说明两条路径使用的缓存键、VMA 和 PTE 是否相同。

解答 7.1 追踪同一文件页的两个入口

两者都到达文件偏移 12288，即页索引 3，因此缓存键相同。A 不需要 B 的 VMA；B 有自己的 VMA 和 PTE。缓存身份相同不意味着地址策略或当前页表项相同。

练习 7.2 区分共享写与私有写

两个进程最初都映射同一缓存页。一个进程执行私有写后，列出文件缓存页、匿名页、两个 PTE 和 dirty 状态的变化。

解答 7.2 区分共享写与私有写

写入者获得复制出的匿名页，其 PTE 改指匿名页并允许写；另一个 PTE 仍指文件缓存页。文件缓存页内容和 dirty 状态不因私有写改变，匿名页的脏状态由匿名内存回收规则管理。

练习 7.3 解释截断后的迟到完成

页索引 5 正在写回时文件被截断到只保留页索引 0。为什么设备完成回调不能无条件把索引 5 重新标为有效？说明需要比较或保护的身份信息。

解答 7.3 解释截断后的迟到完成

请求只证明旧页面的一次传输结束，不证明该文件偏移仍存在。完成路径必须受映射失效协议约束，或携带可识别旧代次的页面引用；它只能结束旧请求和报告错误，不能重新把已从索引移除的页面发布为当前缓存身份。

尚未解决的问题。用户页与文件缓存页已有身份和回收路径，内核自身仍不断申请 task、VMA、inode、请求描述符等不同大小的短寿命对象。下一章从最小的物理页分配开始，逐步处理连续性、碎片、上下文限制和对象复用。

第 8 章 为什么内核还需要自己的分配器

继承的机器。地址空间与页缓存已经明确页面的策略、映射和归属。内核不能把每个几十字节的对象都永久占用一整页，也不能在持锁或中断上下文中任意等待回收；分配动作必须同时表达大小、连续性、可等待性、归属和失败策略。

内核分配面对的对象与上下文

虚拟内存章节讲页表和地址空间，本章看内核自己怎样管理内存。内核需要分配页表页、进程结构、文件对象、socket buffer、目录项缓存、驱动 DMA buffer、日志记录和临时缓冲区。这些对象大小不同、生命周期不同、对齐要求不同，有些可以睡眠等待内存，有些在中断上下文不能睡眠。

先设想内核只提供“分配若干完整物理页”这一种操作。页表页和大块连续缓冲区能够直接使用，但一个只有数百字节的文件对象也会独占整页；成千上万个小对象会把大部分内存留在页内空白中。如果反过来只提供任意字节大小的通用堆，虽然小对象浪费较少，却很难在任意时刻取得满足页对齐、物理连续和特定内存区域要求的大块空间。长期的任意切分还会使空闲区逐渐零散。

因此，内核必须把两个不同问题分开。底层先负责“哪些完整物理页可用，以及怎样取得连续的 2^k 个页”；上层再负责“怎样把已经取得的页切成大量同类型对象，并快速复用这些对象槽”。前一层通常由 buddy allocator 承担，后一层通常由 slab 或 SLUB 承担；通用小对象接口再把不同请求大小映射到若干对象类别。专用的 task、inode 和 dentry 缓存位于更上层。

这种分层不是术语上的分类，而是因为单一分配器无法同时高效处理连续页、大量小对象、对象初始化、缓存局部性和调试信息。后文先推导页级拆分与合并，再说明对象缓存为何建立在页分配器之上。

从物理页到内核对象的分配过程

页框为什么还要分区管理

即使所有物理页大小相同，它们也不一定能够互换。有些设备只能访问特定地址范围；某些页已经被内核固定，不能迁移；多处理器系统中，访问本地内存与远端内存的代价也可能不同。如果分配器只维护一条全局空闲页链表，低地址页可能被普通请求提前耗尽，真正受地址限制的设备随后即使看到大量空闲页也无法工作；所有 CPU 还会争用同一把链表锁。

因此，操作系统先把可分配页按可达范围和拓扑归入不同区域，再在每个区域内维护空闲状态。分配请求不仅给出页数，还可能给出允许使用的区域、能否等待、能否回收以及是否允许移动现有页面。所谓 zone、NUMA node 和迁移类型，分别是在表达这些限制；它们不是对物理内存重复编号，而是分配决策所需的策略边界。

约束	若忽略会发生什么	分配器需要保存的状态
地址可达范围	受限设备取得无法访问的页	页所属区域和请求允许区域
物理连续性	大页或设备缓冲无法建立	各阶连续空闲块
可移动性	内存压缩无法腾出连续区间	movable、reclaimable、unmovable 分类
处理器局部性	频繁访问落到远端节点	节点归属、回退顺序和负载

执行上下文	中断路径进入睡眠或递归回收	是否允许等待、I/O 与文件系统回收
-------	---------------	--------------------

连续页为何使用二次幂分组

设一个区域只有 64 页，当前全部空闲。若把每一种可能长度的连续区间都登记起来，区间彼此重叠，一次分配会使大量记录失效。若只逐页登记，请求 8 个连续页时又要扫描并验证相邻关系。一种更容易维护的折中是只登记长度为 1, 2, 4, 8, ... 页且满足相应对齐的空闲块。

现在请求 8 页。分配器从 64 页块开始，先拆成两个 32 页块，保留其中一个；再把另一个拆成两个 16 页块；最后拆成两个 8 页块并返回一个。每次拆分产生的另一半与返回方向中的块具有确定关系，释放时可以通过块号计算出它的配对块。这种成对拆分与合并关系就是 buddy 名称的来源。

```
64 pages free
-> 32 used for search + 32 free
-> 16 used for search + 16 free + 32 free
-> 8 allocated + 8 free + 16 free + 32 free

free the allocated 8 pages:
8 + its free 8-page buddy -> 16
16 + its free 16-page buddy -> 32
32 + its free 32-page buddy -> 64
```

合并成立有两个条件：配对块当前必须空闲，而且必须处在同一阶。一个相邻的 8 页区间如果属于另一个 16 页父块，就不是当前块的 buddy，不能仅凭地址相邻进行合并。这个规则使分配和释放不需要扫描所有空闲区间，却也规定了请求大小要向上取整到二次幂。

若请求 9 页，分配器必须返回 16 页，其中 7 页暂时不能给其他请求使用，这称为内部碎片。若系统总计有 32 个空闲页，但它们分散成许多不相邻单页，请求 8 个连续页仍然会失败，这称为外部碎片。总空闲量只能说明容量，不能说明最大连续块；这正是高阶分配失败时不能只看剩余内存百分比的原因。

小对象为何从已分配页中复用

接着考虑大小为 256 字节的 inode 内存对象。在 4096 字节页中，扣除少量管理信息后可以排列多个同样大小的槽位。第一次请求需要向页分配器取得一页并建立槽位表，后续请求只需从空闲槽链表取出一个；释放对象也只归还槽位。当整页所有槽位都空闲时，该页才有机会退回页分配器。

这层对象缓存不仅节约空间，还可以保留类型信息。分配器知道该缓存中的槽位都将承载同一种对象，因而能够统一处理对齐、构造、统计、红区与释放后填充值。缓存通常把 slab 分成三类：仍有空槽的 partial slab、已经用满的 full slab，以及完全空闲的 empty slab。分配优先使用 partial slab，避免在许多页上各留下少量无法归还的对象。

对象释放后不能立刻假定其所有引用都消失。引用计数、RCU 宽限期、设备完成和异步工作都可能延长真实生命周期。分配器只负责槽位何时可以复用；对象所属子系统必须先证明再没有合法访问者。将“从全局索引移除”和“把内存交回分配器”混成一步，是释放后使用错误的重要来源。

页框区域、连续性与碎片

实践先写对象分类表：

对象	分配来源	约束
页表页	buddy order-0 页	页对齐，通常不可移动

DMA buffer	连续物理页或 IOMMU 映射	设备可访问，对齐和边界限制
task 结构	slab cache	高频分配释放，需要初始化
socket buffer	slab + page fragment	网络吞吐和生命周期复杂
日志缓冲	kmalloc 或专用池	失败路径也要可用

然后为每类对象写：是否可睡眠、是否可回收、是否可移动、是否需要清零、是否可能从中断上下文分配。

buddy、slab 与分配上下文

buddy allocator

buddy allocator 把空闲内存按 order 管理，order=k 表示 2^k 个连续页。分配时找最小足够 order；没有就拆更大的块。释放时尝试和 buddy 合并。

```
alloc(order):
    for k in order..MAX_ORDER:
        if free[k] not empty:
            block = free[k].pop()
            while k > order:
                k -= 1
                buddy = split(block, k)
                free[k].push(buddy)
            return block
    return null

free(block, order):
    while order < MAX_ORDER:
        buddy = find_buddy(block, order)
        if buddy not free: break
        remove buddy from free[order]
        block = merge(block, buddy)
        order += 1
    free[order].push(block)
```

buddy 的优势是能管理连续页，释放时能合并。问题是内部碎片：请求 9 页必须分配 16 页。长期运行后，高 order 连续块也可能稀缺。

slab cache

slab 分配器为固定大小对象建立缓存。一个 slab 通常由一页或多页组成，切成多个对象槽。分配就是从 freelist 取一个槽；释放就是放回 freelist。对象构造可以在 slab 创建时完成，减少重复初始化成本。

```
slab_alloc(cache):
    if cache.partial not empty:
        obj = pop_free_object(cache.partial.first)
        return obj
    slab = allocate_new_slab(cache)
    cache.partial.push(slab)
    return pop_free_object(slab)
```

slab 的分配释放接近 $O(1)$ 。它适合大量同类型对象，例如 inode、dentry、task。实现要处理 per-CPU cache，以减少多核锁竞争；还要处理调试模式下的红区、poison、double free 检测。

kmalloc 和大小类别

通用 `kmalloc` 通常把请求大小映射到一组 `size class`，例如 32、64、128、256 字节。每个 `size class` 背后是一个 `slab cache`。这样可以用固定对象缓存服务变长请求。

```
kmalloc(size):
    class = size_to_class(size)
    return slab_alloc(kmalloc_cache[class])
```

`size class` 的代价是内部碎片。请求 129 字节可能分到 256 字节槽。分配器设计要在碎片、速度和缓存局部性之间折中。

GFP 标志和上下文

内核分配 API 通常带标志，说明是否允许睡眠、是否允许 I/O 回收、是否要求 DMA 区域等。中断上下文不能睡眠，所以只能使用不会阻塞的分配；普通进程上下文可以等待回收。

```
if in_interrupt():
    page = alloc_page(GFP_ATOMIC)
else:
    page = alloc_page(GFP_KERNEL)
```

这解释了为什么内核代码不能随便分配内存。分配失败不是理论问题，尤其在中断、持锁、内存压力和回收路径中。高质量代码必须定义失败策略：返回错误、丢包、使用预留池、降级还是 `panic`。

per-CPU 缓存

多核系统中，所有小对象分配都争同一把 `slab` 锁会很慢。`per-CPU cache` 让每个 CPU 维护少量本地空闲对象，常见分配释放不需要全局锁。

```
kmem_cache_alloc(cache):
    cpu_cache = cache.percpu[current_cpu()]
    if cpu_cache.freelist not empty:
        return cpu_cache.freelist.pop()
    refill_from_global(cache, cpu_cache)
    return cpu_cache.freelist.pop()

kmem_cache_free(cache, obj):
    cpu_cache = cache.percpu[current_cpu()]
    cpu_cache.freelist.push(obj)
    if cpu_cache.count > HIGH_WATERMARK:
        drain_to_global(cache, cpu_cache)
```

`per-CPU` 缓存降低锁竞争，但会增加内存滞留：每个 CPU 都可能囤积一批对象。高质量实现要有高低水位、CPU 下线 `drain`、内存压力回收和统计对账。

内存调试与防御

分配器是许多内核漏洞的放大器。越界写、`use-after-free`、`double free` 和未初始化读取都可能破坏任意内核对象。调试构建应加入红区、`poison`、隔离队列和调用栈记录。

```
debug_free(obj):
    if obj.magic != LIVE_MAGIC:
        panic("double free or corrupted object")
    fill(obj.memory, POISON_FREE)
    obj.magic = FREE_MAGIC
    quarantine_push(obj)
```

```
debug_alloc(cache):
    obj = real_alloc(cache)
    fill(obj.memory, POISON_ALLOC)
    obj.magic = LIVE_MAGIC
    return obj
```

隔离队列会延迟复用刚释放对象，使 `use-after-free` 更容易在测试中暴露。代价是内存占用和性能下降，所以通常只在调试或安全强化配置中开启。

泄漏、越界与对象生命周期

- buddy 分配释放后，空闲页总数守恒。
- 分裂和合并后，不存在重叠空闲块。
- slab 分配对象满足对齐，释放后能复用。
- double free、越界写和 `use-after-free` 在调试模式能被检测。
- GFP_ATOMIC 路径不能睡眠，失败时有明确降级策略。
- 内存压力下，分配器统计能解释碎片和失败原因。

SLUB、内存压缩与资源归属

buddy 的 `free_area` 和迁移类型

Linux buddy 不只是按 `order` 分链表，还按迁移类型分组，例如 `movable`、`unmovable`、`reclaimable`、`CMA` 等。这样做是为了降低长期碎片：可移动页尽量放在一起，未来需要高 `order` 连续页时更容易通过 `compaction` 得到。

```
struct free_area {
    list_head free_list[MIGRATE_TYPES];
    unsigned long nr_free;
};
```

页分配路径必须同时考虑空闲链表、水位线、内存区域和迁移类型。一次 `order-9` 分配失败，不一定表示系统没有 512 页空闲，也可能只是没有连续的 512 页。

SLAB、SLUB、SLOB 的取舍

SLAB 强调对象缓存和较复杂的队列；SLUB 简化元数据，成为常见默认；SLOB 面向极小系统，结构简单但伸缩性较弱。理解 SLUB 时，应把对象缓存、每 CPU 空闲链表、部分使用的 slab，以及调试用红区和填充值联系起来。

```
kmem_cache_alloc(cache):
    if current_cpu.freelist:
        return pop(current_cpu.freelist)
    if cache.node.partial:
        slab = take_partial(cache.node)
        load_cpu_freelist(slab)
        return pop(current_cpu.freelist)
    slab = new_slab_from_buddy()
    return first_object(slab)
```

SLUB 的快路径很短，但 `debug` 选项会显著增加检查成本。教材中讲“分配是 $O(1)$ ”时，要同时说明这是平均快路径；慢路径可能进入 buddy、回收、`compaction` 或 `OOM`。

kmalloc size class 与内部碎片

`kmalloc(130)` 可能进入 192 或 256 字节 size class，取决于架构和配置。向上取整带来内部碎片，但换来固定大小缓存的速度和对齐。对象若频繁分配，最好建立专用 `kmem_cache`，可以提供构造函数、调试名和更好统计。

GFP_NOIO、GFP_NOFS 和回收递归

`GFP_KERNEL` 允许睡眠和回收；`GFP_ATOMIC` 用于中断或持自旋锁路径；`GFP_NOIO` 禁止通过 I/O 回收；`GFP_NOFS` 禁止进入文件系统回收路径。后两者用于避免递归死锁。

```
filesystem_write_path():
    lock(inode)
    buf = kmalloc(size, GFP_NOFS)
    // avoid reclaim entering filesystem and taking inode locks again
```

若文件系统持有 inode 锁时用 `GFP_KERNEL` 分配，内存压力下回收可能进入同一文件系统并等待 inode 锁，形成递归死锁。GFP flag 是锁和回收的边界，不是性能装饰。

vmalloc 与物理不连续

`kmalloc` 通常返回物理连续内存，适合 DMA 或小对象；`vmalloc` 返回虚拟连续但物理页可不连续的内存，适合大缓冲或模块映射。代价是需要建立页表，TLB 局部性更差，不能直接给不支持 scatter-gather 的设备 DMA。

```
vmalloc(size):
    area = allocate_virtual_area(size)
    pages = alloc_individual_pages(npages)
    map_pages(area, pages)
    return area.addr
```

kmemleak、KASAN 和 KFENCE

内存泄漏和 use-after-free 很难靠肉眼找。kmemleak 通过扫描可达指针寻找未释放对象；KASAN 使用 shadow memory 检测越界和释放后访问；KFENCE 用低频保护页检测堆错误，开销比 KASAN 低但覆盖率也低。

工具	能发现	代价
kmemleak	长期未释放对象	扫描和误报，需要调试配置
KASAN	越界、use-after-free	内存和性能开销高
KFENCE	采样式堆越界和 UAF	开销低，非确定覆盖

内存碎片、compaction 和 CMA

内存碎片分为内部碎片和外部碎片。内部碎片来自 size class 向上取整；外部碎片来自空闲页分散，无法满足高 order 连续页。compaction 通过迁移可移动页，把空闲页聚合成连续块。CMA 预留可迁移区域，给需要连续物理内存的设备使用。

```
compact_zone(zone):
    free_scanner = end_of_zone
    migrate_scanner = start_of_zone
    while scanners_not_met:
        page = find_movable_page(migrate_scanner)
        free = find_free_page(free_scanner)
        migrate(page, free)
```

并非所有页都能迁移。页表页、驱动 pin 住的 DMA 页、mlock 页和某些内核对象不可移动。高 order 分配失败时，诊断要看总空闲页、最大连续块、不可移动页

比例和 compaction 失败原因。

内存控制组和分配归属

cgroup v2 可以限制一组进程的内存。分配器需要把许多用户可归因的内存记到 memcg，例如 page cache、匿名页、部分 slab 对象。这样容器内泄漏就不会无限消耗宿主内存。

```
charge_memcg(page, current.memcg):
    if memcg.current + page.size > memcg.max:
        reclaim_memcg(memcg)
        if still over limit:
            trigger_memcg_oom(memcg)
    memcg.current += page.size
```

memcg OOM 和全局 OOM 必须区分。前者应限制在相应资源组内，后者说明整机资源不足。判断时要同时核对资源组当前用量、内存事件和最终被选择终止的任务。

对象生命周期对账

内核内存质量不只看分配成功，还要能证明对象生命周期闭合。每类对象应有 alloc/free 计数、当前活跃数、高水位和失败次数。对复杂对象，还应记录 owner 和引用来源。

```
object_accounting:
    alloc_count++
    active_count++
    high_watermark = max(high_watermark, active_count)

free:
    active_count--
    free_count++
```

测试结束时 active count 不为零，就是泄漏线索；free count 大于 alloc count，说明 double free 或计数损坏；高水位异常说明负载下对象滞留。生产系统不一定记录所有栈，但调试构建应支持按对象类型打开详细追踪。

尚未解决的问题。内存中的对象可以分配、引用和回收，外部设备却不会在函数返回时同步完成请求。下一章建立长期请求、DMA 缓冲所有权、完成记录和设备移除协议。

第三部分

异步设备与文件接口

本部分导读

内存访问由处理器在指令路径中完成，设备请求却可能在未来某个时刻才结束。为了让执行流可以等待而不占用处理器，内核需要长期请求对象、明确的缓冲区所有权和可核对的完成记录。设备层建立这些边界以后，块层负责拆分、合并、排序与持久化顺序，VFS 再把设备和文件系统对象接到进程可以持有的描述符上。

第 9 章 为什么设备操作需要长期请求对象

继承的机器。内核能够分配请求结构和缓冲页面，也能让 task 在等待队列睡眠。设备寄存器写入只表示请求被提交，并不表示数据已经传输；中断只表示出现了需要处理的事件，也不天然指出哪个请求完成、缓冲区何时可以复用。

异步设备请求与完成证据

设备驱动把硬件寄存器、中断、DMA 和内核对象连接起来。应用看到的是 fd、socket、块设备或帧缓冲；驱动看到的是 MMIO 寄存器、描述符环、设备状态位、中断线、DMA 地址和错误码。驱动的难点在于硬件和 CPU 并发运行：CPU 写描述符，设备异步读取；设备写内存，CPU 稍后读取；中断可能在任意时刻到来。

先从一个错误直觉开始：

```
disk_write(buf, len):
    return disk.write(buf, len)
```

这像普通函数调用，但真实设备不会沿着你的调用栈返回。CPU 能做的是写设备寄存器、填一段内存中的描述符、敲 doorbell 通知设备；设备随后按自己的时钟和状态机工作。它可能成功，可能失败，可能很久以后才完成，也可能在 reset 中丢掉请求。驱动的工作就是把这种异步硬件行为包装成内核可管理的请求。

把最小路径写清楚：

```
submit_write(buf):
    req = allocate_request()
    req.buffer = buf
    req.state = PREPARED
    dma = map_buffer_for_device(buf)
    fill_descriptor(req.id, dma, len)
    publish_descriptor_with_barrier()
    ring_doorbell()
    req.state = IN_FLIGHT
    sleep_until_completion(req)
```

这段伪代码第一次引出几个概念。descriptor 是给设备看的请求摘要，通常包含 DMA 地址、长度、标志和请求身份。doorbell 是通知设备“我放了新活”的寄存器写入。completion 是设备稍后交回的完成结果。request 是内核自己的对象，用来把用户请求、buffer、描述符、等待任务和最终错误码连在一起。

最简单设备可以用 programmed I/O：CPU 直接读写设备寄存器或数据端口。PIO 易懂，但吞吐低，CPU 被数据搬运占住。DMA 让设备直接读写内存，CPU 只准备描述符和缓冲区。DMA 提高吞吐，也引入缓存一致性、地址映射、内存屏障和 buffer 生命周期问题。

PIO 和 DMA 的区别不是“老技术”和“新技术”的区别，而是 CPU 参与程度不同：

方式	CPU 做什么	风险边界
PIO	每次读写设备寄存器搬运数据	CPU 忙等、吞吐低、长时间占用核心
DMA	准备 buffer 和描述符，让设备直接搬运内存	buffer 生命周期、cache 一致性、IOMMU 映射

中断	设备完成后通知 CPU	中断风暴、上半部不能睡眠、completion 要能找回请求
轮询	CPU 定期检查完成队列	延迟/吞吐折中、budget 背压

驱动通常分成上半部和下半部。中断上半部快速确认设备、读取状态、屏蔽或清除中断，把复杂工作推迟到底半部、工作队列、软中断或普通内核线程。这样可以减少中断关闭时间，避免在硬中断上下文执行可能睡眠的操作。

驱动和前面章节的连接也要明确：提交请求时可能让当前 task 睡眠，completion 到来后要唤醒 wait queue；DMA buffer 来自页分配器和虚拟内存系统；设备错误最终要通过 fd、socket、块层或文件系统返回给用户；权限和 namespace 决定用户能否打开这个设备。驱动不是孤立章节，而是硬件异步事件进入 OS 对象系统的入口。

从发现设备到安全移除

为什么设备和驱动要成为两个对象

若内核为每一块具体硬件直接写一段固定初始化代码，单一机器可以启动，但设备替换、热插拔、休眠恢复和同类多设备都会使这套做法失效。内核必须分别保存两类事实：机器上实际出现了哪些设备，以及当前有哪些软件能够控制某类设备。前者形成 device 对象，后者形成 driver 对象。

两者通过可匹配的身份信息建立绑定。例如总线发现一个设备后，先登记设备身份、资源范围和父子关系；驱动注册时声明自己支持哪些身份。匹配成功只说明“可以尝试控制”，还不能立即向用户发布设备，因为初始化过程中仍可能缺少中断、DMA 队列或电源资源。

```
discover device
-> create device object
-> match a compatible driver
-> probe and acquire resources
-> initialize queues and interrupt handling
-> publish usable interface
```

这个顺序使失败保持局部。若初始化队列失败，驱动释放此前取得的资源，device 对象仍可保留为“已发现但尚不可用”，也可以等待另一个驱动；如果在资源尚未齐备时就提前创建设备节点，用户请求可能进入一个半初始化对象。

probe 为什么是一项可回滚事务

一次初始化通常按顺序取得寄存器映射、中断向量、DMA 能力、队列内存、设备固件和上层注册。任一步都可能失败。资源释放必须采用相反顺序，因为较后的对象往往引用较早的对象：队列使用 DMA 映射，完成路径使用中断，用户接口引用队列和设备状态。

```
probe(dev):
    regs = map_registers(dev)
    irq = allocate_interrupt(dev)
    queues = allocate_dma_queues(dev)
    reset_and_negotiate(dev)
    start_device(dev, queues)
    publish_interface(dev)
    return success

on failure:
    unpublish_if_needed()
    stop_device_if_started()
```

```
free_dma_queues()
release_interrupt()
unmap_registers()
```

这里的提交点是向其他子系统发布可用接口。在此之前，错误只需释放私有资源；在此之后，关闭和移除还必须阻止新请求，并处理已经取得对象引用的调用者。托管资源可以减少遗漏，但不会替驱动决定设备是否仍在 DMA、请求是否已完成以及用户接口何时不可见。

移除为何不能直接释放对象

设备被拔出或驱动卸载时，仍可能存在三类活动：尚未提交到设备的新请求、设备已经接收的 in-flight 请求，以及持有 fd 或其他引用的用户。若移除路径直接释放队列和 DMA buffer，迟到的中断或设备写入就可能访问已经复用的内存。

安全移除通常先把设备状态改为 quiescing，使新请求立即失败或留在上层；随后停止队列、屏蔽并同步中断，等待设备不再访问 DMA 映射；未完成请求以明确错误完成；最后撤销用户可见接口并等待对象引用归零。只有这些条件同时成立，寄存器、队列和设备对象才能释放。

阶段	必须建立的事实	过早进入下一阶段的后果
停止接收	新请求不能再进入设备队列	关闭过程永远追不上新提交
终止设备访问	设备不再读取描述符或 DMA buffer	设备访问已释放内存
同步完成路径	旧中断和延后工作已经退出	迟到完成使用旧 request 或 tag
完成未决请求	所有等待者收到成功或错误	线程永久睡眠，引用无法归零
撤销与释放	接口不可见且外部引用为零	用户通过旧 fd 进入已销毁对象

复位是同一问题的较小版本。复位前后的请求 tag 可能数值相同，因此驱动还需要 generation 或等价状态区分旧队列和新队列。完成记录只有同时匹配队列世代和活动请求，才能成为释放 buffer 的证据。

队列所有权与中断分工

先画一个网卡或磁盘的描述符环：

```
descriptor ring:
  head: device consumes descriptors
  tail: driver produces descriptors
  desc[i]:
    dma_address
    length
    flags
    status
```

驱动发送路径通常是：从内核分配 buffer，映射成设备可见 DMA 地址，填描述符，执行内存屏障，更新 tail，通知设备。完成路径通常是：设备 DMA 完成并写 status，触发中断，驱动回收描述符，解除 DMA 映射，释放 buffer，唤醒等待者或上报 completion。

描述符环、DMA 与内存顺序

MMIO 寄存器访问

MMIO 把设备寄存器映射到物理地址。访问 MMIO 不能当成普通内存优化：编译器不能随便重排，CPU 也要遵守设备要求的顺序。驱动通常使用专用 `readl/writel` 一类接口。

```
device_start(dev):
    writel(dev->mmio + CTRL, CTRL_RESET)
    wait_until(readl(dev->mmio + STATUS) & READY)
    writel(dev->mmio + IRQ_MASK, RX_IRQ | TX_IRQ)
```

轮询 `wait` 要有超时。设备可能坏掉，寄存器可能永远不变。驱动若在内核里无限等待，会拖死系统。

描述符环

描述符环是设备驱动的核心队列。生产者和消费者可以是 CPU 和设备，也可以反过来。环满时不能继续提交，环空时不能继续回收。

```
submit_tx(desc_ring, packet):
    next = (tail + 1) % ring_size
    if next == head:
        return -EAGAIN
    desc[tail].addr = dma_map(packet.data)
    desc[tail].len = packet.len
    desc[tail].flags = OWNED_BY_DEVICE
    memory_barrier()
    tail = next
    writel(DOORBELL, tail)
```

这里的屏障非常关键。CPU 必须先把描述符内容写完，再更新 `tail` 或 `doorbell` 通知设备；否则设备可能看到半初始化描述符。

把一次网卡发送请求走细一点。用户态 `send` 最终让内核拿到一段要发送的数据，网络栈构造 `packet buffer`，驱动要把这个 `buffer` 交给网卡。网卡不认识 `struct sk_buff`，也不能读用户虚拟地址；它通常只读一段描述符内存，描述符里写着 DMA 地址、长度、校验和 `offload` 标志、结束位和 `request tag`。CPU 是生产者，设备是消费者。

步骤	驱动和硬件动作	不能错的边界
保留槽位	驱动读取 <code>head/tail</code> ，确认环未滿，选择 <code>tail</code> 槽位	满环必须返回或停止上层队列，不能覆盖设备未消费的描述符
映射 <code>buffer</code>	DMA API 把 <code>packet buffer</code> 映射成设备可见 DMA 地址	设备不能拿用户指针或内核虚拟地址
填写描述符	写入地址、长度、 <code>flags</code> 、 <code>tag</code> ，最后设置 <code>owned/valid</code> 位	<code>valid</code> 位不能早于其他字段对设备可见
发布顺序	执行 DMA 写屏障，保证描述符内容先于 <code>doorbell</code>	少屏障会让设备读到旧字段或半初始化字段
敲门铃	写 MMIO <code>doorbell</code> 或更新 <code>tail</code> ，通知设备取新描述符	<code>doorbell</code> 只是通知，不代表发送完成
设备 DMA	网卡读取描述符和数据，经 IOMMU 访问内存，发送到链路	<code>buffer</code> 在 <code>completion</code> 前不能释放或复用
完成回收	设备写回 <code>completion</code> 或更新 <code>head</code> ，驱动 <code>unmap</code> 并释放 <code>buffer</code>	回收必须以设备完成证据为准

这里有两个常见误解。第一，`tail = next` 只是软件变量变化，设备未必看见；真正通知设备的通常是 MMIO doorbell 或共享队列尾指针的可见更新。第二，`completion` 不等于用户业务语义完成。网卡发送 `completion` 通常说明设备已经不再需要这个 DMA buffer，未必说明对端应用已经收到数据；块设备 `completion` 说明设备处理了请求，是否持久还要看 `flush/FUA` 和文件系统语义。驱动层必须先把“buffer 能否释放”这件事讲清楚，再谈上层协议。

迟到 `completion` 是取消路径最容易出错的地方。假设上层超时，驱动 `reset` 队列并把 `request` 对象释放；如果设备或旧中断稍后又报告同一个 `tag` 完成，而 `tag` 已经被新请求复用，驱动可能唤醒错任务或释放错 `buffer`。可靠实现通常给 `request tag`、队列 `generation`、`in-flight bitmap` 和 `reset` 状态建立明确规则：`reset` 开始后停止新提交，标记旧请求失败，等待或屏蔽旧 `completion`，确认设备不再 DMA 后再复用 `tag` 和 `buffer`。

DMA 和缓存一致性

若 CPU cache 和设备 DMA 不自动一致，驱动必须在设备读内存前清理 `cache`，在设备写内存后失效 `cache`。即使硬件支持 IOMMU，也要建立设备可访问地址映射。

```
prepare_dma_to_device(buf):
    cache_clean(buf)
    dma_addr = iommu_map(buf.phys)
    return dma_addr

finish_dma_from_device(buf):
    cache_invalidate(buf)
    parse_received_data(buf)
```

DMA buffer 的生命周期必须覆盖设备访问全过程。提交后不能释放或复用 `buffer`，直到 `completion` 表明设备不再访问它。这个规则和异步 I/O 的 `buffer` 所有权完全一致。

缓存一致性要分方向看。设备从内存读数据，叫 `TO_DEVICE`：CPU 先把数据写进 `buffer`，但这些写入可能还停在 `cache` 中，设备从内存读会看到旧内容；所以提交前要 `clean` 或通过 `coherent` 映射保证设备可见。设备向内存写数据，叫 `FROM_DEVICE`：设备已经把新数据写进内存，但 CPU `cache` 里可能还有旧 `cache line`；所以 `completion` 后 CPU 读取前要 `invalidate` 或同步。

方向	提交前/完成后要做什么	忘记后的现象
TO_DEVICE	CPU 写 <code>buffer</code> 后， <code>clean cache</code> ，再让设备 DMA 读	设备发送旧包、旧磁盘数据或半初始化描述符
FROM_DEVICE	设备 <code>completion</code> 后， <code>invalidate cache</code> ，再让 CPU 解析	CPU 读到旧包内容，协议栈校验失败或数据错乱
BIDIRECTIONAL	两个方向都要同步，并严格定义所有权切换	CPU 和设备同时修改，结果半新半旧
coherent DMA	平台保证 CPU/设备可见性，但仍需要顺序屏障	数据可见不等于 doorbell 顺序自动正确

再看一个接收包场景。驱动预先分配 `RX buffer`，把它 DMA map 成 `FROM_DEVICE`，挂到 `RX ring`。挂上去之后，`buffer` 暂时归设备所有，CPU 不应把它当普通内存写。网卡收到包后 DMA 写入 `buffer`，并在描述符里写入长度、校验状态和完成位。驱动 `poll` 到完成位后，先执行 DMA 同步或 `invalidate`，再读取包头和长度，构造 `skb` 交给协议栈。若顺序反了，CPU 可能先读旧 `cache line`，看到旧长度或旧以太网头；这类 bug 往往不是每次复现，因为它取决于 `cache` 状态、CPU 迁移和包到达时机。

IOMMU 还提供了另一个边界：设备只能访问被映射的 IOVA 范围。驱动 unmap 后，设备若继续 DMA，正确结果应是 IOMMU fault 或设备错误，而不是静默覆盖任意物理内存。可这要求驱动先确认设备不再持有该 descriptor，再 unmap 并释放页。IOMMU 是安全网，不是生命周期管理的替代品。

中断合并和轮询

高吞吐设备如果每个包都中断一次，CPU 会被中断淹没。中断合并让设备积累多个完成再中断；NAPI 一类机制在高负载下从中断转为轮询，批量处理包，降低中断成本。代价是单个包延迟可能增加。

```
rx_interrupt():
    disable_rx_irq()
    schedule_poll()

poll(budget):
    processed = 0
    while processed < budget and rx_desc_ready():
        handle_packet()
        processed += 1
    if no_more_packets():
        enable_rx_irq()
```

这里的 budget 是背压工具。一次 poll 不能无限处理，否则其他任务得不到 CPU。

超时、复位与迟到完成

- MMIO 等待必须有超时，设备无响应时返回错误。
- 描述符环满时返回背压，不覆盖未完成描述符。
- 提交描述符前执行必要内存屏障。
- DMA buffer 在 completion 前不能释放或复用。
- 中断处理上半部不执行可能睡眠操作。
- 批量 poll 遵守 budget，不能垄断 CPU。
- 设备 reset 后，驱动能清理 in-flight 请求并通知上层失败。

设备完成如何支撑上层提交

块层原本可以单独讲很久，但放在设备章里看更清楚：它是把文件系统的逻辑读写转换成设备队列 request 的中间层。文件系统提交的是 bio，描述“文件系统需要这些扇区读写”；blk-mq 把 bio 合并、排序、分派给硬件队列；驱动把 request 变成设备描述符；completion 再把错误和字节数一路传回等待者。

层次	保存什么账本	不能混淆的边界
bio	一组页、扇区范围、读写方向和完成回调	bio 完成不一定等于文件系统事务提交
request	合并后的设备队列项、tag、超时和状态	request tag 复用必须等旧 completion 收敛
hardware queue	CPU/NUMA 亲和的提交队列	多队列提高吞吐，也引入负载和顺序问题
flush/FUA	写缓存顺序和持久化约束	普通写完成不等于掉电安全

I/O scheduler	合并、排序、公平和延迟控制	吞吐优化不能破坏屏障语义
---------------	---------------	--------------

存储顺序比普通异步完成更严格。日志文件系统、数据库和键值存储经常要求“先写数据，再提交元数据或 manifest”。如果设备或调度器为了吞吐随意重排，就可能在崩溃后看到新 manifest 指向旧数据。块层用 flush、FUA、barrier 或等价机制表达这些顺序约束；文件系统还要把自己的 journal commit、checkpoint 和目录项更新放在正确位置。设备章至少要记住一句话：DMA completion 说明设备不再需要某个 buffer，持久化 completion 才说明恢复路径能相信某个状态。

设备模型、IOMMU 与虚拟设备

设备模型：bus、driver、device、class

Linux 设备模型把发现、匹配、生命周期和 sysfs 统一起来。总线负责枚举和匹配，driver 提供 probe/remove，device 表示具体硬件或虚拟设备，class 给用户空间暴露类别视图，kobject/kset 负责引用计数和 sysfs 层次。

```
struct bus_type      -> match(device, driver)
struct device_driver -> probe(device), remove(device)
struct device        -> parent, bus, driver, kobject
struct class         -> /sys/class view
```

probe 成功后驱动必须拥有明确资源集合：MMIO 映射、IRQ、DMA mask、队列、字符/块/网络注册对象。remove 路径按相反顺序撤销，并拒绝新请求、等待 in-flight 请求完成。

PCI 驱动骨架

```
static const struct pci_device_id ids[] = {
    { PCI_DEVICE(VENDOR, DEVICE) },
    { 0 }
};

probe(pdev, id):
    pci_enable_device(pdev)
    pci_request_regions(pdev)
    mmio = pci_iomap(pdev, bar)
    dma_set_mask_and_coherent(pdev, DMA_BIT_MASK(64))
    request_irq(pdev->irq, handler, IRQF_SHARED, name, dev)
    init_queues(dev)

remove(pdev):
    stop_queues(dev)
    free_irq(pdev->irq, dev)
    pci_iounmap(pdev, mmio)
    pci_release_regions(pdev)
    pci_disable_device(pdev)
```

错误路径要能从任一步失败回滚。例如 request_irq 失败时，要释放 MMIO 和 PCI region；队列初始化失败时，要释放 IRQ。真实源码中这类 goto err_* 标签比正常路径更能体现驱动质量。

字符、块、网络三类设备

类别	核心对象	典型回调
----	------	------

字符设备	<code>cdev</code> 、 <code>file_operations</code>	<code>open/read/write/ioctl/poll</code>
块设备	<code>gendisk</code> 、 <code>request_queue</code> 、 <code>blk-mq</code>	<code>submit_bio</code> 、 <code>queue_rq</code> 、 <code>completion</code>
网络设备	<code>net_device</code> 、 <code>net_device_ops</code> 、 <code>NAPI</code>	<code>ndo_start_xmit</code> 、 <code>poll</code> 、 <code>set_rx_mode</code>

同一个硬件可能暴露多个接口。比如 NVMe 是块设备，网卡是网络设备，串口是字符设备。VFS 统一 `fd` 入口，但驱动对象和 `completion` 路径完全不同。

streaming DMA 与 coherent DMA

`dma_alloc_coherent` 返回 CPU 和设备一致可见的内存，适合描述符环；`dma_map_single/sg` 把已有 `buffer` 临时映射给设备，适合数据包或块 I/O。streaming DMA 要在方向、同步和 `unmap` 上严格配对。

```
tx_submit(buf):
    dma = dma_map_single(dev, buf.data, buf.len, DMA_TO_DEVICE)
    if dma_mapping_error(dev, dma): return -EIO
    fill_desc(dma, buf.len)
    dma_wmb()
    ring_doorbell()

tx_complete(desc):
    dma_unmap_single(dev, desc.dma, desc.len, DMA_TO_DEVICE)
    free_buf(desc.buf)
```

IOMMU 为设备建立地址空间，限制设备只能 DMA 到授权页。没有 IOMMU 或映射错误时，设备 bug 可能覆盖任意内存；有 IOMMU 时，错误 DMA 会变成 IOMMU fault，至少能阻止破坏扩大。

NAPI、NVMe 与 virtio 的共同队列模型

网络设备、NVMe 与 virtio 的具体队列格式不同，但都可以按同一组问题分析：

1. probe 如何发现设备、分配队列、注册上层对象。
2. submit 路径如何填描述符、DMA map、通知设备。
3. completion 路径如何从中断/NAPI/poll 回收资源。
4. reset/remove 路径如何停止新请求并清理 in-flight。
5. 错误统计在哪里暴露给 `ethtool`、`sysfs`、`dmesg` 或 `block stats`。

ACPI platform 设备与 PCIe 自描述设备

并非所有设备都能像 PCIe 一样完整自描述。x86-64 PC/服务器上，可枚举高速设备通常来自 PCIe 配置空间；板载控制器、固件虚拟设备、电源和热管理资源常由 ACPI 表描述。platform driver 的匹配不依赖 `vendor/device id`，而依赖 `ACPI id` 或平台总线注册信息。

```
ACPI resource sketch:
HID/UID identify the platform device
MMIO resource describes register window
IRQ resource describes interrupt routing
_CRS reports current resource settings
_PS0/_PS3 may describe power-state transitions
```

probe 阶段要把描述转换成内核资源：`platform_get_resource` 取得 MMIO 区间，`devm_ioremap_resource` 建立映射，`platform_get_irq` 取得中断，电源管理回

调处理设备 suspend/resume。教学系统可以把 ACPI 资源简化成静态设备表，但仍要保留“描述硬件”和“驱动绑定”两个阶段。

托管资源和错误回滚

Linux 驱动大量使用 devm 托管资源。资源绑定到 device 生命周期，remove 或 probe 失败时自动释放。托管资源不是为了偷懒，而是为了减少 probe 多阶段失败路径中的泄漏。

```
probe(dev):
    state = devm_kzalloc(dev, sizeof(*state))
    mmio = devm_ioremap_resource(dev, res)
    irq = platform_get_irq(dev, 0)
    ret = devm_request_threaded_irq(dev, irq, top, thread, flags, name, state)
    if ret:
        return ret
    ret = register_upper_object(state)
    if ret:
        return ret
    return 0
```

但 devm 不能替代所有生命周期管理。已经暴露给上层的字符设备、网络设备或块设备必须先停止新请求、撤销注册、等待 in-flight 完成，再让托管资源释放。否则用户态仍可能通过旧 fd 进入已经释放的 MMIO 或队列。

ioctl、sysfs 与用户态控制面

驱动给用户态暴露控制面时，常见三种方式：ioctl、sysfs 属性和 netlink。ioctl 适合和 fd 绑定的设备私有命令；sysfs 适合稳定、简单、可文本化的设备属性；netlink 适合网络和复杂事件通知。控制面设计必须有版本、长度、权限和兼容性边界。

```
ioctl(file, cmd, user_arg):
    switch cmd:
        case GET_STATUS:
            status = snapshot_device_status()
            return copy_to_user(user_arg, status)
        case RESET_DEVICE:
            if !capable(CAP_SYS_ADMIN):
                return -EPERM
            return reset_device_safely()
        default:
            return -ENOTTY
```

ioctl 的常见漏洞来自信任用户结构体长度、未检查 padding、把内核指针泄露给用户、或在 reset 中没有和 I/O 路径同步。控制命令不是旁路，它和 read/write/interrupt 一样会并发访问设备状态。

runtime PM、suspend 与 resume

现代设备需要省电。runtime power management 允许设备空闲时关闭时钟或电源域；系统 suspend/resume 要在整机睡眠和唤醒时保存恢复设备状态。驱动必须区分“设备逻辑对象仍存在”和“硬件暂时不可访问”。

```
runtime_suspend(dev):
    stop_new_io(dev)
    wait_inflight(dev)
    save_registers(dev)
    disable_irq(dev)
    disable_clock(dev)
```



```
runtime_resume(dev):
    enable_clock(dev)
    restore_registers(dev)
    enable_irq(dev)
    restart_queues(dev)
```

如果 resume 后忘记恢复 DMA mask、队列基址或中断 mask，设备可能表现为偶发超时。电源管理 bug 常常只在压力、热插拔、suspend 循环或低功耗平台上出现，因此测试必须覆盖这些状态转换。

驱动并发不变量

驱动至少要维护下面不变量：

- 提交给设备的描述符在 completion 前不能释放。
- 设备可 DMA 的 buffer 必须有有效映射，方向必须匹配。
- remove/reset 开始后，不再接受新请求。
- 中断处理看到 dying 状态时只确认和丢弃，不再提交新工作。
- 用户 fd 持有期间，驱动私有对象引用不能降到零。
- 上半部只做不可睡眠操作，复杂恢复进入线程化 IRQ 或 workqueue。

把这些不变量写进测试，才能发现“正常收发能跑”之外的问题。驱动不是能读写设备就合格，而是要在错误、中断风暴、热拔、reset 和并发 close 下仍然收敛。

一次异步写请求：函数返回以后谁仍然拥有状态

设驱动要把内存页 P 中的 8192 字节写到设备逻辑块 4096，设备队列深度为 64。调用线程 T 进入驱动时可以被调度，设备则独立运行。若驱动只把局部变量地址写进寄存器，提交函数返回后局部变量失效，设备仍可能继续 DMA。因而请求对象必须比提交调用活得更久，并明确区分“软件接受”“设备可见”“设备完成”和“等待者观察完成”。

对象	代表字段	所有权与寿命	完成条件
请求 R37	id、generation、operation、range、state、result、waiter	从分配到最终引用释放；队列和等待者可同时持有引用	状态进入终态且结果只发布一次
DMA 映射 D37	设备地址、长度、方向、页引用、映射域	映射成功后由请求拥有；设备不再访问后才能解除	completion 已证明该描述符不再取数
描述符槽 12	命令字段、DMA 地址、长度、请求标签、phase	入队前由驱动拥有，doorbell 后由设备消费	设备通过完成项归还对应标签
完成项 C37	标签、状态、完成长度、phase/generation	设备产生，驱动消费	驱动校验身份并推进 completion head

发布描述符必须先于通知设备。

处理器把描述符字段写入内存，并不自动保证设备在 doorbell 之后看到完整新值。驱动先填写 DMA 地址、长度、操作和标签，再执行适合该体系结构与 DMA 模型的发布屏障，最后写 doorbell。屏障不让设备更快，只建立“看到通知就应看到此前描述符”的顺序。

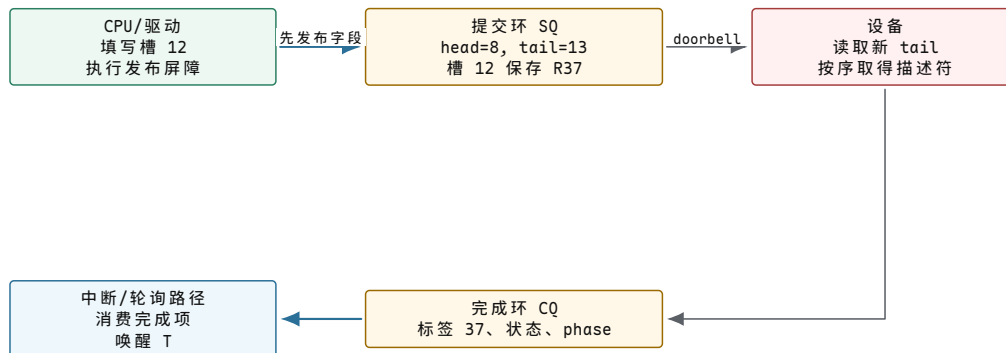

```

submit_write(page, block, length):
    // 映射阶段固定页面并产生设备可使用的地址；失败时尚未发布请求。
    req = allocate_request()
    req.generation = queue.generation
    req.dma = dma_map(page, WRITE_TO_DEVICE)

    // 先完整写描述符，再通过发布屏障和 doorbell 转移消费权。
    slot = reserve_submission_slot()
    fill_descriptor(slot, req.id, req.dma.address, block, length)
    dma_write_barrier()
    ring_doorbell(queue.tail)

    req.state = IN_FLIGHT
    return req

```



图示：提交环和完成环的方向相反。doorbell 只通知设备检查新 tail，不携带完整请求；完成中断只通知软件检查完成环，也不直接等于某个调用已经返回。

等待线程状态与请求状态必须分别推进。

T 可以同步等待 R37，也可以把 R37 交给异步上层后继续执行。同步等待时，T 从 RUNNING 进入 BLOCKED，R37 仍是 IN_FLIGHT；设备完成先把 R37 推入 COMPLETING，再由完成路径写入结果、解除 DMA 映射并唤醒 T。唤醒只把 T 变成 RUNNABLE，真正返回还要等调度器再次选择它。

时刻	T 的状态	R37 的状态	此时可以断言什么
t_0 提交前	RUNNING	PREPARED	页面仍由软件访问规则约束，设备尚不可见
t_1 doorbell 后	RUNNING 或即将等待	IN_FLIGHT	设备可以访问 DMA 区域，buffer 不得提前复用
t_2 T 睡眠	BLOCKED	IN_FLIGHT	CPU 可以运行其他任务，请求没有因此取消
t_3 完成项出现	BLOCKED	COMPLETING	设备给出结果，驱动仍需校验和清理
t_4 驱动发布结果	RUNNABLE	DONE/FAILED	DMA 已解除，等待谓词成立
t_5 T 再次运行	RUNNING	终态	调用者读取稳定结果并释放最后引用

```

complete_from_queue(entry):
    // 标签和队列代次共同识别请求，避免复位前的迟到完成命中新请求。
    req = lookup(entry.request_id)
    if req.generation != queue.generation:

```

```
account_stale_completion(entry)
return

// 先使设备停止拥有 buffer，再向等待者发布最终结果。
dma_unmap(req.dma)
req.result = translate_device_status(entry.status)
store_release(req.state, terminal_state(req.result))
wake(req.wait_queue)
```

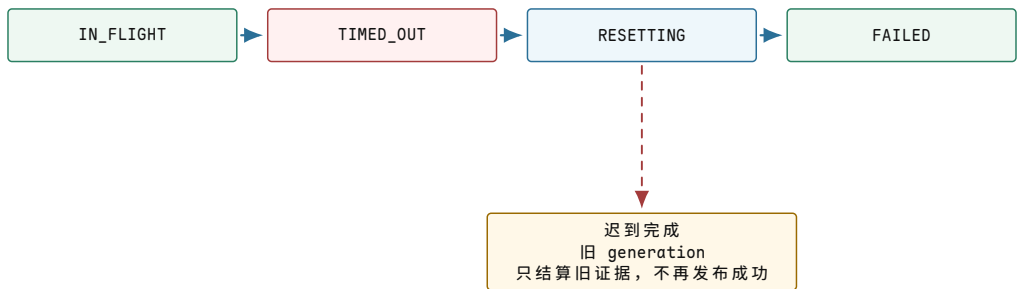
中断只提供“有工作”的证据。

共享中断可能服务多个队列，合并中断可能代表多项完成，轮询模式甚至不为每项完成产生中断。因此中断处理程序不能凭中断号猜测 R37 已完成。上半部确认设备来源并屏蔽或应答事件，下半部或轮询器消费完成环，只有带请求标签的完成项才能推进具体对象。

若完成量很大，上半部逐项处理会长期占用高优先级上下文。常见做法是限制一次消费预算：处理最多 N 项，尚有工作则继续调度轮询；队列清空后才重新允许中断。预算是在中断延迟和吞吐之间建立边界，不改变每个请求必须恰好完成一次的要求。

超时、复位和迟到完成。

超时只说明规定时间内没有观察到完成，不证明设备绝不会再写 buffer。若驱动立即释放 R37 和页面，迟到 DMA 会破坏已经分配给别人的内存。正确路径先把请求转入 TIMED_OUT，阻止新的普通完成发布，再取消队列或复位设备；只有设备访问能力被撤销后才能解除映射。



图示：超时不是释放许可。复位代次把复位前后的请求身份分开；旧完成可以用于统计和清理，但不能完成复位后复用同一槽位的新请求。

故障	直接释放为何错误	收束条件	对上层结果
DMA 映射失败	设备从未取得地址，但部分请求对象已分配	回收准备对象，不写 doorbell	同步返回资源错误
提交后超时	设备可能仍持有描述符和 buffer	取消或复位并撤销设备访问能力	返回超时或设备错误
设备热移除	MMIO 与队列可能随时不可访问	阻止新提交，排空/失败旧请求，等待引用归零	未完成请求统一失败
完成项状态错误	传输已结束但数据结果无效	解除 DMA，保存精确错误与完成长度	上层决定重试或部分完成

IOMMU 限制设备可访问的对象范围。

没有 IOMMU 时，设备地址往往直接或经固定偏移对应物理地址，错误描述符可能访问任意可达内存。IOMMU 为设备建立独立地址域，使 D37 只映射请求所需页面。它不能证明设备写入内容正确，也不能替代请求寿命管理；它提供的是“即使描述符地址错误，访问仍被限制在已授权映射”这一故障边界。

本章复核：四次所有权转移

一次设备请求至少经历四次所有权变化：调用者把 buffer 的设备访问权交给请求；驱动发布描述符后把消费权交给设备；设备写入完成项后把槽位和结果交回驱动；驱动解除 DMA 并发布终态后，调用者才重新取得普通复用权。任何一步省略，都会把函数调用寿命误写成设备寿命。

练习 9.1 区分中断与请求完成

设备触发一次合并中断，完成环中有 R35、R36、R37 三项。为什么不能在中断入口直接唤醒 R37 的等待者？写出驱动必须先取得的证据。

解答 9.1 区分中断与请求完成

中断只说明队列可能有新工作。驱动必须读取完成环、验证 phase、标签和 generation，取得 R37 的状态与完成长度，再解除其 DMA 映射并发布终态。之后的唤醒才对应 R37。

练习 9.2 分析超时后的 buffer 寿命

R37 超时，但控制器仍可能执行旧描述符。说明为什么设置错误码还不够，以及在哪个事件后页面 P 才能交给其他用途。

解答 9.2 分析超时后的 buffer 寿命

错误码只改变软件观察，不撤销设备访问。驱动需要成功取消、排空队列，或复位设备并使旧映射失效；确认设备不能再 DMA 后才能解除 D37，随后页面 P 才可复用。

练习 9.3 解释发布屏障的位置

若驱动先写 doorbell，再补写描述符长度，设备可能观察到什么？正确顺序中的屏障保护哪一条因果关系？

解答 9.3 解释发布屏障的位置

设备可能按新 tail 取得槽位，却读到旧长度或混合字段。正确顺序先完成全部描述符写入，再执行 DMA 发布屏障，最后写 doorbell；它保证设备看到通知后能够看到此前发布的字段。

尚未解决的问题。单个驱动已经能跟踪请求与完成，但不同块设备具有不同队列深度、请求粒度和持久化命令，进程也不能直接持有驱动内部对象。下一章先建立公共块请求层，再建立 fd、打开文件对象和 VFS 分发。

第 10 章 为什么设备请求需要公共块层与文件接口

继承的机器。驱动已经有请求身份、DMA 所有权、完成与复位边界。上层仍不能把任意大小的文件区间直接变成设备命令，也没有稳定对象表达“某进程以何种方式打开了哪个持久对象、当前偏移是多少、操作应分派给谁”。

从 bio 到用户 buffer，I/O 要穿过块层、调度器和 VFS。本章把 request 队列、merge/sort、fd→file→inode 链和 buffered/direct 路径串成一条线。把这条线走通，才能说清 `write` 为什么快返回、`fsync` 为什么慢，以及掉电后哪些数据会丢。

块请求从何而来

文件系统最终要把逻辑块读写交给块设备。若每个上层请求都直接变成一次磁盘操作，系统会遭遇两个问题：机械磁盘上寻道开销巨大，随机请求顺序会让吞吐崩溃；即使在 SSD/NVMe 上，请求大小、队列深度、合并策略和 `flush` 顺序也直接影响延迟和持久化语义。因此操作系统在文件系统和驱动之间放置块层，负责把 bio 合并、拆分、排序、限流、派发和完成。

块层的目标不是“越重排越好”。对普通读写，重排可以提高吞吐；对日志提交、数据库事务和文件系统 barrier，错误重排会破坏崩溃一致性。I/O 调度器必须同时理解性能和顺序约束：哪些请求可合并，哪些可越过，哪些必须在 `flush` 后才能报告完成。

核心思想

块层把“文件系统想读写哪些块”转化为“设备能高效且按约束完成的请求序列”。它是性能优化层，也是持久化契约层。

从设备演进看，机械磁盘偏好按扇区顺序扫描；SATA SSD 降低了寻道成本但仍受擦写块和队列深度影响；NVMe 提供多队列并行，单一全局电梯队列反而会成为瓶颈。教材不能只讲一个 SCAN 算法就结束，而要说明为什么算法随硬件变化。

块层常用对象如下：

对象	含义	关键字段
bio	上层的一段块 I/O 意图	sector、len、page list、op、flags、endio
request	设备队列中的可派发请求	合并后的 bio 链、方向、优先级、deadline
request queue	块设备的软件队列	pending、inflight、depth、scheduler
hardware queue	多队列设备的硬件提交队列	doorbell、completion、CPU 亲和
flush request	缓存刷新或持久化屏障	preflush、FUA、barrier dependency

一次写入的路径可以写成：

```
filesystem_writeback(page):
    bio = build_bio(page.block, page.data, WRITE)
```

```
submit_bio(bio)

submit_bio(bio):
    q = bio.device.queue
    if can_merge(q.last_request, bio):
        merge(q.last_request, bio)
    else:
        req = make_request(bio)
        scheduler_insert(q, req)
    dispatch_if_possible(q)
```

这里的关键是 bio 的生命周期。bio 完成并不等于调用者数据结构一定可释放，除非 endio 回调已经把 page 状态、错误码和等待者唤醒都处理完。对写请求而言，“设备报告完成”也不必然等于掉电后数据存在，除非请求带有 FUA 或之前执行了正确 flush。

合并、拆分与 I/O 调度

合并与拆分

合并减少请求数量，提高吞吐。前向合并是新 bio 紧接已有请求后方；后向合并是新 bio 紧接已有请求前方。

```
try_merge(req, bio):
    if req.op != bio.op or req.device != bio.device:
        return false
    if req.end_sector == bio.start_sector:
        append(req, bio)
        return true
    if bio.end_sector == req.start_sector:
        prepend(req, bio)
        return true
    return false
```

合并复杂度取决于查找结构。只看队尾是 $O(1)$ ，但合并机会少；用按 sector 排序的树查找相邻请求是 $O(\log n)$ ，合并率更高。拆分发生在请求跨越设备最大段数、最大字节数、DMA 边界或 zone 限制时。教学模型可以只实现连续扇区合并，但要保留“设备约束可能要求拆分”的接口。

SCAN 和 deadline

SCAN 电梯算法按当前磁头方向服务请求，到尽头后反向。它适合解释机械磁盘为什么怕随机寻道。

```
pick_scan(queue, head, direction):
    candidates = requests_in_direction(queue, head, direction)
    if empty(candidates):
        direction = reverse(direction)
        candidates = requests_in_direction(queue, head, direction)
    return nearest_by_sector(candidates, head, direction)
```

SCAN 的吞吐较好，但远处请求可能等待很久。deadline 调度器给读写请求设置截止时间，选择时优先处理过期请求，否则继续按 sector 顺序提升吞吐。

```
pick_deadline(queue):
    if has_expired(queue.read_deadline):
        return pop_oldest(queue.read_fifo)
    if has_expired(queue.write_deadline):
        return pop_oldest(queue.write_fifo)
    return pop_by_sector_order(queue)
```

deadline 的本质是用最大等待时间约束吞吐优化。它不保证实时性，但能避免单纯电梯算法在混合负载下让读请求被大量写请求拖垮。

CFQ/BFQ 与公平性

桌面和多租户环境还需要进程间公平。按进程或控制组建立 I/O 队列，可以避免一个大顺序写任务压制交互读。BFQ 这类预算公平队列给每个实体分配服务预算，用“服务了多少字节”和“等待了多久”共同决定下一个队列。

```
pick_fair(queue_set):
    entity = min_virtual_finish_time(queue_set.active)
    req = entity.pop_next_request()
    entity.service += req.bytes
    entity.vfinish = entity.vstart + entity.service / entity.weight
    if entity.has_more():
        reinsert(entity)
    return req
```

公平调度会牺牲部分全局顺序性，尤其在机械磁盘上可能增加寻道。工程选择取决于目标：批处理吞吐、桌面响应、数据库尾延迟还是云主机隔离。

多队列、flush 与 FUA

flush、FUA 与 barrier

现代存储有缓存。普通写完成可能只表示数据到达设备缓存，不表示掉电持久。flush 请求要求设备把缓存写入介质；FUA 要求当前写绕过或强制落到持久介质；barrier 规定顺序，防止前后的写被重排穿越。

```
submit_ordered_write(data_req):
    if data_req.requires_preflush:
        submit(FLUSH)
        wait_flush_done()
    submit(data_req with FUA if requested)
    wait_data_done()
    if data_req.requires_postflush:
        submit(FLUSH)
        wait_flush_done()
    complete_to_filesystem()
```

这段逻辑解释了为什么 `fsync` 慢：它不只是把 page cache 交给设备，还可能要求等待队列中相关写完成，并发出 flush 来建立掉电后的顺序证据。

多队列 NVMe

NVMe 设备支持多个 submission/completion queue。块层会把请求映射到 CPU 本地硬件队列，减少全局锁竞争。

```
submit_nvme(req):
    hctx = map_cpu_to_hw_queue(current.cpu)
    tag = allocate_tag(hctx)
    cmd = build_nvme_command(req, tag)
    sq = hctx.submission_queue
    sq.entries[sq.tail] = cmd
    sq.tail = next(sq.tail)
    ring_doorbell(hctx)
```

多队列提高并行度，但也削弱了全局排序。需要顺序语义的请求必须带明确依赖，不能假设“提交顺序就是完成顺序”。

blk-mq 对象关系

blk-mq 把软件提交队列映射到硬件队列，减少单锁瓶颈。核心对象包括 `request_queue`、`blk_mq_tag_set`、hardware context、software context 和 tag。tag 是请求在硬件队列中的身份，用 completion 找回 request。

```
submit_bio
-> blk_mq_submit_bio
-> get request tag
-> map current CPU to hctx
-> driver queue_rq
-> device completion interrupt
-> blk_mq_complete_request
-> bio end_io
```

分析多队列块层时，应沿着 request 的完整生命周期观察：取得空闲 tag，进入 in-flight，由设备完成，再释放 tag。任何错误或超时路径都必须保证同一 tag 不会同时代表两个请求。

限流、统计与错误完成

I/O 统计从哪里来

`/proc/diskstats`、`/proc/[pid]/io`、`iostat` 看到的数字来自块层和任务 I/O 统计。它们能说明请求数、扇区数、队列时间、服务时间，但不能直接告诉你应用层哪个业务请求慢。需要把应用 trace、系统调用和块层事件串起来。

指标	解释注意点
read/write IOPS	请求次数，受合并影响，不等于应用调用次数
sectors	数据量，可和吞吐换算
await/latency	队列等待加服务时间，受队列深度影响
util	设备忙碌时间比例，多队列设备上不等同饱和度和

错误诊断不要只看 util。NVMe util 高不一定坏，机械盘 util 高更可能意味着排队；await 上升要结合队列深度、flush、写回和 cgroup 限流。

I/O 优先级和 cgroup

`ionice`、I/O priority class 和 blk-cgroup 可影响不同任务的 I/O 服务顺序。云主机和容器环境常通过 cgroup v2 的 io controller 控制权重或限速。公平性会改变尾延迟：低权重任务可能明显变慢，高权重任务不应被后台写回完全淹没。

writeback、dirty throttling 与块层的关系

应用 `write` 把页标脏后，脏页并不会无限积累。内核用 dirty ratio、dirty bytes、writeback 线程和 per-bdi 状态控制写回。当脏页过多时，写入者可能被迫参与平衡，表现为一次普通 `write` 变慢。

```
balance_dirty_pages(task, mapping):
    dirty = global_dirty_pages()
    if dirty < dirty_background:
        return
    wake_writeback_threads()
    if dirty > dirty_limit:
        sleep_or_writeback_until_below_limit(task)
```

这个机制把存储压力反馈给应用。没有 throttling, 快 CPU 可以瞬间制造大量脏页, 把内存挤爆; 有 throttling, 局部写入延迟会上升, 但系统保持可控。诊断写慢时要区分: 是用户态复制慢, page cache 分配慢, dirty throttling, 块层排队, 还是设备 flush。

zone 设备和顺序写约束

某些设备把空间划成 zone, 并要求一个 zone 内按顺序写入。文件系统和块层必须知道 zone write pointer, 不能随意把随机写重排进去。这个例子说明块层不只是性能优化, 还承载设备语义。

```
submit_zone_write(req):
    zone = lookup_zone(req.sector)
    if req.sector != zone.write_pointer:
        return -EIO
    dispatch(req)
    zone.write_pointer += req.length
```

对普通块设备, 逻辑块可随机覆盖; 对 zone 设备, 文件系统可能要采用 log-structured 布局或显式 reset zone。硬件约束改变, 上层算法就必须改变。

请求取消、超时与错误完成

块请求提交后可能超时、被设备 reset、被上层取消或完成失败。取消不是“把请求从队列里删除”这么简单, 因为请求可能已经在设备硬件队列中, 甚至设备正在 DMA。

```
timeout_request(req):
    mark req timed_out
    ask driver eh_timed_out(req)
    if driver says reset:
        freeze_queue()
        reset_device()
        complete_lost_requests(EIO)
        unfreeze_queue()
    else if request completed meanwhile:
        ignore timeout
```

这里存在竞态: timeout 线程准备处理请求时, 中断也可能完成请求。request 状态机必须保证 completion 只发生一次, tag 只释放一次, bio endio 只调用一次。块层测试要专门构造 timeout 和 completion 交错。

从 fio 数字到 OS 解释

I/O benchmark 不能只报 MB/s 和 IOPS。高质量报告要写明 direct/buffered、sync/async、iodepth、block size、read/write mix、文件系统、是否预热、是否清缓存、是否包含 fsync。

参数	含义	影响
bs	每次 I/O 大小	合并、吞吐和延迟
iodepth	in-flight 请求数量	设备并行度和排队
direct	是否绕过 page cache	缓存干扰和对齐要求
fsync	是否等待持久化	flush 延迟和事务成本

rw	顺序/随机、读/写混合	调度器和介质特性
----	-------------	----------

如果不说明这些参数，“磁盘很快”或“磁盘很慢”没有工程意义。OS 学习里的性能测试必须可复现，也必须能解释观测值背后的路径。

常见误区

本章块层部分的合格标准：能说明 bio、request、queue 的区别；能分析 SCAN、deadline、公平调度的取舍；能解释 flush/FUA/barrier 为什么属于正确性而不仅是性能。

fd、file、dentry 与 inode

文件描述符是用户态访问内核对象的句柄。它可以指向普通文件、管道、socket、设备或匿名内核对象。进程拿到的是一个小整数，但内核维护的是对象、offset、权限、引用计数和关闭状态。

先看一段最普通的代码：

```
int fd = open("log.txt", O_CREAT | O_APPEND | O_WRONLY, 0644);
write(fd, "hello\n", 6);
close(fd);
```

初学时容易把它理解成三步：打开文件、把 6 个字节写到磁盘、关闭文件。真实路径更长：路径字符串要解析成目录项和 inode；open 要创建 open file description 并放入进程 fd 表；write 要验证 fd 权限、复制用户 buffer、更新文件 offset、修改 page cache 或提交设备请求；close 只是释放一个引用，不保证已经持久落盘。

把错误版本写出来更清楚：

```
write(fd, buf, len):
    inode = fd_to_inode(fd)
    disk.write(inode.block, buf, len)
    return len
```

这个版本至少错在五处。第一，fd 不直接指向 inode，中间还有 open file description，它保存 offset 和 status flags。第二，O_APPEND 必须在内核里原子决定写入位置，不能让用户态先 lseek。第三，普通文件写通常先进 page cache，write 返回不等于磁盘完成。第四，写入可能短写或被信号打断。第五，错误可能异步出现，例如后台 writeback 失败，之后 fsync 才报告。

文件 I/O 的第一层对象关系是：路径字符串经过路径解析找到目录项和 inode；open 产生 open file description；进程 fd 表保存一个小整数到 open file description 的引用。dup 产生新的 fd，但通常仍指向同一个 open file description，因此共享 offset 和状态。再次 open 同一路径通常产生新的 open file description，因此 offset 分离。

普通文件读写经常经过 page cache。读命中 page cache 时不碰设备；读未命中时可能提交块设备请求并等待完成。写通常先把用户字节复制进 page cache，把页标记为 dirty，再由后台 writeback 或 fsync 推进到设备。write 返回不等于数据安全落盘，这一点必须反复强调。

设备可以看作有特殊行为的文件对象。终端、磁盘、网卡、timerfd、eventfd、epoll、pipe、socket 都可以放进 fd/event 模型，但它们的 read/write 语义不同：有的返回字节流，有的返回结构化事件，有的可能阻塞，有的可能短读短写，有的只表示 readiness。

阻塞、非阻塞和异步是三种控制流模型。阻塞 I/O 让当前线程睡眠直到有结果；非阻塞 I/O 在不能推进时返回 `EAGAIN`，调用者等待 readiness 后重试；异步 I/O 提交请求，completion 后返回结果。三者没有绝对高低，关键是请求身份、buffer 生命周期、短读短写、错误传播和背压。

背压是 I/O 系统的安全阀。无界队列能让短 benchmark 看起来很快，因为生产者从不等待；一旦设备变慢，内存会持续增长，尾延迟扩大，最终进程可能被杀。可靠系统要同时限制 item 数、字节数和 in-flight 请求数，并在关闭时 drain 或 cancel。

核心思想

fd 是入口，file 是打开实例，inode 是文件身份，page cache 是缓存和脏页账本，block request 是设备请求。文件 I/O 的连贯理解来自把这几层一条线串起来，而不是把 `read/write` 当成直接磁盘函数。

fd 分配表

进程的 fd table 可以先看作数组：下标是 fd，元素是指向 open file description 的引用。open 要找最小空闲 fd；close 清空槽位并减少引用计数；dup 找新槽位并增加引用计数。

```
alloc_fd(file):
    for i in 0..max_fd:
        if table[i] == null:
            table[i] = file
            file.refcount += 1
            return i
    return -EMFILE

close(fd):
    file = table[fd]
    if file == null: return -EBADF
    table[fd] = null
    file.refcount -= 1
    if file.refcount == 0:
        file.ops.release(file)
```

线性扫描是 $O(n)$ ，但简单。真实系统会用 bitmap 或 hint 记录下一个可能空闲位置，把常见分配降到接近 $O(1)$ 。无论怎么优化，不变量都是：fd 只是进程局部索引，open file description 才保存 offset、flags 和对象引用。

路径解析

路径解析把字符串变成目录项和 inode。绝对路径从根目录开始，相对路径从当前工作目录开始；每个路径分量都要查目录；遇到符号链接可能要跳转；权限和 mount namespace 会改变可见结果。

```
resolve(path):
    node = path.starts_with("/") ? root : current.cwd
    for component in split(path, "/"):
        if component == "." continue
        if component == "..": node = node.parent
        else:
            node = lookup_child(node, component)
            if node is symlink:
                node = resolve_symlink(node)
    return node
```

路径解析的复杂度大致和路径分量数相关，每个目录查找又取决于文件系统目

录结构。缓存 dentry 可以避免每次都从磁盘读目录。安全实现还要限制符号链接递归深度，避免循环。

fd、file 和 inode 的并发引用

fd 是进程 fd 表下标，不是对象身份。内核执行 `read(fd)` 时，先把 fd 转成 `struct file` 引用；只要引用拿到，即使另一个线程随后 `close(fd)`，当前 I/O 也能在 file 对象上完成。close 删除的是 fd 表槽位，并减少 file 引用；它不能把正在执行的 I/O 用到的 file 直接释放。

```
sys_read(fd, buf, len):
    file = fget(fd)
    if file == null:
        return -EBADF
    ret = vfs_read(file, buf, len)
    fput(file)
    return ret

sys_close(fd):
    file = files_table_remove(fd)
    if file == null:
        return -EBADF
    fput(file)
    return 0
```

这个规则解释 fd reuse 风险：线程 A 保存整数 fd=5，线程 B `close(5)`，线程 C `open` 得到新的 fd=5。A 再用整数 5 时，访问的可能已经是另一个对象。内核靠 file 引用保护内部生命周期，用户态也要避免把裸 fd 长期跨线程传递而没有所有权约定。

open flags 的作用域

flag 不是都绑定在同一层。fd flag 绑定 fd 表项；file status flag 绑定 open file description。dup 后两个 fd 指向同一个 open file description，所以共享 file status flag 和 offset；但 close-on-exec 是 fd flag，每个 fd 可以不同。

标志	作用范围	边界
<code>O_CLOEXEC</code>	fd 表项	exec 提交时关闭该 fd，不影响其他 fd 的标志
<code>O_NONBLOCK</code>	open file description	dup 后共享，影响 read、write、accept、connect 的等待行为
<code>O_APPEND</code>	open file description	每次写前在内核内原子移动到文件尾
<code>O_DIRECT</code>	file I/O 路径	对齐、页 pin、缓存一致性和短 I/O 更复杂
<code>O_SYNC/O_DSYNC</code>	写入持久化语义	返回前要推进数据或元数据到更强边界

`O_APPEND` 不是用户态先 `lseek` 到末尾再 `write`。后者在并发下会竞争：两个线程都看到同一个旧文件尾，然后互相覆盖。append 语义要求内核在持有文件系统相关锁的状态下决定写入位置。

Linux VFS 的关键对象关系

Linux VFS 以 fd table、file、dentry、inode 和 file operations 组织打开文件；page cache 承载普通文件的缓存页，epoll 和 io_uring 则在对象引用之上建立事件与异步完成关系。理解实现时应跟住一个 fd：它如何从整数索引解析为 file，引用如何增加，操作如何进入具体 file operations，错误如何返回，以及最后一个引用在何时释放。

缓冲 I/O、直接 I/O 与异步完成

VFS 的 read/write 并不直接落到设备。普通文件读写经过 page cache，未命中时才生成 bio 进入块层；direct I/O 绕过 page cache 但仍走块层提交。本节把缓存命中与未命中、buffered 与 direct、同步与异步的路径分别拆开。

page cache 读写

普通文件读路径先查 page cache。若缓存命中，从内核页复制到用户 buffer；若未命中，分配 page cache 页，提交块 I/O，等待完成，再复制。写路径通常复制用户数据到 page cache，标记 dirty，并在之后由 writeback 或 fsync 推进设备写。

```
buffered_read(file, offset, len):
    while len > 0:
        page_index = offset / PAGE_SIZE
        page = page_cache_lookup(file, page_index)
        if page == null:
            page = read_page_from_storage(file, page_index)
        n = copy_page_to_user(page, offset % PAGE_SIZE, len)
        offset += n
        len -= n
```

这个算法解释了冷读和热读差异。热读可能完全不碰设备；冷读可能等待存储。benchmark 若不说明缓存状态，就无法比较。

把一次冷读走完整。进程调用 `read(fd, buf, 6000)`，当前文件 offset 是 3000，页大小是 4096。第一次要读的是文件第 0 页的后 1096 字节，第二次要读第 1 页的前 4096 字节，最后还要读第 2 页的 808 字节。若这三页都不在 page cache，内核要为每页建立 page cache 条目，发起块 I/O，把线程挂到等待队列。设备 completion 到来后，页状态从 locked/loading 变成 uptodate，等待线程被唤醒，再把页内容复制到用户 buffer。

阶段	page cache 状态	线程看到什么
查第 0 页	miss，分配 cache 页并标记 locked	不能直接返回，需要等待 I/O
提交 I/O	块层生成 request，设备 DMA 填页	线程 sleeping，不占 CPU
completion	页标记 uptodate，清 locked，唤醒等待者	线程变 runnable，不一定立即运行
复制用户 buffer	从 page cache 页复制对应片段	可能再次遇到用户页 fault
推进 offset	成功复制多少，offset 推进多少	短读时不能假设请求全完成

热读路径不同：若页已经 uptodate，内核只需要拿到页引用并复制到用户 buffer，不发设备 I/O。这里的“快”来自 page cache 保存了文件内容的内存副本，而不是文件系统跳过权限和边界检查。读性能测试若第一轮冷读、第二轮热读，却

把两者当同一种读，就会得出错误结论。

写路径可以按这条线理解：

```
sys_write(fd, user_buf, len):
    file = fget(fd)
    check file allows write
    decide offset, handling O_APPEND under lock
    copy bytes from user into page cache pages
    mark pages dirty
    update file size if needed
    return copied bytes

later:
    writeback finds dirty pages
    filesystem maps file offsets to blocks
    block layer submits requests
    device completes DMA
    fsync reports delayed writeback errors
```

这条线解释了三个常见困惑。第一，`write` 很快返回，是因为它可能只完成了“复制到内核缓存并标脏”。第二，系统掉电时，dirty page 还没写到持久介质就会丢。第三，`fsync` 的意义不是“再写一遍”，而是把先前的脏数据和必要元数据推进到更强的持久化边界，并把异步错误交还给调用者。

设备队列和完成

块设备、网卡和异步 I/O 都可以抽象成提交队列和完成队列。提交时生成 request，放入设备或驱动队列；完成时中断或轮询拿到结果，唤醒等待任务或投递 completion。

```
submit(req):
    req.id = next_id()
    req.state = IN_FLIGHT
    device_queue.push(req)
    kick_device()

complete(req_id, status, bytes):
    req = lookup(req_id)
    req.status = status
    req.bytes = bytes
    req.state = COMPLETED
    wake(req.waiter)
```

请求身份非常重要。异步完成可能乱序返回；取消后的请求也可能迟到完成；buffer 所有权必须跟着请求身份走。没有 request id，就无法把 completion 正确对应到提交者。

短读短写不是异常

普通文件顺序读常尽量返回请求长度，但很多 fd 都允许短读短写。pipe 可能只剩一部分数据；socket 是字节流，接收缓冲里有什么就先给什么；非阻塞 fd 无法继续推进时返回 `EAGAIN`；信号可能让系统调用返回 `EINTR`；存储错误可能让已经完成的部分和错误同时出现。

```
write_all(fd, buf, len):
    done = 0
    while done < len:
        n = write(fd, buf + done, len - done)
        if n > 0:
            done += n
```

```

    continue
    if errno == EINTR:
        continue
    if errno == EAGAIN:
        wait_writable(fd)
        continue
    return error
return done

```

读路径也需要协议层状态机。对 TCP, `recv` 返回 20 字节不表示一个业务消息完整；对 UDP, 一次 `recvmsg` 对应一个 datagram, buffer 太小会截断；对终端, 行规程可能改变返回时机。OS 提供 fd 语义, 应用协议必须自己定义消息边界。

短写也要用具体事故理解。假设应用要把 1MiB 日志写到非阻塞 socket, 第一次 `write` 返回 64KiB, 第二次返回 `EAGAIN`。如果应用把第一次返回当作“整条日志已经发送”, 就会丢掉后 960KiB; 如果它在 `EAGAIN` 时忙循环重试, 会把 CPU 打满; 如果它没有保存剩余 buffer 和当前偏移, 等 socket 可写时也不知道从哪里继续。正确实现需要一个输出队列, 记录每条待发送数据、已发送 offset 和连接关闭时的清理策略。

```

connection_on_writable(conn):
    while !conn.outq.empty():
        item = conn.outq.front()
        n = write(conn.fd, item.buf + item.off, item.len - item.off)
        if n > 0:
            item.off += n
            if item.off == item.len:
                conn.outq.pop_front()
                continue
        if errno == EAGAIN:
            arm_epollout(conn)
            return
        if errno == EINTR:
            continue
        fail_connection(conn, errno)
    return

```

这段代码把“短写不是异常”落实成状态机：数据没写完时必须保留；`EAGAIN` 表示当前 fd 暂不可推进；真正错误才关闭连接或上报。文件 fd、pipe、socket、终端的短 I/O 原因不同, 但调用者都不能假设一次系统调用等于业务消息完整提交。

buffered I/O 和 direct I/O 的不同风险

buffered I/O 经过 page cache, 适合普通应用和共享缓存；direct I/O 尽量绕过 page cache, 适合数据库一类自己管理缓存和写入顺序的程序。direct I/O 不是“更高级的 read/write”, 它把对齐、页 pin、设备队列、缓存一致性和错误处理暴露得更多。

路径	优点	代价
buffered read	命中 page cache 很快, 可共享缓存	多一次复制, 缓存污染可能影响其他任务
buffered write	快速标脏, 后台批量写回	write 返回不代表持久化
direct read/write	减少 page cache 干扰, 适合自管缓存	对齐、pin 页、短 I/O 和并发一致性复杂

mmap	load/store 直接访问映射 页	缺页、SIGBUS、 dirty/writeback 语义 更隐蔽
------	------------------------	---

direct I/O 和 buffered I/O 混用尤其危险。一个路径绕过 page cache，另一个路径读写 page cache，若文件系统没有正确失效或同步缓存，应用可能看到旧数据。真实系统会在文件系统层处理这类一致性，但应用设计仍应尽量避免无计划混用。

epoll item 绑定的是 file

`epoll_ctl` 把一个 fd 对应的 file 和 interest mask 加进 epoll 实例。之后整数 fd 关闭并复用，不代表旧 epoll item 自动变成新文件的监听项。事件返回里带着用户当初登记的 fd 数字，但内核里的等待关系绑定的是旧 file。

```
epoll_ctl(epfd, ADD, fd):
    file = fget(fd)
    epitem.file = file
    epitem.fd_number_for_user = fd
    attach_poll_callback(file, epitem)

event delivery:
    file state changes
    callback enqueues epitem
    epoll_wait returns stored fd_number and event mask
```

事件循环通常要给每个连接对象加 generation 或指针身份，而不是只根据 fd 整数找状态。close、dup、fork、exec、线程并发都可能让 fd 数字的直觉失效。

io_uring 的提交、完成与取消

io_uring 用共享提交队列 SQ 和完成队列 CQ 减少系统调用往返，支持固定 buffer、固定文件、poll、timeout、链式请求和异步取消。它不是“绕过内核”，而是把提交和完成批量化，让内核在更少入口中处理更多请求。

io_uring 把“提交请求”和“收完成结果”拆成两个环。SQE 中的 `user_data` 是应用识别 completion 的关键；CQE 中的 `res` 是最终字节数或负 `errno`。提交成功只表示内核接收了请求，不表示 I/O 完成。

```
userspace:
    sqe = get_sqe()
    sqe.op = READV
    sqe.fd = fixed_file
    sqe.buf = fixed_buffer
    sqe.user_data = request_id
    submit_sq_tail()
    io_uring_enter()          # or rely on SQPOLL

kernel:
    consume SQEs
    issue async work or direct submit
    post CQEs with res = bytes or -errno
```

fixed buffer/file 提高性能，但也把生命周期责任推给应用：buffer 在 completion 前不能释放，取消可能和完成竞态，部分完成必须按 CQE 结果处理。

取消请求要按竞态理解：取消到达时，请求可能还在队列里，可能已经派发给设备，可能刚刚完成。应用必须能接受“取消成功”“取消失败因为已经完成”“先收到完成后收到取消结果”等情况。只要 completion 可能到来，buffer 和请求对象就不能提前复用。

更具体地说，`io_uring` 取消不是“把那次 I/O 从世界上抹掉”。假设应用提交读请求 R，`user_data=42`，`buffer` 指向一块复用池内存。100ms 后应用超时，提交 `cancel(42)`。此时可能有三种真实状态：R 还在内核队列，取消可以把它摘掉；R 已经交给块设备，设备可能无法撤销，只能等 `completion`；R 已经完成，CQE 还没被应用收走。应用若在提交 `cancel` 后立刻把 `buffer` 给另一个请求使用，就可能和迟到的 R `completion` 冲突。

取消到达时	内核可能返回什么	应用必须怎样处理
请求未派发	<code>cancel</code> CQE 表示取消成功，原请求不会再完成	可以释放 <code>buffer</code> ，但仍要消费 <code>cancel</code> 结果
请求已派发	<code>cancel</code> 可能失败或标记取消中，原请求仍会完成	<code>buffer</code> 继续保持有效，直到看到原请求 CQE
请求已完成	<code>cancel</code> 返回未找到或已完成	根据原 CQE 的结果处理，不重复释放
完成和取消乱序到达	先看到哪一个都可能	用 <code>request</code> 状态机合并结果，避免双重释放

所以异步 I/O 的基本规则是：提交记录、`buffer` 所有权和 `completion` 消费必须绑定。真正能释放或复用 `buffer` 的点，不是“我不想等了”，而是“所有可能引用这个 `buffer` 的请求状态都已经收敛”。这和驱动层 DMA request 的规则是同一件事，只是对象从设备 `descriptor` 换成了用户可见的 `SQE/CQE`。

一次文件访问的跨层路径

第一项分析是画出 `fd` 引用图。

```
process fd table:
fd 3 -> open file description A -> inode X
fd 4 -> open file description A -> inode X    # dup
fd 5 -> open file description B -> inode X    # open same path again
```

这张图能解释 `offset` 行为：`fd 3` 和 `fd 4` 共享 `offset`，`fd 5` 有独立 `offset`。并行文件读取如果使用 `fd 3` 和 `fd 4`，就可能因为共享 `offset` 得到不可预期的分片；使用 `pread` 或独立 `open file description` 更清楚。

第二项分析是写出短写循环。任何 `write` 都可能只写入一部分字节，尤其是 `pipe`、`socket`、非阻塞 `fd` 或被信号打断时。正确代码必须记录已完成字节数，继续写剩余部分，遇到 `EINTR` 重试，遇到 `EAGAIN` 进入等待，不能把短写当成完整成功。

第三项分析用于区分 `readiness` 和 `message`。`epoll` 告诉进程 `fd` 可能可读或可写，并不表示协议帧已经完整。网络程序仍要维护用户态 `receive buffer`、解析状态机和背压策略；把 `fd readiness` 当成完整请求，会破坏事件驱动程序的消息边界。

```
PreparedBuffer
-> Submitted
-> InFlight
-> Completed(bytes, error)
-> ConsumedByProtocol
-> BufferReleased
```

异步 I/O 中，`buffer` 在 `completion` 前不能被上游复用；`completion` 失败不能被提交路径吞掉；`cancel` 后也可能收到迟到 `completion`，所以请求身份必须稳定。

短读写、取消与引用生命周期

块层侧的测试覆盖合并、拆分、调度、flush 顺序和多队列身份：

1. 合并测试：连续写 bio 应合并成较少 request，非连续不得错误合并。
2. 拆分测试：超过最大请求大小时必须拆分，完成回调仍只向上层报告一次整体结果或清晰的部分错误。
3. SCAN 测试：给定 head 和方向，选择顺序可手算。
4. deadline 测试：过期读请求应越过大量未过期写请求。
5. flush 顺序测试：日志数据、flush、commit block 的完成顺序不能被重排破坏。
6. 多队列测试：同一队列 tag 唯一，completion 能找到原请求。
7. 错误注入：设备返回 I/O error 时，page、bio、request、等待者状态都能收敛。

文件 I/O 侧的测试覆盖 open/write/read、dup 后共享内容、close 一个 fd 后另一个 fd 仍有效、关闭所有引用后访问失败、未知 fd 返回诊断。还要测试 fd 分配单调或复用策略是否有明确规则。

- dup 后 offset 共享，独立 open 后 offset 分离。
- pread 不改变 current offset。
- pipe 空读阻塞或非阻塞返回 EAGAIN，写端关闭后读端得到 EOF。
- 读端关闭后写 pipe 失败，不应静默丢数据。
- socket 或 pipe 短写后，调用者继续写剩余字节。
- readiness 事件只触发解析尝试，不被当成完整消息。
- 异步请求完成前，buffer 所有权不能释放。
- 队列达到字节上限时，生产者受到背压。

第一版必须先保证句柄生命周期可解释。持久化和崩溃恢复不要另起一条线，而要回到 fd、file、inode、page cache、block request 和目录项这些对象上验收。

VFS 与文件系统的持久化边界

VFS 给用户一个统一 fd 接口，具体文件系统负责目录项、inode、块映射、权限、日志和恢复。路径解析得到 dentry/inode，open 生成 file，read/write 进入 file operations，普通文件再落到 address_space、page cache 和文件系统块映射。主线里先抓住对象边界，而不是先背某个文件系统格式。

对象	保存什么	关键不变量
dentry	名字到 inode 的缓存关系	rename/unlink 后缓存不能让旧名字错误复活
inode	文件身份、权限、大小、块映射、时间戳	多个路径和 fd 可引用同一 inode
file	一次打开实例、offset、status flags、ops	dup 共享，独立 open 分离

address_space	文件页缓存和写回状态	read/write/mmap 必须看到同一份 page cache 账本
journal/log	元数据或数据提交记录	崩溃恢复只能重放完整事务

文件系统恢复的核心不是“写得更勤”，而是“崩溃后能判断哪些写已经构成完整状态”。以创建文件并重命名为例，数据块、inode 大小、目录项、父目录元数据、日志 commit 和设备 flush 之间必须有顺序。若应用写临时文件、`fsync(tmp)`、`rename(tmp, final)`，但忘记 `fsync` 父目录，崩溃后目录项是否存在就依赖文件系统和挂载语义，不能当作通用保证。

```

durable_replace(tmp, final):
    write tmp contents
    fsync(tmp_fd)
    rename(tmp, final)
    fsync(parent_dir_fd)

```

这段不是所有场景的万能配方，但它展示了恢复协议的写法：每一步都要说明崩溃发生后恢复程序能看到什么。若看到旧文件，旧版本必须仍完整；若看到新名字，新内容必须已经同步；若看到临时文件，恢复程序应能删除或继续提交。文件系统、块层和设备只提供若干提交边界，业务正确性还要靠应用自己的版本号、校验和、manifest 或事务日志。

常见误区

文件与 I/O 章的最终验收：跟住一个字节从用户 buffer 到 page cache、bio、request、设备 completion、writeback error、`fsync` 返回和恢复程序判断。若不能指出每层的状态对象和错误返回点，就不能声称理解了文件 I/O。

从文件对象到持久块的综合验证

I/O 与文件系统深讲：从 fd 到磁盘块

文件系统和 I/O 是许多教材最难讲清的部分，因为它们跨越对象模型、缓存、设备、并发和持久化。用户看到的是 `read`、`write`、`open`、`close`，内核看到的是 fd 表、file、inode、page cache、bio、request、DMA 和 completion。

一次 buffered read 的完整路径。

```

read(fd, user_buf, len)
-> fd table lookup
-> file object permission check
-> inode and page cache lookup
-> if cache miss: submit block I/O
-> sleep waiting for completion
-> driver interrupt completes request
-> wake reader
-> copy page cache bytes to user buffer
-> update file offset
-> return bytes read

```

这个路径里有三个不同的“成功”。块设备成功读取页面，不代表用户 buffer 已经复制；用户复制成功，不代表文件 offset 更新一定已经对其他共享 fd 可见；`read` 返回字节数，也不代表下一次读不会因为 EOF、信号或错误返回不同结果。OS 教材必须不断区分这些边界。

一次 buffered write 的完整路径。

`write` 更容易误解。普通 buffered write 通常只是把数据复制到 page cache，并把页标脏。数据什么时候写到设备，由后台 writeback、内存压力、`fsync` 或文件系统策略决定。

```
write(fd, user_buf, len)
-> fd table lookup
-> check write permission
-> copy user bytes into page cache
-> mark page dirty
-> update inode size if needed
-> return bytes written

later:
writeback dirty page
-> build bio
-> block scheduler
-> driver DMA
-> completion
-> mark page clean
```

因此 `write` 返回和持久化不是一回事。可靠程序需要 `fsync` 文件，必要时还要 `fsync` 父目录，并用 `manifest` 表达业务提交。

fd、file、inode 的引用关系。

对象	保存内容	生命周期
fd slot	进程局部整数到 file 的引用	<code>open/dup/close</code> 改变
file object	offset、flags、ops、inode 引用	引用计数为 0 时释放
inode	文件元数据、数据块索引、权限	link count 和 open count 共同决定回收
dentry	名字到 inode 的缓存映射	<code>rename/unlink/mount</code> 会影响有效性

`dup` 后两个 fd 指向同一个 file object，所以共享 offset；独立 `open` 同一路径得到不同 file object，所以 offset 分离；`unlink` 删除目录项，但已打开 file 仍引用 inode，所以文件内容可能继续存在到最后一个 fd 关闭。

块层为什么存在。

文件系统关心逻辑块，设备关心队列、扇区、对齐、最大请求、flush 和 completion。块层在中间做合并、拆分、排序和顺序约束。

```
file system:
write inode block 100
write data blocks 200..240
flush journal commit

block layer:
merge adjacent writes
split too-large requests
enforce flush/FUA order
dispatch to hardware queue
```

没有块层，文件系统要直接理解每种设备队列；没有顺序约束，文件系统日志可能被设备重排破坏；没有合并，吞吐会被小请求拖垮。

文件系统练习。

1. 画出 `dup`、`open`、`fork` 后 `fd`、`file`、`inode` 的引用图。
2. 为什么 `unlink` 一个打开文件后，磁盘空间可能不立刻释放？
3. 为什么 `write` 成功后立刻断电，重启不一定看到新数据？
4. 为什么 `direct I/O` 和 `mmap` 同时访问同一文件会有一致性风险？
5. 设计一个测试，证明 `page cache` 热读没有访问块设备。

持久化故障矩阵：从 `write` 到 `fsync`。

可靠文件输出要把每个阶段的崩溃结果写清楚。

阶段	崩溃后可能看到	正确恢复态度
<code>write</code> 返回后	数据只在 <code>page cache</code> ，磁盘可能无变化	不能宣布业务提交
<code>fsync(file)</code> 后	文件内容和部分元数据持久	若路径是新建文件，还要考虑目录
<code>rename(tmp, final)</code> 后	名字可能切换，目录项未必持久	需要 <code>fsync(parent dir)</code>
<code>manifest</code> 写入前	数据文件可能存在但未被承认	读取者忽略孤儿文件
<code>manifest fsync</code> 后	版本记录可校验	读取者按 <code>manifest</code> 校验并采用

这个矩阵解释了为什么可靠系统常用临时文件、`rename`、目录 `fsync` 和 `manifest`。不是每个程序都需要这么重，但如果程序声称“崩溃后不产生半成品”，就必须定义这些阶段。

文件系统常见误区。

常见误区

- `write` 成功不等于数据落盘。
- `close` 不是可靠提交协议，错误可能来自之前的 `writeback`。
- `rename` 原子可见不等于目录项已经持久。
- 删除路径名不等于已打开文件对象立即消失。
- `page cache` 不是“可有可无的缓存”，它参与 `mmap`、回收和写回。
- `direct I/O` 绕过 `page cache` 不等于绕过所有一致性问题。

本章小结。

- `fd` 是进程局部句柄，`file object` 保存 `offset` 和 `flags`，`inode` 保存文件身份。
- `buffered I/O` 通过 `page cache`，冷/热缓存性能差异巨大。
- 块层负责合并、拆分、排序和 `flush` 顺序。
- 文件系统必须同时维护运行时一致性和崩溃后可恢复性。
- 可靠写出需要明确业务提交点，通常不能只依赖 `write`。

VFS：为什么需要统一文件对象模型。

不同文件系统的磁盘布局差异很大，设备文件、管道、socket 和普通文件也不是同一种存储对象。用户态却希望都能通过 `open/read/write/close` 操作。VFS 的作用是提供统一对象模型：路径解析得到 dentry 和 inode；打开后得到 file object；具体操作通过 file operations 或 inode operations 分发给底层实现。

VFS 对象	语义	例子
superblock	一个挂载文件系统实例	ext4、tmpfs、procfs 的挂载点
inode	文件身份和元数据	普通文件、目录、设备节点
dentry	名字到 inode 的缓存关系	<code>/usr/bin/sh</code> 的路径组件
file	一次打开实例	offset、flags、权限和 ops
mount	命名空间中的挂载关系	bind mount、overlay、chroot 边界

VFS 解释了“一切皆文件”的边界。普通文件的 `read` 走 page cache；管道的 `read` 走内存环形缓冲区；socket 的 `read` 走网络接收队列；设备节点的 `read` 进入驱动。统一 API 不等于统一实现，调用者仍要根据对象类型处理短读、阻塞、错误和 readiness。

路径解析：名字查找不是字符串拼接。

路径解析要逐级查找目录项，处理 `...`、符号链接、挂载点、权限、并发 `rename` 和命名空间边界。它不能简单把字符串 `split` 后查哈希表，因为每一级都可能跨文件系统或触发权限检查。

```

path_lookup(start, path):
    current = start
    for component in split(path):
        if component == ".":
            continue
        if component == "..":
            current = parent_respecting_mount_namespace(current)
            continue
        dentry = lookup_dentry(current, component)
        if dentry.is_symlink:
            path = expand_symlink(dentry, remaining_path)
            restart_with_symlink_limit()
        if crosses_mount(dentry):
            current = mounted_root(dentry)
        else:
            current = dentry.inode
    return current

```

路径查找的失败也要分类。组件不存在返回 `ENOENT`；中间组件不是目录返回 `ENOTDIR`；权限不足返回 `EACCES`；符号链接层数太多返回 `ELOOP`；路径过长返回 `ENAMETOOLONG`。这些错误让调用者知道是创建父目录、修正路径还是请求权限。

目录、`rename` 和原子可见性。

目录是名字到 inode 的映射。它本身也是文件，只是内容由文件系统解释。`rename` 的关键价值是原子可见：观察者要么看到旧名字，要么看到新名字，不应看到半个名字。这个性质让临时文件发布成为可能。

```

atomic_replace(path, data):
    tmp = create(path + ".tmp")
    write_all(tmp, data)
    fsync(tmp)

```

```
rename(tmp, path)
fsync(parent_directory(path))
```

这里的原子可见和持久化仍然不同。`rename` 完成后，其他进程查路径应看到新文件；但断电后目录项是否仍在，要看父目录是否持久化。很多可靠更新协议必须同时关心“运行时读者看到什么”和“崩溃后恢复看到什么”。

page cache 一致性：read、write、mmap。

普通 buffered I/O 和文件 `mmap` 通常共享 page cache。进程 `write` 修改 page cache 页后，另一个进程通过 `mmap` 读同一页，应该看到一致内容；反过来，`mmap` 写共享映射后，`read` 也应看到页缓存中的新数据。这个共享减少了重复缓存，但增加了一致性规则。

访问方式	数据所在位置	一致性关注点
buffered read	page cache 到用户 buffer	热页不访问设备，冷页触发 I/O
buffered write	用户 buffer 到 page cache	返回不等于落盘，页变 dirty
MAP_SHARED mmap	PTE 映射 page cache 页	写入变 dirty，回写时机独立
MAP_PRIVATE mmap	COW 匿名页	写入不改变文件内容
direct I/O	用户页到设备 DMA	与 page cache 的失效/同步边界

direct I/O 试图绕过 page cache，常用于数据库。它减少双重缓存，但要求处理对齐、短 I/O、设备错误和与 buffered/mmap 混用的一致性。若同一文件同时被 direct write 和 buffered read 访问，内核必须失效或同步相关 page cache 页，否则读者可能看到旧数据。

inode 元数据和数据块分配。

文件系统不只保存文件内容，还要保存大小、权限、时间戳、link count、extent 或 block map。写入一个新文件块可能同时改变多个结构：分配位图、inode size、extent tree、目录项和 journal。

```
append_block(file, data):
    block = allocate_free_block()
    update_allocation_bitmap(block)
    write_data(block, data)
    update_inode_extent(file.inode, block)
    update_inode_size(file.inode)
    mark_metadata_dirty(file.inode)
```

崩溃一致性的难点就在这里。如果位图显示块已分配，但 inode 没有引用它，出现空间泄漏；如果 inode 引用块，但位图仍显示空闲，未来可能把同一块分给另一个文件；如果目录项指向未初始化 inode，路径查找会读到坏对象。日志和写序约束用于避免这些不一致状态在重启后被当成合法事实。

日志文件系统的三种常见模式。

日志文件系统把元数据或数据更新先写入 journal，再提交。恢复时根据 commit record redo 完整事务，忽略不完整事务。不同模式在性能和数据保证之间取舍。

模式	写入内容	崩溃后保证
----	------	-------

metadata journal	journal 只记录元数据	文件系统结构一致，文件新数据可能旧或空洞
ordered mode	数据先写，再提交元数据 journal	新元数据不会指向未写入垃圾数据
data journal	数据和元数据都写 journal	语义更强，写放大更高

日志不是万能。设备可能重排写入，所以 commit 前后需要 flush/FUA 等顺序保证；journal 空间有限，所以要 checkpoint；checksum 用于发现 torn write；事务太大影响延迟，太小影响吞吐。文件系统设计的核心是把这些成本变成可解释的提交协议。

块调度算法：从磁头到 NVMe。

传统磁盘有寻道成本，因此电梯算法会按扇区顺序合并和排序请求，减少磁头来回移动。SSD 和 NVMe 没有机械磁头，但仍有队列深度、并行通道、写放大、flush 成本和公平性问题。块调度从“减少寻道”演进到“控制延迟、合并请求、按 cgroup 分配带宽”。

策略	优点	问题
FCFS	简单，保持提交顺序	随机 I/O 性能差，尾延迟不可控
SSTF	优先近距离请求，减少寻道	远处请求可能饥饿
SCAN/elevator	有方向地扫过磁盘	参数依赖介质，交互延迟仍需控制
deadline	给请求设置过期时间	吞吐和延迟需要权衡
BFQ/权重公平	按进程或 cgroup 分配带宽	实现复杂，需识别同步/异步流
mq 调度	多硬件队列并行提交	CPU 亲和性、锁和合并难度增加

现代块层还要处理 flush 和 FUA。普通写请求完成，可能只是设备缓存接收；flush 要求设备把缓存内容推到持久介质；FUA 要求特定写绕过或强制落到稳定介质。文件系统的 fsync 正确性依赖这些语义。

驱动生命周期：probe、open、submit、remove。

设备驱动不是只有 read/write。驱动要在设备发现时 probe，分配队列和 DMA 资源；打开设备时建立引用；提交请求时保证 buffer 生命周期；中断或 polling 完成请求；设备移除时拒绝新请求并等待旧请求完成。

```

driver_probe(dev):
    map_mmio(dev)
    allocate_queues(dev)
    register_interrupt(dev)
    register_device_node(dev)

driver_remove(dev):
    mark_dying(dev)
    wake_all_waiters(dev)
    stop_queues(dev)
    wait_inflight_requests(dev)
    unregister_interrupt(dev)
    unmap_mmio(dev)
    
```

热插拔和错误恢复把生命周期变复杂。用户线程可能正在阻塞读设备时设备被拔出；中断可能在 remove 期间到达；DMA 可能尚未完成。驱动必须有 dying 状态、

引用计数、in-flight 计数和唤醒所有等待者的关闭路径。

I/O 错误传播。

写回错误可能晚于 `write` 出现。用户调用 `write` 时数据进入 page cache，后台 writeback 才发现设备 EIO。内核必须把错误记录在 inode 或 file 的错误状态里，让后续 `fsync`、`close` 或再次写入时能报告。

```
writeback_complete(page, status):
    if status == EIO:
        mapping.error = EIO
        mark_page_error(page)
    end_writeback(page)

fsync(file):
    writeback_dirty_pages(file.mapping)
    err = file.mapping.consume_error_since(file.open_time)
    if err:
        return -err
    flush_device()
    return 0
```

这也是为什么可靠程序不能忽略 `fsync` 和 `close` 错误。错误可能代表先前看似成功的 buffered write 实际没有持久化。高质量 API 文档应明确错误是在当前调用中发生，还是报告此前异步写回失败。

文件系统测试：不仅测功能，还测崩溃。

文件系统测试要覆盖运行时一致性、权限、并发和崩溃恢复。只测试“创建文件后能读回”远远不够。

测试类型	方法	期望
路径错误	构造 <code>ENOENT/ENOTDIR/ELOOP/EACCES</code>	错误码准确，目标不被破坏
引用生命周期	open 后 unlink、dup、fork、close	inode/file 引用计数正确
page cache	buffered read/write/mmap 混用	内容一致，dirty 状态正确
direct I/O	非对齐和混用 buffered I/O	返回明确错误或同步缓存
写回错误	注入设备 EIO	fsync/close 可观察到错误
崩溃恢复	在事务各阶段断电重放	fsck/journal 恢复到合法状态
并发 rename	多线程查找和替换路径	观察到旧或新，不能看到坏中间态

崩溃测试通常需要 fault injection 或模拟块设备：在第 N 次写、flush 或 journal commit 后强制崩溃，然后重挂载检查不变量。没有这类测试，文件系统只能证明正常路径可用，不能证明持久化语义可信。

尚未解决的问题。块层能够提交有顺序约束的请求，VFS 能把 fd 分派到具体实现，但一片可寻址块仍没有空闲记录、文件身份、名称映射或重新挂载入口。下一章从这些最小持久事实开始形成文件系统。

第四部分

持久对象与网络端点

本部分导读

可寻址块还不是文件：系统必须先记录空闲空间，再建立稳定的文件身份和名称关系，随后解决大文件、大目录、缓存写回和断电恢复。只有基本格式与恢复协议已经成立，现代文件系统之间的不同取舍才有可比较的基准。网络端点随后把同样的对象、队列、等待和完成思想应用到持续通信，但其提交边界由确认、缓冲和连接状态决定，而不是由介质持久化决定。

第 11 章 文件系统的形成：从可寻址空间到持久对象

核心思想

文件系统并不是从 inode、超级块或目录树开始的。最初只有一片能够在重启后保留内容的可寻址空间；只有在多个对象共享这片空间之后，系统才依次需要管理单位、分配状态、对象身份、名称关系和重新挂载的入口。本章沿着这条需求链建立最小磁盘格式，并说明每一种结构为何出现、保存什么事实，以及它不能替代什么。

从持久字节到固定大小的块

理解文件系统，可以先暂时放下 inode、超级块和 VFS 等术语，从一个更早的问题开始。系统现在得到一片掉电后仍能保留内容的空间，并且能够读取或改写其中某个范围。这个接口只回答“某个位置有哪些字节”，并不知道这些字节是否构成文件，更不知道文件名和目录。接下来建立的每一种结构，都对应一个尚未解决的持久化问题。

如果直接按单个字节管理整片空间，一块 1 TiB 的设备就需要面对约 2^{40} 个可分配位置。为每个字节记录是否占用会产生巨大的管理信息；分配一个 8 MiB 文件也要登记数百万个位置。另一个极端是把整个设备视为一个单位，这虽然容易登记，却无法让多个文件独立增长和回收。系统需要在“管理记录不能过多”和“小文件不能浪费过大”之间选择一个固定粒度：每次以一组连续字节作为分配、映射和回收的基本单位。

这一步才得到文件系统块。本节采用 4096 字节作为块大小，所以块 7 表示从字节偏移 7×4096 开始的 4096 字节区间。固定大小使任意位置能够由整数块号表示，也使分配状态可以压缩成位图。块越小，小文件尾部浪费通常越少，但管理项和索引项更多；块越大，顺序 I/O 更容易成批处理，但文件末尾未使用空间可能增加。块大小因此是格式设计的取舍，而不是由“文件”这个概念天然规定的常数。

设备也可能按自己的逻辑扇区接受请求，内存则按页管理映射。设备逻辑块、文件系统块和内存页是三个不同层次的单位：前者属于设备接口，中者属于持久空间组织，后者属于内存映射。这里让文件系统块与算例中的内存页同为 4096 字节，只是为了便于手算，不能据此认为三者在所有系统中都相同。

固定管理单位仍然不是文件

块给出了位置和管理粒度，却没有说明其中哪些字节有效、这些字节属于哪个对象，也没有提供名字或重启后的查找入口。假设把字符串 “hello” 写在一个 4096 字节块的开头，重启后至少还需要回答：

- 1. 有效内容是 5 字节，还是整个块都属于数据？
- 2. 这段内容应当按普通文件、目录还是其他记录解释？
- 3. 写入过程中发生掉电后，块里的字段是否仍然可信？

这些信息需要和内容一起保存。描述数据含义、边界和状态的数据称为 元数据 (metadata)。它在介质上仍是普通字节，只是在格式中承担解释作用。一个最小记录可以写成：

+-----+-----+-----+-----+
type length payload unused
1 byte 4 bytes length bytes rest of block
+-----+-----+-----+-----+

字段 `length=5` 规定读取时只返回 `"hello"`。此时得到的是一条带有解释边界的持久记录，还没有形成能够管理多个文件的文件系统。

空闲空间为何需要统一记录

当 `a.txt` 和 `b.txt` 共用同一设备时，两个创建者不能各自随意选择数据块。若它们都选择块 20，后一次写入就会覆盖前一次写入。系统需要维护一份全局分配状态，明确哪些块已经占用、哪些块仍可使用。

最直接的表示方法是为每个块保存一位：0 表示空闲，1 表示已分配。各位按块号排列，形成块位图 (*block bitmap*)：

```
block number: 0 1 2 3 4 5 6 7 8 9 ...
bitmap bit:   1 1 1 1 0 0 1 0 0 0 ...
```

解释：块 0--3 和块 6 已分配；值为 0 的位置可供后续分配。

位图记录的是空间状态。它能够防止同一块被重复分配，却不能说明块 6 属于哪个文件，也不能给出文件长度和块的排列顺序。后一组关系需要由文件自身的身份记录承担。

文件为何需要独立于数据块的身份

例如，一个 9000 字节文件至少占用三个 4096 字节块；这些块可能位于 20、21 和 90，不必连续。文件系统需要为这个文件保存一份记录，其中包括：

- 文件类型、权限和精确字节长度；
- 从文件偏移到数据块的映射；
- 链接计数、时间戳等生命周期信息。

这份“每个文件一份”的持久记录称为 *inode* (*index node*，索引节点)。给定 *inode*，文件系统能够解释文件属性并找到其数据块。文件内容可以迁移到新的块，目录中的名字也可以改变，而 *inode* 仍可代表同一个文件对象。

inode 编号本身也需要分配和回收，因此文件系统通常再维护一张 *inode* 位图。块位图管理可分配的数据空间，*inode* 位图管理可复用的对象编号；两者对应不同资源。*inode* 既不是文件内容或文件名，也不同于进程打开文件后得到的文件描述符 `fd`。

名称为何必须与对象身份分离

用户通过 `/notes/today.txt` 访问文件，不会直接使用 *inode* 37。文件系统因而需要持久保存名字到 *inode* 编号的映射：

```
"today.txt" -> inode 37
"todo.txt"  -> inode 52
```

保存一组这类映射的文件称为目录 (*directory*)；其中一条映射称为目录项 (*directory entry*，简称 *dirent*)。目录在磁盘上同样是一类文件，只是它的数据按目录项格式解释，而普通文件的数据由应用解释。

文件名与 *inode* 分开保存后，同一个 *inode* 可以由多个目录项引用，这构成硬链接；改名时主要修改目录项，无须搬动文件数据；固定大小的 *inode* 记录也不必为可变长名字预留空间。相应地，`unlink` 删除的是一个名字映射。链接计数已经降为零且没有进程继续打开该文件时，*inode* 和数据块才进入最终回收阶段。

路径解析需要一个固定起点

目录项能够把一个路径分量转换成 inode，但第一项查找仍需起点。解析 `/notes/today.txt` 时，系统先从 `/` 查找 `notes`，再在得到的目录中查找 `today.txt`。这个固定起点就是根目录 (*root directory*)。

根目录仍由普通的目录 inode 和目录数据构成。“根”描述的是它在路径解析中的位置，并不表示它采用另一种磁盘格式。文件系统只需在全局元数据中保存根 inode 编号，路径遍历便可从这里开始。

重新挂载为何需要超级块

运行期间，内存对象已经记录 inode 表、位图和根目录的位置；关机后这些指针全部消失。下一次挂载面对的仍是一串块，需要重新确认：

块有多大、共有多少块、inode 表从哪里开始、根 inode 是多少，以及设备上的字节是否属于当前能够识别的文件系统格式？

此时还不能借助 inode，因为挂载代码尚不知道 inode 表的位置。磁盘格式必须约定一个可直接定位的入口，在其中保存解释整体布局所需的全局元数据。这个入口记录称为 超级块 (*superblock*)。它描述整个文件系统，而 inode 描述单个文件对象。

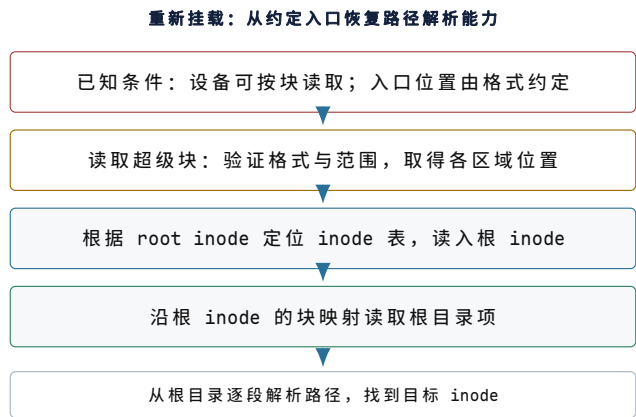
也可以设想另一种方案：挂载时扫描所有块，尝试猜测哪些块像 inode、哪些块像目录。这个方案既慢又没有唯一答案。普通文件内容完全可能恰好拥有与某种元数据相同的字节模式；如果没有预先约定的格式标识和范围，系统无法判断哪一种解释才有效。超级块解决的正是这个“解释必须先有入口”的问题。它不是因为名称听起来重要而额外增加的总表，也不会保存每个文件的内容；它只保存足以启动后续解释过程的全局事实。

超级块的位置本身不能再依赖超级块中的字段，否则会形成循环依赖。因此磁盘格式必须约定主超级块的固定位置或一组可枚举位置。挂载过程先按约定读取候选入口，验证格式标识、版本、块大小与各区域范围，再允许这些字段指导后续读取。验证之前，介质中的所有数值都只是不可信字节，不能直接作为数组下标、块号或长度使用。

一个教学格式的超级块至少包含：

- magic：识别文件系统格式；
- version：选择对应的磁盘格式解释规则；
- block size、block count 和 inode count：给出粒度与合法编号范围；
- inode 位图、块位图和 inode 表的位置；
- data start：普通数据区的起点；
- root inode：路径解析的起点。

magic 只能标识格式，不能替代损坏检测；version 指磁盘格式版本，也不同于应用或内核版本。字段范围检查、校验和、副本和日志会在后文逐步加入。



图示：超级块承担重新挂载的入口职责。挂载代码先在约定位置取得全局布局，再定位根 inode 和根目录；后续路径查找才有稳定起点。

各类结构的职责边界

结构	负责的关系	职责边界
块	用编号表示固定大小的设备区间	不表达文件身份、名字和有效长度
块位图	记录每个块是否已经分配	不记录已用块属于哪个文件
inode	保存单个文件的属性和块映射	通常不保存文件名，也不表示编号是否空闲
inode 位图	记录 inode 编号的分配状态	不保存 inode 的属性与块映射
目录项	保存名字到 inode 编号的映射	不保存普通文件内容
超级块	保存全局格式、区域位置和根 inode	不容纳每个文件的全部元数据

这些结构并非彼此替代：位图记录资源状态，inode 记录文件与数据块的关系，目录项记录名字与文件的关系，超级块提供解释整个布局的入口。后文加入的新字段也应落入明确职责，否则相同事实会出现多个权威来源，更新时很容易失去一致性。

从职责关系得到最小磁盘布局

MiniFS-Lab 使用 64 个块，每块 4096 字节，总容量 256 KiB。它只保留观察机制所需的结构，布局如下：

block 0	superblock: 全局格式与区域位置
block 1	inode bitmap: 哪些 inode 编号已分配
block 2	block bitmap: 哪些块已分配
block 3--6	inode table: 32 个固定大小 inode
block 7	root directory data: 根目录的目录项
block 8--62	ordinary data blocks
block 63	reserved: 后续日志实验使用

块 0 是这个教学格式约定的入口位置。真实文件系统可以预留引导区，在固定偏移保存主超级块，并在其他位置保存副本；共同要求是挂载代码能够在尚未了解布局时定位入口。

inode 表采用固定大小记录。给定 inode 编号 i ，其字节位置可以直接计算：


```
byte offset = inode table start + i × inode size.
```

这种布局避免为定位一个 inode 而扫描整张表，代价是每条记录的内联空间固定，较大的可变信息需要放在其他块中。

格式化后必须立即存在根目录。MiniFS-Lab 的 mkfs 会写入超级块，在两张位图中标记元数据、根 inode 和根目录块已用，初始化根 inode，并在块 7 写入 “.” 和 “..”。只写超级块会留下一个可识别却无法解析路径的格式；若根目录块没有在位图中标记，后续分配又可能覆盖它。mkfs 因而建立的是一组相互一致的初始状态。

创建 /hello 时发生的结构变化

现在执行 `create("/hello", "abc")`。暂不考虑并发和崩溃，涉及的结构变化依次为：

1. 依据超级块中的 root inode 和区域位置开始路径解析；
2. 读取根 inode 与根目录，确认名字 `hello` 尚不存在；
3. 从 inode 位图选择一个空闲编号；
4. 从块位图选择一个空闲数据块；
5. 初始化新 inode，记录普通文件类型、长度 3 和数据块映射；
6. 将 `abc` 写入已分配的数据块；
7. 在根目录加入 `"hello" -> new inode`，使新文件可由路径访问。

一次 create 同时改变两张分配表、一个 inode、一组目录项和数据块。若任意写入只完成一部分，位图、块映射与路径可达性之间便可能失去一致：资源会重复分配或泄漏，目录项也可能指向未初始化的 inode。后文的操作协议、日志和 COW 都围绕这些多对象更新展开。

磁盘格式与内存对象

磁盘 inode 是按格式编码的持久字节；内核中的 `struct inode` 是运行时对象，还包含锁、引用计数、page cache 入口和操作表。类似地，磁盘目录项保存持久名字映射，内存 dentry 缓存路径查找结果；磁盘超级块与内存 `struct super_block` 也各有职责。

磁盘格式需要跨重启和软件升级保持可解释，不能随编译器的结构体布局变化；内存对象只服务当前运行，可以使用指针、锁和缓存。将 C++ 结构体直接 `memcpy` 到磁盘会引入字节序、对齐、填充和版本兼容问题。MiniFS-Lab 采用显式定宽整数编码，并在 mount 时逐字段检查范围，使磁盘上的每个字段都有稳定含义。

这一组关系可以概括为：

固定位置与管理粒度	-> block
多个对象共享空间	-> allocation bitmap
文件属性与非连续块映射	-> inode
用户名字到文件对象	-> directory entry
绝对路径的固定起点	-> root inode
重启后重新解释整个布局	-> superblock
多对象更新的崩溃恢复	-> journal / copy-on-write (后文)

接下来的内容先把教学块号接到真实设备 I/O，再讨论 ext4 inode、extent、目录索引、page cache 和日志如何在大容量、并发与故障条件下维护同一组关系。

MiniFS-Lab 中的编码与恢复逻辑

MiniFS-Lab 使用 `put_u16`、`put_u32`、`get_u16` 和 `get_u32` 显式转换定宽整数。写入函数把整数按小端顺序拆成字节，读取函数再按相同顺序组合；每次访问都先

确认字段没有越过块边界。磁盘格式因此由字段宽度、字节序和偏移共同确定，不依赖 C++ 结构体的填充与对齐。

代码中的三条执行路径分别说明正常操作、资源回收和中途掉电时的状态变化：

1. **格式化与重挂载**：`format` 写入超级块、两张位图、根 inode 和根目录项；`mount` 不沿用先前的内存对象，而是读取超级块、确认布局参数，再从根 inode 开始恢复路径解析。
2. **删除与资源回收**：`unlink` 先移除目录项，再清除数据块位图和 inode 位图中的占用标记，同时将相应记录复位。`audit` 从根目录计算可达对象，并将该结果与两张位图及 inode 块映射相互核对。
3. **掉电注入**：`create` 已经分配 inode 和数据块并写入内容，在目录项发布前中断执行。重新挂载后，该名字尚不可见；`audit` 会识别不可达 inode 和没有可达所有者的已分配块，`reclaim_unreachable` 再依据可达关系回收这些资源。

这个故障场景展示了无事务更新的恢复边界。`mount` 只能读取当前磁盘状态，无法推知半完成操作原本打算发布的名字；`fsck` 可以修复结构和回收泄漏，却未必能够恢复用户意图。日志为完整事务保存提交证据，使恢复程序能够重放已提交事务，并忽略没有完成提交的尾部更新。

布局字段如何编码为磁盘字节

前一节没有预先规定“文件系统必须有哪些结构”，而是从尚未解决的问题逐项得到块、位图、inode、目录项和超级块。现在才把这些职责落实为具体布局。阅读下面的字段时，应当始终追问它代表哪一项持久事实；如果两个字段同时声称自己是同一事实的权威来源，更新时就可能产生无法判定的冲突。

磁盘参数

为了具体化讨论，我们定义一个教学文件系统（Minimal FS, MDFS），运行在一块 1 GiB 磁盘上：

- 磁盘容量：1 GiB = 2^{30} 字节 = 1,073,741,824 字节
- 块大小：4096 字节（4 KiB）
- 总块数： $2^{30}/2^{12} = 2^{18} = 262144$ 块
- inode 大小：256 字节
- inode 总数：512
- inode 表大小： $512 \times 256 / 4096 = 32$ 块

磁盘布局如下：

块号	内容	说明
0	引导扇区	保留，包含引导加载程序。文件系统不使用。
1	超级块	文件系统全局元数据：总块数、空闲块数、inode 表位置等。
2	inode 位图	每位对应一个 inode，1 = 已分配。512 位 = 64 字节，远小于 1 块。
3-10	块位图	每位对应一个数据块，1 = 已分配。262144 位 = 32768 字节 = 8 块。
11-42	inode 表	$32 \text{ 块} \times 4096 / 256 = 512$ 个 inode。inode N 在块 $11 + N/16$ ，偏移 $(N\%16) \times 256$ 。

块号	内容	说明
43- 262143	数据块	实际存放文件数据。

这个文件系统需要管理四类磁盘上的数据结构：

1. 超级块 (superblock)：全局元数据，1 块。
2. inode 表 (inode table)：每个文件的元数据，32 块。
3. 目录项 (directory entries)：目录的内容，存在数据块中。
4. 空闲空间位图 (block bitmap + inode bitmap)：分配状态， $8 + 1 = 9$ 块。

超级块：全局元数据

```
/* Block 1: superblock (64-byte header, rest is padding) */
struct superblock {
    uint32_t magic;           /* 0x4D534653 = "MDFS" magic number */
    uint32_t total_blocks;    /* 262144 */
    uint32_t free_blocks;     /* 262100 (initially, after mkfs) */
    uint32_t total_inodes;    /* 512 */
    uint32_t free_inodes;     /* 511 (inode 1 used by root dir) */
    uint32_t block_size;      /* 4096 */
    uint32_t inode_size;      /* 256 */
    uint32_t root_inode;      /* 1 (root directory's inode number) */
    uint32_t bitmap_start;    /* 2 (block where inode bitmap starts) */
    uint32_t inode_table_start; /* 11 */
    uint32_t data_start;      /* 43 (first data block) */
    uint32_t reserved[5];     /* future use */
}; /* total: 16 * 4 = 64 bytes, rest of 4096-byte block is zero */
```

当 mkfs 创建文件系统后，块 1 的前 64 字节（小端序）如下：

```
偏移  字节 (hex)
0000:  53 46 53 4D  00 00 04 00  FF FF 03 00  00 02 00 00
      |magic----| |total_blk-| |free_blk--| |total_ino|
0010:  FF 01 00 00  00 10 00 00  00 01 00 00  01 00 00 00
      |free_ino-| |block_sz-| |inode_sz-| |root_ino-|
0020:  02 00 00 00  0B 00 00 00  2B 00 00 00  00 00 00 00
      |bitmap---| |inode_tbl-| |data_strt| |reserved0|
0030:  00 00 00 00  00 00 00 00  00 00 00 00  00 00 00 00
      |reserved1| |reserved2| |reserved3| |reserved4|
```

逐字段解读：

- `magic = 0x4D534653`：小端序读为字节 53 46 53 4D，即 ASCII "MDFS"（反序）。文件系统挂载时检查此值，不匹配则拒绝挂载。
- `total_blocks = 0x00040000` = 262144。
- `free_blocks = 0x0003FFD5` = 262101。初始时 262144 块减去 43 块元数据 = 262101。再减去根目录占用的 1 块（块 43）= 262100。
- `total_inodes = 0x00000200` = 512。
- `free_inodes = 0x000001FF` = 511（inode 1 已分配给根目录）。
- `block_size = 0x00001000` = 4096。
- `inode_size = 0x00000100` = 256。
- `root_inode = 1`, `bitmap_start = 2`, `inode_table_start = 11` (0x0000000B), `data_start = 43` (0x0000002B)。

inode：文件的元数据

inode (index node) 是文件系统中每个文件的元数据锚点。它存储除文件名以外的所有文件属性。我们的教学文件系统使用 256 字节的 inode (与 ext4 默认值相同)，但核心字段布局沿用 ext2 的经典 128 字节格式：

```
/* 256-byte inode (first 128 bytes shown, matching ext2 layout) */
struct mdfs_inode {
    uint16_t i_mode;           /* file type + permission bits */
    uint16_t i_uid;           /* owner user ID (low 16 bits) */
    uint32_t i_size;           /* file size in bytes */
    uint32_t i_atime;          /* last access time (Unix epoch) */
    uint32_t i_ctime;          /* inode change time */
    uint32_t i_mtime;          /* data modification time */
    uint32_t i_dtime;          /* deletion time (0 if alive) */
    uint16_t i_gid;            /* group ID (low 16 bits) */
    uint16_t i_links_count;    /* hard link count */
    uint32_t i_blocks;         /* sectors used (NOT block count!) */
    uint32_t i_flags;          /* ext4 flags (immutable, append, etc.) */
    uint32_t i_osd1;           /* OS-specific data */
    uint32_t i_block[15];      /* 12 direct + 1 indirect +
                               /* 1 double + 1 triple indirect */
    uint32_t i_generation;     /* inode version (for NFS) */
    uint32_t i_file_acl;       /* extended attribute block */
    uint32_t i_dir_acl;        /* high 32 bits of size (dirs) */
    uint32_t i_faddr;          /* fragment address (obsolete) */
    uint8_t i_osd2[12];        /* OS-specific (uid/gid high,
                               /* reserved blocks, etc.) */
    /* --- ext4 extension: bytes 128--255 --- */
    uint32_t i_ctime_extra;     /* sub-second ctime precision */
    uint32_t i_mtime_extra;     /* sub-second mtime precision */
    uint32_t i_atime_extra;     /* sub-second atime precision */
    uint32_t i_crtime;         /* creation time */
    uint32_t i_crtime_extra;    /* sub-second crtime precision */
    uint32_t i_version_hi;      /* high 32 bits of i_generation */
    uint32_t i_projid;          /* project ID */
    uint8_t i_checksum[8];      /* inode checksum (crc32c) */
};
```

inode 的字节级视图

假设有一个 10000 字节的普通文件，拥有者 uid=0, gid=0, 权限 0644, 创建时间戳为 1700000000 (2023-11-14 UTC)。文件数据占用 3 个块 (块 68, 69, 70)，因为 $\lceil 10000/4096 \rceil = 3$ 。

该 inode (前 64 字节，小端序) 如下：

偏移	字节 (hex)	字段
0000:	A4 81 00 00 10 27 00 00 00 F1 53 65 00 F1 53 65	i_mode--- i_size--- i_atime- i_ctime-
0010:	00 F1 53 65 00 00 00 00 00 00 01 00 18 00 00 00	i_mtime- i_dtime-- uid gid i_links- i_blocks
0020:	00 00 00 00 00 00 00 00 44 00 00 00 45 00 00 00	i_flags- i_osd1--- block[0]- block[1]-
0030:	46 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	block[2]- block[3]- block[4]- block[5]-

逐字段解读：

- `i_mode = 0x81A4`。高 4 位 `0x8` 表示 `S_IFREG` (普通文件)。低 12 位 `0x1A4 = 0644` (`rw-r--r--`)。完整值：`S_IFREG | 0644 = 0100644` (八进制) = `0x81A4`。
- `i_size = 0x00002710 = 10000`。文件精确大小 10000 字节。
- `i_atime / i_ctime / i_mtime = 0x6553F100 = 1700000000`，即 2023-11-14T22:13:20Z。
- `i_dtime = 0` (文件未被删除)。
- `i_uid = 0` (root), `i_gid = 0` (root)。
- `i_links_count = 0x0001 = 1` (1 个硬链接)。
- `i_blocks = 0x00000018 = 24`。注意：不是 3 (块数)，而是 24 (512 字节扇区数)。3 块 × 8 扇区/块 = 24 扇区。
- `i_block[0] = 0x00000044 = 68` (物理块号)。`i_block[1] = 69`, `i_block[2] = 70`。`i_block[3..14] = 0` (未使用)。

常见误区

`i_blocks` 采用 512 字节扇区数作为单位，并可能计入间接指针块、扩展属性块等元数据；它不等于当前文件系统块的数量。一个 1 字节文件在 4K 块系统上 `i_blocks = 8` (1 块 × 8 扇区)，而不是 1。读取该字段时应先确认单位，再换算实际占用量。

目录项：文件名的存放

文件名不在 `inode` 中——文件名存放在目录文件的数据块里，以目录项 (directory entry, `dirent`) 的形式组织。`ext2/ext4` 的目录项格式如下：

```
/* ext4 directory entry (variable length) */
struct ext4_dirent {
    uint32_t inode;          /* inode number of this entry */
    uint16_t rec_len;        /* total size of this entry (bytes) */
    uint8_t  name_len;       /* length of filename (bytes) */
    uint8_t  file_type;      /* file type (see enum below) */
    char     name[];         /* filename, NOT null-terminated */
                          /* padded to 4-byte boundary */
};

/* file_type values */
#define EXT4_FT_UNKNOWN 0
#define EXT4_FT_REG_FILE 1 /* regular file */
#define EXT4_FT_DIR 2 /* directory */
#define EXT4_FT_CHRDEV 3 /* character device */
#define EXT4_FT_BLKDEV 4 /* block device */
#define EXT4_FT_FIFO 5 /* named pipe */
#define EXT4_FT_SOCK 6 /* socket */
#define EXT4_FT_SYMLINK 7 /* symbolic link */
```

`rec_len` 的设计是 `ext2/ext4` 目录项的核心：它允许目录项在块内 紧凑排列，同时支持 `rec_len` 覆盖被删除的条目（删除时不移动后续条目，只把被删条目的 `inode` 置 0 并把它的 `rec_len` 加到前一条目）。

目录项的字节级视图

假设根目录 (`inode 1`) 的数据块中有一个条目指向文件 “foo” (`inode 5`)，以及标准的 “.” 和 “..” 条目：

目录数据块 (块 43) 内容：

偏移	字节 (hex)	解读
0000:	01 00 00 00 0C 00 01 02 2E 00 00 00	inode=1, rec_len=12, namelen=1, type=DIR, "."
000C:	01 00 00 00 0C 00 02 02 2E 2E 00 00	inode=1, rec_len=12, namelen=2, type=DIR, ".."
0018:	05 00 00 00 0C 00 03 01 66 6F 6F 00	inode=5, rec_len=12, namelen=3, type=REG, "foo"
0024:	00 00 00 00 F4 0F 00 00 ...	inode=0 (deleted), rec_len=4012 (rest of block)

逐条解读：

- 偏移 0x0000: "." 条目。inode=1 (根目录自身), rec_len=12 (8 字节头 + 1 字节名 + 3 字节填充), name_len=1, file_type=2 (DIR), 名字 = "."。
- 偏移 0x000C: ".." 条目。inode=1 (父目录也是根目录自身), rec_len=12, name_len=2, file_type=2 (DIR), 名字 = ".."。
- 偏移 0x0018: "foo" 条目。inode=5, rec_len=12, name_len=3, file_type=1 (REG_FILE), 名字 = "foo" (0x66 0x6F 0x6F), 填充 1 字节 0x00。
- 偏移 0x0024: 剩余空间。inode=0 标记为空闲, rec_len=4012 (从 0x0024 到块末 = 0x1000 的剩余字节数)。

空闲空间管理：块位图与 inode 位图

MDFS 使用位图 (bitmap) 管理空闲资源。每个块对应位图中一位：1 = 已分配，0 = 空闲。

- 块位图：262144 个块需要 262144 位 = 32768 字节 = 8 个块 (块 3-10)。
- inode 位图：512 个 inode 需要 512 位 = 64 字节，远少于 1 个块 (块 2)。

块位图在创建 3 个文件后的状态

假设 mkfs 后，创建了根目录 (占用块 43)，然后创建 3 个文件：/foo (100 字节，块 68)、/bar (200 字节，块 69)、/baz (50 字节，块 70)。

已分配的块：0-42 (元数据)、43 (根目录数据)、68、69、70。

位图前 10 字节如下 (bit 0 = 块 0, LSB first):

```

字节 0 (块 0-7):  FF = 11111111  块 0-7 全部已分配
字节 1 (块 8-15): FF = 11111111  块 8-15 全部已分配
字节 2 (块 16-23): FF = 11111111  块 16-23 全部已分配
字节 3 (块 24-31): FF = 11111111  块 24-31 全部已分配
字节 4 (块 32-39): FF = 11111111  块 32-39 全部已分配
字节 5 (块 40-47): 0F = 00001111  块 40-43 已分配, 44-47 空闲
字节 6 (块 48-55): 00 = 00000000  全部空闲
字节 7 (块 56-63): 00 = 00000000  全部空闲
字节 8 (块 64-71): 70 = 01110000  块 68-70 已分配, 其余空闲
字节 9 (块 72-79): 00 = 00000000  全部空闲
...
```

常见误区

ext2/ext4 中位图是小端序 bit order: 字节内的 bit 0 (LSB) 对应最低编号的块。即字节 0x0F = 0b00001111 表示块 0-3 已分配，块 4-7 空闲。这与大端序的直觉相反。x86 架构下，testb 指令测试的是 LSB，与这种 bit order 一致。

完整追踪：创建 /foo (100 字节)

现在我们追踪一次完整的 `create_file` 操作，从路径解析到磁盘写入。

```
=== create_file("/foo", 100 bytes) ===

Step 1: path_resolve("/foo")
- Start at root inode (superblock.root_inode = 1)
- Load inode 1 from inode table
- Root is a directory (i_mode & S_IFDIR), proceed

Step 2: dir_lookup(root_inode=1, "foo")
- Read root's data block (block 43)
- Scan dirents: ".", "..", (deleted entry)
- "foo" not found -> ENOENT (good, we can create it)

Step 3: alloc_inode()
- Load inode bitmap (block 2)
- Scan for first 0 bit: bit 5 is free
- Set bit 5 -> inode 5 is now allocated
- Mark inode bitmap block dirty

Step 4: init_inode(5)
- Write inode 5 at block (11 + 5/16) = block 11, offset (5%16)*256 = 1280
- Fields:
  i_mode      = S_IFREG | 0644 = 0x81A4
  i_uid       = 0
  i_size      = 0 (will be 100 after write)
  i_atime     = i_ctime = i_mtime = current_time
  i_dtime     = 0
  i_gid       = 0
  i_links_count = 1
  i_blocks    = 0 (no data blocks yet)
  i_block[0..14] = 0 (all zero)
- Mark inode table block 11 dirty

Step 5: alloc_blocks(1)
- Load block bitmap (blocks 3-10)
- Scan for first free bit after data_start (block 43)
- Block 68 is free (bit 68 in bitmap = 0)
- Set bit 68 -> block 68 allocated
- Mark block bitmap block dirty
- Update superblock: free_blocks--

Step 6: set_inode_5_block[0] = 68
- Write i_block[0] = 68 in inode 5's on-disk record
- Update i_blocks = 8 (1 block * 8 sectors/block)
- Update i_size = 100
- Mark inode table block 11 dirty

Step 7: write data
- Copy 100 bytes of user data into block 68
- Zero-fill remaining 4096 - 100 = 3996 bytes of block 68
- Mark block 68 dirty in page cache

Step 8: dir_add_entry(root_inode=1, "foo", inode=5)
- Read root's data block (block 43, already in page cache)
- Find the "deleted" entry at offset 0x0024
  (inode=0, rec_len=4012)
- Split: new entry takes 12 bytes (8 header + 4 name)
  remaining rec_len = 4012 - 12 = 4000
```

- Write dirent:
 - inode = 5
 - rec_len = 12
 - name_len = 3
 - file_type = 1 (EXT4_FT_REG_FILE)
 - name = "foo\0" (padded to 4 bytes)
- Update the following entry's rec_len to 4000
- Mark root's data block (block 43) dirty
- Update root inode: i_mtime = current_time, i_size may grow
- Mark root inode dirty

Step 9: mark dirty

- Inode 5 dirty (metadata changed)
- Inode 1 (root) dirty (directory modified)
- Inode bitmap block dirty (bit 5 set)
- Block bitmap block dirty (bit 68 set)
- Superblock dirty (free_blocks decremented)
- Block 68 dirty (file data written)
- Block 43 dirty (dirent added)

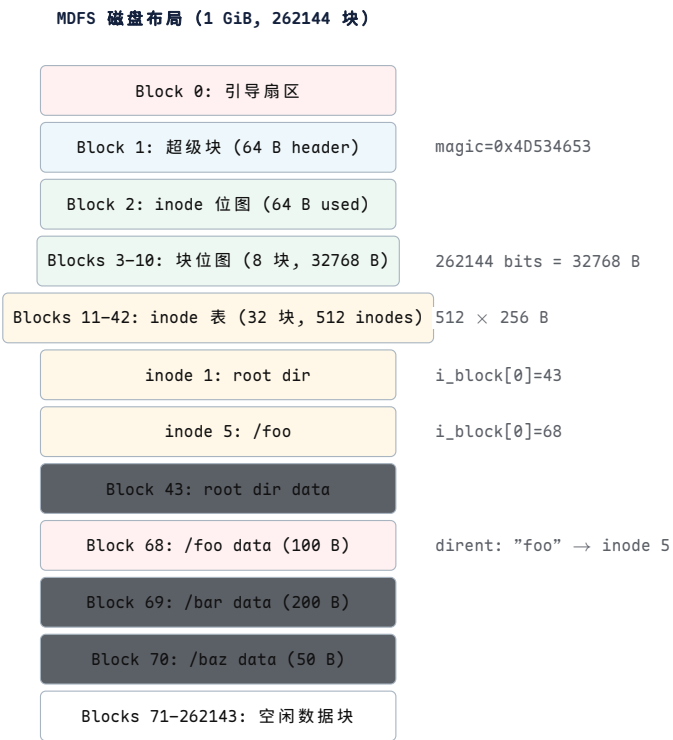
Step 10: writeback

- Eventually, the writeback mechanism selects dirty folios
- Flushes all dirty pages to disk via submit_bio()
- Order may vary (depends on journal mode)
- On ext4 with journal=data: metadata changes go through journal first
- On ext4 with journal=ordered: data is written before metadata
- fsync() waits for all these writes to complete

常见误区

Step 10 是最危险的一步。如果在 Step 8 写入了目录项但 Step 5 的块位图还没落盘，断电后磁盘状态不一致：inode 5 的数据块 68 被标记为已分配（因为目录项指向 inode 5），但块位图说 68 空闲，导致块泄漏（leaked block）。反之，如果块位图先落盘但目录项没落盘，则块 68 被标记为已分配但没有文件引用它——也是泄漏。ext4 用日志（journal）解决这个问题：将元数据变更先写入日志区，原子提交后再应用到主区域。

磁盘布局图



图示：红色 = 数据块，蓝色 = 超级块，绿色 = 位图，琥珀色 = inode 表，灰色 = 根目录数据。注意块 43（根目录数据）和块 68（/foo 数据）都在数据区（块 43+），它们没有特殊的存储位置—文件系统通过 inode 的 `i_block[]` 指针找到它们。

第 12 章 文件身份、空间索引与路径命名

核心思想

文件内容可以移动，文件名可以改变，同一个文件还可以拥有多个名字。因此，位置、身份和名称不能由同一个字段承担。本章先建立稳定的文件身份，再从文件增长问题推导块映射，从大量名字的查找问题推导目录索引，最后说明 VFS 如何把这些持久对象接入统一接口。

inode 记录与文件生命周期

文件为什么需要独立于名字和位置的身份

先设想只用目录项保存“文件名、长度和全部数据块号”。这种做法在一个名字对应一个小文件时可以工作；一旦文件改名，整份记录就要移动；一旦建立硬链接，两份目录记录又会重复保存权限、长度和块映射。此后修改文件时，系统无法判断哪一份记录才是权威状态。

解决办法是把“文件本身的稳定记录”从“用户看到的名字”中分离。目录只保存名字到文件编号的映射，编号再定位一份独立记录。这样，改名只改变名称关系，硬链接只增加一个名称引用，数据块移动也不会改变文件身份。这份按文件编号定位的持久记录就是 inode。

inode 因而不是为了给某个数据块命名，也不是把每个块再切出一部分。文件系统在格式化时为 inode 记录保留独立区域；一个 inode 描述一个文件，并把文件内的字节偏移映射到零个或多个数据块。空文件已经可以拥有 inode，却不需要数据块；大文件则由一个 inode 管理许多数据块。明确这一点之后，inode 中各类字段的职责才容易理解：

- 身份：inode 编号唯一标识文件。
- 类型：普通文件、目录、符号链接、设备文件.....
- 权限：谁可以读、写、执行。
- 大小：文件有多少字节。
- 时间戳：访问、修改、创建、删除时间。
- 数据位置：通过 `i_block[15]` 数组指向数据块。
- 链接计数：有多少个目录项指向这个 inode。

文件名不在 inode 中。文件名在目录文件的数据块中，作为目录项的 `name` 字段。一个 inode 可以有多个名字（硬链接），也可以没有名字但仍然存在（被进程通过 `open()` 持有但已从目录中删除）。

ext4 on-disk inode 格式

下面以简化的 ext4 磁盘 inode 定义说明各字段的偏移和职责：

```
/* ext4 on-disk inode (256 bytes, little-endian) */
struct ext4_inode {
    /* --- bytes 0--127: ext2-compatible core --- */
    __le16 i_mode;           /* +0:  file mode (type + perms)    */
    __le16 i_uid;            /* +2:  low 16 bits of owner UID      */
    __le32 i_size_lo;        /* +4:  low 32 bits of size (bytes)   */
    __le32 i_atime;          /* +8:  last access time              */
}
```

```

__le32 i_ctime;          /* +12: inode change time */
__le32 i_mtime;          /* +16: data modification time */
__le32 i_dtime;          /* +20: deletion time */
__le16 i_gid;            /* +24: low 16 bits of group GID */
__le16 i_links_count;    /* +26: hard link count */
__le32 i_blocks_lo;      /* +28: low 32 bits of sector count */
__le32 i_flags;          /* +32: ext4 flags */
__le32 i_osd1;           /* +36: OS-specific (Linux: version) */

/* +40: block pointers (60 bytes) */
__le32 i_block[15];      /* 12 direct, 1 indirect,
                          /* 1 double indirect, 1 triple

__le32 i_generation;     /* +100: inode version (NFS) */
__le32 i_file_acl_lo;    /* +104: extended attribute block */
__le32 i_size_high;      /* +108: high 32 bits of size */
__le32 i_obso_faddr;     /* +112: obsolete fragment address */

/* +116: OS-specific area (12 bytes) */
__le16 i_blocks_hi;      /* high 16 bits of i_blocks */
__le16 i_file_acl_hi;    /* high 16 bits of file ACL */
__le16 i_uid_high;       /* high 16 bits of UID */
__le16 i_gid_high;       /* high 16 bits of GID */
__le16 i_checksum_lo;    /* low 16 bits of checksum */
__le16 i_reserved2;      /* reserved */

/* --- bytes 128--255: ext4 extensions --- */
__le32 i_ctime_extra;     /* +128: extra ctime (ns precision) */
__le32 i_mtime_extra;     /* +132: extra mtime (ns precision) */
__le32 i_atime_extra;     /* +136: extra atime (ns precision) */
__le32 i_crtime;          /* +140: creation time */
__le32 i_crtime_extra;    /* +144: extra crtime (ns precision) */
__le32 i_version_hi;      /* +148: high 32 bits of version */
__le32 i_projid;          /* +152: project ID */
/* bytes 156--255: checksums, inline data, etc. */
};

```

逐字段深入

字段	偏移	含义与设计理由
<code>i_mode</code>	+0	16 位。高 4 位是文件类型 (<code>S_IFREG=0100</code> , <code>S_IFDIR=0040</code> , <code>S_IFLNK=0120</code> 等), 低 12 位是权限位 (<code>rw-rw-rw+</code> + <code>setuid/setgid/sticky</code>)。用 16 位而非 32 位是为了节省空间——512 个 inode $\times$ 2 字节 = 1 KiB 节省。
<code>i_uid</code>	+2	16 位低部。加上 <code>i_uid_high</code> (+118) 组成 32 位 UID。早期 Unix 只有 16 位 UID (最多 65536 用户), 足够小型系统。
<code>i_size_lo</code>	+4	32 位低部。加上 <code>i_size_high</code> (+108) 组成 64 位文件大小。ext2 时代 <code>i_size_high</code> 用于目录 (<code>i_dir_acl</code>), ext4 重新定义为大小高位, 支持 > 4 GiB 文件。

字段	偏移	含义与设计理由
i_atime	+8	最后访问时间。32 位 Unix epoch 秒。ext4 扩展 i_atime_extra 提供纳秒精度。注意：atime 更新会产生写 I/O！现代系统用 noatime 挂载选项避免此开销。
i_ctime	+12	inode 元数据修改时间（改权限、改属主时更新）。i_mtime 是数据修改时间（写文件内容时更新）。
i_dtime	+20	删除时间。文件被删除时写入此字段。用于恢复工具（如 extundelete）判断文件何时被删。非零意味着此 inode 曾被删除。

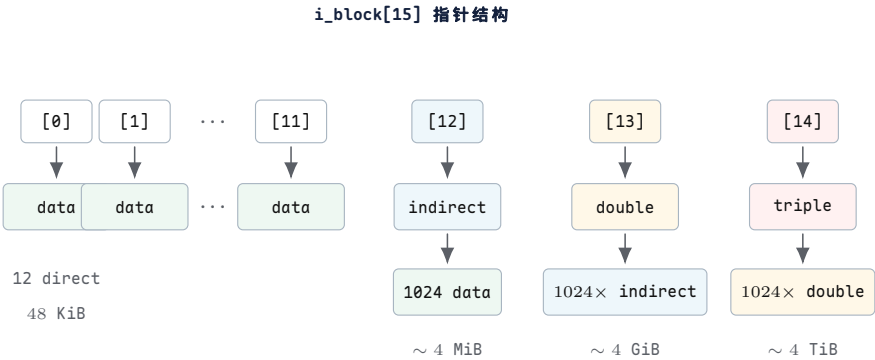
前一组字段描述文件身份、大小和时间；下一组字段描述引用关系、块映射方式和 inode 复用后的识别依据：

字段	偏移	含义与设计理由
i_links_count	+26	硬链接计数。每创建一个硬链接 +1，每删除一个 -1。降为 0 时，inode 被释放（位图中对应位清零，数据块回收）。
i_blocks_lo	+28	512 字节扇区数，不是块数！包括间接指针块和扩展属性块。加上 i_blocks_hi (+116) 组成 48 位值。
i_flags	+32	ext4 特性标志：EXT4_EXTENTS（使用区段树而非经典间接指针）、EXT4_IMMUTABLE_FL（不可修改）、EXT4_APPEND_FL（仅追加）等。现代 ext4 默认设置 EXT4_EXTENTS，此时 i_block[15] 的含义改变为区段树头。
i_block[15]	+40	60 字节的块指针数组。经典模式下：12 直接 + 1 间接 + 1 双重间接 + 1 三重间接。ext4 区段模式下：存储区段树根。
i_generation	+100	inode 版本号。每次 inode 被回收再分配时递增。NFS 用它检测“这个 inode 号是否还指向原来的文件”（防止 ABA 问题）。

i_block[15]：经典间接指针方案

当 i_flags 不包含 EXT4_EXTENTS 时，i_block[15] 使用经典的多级间接指针方案。这是 ext2/ext3 的设计，也是理解 inode 数据定位的基础。

15 个 uint32_t 槽位分为 4 组：



图示：蓝色 = 单重间接，琥珀色 = 双重间接，红色 = 三重间接，绿色 = 数据块。每个间接块包含 1024 个 4 字节指针（4096 / 4 = 1024）。颜色越深表示间接层级越多、寻址时需要的磁盘读次数越多。

精确容量计算

块大小 4096 字节，指针 4 字节，每块可容纳 4096/4 = 1024 个指针。

层级	计算	容量（字节）	磁盘读次数
12 直接	12×4096	49,152（48 KiB）	1
1 间接	1024×4096	4,194,304（4 MiB）	2
1 双重间接	$1024^2 \times 4096$	4,294,967,296（4 GiB）	3
1 三重间接	$1024^3 \times 4096$	4,398,046,511,104（4 TiB）	4
总计	以上之和	~ 4 TiB	最多 4

核心思想

多级指针方案的核心权衡是查找深度 vs 元数据紧凑性：小文件（< 48 KiB）只需 1 次磁盘读（直接指针直达数据），大文件需要 2-4 次。由于大多数文件很小（Unix 系统中 > 90% 的文件 < 48 KiB），这种设计在实践中非常高效。ext4 的区段（extent）方案进一步优化：用一棵 B+ 树替代 15 个指针槽，使连续区域用单个区段描述符表示。

追踪：读取偏移 5,000,000 处的数据

假设文件使用了所有四种指针类型。我们追踪读取文件内偏移 5,000,000 字节处所在的数据块。

目标：读取 byte offset 5,000,000

Step 1: 计算逻辑块号

```
logical_block = 5,000,000 / 4096 = 1220（整数除法）
byte_offset_in_block = 5,000,000 % 4096 = 3120
```

Step 2: 判断指针层级

```
Direct blocks cover logical 0..11    -> 12 blocks
1220 >= 12, so NOT in direct range.
offset_in_indirect = 1220 - 12 = 1208
```

```
Single indirect covers logical 12..1035 -> 1024 blocks
1208 >= 1024, so NOT in single indirect range.
offset_in_double = 1208 - 1024 = 184
```

```
Double indirect covers logical 1036..1,049,611 -> 1,048,576 blocks
184 < 1,048,576, so it IS in double indirect range. ✓
```

Step 3: 读取双重间接块

```
double_indirect_block = read_block(inode.i_block[13])
-- DISK READ #1: read the double indirect block
-- This block contains 1024 pointers to single indirect blocks
-- We need entry index = 184 / 1024 = 0
```

Step 4: 读取单重间接块

```
indirect_ptr = double_indirect_table[0] -> block 6000 (example)
indirect_block = read_block(6000)
```

```
-- DISK READ #2: read the single indirect block
-- This block contains 1024 pointers to data blocks
-- We need entry index = 184 % 1024 = 184
```

Step 5: 读取数据块

```
physical_block = indirect_table[184] -> block 8000 (example)
data = read_block(8000)
-- DISK READ #3: read the actual data block
-- The byte we want is at offset 3120 within this block
```

Result: byte at physical block 8000, offset 3120

为什么是 3 次磁盘读？

在没有缓存的情况下，读取这一个数据块需要 3 次磁盘 I/O：

1. 读双重间接块 (`i_block[13]` 指向的块)。
2. 读单重间接块 (双重间接块中第 0 项指向的块)。
3. 读数据块 (单重间接块中第 184 项指向的块)。

如果磁盘是 HDD，每次随机读约需 5–10 ms，则总延迟 15–30 ms。如果是 NVMe SSD，每次约 50 μ s，总延迟约 150 μ s。

实际中，Linux 的预读 (readahead) 机制会提前读取间接块周围的数据，以及 inode 所在的整个块组。页缓存也会缓存间接块，后续访问同一区域的间接块时不再需要磁盘 I/O。

常见误区

别混淆逻辑块和物理块。逻辑块号 1220 是文件内的第 1220 个块 (从文件头算起)。物理块号 8000 是磁盘上的第 8000 个块 (从磁盘头算起)。inode 的 `i_block[]` 数组做的正是从逻辑块号到物理块号的映射。这个映射可能不是连续的 (逻辑块 1220 在物理块 8000，逻辑块 1221 可能在物理块 5000)，这就是碎片化。

稀疏文件 (Sparse Files)

当 `i_block[]` 中的某个指针为 0 时，意味着该逻辑块没有被分配。这不是错误——读取该块时，内核返回一个全零页，而不是报错。这就是稀疏文件 (sparse file) 的工作原理。

```
// 创建一个稀疏文件：只有块 0 和块 1000 有数据
int fd = open("sparse.dat", O_CREAT | O_WRONLY, 0644);
write(fd, "hello", 5);           // 写块 0 (offset 0)
lseek(fd, 1000 * 4096, SEEK_SET); // 跳到块 1000
write(fd, "world", 5);           // 写块 1000
close(fd);

// inode 的 i_block[] 状态:
// i_block[0] = 68      (块 0 有数据)
// i_block[1..11] = 0   (块 1-11 是 "hole", 未分配)
// i_block[12] = 5000   (间接块, 指向块 1000 的间接指针)
// 间接块中: entry[988] = 70 (块 1000 的物理块号)
// 间接块中: entry[0..987] = 0 (块 12-999 是 hole)
//
// 磁盘实际占用: 3 块 (块 0 的数据 + 间接块 + 块 1000 的数据)
// 逻辑大小: 1001 * 4096 + 5 = 4,102,405 字节
// 实际占用: 3 * 4096 = 12,288 字节 (节省 99.7%)
```

```
// 读取 hole 区域:
char buf[4096];
lseek(fd, 500 * 4096, SEEK_SET); // 偏移在 hole 中间
read(fd, buf, 4096); // 返回全零页, 不产生磁盘 I/O!
// 内核发现 i_block[] 指针 = 0, 直接返回 zero-filled page
```

在经典间接指针表示中, `i_block[]` 的 0 指针表示相应逻辑块尚未分配, 读取结果按零填充。这个区间称为 *hole*。读取 *hole* 无须读取数据块; 写入其中某一段时, 文件系统才为相应范围分配物理空间。虚拟机镜像和数据库文件常利用这一性质表示逻辑容量很大、实际写入范围较小的文件。

inode 表的布局

inode 表是一段连续的磁盘块区域, 存储所有 inode 的 on-disk 记录。每个 inode 256 字节, 每个块 4096 字节, 因此每个块容纳 $4096/256 = 16$ 个 inode。

```
inode 表布局 (blocks 11--42, 32 blocks, 512 inodes):

Block 11: inode 0    inode 1    inode 2    ... inode 15
Block 12: inode 16   inode 17   inode 18   ... inode 31
Block 13: inode 32   inode 33   inode 34   ... inode 47
...
Block 42: inode 496  inode 497  inode 498  ... inode 511

inode N 的位置:
    block = inode_table_start + N / 16
           = 11 + N / 16
    offset = (N % 16) * 256

例: inode 5
    block = 11 + 5 / 16 = 11 + 0 = 11
    offset = (5 % 16) * 256 = 5 * 256 = 1280

    inode 5 位于块 11 的字节偏移 1280 (0x500)
```

常见误区

ext2 中 inode 编号从 1 开始 (inode 0 无效), 但 inode 表在磁盘上从偏移 0 开始存储。因此 inode 1 在表偏移 0, inode N 在表偏移 $(N - 1) \times \text{inode_size}$ 。MDFS 为了简化教学, 让 inode N 在表偏移 $N \times \text{inode_size}$, 即 inode 1 在偏移 256。真实 ext2/ext4 中需要减 1。

硬链接与符号链接

硬链接

硬链接 (hard link) 是目录中的一个额外条目, 指向一个已存在的 inode 编号。创建硬链接不创建新 inode, 而是:

1. 在目标目录的数据块中追加一个 `dirent`, `inode` 字段填为被链接文件的 inode 编号, `name` 填为新名字。
2. 将被链接文件的 `i_links_count` 加 1。

```
// 创建硬链接
link("/foo", "/bar");
// 效果:
//   /foo 的目录项: inode=5, name="foo"
//   /bar 的目录项: inode=5, name="bar" (新增)
```

```
// inode 5 的 i_links_count: 1 -> 2
//
// 删除 /foo:
unlink("/foo");
// /foo 的目录项被删除 (inode=0, rec_len 合并)
// inode 5 的 i_links_count: 2 -> 1
// inode 5 不被释放 (links_count > 0)
//
// 删除 /bar:
unlink("/bar");
// /bar 的目录项被删除
// inode 5 的 i_links_count: 1 -> 0
// inode 5 被释放! 数据块回收, inode 位图清零
```

符号链接

符号链接 (symbolic link, symlink) 是一个独立的文件，其 `i_mode` 为 `S_IFLNK`。文件内容是目标路径的字符串。

```
// 创建符号链接
symlink("/foo", "/baz");
// 效果:
// 分配新 inode 6
// inode 6: i_mode = S_IFLNK | 0777
// inode 6: i_links_count = 1
// inode 6: 数据 = "/foo" (4 bytes, 存在 i_block[] 内联)
//
// 路径解析 open("/baz"):
// 1. dir_lookup(root, "baz") -> inode 6
// 2. read inode 6: i_mode = S_IFLNK
// 3. read symlink data: "/foo"
// 4. 重新解析 open("/foo")
// 5. dir_lookup(root, "foo") -> inode 5
// 6. open inode 5

// 删除 /foo 后:
unlink("/foo");
// inode 5 被释放 (links_count -> 0)
// /baz 仍然存在, 但指向一个不存在的路径 -> "dangling symlink"
```

对比

方面	硬链接	符号链接
实现	目录项指向同一 inode 编号	独立 inode, 内容为目标路径字符串
inode	与原文件共享同一 inode	有自己的 inode
i_links_count	创建时 +1	新 inode 的 links_count = 1
原文件删除	数据不丢 (links_count > 0)	链接悬空 (dangling)
跨文件系统	不行 (inode 号是 FS 局部的)	可以 (只是路径字符串)
链接到目录	通常禁止 (防止循环)	允许
路径解析	直接找到 inode (1 次 lookup)	需读链接内容、重新解析 (2+ 次 lookup)
inode 大小	无额外空间 (仅目录项)	需要新 inode + 路径字符串

核心思想

硬链接和符号链接代表了两种不同的引用模型：硬链接是物理引用（直接指向存储对象的编号），符号链接是逻辑引用（通过路径名间接指向）。硬链接的代价是不能跨文件系统、不能链接目录；符号链接的代价是解析时有额外开销，且原文件移动/删除后链接失效。Unix 的设计哲学是同时提供两种机制，让用户根据场景选择。

inode 分配：Orlov 分配器

ext2/ext3/ext4 面临一个经典问题：新创建的 inode 应该放在 inode 表的哪个位置？这个决策影响数据局部性(data locality)—如果同一目录下的文件 inode 被放在相邻的块组 (block group) 中，读取这些文件时磁头移动更少。

ext4 默认使用 Orlov 分配器（由 Grigory Orlov 提出），其策略是：

- 1. 目录分散：新目录应分散到不同块组，避免同一块组中出现过多目录（目录会产生大量子文件，导致该块组 inode 和数据块耗尽）。Orlov 倾向于选择空闲 inode 和空闲块最多的块组。
- 2. 文件聚集：新文件应与父目录在同一块组中。这样，遍历目录时的 I/O 集中在一个块组内，磁头移动最小。

Orlov 策略示意：

Block Group 0	Block Group 1	Block Group 2
+-----+ inode: root/ /src/ /src/a /src/b +-----+	+-----+ inode: /docs/ /docs/a /docs/b /docs/c +-----+	+-----+ inode: /media/ /media/x /media/y /media/z +-----+

/root 和 /src 在 BG0；/docs 及其文件在 BG1；/media 及其文件在 BG2
目录分散到不同 BG；文件与父目录同 BG

Orlov 的核心启发式：目录是“重”的（会产生大量子文件），应分散；文件是“轻”的（不会产生子节点），应聚集。这种“重者散、轻者聚”的策略在大规模文件系统中显著减少了碎片化和寻道开销。

常见误区

ext4 默认启用 EXT4_EXTENTS 标志 (i_flags 中)，此时 i_block[15] 不再是 12+1+1+1 的经典指针方案，而是存储一棵 区段树 (extent tree) 的根节点。区段树用一个 (logical_block, length, physical_block) 三元组描述一段连续的物理块，大幅减少了大文件的元数据开销。经典间接指针方案仅在不支持区段的旧文件系统中使用。但理解经典方案对于理解 inode 的设计哲学仍然至关重要—它是区段方案的前身和对照。

文件偏移到数据块：从指针到 extent 树

问题：间接指针的大文件代价

经典的 UNIX 文件系统 (ext2、UFS) 在 inode 中保存若干直接指针、一个单间接指针、一个双间接指针和一个三间接指针。对小文件这足够了，但当文件增大时，随机访问一个偏移需要多次磁盘 I/O 才能定位数据块。

考虑一个 1 GiB 的文件。在 4 KiB 块、4 字节指针的前提下：直接指针只能覆盖前 12 个块 (48 KiB)，其余都要走间接。在 inode 已经驻留内存而间接块均未缓存时，双间接区要读取 2 个间接块，三间接区要读取 3 个间接块，然后才能读取数据块。对 HDD，这些元数据若分散在不同位置，机械寻道会显著放大延迟；具体数值取决于缓存命中、布局和设备，不能用一个固定毫秒数代表。

常见误区

间接指针的本质缺陷：元数据散布在许多不连续的磁盘块中。一个 100 MiB 的文件需要 25 个间接块，它们随机散布在磁盘各处，导致寻道时间随文件大小线性增长。

各级指针的精确容量

设块大小 $B = 4096$ 字节，指针宽度 $P = 4$ 字节。一个间接块可容纳的指针数为：

$$N = \frac{B}{P} = \frac{4096}{4} = 1024 \text{ (个 uint32 指针)}$$

各级指针的覆盖范围如下表所示。

指针级别	指针数	覆盖块数	覆盖字节
直接 (12 个)	12	12	48 KiB
单间接 (1 个)	1024	1024	4 MiB
双间接 (1 个)	1024^2	1 048 576	4 GiB
三间接 (1 个)	1024^3	1 073 741 824	4 TiB

表 5.1: ext2 inode 各级指针容量 (4 KiB 块、4 字节指针)。

注意双间接理论上可覆盖 4 GiB，但 ext2 inode 的 `i_size` 字段限制了实际最大文件。真正的大文件落在三间接区，随机读需要读 3 个间接块再读数据块 = 4 次 I/O。

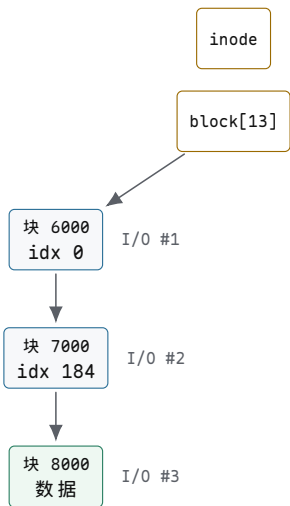
追踪：在双间接区读取偏移 5000000

给定文件偏移 `offset = 5\protect \protect \leavevmode@ifvmode \kern +.1667em\relax 000\protect \protect \leavevmode@ifvmode \kern +.1667em\relax 000` 字节，块大小 4096：

```
logical_block = 5000000 / 4096 = 1220    // 落在双间接区
// 直接区：0..11, 单间接区：12..1035, 双间接区：1036..(1036+1024*1024-1)
// 1220 在双间接区：index_into_double = 1220 - 1036 = 184
// first_level_index = 184 / 1024 = 0
// second_level_index = 184 % 1024 = 184
```

逐步追踪：

1. `logical_block = 1220`，落在双间接区 (1036 起)。
2. inode 已在内存中，其 `block[13]` 给出双重间接根块号 6000；读取块 6000 (第 1 次 I/O)。
3. 在块 6000 中取索引 0 的指针得到二级间接块号 7000；读取块 7000 (第 2 次 I/O)。
4. 在块 7000 中取索引 184 的指针得到数据块号 8000。
5. 读块 8000 → 数据。
6. 总计：2 次间接块 I/O + 1 次数据读 = 3 次 I/O (inode 已在内存、间接块均未缓存时)。若连 inode 所在磁盘块也未缓存，还需另计读取 inode 的 I/O。



图示：双间接读偏移 5000000 的三次 I/O 链：两个间接块加一个数据块。实际延迟由缓存命中、介质与队列状态决定。

间接指针的问题总结

- **元数据膨胀**：100 MiB 文件需要 $\lceil (100 \times 1024 / 4) \rceil = 25600$ 个数据块，其中间接块约 25 个（每 1024 个数据块 1 个间接块），共 $25 \times 4 = 100$ KiB 纯元数据。
- **随机访问延迟**：三间接读在最坏的无缓存路径上需要 3 次间接块 I/O 加 1 次数据 I/O；SSD 消除了机械寻道，但没有消除额外请求、队列和控制器处理成本。
- **元数据碎片化**：间接块本身可能不连续，进一步加剧寻道。
- **无范围信息**：无法高效表达“这 1000 个块连续”，导致 FIEMAP 类查询效率低。

Extent：用 (start,length) 表达连续性

核心思想

Extent 把“一个数据块指针”升级为“一段连续数据块的描述”：一个 extent 记录 (logical_block, start_block, length)，能覆盖任意长的连续区域。100 MiB 的连续文件只需 1 个 extent = 12 字节，而间接指针方案需要 25 个间接块 = 100 KiB 元数据。

ext4 extent 的字段组织

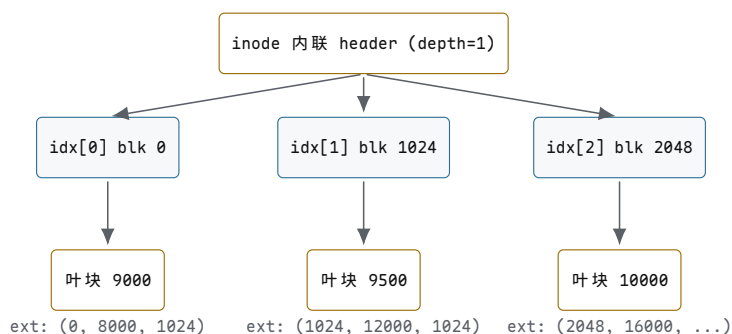
```
// 叶子节点中的 extent：描述一段连续物理块
struct ext4_extent {
    __le32 ee_block;           // 逻辑块号（文件内偏移）
    __le16 ee_len;             // 覆盖的块数（最多 32768）
    __le16 ee_start_hi;        // 物理起始块高 16 位
    __le32 ee_start_lo;        // 物理起始块低 32 位
};

// 索引节点：指向子节点（另一 extent 块）
struct ext4_extent_idx {
    __le32 ei_block;           // 此索引覆盖的逻辑块号下界
    __le32 ei_leaf_lo;         // 子节点块号低 32 位
    __le16 ei_leaf_hi;         // 子节点块号高 16 位
    __u16 ei_unused;           // 保留
};
```

```
// 每个 extent 块 (含 inode 内联) 的头部
struct ext4_extent_header {
    __le16 eh_magic;        // 0xF30A, 标识 extent
    __le16 eh_entries;      // 实际条目数
    __le16 eh_max;          // 最大条目数
    __le16 eh_depth;        // 0=叶子, 1=一层索引, 2=两层
    __le16 eh_generation;   // 用于一致性校验
};
```

inode 尾部的 60 字节 (`i_block[15]`) 内联一个 extent header + 4 个 extent。`eh_depth=0` 时这 4 个 extent 直接描述数据；文件更大时 `eh_depth=1`，内联区变为 4 个 `ext4_extent_idx`，每个指向一个叶子块；再大则 `eh_depth=2`，极端情况可覆盖 ~ 4 PiB。

Extent 树结构



图示：ext4 extent 树。根内联于 inode，depth=1 时指向若干叶块。

追踪：在 depth=1 树中查找逻辑块 1500

前提：inode 内联 header `depth=1`，有 3 个索引条目。

1. 读内联 header: `depth=1`，3 个 `ext4_extent_idx`。
2. 二分查找：`ei[0].block=0`，`ei[1].block=1024`，`ei[2].block=2048`。1500 落在 $[1024, 2048)$ ，即 `ei[1]`。
3. 读 `ei[1].leaf` → 叶块 9000。
4. 块 9000 header: `depth=0`，2 个 extent: `ext[0]=\{block=1024, start=8000, len=1024\}`，`ext[1]=\{block=2048, start=12000, ...\}`。
5. 1500 落在 `ext[0]`: 物理块 = $8000 + (1500 - 1024) = 8476$ 。
6. 总计：1 次 I/O (叶块) + 1 次数据读 = 2 次 I/O。

对比：在 inode 驻留内存且元数据均未缓存时，双间接方案需要 2 次间接块 I/O，extent 树在这个示例里只需读取 1 个叶块，因为内联 header 不需要 I/O，而索引层只多一层。

Extent 的分裂与合并

- **合并**：当新写入恰好紧接在已有 extent 末尾（逻辑块连续、物理块连续），直接把 `ee_len` 加 1，不新增条目。
- **分裂**：当 extent 块已满 (`eh_entries == eh_max`)，需要分裂：把一半条目移到新块，在父层插入新索引。若父层也满，递归分裂。
- **延迟分裂**：ext4 默认不立即分裂，先尝试在现有 extent 上扩展 `ee_len`，减少树高度。

Extent 树中的洞

两个 extent 之间的逻辑块间隙即为洞 (hole)。读洞时返回零页—ext4 不为洞分配物理块。FIEMAP ioctl 返回 FIEMAP_EXTENT_UNKNOWN 标志标记洞。写入洞的某个偏移时，ext4 分配块并在 extent 树中插入新条目。

XFS B+ 树：无历史包袱的设计

XFS 从设计之初就以 B+ 树为映射结构，没有间接指针的历史包袱。其 extent 记录格式更简洁：

```
// XFS B+ tree 叶子节点 (extent 记录)
struct xfs_bmbt_rec {
    __be64 br_startoff;    // 逻辑块号
    __be64 br_startblock;  // 物理块号
    __be64 br_blockcount;  // 块数
};
// 内部节点：键 + 指针
struct xfs_bmbt_key { __be64 br_startoff; };
struct xfs_bmbt_ptr  { __be64 br_startblock; };
```

XFS 的 B+ 树是真正的平衡树：插入/删除后自动调整高度与分裂/合并，而 ext4 extent 树的深度在多数文件上固定为 0-2。XFS 的优势在于动态伸缩：小文件 1 层即可，超大文件自动增到 3-4 层，查找代价始终 $O(\log n)$ 。

映射方案对比

方案	元数据大小 (100 MiB 连续文件)	查找复杂度	碎片化倾向
ext2 间接指针	~ 100 KiB (25 间接块)	$O(\text{层数})$ ，最多 4 I/O	高
ext4 extent	12 字节 (1 extent)	$O(\log n)$ ，1-2 I/O	低
XFS B+ 树	~ 24 字节 (1 叶记录)	$O(\log n)$ ，严格平衡	最低

表 5.2：三种块映射方案的定量对比。

空闲空间、块组与分配局部性

分配问题

当 write() 写入新数据时，文件系统必须决定把新块放在磁盘哪里。看似简单的选择，却影响三个关键指标：

- 连续性：连续布局使预读和顺序读受益，HDD 上减少寻道。
- 空间利用率：低碎片化意味着空闲空间可被大请求复用。
- 分配延迟：write() 路径上的分配必须快—它是热路径。
- 局部性：数据块应靠近其 inode 和同目录的其他文件，使目录遍历（如 ls -R）时的寻道最小。

这些目标常常冲突：best-fit 减少碎片但扫描慢；first-fit 快但可能产生外部碎片。现代文件系统用多层策略逼近帕累托前沿。

Bitmap 分配

最朴素的方案：每个块用 1 bit 标记空闲（0= 空闲，1= 已用）。1 GiB 磁盘、4 KiB 块：

$$\text{块数} = \frac{1 \text{ GiB}}{4 \text{ KiB}} = 262144 \Rightarrow \text{bitmap 字节} = \frac{262144}{8} = 32768 \text{ B} = 32 \text{ KiB}$$

即 8 个块大小的 bitmap。分配时扫描 bitmap 找第一个 0 bit，复杂度 $O(n)$ (n 为块数)。对 1 TiB 磁盘，bitmap 达 32 MiB，全扫描不可接受，因此实际中 bitmap 按块组分区，配合缓存与索引。

first-fit / best-fit / worst-fit

策略	规则	碎片化	速度
first-fit	选第一个足够大的空闲区	中等，外部碎片在前部	快
best-fit	选最小的足够大空闲区	低内部碎片，高外部碎片	慢（全扫描）
worst-fit	选最大的空闲区	高（浪费大区）	慢

表 6.1：三种经典分配策略。

first-fit 追踪（空闲区链表：[0-31]，[40-71]，[80-95]，请求 8 块）：选 [0-31] 的前 8 块 → 剩 [8-31]。下次请求 28 块：[8-31] 只有 24 不够 → 跳到 [40-71]，分走 28，剩 [68-71]。外部碎片出现在前部。

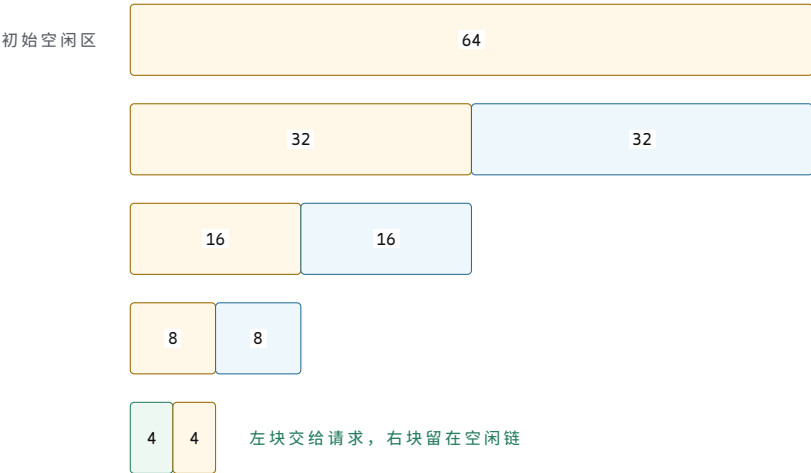
best-fit 追踪（同上，请求 8 块）：扫描全部：[0-31](32)、[40-71](32)、[80-95](16)。8 最接近 16，选 [80-95]，分 8 块 → 剩 [88-95](8)。碎片被挤到小区域，但每次都要全扫描。

Buddy 分配器

Buddy 系统把空闲空间按 2^k 大小组织。分配请求 m 块时向上取整到 $2^{\lceil \log_2 m \rceil}$ ，递归分裂更大的块。

追踪：初始一个 64 块的空闲区，请求 4 块：

```
free[6] = {64}           // 只有 1 个大小为 64 的区
请求 4 块：
  分裂 64 -> 32 + 32,    选第一个 32
  分裂 32 -> 16 + 16,   选第一个 16
  分裂 16 -> 8 + 8,     选第一个 8
  分裂 8 -> 4 + 4,      选第一个 4 -> 分配
free[3] = {4}            // 剩下的 4
free[4] = {8}            // 剩下的 8
free[5] = {16}           // 剩下的 16
free[6] = {32}           // 剩下的 32
```

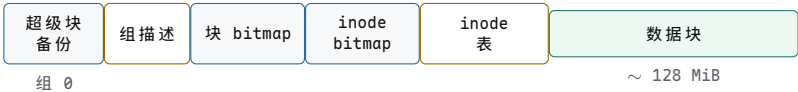


图示：Buddy 分配器：64 块区为服务 4 块请求，分裂 3 次，产生 3 个“伙伴”空闲块。

Buddy 优势：合并 $O(1)$ （释放时检查伙伴是否也空闲），但内部碎片高：请求 5 块也要分 8 块。ext4 的 `mballoc` 在 buddy 之上做了多块优化。

ext2 块组

ext2 把磁盘分成块组（block group），每组约 32768 个块（4 KiB 块时 ~ 128 MiB/组）。每组内部布局：



图示：ext2 块组布局。元数据与数据同居一组，利于 HDD 局部性。

核心理想

块组的三大设计理由：

1. 局部性：inode 与其数据块在同一组内，目录遍历寻道小。

2. 并行性：不同组可被不同 CPU 并行分配（各组有独立 bitmap 锁）。

3. 故障隔离：一个组 bitmap 损坏不影响其他组。

ext4 mballoc：多块分配器

ext4 的 `mballoc`（multi-block allocator）在块组之上叠加多层策略：

- **Buddy bitmap**：每组维护 buddy 结构，查找连续 2^k 块的代价 $O(\log(\text{组内块数}))$ 而非 $O(n)$ 。
- **Locality group（局部性组）**：尝试把同一 inode 的数据和同目录的新文件分配到同一组。
- **Preallocation（预分配）**：对正在增长的文件，预留 8-32 KiB 连续块，使后续 `write()` 直接命中预留区。
- **Goal（目标）**：记录上次分配的最后块号，新分配尽量靠近 goal，使 extent 可合并。
- **Multi-block 一次性分配**：一次 `writeback` 可能脏了上百个页，`mballoc` 一次分配所有块，减少锁竞争。

延迟分配 (delalloc)

延迟分配在 `buffered write()` 阶段先记录脏缓存和逻辑范围，暂缓决定物理块。到 `writeback` 阶段，分配器能够观察更完整的脏范围，并尝试一次分配连续 `extent`。这改善了布局机会，但也意味着空间不足等错误可能延迟到写回或 `fsync` 才出现。

对比两种场景（写 3 个 4 KiB 页）：

```
// 无 delalloc: 每次 write() 立即分配
write(0..4095):    alloc block 5000
write(4096..8191): alloc block 8000    // 5000 已被占用，跳到 8000
write(8192..12287):alloc block 3500    // 又跳到 3500
结果: 3 个 extent, 高碎片化

// 有 delalloc: writeback 看到全部脏页
write(0..4095):    page cache only
write(4096..8191): page cache only
write(8192..12287):page cache only
flush / writeback: alloc 5000-5002 (连续)
结果: 1 个 extent, 无碎片
```



图示：延迟分配让 `writeback` 一次性分配连续块，消除逐次分配导致的碎片化。

delalloc 的 ENOSPC 困境

常见误区

延迟分配带来一个微妙问题：`write()` 在页缓存中成功，但 `writeback` 时可能发现磁盘已满（`ENOSPC`）。此时 `write()` 早已返回 0（成功），用户以为数据已落盘，却在后台刷盘时失败。

`ext4` 的缓解措施：

- **预留块**：可为特权用户保留一部分空间，降低普通用户写满文件系统后管理员完全无法修复或腾挪数据的风险；这不是某个名为 `pdflush` 的进程专用空间。
- **写时检查**：在 `write()` 路径上做粗略空间检查（`ext4_nospace_check`），但这是尽力而为。
- **回退到立即分配**：当剩余空间低于阈值，关闭 `delalloc` 改为同步分配，保证 `ENOSPC` 在 `write()` 时即报错。

Orlov 分配器

Orlov 策略针对目录分配：新目录应分散到不同块组，避免所有热目录挤在一个组导致该组 `inode` 表与 `bitmap` 成为瓶颈。规则：

- 新目录选空闲率最高的组（`free_blocks` 和 `free_inodes` 都较高）。
- 同一目录下的文件（非子目录）仍跟随父目录，保证目录内局部性。
- 目录分散 → 子树分散 → 负载均衡。

XFS 分配组 (Allocation Group, AG)

- XFS 把磁盘分成独立的 AG，每组有自己的超级块、inode 表、两棵 B+ 树：
- **by-size 树**：键 = 空闲区长度，便于 best-fit 查找。
 - **by-location 树**：键 = 起始块号，便于局部性分配。
- 两棵树互补：by-size 满足空间效率，by-location 满足局部性。不同 AG 可被不同 CPU 并行分配，每组各持一把自旋锁，mp_io_resource 级别的锁竞争极低。

名称索引：线性目录、hash 与 B-tree

目录问题

目录本身也是一个文件，其”数据”是名字 → inode 号的映射表。关键问题：如何组织这张表使查找快、增删快、且向后兼容？

理想目录应满足：查找 $O(\log n)$ 或 $O(1)$ ，插入 $O(1)$ amortized，删除不移动数据（避免大目录整块搬移）。不同文件系统选择了不同平衡点。

线性目录 (ext2/ext4 基础格式)

ext2/ext4 的目录数据块是一串变长 dirent：

```
struct ext4_dirent {
    __le32 inode;        // 0 表示已删除
    __le16 rec_len;      // 本条目总长度 (含 padding)
    __u8  name_len;      // 文件名实际字节数
    __u8  file_type;     // DT_REG, DT_DIR, DT_LNK, DT_CHR, DT_BLK, DT_FIFO, DT_SOCK
    char  name[];        // name_len 字节, NUL 填充到 4 字节边界
};
```

rec_len 是变长布局的关键：它允许条目大小不等，删除时只需把被删条目的 rec_len 加到前一条目上一零数据搬移。

具体目录块 (块 4096 字节，含 .、..、foo.txt、bar.c)：

偏移	inode	rec_len	name_len	file_type	name
0	12	12	1	DT_DIR	.
12	2	12	2	DT_DIR	..
24	15	20	7	DT_REG	foo.txt
44	16	16	5	DT_REG	bar.c
60	0	4036	0	—	(剩余空间, 空闲)

表 7.1：一个 ext4 目录块的具体布局。rec_len 使最后一条目吞掉所有剩余空间。

删除 foo.txt：把偏移 24 的条目 rec_len 累加到偏移 12 的 ..：.. 的 rec_len 变为 12 + 20 = 32。foo.txt 的空间被”折叠”进前一条，无需移动 bar.c。查找时遇到 inode==0 直接跳过。

线性查找复杂度

线性扫描是 $O(n)$ 。对 10 个文件的目录， $< 1 \mu s$ ，完全可接受。对 100000 个文件的目录 (如 /usr/lib、邮件 spool)，每次 stat 都要扫半个目录—灾难性。这驱动了 HTree 和 B-tree 目录的诞生。

HTree: ext3/ext4 的 hash B-tree

HTree (也叫 `dx_tree`) 是在线性目录之上叠加的 `hash` 索引。目录数据块仍是线性 `dirent`，但额外维护一个 B-tree 索引：

- **Hash 函数**：`half_md4` 或 `dx_hash` (tear、rupasov 等)。对文件名算 `hash`，`hash` 决定它落在哪个数据块。
- **索引节点**：一个 (`hash`, `child_block`) 对的数组。
- **查找**：`hash(name)` → 在索引中二分 → 读 `child_block` → 在该块的线性 `dirent` 中扫描。

```
struct dx_node {
    struct dx_entry entries[]; // 变长
};
struct dx_entry {
    __le32 hash;           // 子树的最小 hash
    __le32 child_block;    // 子目录块号
};
// 根索引内联于目录第一个数据块的开头
// (与 dirent 共存, 通过 dirent 的 rec_len 隐藏索引部分)
```

查找追踪 (目录有 4 个数据块, HTree depth=1):

1. `hash("foo.txt") = 0xA3B7`。
2. 根索引 4 个 entry: `[0x0000$ \rightarrow $blk100, 0x4000$ \rightarrow $blk200, 0x8000$ \rightarrow $blk300, 0xC000$ \rightarrow $blk400]`。
3. 二分: `0xA3B7` 落在 `[0x8000, 0xC000)` → `blk300`。
4. 读 `blk300`，线性扫描找 `name=="foo.txt"` → `inode 15`。
5. 总计: 1 次索引 I/O + 1 次数据 I/O = 2 I/O (对比纯线性 4 I/O)。

核心理想

HTree 的精妙在于向后兼容：不理解 HTree 的工具 (旧版 `e2fsck`、只读挂载) 看到的仍是线性 `dirent`，因为索引被巧妙隐藏在第一个条目的 `rec_len` 里。只有 `EXT4_INDEX_FL` 标志位告知新内核“这里有索引”。

B-tree 目录 (XFS、Btrfs)

XFS `dir2` 和 Btrfs 把目录本身做成 B-tree，不再有线性 `dirent` 层：

- **XFS `dir2`**：叶节点存 `dirent` 数据，内部节点存 (`hash`, `ptr`) 索引。查找 $O(\log n)$ ，插入/删除自动再平衡。
- **Btrfs**：所有元数据统一为 B-tree，键为 (`objectid`, `type`, `offset`)，目录条目的 `type=BTRFS_DIR_ITEM_KEY`，值为完整 `dirent`。整个文件系统就是一棵大 B-tree 的不同子树。

B-tree 目录的优势：所有操作 $O(\log n)$ ，可以扩展到百万级条目 (如容器镜像层的 `/usr/lib`)。代价是结构复杂、小目录有额外开销 (一个节点至少一个块)。

dentry 缓存 (dcache)

`dcache` 是 VFS 层的路径解析结果缓存，与具体文件系统无关：

- **正向 dentry**：名字存在，缓存了对应 `inode` 指针。再次访问该名字直接命中，无需调用底层 `lookup`。

- **负向 dentry**：名字不存在。缓存“不存在”这一事实，避免对缺失文件的反复磁盘查找（典型场景：`PATH` 搜索中 90% 的路径不存在）。

`dcache` 是基于路径的：`/home/user/foo` 被缓存为一串 dentry: `root $ \rightarrow $ home $ \rightarrow $ user $ \rightarrow $ foo`。每段都是独立缓存条目，可被不同路径复用。

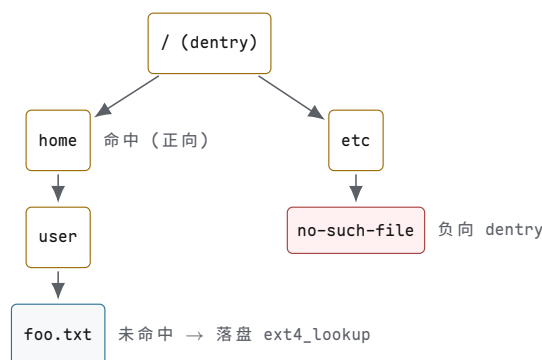
RCU-walk：Linux 3.2+ 引入的无锁路径遍历。遍历 dentry 树时用 RCU (Read-Copy-Update) 保护，不取任何锁，只在需要修改（创建/删除 dentry）时降级为 ref-walk。热路径 `stat("/usr/bin/gcc")` 几乎全在 RCU 下完成。

失效：`rename`、`unlink`、跨挂载点变化必须使受影响 dentry 失效 (`d_invalidate`)，否则返回过时结果。

带 dcache 的完整路径解析

```
vfs_lookup("/home/user/foo.txt"):
root_dentry = dcache_lookup(NULL, "/")          // 命中：根 dentry
home_dentry = dcache_lookup(root_dentry, "home") // 命中?
if !home_dentry:
    home_inode = ext4_lookup(root_inode, "home") // 落盘
    home_dentry = d_add(root_dentry, "home", home_inode)
user_dentry = dcache_lookup(home_dentry, "user") // 命中?
if !user_dentry:
    user_inode = ext4_lookup(home_inode, "user")
    user_dentry = d_add(home_dentry, "user", user_inode)
foo_dentry = dcache_lookup(user_dentry, "foo.txt") // 命中?
if !foo_dentry:
    foo_inode = ext4_lookup(user_inode, "foo.txt")
    foo_dentry = d_add(user_dentry, "foo.txt", foo_inode)
return foo_dentry->d_inode
```

热路径下，每一级 `dcache_lookup` 都命中，`ext4_lookup` 永不被调用——整个解析在内存中 $O(\text{路径深度})$ 。



图示：dcache 树：正向 dentry 缓存存在路径，负向 dentry 缓存“不存在”。

VFS 对象与统一分发

VFS 问题

Linux 支持 `ext4`、`xfs`、`btrfs`、`nfs`、`proc`、`sysfs`、`tmpfs`、`fuse` 等数十种文件系统。应用程序只想用 `open/read/write/close`，不关心底层是 `ext4` 还是网络协议。

核心思想

VFS (Virtual File System Switch) 是内核中的一层抽象接口与分发器：它定义了一组抽象对象和操作方法表，每个具体文件系统实现这些方法。应用层看到统一 API，内核根据挂载信息把调用分发到具体实现。

四个核心 VFS 对象

super_block：每个已挂载文件系统一个

```
struct super_block {
    unsigned long s_blocksize;    // 块大小（字节）
    unsigned long s_maxbytes;    // 最大文件大小
    unsigned long s_flags;       // MS_RDONLY, MS_NOSUID, ...
    unsigned long s_magic;       // 0xEF53 (ext2/3/4), 0x58465342 (xfs)
    struct dentry *s_root;       // 此挂载实例的根 dentry
    struct list_lru s_dentry_lru; // 可回收 dentry 的运行时状态（示意）
    const struct super_operations *s_op;
    // ...
};

struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb);
    void (*destroy_inode)(struct inode *);
    void (*evict_inode)(struct inode *);
    int (*write_inode)(struct inode *, struct writeback_control *);
    int (*sync_fs)(struct super_block *, int wait);
    // ...
};
```

`super_block` 在 `mount()` 时由 `fill_super` 回调创建，卸载时销毁。它持有全文件系统级别的状态（空闲块/inode 计数、脏标志）。

inode：每个文件一个（在内存中）

```
struct inode {
    umode_t i_mode;    // 文件类型与权限
    loff_t i_size;     // 文件大小（字节）
    blkcnt_t i_blocks; // 占用的 512 字节块数
    const struct inode_operations *i_op;
    const struct file_operations *i_fop;
    struct address_space *i_mapping; // 页缓存
    struct super_block *i_sb;
    // ...
};

struct inode_operations {
    struct dentry *(*lookup)(struct inode *, struct dentry *, unsigned int);
    int (*create)(struct inode *, struct dentry *, umode_t, bool);
    int (*link)(struct dentry *, struct inode *, struct dentry *);
    int (*unlink)(struct inode *, struct dentry *);
    int (*mkdir)(struct inode *, struct dentry *, umode_t);
    int (*rename)(..., unsigned int);
    const char *(*get_link)(struct dentry *, struct inode *, void **);
    int (*permission)(struct inode *, int);
    // ...
};
```

内存 inode 由磁盘 inode 实例化，但额外维护运行时状态（页缓存引用、脏标志、锁、引用计数）。

dentry: 路径组件缓存

```
struct dentry {
    struct qstr d_name;      // 名字 + hash
    struct inode *d_inode;   // NULL = 负向 dentry
    struct dentry *d_parent;
    const struct dentry_operations *d_op;
    struct list_head d_subdirs;
    unsigned char d_flags;   // DCACHE_MOUNTED, ...
    struct lockref d_lockref; // 自旋锁与引用计数的组合
};
```

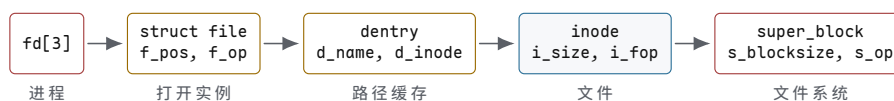
dentry 形成镜像目录结构的树，是 dcache 的载体。

file: 每次 open() 一个

```
struct file {
    struct path f_path;      // dentry + vfsmount
    struct inode *f_inode;
    const struct file_operations *f_op;
    loff_t f_pos;            // 当前偏移
    fmode_t f_mode;          // FMODE_READ, FMODE_WRITE
    unsigned int f_flags;     // O_RDONLY, O_NONBLOCK, ...
    atomic_long_t f_count;    // 引用计数
    // ...
};

struct file_operations {
    ssize_t (*read_iter)(struct kiocb *, struct iov_iter *);
    ssize_t (*write_iter)(struct kiocb *, struct iov_iter *);
    int (*open)(struct inode *, struct file *);
    int (*release)(struct inode *, struct file *);
    int (*mmmap)(struct file *, struct vm_area_struct *);
    int (*fsync)(struct file *, loff_t, loff_t, int);
    long (*ioctl)(struct file *, unsigned int, unsigned long);
    // ...
};
```

对象关系: fd → file → dentry → inode → super_block



图示: VFS 对象链: fd 引用 file, file 指向 dentry 和 inode, inode 属于 super_block。

分发机制

VFS 的核心是函数指针表分发。vfs_read 不直接读数据，而是调用 file->f_op->read_iter，后者对 ext4 是 ext4_file_read_iter，对 xfs 是 xfs_file_read_iter:

```
ssize_t vfs_read(struct file *file, char __user *buf, size_t len, loff_t *pos) {
    if (file->f_op->read_iter)
        return new_sync_read(file, buf, len, pos); // 调 read_iter
    else if (file->f_op->read)
        return file->f_op->read(file, buf, len, pos); // 旧接口
    return -EINVAL;
}
```

```
// new_sync_read 最终调用:
// file->f_op->read_iter(kiocb, iov_iter)
// ext4: ext4_file_read_iter -> generic_file_read_iter -> pagecache
// xfs: xfs_file_read_iter -> generic_file_read_iter
// nfs: nfs_file_read_iter -> 网络 RPC
```

VFS 通过延迟绑定完成分发：打开文件时，`file->f_op = inode->i_fop`；具体文件系统在实例化相应 `inode` 时设置操作表。此后每次 `read/write/fsync` 都通过这张函数指针表分发到正确的实现。应用代码、VFS 通用代码、文件系统特定代码三者解耦。

open() 完整路径追踪

1. 用户态 `open()` 通常经 `libc` 进入 `openat` 系统调用；内核当前实现可沿 `do_sys_openat2` 一类入口进入路径打开逻辑。函数名会随版本调整，本追踪强调机制而非背诵符号。
2. `path_openat` 协调路径遍历与最终打开，逐段解析得到 `dentry + inode`。遍历过程中若遇挂载点，`follow_mount()` 跳转到挂载的子树。
3. 权限检查：`inode_permission(inode, MAY_OPEN)` → 调用 `inode->i_op->permission` (`ext4: ext4_permission`)。
4. 分配 `struct file`，设置 `f_pos=0`，`f_op = inode->i_fop`，`f_inode = inode`。
5. 调用 `file->f_op->open`（若存在）做文件系统特定初始化，再调 `security_file_open` (SELinux/AppArmor 钩子)。
6. `fd_install`：把 `file` 装入当前进程的 `fd` 表，返回 `fd` 号。

挂载命名空间

每个容器/命名空间有自己的挂载树。`/proc/mounts` 只显示当前命名空间的挂载。`mount --bind /src /dst` 为同一 `inode` 子树创建第二个 `dentry` 入口——两路径访问同一数据。`mount --move` 可在树间搬移子树。

特殊文件系统

文件系统	后端存储	语义	典型用途
tmpfs	内存 (+swap)	易失，读写	<code>/tmp</code> , <code>/dev/shm</code>
procfs	无	读即生成	<code>/proc/<pid>/status</code>
sysfs	无	属性 ↔ 驱动函数	<code>/sys/class/...</code>
overlayfs	lower+upper	写时复制	容器镜像层叠
fuse	用户态守护进程	请求经 <code>/dev/fuse</code> 往返	自定义文件系统

表 8.1：常见特殊文件系统。

tmpfs

数据在页缓存中，无磁盘 `inode`。可用 `swap` 作后端：内存压力时换出。`shmem_alloc` 在 `shmem_fs_type` 下注册。`ls /dev/shm` 看到的“文件”实为 `shmem_inode_info`。

procfs

`/proc` 无后端存储。`read()` 触发 `proc_generate()` 动态生成内容。如 `/proc/meminfo` 的 `seq_file` 回调 `meminfo_show` 实时遍历 `global_node_page_state` 等计数器。

sysfs

`/sys` 是 `kobject` 层次的镜像。每个 `kobject` 对应一个目录，每个 `attribute` 对应一个文件。`read(attribute)` 调用驱动注册的 `show` 回调，`write(attribute)` 调用 `store` 回调——直接映射到驱动函数，无“文件”实体。

overlayfs

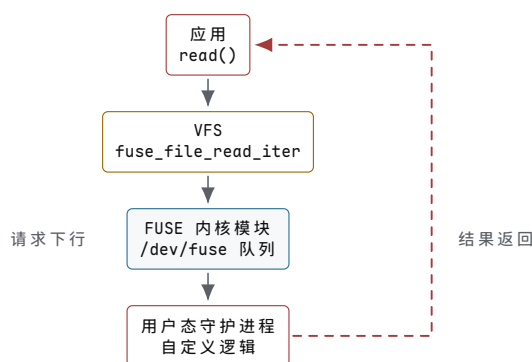
叠加 `lower`（只读）和 `upper`（可写）层。读时自顶向下找第一层命中；首次写时写时复制（`copy-up`）：把 `lower` 文件复制到 `upper` 再修改。Docker 镜像层叠正是基于 `overlayfs`。

FUSE

`fuse` 把文件操作路由到用户态守护进程：

1. 内核 VFS 调用 `fuse_file_read_iter`。
2. 内核把请求写入 `/dev/fuse` 字符设备，守护进程阻塞在 `read(/dev/fuse)`。
3. 守护进程处理请求（可能是网络、内存、任意逻辑），把结果 `write` 回 `/dev/fuse`。
4. 内核唤醒等待的 `vfs_read`，返回数据给应用。

代价是每次 I/O 两次内核 ↔ 用户态切换，延迟 $\sim \mu\text{s}$ 级，不适合高吞吐场景。优势是文件系统逻辑可在用户态实现，无需内核权限。



图示：FUSE 请求往返：应用 → VFS → FUSE 内核 → 用户态守护进程，结果原路返回。

小结

本节覆盖了从块映射到 VFS 的完整栈：

- 块映射从间接指针演化到 extent 树与 B+ 树，把大文件的元数据从 100 KiB 降到 12 字节，查找从 4 I/O 降到 2 I/O。
- 空间分配用块组 + buddy + 延迟分配的组合，在连续性、利用率、延迟间逼近帕累托前沿。
- 目录结构从线性表升级到 HTree 和 B-tree，把 $O(n)$ 查找变为 $O(\log n)$ ，并辅以 dcache 实现热路径零 I/O。
- VFS 用函数指针表分发统一 API，使 `ext4/xfs/nfs/tmpfs/fuse` 共享同一套 `open/read/write` 接口。

这些机制层层叠叠：VFS 分发 → 文件系统特定操作 → extent 树查找 → 块分配 → 目录索引 → dcache 命中。理解每一层的权衡与数据结构，才能诊断从“大文件随机读慢”到“容器挂载层叠”的各类性能问题。

第 13 章 缓存、写回与崩溃一致性

核心思想

缓存把“程序已经修改的状态”和“介质已经保存的状态”分离开来。只要一次文件操作会改变多个持久对象，崩溃就可能落在这些写入之间。本章先确定可见性与持久性的不同边界，再从半完成更新推导写回顺序、日志提交、检查点和恢复规则。

create、unlink、rename 与 truncate

前一节说明了 create 会修改位图、inode、目录项和数据块。结构清单还不足以构成正确实现：并发线程可能同时修改同一目录，资源分配可能在中途失败，设备也可能只持久化部分写入。因此，一个文件操作还需要规定加锁范围、错误回收、对外可见点和持久化顺序。

可见性、持久性与一致性

分析操作结果时，需要分别观察三个边界：

可见性 (visibility) 其他线程现在能否通过名字或已打开的文件描述符观察到修改。它讨论的是运行中内存状态，不自动意味着数据已经掉电不失。

持久性 (durability) 系统成功返回某个同步边界后，掉电重启是否仍能恢复该修改。它依赖文件系统提交协议、块层顺序和设备对 flush/FUA 的兑现。

一致性 (consistency) 位图、inode、目录项和块映射是否仍满足格式规定的不变量。一致的文件系统也可能保存旧数据；“结构能挂载”不等于“应用的最后一次写已持久”。

`write` 返回通常建立的是新字节对进程的可见性；`rename` 可以建立名字切换的原子可见性；`fsync` 请求建立持久性。三个边界有关联，却不是同一个边界。把它们统称为“原子写”会掩盖真正需要证明的性质。

锁、事务、持久化顺序与回滚

锁 (*lock*) 是运行期间的互斥或同步规则。例如父目录锁可以防止两个 `create` 同时把同一个名字插入目录；inode 锁可以协调 `truncate` 与 `write`。锁保存在内存中，断电后会消失，所以锁本身不能提供崩溃恢复。

事务 (*transaction*) 把多项持久修改归为一个恢复单位。日志型文件系统先把“本事务要发布哪些元数据版本”写入日志，并用 `commit record` (提交记录) 表示事务已经完整。恢复程序只重放具有有效提交证据的事务。事务解决的是“掉电后该接受哪组修改”，并不替代运行中的锁。

持久化顺序 (*write ordering*) 规定某个写在另一个写获得持久性之前必须先完成。例如 `ordered` 模式需要先保证新文件数据已写出，再提交让该块可达的元数据，否则重启后可能通过正确目录名读到旧块残留。设备可以重排普通写，因此文件系统还要通过依赖关系、flush、FUA 或等价协议把逻辑顺序传到设备。

回滚 (*rollback*) 处理的是尚未发布时的运行期错误。例如分配 inode 成功、分配数据块却返回 `ENOSPC`，就要撤销 inode 位图和计数的修改。日志恢复常用的是重放已提交事务，而不一定在磁盘上逐步执行传统意义的“反向撤销”。运行期回滚与崩溃恢复处理的时间边界不同，所需证据也不同。

机制	解决的问题	适用边界
锁	防止并发交错破坏运行状态	只在系统运行期间有效，不能跨越掉电
运行期回滚	回收系统调用中途失败后已取得的资源	在错误返回前恢复内存与事务状态
日志事务	识别一组元数据更新是否已经完整提交	用于 mount 恢复与 checkpoint；不自动提供普通数据内容原子性
持久化顺序	防止设备重排或易失缓存破坏写入依赖	约束 I/O 到达稳定存储的先后，不负责线程互斥
fsck 式扫描	修复已经违反全局不变量的磁盘结构	可恢复结构与空间，通常无法还原应用的原始意图

create：目录项是名字的可见性发布点

设两个线程同时执行 `open("/d/x", O_CREAT|O_EXCL)`。如果都先查到名字不存在，再分别插入，就可能产生重名目录项或泄漏其中一个 inode。因此检查和插入必须受同一目录修改协议保护，不能在两步之间释放父目录的排他锁。

一次简化但完整的 create 可以按下面的状态机理解：

```

create(parent, "x"):
1. lock(parent directory)           // 串行化同目录名字修改
2. lookup("x")                     // 在锁内再次确认不存在
3. reserve journal credits          // 预留本事务最坏所需日志空间
4. allocate inode; initialize it    // 还没有目录项，外部路径不可达
5. if initial data is required:
    allocate block; write data
6. insert dirent "x" -> inode       // 发布名字：路径开始可达
7. update parent time/counts
8. attach all metadata changes to transaction
9. unlock(parent directory)

on error before step 6:
free block if allocated
free inode if allocated
cancel or stop transaction
unlock(parent directory)
return error

```

第 6 步建立目录项，是新名字的可见性发布点。在此之前，新 inode 即使已经存在，也没有从根目录出发的路径；在此之后，路径解析可以找到它。发布之前出错，资源应被回滚。发布之后若向用户返回失败，调用者也不能武断地认为名字不存在：网络文件系统、I/O 错误或崩溃边界都可能造成“操作已经生效但答复丢失”，稳健程序要重新查询状态。

日志空间预留发生在结构修改之前。这样，即使操作走到最坏路径，也有足够空间记录维持一致性所需的元数据；若等目录项已经发布后才发现日志已满，提交和回滚都会失去可靠空间保障。

create 各阶段的掉电结果

下面假设没有日志，只按顺序写 MiniFS-Lab 的几个块。表中的“泄漏”是指位图认为资源已占用，却没有从根目录可达的对象拥有它。

掉电点	重启后可能看到	形成原因
-----	---------	------

只写 inode 位图后	一个已占用但未初始化/不可达的 inode	分配状态先于对象正文持久
再写块位图后	不可达 inode，加一个无所有者数据块	两种资源都分了，但映射尚未建立
再写 inode 与数据后	完整数据仍没有名字可达	目录项尚未持久
只持久目录项、inode 未持久	名字指向无效或旧 inode	发布发生在被发布对象之前
全部写完后	结构可能正确	仍要确认设备是否兑现写序与持久化命令

日志把相关元数据版本连同事务号写入日志，完整写出 commit record 后才把事务视为可重放；没有完整提交证据的尾部事务被忽略。ordered data 模式还建立“数据先于使其可达的元数据提交”的顺序。恢复时由提交记录区分完整事务与尾部半成品：前者重放，后者忽略。

unlink：名字消失后的对象生命周期

目录项对 inode 的持久引用数由链接计数表示；进程已打开文件所持有的是运行时引用。这两种引用必须分开。考虑：

```
fd = open("/tmp/report", O_RDWR); // 得到运行时 file/inode 引用
unlink("/tmp/report");           // 删除名字，链接计数可能变为 0
write(fd, "tail", 4);           // 仍然合法
close(fd);                       // 最后一个运行时引用消失
```

unlink 的直接语义是从父目录移除“名字 → inode”映射并减少链接计数。链接计数降为零后，新的路径查找找不到该对象；但只要 fd 仍引用它，内核就不能回收 inode 和数据块。最后一个打开引用关闭后，回收路径才释放块映射和 inode 编号。

这里产生一个新的崩溃窗口：目录项已删除、链接计数为零，但系统在释放全部数据块前掉电。很多日志型文件系统维护类似 orphan list（孤儿链表）的持久恢复线索。它记录“这个 inode 已经没有名字，但清理尚未完成”；mount 恢复时继续 truncate/删除，避免永久泄漏。这条记录是删除协议主动保存的清理进度，不是恢复程序根据残留块进行猜测。

rename：原子可见性与持久发布是两回事

常见安全更新模式是先写临时文件，再把它改名到正式名字：

```
fd = open("config.tmp", O_CREAT | O_TRUNC | O_WRONLY, 0600);
write_all(fd, new_bytes);
fsync(fd); // 先持久化临时文件内容
close(fd);
rename("config.tmp", "config"); // 名字切换对并发查找原子可见
dirfd = open(".", O_DIRECTORY | O_RDONLY);
fsync(dirfd); // 请求持久化目录项变化
close(dirfd);
```

这里的原子可见表示其他进程不会观察到半个目录项：目标名在某个线性化点从旧对象切到新对象。但 rename 成功返回本身不应被泛化成“掉电后新名字必然存在”。文件数据和目录项是两个持久对象，所以先同步文件，再执行 rename，再同步父目录。跨两个目录 rename 时，若应用要求两个目录的变更都可靠，还要按目标平台契约处理相关目录同步。

rename 还暴露了锁顺序问题。两个线程若一个先锁目录 A 再锁 B，另一个先锁 B 再锁 A，就会形成死锁：双方都拿着对方需要的锁等待。VFS 因此规定目录操作的

锁层级与祖先关系规则；跨目录 `rename` 还必须防止把目录移动进自己的后代，从而制造目录环。由此，锁顺序同时承担并发安全和目录无环不变量的维护职责。

truncate：同时收缩长度、缓存和块映射

`truncate(file, 5000)` 看似只改一个长度字段，实际上旧文件若长 20 KiB，它还要：

1. 在 inode 锁与映射失效协议保护下发布新的 `i_size`；
2. 处理 page cache 中新 EOF 之后的 folio，不能让旧缓存内容再次被写回；
3. 将包含新 EOF 的最后一个块尾部清零，避免文件以后增长时泄露旧字节；
4. 删除 EOF 之后的 extent/间接块映射并释放物理块；
5. 与正在进行的 buffered write、direct I/O、writeback 和 mmap page fault 协调；
6. 把长度、块映射和空闲空间修改纳入可恢复的元数据事务。

顺序错误会形成真实漏洞。如果先把物理块分给文件 B，再允许文件 A 的旧脏 folio 写回，A 的写回就可能覆盖 B 的数据；如果不清理最后一个部分块，A 随后扩展时可能读到截断前的旧内容。完整的 truncate 需要撤销 EOF 之后各层次对旧数据的引用，并与仍可能使用这些引用的并发路径建立同步屏障；修改 `i_size` 只是其中一步。

对已 mmap 的文件，映射地址也不是永久拥有物理块。截短后访问新 EOF 之外的页面可能触发 `SIGBUS`；内核需要让相关页表映射和 page cache 状态失效。这个例子再次说明：磁盘 inode、内存 inode、page cache 与进程页表是同一文件的不同层次，不能只改其中一个。

fsync 的对象范围与调用顺序

对普通文件调用 `fsync(fd)`，目标是该文件的数据和恢复它所需的相关元数据；`fdatasync` 可以不强求与数据读取无关的部分元数据，但仍必须同步影响数据正确读取的长度等信息。二者都必须检查返回值，因为延迟分配、writeback 或设备错误可能直到同步时才报告。

文件名属于目录数据，不属于普通文件 inode。因此新建文件的可靠发布通常还需要同步父目录；`rename`、`link`、`unlink` 改的也是目录命名关系。不能因为文件内容已同步，就推出重启后包含它的名字也一定存在。反过来，只同步目录也不能替代先把新文件内容同步好。

持久性检查应明确对象集合和顺序。需要同步哪些文件数据、inode 元数据与父目录项，各操作采用什么先后顺序，返回错误怎样处理，以及设备是否提供正确的持久化语义。应用若需要跨多个文件原子提交，通常还要使用 WAL、manifest 或数据库事务；单个文件的 `fsync` 不会自动把多个文件合并成一个事务。

本节涉及的操作可以用四条不变量统一检查：

1. 每个可达目录项都指向一个已分配、类型合法的 inode；
2. 每个可达 inode 引用的数据块都已分配，且非共享格式中不会被别的 inode 重复拥有；
3. 不可达且无运行时引用的零链接 inode 最终能够回收，清理中断后仍有恢复线索；
4. 对外声称已经持久的状态，必须存在一条从有效超级块/日志提交点到该状态的完整恢复路径。

前两条是空间与引用一致性，第三条是生命周期闭环，第四条是崩溃后的可证明性。后面的 page cache、日志和 COW 看似是新主题，其实都在实现或优化这四条约束。

页缓存、脏页与写回

问题：磁盘比内存慢一千倍

上一节讨论了 VFS 的统一接口与分发机制，但还没有回答一个更根本的问题：每次 `read` 和 `write` 都访问磁盘，系统会慢到什么程度？

先看数字。DDR4 内存访问延迟 ~ 80 ns，带宽 ~ 25 GB/s。NVMe SSD 读取延迟 ~ 100 μ s（慢 1250 倍），带宽 ~ 3 GB/s（慢 8 倍）。HDD 随机读取延迟 ~ 10 ms（慢 125000 倍），带宽 ~ 200 MB/s（慢 125 倍）。如果用户程序每次 `read` 都要等磁盘返回，一个 4 KiB 的读请求在 HDD 上就要花 10 ms；如果一个程序依次读 10000 个 4 KiB 页面，总耗时 100 秒。而如果把这 40 MiB 数据缓存在内存中，10000 次内存拷贝只需 ~ 0.8 ms。差距是五个数量级。

因此，普通文件的 buffered I/O 默认经过内核的文件数据缓存——*page cache*。读操作先查缓存，命中则不产生本次磁盘 I/O；buffered write 通常先修改缓存并标脏，随后由后台写回或同步调用送往设备。Direct I/O 和 DAX 是后文单独讨论的旁路，不能把这段默认路径扩大成所有文件 I/O 的唯一实现。

页缓存按文件映射与页索引组织 folio，使同一文件区间能够被 `read`、`write` 和 `mmap` 共享。buffered read 优先查找缓存；buffered write 通常修改缓存并标脏，随后由后台写回或同步调用提交 I/O。这里描述的是默认 buffered 路径，不包括 Direct I/O 与 DAX。

address_space: 从 (inode, offset) 到 folio

page cache 不是一张全局哈希表。内核为每个 inode 关联一个 `address_space` 对象，它是该 inode 所有缓存页的管理容器。

```
struct address_space {
    struct inode          *host;           // 拥有此 address_space 的 inode
    struct xarray          i_pages;        // page cache 页的索引
    pgoff_t               writeback_index; // 上次 writeback 停止位置
    const struct address_space_operations *a_ops; // 操作表
    unsigned long          nrpages;        // 已缓存页数
    struct rb_root_cached  i_mmap;         // 私有映射的 VMA 树
    struct rw_semaphore    invalidate_lock; // 截断/失效与填充的互斥边界
    struct rw_semaphore    i_mmap_rwsem;   // mmap 信号量
    errseq_t              wb_err;          // 异步写回错误序列
    unsigned long          flags;
};
```

`host` 指回拥有此 `address_space` 的 inode。`i_pages` 是 XArray 索引，键表示文件中的页索引，值通常指向 folio。folio 是一个或多个连续基础页的内存管理单位；引入它是为了让页缓存不必永远以单个 4 KiB 页为粒度。`nrpages` 仍以基础页数量记账，不能简单理解成“XArray 中有多少个 folio”。

`a_ops` 是操作表，决定了具体的读、写、写回行为如何落到底层文件系统：

```
struct address_space_operations {
    int (*read_folio)(struct file *, struct folio *);
    void (*readahead)(struct readahead_control *);
    int (*writepages)(struct address_space *,
                      struct writeback_control *);
    int (*write_begin)(const struct kiocb *, struct address_space *,
                      loff_t, unsigned, struct folio **, void **);
    int (*write_end)(const struct kiocb *, struct address_space *,
                     loff_t, unsigned, unsigned,
                     struct folio *, void *);
    bool (*dirty_folio)(struct address_space *, struct folio *);
    void (*invalidate_folio)(struct folio *, size_t, size_t);
    bool (*release_folio)(struct folio *, gfp_t);
};
```



```
};
```

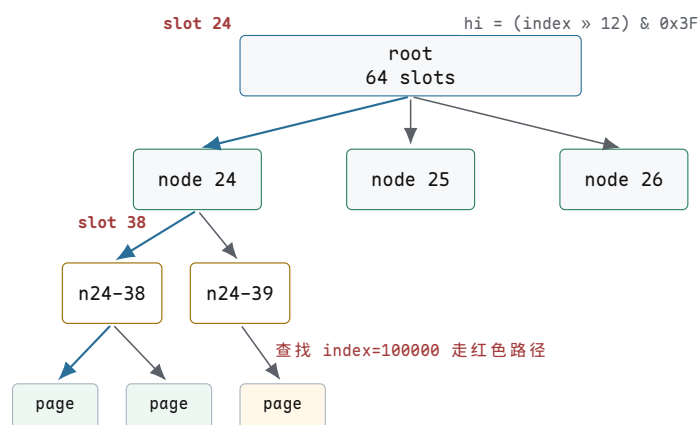
具体字段会随内核版本变化；这里以 Linux 6.16 基线附近的 folio 接口说明职责。许多现代文件系统还通过 iomap 复用 buffered I/O、direct I/O 和块映射的公共实现，从而避免分别维护功能重叠的 page 回调。

XArray：低树高的稀疏索引

`address_space.i_pages` 是一棵 XArray（历史上使用 radix tree）。键以 `pgoff_t` 页索引表达文件位置，普通叶项可指向 folio/page；同一 folio 覆盖多个页索引时还会使用 XArray 的多索引表示。常见 64 位配置中内部节点通常有 64 个槽位（`XA_CHUNK_SHIFT = 6`），但应把它理解为低树高稀疏索引，而不是对所有架构和配置都写死的磁盘格式常量。

```
// 1 GiB 文件，页大小 4 KiB
// 总页数 = 1 GiB / 4 KiB = 262144 = 2^18
// XArray 层级计算：
//   每级 64 个槽 (2^6)
//   2^18 个键需要 ceil(18/6) = 3 级
//   level 0 (root):    64 个槽 → 覆盖 2^6 = 64 页
//   level 1:          64^2 个槽 → 覆盖 2^12 = 4096 页
//   level 2:          64^3 个槽 → 覆盖 2^18 = 262144 页
//
// 查找 page index = 100000:
//   index = 100000 = 0x186A0
//   slot[0] = (100000 >> 0) & 0x3F = 0x20 = 32    // 实际用高 12 位
//   实际分三段：高6位、中6位、低6位
//   hi = (100000 >> 12) & 0x3F = 24    // root → level 1 节点 24
//   mi = (100000 >> 6) & 0x3F = 38    // level 1 → level 2 节点 38
//   lo = (100000 >> 0) & 0x3F = 32    // level 2 → slot 32 = page*
```

在这个 1 GiB、4 KiB 粒度的例子中，路径只有 3 层，因此常数很小；但严格复杂度仍是 $O(\log_{64} n)$ ，不是与索引范围无关的 $O(1)$ 。XArray 的优势还包括稀疏分配、标记和并发访问接口，不能只用“大分支树所以等于常数”概括。



图示：XArray 查找路径。这个教学参数下需要 3 级节点；树高随可表示的索引范围缓慢增长，因此应写成低树高的对数查找，而不是严格 $O(1)$ 。

读路径深追踪

下面追踪一次 `vfs_read(file, buf, 4096, offset=8192)` 的完整执行路径。文件使用 ext4，页大小 4 KiB，offset 8192 对应 page index = 2。

```
// 步骤 1: VFS 入口
ssize_t vfs_read(struct file *file, char __user *buf,
```

```

        size_t count, loff_t *pos)
{
    // offset = 8192, count = 4096
    // 检查权限、锁、文件模式
    if (ret = rw_verify_area(READ, file, pos, count))
        return ret;
    // 分发到 file->f_op->read_iter
    // ext4: ext4_file_read_iter -> generic_file_read_iter
    return generic_file_read_iter(iocb, to);
}

// buffered read 的机制级伪代码
// 步骤 2: 按文件页索引查找 page cache
pgoff_t index = 8192 / 4096; // index = 2
struct address_space *mapping = file->f_mapping;
struct folio *folio = lookup_cached_folio(mapping, index);

// 步骤 3a: 如果命中且 folio 内容有效
if (folio && folio_test_uptodate(folio)) {
    copy_folio_to_iter(folio, 0, 4096, to);
    // 没有设备 I/O; 耗时取决于复制长度、CPU cache、缺页和调度状态
    return 4096;
}

// 步骤 3b: 如果未命中
// 分配并锁住新 folio, 再加入 mapping->i_pages。
// 先发布“正在装入”的 locked folio, 是为了让并发读者等待同一次 I/O,
// 而不是为同一 (inode,index) 重复发起设备读取。
folio = allocate_locked_folio(index);
add_folio_to_mapping(mapping, folio, index);

// 步骤 4: 对 folio 发起读取; 具体文件系统可复用 iomap 等公共路径
mapping->a_ops->read_folio(file, folio);
// read_folio / iomap 路径内部:
// 1. 查 extent 或其他映射, 找到 folio 对应的物理范围
// 2. 按设备约束构造一个或多个 bio
// 3. 提交到块层、驱动和设备
// 4. I/O 完成后设置 uptodate/error, 并解锁 folio

// 步骤 5: 等待 I/O 完成
folio_wait_locked(folio);
if (!folio_test_uptodate(folio))
    return -EIO;

// 步骤 6: 拷贝数据到用户空间
copy_folio_to_iter(folio, 0, 4096, to);
// folio 留在 page cache 中, 下次读同一区间可能直接命中
return 4096;

```

整个读路径的两种结局:

路径	执行步骤	磁盘 I/O	延迟
缓存命中	XArray 查找 → <code>copy_page_to_user</code>	无	~100 ns

缓存未命中	XArray 查找 → 分配 folio → <code>read_folio</code> / iomap → 提交 I/O → 等待 → 拷贝	取决于映射与预读	取决于设备和队列
-------	--	----------	----------

第二次读同一页面时走缓存命中路径，延迟降到 ~ 100 ns。这就是 page cache 对重复读的加速比： ~ 1000 倍。

预读：检测顺序访问并预取

当内核检测到访问模式是顺序的，会主动把后续页面提前读入 cache，避免每次都 cache miss。预读窗口按指数增长：

```
// read-ahead 窗口增长（简化）
// 第一次读 offset=0: cache miss, 读 1 页
// 预读窗口 = 4 KiB (1 page)
// 第二次读 offset=4096: cache miss, 读 1 页 + 预读 3 页
// 预读窗口 = 8 KiB (2 pages), 实际预读 4-16 KiB
// 第三次读 offset=8192: cache hit (被预读)
// 预读窗口增长到 16 KiB (4 pages)
// 内核继续预读下一批
// 第四次读 offset=12288: cache hit
// 预读窗口 = 32 KiB (8 pages)
// ...
// 最终预读窗口稳定在 128 KiB (32 pages)

// 内核使用 onstack_readahead_control 结构
struct readahead_control {
    struct file *rac_file;           // 关联的 file
    struct address_space *mapping;
    pgoff_t _index;                 // 预读起始页索引
    unsigned int _nr_pages;          // 本次预读页数
    unsigned int _batch_count;       // 已提交页数
};

// 预读决策在 page_cache_sync_readahead 中
void page_cache_sync_readahead(mapping, file, page, index)
{
    // ondemand_readahead 判断是否应该预读
    if (ondemand_readahead(mapping, ra, file, true, index)) {
        // 预读窗口增长逻辑
        ra->size = next_ra_size(ra); // 指数增长
        // ra_pages = min(ra->size, 128) // 上限 128 KiB
        __do_page_cache_readahead(mapping, file,
                                   ra->start, ra->size);
    }
}
```

次序	offset	用户读	预读动作	预读窗口
1	0	4 KiB	读 1 页，启动预读	4 KiB
2	4K	4 KiB	命中（预读命中），继续预读 4 页	8 KiB
3	8K	4 KiB	命中，预读 8 页	16 KiB
4	12K	4 KiB	命中，预读 16 页	32 KiB

5	16K	4 KiB	命中, 预读 32 页	64 KiB
6	20K	4 KiB	命中, 预读 64 页	128 KiB
7+	...	4 KiB	命中, 稳定预读 128 KiB	128 KiB

预读的关键收益: 当窗口稳定在 128 KiB 时, 后续每次 4 KiB 读都命中 cache, 没有磁盘 I/O。磁盘在后台以一次大 I/O 读取 128 KiB (32 页), 而非 32 次小 I/O。大 I/O 的吞吐远高于小 I/O—HDD 上一次 128 KiB 顺序读 ~ 0.6 ms, 而 32 次 4 KiB 随机读 ~ 320 ms, 快 500 倍。

常见误区

预读是投机 (speculation)。如果用户程序只读了文件头部就退出, 预读的 128 KiB 全部浪费。内核的预读算法因此需要检测随机访问模式并关闭预读: 如果连续两次读的 offset 不连续, 预读窗口立即缩减到 0。

写路径深追踪

写路径与读路径有一个根本区别: 写操作不等待磁盘。数据拷入 page cache、标记为脏页后立即返回。

```
// vfs_write(file, buf, 4096, offset=8192)
ssize_t vfs_write(struct file *file, const char __user *buf,
                  size_t count, loff_t *pos)
{
    // offset = 8192, count = 4096
    // 分发到 generic_file_write_iter
    return generic_file_write_iter(iocb, from);
}

// generic_file_write_iter 内部:
// 1. 获取 page cache 中的 page
pgoff_t index = 8192 / 4096; // index = 2
struct page *page;

// write_begin 回调 (ext4: ext4_write_begin)
// 如果 page 不在 cache 中, 分配并加入
page = grab_cache_page_write_begin(mapping, index);
// ext4_write_begin 可能还需要处理延迟分配预留

// 2. 从用户空间拷贝数据到 page
// page 内偏移 = 8192 % 4096 = 0
copied = copy_from_user(page_address(page) + 0, buf, 4096);
// 此时数据在内核 page cache 中

// 3. write_end 回调 (ext4: ext4_write_end)
// 更新 inode 的 i_size
// 标记 page 为脏
set_page_dirty(page);
// SetPageDirty(page) → 把 page 加入 address_space 的脏页树

// 4. 解锁 page, 返回用户空间
unlock_page(page);
put_page(page);
// write() 系统调用返回 4096
// 数据在内存中, 磁盘上还没有
```

`write()` 返回时, 数据的安全级别取决于缓存层次:

时刻	数据位置	断电后
普通 buffered write() 返回	page cache (内存)	掉电时通常丢失
writeback 完成	磁盘写缓存	可能丢失 (易失缓存)
fsync() 成功返回	文件系统与设备已完成其持久化协议	设备兑现 flush/FUA 等承诺且硬件无故障时保留

对这里讨论的普通 buffered write, write() 成功返回只说明内核已经接收数据；持久化仍取决于后续写回、文件系统提交和设备协议。

脏页追踪

page cache 需要区分“干净 folio”(内容不需要写回)和“脏 folio”(内存中修改过, 需要写回)。address_space.i_pages 是保存 page-cache folio 的 XArray; 脏状态由 folio 标志表示, 并通过同一 XArray 上的 mark 让扫描者快速筛选。这里没有另一棵与缓存索引平行的“脏页树”。下面继续使用单页大小的伪代码, 以便突出标记机制:

```
// 每个 page 有一个 page->flags 字段
// 其中包含 PG_dirty 位:
//   PG_dirty = 1: 页被修改过, 需要 writeback
//   PG_dirty = 0: 页与磁盘一致 (或刚从磁盘读入)

// SetPageDirty(page) 的简化逻辑:
int set_page_dirty(struct page *page)
{
    struct address_space *mapping = page->mapping;
    // page 已在 mapping->i_pages 中; 设置 folio/page 脏标志,
    // 并在同一 XArray 上设置 DIRTY mark
    if (!TestSetPageDirty(page)) {
        // 之前不是脏页, 给其 XArray 索引设置脏 mark
        __xa_set_mark(&mapping->i_pages, page->index,
                     PAGECACHE_TAG_DIRTY);
        // 增加全局脏页计数
        __inc_node_page_state(page, NR_FILE_DIRTY);
        // 增加 inode 的脏页计数
        __mark_inode_dirty(mapping->host, I_DIRTY_PAGES);
        return 1;
    }
    return 0;
}
```

脏页的追踪使用三个标记: PAGECACHE_TAG_DIRTY (需要写回)、PAGECACHE_TAG_WRITEBACK (正在写回中)、PAGECACHE_TAG_TOWRITE (等待写回)。writeback 代码通过扫描这些标记找到需要处理的页。

写回机制: 阈值与触发

Linux 的脏页写回由 per-CPU 计数器和全局阈值驱动。核心计数器是 nr_dirty (全局脏页数) 和 nr_writeback (正在写回的页数), 通过 per-CPU 变量维护以减少锁竞争。

参数	默认值	可调范围	含义
----	-----	------	----

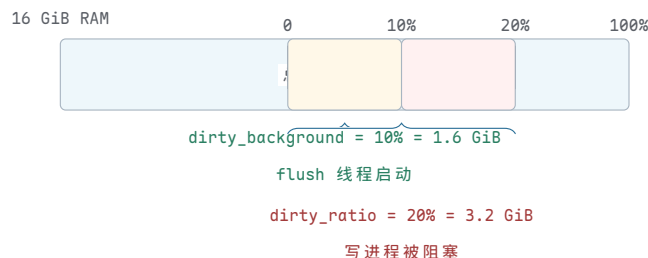
<code>vm.dirty_ratio</code>	20%	0-100	脏页达到总内存的比例时，写进程被阻塞
<code>vm.dirty_background_ratio</code>	10%	0-100	脏页达到此比例时，后台 flush 线程启动
<code>vm.dirty_expire_centisecs</code>	3000 (30s)	—	脏页存活超过此时长后优先写回
<code>vm.dirty_writeback_centisecs</code>	500 (5s)	—	flush 线程周期性唤醒间隔

以 16 GiB RAM 为例：

```
// 16 GiB RAM 的脏页阈值计算
total_memory = 16 * 1024 * 1024 KiB = 16777216 KiB
// 假设每页 4 KiB
total_pages = 16777216 / 4 = 4194304

dirty_background_ratio = 10% // 默认
dirty_background_threshold = 4194304 * 0.10 = 419430 pages
                        = 419430 * 4 KiB = 1638 MiB
// 脏页超过 1638 MiB → kworker flush 线程启动

dirty_ratio = 20% // 默认
dirty_threshold = 4194304 * 0.20 = 838860 pages
                = 838860 * 4 KiB = 3276 MiB
// 脏页超过 3276 MiB → 写进程被阻塞 (throttle)
// 写进程被迫同步执行 writeback，不能继续产生脏页
```



图示：脏页阈值示意。16 GiB RAM 时，脏页超过 1.6 GiB (10%) 触发后台写回；超过 3.2 GiB (20%) 阻塞写进程。黄色区域为后台写回触发范围，红色区域为写进程阻塞范围。

写回追踪

当脏页达到阈值或时间过期，`kworker` 线程被唤醒执行 `wb_workfn`：

```
// 写回核心逻辑（简化）
void wb_workfn(struct writeback_control *wbc)
{
    while (wbc->nr_to_write > 0) {
        // 1. 收集脏 folio：按 address_space.i_pages 的 DIRTY mark 扫描
        // 扫描最多 1024 页
        struct page *pages[1024];
        int nr = find_dirty_pages(mapping, wbc,
                                pages, 1024);

        if (nr == 0)
            break; // 没有脏页了
    }
}
```



```

    for (int i = 0; i < nr; i++) {
        struct page *page = pages[i];

        // 2. 如果是延迟分配, 此时才分配物理块
        if (test_bit(PG_delalloc, &page->flags)) {
            // ext4: ext4_mballocc 在块组中找连续块
            ext4_lblk_t lblk = page->index;
            ext4_fsblk_t pblk;
            int allocated = ext4_mb_new_blocks(
                handle, &ar, &pblk);
            // 设置 extent: (lblk, pblk, 1)
            ext4_ext_map_blocks(handle, inode, &map,
                                EXT4_GET_BLOCKS_CREATE);
            clear_bit(PG_delalloc, &page->flags);
        }

        // 3. 构造 bio
        struct bio *bio = bio_alloc(GFP_NOIO, 1);
        bio->bi_bdev = inode->i_sb->s_bdev;
        bio->bi_iter.bi_sector =
            pblk * (blocksize >> 9);
        // bio 记录 page、页内偏移和长度; DMA 映射在更低层建立
        bio->bi_io_vec[0].bv_page = page;
        bio->bi_io_vec[0].bv_len = 4096;
        bio->bi_io_vec[0].bv_offset = 0;

        // 4. 标记为正在写回
        SetPageWriteback(page);
        ClearPageDirty(page);
        // 在同一 XArray 中把 DIRTY/WRITEBACK marks 转换

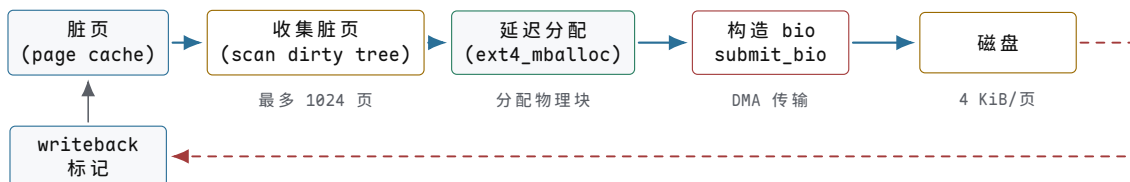
        // 5. 提交给块层
        submit_bio(WRITE, bio);
    }

    // 6. 等待这批 bio 完成
    // (部分 writeback 模式异步等待)
    wbc->nr_to_write -= nr;
}

// bio 完成回调 (中断上下文)
void bio_endio(struct bio *bio)
{
    for (page in bio->bi_io_vec) {
        // 清除 writeback 标记
        end_page_writeback(page);
        // 从 writeback 树移除
        // page 现在是干净页, 留在 cache 中
    }
    bio_put(bio); // 释放 bio
}

```

脏页到磁盘的完整数据流:



图示：写回数据流。脏页从 page cache 出发，经过脏页扫描、延迟分配、bio 构造、DMA 传输到达磁盘。I/O 完成后中断回调清除 writeback 标记，page 变为干净页留在 cache 中。

fsync：从内存到非易失介质

`fsync(fd)` 是 POSIX 文件 API 中请求文件数据与相关元数据完成同步的核心接口，但不是所有持久化场景的“唯一手段”：`fdatasync`、同步打开标志、数据库自己的日志协议以及目录 `fsync` 都可能参与。成功返回表示内核和文件系统已完成该对象要求的写回与提交，并向设备发出了必要的顺序/持久化命令；这个承诺依赖设备正确实现 `flush`、`FUA` 或非易失缓存，也不等于新建文件的父目录项必然已经持久。下面的代码是机制级伪代码，不是某个内核版本的逐行 `ext4_fsync`：

```
// 文件系统 fsync 的机制级伪代码
int filesystem_fsync(struct file *file, loff_t start,
                    loff_t end, int datasync)
{
    struct inode *inode = file->f_inode;

    // 步骤 1: 把此 inode 的所有脏页刷到磁盘
    // filemap_write_and_wait 按 address_space.i_pages 的脏 mark 扫描
    // 对每个脏页：构造 bio → submit_bio → 等待完成
    ret = filemap_write_and_wait_range(
        inode->i_mapping, start, end);
    // 此时数据已到磁盘（或磁盘写缓存）

    // 步骤 2: 如果文件系统有日志，强制提交日志事务
    // 确保元数据修改也持久化
    if (!datasync || ext4_should_journal_data(inode)) {
        ret = ext4_fc_commit(journal, start, end);
        // 或旧版：jbd2_complete_transaction(journal, tid)
        // 这会触发 jbd2 的 commit 流程：
        //   写 descriptor + metadata + commit block
        //   flush 日志区域
    }

    // 步骤 3: 按文件系统、块层和设备能力完成顺序/持久化协议
    // 可能使用 preflush、FUA、单独 flush，或由事务提交路径承担；
    // 不能假定每次 fsync 都恰好直接调用一次 blkdev_issue_flush。
    ret = finish_persistence_protocol(inode->i_sb);

    return ret;
}
```

常见误区

许多设备有易失性写缓存 (volatile write cache)。普通写完成可能只表示数据进入控制器缓存；掉电后仍会丢失。块层与驱动必须把文件系统的持久化要求翻译成设备支持的 `flush`/`FUA` 等命令，且设备必须诚实兑现协议。若设备具有受保护的易失性写缓存，某些 `flush` 可以快速完成；若设备或虚拟化层谎报能力，软件单靠 `fsync` 无法修复硬件契约的破坏。因此“成功返回”是分层协议承诺，不是软件越过控制器亲眼观察了 NAND 或盘片。

屏障与磁盘命令顺序

在有易失写缓存的磁盘上，写操作的完成顺序与提交顺序可能不同。内核用屏障(barrier)来保证顺序。考虑以下场景：先写数据块 5000，再写元数据块 6000，必须保证 5000 先到非易失介质。

```
// 磁盘命令序列（有屏障）
WRITE block 5000    // 数据块
WRITE block 5001    // 数据块
FLUSH_CACHE        // ← 屏障：强制 5000/5001 到非易失介质
--barrier--        // 后续写必须在此之后
WRITE block 6000    // 元数据块（依赖 5000 已落盘）
FLUSH_CACHE        // 确保 6000 也落盘

// 如果在 FLUSH_CACHE 之前断电：
//   block 5000/5001 可能没落盘
//   block 6000 一定没落盘（还在屏障之后）
//   元数据不会指向未落盘的数据块 → 一致

// 如果没有屏障，磁盘可能重排：
WRITE block 6000    // 先落盘
WRITE block 5000    // 后落盘
// 断电：block 6000 在盘上，block 5000 不在
//   元数据指向不存在的数据 → 不一致
```

现代 SATA 和 NVMe 协议提供了原语的 FUA (Force Unit Access) 和 FLUSH 命令来支持屏障语义。内核在 `submit_bio` 时设置 `BIO_RW_BARRIER` 或使用 `blkdev_issue_flush` 来插入屏障。

Direct I/O：绕过 page cache

有些应用程序不需要内核 page cache——它们自己管理缓存。数据库系统 (Oracle、PostgreSQL) 在用户态维护 buffer pool，数据缓存在自己的内存中。如果再经过 page cache，就会产生双重缓存：同一份数据既在数据库 buffer pool 中，又在内核 page cache 中，浪费内存且增加一次拷贝。

`O_DIRECT` 标志让 `read/write` 绕过 page cache，直接在用户 buffer 和块设备之间通过 DMA 传输数据。

```
// O_DIRECT 读路径
vfs_read(file, buf, 4096, offset=8192, O_DIRECT):
    // 1. 不查 page cache，不分配 page
    // 2. 检查对齐约束
    //   buf 必须页对齐 (buf % 4096 == 0)
    //   offset 必须块对齐 (offset % 512 == 0)
    //   len 必须块对齐
    if ((unsigned long)buf % blocksize != 0)
        return -EINVAL;
    if (offset % blocksize != 0)
        return -EINVAL;
    if (len % blocksize != 0)
        return -EINVAL;

    // 3. 直接构造 bio，用户 buffer 作为 DMA 目标
    //   需要 pin 住用户 page 防止换出
    bio = bio_alloc(GFP_KERNEL, 1);
    bio->bi_bdev = inode->i_sb->s_bdev;
    bio->bi_iter.bi_sector = offset / 512;
    bio->bi_io_vec[0].bv_page = virt_to_page(buf);
    bio->bi_io_vec[0].bv_offset = offset_in_page(buf);
    bio->bi_io_vec[0].bv_len = 4096;
```

```
// 4. 提交给块层，等待完成
submit_bio(READ, bio);
wait_for_completion(bio);

// 5. 不经过 page cache，不产生缓存页
// 数据直接 DMA 到用户 buffer
return 4096;
```

条件	要求	原因
buffer 对齐	页对齐	DMA 需要物理连续页帧
offset 对齐	块对齐 (512B 或 4K)	块设备最小寻址单元
length 对齐	块对齐	不能 DMA 半个块

何时使用 `O_DIRECT`:

- 数据库 (Oracle、PostgreSQL、MySQL InnoDB): 自己管理 buffer pool, 避免双重缓存, 减少一次内存拷贝。
- 大文件顺序写 (视频录制、日志归档): 不需要 page cache 的读缓存, 直接写入即可。

何时不使用 `O_DIRECT`:

- 小文件随机读: 没有预读收益, 每次都直接访问磁盘。
- 写密集型小写: 没有写回批量化, 每次写直接落盘。
- 通用文件访问: page cache 的读缓存和写批量对小 I/O 至关重要。

DAX: 持久内存直接访问

DAX (Direct Access) 面向可由 CPU load/store 访问的持久内存 (persistent memory, PMEM) 或支持 DAX 的设备映射。具体延迟依平台而变; 稳定特征是它能绕过传统 page cache 数据路径, 并保留非易失性介质的持久化要求。

DAX 模式下, 文件 `mmap` 返回用户虚拟地址, 页表把该地址映射到持久内存页。应用可用 CPU load/store 访问映射内容, 数据不经过传统 page cache, 也不靠设备 DMA 完成每次 load/store; 但地址仍然是受页表权限约束的虚拟地址。

```
// DAX 写路径
struct data_t { int data; };
int fd = open("/pmem/file.dat", O_RDWR);
struct data_t *p = mmap(NULL, 4096,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
// p 是映射到持久内存页的用户虚拟地址

p->data = 42;    // CPU store 指令直接写 PMEM
                // 这是一条 mov 指令, 不经过 page cache

// 但 CPU 有自己的 L1/L2 cache
// store 先进入 CPU cache, 可能还没到 PMEM
// 要保证持久化, 需要显式刷 cache line

clwb(p);        // cache line writeback
                // 把包含 p 的 cache line 刷到 PMEM
sfence();       // store fence
                // 确保之前的所有 store (包括 clwb) 完成
                // 之后才能继续执行
```

```
// 此刻 p->data = 42 已在持久内存上，断电不丢
```

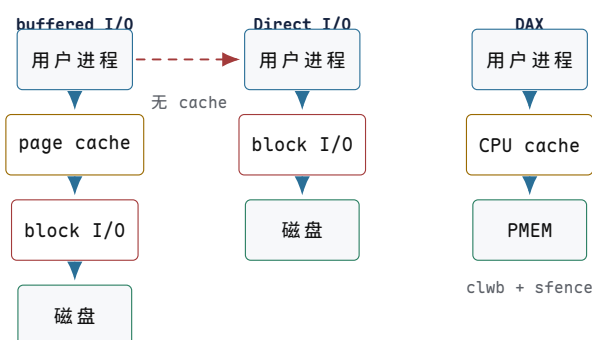
```
munmap(p, 4096);
close(fd);
```

DAX 下的 `fsync` 语义完全不同：

```
// DAX fsync
int dax_fsync(struct inode *inode)
{
    // 遍历此 inode 的所有 dirty PMEM 映射
    // 对每个 dirty cache line:
    //   clwb(addr) // flush to PMEM
    //   sfence()   // ensure ordering
    // 不需要 submit_bio, 不需要 blkdev_issue_flush
    // 因为 PMEM 本身就是非易失介质

    struct address_space *mapping = inode->i_mapping;
    // 扫描 DAX dirty 标记的页面
    pgoff_t index = 0;
    while ((page = find_get_page_tag(mapping, &index,
        PAGECACHE_TAG_DIRTY))) {
        // 对 PMEM 中的地址执行 clwb
        void *addr = dax_get_addr(page);
        arch_wb_cache_pmem(addr, PAGE_SIZE);
    }
    arch_wmb_cache(); // sfence
    return 0;
}
```

	buffered I/O	Direct I/O	DAX
数据路径	page cache → block I/O	block I/O	CPU store → PMEM
<code>fsync</code>	刷脏页 + 屏障	屏障	<code>clwb + sfence</code>
缓存层	内核 page cache	应用自管	CPU cache line
延迟	~100 ns (命中)	~100 μs	~300 ns
对齐要求	无	页/块对齐	无
适用	通用	数据库	持久内存设备



图示：三种 I/O 路径对比。buffered I/O 经过 page cache 和 block I/O 两层；Direct I/O 绕过 page cache 但仍走 block I/O；DAX 完全绕过 page cache 和 block I/O，CPU store 直接写持久内存。

崩溃一致性、日志与检查点

崩溃问题：多步写不是原子的

文件系统的一次逻辑操作（如创建文件）在磁盘上需要修改多个独立的块。磁盘每次只写一个块（或者一个扇区），而电源可能在任意时刻中断。这导致磁盘上的状态可能停留在“半完成”的状态——部分块已更新，部分块还是旧的。

以创建文件 `/foo` 为例，需要四步磁盘写入：

```
// 创建 /foo 需要的磁盘修改
create_file(root_dir, "foo"):
    // 步骤 1: 分配 inode 5
    //   修改 inode bitmap: 标记 inode 5 为已用
    //   写磁盘块 2 (inode bitmap)
    mark_inode_used(inode_bitmap, 5)
    write_block(2, inode_bitmap)

    // 步骤 2: 初始化 inode 5
    //   修改 inode table: 写 inode 5 的内容
    //   写磁盘块 4 (inode table 中 inode 5 所在块)
    init_inode(5, mode=REG_FILE, size=0, ...)
    write_block(4, inode_table_block)

    // 步骤 3: 添加目录项
    //   修改根目录数据块: 添加 "foo" -> 5
    //   写磁盘块 68 (根目录数据块)
    dir_add_entry(root_dir, "foo", inode=5)
    write_block(68, dir_data_block)

    // 步骤 4: 分配数据块 (如果文件有内容)
    //   修改 block bitmap: 标记块 68 为已用
    //   写磁盘块 3 (block bitmap)
    mark_block_used(block_bitmap, 68)
    write_block(3, block_bitmap)
```

崩溃在不同时刻发生，造成不同的不一致状态：

崩溃时刻	已完成的写	不一致后果
步骤 1 之后	块 2 (inode bitmap)	inode 5 标记为已用，但 inode table 中内容未初始化。inode 5 有垃圾数据。
步骤 2 之后	块 2 + 块 4	inode 5 已分配且初始化，但无目录项指向它。inode 泄漏。bitmap 说已用，但无路径可达。
步骤 3 之后	块 2 + 4 + 68	目录指向 inode 5，但 block bitmap 未标记块 68 为已用。另一个文件可能分配到同一块。
步骤 4 之后	全部完成	一致。

步骤 3 之后崩溃尤其危险：目录项说 `"foo" → inode 5`，inode 5 的 `block[]` 指向块 68，但 block bitmap 仍标记块 68 为空闲。下次分配时，另一个文件可能拿到块 68，两个文件共享同一数据块——数据损坏。

核心思想

崩溃一致性的核心困难是：一次逻辑修改需要多个磁盘写，而磁盘写之间没有原子性。任何时刻断电都可能留下部分更新的状态。文件系统必须能恢复到一致状态，无论崩溃发生在哪一步。

fsck：旧方案的代价

ext2 没有日志，崩溃恢复依赖 `e2fsck` 在挂载前扫描整个文件系统。`fsck` 分五步检查：

```
e2fsck(dev):
// Pass 1: 检查每个 inode
// 遍历 inode table 中所有 inode
// 验证 mode 合法、size 非负
// 验证 block 指针在合法范围 [0, total_blocks]
// 统计每个 inode 使用的块
for i in 0..inode_count:
    inode = read_inode(i)
    if inode.mode == 0: continue // 未使用
    for block in inode.block_pointers:
        if block < 0 or block >= total_blocks:
            mark_bad_inode(i)
        block_usage[block].add(i) // 谁用了这个块

// Pass 2: 检查目录结构
// 从根目录开始 BFS
// 每个目录项指向的 inode 必须存在且类型正确
// "." 必须指向自身，".." 必须指向父目录
bfs_from_root():
    queue = [root_inode]
    while queue:
        dir = dequeue(queue)
        for entry in dir.entries:
            if not inode_exists(entry.inode):
                error("dangling dir entry")
            if entry.name == "." and entry.inode != dir:
                error("bad . entry")
            enqueue(entry.inode)

// Pass 3: 检查连通性
// 从根目录可达的 inode 集合 vs 全部已用 inode
// 不可达的 inode 是“孤儿” (orphan)
// 将孤儿 inode 链到 lost+found
reachable = bfs_from_root()
for i in 0..inode_count:
    if inode_used(i) and i not in reachable:
        link_to_lost_found(i)

// Pass 4: link count 验证
// inode.links 字段 vs 实际目录引用数
// 不匹配则修正 links 字段
for i in 0..inode_count:
    actual_links = count_dir_references(i)
    if inode[i].links != actual_links:
        fix_links(i, actual_links)

// Pass 5: bitmap 一致性
// inode bitmap: 标记已用的 inode 是否确实在使用
// block bitmap: 标记已用的块是否被某 inode 引用
```

```
// 不匹配则修正 bitmap
for block in 0..total_blocks:
    if bitmap[block] == USED and block not in block_usage:
        bitmap[block] = FREE
    if bitmap[block] == FREE and block in block_usage:
        bitmap[block] = USED // 可能是泄漏
```

全盘检查的工作量随 inode 与块数增长。一个 1 TiB 文件系统若采用 4 KiB 块，共有 $2^{40}/2^{12} = 2^{28} \approx 2.68$ 亿个块；即使以每秒 100000 块的理想速度线性扫描，仅块扫描也约需 45 分钟。真实耗时还受 inode 数量、元数据局部性、设备并行度和修复工作影响，因此大容量文件系统的完整检查可能持续更久。

常见误区

fsck 在大型文件系统上恢复时间长达数十分钟到数小时。服务器重启窗口要求几秒到几分钟，**fsck** 的恢复时间是不可接受的。这正是日志文件系统被发明的直接动机：把恢复时间从 $O(\text{文件系统大小})$ 降到 $O(\text{日志大小})$ 。

日志：write-ahead logging

日志采用写前记录 (write-ahead logging, WAL)：相关主位置更新获得持久性之前，先将事务所需的元数据版本和提交信息写入日志区域。事务提交后，主文件系统位置可以在 checkpoint 阶段更新，已完成 checkpoint 的日志空间随后回收。

```
// 带日志的文件创建
journal_create_file(root_dir, "foo"):
    // 1. 开启一个日志事务 (handle)
    handle = journal_start()

    // 2. 在日志中记录所有要做的修改
    // 这些记录描述了修改后的“完整块内容”
    journal_log(handle, "inode 5: set mode=FILE, size=0")
    journal_log(handle, "dir block 68: add entry foo->5")
    journal_log(handle, "inode bitmap: set bit 5")
    journal_log(handle, "block bitmap: set bit 68")

    // 3. 提交日志事务
    // commit record 写到日志区域的磁盘上
    // 这是“事务完成”的不可撤销标记
    journal_commit(handle)
    // ← 此刻之前断电：事务未提交，恢复时丢弃
    // ← 此刻之后断电：事务已提交，恢复时重放

    // 4. 把修改写到主文件系统位置
    // (checkpoint, 可延迟执行)
    write_inode(5, ...)
    write_dir_block(68, ...)
    write_inode_bitmap(...)
    write_block_bitmap(...)
```

崩溃恢复规则因此变得简单：

崩溃时刻	日志状态	恢复动作
<code>journal_commit</code> 之前	无 commit 块	丢弃此事务，所有修改未应用
<code>journal_commit</code> 之后	有 commit 块	重放日志中的修改到主文件系统

checkpoint 之后	日志已回收	无需恢复（修改已在主位置）
---------------	-------	---------------

核心理念

日志先行约束要求恢复所需的日志记录先于受保护的主位置更新持久化。恢复程序还要验证事务序号、描述块、校验和与 commit 块；只有满足格式规定的完整事务才可重放。commit 块提供提交证据，但不能脱离写序和完整性检查单独解释。

jbd2 日志格式

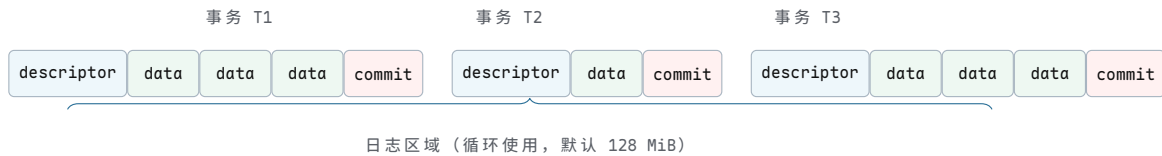
ext4 的日志由 jbd2 (Journaling Block Device v2) 实现。日志是一块循环使用的磁盘区域，内部由一系列块组成。每个事务在日志中的布局如下：

```
// jbd2 日志区域布局（磁盘上）
// 每个块 4 KiB
//
// 事务 T1:
// [descriptor][data][data][data][commit]
// 块 100      101    102    103    104
//
// 事务 T2:
// [descriptor][data][commit]
// 块 105      106      107
//
// 事务 T3:
// [descriptor][data][data][data][commit]
// 块 108      109     110     111     112

// descriptor block 格式:
struct journal_header_s {
    __be32 h_magic;           // 0xC03B3998
    __be32 h_blocktype;       // 1=descriptor, 2=commit, 5=revoke
    __be32 h_sequence;        // 事务序号
};
// 后跟一组 block_tag:
struct journal_block_tag_s {
    uint32_t t_blocknr;        // 被修改块在主 FS 中的块号
    uint16_t t_checksum;        // 数据块校验和 (CRC32C)
    uint16_t t_flags;          // JBD2_FLAG_ESCAPE 等
    uint32_t t_blocknr_high;    // 高 32 位块号 (大 FS)
};

// commit block 格式:
struct journal_commit_header {
    journal_header_s h;        // h_blocktype = 2 (commit)
    uint8_t  checksum_type;     // 校验类型
    uint8_t  checksum_size;
    uint8_t  padding[2];
    __be32   checksum[8];      // 具体解释取决于 checksum feature
};
```

jbd2 的磁盘字段采用大端序，这一点与 ext4 主体结构采用小端序相反。descriptor 后面可以有数据块或 revoke 记录；revoke 用来阻止恢复时重放已经被释放并重新解释的旧块。若忽略 revoke，只靠“看到 commit 就逐块覆盖”，恢复器可能把旧元数据写到后来已经属于文件数据的块上。



图示：jbd2 日志布局。每个事务以 descriptor 块开始（列出后续数据块对应的主 FS 块号），中间是数据块（修改后的完整块内容），最后以 commit 块结束（含 CRC32C 校验和）。恢复时，有 commit 块且校验通过的事务被重放。

崩溃恢复追踪

系统启动挂载文件系统时，jbd2 执行日志恢复：

```
journal_recover(journal):
    // 从日志区域的头部开始扫描
    log_start = journal->j_head;    // 下一个要写的位置
    log_end   = journal->j_tail;    // 最老的未 checkpoint 位置

    // 扫描每个事务
    seq = journal->j_tail_sequence;
    block = log_end;

    while block < log_start:
        // 读 descriptor 块
        desc = read_block(block);
        if desc.h_magic != MAGIC || desc.h_blocktype != DESCRIPTOR:
            break;    // 不是有效的日志内容

        // 读后续的数据块，直到遇到 commit 块
        data_blocks = [];
        tag = desc.tags;
        while tag != NULL:
            data = read_block(block + offset);
            // 校验：data 的 CRC32C == tag.t_checksum?
            if crc32c(data) != tag.t_checksum:
                // 数据损坏，事务不可信
                break;
            data_blocks.append({target: tag.t_blocknr,
                               content: data});
            tag = tag->next;

        // 读 commit 块
        commit = read_block(block + data_count + 1);
        if commit.h_blocktype == COMMIT:
            // 有 commit 块 → 事务已提交
            // 校验 commit 块的 checksum
            if verify_commit_checksum(commit, data_blocks):
                // 重放：把每个数据块写到主 FS 位置
                for entry in data_blocks:
                    write_block(entry.target, entry.content);
                // 事务重放完成
            else:
                // commit 校验失败，跳过
                break;
        else:
            // 没有 commit 块 → 事务未完成，丢弃
            break;

    // 恢复完成
    // 恢复时间 = 0(日志大小)，通常几秒
    // 128 MiB 日志 / 4 KiB 块 = 32768 块
```

```
// 每块读 + 可能重写 = 最坏 32768 * 2 次 I/O
// NVMe 上约 32768 * 0.1ms = 3.3 秒
```

恢复时间与日志大小成正比，与文件系统总大小无关。128 MiB 日志的恢复在 NVMe 上约 3 秒，远优于 `fsck` 的 10–30 分钟。

ext4 + jbd2 事务模型

jbd2 有两级抽象：

对象	粒度	生命周期
<code>handle</code>	一次逻辑操作（创建一个文件、一次 <code>rename</code> ）	线程局部， <code>journal_start</code> 到 <code>journal_stop</code>
<code>transaction</code>	一个 <code>commit</code> 周期内的所有 <code>handle</code>	一个 jbd2 线程唤醒周期，批量提交

一个 `transaction` 可以收集来自不同线程的多个 `handle`。jbd2 线程定期唤醒（默认 5 秒或脏数据达到阈值），把当前 `transaction` 的所有修改一次性提交到日志。

```
// jbd2 commit 流程（简化）
void jbd2_commit_transaction(journal_t *journal)
{
    transaction_t *commit_transaction =
        journal->j_running_transaction;

    // 1. 等待所有 handle 关闭
    // 新的 handle 会进入下一个 transaction
    wait_for_all_handles_closed(commit_transaction);
    journal->j_running_transaction = new_transaction();

    // 2. 切换到 commit 状态
    commit_transaction->t_state = T_LOCKED;

    // 3. 写 descriptor 块
    // descriptor 列出本事务中所有被修改的 buffer
    // 及其在校 FS 中的目标块号
    write_descriptor_block(journal, commit_transaction);

    // 4. 写数据/元数据块到日志
    // 把每个被修改的 buffer 的完整内容写入日志
    for (jh in commit_transaction->t_buffers):
        // 如果是 data=journal 模式，数据块也写入
        // 如果是 data=ordered, 此时不写数据块
        journal_write_buffer(journal, jh);

    // 5. 写 commit 块
    // 包含本事务所有块的 CRC32C 校验和
    write_commit_block(journal, commit_transaction);

    // 6. flush 日志区域
    // 确保日志内容在非易失介质上
    blkdev_issue_flush(journal->j_dev);

    // 7. 标记旧 transaction 可以被 checkpoint
    commit_transaction->t_state = T_COMMIT;

    // 8. 后续 checkpoint（可以延迟）:
```

```
// 把已提交的修改从日志复制到主 FS 位置
// 完成后释放日志空间
}
```

三种日志模式

ext4 支持三种日志模式，在一致性和性能之间做不同取舍：

模式	日志内容	数据顺序	崩溃后保证
<code>data=journal</code>	元数据 + 数据	数据先入日志	最强：数据不丢。写放大 2 倍。
<code>data=ordered</code>	只元数据	数据先写盘，再提交元数据	中等：要么旧数据要么新数据。
<code>data=writeback</code>	只元数据	无顺序保证	最弱：可能看到旧垃圾数据。

下面追踪三种模式下 `write(fd, buf, 4096); fsync(fd);` 的磁盘 I/O 序列：

```
// ===== data=journal 模式 =====
// write() 触发：数据进入 page cache (无 I/O)
// fsync() 触发：
//   1. jbd2 提交事务，包含数据和元数据
//   日志：[data: buf 内容][metadata: inode 更新][commit]
//   flush 日志区域
//   2. checkpoint: 把数据写入主 FS 位置
//   主 FS: [data: buf 内容][metadata: inode 更新]
//   3. flush 主 FS
//
// 磁盘写：数据写 2 次 (日志 + 主 FS)，元数据写 2 次
// 写放大：2x
// 崩溃后：日志重放恢复数据，数据完整

// ===== data=ordered 模式 (ext4 默认) =====
// write() 触发：数据进入 page cache (无 I/O)
// fsync() 触发：
//   1. 先把数据脏页刷到主 FS
//   主 FS: [data: buf 内容] ← 先落盘
//   flush 数据 (barrier)
//   2. jbd2 提交元数据事务
//   日志：[metadata: inode 更新][commit]
//   flush 日志
//   3. checkpoint: 元数据写入主 FS
//   主 FS: [metadata: inode 更新]
//
// 磁盘写：数据写 1 次 (主 FS)，元数据写 2 次 (日志 + 主 FS)
// 写放大：~1.5x (元数据部分)
// 崩溃后：
//   如果 commit 之前崩溃：数据在盘上，元数据不在
//   → inode 不指向新数据 → 看到旧数据 (或 hole)
//   如果 commit 之后崩溃：数据和元数据都在
//   → 看到新数据，完整一致

// ===== data=writeback 模式 =====
// write() 触发：数据进入 page cache (无 I/O)
// fsync() 触发：
//   1. jbd2 提交元数据事务
//   日志：[metadata: inode 更新][commit]
//   flush 日志
```



```
// 2. checkpoint: 元数据写入主 FS
// 主 FS: [metadata: inode 更新]
// 3. 数据可能在 fsync 时也刷了, 也可能没有
// (取决于实现, writeback 模式不强制数据顺序)
//
// 磁盘写: 元数据写 2 次 (日志 + 主 FS), 数据写 0 或 1 次
// 写放大: ~1x (最快)
// 崩溃后:
// 元数据可能指向一个块, 但数据没写
// → 读到的是磁盘上的旧垃圾数据
// → 可能暴露上一个文件的残留内容!
```

常见误区

data=writeback 模式在崩溃后可能暴露旧数据: 元数据 (inode 的块指针) 已更新指向新分配的块, 但数据内容还没写。这个块上可能存着另一个已删除文件的内容。这是安全漏洞——一个用户可能看到另一个用户的旧数据。**data=ordered** (默认模式) 通过强制数据先于元数据落盘来避免此问题。

checkpoint 与日志空间回收

日志区域大小有限 (ext4 默认 128 MiB)。提交后的事务修改记录留在日志中, 直到被 checkpoint 到主文件系统位置后才能回收。日志因此是一块循环使用的环形缓冲区。

```
// 日志环形缓冲区
// j_head: 下一个写入位置 (指向日志的最新事务末尾之后)
// j_tail: 最老的未 checkpoint 事务的起始位置
// j_free: j_head 到 j_tail 之间的可用空间
//
// ... [T_old][T_newer][T_current] ... [free space] ... [T_oldest]
//                                     ↑head           ↑tail
// (日志是循环的, head 可能绕回 tail 之前)

// checkpoint 流程
jbd2_checkpoint(journal):
// 找到最早未 checkpoint 的 transaction
t = journal->j_checkpoint_transactions;
while t:
// 把 t 中记录的修改写到主 FS 位置
for jh in t->t_buffers:
// jh->b_bh 是被修改的 buffer
// buffer 内容已在日志中, 现在写到主位置
write_block(jh->b_blocknr, jh->b_data);
// 此事务的日志空间可以回收
free_log_space(journal, t);
t = t->t_next;

// 更新 j_tail
journal->j_tail = next_oldest_start;
// 写日志 superblock (更新 tail 指针)
update_journal_superblock(journal);
```



图示：日志环形缓冲区。head 指向下一个写入位置，tail 指向最老的未 checkpoint 事务。已 checkpoint 的事务 (T0、T1) 空间可回收。如果 checkpoint 跟不上新事务产生速度，free 空间耗尽，新事务必须等待——导致写入停顿。

写时复制、快照与根切换

COW 原理：不覆盖，只追加

ext4、XFS 一类文件系统可以原地更新部分数据或元数据，但它们并不因此共享一个简单的“覆盖后由日志回滚全部文件数据”模型。以 ext4 默认 `data=ordered` 为例，日志主要保护元数据结构；普通文件数据通常不进入日志，覆盖写仍可能受到设备原子写粒度和掉电时序影响。必须把“结构可恢复”和“用户数据原子更新”分开。

COW (Copy-on-Write) 文件系统的基本策略是：对受 COW 保护的块，不在原地发布新版本，而是写到新位置，再逐层发布新的引用关系。真实系统还需要 generation、校验和、写序、超级块副本和恢复选择规则；不能把它缩成一次天然原子的指针赋值。

```
// 传统原地覆盖写
update_block(inode, offset, new_data):
    block = lookup(inode, offset) // 找到物理块 5000
    write_block(5000, new_data)   // 直接覆盖块 5000
    // 如果写到一半断电：
    //   块 5000 一半旧一半新 → 数据损坏
    //   需要 journal 回滚

// COW 方式
cow_write(inode, offset, new_data):
    // 1. 分配新物理块
    new_data_block = alloc_block() // 分配块 8000
    write_block(8000, new_data)    // 写新块，旧块不动

    // 2. 更新 B-tree 叶子指向新块
    //   叶子也要 COW (不能原地改)
    new_leaf = copy_and_modify(old_leaf, offset -> 8000)
    new_leaf_block = alloc_block() // 分配块 8100
    write_block(8100, new_leaf)

    // 3. 父节点也要 COW (因为叶子指针变了)
    new_parent = copy_and_modify(old_parent,
                                  leaf_ptr -> 8100)
    new_parent_block = alloc_block() // 分配块 8200
    write_block(8200, new_parent)

    // ... 递归 COW 到根 ...

    // 4. 按格式要求写入带 generation/checksum 的新超级块版本
    publish_superblock_copy(new_root_block, generation)
    // mount/recovery 选择校验通过且代数完整的版本

    // 旧块 (5000, 旧叶子, 旧父节点...) 仍被旧根引用
    // 如果有 snapshot, 旧根仍然有效
    // 如果没有 snapshot, 旧块可以回收
```

	原地覆盖 (ext4/XFS)	COW (Btrfs/ZFS)
崩溃一致性	依靠日志、写序和恢复规则	依靠 COW、generation、校验和与提交协议

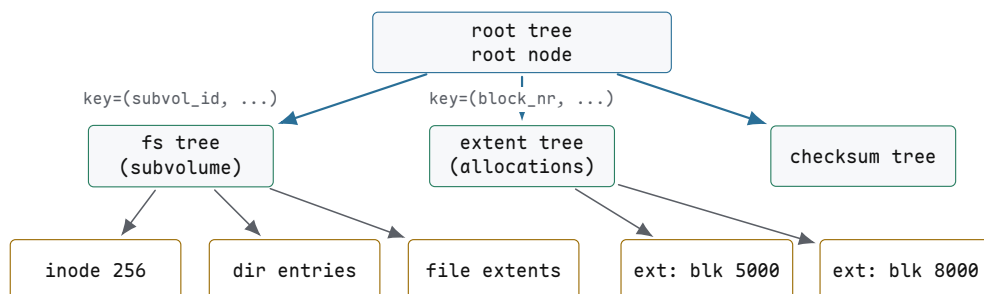
snapshot	需要额外 LVM 层	原生支持（保留旧根指针）
写放大	低（1 次写服务 1 次用户写）	高（3-6 倍）
碎片	低（原地覆盖）	高（每次写新位置）
空间回收	仍需位图/B+ 树等空间管理	还需引用追踪与延迟回收

Btrfs：一切都是 B-tree

Btrfs (B-tree file system) 的核心设计是把所有数据结构统一为 *B-tree*。超级块、extent 分配、目录、inode、校验和都是不同类型的 B-tree。

每个 B-tree 节点是一个磁盘块（4 KiB-64 KiB）。键是三元组 (*objectid*, *type*, *offset*)，这种统一键空间让不同类型的树可以共享同一套查找、插入、删除代码。

B-tree	key	value
root tree	tree id	指向子树根节点
extent tree	逻辑块号	extent ref（物理块号 + 引用计数 + owner）
chunk tree	逻辑 chunk 号	物理 chunk 映射（设备号 + 偏移）
dev tree	物理 extent	设备号 + 偏移
fs tree（每个子卷）	(inode#, type, offset)	inode / dir entry / file extent
checksum tree	(inode#, offset)	数据块校验和



图示：Btrfs 的多棵 B-tree 结构。root tree 是顶层，指向 fs tree（每个 subvolume 一棵）、extent tree（块分配跟踪）、checksum tree 等。所有树共享统一的 (*objectid*, *type*, *offset*) 键空间。

Btrfs snapshot：不复制文件数据的快速创建

Btrfs 的 snapshot 创建只需复制 root tree 的根节点指针，不复制任何数据块：

```

// Btrfs snapshot 创建
btrfs_snapshot(src_subvol, dst_subvol):
    // src_subvol 的 fs tree root 在块 5000
    // 创建 dst_subvol:
    //   1. 在 root tree 中添加一个新条目
    //   指向 src 的 fs tree root（块 5000）
    //   2. 增加块 5000 的引用计数（从 1 到 2）
    // 完成！没有复制任何数据块
    dst_subvol->fs_tree_root = src_subvol->fs_tree_root;
  
```

```

// 即块 5000
increase_refcount(5000); // extent tree 中 refcount 1->2
// 关键性质：不遍历并复制全部文件数据；
// 实际元数据工作量与实现和待提交事务状态有关。

// 之后在 dst 中修改文件：
btrfs_write(dst_subvol, inode, offset, data):
// COW 路径：
// 1. alloc new block 8000, write data
// 2. COW leaf: 新叶子指向 8000 (新块 8100)
// 3. COW parent: ... (新块 8200)
// 4. ...
// 5. COW 到 fs tree root: 新 root 在块 8400
// 6. 更新 dst_subvol->fs_tree_root = 8400
// dst 现在有独立的树，指向自己的新块

// src 的 fs tree root 仍然指向块 5000
// 块 5000 的 refcount 降回 1 (被 src 引用)
// 旧数据块 (5000 的子树) 仍被 src 引用，不回收

```

snapshot 创建后，两个 subvolume 共享未修改的数据块；只有后续被修改的块及其相关元数据路径才会分叉。因此创建通常远快于复制整棵文件树，但不能据此许诺与任意系统状态无关的严格常数时间。

旧树根在 COW 更新期间仍然有效，新快照通过共享引用保留既有数据，后续写入才使相关路径逐渐分叉。共享的数据块通过 extent tree 的引用记录管理，最后一个引用消失后才具备回收条件。创建时不复制全部文件数据；真实完成时间还受事务提交、元数据更新和系统负载影响。

ZFS：事务对象存储

ZFS (Zettabyte File System) 的设计哲学与 Btrfs 类似 (COW + snapshot)，但组织方式和数据完整性保证不同。

概念	含义	作用
pool	存储池，由多个 vdev 组成	聚合空间、提供冗余
vdev	虚拟设备 (disk、mirror、RAID-Z)	冗余层
dataset	文件系统或卷	独立的 snapshot/quota/属性
objset	dataset 内的对象集合	包含 dnode 和数据
dnode	ZFS 的 inode 等价物	存储元数据和 block pointer
block pointer	带校验和的块指针	端到端数据完整性
TXG	transaction group	批量原子提交
ZIL	ZFS Intent Log	同步写加速
ARC	Adaptive Replacement Cache	内存读缓存
L2ARC	SSD 二级缓存	读缓存扩展

ZFS 的每个 block pointer 都携带校验和 (256 位 SHA-256 或 fletcher4)。读数据时，ZFS 先读块、再校验 checksum。如果不匹配，说明数据静默损坏 (silent data corruption) — 磁盘返回了数据但内容不对。此时 ZFS 从冗余副本 (mirror 或 RAID-Z) 读取并修复。

```
// ZFS block pointer 结构 (简化)
struct blkptr_t {
    uint64_t blk_dva[3];    // 3 个 Data Virtual Address
                          // (可存 3 个副本的位置)
    uint64_t blk_prop;      // 压缩级别、加密、类型
    uint64_t blk_birth;     // 创建此块的 TXG 号
    uint64_t blk_fill;      // 填充信息
    uint64_t blk_cksum[8];  // 256-bit checksum
                          // (ZIO_CHECKSUM_SHA256)
};

// 读取一个 block pointer 指向的数据:
zio_read(bp):
    data = read_block(bp->blk_dva[0]); // 从第一副本读
    if checksum(data) == bp->blk_cksum:
        return data; // 校验通过, 数据完好

    // 校验失败 → 数据损坏!
    // 从第二副本读 (如果有 mirror/RAID-Z)
    data = read_block(bp->blk_dva[1]);
    if checksum(data) == bp->blk_cksum:
        // 第二副本完好
        // 修复第一副本: 重写 blk_dva[0]
        zio_rewrite(bp->blk_dva[0], data);
        return data;

    // 都坏了 → 不可恢复, 返回 EIO
```

ZFS 的 TXG (Transaction Group) 机制批量提交写操作。系统把修改组装成 TXG, COW 写入新块并更新 block pointer, 最后按格式协议写出新的 uberblock 槽位。恢复时从多个候选 uberblock 中选择校验和与事务号有效的版本; 这不是假设任意宽度的根指针更新天然具有硬件原子性。

```
// ZFS TXG 提交流程
txg_commit(pool):
    // 1. 冻结当前 TXG, 收集所有 dirty 数据
    txg = pool->txg_current;
    txg->state = TXG_STATE_QUIESCING;
    // 新写入进入下一个 TXG

    // 2. COW 写所有新数据块
    for dnode in txg->dirty_dnodes:
        for block in dnode->dirty_blocks:
            new_block = alloc_block(); // 从空间分配
            write_block(new_block, block.new_data);
            // 更新 block pointer: 指向新块, 带新 checksum
            block.bp = make_blkptr(new_block,
                                   checksum(block.new_data),
                                   txg->txg_num);
            // 旧块不覆盖, 旧 block pointer 还在旧 uberblock

    // 3. 更新所有 dnode (COW)
    for dnode in txg->dirty_dnodes:
        new_dnode_block = alloc_block();
        write_block(new_dnode_block, dnode);
        // dnode 指向新的 block pointers

    // 4. 更新 objset (COW)
    new_objset_block = alloc_block();
```

```

write_block(new_objset_block, objset);
// objset 指向新的 dnode blocks

// 5. 按格式要求写新的 uberblock 槽位并完成持久化顺序
//   uberblock 包含指向 root objset 的指针
new_uberblock = make_uberblock(
    new_objset_block, txg->txg_num, timestamp);
publish_uberblock_slot(pool, new_uberblock);
// 旧 uberblock 仍然存在 (ZFS 存多个 uberblock)
// 崩溃恢复：读最新的有效 uberblock

```

ZIL (ZFS Intent Log) 记录尚未进入已提交 TXG、却已向调用者承诺持久的同步写意图。同步调用必须等相应日志记录达到稳定存储后才可成功返回，而不是“写进内存后立即返回”。后续 TXG 提交把修改纳入主存储状态，相关日志记录即可回收；崩溃恢复时重放尚未被已提交 TXG 覆盖的有效同步记录。独立的 SLOG 设备是 ZIL 记录的一种承载位置，ZIL 本身不是必须单独购买的一块设备。

写放大：COW 的核心代价

COW 文件系统的写放大是核心代价。一次 4 KiB 的随机写，最坏情况下需要：

```

// 4 KiB 随机写在 Btrfs 中的写放大
// 假设 B-tree 深度 3 (root → internal → leaf → data)

// 步骤 1: 写新数据块
new_data_block = alloc_block(); // 块 8000
write_block(8000, user_data); // 1 次 4 KiB 写

// 步骤 2: COW 叶子节点 (data → 8000)
new_leaf = copy_modify(old_leaf);
new_leaf_block = alloc_block(); // 块 8100
write_block(8100, new_leaf); // 1 次 4 KiB 写

// 步骤 3: COW 父节点 (leaf → 8100)
new_parent = copy_modify(old_parent);
new_parent_block = alloc_block(); // 块 8200
write_block(8200, new_parent); // 1 次 4 KiB 写

// 步骤 4: COW 祖父节点
new_grandparent = copy_modify(old_gp);
new_gp_block = alloc_block(); // 块 8300
write_block(8300, new_gp); // 1 次 4 KiB 写

// 步骤 5: COW 根节点
new_root = copy_modify(old_root);
new_root_block = alloc_block(); // 块 8400
write_block(8400, new_root); // 1 次 4 KiB 写

// 步骤 6: 原子更新超级块根指针
atomic_write(superblock.root, 8400); // 1 次 4 KiB 写

// 总计：6 次 4 KiB 写，服务 1 次用户写
// 写放大 = 6x

```

步	写什么	块号	说明
1	数据块	8000	新数据内容
2	叶子节点	8100	COW: extent 指向 8000

3	父节点	8200	COW: 指向新叶子 8100
4	祖父节点	8300	COW: 指向新父节点 8200
5	根节点	8400	COW: 指向新祖父 8300
6	超级块	—	原子更新根指针 → 8400

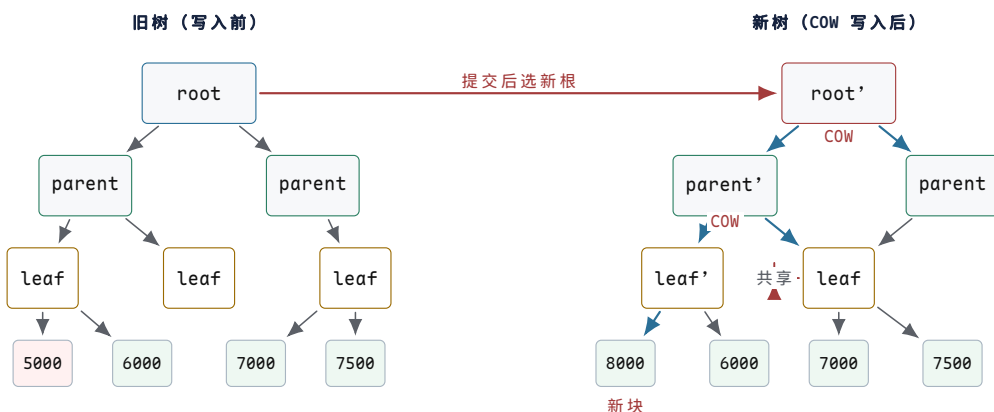
Btrfs 和 ZFS 通过以下策略缓解写放大：

- 批量提交：把许多写操作收集到一个 TXG/transaction 中，一次提交。多用户写共享同一批 COW 元数据节点，分摊开销。
- ZIL / journal：同步写先记到小日志，不等完整 COW 路径。
- Clone-friendly allocator：分配器尽量让新块在物理上靠近，减少碎片和元数据分散。
- B-tree 节点变大：Btrfs 可用 64 KiB 节点，每个节点容纳更多条目，减少树深度和 COW 层数。

对比：ext4 vs Btrfs vs ZFS

	ext4 (journal)	Btrfs (COW)	ZFS (COW)
崩溃恢复	日志重放与有序数据规则	COW 事务、tree-log 与超级块代数选择	TXG、ZIL 与 uberblock 选择
snapshot	需 LVM 等上层机制	原生、创建元数据开销较小	原生、创建元数据开销较小
写放大	1-1.5x	3-6x	3-6x
碎片	低（原地覆盖）	高（COW 散布）	高（COW 散布）
空间效率	需日志空间	需 refcount 元数据	需校验和 + refcount
数据完整性	无校验	有校验（checksum tree）	端到端校验 + 自修复
冗余集成	通常依赖 md/device-mapper	原生 block-group profiles, RAID56 有额外风险	原生 mirror/RAID-Z
压缩	否	是（zlib/zstd）	是（lz4/gzip）

COW 设计通过写新版本与提交协议提供恢复点和原生 snapshot，但会引入元数据更新、引用追踪和碎片等成本。允许原地更新的文件系统通常以日志保护特定元数据，并获得不同的写放大与延迟特征。两类设计对用户数据原子性、校验和和恢复范围的承诺仍取决于具体实现与配置，不能只按“COW”或“日志”标签推断。



图示：COW 更新路径。写入块 5000 的数据时，不覆盖块 5000，而是分配新块 8000 写入数据。从叶子到根的路径上的每个节点都 COW 出新版本（红色标记）。格式规定的提交协议完成后，恢复规则才选择新根；这不依赖一个抽象根指针天然原子。旧块 5000 仍被旧树引用，可用于 snapshot。共享的子树（parent 右子树、leaf 和 6000/7000/7500 块）不被复制。

闪存约束、FTL 与日志结构

NAND Flash 的物理约束

NAND Flash 的物理特性与 HDD 有根本区别，直接决定了闪存文件系统的设计方向。

特性	NAND Flash	HDD
读写单位	page (4-16 KiB)	sector (512 B)
擦除单位	block (128-512 KiB = 32-128 pages)	无需擦除，直接覆盖
写约束	只能写已擦除的页（全 1）	无约束
寻址	电气 (μ s 级)	机械寻道 (ms 级)
寿命	P/E 循环有限	无限（理论上）

NAND 的核心约束是写之前必须先擦除，而擦除单位远大于写入单位。一个 block 包含 64-128 个 page。要修改一个 page 的内容，不能直接覆盖—必须先擦除包含该 page 的整个 block，然后重新写入。但 block 中其他 page 可能还有有效数据，擦除会丢失它们。

```
// NAND Flash 的操作约束
// page: 4 KiB (读写单位)
// block: 256 KiB = 64 pages (擦除单位)
//
// 写一个 page:
// 如果该 page 已被擦除 (所有 bit = 1):
//   page_program(page_addr, data) // 直接写入
// 如果该 page 之前写过 (有非 1 bit):
//   不能直接覆盖! 会损坏数据
//   必须:
//   1. 找一个已擦除的新 page
//   2. 写入新数据到新 page
//   3. 旧 page 标记为 "invalid" (stale)
//   4. 当整个 block 的有效页很少时:
//       a. 把有效页复制到新 block
//       b. erase_block(old_block) → 全部变 1
//       c. 旧 block 可以重新写
```

```
// P/E 循环寿命 (Program/Erase cycles)
// SLC: 100,000 次/block
// MLC: 3,000-10,000 次/block
// TLC: 1,000-3,000 次/block
// QLC: 100-1,000 次/block
// 超过 P/E 寿命的 block 变成坏块
```

除了 P/E 寿命限制, NAND 还有其他可靠性问题:

- *Read Disturb* (读干扰): 读取同一个 page 太多次, 可能导致相邻 page 的 bit 翻转。典型阈值: 每个 block 读取 10^5 – 10^6 次后需要刷新。
- *Write Disturb* (写干扰): 编程一个 page 时, 高压脉冲可能影响同一 block 中相邻 page 的电荷。
- *Retention* (保持性): 存储的电荷随时间泄漏。TLC 闪存在室温下数据保持约 1–3 年, 高温环境保持时间更短。
- *Bad blocks* (坏块): 出厂时有初始坏块 (通常 $\leq 2\%$), 运行中会持续产生新的坏块。

常见误区

NAND Flash 的写前擦除约束是所有闪存文件系统设计的出发点。传统文件系统 (ext4) 假设可以随时覆盖任何块, 这在 NAND 上不成立—要么通过 FTL 翻译层隐藏这个约束, 要么设计专门的日志结构文件系统 (F2FS) 直接适配它。

FTL: 闪存翻译层

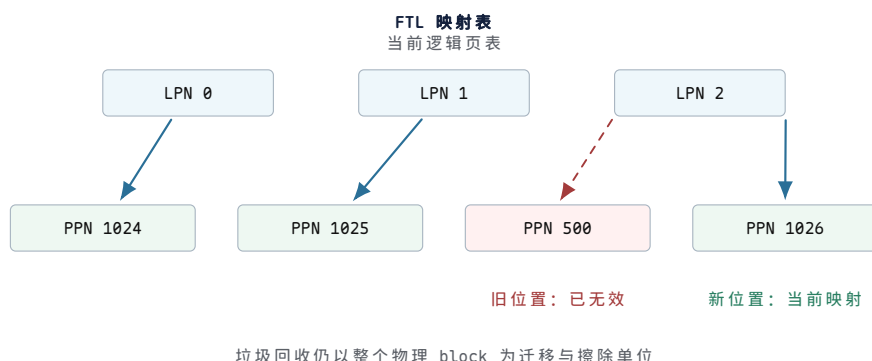
大多数 SSD 内部有 FTL (Flash Translation Layer), 把 NAND 的页/块语义翻译成块设备的扇区语义, 对上层文件系统透明。FTL 的核心职责是地址映射、垃圾回收和磨损均衡。

```
// FTL 地址映射表
// 逻辑扇区号 → 物理页号
//
// 1 TiB SSD, 4 KiB pages
// 逻辑扇区数 = 1 TiB / 512 B = 2 * 10^9
// 逻辑页数 = 1 TiB / 4 KiB = 256 * 10^6 = 256M
// 映射表项 = 256M entries * 4 bytes = 1 GiB
//
// 这张表存在 SSD 的 DRAM 中 (SSD 有自己的 DRAM)
// 断电时持久化到 NAND 的保留区域

struct ftl_mapping_entry {
    uint32_t logical_page; // 逻辑页号
    uint32_t physical_page; // 物理页号
};
// 映射表: logical_page -> physical_page

// FTL 写操作
ftl_write(logical_page, data):
    // 1. 分配一个已擦除的物理页
    physical_page = alloc_erased_page();
    // 2. 编程写入数据
    nand_page_program(physical_page, data);
    // 3. 更新映射表
    old_physical = mapping_table[logical_page];
    mapping_table[logical_page] = physical_page;
    // 4. 旧物理页标记为无效
```

```
mark_page_invalid(old_physical);
// 注意：旧页不能直接擦除
//      要等整个 block 的有效页都迁移后才能擦除
```



图示：FTL 地址映射表。逻辑页号（LPN）映射到物理页号（PPN）。LPN 2 的当前映射指向 PPN 1026；虚线保留旧位置 PPN 500 已失效这一历史事实。当旧页所在 block 的有效页比例足够低时，FTL 垃圾回收会迁移仍有效的页并擦除整个 block。

FTL 垃圾回收

当空闲的已擦除 block 不足时，FTL 执行垃圾回收，回收包含大量无效页的 block：

```
ftl_gc():
// 1. 选择 victim block: 无效页最多的 block
victim = find_block_with_most_invalid_pages();
// 假设 victim 有 60 个无效页, 4 个有效页

// 2. 分配一个已擦除的新 block
new_block = alloc_erased_block();

// 3. 遍历 victim 中的每个 page
new_page_offset = 0;
for page in victim.pages:
    if is_page_valid(page):
        // 有效页: 复制到新 block
        data = nand_page_read(page.physical);
        nand_page_program(new_block[new_page_offset], data);
        // 更新映射表
        mapping_table[page.logical] = new_block[new_page_offset];
        new_page_offset++;
    // 无效页: 跳过, 不复制

// 4. 擦除旧 block
nand_block_erase(victim);
// 现在 victim 是全 1, 可以重新写入

// 5. 新 block 还有剩余已擦除页, 加入空闲池
// victim 有 64 pages, 4 个被复制, 60 个空闲
// → 新 block 提供 60 个空闲页

// 写放大计算:
// 用户写入 4 个页 (4 * 4 KiB = 16 KiB)
// GC 迁移了 4 个有效页 (4 * 4 KiB = 16 KiB)
// 总闪存写 = 用户写 + GC 写 = 16 + 16 = 32 KiB
// 写放大 = 32 / 16 = 2x
```

```
//
// 如果 victim 只有 1 个有效页：
// 用户写 63 页，GC 迁移 1 页
// 写放大 = (63 + 1) / 63 ≈ 1.02x （很好）
//
// 如果 victim 有 32 个有效页：
// 用户写 32 页，GC 迁移 32 页
// 写放大 = (32 + 32) / 32 = 2x
//
// 最坏情况：几乎所有页都有效，GC 频繁迁移
// 写放大可达 3-4x 甚至更高
```

磨损均衡

P/E 循环有限，必须把写操作均匀分布到所有 block，避免热点 block 过早耗尽。磨损均衡分两种：

```
// 动态磨损均衡 (Dynamic Wear Leveling)
// 分配器选择 P/E 循环最少的已擦除 block
ftl_alloc_page_dynamic():
    // 从空闲 block 池中选择 erase_count 最小的
    block = min(free_blocks, key=lambda b: b.erase_count);
    return block.alloc_page();

// 静态磨损均衡 (Static Wear Leveling)
// 后台任务：把冷数据从高磨损 block 迁移到低磨损 block
// 释放高磨损 block 给新写入使用
ftl_static_wl():
    // 找到 erase_count 最高的 block (写多，存冷数据)
    hot_block = max(all_blocks, key=lambda b: b.erase_count);
    // 找到 erase_count 最低的 block (写少，存热数据)
    cold_block = min(all_blocks, key=lambda b: b.erase_count);

    if hot_block.erase_count - cold_block.erase_count > threshold:
        // 交换：把冷数据移到低磨损 block
        for page in hot_block.valid_pages:
            data = read_page(page);
            new_page = cold_block.alloc_page();
            write_page(new_page, data);
            update_mapping(page.logical, new_page);
        // 高磨损 block 现在空了，擦除后可用于新写入
        erase_block(hot_block);
```

TRIM/DISCARD

文件系统删除文件时只更新元数据 (bitmap、inode)，不擦除数据块内容。在 HDD 上这不影响性能—磁盘不需要擦除就能覆盖写。但 SSD 的物理块需要先擦除才能写入，FTL 不知道哪些逻辑块对应的物理块可以回收。

TRIM (也叫 **DISCARD**) 命令让文件系统告诉 SSD “这些逻辑块不再被使用”：

```
// 文件系统删除文件后发 TRIM
unlink(path):
    inode = resolve(path)
    // 释放 inode 和数据块 (更新文件系统元数据)
    free_inode(inode)
    free_blocks(inode.block[])
    // 告诉 SSD 这些逻辑块不再使用
    // blkdev_discard 发送 DISCARD/TRIM 命令
    blkdev_discard(bdev,
```

```

        inode.block[] * blocksize,
        inode.block_count * blocksize)

// FTL 收到 TRIM 后
ftl_trim(logical_blocks):
    for lb in logical_blocks:
        physical = mapping_table[lb]
        // 标记物理页为无效
        mark_page_invalid(physical)
        // 不需要立即擦除
        // 垃圾回收时这些无效页不需要迁移
        // → GC 更快, 写放大更低

// ext4 挂载选项:
// mount -o discard /dev/nvme0n1 /mnt
//     → 同步 TRIM: 删除时立即发 TRIM
// mount -o discard=async /dev/nvme0n1 /mnt
//     → 异步 TRIM: 收集后批量发
// mount -o noDiscard /dev/nvme0n1 /mnt
//     → 禁用 TRIM

```

	无 TRIM	有 TRIM
GC 迁移	迁移已删除数据的无效页	不迁移（已知无效）
写放大	高	低
空闲块	少（GC 不知哪些可回收）	多

F2FS：日志结构闪存文件系统

F2FS (Flash-Friendly File System) 是 Samsung 开发的 Linux 文件系统，专门为 NAND Flash 优化。核心设计是日志结构 (log-structured)：所有写操作都是顺序追加，不覆盖旧数据。

```

// F2FS 磁盘布局
// 整个设备分为以下区域（按顺序）：
//
// [Superblock] [Checkpoint] [SIT] [NAT] [SSA] [Main Area]
//
// Superblock: 全局参数（块大小，段大小，总段数）
// Checkpoint: 两个交替的 checkpoint（崩溃恢复）
// 每个 checkpoint 记录：
//     - NAT 的根位置
//     - SIT 的根位置
//     - Main Area 的空闲段位置
//     - 活跃 segment 列表
//
// SIT (Segment Info Table):
// 每个 segment 的状态：有效块数，无效块数
// GC 用 SIT 找到无效块最多的 segment
//
// NAT (Node Address Table):
// inode 号 → node block 的物理地址
// 类似于 ext4 的 indirect block，但用一张表而非指针
// 作用：数据块地址不直接存在 inode 中
//     inode → NAT → node block → data block
//
// SSA (Segment Summary Area):
// 每个 segment 内每个块的：(inode 号，偏移)

```



```
// GC 用 SSA 判断一个块是否有效
// 有效 = 该逻辑块仍被某 inode 引用
//
// Main Area: 6 种 segment 类型
// hot node: 直接 inode block (频繁更新)
// warm node: 间接 node block
// cold node: 目录/很少更新的 node
// hot data: 目录数据 (频繁更新)
// warm data: 普通文件数据
// cold data: 很少访问的数据 (多媒体等)
```



图示：F2FS 磁盘布局。Superblock 和 Checkpoint 是元数据入口。SIT/NAT/SSA 是三张辅助表，分别管理段状态、节点地址映射和块有效性。Main Area 分 6 种 segment 类型 (hot/warm/cold × node/data)，按热度分离写入。所有写入都是顺序追加到 log tail。

F2FS 写路径与 GC

F2FS 的写路径完全顺序化：新数据始终追加到 log tail 的下一个空闲 segment。旧数据所在的 segment 逐渐变为只有少量有效页。

```
// F2FS 写路径
f2fs_write(inode, offset, data):
// 1. 找到当前 log tail 的位置
// (hot/warm/cold 决定写入哪个 segment 类型)
segment = get_active_segment(SEG_WARM_DATA);
page_offset = segment.next_free_page;

// 2. 写数据到 segment
write_page(segment, page_offset, data);
segment.next_free_page++;

// 3. 更新 NAT: inode 的 node block 指向新数据位置
// node block 本身也要 COW (日志结构)
node_block = get_node_block(inode);
new_node_block = copy_modify(node_block,
    offset -> segment:page_offset);
node_segment = get_active_segment(SEG_WARM_NODE);
new_node_addr = node_segment.next_free_page;
write_page(node_segment, new_node_addr, new_node_block);

// 4. 更新 NAT 表: inode -> new_node_block 地址
nat_update(inode, new_node_addr);
// 旧 node block 所在 page 变为无效

// 5. 更新 SSA: 记录新数据页和 node 页的归属
ssa_update(segment, page_offset, (inode, offset));
ssa_update(node_segment, new_node_addr, (inode, NODE));

// 6. 如果当前 segment 写满, 分配新 segment
if segment.next_free_page >= segment_size:
    segment = alloc_new_segment(SEG_WARM_DATA);
```

```

// F2FS GC
f2fs_gc():
// 1. 用 SIT 找无效块最多的 segment
victim = find_segment_with_most_invalid_blocks();
// 例如: segment 有 512 blocks, 480 无效, 32 有效

// 2. 用 SSA 确定每个有效块的归属
for block in victim.valid_blocks:
    (inode, offset) = ssa_lookup(block);
// 3. 复制有效块到 log tail
data = read_block(block);
f2fs_write(inode, offset, data); // 追加到新位置
// NAT 和 SSA 自动更新

// 4. 整个 segment 现在全是无效块, 回收
free_segment(victim);
// segment 加入空闲池, 可重新使用

```

F2FS 的 hot/warm/cold 分离是关键优化。元数据 (hot node, 频繁更新) 与用户数据 (warm/cold data) 写入不同的 segment。这样 GC 时不会因为迁移热元数据而连带迁移冷数据——热 segment 会更频繁地被 GC 回收 (因为无效页增长快), 但冷 segment 不受影响。

segment 类型	内容	GC 特点
hot node	直接 inode block	更新频繁, 无效快, GC 频繁但小
warm node	间接 node block	中等频率
cold node	目录 node	很少更新, GC 几乎不碰
hot data	目录数据	频繁更新
warm data	普通文件数据	中等
cold data	多媒体等	写一次读多次, GC 不碰

日志结构文件系统原理 (LFS)

F2FS 的设计思想源自 LFS (Log-structured File System), 由 Rosenblum 和 Ousterhout 在 1992 年提出。LFS 的核心原则: 磁盘是一个只追加的日志。

```

// LFS 核心原理
// 1. 所有写操作都是追加:
// 数据块、inode、目录项、间接块
// 全部追加写入日志尾部
// 旧版本数据变成 "garbage" (垃圾)

// 2. inode map (类似 F2FS 的 NAT):
// inode 号 -> inode 最新版本的磁盘位置
// inode map 本身也写入日志

// 3. 读操作:
// 查 inode map 找到 inode 位置
// 读 inode 获取数据块指针
// 读数据块 (位置在日志中, 可能很分散)
// → 随机读可能慢 (数据散布在日志各处)

// 4. 清理 (cleaner / GC):
// 后台进程扫描旧 segment

```

```
// 复制有效块到 log tail
// 释放旧 segment

// LFS 的基本权衡：
// 写：快（顺序追加，无需寻道/擦除）
// 读：可能慢（数据散布在日志中，随机访问）
// GC：后台开销，可能影响前台延迟
```

	LFS/F2FS（日志结构）	ext4（原地覆盖 + 日志）
写模式	顺序追加	原地覆盖 + 独立日志
写放大	低（无覆盖写）	中（日志 + 主 FS）
随机读	慢（数据散布）	快（extent 连续）
GC 开销	有（cleaner）	无
空间开销	NAT/SIT/SSA 元数据	bitmap + 日志
碎片	随时间增长	延迟分配缓解

核心思想

日志结构文件系统的核心收益是写性能：NAND Flash 的顺序写比随机写快得多（无需先擦除，利用 page program 的顺序特性），且写放大更低。代价是读性能可能下降（数据散布在日志各处）和 GC 的后台开销。F2FS 通过 hot/warm/cold 分离、NAT 间接寻址和前台 GC 控制来缓解这些代价。

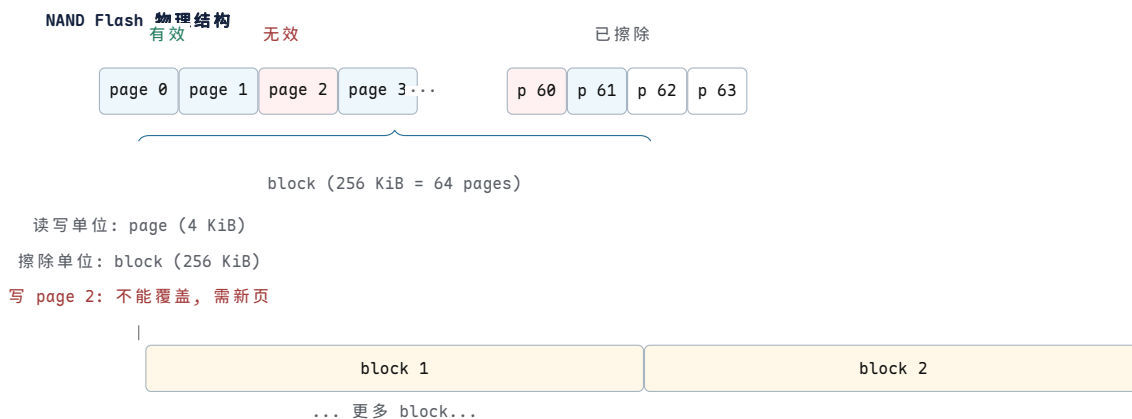
F2FS vs ext4 on SSD

在 SSD 上，F2FS 和 ext4 都能工作，但性能特征不同：

	F2FS	ext4
写模式	顺序追加（闪存友好）	原地覆盖（FTL 翻译）
写放大	低（无覆盖，GC 高效）	中（FTL GC + ext4 日志）
元数据开销	高（NAT + SIT + SSA）	低（bitmap + inode table）
随机读	中（NAT 间接一层）	快（extent 直接映射）
崩溃恢复	checkpoint 交替	jbd2 日志重放
TRIM	集成到 GC	需挂载选项

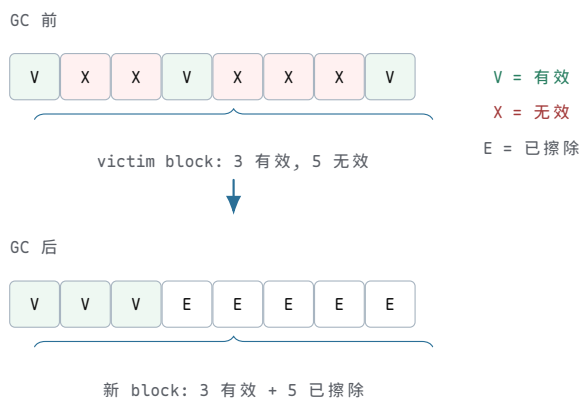
F2FS 的优势在写密集场景：闪存的顺序追加写不需要 FTL 做额外的地址翻译和 GC，写放大接近 1x。ext4 的原地覆盖写触发 FTL 的写前擦除约束，FTL 内部 GC 产生额外写放大。

ext4 的优势在读密集和元数据密集场景：extent 树直接映射逻辑块到物理块，而 F2FS 需要通过 NAT 间接一层。对于小文件和目录操作，ext4 的 bitmap 分配比 F2FS 的 segment 管理更轻量。



图示: NAND Flash 物理结构。一个 block 包含 64 个 page (256 KiB)。绿色为有效页, 红色为无效页 (旧数据), 白色为已擦除页 (全 1, 可写)。写单位是 page, 擦除单位是 block。修改 page 2 不能直接覆盖—必须分配新页写入, 旧页标记无效。当 block 的无效页足够多时, GC 迁移有效页并擦除整个 block。

FTL 垃圾回收过程



图示: FTL 垃圾回收过程。GC 前 victim block 有 3 个有效页和 5 个无效页。GC 把 3 个有效页复制到新 block, 然后擦除 victim block。结果是一个新 block 包含 3 个有效页和 5 个已擦除页, 可以接受新写入。写放大 = (3 迁移 + 3 用户写) / 3 = 2x。

核心理想

从 page cache 到 COW 到闪存文件系统, 本章展示了文件系统设计的核心张力: 性能、一致性、介质适应性三者之间的取舍。page cache 用内存换延迟; 日志用额外写换恢复速度; COW 用写放大换崩溃一致性和 snapshot; 日志结构用顺序写换闪存友好性。没有免费的午餐—每个优化都在另一个维度产生代价。理解这些取舍, 才能根据工作负载和硬件选择正确的文件系统。

第 14 章 现代文件系统的组织与取舍

核心思想

基础文件系统已经能够保存、查找和恢复文件，但容量、并发、介质特征与可靠性要求会继续放大元数据访问、随机写、空间碎片和静默损坏等成本。现代设计不是术语的集合；每一种优化都应当从明确的工作负载或故障条件出发，并说明它增加的状态、写放大与恢复责任。

小文件、校验和与空间效率

前几节建立了文件系统的基础机制：索引节点、目录项、日志、B+ 树、空间分配。然而，真实世界中的文件系统还必须应对一系列工程挑战——小文件膨胀导致的空间浪费、静默数据腐烂 (bit rot)、闪存设备的写放大、延迟敏感型工作负载、容器镜像的层叠结构。本节深入分析现代文件系统采用的核心优化技术，每一项都是资源 (CPU、内存、复杂性) 与收益 (I/O、空间、可靠性) 之间的权衡。

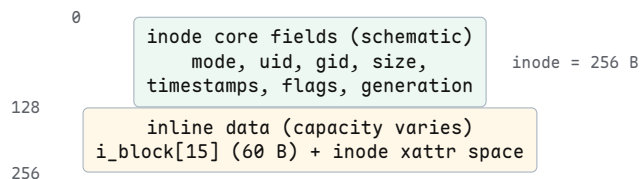
inline data: 小文件内嵌

一个典型的配置文件——`/etc/hostname`——通常只有 10-20 字节。在传统的 ext4 布局中，即使文件只有 1 字节，文件系统也会为其分配至少一个 4 KiB 数据块。inode 本身占据 256 字节，其中 `i_block[15]` 数组 (60 字节) 用于存放区段树 (extent tree) 的根节点。对于小文件而言，这些空间大部分被浪费。

ext4 的 `inline data` 特性 (标志 `EXT4_FEATURE_INCOMPAT_INLINE_DATA`) 解决了这个问题：当文件足够小时，直接将文件数据内嵌到 inode 结构中，不需要任何独立的数据块。`i_block[]` 数组原本用于存放区段指针，现在被重新利用为内嵌数据区域。对于一个 256 字节的 inode，标准元数据占据前 128 字节，剩余 128 字节的额外 inode 空间 (extra inode space) 也可以用于内嵌数据，合计 ~ 128 字节。ext4 进一步支持通过 `i_data` 字段 (位于额外 inode 空间内) 将内嵌容量扩展到 ~ 384 字节。

```
/* simplified ext4 on-disk inode */
struct ext4_inode {
    __le16 i_mode;           /* 0 file mode */
    __le16 i_uid;            /* 2 low 16 bits of UID */
    __le32 i_size_lo;        /* 4 size (low 32 bits) */
    __le32 i_atime;          /* 8 access time */
    __le32 i_ctime;          /* 12 inode change time */
    __le32 i_mtime;          /* 16 modification time */
    __le32 i_dtime;          /* 20 deletion time */
    __le16 i_gid;            /* 24 low 16 bits of GID */
    __le16 i_links_count;    /* 26 hard link count */
    __le32 i_blocks_lo;      /* 28 blocks count */
    __le32 i_flags;          /* 32 inode flags */
    __le32 i_osd1;           /* 36 OS-specific */
    __le32 i_block[15];      /* 40 block pointers / extents
                             /* repurposed for inline_data */
    __le32 i_generation;     /* 100 version (NFS) */
    __le32 i_file_acl_lo;    /* 104 extended attr block */
    /* --- standard inode ends at offset 128 --- */
    __le32 i_size_high;      /* 108 size (high 32 bits) */
    __le32 i_obso_faddr;     /* 112 obsolete fragment addr */
    __le16 i_checksum_lo;    /* CRC32C checksum (low) */
    /* --- extra inode space: up to 256 bytes total --- */
}
```

```
/*      inline data continues here via xattr mechanism      */
};
```



图示:ext4 inline data inode 布局示意。前一部分数据可放在 60 字节的 `i_block[]` 区域,更多内容可使用 inode 的扩展属性空间;实际容量取决于 inode 大小和已有 xattr,不能固定写成 128 字节。足够小的文件可以不分配独立数据块。

Btrfs 提供了类似但更慷慨的内嵌支持。Btrfs 的 *inline extent* 可以将整个小文件存储在 B 树叶子节点中,容量上限取决于叶子大小(通常 4 KiB)减去头部和其他条目,实际上可达 ~ 3.8 KiB。由于 Btrfs 的 B 树叶子已经在内存中,内嵌文件的读取不需要额外的 I/O。

影响分析:考虑一个典型的 `/etc` 目录,包含 ~ 3000 个配置文件,其中大多数 < 128 字节。在不启用 inline data 时,每个小文件至少需要 1 个 inode 块(与其他 inode 共享)加 1 个数据块 = 8 KiB I/O(最坏情况)。启用 inline data 后,文件数据随 inode 一起读取,零额外 I/O。对于遍历 `/etc` 的全量扫描,I/O 从 ~ 3000 次数据块读取降至 0 次——所有数据已在 inode 块中。

inline data 利用 inode 内的可用空间保存小文件内容,省去独立数据块和一次间接寻址。当内容超过内联容量时,文件系统需要把它迁移到普通块映射;实际读取是否产生 I/O 还取决于 inode 所在元数据块是否已经缓存。

元数据校验和

存储介质会发生 静默数据腐烂 (silent data corruption, bit rot)。原因包括:宇宙射线导致的位翻转、控制器固件 bug、NAND 读取扰动 (read disturb)、磁介质磁畴退化、固件写入不完整。一个被损坏的 inode 块如果不被检测到,会导致文件系统将错误的物理块号映射给文件——这是灾难性的。

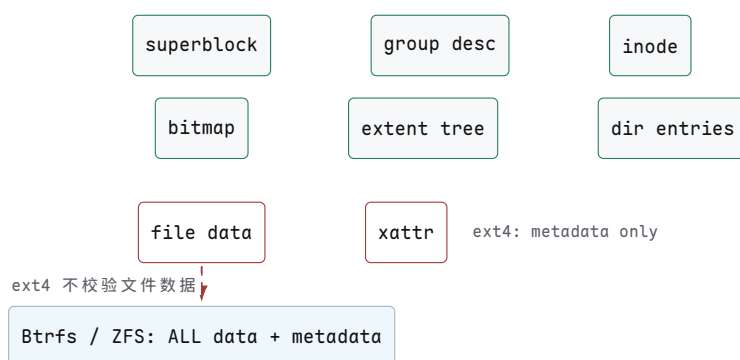
ext4 的 *metadata checksums* 特性(标志 `EXT4_FEATURE_RO_COMPAT_METADATA_CSUM`)为以下结构添加 CRC32C 校验和:

结构	校验和字段	覆盖范围
superblock	<code>s_checksum</code>	superblock 全部字段
group descriptor	<code>bg_checksum</code>	group descriptor 全部字段
inode	<code>i_checksum_lo/hi</code>	inode + inode number + generation
extent header	<code>eh_checksum</code>	extent header + 所有 extent 条目
directory htree	<code>dx_tail</code> (dx_node 尾部)	目录块全部内容
block bitmap	(group descriptor 中的校验和)	bitmap 块全部内容

CRC32C (Castagnoli 多项式 0x1EDC6F41) 被选中是因为 x86 SSE4.2 提供了硬件加速指令 `crc32`, 使得校验计算几乎零开销。ARMv8 也提供了 `CRC32` 指令。在没有硬件支持的平台, 内核回退到软件 CLMUL 实现。

```
/* ext4 extent checksum computation (simplified) */
static __le32 ext4_extent_block_csum(struct inode *inode,
                                     struct ext4_extent_header *eh)
{
    struct ext4_inode_info *ei = EXT4_I(inode);
    struct ext4_extent_tail *tail;

    /* checksum covers: extent header + all extent entries
       (but NOT the tail itself, which holds the checksum) */
    tail = find_ext4_extent_tail(eh);
    return cpu_to_le32(
        ext4_crc32c(inode->i_sb,
                    (void *)eh,
                    (__u8 *)tail - (__u8 *)eh, /* data to checksum */
                    ei->i_crc_seed));           /* seed = inode
                                                number + generation */
}
```



图示: ext4 仅对元数据计算 CRC32C 校验和。文件数据块本身没有校验。Btrfs 和 ZFS 对所有数据和元数据均计算校验和, 并在每次读取时验证。

常见误区

ext4 的元数据校验和不保护文件数据内容。如果一个数据块在磁盘上发生了位翻转, ext4 在读取该块时不会检测到错误, 应用程序会收到损坏的数据。只有 Btrfs 和 ZFS 的全数据校验才能在读取时检测并 (通过镜像/校验) 修复数据损坏。

ZFS 的校验策略更为彻底: 每个块 (数据块和元数据块) 都有 256 位校验和 (默认 `fletcher4`, 可选 `sha256` 或 `edon-R`)。ZFS 在每次读取时都会重新计算并验证校验和。如果校验失败, ZFS 会尝试从镜像副本或 RAID-Z 校验数据中恢复正确的块。Btrfs 默认使用 CRC32C, 也支持 `xxhash64`、`sha256` 和 `blake2b`。

透明压缩

存储 I/O 是许多工作负载的瓶颈。如果数据可以被压缩, 那么在写入时压缩、在读取时解压, 可以减少实际 I/O 量—以 CPU 时间换取 I/O 带宽。Btrfs、ZFS 和 F2FS 支持透明压缩 (transparent compression), 应用程序无需感知。

写入路径: 应用程序调用 `write()`, 数据进入页缓存 (page cache)。在写回时, 文件系统对每个数据块执行压缩算法。如果压缩后的块比原始块小, 文件系统分配

更少的物理空间并记录压缩信息（压缩算法、压缩后大小）。读取路径：文件系统读取压缩后的块，执行解压，返回原始数据。

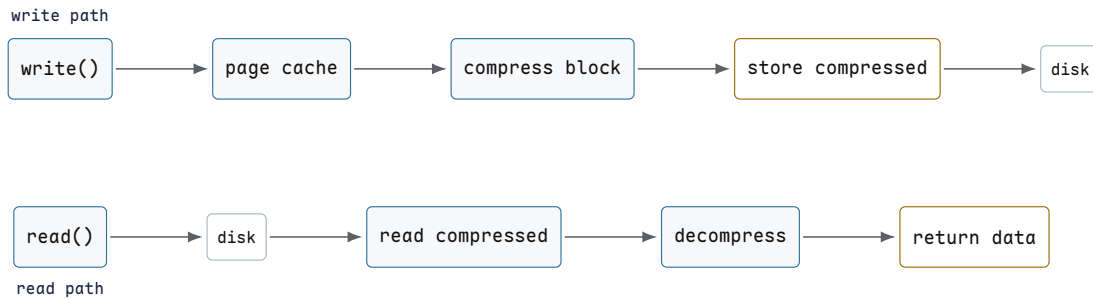
```
/* ZFS compression: lz4 early-abort logic (simplified) */
int lz4_compress(void *src, void *dst, size_t s_len, size_t d_len)
{
    /* Early abort: compress first 4 KiB.
       If ratio < 1/8 (12.5% improvement), skip entirely. */
    if (s_len >= 4096) {
        int prefix = lz4_compress_4k(src, dst);
        if (prefix > 4096 * 7 / 8) {
            /* Not compressible enough - store uncompressed */
            return -1; /* signals: store raw block */
        }
    }
    /* Full compression attempt */
    return lz4_compress_full(src, dst, s_len, d_len);
}
```

文件系统	默认算法	可选算法	早期中止	特点
ZFS	lz4	gzip, zstd	lz4: 前 4 KiB	lz4 速度 ~ 500 MB/s 压缩, ~ 2.5 GB/s 解压
Btrfs	zlib	zstd, lzo, none	zlib/zstd: 是	zstd 压缩比更好, 速度接近
F2FS	lz4	zstd, none	是	针对 NAND 优化的压缩
ext4	—	—	—	不支持透明压缩

具体数字：1 GiB 文本文件，4 KiB 块大小 = 262144 个块。假设压缩比为 3:1（典型文本文件），压缩后需要 87381 个块 = $87381 \times 4 \text{ KiB} = 341 \text{ MiB}$ 。I/O 减少了 67%。

```
=== 压缩 I/O 节省计算 ===
原始: 1 GiB / 4 KiB = 262144 blocks
压缩比: 3:1
压缩后: 262144 / 3 = 87381 blocks (向上取整)
空间: 87381 * 4 KiB = 341 MiB (节省 683 MiB, 67%)
I/O: 写入减少 67%, 读取减少 67%
CPU: 压缩和解压成本必须在目标 CPU、数据集和算法级别上实测
```

```
=== 性能权衡 ===
I/O-bound: CPU 空闲等 I/O → 压缩用 CPU 换 I/O → 吞吐 ↑
CPU-bound: CPU 已满 → 压缩增加 CPU 负载 → 吞吐 ↓
存储受限且 CPU 有余量 → 压缩可能提升吞吐
CPU 已饱和或数据不可压缩 → 压缩可能降低吞吐
```



图示：透明压缩的写入与读取路径。写入时压缩每个数据块，仅存储压缩后的数据。读取时先读取压缩块，再解压返回。对 I/O-bound 的工作负载，CPU 用在压缩上的时间通常远小于节省的 I/O 等待时间。

块级去重

许多存储系统包含大量重复数据：100 台虚拟机运行相同的操作系统镜像，容器镜像共享相同的基础层，备份系统存储逐日累积的相似快照。块级去重(block-level deduplication) 在块内容粒度上消除冗余：如果两个不同的逻辑块包含完全相同的字节序列，文件系统只存储一份物理副本。

机制：ZFS 维护一张去重表 (Dedup Table, DDT)。对于每个写入的块，ZFS 计算 SHA-256 哈希，然后在 DDT 中查找：

```

/* ZFS dedup write path (simplified) */
int ddt_lookup(zfs_sb_t *zsb, ddt_entry_t *dde, void *data)
{
    /* 1. Compute SHA-256 hash of block content */
    dde->dde_hash = sha256_hash(data, blocksize);

    /* 2. Check dedup table */
    ddt_entry_t *existing = ddt_table_lookup(zsb, &dde->dde_hash);
    if (existing != NULL) {
        /* 3a. Block already exists: increment refcount, return */
        existing->dde_phys.ddp_refcnt++;
        dde->dde_phys = existing->dde_phys; /* physical block addr */
        return DDT_EXISTING; /* no actual write performed */
    }

    /* 3b. New block: write to disk, add to DDT */
    ddt_table_add(zsb, dde);
    return DDT_NEW; /* caller performs actual write */
}

```

内存代价：去重表必须常驻内存（或在 ARC 缓存中），否则每次写入都需要额外的 I/O 来查表。对于 1 TiB 存储空间，4 KiB 块大小：

```

=== 去重表内存计算 ===
存储容量：    1 TiB = 2^40 B
块大小：      4 KiB = 2^12 B
总块数：      2^40 / 2^12 = 2^28 = 268,435,456 (≈ 256M)

```

每个 DDT 条目：

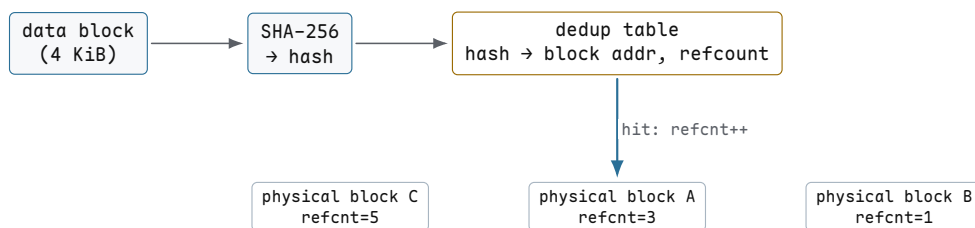
```

SHA-256 哈希：    32 B
物理块指针：      8 B
引用计数：        4 B
哈希链表指针：    16 B
填充/对齐：       ~40 B

```

合计： ≈ 100 B

总内存: $268\text{M} \times 100\text{B} \approx 25.6\text{GiB}$
 → 1 TiB 存储需要 25 GiB RAM 仅仅用于去重表
 → 10 TiB 需要 256 GiB RAM → 不现实



图示：块级去重流程。写入时计算块内容的 SHA-256 哈希，在去重表中查找。如果哈希已存在，增加引用计数并返回已有物理块地址——不执行实际写入。新块才真正写入磁盘并添加到去重表。

常见误区

去重表的成本取决于唯一块数量、记录格式、缓存命中率和实现。容量相同的数据集，块越小、唯一块越多，索引记录就越多；索引不能在内存中保持有效工作集时，写入路径会增加持久索引访问。部署前应以目标数据集测量重复率、内存占用、写入放大和恢复时间，不按总容量套用固定内存比例。

最佳用例：VM 镜像（多台虚拟机安装相同操作系统，90%+ 块重复）、容器镜像（共享基础层，层间大量重复块）、备份目标（每日增量备份的快照间重复率极高）。对于这些工作负载，去重可以节省 50-90% 的存储空间，且 DDT 大小可控（因为重复块数量有限）。

预读和预取

当应用程序顺序读取文件时，每次 `read()` 触发一次页缓存缺失 (cache miss)，导致一次磁盘 I/O。如果文件系统能预测应用程序接下来会读哪些页，它可以提前将这些页加载到页缓存中。Linux 内核的 *read-ahead* 机制实现了这一预测。

Linux 的预读算法采用自适应状态机。它跟踪每个文件的访问模式，并据此动态调整预读窗口大小：

=== 顺序读取：预读窗口增长 ===

```

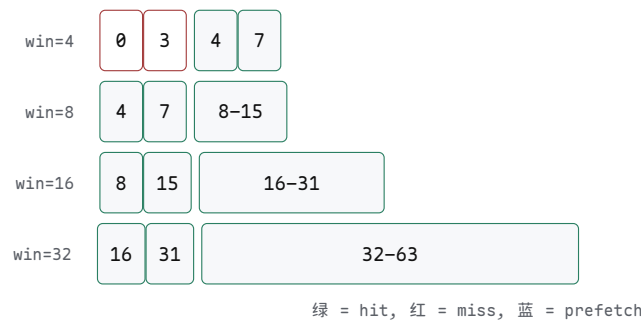
read(0):  cache miss → read pages [0-3],    prefetch [4-7]      win=4
read(4):  cache hit  → win × 2 = 8,         prefetch [8-15]     win=8
read(8):  cache hit  → win × 2 = 16,        prefetch [16-31]    win=16
read(16): cache hit  → win × 2 = 32,        prefetch [32-63]    win=32
read(32): cache hit  → win × 2 = 64,        prefetch [64-127]   win=64
read(64): cache hit  → win × 2 = 128,       prefetch [128-255]  win=128 (cap)
  
```

→ 128 pages = 512 KiB 预读窗口上限 (4 KiB page)

=== 随机回退读取：窗口重置 ===

```

read(0):  cache miss → reset win=4, read [0-3]      win=4
          (检测到回退 → 标记为随机访问 → 停止预读)
  
```



图示：预读窗口增长过程。每次缓存命中（绿色）后，窗口大小翻倍。蓝色区域为预取的页。窗口从 4 页增长到 128 页（512 KiB），在连续命中 5 次后达到上限。

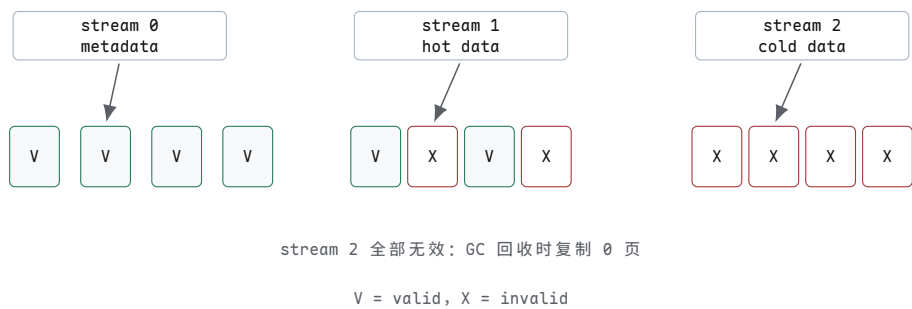
预读算法的关键在于区分顺序访问与随机访问。如果检测到 `lseek()` 回退（当前读取位置在上次预读窗口之前），算法判定为随机访问，将窗口重置为 0（不预读）。这避免了为随机访问工作负载预取无用的页——随机 4 KiB 读取时预读 512 KiB 会浪费 99% 的 I/O 带宽。

多流写入

SSD 的垃圾回收 (garbage collection, GC) 是写放大的主要来源。当 GC 需要回收一个擦除块时，必须先将其中的有效页复制到其他块。如果一个擦除块中混合了长寿命数据（如文件系统元数据）和短寿命数据（如临时文件），GC 在回收短寿命数据的同时被迫复制长寿命数据——这完全是不必要的写放大。

NVMe streams (NVMe 1.3+ 规范) 通过 *Directive* 机制解决了这个问题。SSD 暴露多个流 (stream)，每个流被映射到不同的擦除块组。文件系统将不同生命周期的写入分配到不同的流：

- 流 0：文件系统元数据 (journal, inode table, directory blocks) ——长寿命
- 流 1：热数据（数据库活跃表、日志文件）——中寿命
- 流 2：冷数据/临时数据（日志轮转、临时文件）——短寿命



图示：NVMe 多流写入。不同生命周期的数据分配到不同流，SSD 内部将流映射到不同的擦除块组。当 GC 回收 stream 2 的块时，所有页已无效——零次有效页复制，写放大 = 1.0。

收益：当 GC 回收 stream 2 的擦除块时，由于该流中的数据都是短寿命的，很可能所有页都已经失效——无需复制任何有效页，直接擦除即可。而对于 stream 0 的块，由于元数据是长寿命的，GC 很少需要回收它们。多流将 生命周期相似的数据隔离到不同的擦除块，从根本上减少了 GC 的有效页复制量。

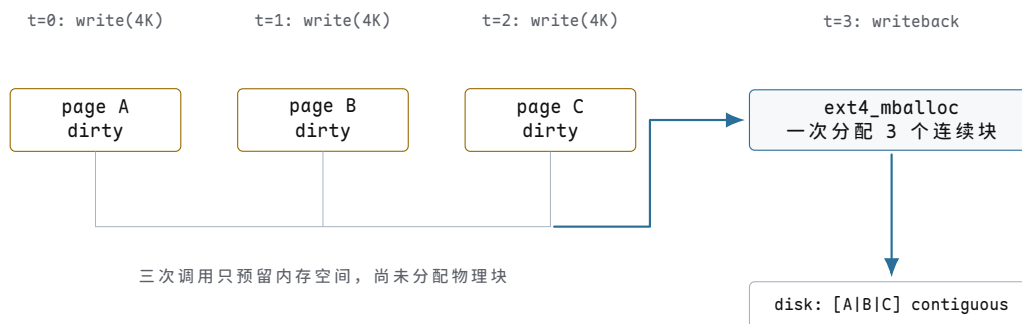
ext4 和 XFS 的多流支持通过 `write_hint` 接口 (`f_hint` 字段在 `struct inode` 中) 实现，传递到块层的 `REQ_OP_WRITE` 请求中，最终通过 NVMe Directive 命令传递到 SSD。

延迟分配深入

传统文件系统在 `write()` 系统调用时立即分配物理块。但这有几个问题：(1) 如果应用程序先写 4 KiB 块 A，再写 4 KiB 块 B（不相邻的偏移量），文件系统可能为 A 和 B 分配不相邻的物理块——即使 B 的写入随后紧随 A。(2) 对于追加写入 (append)，立即分配无法利用未来的写入模式来优化布局。

ext4 的延迟分配 (delayed allocation, `delalloc`) 将物理块分配推迟到写回 (writeback) 时间：

1. `write()` 系统调用：数据进入页缓存，标记为脏页。ext4 在内存中为该 inode 预留所需块数 (per-inode reservation)，但不分配物理块。`write()` 返回成功。
2. 写回触发 (定时器、脏页比例超限、`fsync()`)：ext4 的多块分配器 `ext4_mballoc` 一次性为该 inode 的所有脏页分配物理块。分配器在选择合适的连续区段 (contiguous extent) 时拥有全局视角——它看到所有待写入的页，可以选择最大连续物理区段。
3. 数据写入磁盘：所有脏页按分配的连续物理块顺序写入，最大化 I/O 效率。



图示：延迟分配时间线。三次 `write()` 调用仅在内存中预留空间。写回时，`ext4_mballoc` 一次性为所有脏页分配连续物理块，实现最优布局。

预留与超额预留：每个 inode 按需预留块数，但多个 inode 的预留可能重叠 (即总预留量可能超过实际可用空间)。ext4 使用全局空闲块计数器 (`s_freeclusters_counter`) 作为权威判断：在 `write()` 时检查全局计数器是否有足够空间，但实际分配在写回时才发生。如果写回时发现空间不足 (其他 inode 的写回消耗了空间)，`write()` 已经返回了成功——应用程序会收到一个延迟的 `ENOSPC` 错误。

常见误区

延迟分配的 `ENOSPC` 延迟问题：`write()` 返回成功但写回时空间不足。ext4 通过 `li_get_block_create_hint` 和预留机制尽量避免此问题，但在接近满盘时仍可能发生。应用程序应当检查 `write()` 和 `fsync()` 的返回值——`fsync()` 会冲刷所有延迟分配的块并返回最终的错误状态。

持久内存与 DAX

非易失性内存 (non-volatile memory, NVM) 可以提供字节寻址或可直接映射的持久存储。Intel Optane PMEM (基于 3D XPoint) 是历史上的典型代表，虽然产品已经停产，DAX、缓存行写回和持久化域等概念仍然重要。延迟会随介质、CPU、NUMA 拓扑和访问模式变化，本章不使用固定纳秒数代表整个类别。

DAX 模式 (Direct Access, `CONFIG_FS_DAX`)：当文件系统以 DAX 模式挂载时 (`mount -o dax`)，`mmap()` 系统调用返回的虚拟地址直接映射到 PMEM 的物理地址。没有页缓存 (page cache) 介入，没有块设备层 (block layer) 介入。CPU 的 `load/store` 指令直接访问持久内存。


```

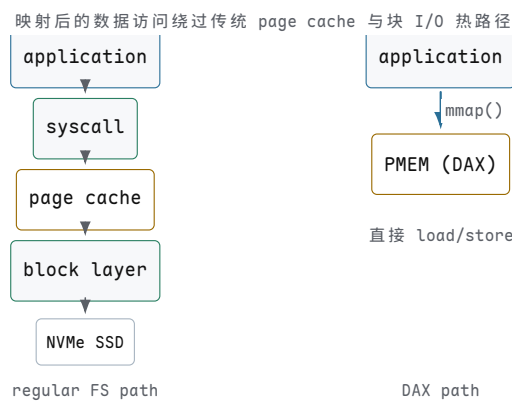
/* PMEM 持久性写入流程 (PMDK libpmem, simplified) */
void pmem_persist(const void *addr, size_t len)
{
    /* 1. 将脏缓存行写回 PMEM */
    pmem_clwb(addr, len);          /* clwb (Cache Line Write Back) */

    /* 2. 确保所有 store 对其他 CPU 和 PMEM 可见 */
    asm volatile("sfence" ::: "memory"); /* sfence (Store Fence) */

    /* 此后 addr 处的数据已持久化到 PMEM。
       断电也不会丢失。 */
}

/* 对比：传统文件系统写持久化 */
void fs_persist(int fd)
{
    write(fd, buf, len); /* 数据进入页缓存 */
    fsync(fd);           /* 刷页缓存到块设备 → I/O syscall */
    /* 需要 syscall 上下文切换 + 块层 + 驱动 */
}

```



图示：DAX 与传统 buffered I/O 数据路径对比。建立映射和处理缺页仍需系统调用、文件系统与驱动参与；映射建立后的普通 load/store 不再为每次访问走传统 page cache 和块 I/O 提交路径。

持久性保证：DAX 模式下，CPU store 指令将数据写入 L1/L2 缓存，然后通过写缓冲 (write buffer) 异步刷新到 PMEM。如果在刷新完成前断电，数据会丢失。必须显式执行 `clwb` (Cache Line Write Back) 将缓存行写回 PMEM，然后执行 `sfence` (Store Fence) 确保所有之前的 store 指令对 PMEM 可见。ext4 和 XFS 在 DAX 模式下的 `fsync()` 会遍历 inode 的脏页，对每页执行 `clwb` + `sfence`。

大页支持

页缓存 (page cache) 传统上使用 4 KiB 页。对于大文件 (例如 10 GiB 数据库)，4 KiB 页意味着 2.5×10^6 个页表项，产生大量 TLB miss。x86-64 的 TLB 容量有限 (L1 dTLB 通常 64-72 项 for 4 KiB 页)，覆盖范围仅 256-288 KiB。

页缓存可以使用包含多个基础页的 large folio，减少逐页管理开销并形成更大的 I/O。这不等同于承诺用户映射一定得到 2 MiB TLB huge-page 映射；页缓存 folio 大小、页表映射粒度和文件系统支持是三个需要分别验证的条件。现代文件系统通常通过 iomap 逐步获得 large-folio 支持，具体能力必须绑定内核版本和文件系统配置。

对于顺序读取 10 GiB 文件：

- 4 KiB 页： 2.5×10^6 个页，TLB miss 频繁

- 2 MiB folio: 只需约 5120 个 folio 对象，显著减少页缓存管理对象数量
- I/O 层面: 较大 folio 有机会形成更大请求，但仍受 extent 连续性、设备限制和预读策略约束

overlayfs 与容器

容器镜像通常由多个层 (layer) 组成: 基础 OS 层 + 应用层 + 依赖层。每个层是一个只读的目录树。容器运行时需要一个可写层来记录修改。OverlayFS 将这些层叠合并为一个统一的目录视图。

- *lowerdir* (下层): 一个或多个只读目录，叠加顺序从下到上
- *upperdir* (上层): 一个可写目录，所有修改写入此层
- *workdir* (工作目录): overlayfs 内部使用，用于 copy-up 原子操作

挂载命令:

```
mount -t overlay overlay -o lowerdir=/lower1:/lower2,upperdir=/upper,workdir=/work /merged
```

copy-up 机制: 当应用程序尝试写入一个位于 lower 层的文件时, overlayfs 不能直接修改只读的 lower 层。它先将文件从 lower 层完整复制到 upper 层, 然后在 upper 层的副本上执行写入。lower 层的原始文件保持不变 (其他容器可能正在使用它)。

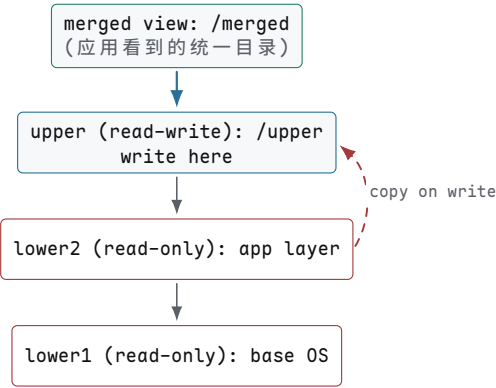
```
=== overlayfs copy-up trace ===

open("/etc/passwd", O_WRONLY)
├─ lookup: /etc/passwd found in lower layer (read-only)
├─ cannot write to lower layer
├─ copy /etc/passwd: lower → upper (full file copy, atomic)
├─ open upper copy for writing
└─ return fd (points to upper copy)

write(fd, "newuser:x:1000:1000::/home/newuser:/bin/bash\n", 47)
└─ writes 47 bytes to upper copy
   lower layer copy is unchanged

close(fd)
└─ upper layer now has modified /etc/passwd

=== 后续读取 ===
open("/etc/passwd", O_RDONLY)
└─ overlayfs checks upper layer first → returns upper copy
   (lower layer shadowed by upper copy)
```



图示：OverlayFS 层叠结构。merged 是应用程序看到的统一视图。upper 是可写层。lower1/lower2 是只读层。copy-up：首次写入 lower 层文件时，文件被复制到 upper 层，后续操作在 upper 副本上进行。

性能注意：copy-up 是同步操作——大文件的首次写入需要先完整复制文件到 upper 层。对于一个 1 GiB 的日志文件，首次 `open(O_WRONLY)` 会触发 1 GiB 的复制 I/O。此外，overlayfs 的目录查找需要在多个层中搜索，增加了 `lookup()` 的开销。

文件系统设计维度与取舍

不同的文件系统做出了不同的设计选择，反映了对工作负载、兼容性和故障模型的取舍。功能开关、容量上限和算法会随版本、格式参数和操作系统集成变化，因此这里比较稳定的设计轴，不把易过期的“yes/no”和最大容量数字伪装成永久事实。

表 14.3：主流文件系统的稳定设计取舍

系统	更新与索引思路	主要优势	边界与代价
ext4	extent 映射，jbd2 日志保护元数据更新	成熟、兼容性广，适合通用 Linux 工作负载	不原生提供 Btrfs/ZFS 式子卷快照和透明压缩
XFS	allocation group、B+ 树和元数据日志	大文件、并行分配和高吞吐扩展性强	管理工具与恢复模型不同于 ext 系列；reflink 不等同于快照产品
Btrfs	数据和元数据采用 COW B 树	校验和、子卷、快照、压缩和 send/receive 集成	重写型负载可能碎片化；defrag 会破坏 extent sharing 并增加空间占用
ZFS	COW 树、事务组、ZIL 与存储池	端到端校验、快照、压缩和冗余修复能力强	内存、运维模型和平台集成成本较高
F2FS	日志结构写入，NAT/SIT、checkpoint 与 GC	面向闪存，支持冷热数据分离和自适应分配	空间压力下前台 GC 会形成尾延迟；不提供原生通用快照
NTFS	MFT、run list 与 \$LogFile 元数据日志	与 Windows 权限、加密和管理生态紧密集成	VSS 是上层卷影复制机制，不能简单等同于文件系统原生快照
APFS	COW 元数据、空间共享、clone 与 snapshot	面向 Apple 平台，集成加密、快照和快速克隆	实现与可用特性受 Apple 系统版本约束，跨平台可观测性有限

设计哲学

ext4：兼容演进是重要约束。ext4 继承 ext2/ext3 的磁盘格式家族，并通过 compatible/incompatible feature flags 区分旧实现能否安全挂载；新建文件系统也可以选择更大的 inode 和新特性。ext4 通过区段树（extent tree）扩展块映

射，通过 jbd2 日志提供崩溃一致性，通过 inline_data 和元数据校验和增量地添加现代特性。这种保守的演进路径使 ext4 成为 Linux 最广泛使用的文件系统，但也限制了它支持快照和透明压缩等需要根本性结构改变的功能。

XFS: 源自 SGI IRIX 系统，设计之初就面向大文件吞吐量。XFS 使用并行分配组 (allocation groups, AG)，每个 AG 有自己的 inode 表和空闲空间 B+ 树，允许多 CPU 并行分配而无需全局锁。XFS 的 B+ 树框架 (xfs_btree) 是一个通用实现——空闲空间、inode 分配、区段映射和大目录都使用同一框架。XFS 不支持快照和压缩，但其对大文件顺序 I/O 的优化和并行性使其在高性能计算和数据库工作负载中表现优异。

Btrfs: Oracle 赞助开发，目标是成为 Linux 的“下一代文件系统”。Btrfs 的核心是 COW B 树——元数据组织在多棵 B 树中，文件数据由 extent item 引用，更新通过 COW 发布新树路径。这使得创建快照通常很快，因为可以共享既有 extent。Btrfs 支持透明压缩、子卷 (subvolume)、快照、发送/接收 (send/receive) 等现代特性。但 COW 的代价是碎片化——频繁修改的文件会被分散到不同位置。Btrfs 支持在已挂载文件系统上整理文件；代价是整理会重写 extent，并可能打破 reflink 或快照之间的 extent sharing，显著增加空间占用。

ZFS: Sun Microsystems (后被 Oracle 收购) 工程团队设计，将数据完整性置于一切之上。ZFS 的设计原则是“永不静默损坏”：每个块 (数据和元数据) 都有校验和，每次读取都验证。ZFS 的 COW + intent log (ZIL) 提供了强一致性保证。ZFS 的存储池 (zpool) 模型将物理设备与文件系统解耦——多个文件系统可以共享一个池的存储空间。ZFS 支持透明压缩、去重、快照、克隆和纠错 (RAID-Z)。代价是复杂性和内存消耗 (ARC 缓存通常占用大量 RAM)。

F2FS: Samsung 开发，专门为 NAND 闪存优化，主要目标是 Android 设备。F2FS 的设计围绕闪存的物理特性：日志结构写入 (log-structured write) 将随机写转换为顺序写，减少闪存擦除次数；冷热数据分离 (warm/cold separation) 将不同生命周期的数据写入不同区域，减少 GC 开销。后台 GC 可以在 I/O 空闲时清理；空间压力触发的前台 GC 则可能直接增加应用延迟。F2FS 不提供原生通用快照和去重，其目标是在闪存上平衡写放大、吞吐和尾延迟。

NTFS: Microsoft 为 Windows NT 设计，使用 \$LogFile 日志保证元数据一致性。小文件数据可以作为 resident attribute 保存在 MFT 记录中，大文件则用 run list 描述簇范围。NTFS 支持每文件压缩和 EFS 加密；Windows 的 VSS 与服务器数据去重属于更大的存储栈，讨论时不能直接当成 NTFS 磁盘格式本身的能力。

APFS: Apple 为 macOS/iOS 设计，2017 年推出。APFS 针对闪存优化——所有写入都是 COW，原生支持 TRIM，并为 SSD 优化了空间分配。APFS 的快照可被 Time Machine 等上层功能使用。APFS 原生支持加密、snapshot 和 clone；clone 利用共享 extent 避免创建时复制全部数据，但后续写入仍会分配新空间，不能把它理解成永远零成本。

核心思想

没有“最好的”文件系统，只有不同约束下的权衡。ext4 追求稳定兼容，XFS 追求大文件吞吐，Btrfs 追求功能丰富，ZFS 追求数据完整性，F2FS 追求闪存友好，NTFS 追求 Windows 生态，APFS 追求 Apple 生态。选择取决于工作负载、硬件平台和可靠性要求。

结构不变量与崩溃恢复检验

文件系统的正确性和性能不能仅靠代码审查保证。一个文件系统必须在各种异常条件下经过系统性测试——包括崩溃恢复、并发竞争、空间碎片和闪存磨损。以下测试分类和具体场景构成了一个文件系统验证的最小集合。

测试分类

验收标准

Correctness (正确性)

- 基本 API 正确性: `create`、`unlink`、`rename`、`link`、`fsync`、`fdatasync` 在所有文件类型上的行为
- 边界条件: 空文件、最大文件、路径长度上限、文件名特殊字符
- 崩溃恢复: 在每个操作序列的每个中间状态下断电, 验证恢复后状态

验收标准

Performance (性能)

- 顺序读/写吞吐量: 1 MiB 块大小, 测量 MB/s 和 IOPS
- 随机读/写吞吐量: 4 KiB 块大小, 队列深度 1/16/64
- 不同块大小 (512 B - 1 MiB) 下的吞吐量曲线
- 元数据操作延迟: `stat`、`open`、`readdir` 的 p99 延迟

验收标准

Scalability (可扩展性)

- 单目录 1M 文件: `readdir`、`lookup`、`unlink` 性能
- 10 TiB 单文件: 顺序读写性能不随偏移量退化
- 100 万并发文件描述符: 内核内存消耗和调度延迟

验收标准

Crash Recovery (崩溃恢复)

- 故障注入: 在每个操作 (`create`、`write`、`fsync`、`rename`、`unlink`) 的每个步骤后断电
- 日志恢复: 验证 jbd2/ZIL/checkpoint 在各种中断场景下能正确恢复
- 校验和验证: 注入位翻转后, 文件系统是否能检测并报告损坏

验收标准

Concurrency (并发)

- 100 个进程在同一目录中并发创建文件—无数据竞争, 无死锁
- 并发 `rename` 到同一目标路径—最终状态一致
- 并发 `fsync` 与 `write`—崩溃后数据状态可预测

验收标准

Space Efficiency (空间效率)

- 1M 次 `create` + `delete` 循环后测量碎片化程度

- 填充至 99% 后随机 create/delete—验证分配器在满盘时的退化
- 空闲空间报告准确性: `statfs()` 报告的空闲块数与实际一致

验收标准

Flash Optimization (闪存优化)

- 写放大测量: 写入 100 GiB 数据, 测量 SSD 实际闪存写入量
- TRIM 有效性: 删除大量文件后验证 SSD 回收了擦除块
- 对齐: 验证文件系统块大小与 SSD 页大小对齐

具体测试场景

以下场景覆盖了最常见的文件系统故障模式:

1. **崩溃在同步之前:** 创建并写文件 → 在任意步骤崩溃 → 恢复。未建立持久化承诺时, 文件可能不存在, 也可能以某个已提交状态存在; 测试应验证文件系统结构合法、旧状态仍可解释, 并且不会暴露越界块、悬空目录项或未初始化数据, 而不是强行要求“文件不应存在”。
2. **新文件的持久发布:** 创建并写临时文件 → `fsync(file)` → `rename` 到目标名 → `fsync(parent_dir)` → 崩溃 → 恢复 → 文件名和正确数据都必须存在。只同步文件不能单独保证包含它的目录项已经持久化。
3. **每 4K fsync 的吞吐量:** 顺序写入 1 MiB, 每 4 KiB 后调用 `fsync` → 测量吞吐量和 p50/p95/p99 延迟。这是数据库 WAL 的典型模式。报告必须记录设备型号、缓存与断电保护、文件系统版本、挂载参数、队列深度和测试工具; 没有这些条件, 不给固定“预期 IOPS”。
4. **并发创建:** 100 个进程各在同一目录中创建 1000 个文件 → 无文件丢失、无数据竞争、无死锁。验证目录锁的正确性。
5. **满盘碎片化:** 填充至 99% → 随机 create/delete 10000 次 → 测量碎片化 (最大连续空闲区段 vs 总空闲空间)。验证分配器不会退化到无法分配连续区段。
6. **SSD 写放大:** 在 SSD 上写入 100 GiB → 读取 SSD SMART `Media_Wearout_Indicator` 或厂商提供的 NAND 写入计数 → 计算实际闪存写入量与逻辑写入量之比。不同 SSD 对 SMART 字段的定义不同, 必须先验证计数单位, 并把文件系统写放大与设备 FTL 写放大分开报告。
7. **快照一致性:** 创建快照 → 修改 10% 的数据 → 验证快照仍然保存原始数据 → 删除快照 → 验证空间被正确回收。
8. **累积崩溃:** 在持续写入工作负载下, 执行 1000 次随机断电 → 每次恢复后验证文件系统一致性 → 无累积损坏。验证日志恢复是幂等的。

关键机制的运行路径

下面通过经过删减的 Linux 关键函数说明路径查找、区段映射、日志提交、写回和目录项缓存之间的控制关系。代码保留影响机制理解的分支和状态转换, 省略与主题无关的参数处理及平台细节。

Linux VFS 核心路径

路径查找 (path lookup) 是文件系统最频繁的操作—每次 `open()`、`stat()`、`read()` 都需要将路径名解析为 inode。Linux VFS 通过 `path_lookupat` 组织这条查找路径。

路径查找有两条路径：*RCU walk*（无锁快速路径）和 *ref walk*（引用计数慢速路径）。RCU walk 使用 Read-Copy-Update 机制在无锁状态下遍历 dentry 和 inode——这是 *stat* 和 *open* 热路径上的常见情况（缓存命中）。当遇到复杂情况（跨挂载点、符号链接、dentry 缺失、竞争检测到修改）时，RCU walk 回退到 ref walk，后者使用 *d_lock* 和 *i_rwsem* 保证安全。

```
/* VFS path lookup (simplified) */
static int path_lookupat(struct nameidata *nd,
                        unsigned flags, struct path *path)
{
    /* Phase 1: try RCU walk (lockless, fast path) */
    rcu_read_lock();
    nd->flags |= LOOKUP_RCU;
    err = link_path_walk(nd);          /* walk path components */
    if (!err && can_lookup_fast(nd))
        err = walk_component(nd);      /* try dcache hit */
    rcu_read_unlock();

    /* Phase 2: fallback to ref walk on contention */
    if (err == -ECHILD || err == -ESTALE) {
        err = link_path_walk(nd);      /* ref walk: d_lock + i_rwsem */
    }
    return err;
}

/* walk_component: fast path vs slow path */
static int walk_component(struct nameidata *nd)
{
    /* lookup_fast: dcache hit (RCU-protected) */
    dentry = lookup_fast(nd);          /* d_lookup in dcache */
    if (dentry)
        return step_into(nd, dentry); /* consume one component */

    /* lookup_slow: dcache miss → call filesystem */
    dentry = lookup_slow(nd);          /* inode->i_op->lookup() */
    return step_into(nd, dentry);
}
```

RCU walk 的性能优势在于完全避免了原子操作和缓存行争用。在高度并行的场景下（例如 Web 服务器处理数千并发请求），RCU walk 能显著减少共享锁和引用计数争用。不过路径遍历仍与路径分量数量相关，并没有从算法上的 $O(n)$ 变成 $O(1)$ ；改善的是并发扩展性和常数开销。

ext4 extent 查找

ext4 使用 *ext4_ext_map_blocks* 查找区段树（extent tree）。给定一个逻辑块号，该函数在 B+ 树中查找对应的物理块号。

```
/* ext4 extent lookup (simplified) */
int ext4_ext_map_blocks(handle_t *handle, struct inode *inode,
                        struct ext4_map_blocks *map, int flags)
{
    struct ext4_extent_header *eh;
    struct ext4_extent *ex;
    ext4_lblk_t block = map->m_lblk; /* logical block number */

    /* Start from inline header in inode */
    eh = ext_inode_hdr(inode); /* i_block[0] as extent header */

    /* Traverse B+ tree: descend from root to leaf */
}
```

```

while (eh->eh_depth > 0) {
    /* Index node: find correct child pointer */
    struct ext4_extent_idx *idx;
    idx = ext4_ext_find_index(eh, block);
    bh = sb_bread(sb, ext4_idx_pblock(idx)); /* read child */
    eh = ext_block_hdr(bh);
}

/* Leaf level: search for extent covering 'block' */
ex = ext4_ext_search_right(eh, block);
if (ex == NULL) {
    /* No extent covers this range → it's a hole */
    map->m_flags |= EXT4_MAP_HOLE;
    map->m_pblk = 0; /* zero-filled page on read */
    return 0;
}

/* Found: compute physical block */
map->m_pblk = ext4_ext_pblock(ex) + (block - ex->ee_block);
map->m_len = ext4_ext_get_actual_len(ex)
            - (block - ex->ee_block);
return map->m_len;
}

```

空洞 (*hole*) 处理：如果逻辑块号不在任何 extent 的覆盖范围内，`ext4_ext_map_blocks` 返回 `EXT4_MAP_HOLE` 标志。VFS 层据此返回一个全零页—读取空洞区域得到零字节，不需要分配物理块。

jbd2 提交

ext4 的日志由 jbd2 (Journaling Block Device 2) 驱动。`jbd2_journal_commit_transaction` 将内存中的事务，也就是一组已经登记的元数据修改，提交到磁盘日志区域。

```

/* jbd2 transaction commit (simplified) */
int jbd2_journal_commit_transaction(journal_t *journal)
{
    transaction_t *commit_txn = journal->j_running_transaction;

    /* Phase 1: write all descriptors and data buffers */
    /* For each journal block: write to log area */
    err = journal_write_data_buffers(commit_txn);

    /* Phase 2: write commit block */
    commit_blk = jbd2_journal_get_descriptor(journal);
    commit_blk->h_magic = JBD2_MAGIC_NUMBER;
    commit_blk->h_blocktype = JBD2_COMMIT_BLOCK;

    /* CRC32C checksum of all blocks in this transaction */
    commit_blk->h_chksum_type = JBD2_CRC32C;
    commit_blk->h_chksum[0] = crc32c_journal(commit_txn);

    /* Write commit block → this is the atomic commit point */
    submit_bio(WRITE, commit_bio);
    wait_for_completion(); /* commit record is now durable */

    /* Phase 3: checkpoint */
    /* Move committed data from log to final locations,
       then free journal space for reuse */
    jbd2_journal_checkpoint_transaction(journal);
}

```

```
}
```

提交块 (commit block) 是事务的原子提交点——只有在提交块成功写入后，事务才算“已提交”。如果在写入提交块之前崩溃，恢复时该事务被视为未提交，所有相关修改回滚。提交块中的 CRC32C 校验和用于在恢复时验证提交块本身没有损坏。

XFS B+ 树框架

XFS 的核心索引结构共用一套 B+ 树框架。这个框架提供查找、插入和删除的通用实现，具体的数据结构再通过函数指针定制键值提取和比较规则。

```
/* XFS generic B+ tree lookup (simplified) */
int xfs_btree_lookup(struct xfs_btree_cur *cur,
                    xfs_lookup_t dir, int *stat)
{
    /* Traverse from root to leaf, comparing keys */
    for (level = cur->bc_nlevels - 1; level >= 0; level--) {
        /* Binary search within node */
        xfs_btree_binary_search(cur, level, &key, &ptr);
        if (ptr > xfs_btree_get_numrecs(block))
            ptr = xfs_btree_get_numrecs(block);
        if (level > 0)
            block = xfs_btree_read_block(cur, ptr); /* descend */
    }
    *stat = (found_exact_match) ? 1 : 0;
    return 0;
}

/* Users of the generic framework:
 * - xfs_btree_cur for free space by block number (AGFL)
 * - xfs_btree_cur for free space by block count (AGFL)
 * - xfs_btree_cur for inode allocation (AGI)
 * - xfs_btree_cur for extent map (BMBT)
 * - xfs_btree_cur for directory (large dirs) (BMDA)
 */
```

所有 XFS 的 B+ 树 (空闲空间索引、inode 分配、区段映射、大目录) 都通过 `struct xfs_btree_cur` 配置不同的比较函数和键值类型，共享同一套遍历、插入和删除代码。这种设计避免了代码重复，也保证了所有 B+ 树操作的一致性。

Btrfs 事务提交

Btrfs 的 `btrfs_commit_transaction` 使用 COW 树、generation、校验和与超级块提交协议维护一致状态。它没有 `ext4/jbd2` 那样的块日志，但存在专门的 `tree-log`，用于在完整事务提交前记录 `fsync` 等需要持久化的更新；不能概括成“B 树本身就是全部日志”。

```
/* Btrfs transaction commit (simplified) */
int btrfs_commit_transaction(struct btrfs_trans_handle *trans)
{
    struct btrfs_transaction *cur = trans->transaction;

    /* Phase 1: COW all modified B-tree nodes */
    /* Each modified node is written to a NEW disk location.
     * Old locations remain intact until superbblock is updated. */
    btrfs_write_dirty_block_groups(trans);
    btrfs_write_dirty_roots(trans); /* COW B-tree nodes */

    /* Phase 2: write superbblock with new root pointers */
    /* Write checksummed superbblock mirrors carrying new generations. */
    btrfs_write_super(trans);
```

```

/* Phase 3: mark old data as reclaimable */
/* Old B-tree nodes are no longer referenced.
   Future writes can reuse those disk blocks. */
btrfs_run_delayed_refs(trans);
}

/* Two ways to join a transaction:
 * btrfs_join_transaction: read-only join, does NOT force commit
 * btrfs_start_transaction: write join, MAY force commit if
 *                           transaction is large or old
 */

```

Btrfs 在固定物理位置保存超级块镜像，超级块带 checksum 和 generation，并指向各棵树的已提交根。恢复不能假定“任意单扇区写天然原子”，而要验证超级块及其引用树是否属于完整代数。旧块只在仍被已提交树或快照引用时保留；引用解除并跨过安全事务边界后，空间最终可以复用，所以“旧数据永远不覆盖”也不是永久承诺。

page cache 与写回

Linux 的脏页管理分为全局调节和文件系统写回两个层次：前者计算全局脏页阈值并实施写回节流，后者组织写回线程的主循环和逐 inode 的写回逻辑。

```

/* global dirty-page throttling (simplified) */
/* vm_dirty_ratio (default 20%): max dirty pages as % of memory */
/* vm_dirty_background_ratio (default 10%): when writeback starts */

void global_dirty_limits(unsigned long *pbackground,
                        unsigned long *pdirty)
{
    unsigned long total = global_page_state(NR_FREE_PAGES)
                        + global_reclaimable_pages();
    *pbackground = total * vm_dirty_background_ratio / 100;
    *pdirty      = total * vm_dirty_ratio / 100;
    /* When dirty pages > pdirty: new write() blocks
       When dirty pages > pbackground: writeback thread wakes */
}

/* filesystem writeback thread (simplified) */
void wb_workfn(struct work_struct *work)
{
    while (!list_empty(&bdi->work_list)) {
        /* Pop next inode to write back */
        inode = wb_writeback_single(bdi, &wbc);
        __writeback_single_inode(inode, &wbc);
        /* Calls mapping->a_ops->writepages() → filesystem writeback
           which calls ext4_mballocc to allocate blocks
           for all dirty pages at once (delayed alloc!) */
    }
}

```

`__writeback_single_inode` 通过 `mapping->a_ops->writepages` 进入文件系统写回路径，后者负责将脏 folio 变成块映射与 I/O。对于延迟分配的文件系统，`writepages` 是实际物理块分配发生的时刻——多块分配器 `ext4_mballocc` 在此处一次性为所有脏页分配连续物理块。

dcache RCU 遍历

目录项缓存 (dcache) 保存路径分量到 inode 的查找结果。dcache 的查找使用 RCU 实现无锁快速路径。

```
/* dcache lookup (simplified) */
struct dentry *d_lookup(const struct dentry *parent,
                        const struct qstr *name)
{
    /* RCU fast path: lockless dcache lookup */
    rcu_read_lock();
    hlist = rcu_dereference(
        parent->d_flags & DCACHE_OP_COMPARE
        ? d_compare_rcu : d_hash_rcu);
    hlist_for_each_entry_rcu(dentry, hlist, d_hash) {
        if (dentry->d_parent != parent) continue;
        if (qstr_casecmp(&dentry->d_name, name)) continue;
        if (d_unhashed(dentry)) continue; /* being removed */
        /* Hit! But must verify under RCU */
        if (lockref_get_not_dead(&dentry->d_lockref)) {
            rcu_read_unlock();
            return dentry; /* cache hit */
        }
    }
    rcu_read_unlock();
    return NULL; /* cache miss → call filesystem lookup */
}
```

RCU walk 依次调用 `link_path_walk`、`walk_component`、`lookup_fast` 和 `d_lookup`。整个路径在 `rcu_read_lock()` 保护下执行，不持有任何自旋锁。当 `d_lookup` 返回 `NULL` (缓存缺失) 或检测到 `dentry` 正在被删除时，VFS 回退到 `lookup_slow`，后者获取 `i_rwsem` 信号量并调用文件系统的 `i_op->lookup` 方法。

应用层一致性不能外包给文件系统

文件系统的日志保证的是 文件系统自身元数据的一致性——inode 表、目录项、空闲块位图不会因为崩溃而损坏。但文件系统 不保证应用层面的多文件事务一致性。如果一个应用需要原子地更新两个文件 (例如：写入数据文件 A 和更新指向 A 的索引文件 B)，文件系统的日志无法帮助——每个文件系统操作 (`write`、`fsync`) 是独立的，崩溃可能发生在两者之间。

应用必须自行实现原子性。清单发布协议 (manifest publish protocol) 是一种常用模式，利用 `rename` 的原子性作为提交点：

```
/* Manifest publish protocol - atomic multi-file update */

/* Step 1: Write data object to a unique, versioned path */
int fd = open("objects/v42/data", O_WRONLY|O_CREAT|O_EXCL, 0644);
write_all(fd, data, data_len);
fsync(fd); /* persist data to disk */
close(fd);

/* Step 2: Write manifest referencing the object */
fd = open("manifests/v42", O_WRONLY|O_CREAT|O_EXCL, 0644);
write_all(fd, manifest_content); /* includes checksums, metadata */
fsync(fd); /* persist manifest */
close(fd);

/* Step 3: Atomic publish via rename */
rename("manifests/v42", "CURRENT");
/* rename is atomic: either readers see old CURRENT or new CURRENT,
```

```

    never an intermediate state */

/* Step 4: Persist the directory entry change */
int dirfd = open(".", O_RDONLY|O_DIRECTORY);
fsync(dirfd);           /* ensure rename is durable */
close(dirfd);

/* Readers: always read CURRENT, which points to a consistent
   manifest+data pair. No partial state is ever visible. */

```

核心思想

不同层次的一致性由不同的日志机制保护，各司其职，不可混淆：

1. 文件系统日志（jbd2/ZIL/Btrfs COW）保护文件系统结构的一致性——inode、目录项、位图不会损坏。
2. 数据库 WAL（Write-Ahead Log）保护数据库事务的一致性——事务要么全部提交，要么全部回滚。
3. 应用清单（manifest + atomic rename）保护应用版本的一致性——读者要么看到旧版本，要么看到新版本，不会看到中间状态。

文件系统的崩溃一致性保证止步于元数据。应用程序的跨文件原子性是应用自身的责任——不能外包给文件系统。

第 15 章 为什么网络通信需要持久端点

继承的机器。内核已经能把异步设备请求组织成长期对象，也能用 fd 引用文件。单个网络数据报仍不能表达长期会话：地址由谁占用、连接处于哪个阶段、发送者何时需要背压、接收者等待什么条件、关闭后迟到的数据属于谁，都需要独立保存。

网络端点为何需要内核对象

先设想两个进程不在同一台机器上。共享内存和普通文件描述符不能直接跨越机器边界，发送方只能把字节交给网络协议，接收方则在稍后某个时刻得到这些字节或错误。操作系统必须记住：这次通信使用哪个协议，本地与远端是谁，尚未发送和尚未读取的数据在哪里，线程正在等待什么条件，以及连接关闭或网络失败后怎样报告结果。

这些状态不能只保存在一次 `send` 调用的栈上。调用返回后，数据可能仍在排队或等待重传；进程也可能先登记等待，再由中断和协议处理在未来唤醒。因此内核需要一个具有引用计数、协议状态、收发队列和等待队列的持久运行时对象。进程仍需要用整数句柄引用它，以复用已有的关闭、继承、阻塞和事件等待规则。由这条需求链才得到套接字对象以及指向它的 `socket fd`。

网络协议会在后续卷册继续展开。本章只讨论操作系统必须提供的网络入口：`socket` 为什么进入 `fd` 模型，连接和监听如何形成不同内核对象，阻塞和非阻塞 I/O 如何与调度器协作，以及 `poll/epoll` 为什么不仅是“循环 `read`”的语法变化。

`socket` 与普通文件的差异在于它不是稳定字节数组，而是协议状态机和队列集合。TCP `socket` 至少包含发送缓冲、接收缓冲、连接状态、重传计时器、拥塞控制状态和等待队列。UDP `socket` 没有连接字节流，但有端口绑定、报文边界和接收队列。应用看到的是 `send/recv`，内核维护的是协议栈状态、网卡队列和计时器。

核心思想

`socket` 是 `fd` 表中的一个内核对象，但它的可读可写由协议状态、缓冲区、水位线和错误队列共同决定。

网络入口必须先把 OS 契约讲清楚，再进入协议细节。阻塞 `recv` 在没有数据时会让线程睡在 `socket` 等待队列；非阻塞 `recv` 返回 `EAGAIN`；`poll` 把“是否可读可写”注册为等待条件；`epoll` 用就绪队列降低大量 `fd` 的重复扫描成本。这些都是调度、等待队列和 `fd` 生命周期的延伸。

`bind`、`listen`、`accept` 与 `connect`

最小 `socket` 子系统可以包含：

对象	作用	不变量
<code>socket fd</code>	进程引用网络端点的句柄	<code>fd</code> 引用有效时 <code>socket refcount</code> 大于零
<code>socket object</code>	保存协议族、类型、状态、ops	状态决定允许的系统调用
<code>bind table</code>	本地地址端口到 <code>socket</code> 的映射	同一四元组不能被非法重复绑定
<code>listen queue</code>	半连接和已完成连接队列	<code>backlog</code> 限制必须生效

send buffer	用户写入但未发送或未确认的数据	字节数受 SO_SNDBUF 限制
receive buffer	已到达但未被用户读取的数据	可读事件与缓冲水位一致

典型 TCP 服务端路径：

```
s = socket(AF_INET, SOCK_STREAM)
bind(s, 0.0.0.0:8080)
listen(s, backlog=128)
while true:
    c = accept(s)
    handle_client(c)
```

内核对应状态是：`socket` 创建未绑定对象；`bind` 加入端口表；`listen` 把对象转成监听状态并初始化队列；三次握手完成后，新连接进入 `accept queue`；`accept` 取出一个已完成连接，创建新的 `fd` 指向新的 `connected socket`。监听 `socket` 和连接 `socket` 不是同一个对象。

发送缓冲、接收缓冲与背压

`bind` 冲突检查

端口绑定要考虑地址、端口、协议、`reuse` 标志和 `namespace`。教学模型可以先实现严格唯一：

```
bind(sock, local_addr):
    lock(bind_table.lock)
    if bind_table.contains(sock.netns, sock.proto, local_addr):
        unlock(bind_table.lock)
        return EADDRINUSE
    sock.local = local_addr
    sock.state = BOUND
    bind_table.insert(sock)
    unlock(bind_table.lock)
    return OK
```

真实系统的 `SO_REUSEADDR` 和 `SO_REUSEPORT` 会放宽规则，但不能简化成“允许重复”。复用规则还要区分监听 `socket`、已连接四元组、`TIME_WAIT` 和负载均衡策略。先掌握严格唯一模型，再扩展复用规则，能避免把安全边界讲错。

`listen` 与 `accept` 队列

监听 `socket` 有两个不同队列：握手未完成的半连接队列，以及握手完成等待用户 `accept` 的队列。简化模型可以只保留完成队列：

```
tcp_connection_established(listener, child):
    lock(listener.lock)
    if listener.acceptq.len == listener.backlog:
        drop_or_reset(child)
    else:
        enqueue(listener.acceptq, child)
        wake_waiters(listener.read_waitq)
    unlock(listener.lock)

accept(listener):
    lock(listener.lock)
    while empty(listener.acceptq):
        if listener.nonblock:
```

```

    unlock(listener.lock)
    return EAGAIN
    sleep_on(listener.read_waitq, listener.lock)
    child = dequeue(listener.acceptq)
    fd = install_fd(child)
    unlock(listener.lock)
    return fd

```

不变量是: accept queue 中的 child socket 已经是 connected 状态; 队列非空时监听 socket 对 poll 表现为可读; 关闭监听 socket 必须唤醒等待 accept 的线程并让它们返回错误。

send/recv 与背压

发送路径不是把用户数据直接交给网卡。数据先进入 socket send buffer, 再由协议栈分段、排队、重传和确认。

```

send(sock, user_buf, len):
    copied = 0
    lock(sock.lock)
    while copied < len:
        space = sock.sndbuf_limit - sock.sndbuf_used
        if space == 0:
            if sock.nonblock:
                break_or_EAGAIN(copied)
            sleep_on(sock.write_waitq, sock.lock)
            continue
        n = min(space, len - copied)
        copy_from_user_into_skb(sock, user_buf + copied, n)
        copied += n
        tcp_push_pending(sock)
    unlock(sock.lock)
    return copied

```

这段伪代码解释短写: 阻塞 socket 通常尽量等待空间, 非阻塞 socket 在缓冲区满时可能只写入部分数据或返回 EAGAIN。接收路径类似: 接收缓冲为空时阻塞或返回 EAGAIN; 对 TCP 返回字节流, 对 UDP 每次读取保留报文边界语义。

背压要从两端同时看。发送方应用调用 send(sock, buf, 1MiB), 内核不可能把 1MiB 立即塞进网卡。它先受本机 socket send buffer 限制; 再受 TCP 拥塞窗口和接收方通告窗口限制; 再受 qdisc、驱动队列和网卡 DMA ring 限制。任一层满了, send 都可能停住、短写或返回 EAGAIN。

队列或窗口	它限制什么	满了以后会怎样
socket sndbuf	应用已交给内核但尚未确认释放的字节	阻塞 send、短写或 EAGAIN
TCP send queue	已分段、待发送、待重传的数据	受拥塞控制和对端窗口约束
qdisc 队列	本机网络调度层待发送 packet	排队延迟增加, 可能丢包或限速
驱动 TX ring	网卡可消费的 DMA 描述符槽位	停止网络队列, 等 completion 回收槽位
对端 receive window	接收方还能缓存多少未读数据	发送方必须减速, 甚至进入零窗口等待

一个常见现象是: 程序以为 send 返回成功就表示对方收到了消息。实际并不是。返回成功通常只说明数据已经被本机内核接收进入发送路径, 后续还可能因为

连接 reset、重传失败、超时或本机崩溃而没有到达对端应用。若业务需要“对方已经处理”，必须在应用协议里设计确认，而不是把 socket 写入成功当业务提交点。

接收方向也一样。网卡收到包后，驱动把 skb 交给协议栈，TCP 按序重组后放进 socket receive queue。应用不读，receive queue 会涨满；涨满后接收窗口变小，发送方会被迫降速。若应用只看“网络带宽不够”，就会误诊。真正问题可能是本进程处理慢、单线程事件循环被阻塞、receive buffer 太小，或者应用协议把消息边界处理错。

poll 与 epoll

朴素 poll 每次扫描 fd 数组，复杂度 $O(n)$ ：

```
poll(fds, timeout):
    ready = []
    for fd in fds:
        mask = fd.file.ops.poll(fd.file)
        if mask != 0:
            ready.append((fd, mask))
    if ready not empty:
        return ready
    register_waiters(fds, current)
    schedule_timeout(timeout)
    retry scan
```

epoll 把关注集合保存在内核中。当 socket 状态变化时，回调把对应 epitem 放入 ready list，用户调用 epoll_wait 主要消费 ready list。这样大量空闲连接不会每次都被全量扫描。

```
socket_state_change(sock):
    for epitem in sock.poll_subscribers:
        if sock.poll_mask & epitem.interest:
            enqueue_once(epitem.epoll.ready_list, epitem)
            wake_waiters(epitem.epoll.waitq)
```

边沿触发和水平触发的区别也来自 ready 条件。水平触发下，只要缓冲区仍有数据，下一次等待还会报告；边沿触发只在状态从不可读变可读时报告，用户必须一直读到 EAGAIN，否则可能再也收不到事件。

用一个具体 bug 看边沿触发为什么容易错。socket S 原来不可读，随后内核收到 100 字节，状态从不可读变为可读，epoll edge-triggered 把 S 放入 ready list。应用从 epoll_wait 取到事件，只调用一次 recv(S, buf, 20)，读走 20 字节后就回到等待。此时 socket 里还剩 80 字节，但状态没有从“不可读”再次变成“可读”，所以边沿触发可能不会再报告。应用就把 80 字节卡在内核 receive queue 里。

正确写法是：收到边沿事件后，对非阻塞 fd 循环读取，直到 recv 返回 EAGAIN。这个 EAGAIN 不是错误，而是证明“当前 ready 条件已经被消费干净”。写方向也类似，收到可写事件后应尽量写到 EAGAIN 或应用输出队列为空。

```
on_epollin_edge(fd):
    while true:
        n = recv(fd, buf, sizeof(buf), NONBLOCK)
        if n > 0:
            append_to_protocol_parser(buf, n)
            continue
        if n == 0:
            close_connection(fd)
            break
        if errno == EAGAIN:
            break
```

```

if errno == EINTR:
    continue
handle_error(fd, errno)
break

```

epoll 还要求应用区分“事件身份”和“fd 数字”。线程 A 把 fd=7 加入 epoll, 线程 B 关闭 fd=7, 线程 C 立即打开新连接又得到 fd=7。旧 epoll item 在内核里绑定的是旧 file, 不是新连接; 但事件返回给用户时可能仍带着当初保存的数字 7。健壮事件循环通常把 `data.ptr` 指向连接对象, 并给对象加 generation 或关闭标记, 避免用裸 fd 数字误操作新连接。

TCP 状态边界

操作系统教材不必在本册完整推导 TCP, 但要能解释 socket 系统调用面对的状态:

状态	用户可见行为	内核重点
LISTEN	可 accept, 不能普通 recv 数据	accept queue、backlog、SYN 处理
SYN-SENT	connect 进行中	超时、错误返回、非阻塞完成事件
ESTABLISHED	可 send/recv	缓冲、重传、拥塞控制、窗口
半关闭	FIN-WAIT 或 CLOSE-WAIT	EOF、shutdown、剩余数据读取
TIME-WAIT	fd 通常已关但四元组保留	防旧包污染新连接

连接关闭、错误与重试边界

1. fd 生命周期: dup socket 后关闭一个 fd, 连接对象不能提前释放。
2. bind 冲突: 同一地址端口重复绑定应失败, 释放后可再次绑定。
3. listen backlog: 超过 backlog 的完成连接必须被拒绝或延迟, 不能无限增长。
4. accept 阻塞: 无连接时阻塞, 有连接到达后唤醒; 非阻塞返回 `EAGAIN`。
5. send 背压: 发送缓冲满时阻塞或短写, 空间释放后唤醒写等待者。
6. recv EOF: 对端关闭后, 已缓冲数据读完再返回 EOF。
7. poll 水平触发: 缓冲仍有数据时重复报告可读。
8. epoll 边沿触发: 未读尽数据时不会凭空产生新边沿事件。

验收标准

理解网络入口的判断基准是: 能够说明 socket 为什么由 fd 引用, 画出 bind/listen/accept/connect/send/recv 引起的对象变化, 并解释阻塞、非阻塞、poll 与 epoll 各自的等待语义。

传输完成与业务完成之间的边界

网络补充材料可以继续展开路由、Netfilter、NAPI、skb、拥塞控制和持久化服务协议。主线里先守住一个边界: TCP 的可靠字节流不等于业务请求可靠提交。操作系统能告诉你连接状态、字节是否进入本机内核、对端是否关闭、socket 错误是什么; 它不能替应用判断“远端服务是否已经执行这笔订单、写入这条记录或只执行了一半”。

现象	OS/协议层含义	应用层仍要处理什么
<code>send</code> 返回正数	字节被本机发送路径接收	对端应用未必收到或处理
<code>recv</code> 返回 0	对端有序关闭且缓冲已读完	未完成请求可能成功也可能失败
<code>ECONNRESET</code>	连接被复位，流状态不再可信	当前业务操作结果未知
超时	等待期限内未得到事件	远端可能稍后处理或已处理但响应丢失
重连成功	新 TCP 流建立	旧流上的请求语义不能自动继承

可靠服务要在 TCP 之上建立自己的提交证据。常见方法包括请求 ID、幂等键、服务端去重表、事务日志、单调版本、应用级 ACK 和恢复查询。客户端超时后重试同一个请求 ID，服务端若发现已经提交，就返回旧结果；若未提交，才执行并记录。这样网络断开、进程崩溃、重复发送和响应丢失都能收敛到同一条业务语义，而不是把 socket 错误码当作业务真相。

```
server_handle(req):
    if committed_log.contains(req.id):
        return committed_log.result(req.id)
    result = execute_transaction(req.payload)
    committed_log.append(req.id, result)
    fsync(committed_log)
    return result
```

这个模型也解释了为什么网络服务常和文件系统持久化绑在一起。若服务端先回 ACK 再写日志，崩溃后客户端以为成功而服务端忘记结果；若先写日志再回 ACK，客户端即使没收到响应，也能用同一个请求 ID 查询或重试。OS 网络栈负责传字节和报告连接状态，业务一致性由应用协议和存储提交共同证明。

数据包从设备队列到套接字

sk_buff 是网络栈的“页”和“bio”

Linux 网络栈中，`sk_buff` 是数据包在内核中的主要载体。它不仅保存字节，还保存协议头指针、校验和状态、路由信息、引用计数、分片信息和共享数据区。理解 `skb`，才能把驱动、协议栈和 socket 接起来。

```
sk_buff:
    head, data, tail, end
    len, data_len
    mac_header, network_header, transport_header
    dev, sk
    checksum state
    shared_info: fragments, gso, refcount
```

`skb_clone` 共享数据但复制元数据，`skb_copy` 复制数据，`kfree_skb` 释放引用。性能问题常来自不必要 copy；正确性问题常来自共享 `skb` 被误写、引用计数错误或线性区和 fragment 区理解错误。

接收路径：从 NAPI 到 socket 队列

高吞吐网络接收通常走 NAPI。中断只负责调度 poll，poll 在 budget 内批量取包，把 `skb` 送入协议栈。


```

nic_interrupt:
    disable_rx_interrupt()
    napi_schedule()

napi_poll(budget):
    while work < budget and rx_desc_ready:
        skb = build_skb_from_rx_desc()
        napi_gro_receive(skb)
    if no_more_work:
        napi_complete()
        enable_rx_interrupt()

ip_rcv -> tcp_v4_rcv -> tcp_v4_do_rcv
    -> append to socket receive queue
    -> wake waiters

```

GRO 可以把多个小包合并成较大 skb，降低协议栈成本；但它也改变了观测粒度。抓包、trace 和应用 recv 的边界不一定一一对应。测试网络栈时要区分 wire packet、skb、TCP segment 和应用读出的字节。

发送路径：tcp_sendmsg 到驱动队列

发送路径从用户 buffer 复制或引用到 skb，进入 TCP 发送队列，按拥塞窗口、接收窗口和 MSS 分段，最后到 qdisc 和驱动。

```

sendmsg
-> inet_sendmsg
-> tcp_sendmsg
-> copy into skb or attach pages
-> tcp_push_pending_frames
-> ip_queue_xmit
-> qdisc enqueue/dequeue
-> dev_queue_xmit
-> driver ndo_start_xmit

```

这个路径解释了 send 成功的边界：数据进入本机 TCP 发送状态，不等于对端应用收到，更不等于业务提交。若连接稍后 reset，应用必须根据协议状态判断请求是否需要重试。

Netfilter hook 点

Netfilter 在 IP 路径上提供 hook：PRE_ROUTING、LOCAL_IN、FORWARD、LOCAL_OUT、POST_ROUTING。防火墙、NAT、容器端口映射和连接跟踪都依赖这些点。

hook	典型用途
NF_INET_PRE_ROUTING	入站后、路由前，DNAT、早期过滤
NF_INET_LOCAL_IN	目的地是本机，进入 socket 前过滤
NF_INET_FORWARD	转发路径过滤
NF_INET_LOCAL_OUT	本机发出包，路由后处理
NF_INET_POST_ROUTING	出站前，SNAT、最后修改

Netfilter 让网络语义不仅由 socket 代码决定，还受规则表、namespace、连接跟踪和 NAT 影响。诊断“包为什么没到应用”时，要检查驱动、IP、路由、Netfilter、socket 绑定和应用读取，而不是只看一个层次。

socket option 与内核策略

socket option 是应用调整内核协议策略的入口。例如 `SO_RCVBUF` 和 `SO_SNDBUF` 控制缓冲区，`TCP_NODELAY` 影响 Nagle，`SO_REUSEPORT` 允许多个监听 socket 分担连接，`SO_KEEPALIVE` 打开保活探测。

选项	影响	风险
<code>TCP_NODELAY</code>	小包立即发送	更多包，可能降低吞吐
<code>SO_REUSEPORT</code>	多 socket 绑定同端口	负载分配和安全检查更复杂
<code>SO_RCVBUF</code>	接收缓冲上限	太小丢吞吐，太大耗内存
<code>SO_LINGER</code>	close 时处理未发送数据	可能阻塞或丢弃

选项不是调优魔法。每个选项都改变状态机和资源边界，必须通过负载测试和故障测试验证。

connect 的阻塞、非阻塞和错误取回

`connect` 不是简单地“连上或失败”。阻塞 TCP `connect` 会发起握手并睡眠等待结果；非阻塞 `connect` 通常返回 `EINPROGRESS`，之后用户用 `poll/epoll` 等待可写，再用 `getsockopt(SO_ERROR)` 取真正结果。可写事件只表示连接状态有变化，不表示一定成功。

```
nonblocking_connect(sock, addr):
    ret = connect(sock, addr)
    if ret == 0:
        connected
    else if errno == EINPROGRESS:
        wait EPOLLOUT
        err = getsockopt(sock, SOL_SOCKET, SO_ERROR)
        if err == 0:
            connected
        else:
            connect failed with err
    else:
        immediate failure
```

程序只看到 `EPOLLOUT` 就开始发送，是常见错误。连接拒绝、超时、路由不可达都可能通过 socket 错误状态返回。内核把异步错误挂在 socket 上，应用必须取走并处理。

recv 返回值的含义

TCP 是字节流，`recv` 的返回值表达当前 socket 状态，不表达业务协议状态。

结果	内核含义	应用动作
<code>n > 0</code>	从接收队列取出 n 字节	放入协议解析缓冲，继续解析消息边界
<code>0</code>	对端有序关闭，且本端已读完缓冲数据	处理 EOF，不能再期待新字节
<code>-EAGAIN</code>	非阻塞 socket 暂无数据	等待下一次可读事件

-EINTR	被信号打断	根据业务决定重试或退出
-ECONNRESET	连接被复位	丢弃未完成请求，按协议决定是否重试

EOF 不等于错误，reset 也不等于普通 EOF。若对端发送 FIN，已经在接收缓冲里的数据仍可读；读完后才返回 0。若收到 RST，未完成的字节流不再可信，很多协议必须把当前请求标成结果未知。

TCP 不是消息队列

应用写入两次，接收端可能一次读到合并后的字节；应用写入一次，接收端可能分多次读到。TCP 保证有序字节流，不保留 `write` 调用边界。业务协议必须显式 framing。

```
length_prefixed_protocol:
    read bytes into buffer
    while buffer has at least 4 bytes:
        len = parse_u32(buffer[0:4])
        if buffer has less than 4 + len:
            break
        message = buffer[4:4+len]
        dispatch(message)
        remove consumed bytes
```

这个状态机必须处理半包、粘包、超长长度、恶意长度、连接关闭和超时。把一次 `recv` 当成一条完整消息，是 TCP 服务端最常见的教学级错误。

Nagle、delayed ACK 和小包延迟

`TCP_NODELAY` 关闭 Nagle 后，小写入更快发出，但包数增加；Nagle 打开时，小包可能等待未确认数据；接收端 delayed ACK 又可能延迟确认。两者组合会让“写了几个字节为什么几十毫秒后才到”变成真实问题。

机制	目的	可能副作用
Nagle	合并小包，减少网络开销	请求/响应小消息延迟上升
Delayed ACK	减少纯 ACK 包	与 Nagle 组合造成等待
<code>TCP_NODELAY</code>	降低小消息等待	包数和 CPU 开销上升
<code>TCP_CORK</code>	手动聚合发送	忘记 uncork 会拖延发送

调优不能只背“低延迟就开 `TCP_NODELAY`”。如果协议能批量发送，保留聚合可能更好；如果是交互 RPC，小消息阻塞会直接进入尾延迟。

超时、重试和幂等不由 socket 自动解决

socket 超时只能告诉应用某个等待没有在期限内完成，不能告诉远端是否处理了请求。发送超时、接收超时、连接 reset、进程崩溃都可能让请求处在“可能提交、可能未提交”的灰区。应用协议要用请求 ID、幂等键、事务日志或去重表解决语义。

```
rpc_with_idempotency:
    id = allocate_request_id()
    send(id, operation, payload)
    if timeout:
        reconnect_or_retry(id)
    server:
```

```

if id already committed:
    return old result
execute operation
record result by id
return result

```

OS 负责提供字节流、错误码、关闭语义和等待机制；“这笔业务是否执行过”是应用协议层的责任。网络章节若不讲这个边界，读者会把 TCP 可靠传输误解成业务可靠提交。

网络事件、流量控制与提交边界

网络与持久化深讲：流、事件和提交

网络和持久化看起来相距很远，一个处理远端通信，一个处理本地存储。但它们共享几个 OS 问题：缓冲区所有权、阻塞与唤醒、错误传播、部分完成、重试与幂等。网络 send 可能短写，磁盘 write 可能只进 page cache；TCP 连接可能 reset，块设备写入可能返回 EIO；应用都必须写出可恢复状态机。

TCP 字节流为什么没有消息边界。

TCP 提供可靠有序字节流，不提供应用消息边界。一次 send 的 100 字节，接收方可能分两次 recv 到 40 和 60 字节，也可能和下一次 send 合并读到。应用协议必须自己定义长度、分隔符或帧格式。

```

read_frame(sock):
    while buffer.len < HEADER_SIZE:
        recv_more(sock, buffer)
    len = parse_length(buffer.header)
    while buffer.len < HEADER_SIZE + len:
        recv_more(sock, buffer)
    return extract_frame(buffer)

```

这解释了为什么 readiness 不是消息。epoll 告诉你 socket 可能可读，不告诉你完整帧已经到达。若把一次可读事件当成完整请求，协议解析会在高负载或网络分片时失败。

网络背压。

发送缓冲区、接收缓冲区、协议队列和用户态队列都必须有限。若应用生产速度超过网络发送速度，队列会增长。正确系统要让生产者感受到背压，而不是无限缓存。

```

send_with_backpressure(sock, data):
    while data not empty:
        n = send(sock, data)
        if n > 0:
            data = data[n:]
        else if errno == EAGAIN:
            wait_for_writable(sock)
        else:
            fail_connection(errno)

```

背压也适用于持久化。后台写盘跟不上，dirty page 不能无限增长；内核会限制脏页比例，让写入者参与 writeback 或阻塞。这样牺牲局部延迟，换取系统整体不被内存耗尽。

提交协议和幂等恢复。

网络服务通常要处理“请求已经执行但响应丢失”的情况。持久化系统要处理“数据已经写一部分但 commit 记录没写”的情况。二者都需要幂等标识。

```

request:
  client_id
  request_id
  operation
  payload

server:
  if request_id already committed:
    return previous_result
  prepare operation
  commit durable record
  return result

```

幂等不是“重复执行也没事”这么简单，而是系统能识别重复请求，并返回同一个已提交结果。没有 request id，网络重试可能造成重复扣款、重复写文件或重复发布版本。

断电恢复和网络重试的共同思维。

两者都要求你列出中间状态。

领域	中间状态	恢复规则
网络请求	received、processing、committed、replied	committed 后重复请求返回旧结果
文件输出	temp、synced、renamed、manifest	manifest 后才被读取者信任
日志事务	intent、data written、commit	commit 存在则 redo，不存在则忽略

学习 OS 时要养成这种状态枚举习惯。它比背诵某个 API 更接近工程正确性。

网络与持久化练习。

1. 为什么 TCP 服务端不能假设一次 `recv` 得到一个完整请求？
2. 设计一个带 request id 的幂等接口，说明响应丢失后客户端如何重试。
3. 比较 socket send buffer 背压和 dirty page 背压的共同点。
4. 写出 manifest 提交前后崩溃的恢复规则。
5. 为什么 commit record 需要 checksum 和 fsync？

网络错误和文件错误的相似性。

网络错误常被误解成“连接断了”，文件错误常被误解成“磁盘坏了”。实际上二者都需要精细分类。

领域	错误	处理
socket	EAGAIN	等可写/可读事件后继续
socket	connection reset	丢弃连接状态，决定请求是否可重试
socket	short write	保存剩余 buffer，下次继续发送
文件	short write	循环写或上报部分完成
文件	fsync failure	业务提交失败，不能发布 manifest

日志	checksum mismatch	停止恢复到最后完整记录
----	-------------------	-------------

共同原则是：先判断是否部分完成，再判断是否可重试，再判断重试是否幂等。没有这个顺序，网络服务和持久化系统都会在故障时产生重复、丢失或半提交。

案例：上传文件服务。

上传文件服务同时涉及网络 and 持久化。服务端从 socket 接收字节，写入临时文件，计算 checksum，fsync，rename，更新 manifest，再返回成功。任何一步失败都要有恢复规则。

```
upload(request_id, socket):
    tmp = create_temp(request_id)
    while not end_of_body:
        bytes = recv(socket)
        if bytes.error: abort_tmp(tmp)
        write_all(tmp, bytes)
        update_checksum(bytes)
    fsync(tmp)
    rename(tmp, final_name(request_id))
    fsync(parent_dir)
    commit_manifest(request_id, final_name, checksum)
    send_success(socket)
```

若成功响应丢失，客户端可能重试同一个 request id。服务端必须识别 manifest 中已有提交，返回同一结果，而不是再创建第二个文件。这就是网络幂等和持久化提交的交叉点。

本章小结。

- TCP 是字节流，应用协议必须自己定义消息边界。
- 网络和存储都可能部分完成，调用者必须保存状态继续推进。
- 背压是系统稳定性的机制，不是性能失败。
- 幂等 request id 能把网络重试和持久化提交连接起来。
- 日志、manifest 和 checksum 让崩溃恢复有可判定事实。

网络栈分层：从网卡帧到 socket。

网卡收到的是链路层帧，用户程序读到的是 socket 字节或数据报。中间要经过驱动、软中断或 polling、链路层解析、IP 层路由和分片处理、传输层状态机、socket 缓冲区和唤醒机制。

```
NIC DMA frame
-> driver RX ring
-> network poll budget
-> Ethernet header parse
-> IP validation and routing
-> TCP/UDP demux by ports
-> socket receive queue
-> wake epoll/read waiter
```

每层都有丢弃理由。网卡 ring 满会丢包；校验和错会丢包；IP TTL 为 0 会丢包；没有匹配 socket 会发 RST 或 ICMP；socket receive buffer 满会产生背压或丢弃。网络栈不是“把数据送给进程”，而是一系列有限队列和状态机。

UDP 和 TCP 的不同契约。

UDP 保留报文边界，但不保证可靠、有序、去重和拥塞控制；TCP 保证可靠有序字节流，但不保留消息边界。应用选择协议时，实际上是在选择哪些语义由内核提供，哪些语义由应用自己实现。

协议	提供	不提供
UDP	数据报边界、端口复用、校验和	重传、排序、拥塞控制、连接状态
TCP	可靠有序字节流、流量控制、拥塞控制	应用消息边界、请求幂等、业务提交
QUIC 类协议	用户态连接、流、多路复用、加密集成	仍需应用定义业务语义和提交规则

UDP 适合应用能容忍丢包或自己实现重传的场景，例如实时音视频、查询协议或自定义传输。TCP 适合需要可靠字节流的场景，但应用仍要处理半关闭、reset、短读短写和重复请求。

TCP 状态机和关闭语义。

TCP 连接有状态：LISTEN、SYN-SENT、ESTABLISHED、FIN-WAIT、CLOSE-WAIT、TIME-WAIT 等。关闭不是单个动作，而是双向字节流各自结束。收到 FIN 表示对方不再发送数据，但本端仍可发送；reset 表示连接异常终止，未读或未确认数据可能丢失。

事件	socket 读写表现	应用含义
peer FIN	<code>recv</code> 返回 0	对方正常关闭发送方向
local shutdown write	本端发送 FIN	告诉对方本端不再发送
RST	ECONNRESET 等错误	连接状态丢弃，业务是否完成未知
TIME_WAIT	连接四元组暂时保留	吸收迟到报文，避免旧连接污染新连接

服务端常见 bug 是把 `recv` 返回 0 当成错误，或把 `send` 成功当成对方业务已经处理。TCP 只能说明字节进入本机协议栈或被对端确认到传输层，不能说明远端应用已经提交业务。

流量控制和拥塞控制。

流量控制保护接收方：接收窗口告诉发送方还能接收多少字节。拥塞控制保护网络：发送方根据丢包、延迟或 ACK 估计网络可承受速率。二者都能让 `send` 变慢或返回 `EAGAIN`，但原因不同。

```
tcp_send_loop:
while app_has_data:
    if send_buffer_full:
        wait_app_backpressure()
    if receiver_window_zero:
        wait_window_update()
    if congestion_window_limited:
        wait_ack_or_timer()
    transmit_allowed_segments()
```

这解释了为什么“网络慢”不能只看带宽。可能是接收方应用不读导致窗口关闭；可能是中间网络拥塞导致拥塞窗口缩小；可能是本机 socket buffer 太小；可能是 Nagle、delayed ACK 或小包策略影响延迟。诊断要看队列、窗口、重传、RTT 和应用读写速率。

epoll/readiness：事件不是工作完成。

readiness API 告诉应用“某 fd 可能不会阻塞”，而不是“完整请求已经准备好”。边缘触发模式下，应用必须一直读到 `EAGAIN`，否则可能错过后续事件；水平触发模式下，只要条件仍成立，事件会重复出现。

```
on_readable(fd):
    while true:
        n = recv(fd, buf)
        if n > 0:
            parser.feed(buf[0:n])
            while parser.has_frame():
                handle_frame(parser.next_frame())
        else if n == 0:
            close_connection(fd)
            break
        else if errno == EAGAIN:
            break
        else:
            fail_connection(fd, errno)
            break
```

事件循环必须把连接状态保存在对象里：已读但未解析的字节、待发送但未写完的响应、超时时间、是否半关闭、request id 和持久化提交状态。没有这些状态，短读短写一出现，程序就会丢帧或重复发送。

accept 队列、listen backlog 和过载。

服务器调用 `listen(backlog)` 后，内核维护连接建立相关队列。过载时，SYN 队列、accept 队列、应用线程池、socket buffer 或业务队列都可能成为瓶颈。只调大一个 backlog 不一定解决问题。

队列	堵塞表现	对策
SYN 队列	新连接握手失败或重传	SYN cookie、限流、扩容
accept 队列	连接已建立但应用未 accept	更快 accept、多进程、backlog 调整
worker 队列	已接收请求等待处理	背压、负载均衡、降级
send buffer	响应写不出去	非阻塞写、限速慢客户端
持久化队列	fsync 或日志提交积压	批量提交、限流、隔离磁盘

一个健壮服务会在每层设置上限，并在上限触发时明确拒绝或降级。无限 accept、无限读 body、无限缓存响应，都会把远端慢客户端或恶意客户端变成本机内存问题。

持久化日志：redo、undo 和幂等。

崩溃恢复常见两类思路。redo 日志记录“提交后应当产生的结果”，恢复时重做已提交事务；undo 日志记录“修改前的旧值”，未提交事务崩溃后回滚。实际数据库可能使用更复杂的 WAL，但核心仍是先写可恢复证据，再改变主数据。

日志类型	写入顺序	恢复规则
redo	日志 intent/data 先持久，commit 后主数据可稍后写	有 commit 则 redo，无 commit 忽略
undo	旧值日志先持久，再覆盖主数据	无 commit 则 undo，有 commit 保留

manifest	内容文件先完整持久，再发布版本记录	只采用 manifest 中可校验版本
----------	-------------------	---------------------

幂等 request id 与日志结合后，服务才能处理响应丢失。commit record 中记录 request id 和结果摘要；客户端重试时，服务查到已提交记录，直接返回同一结果。没有这层记录，重试只能变成“再执行一次”，这对扣款、发货、发布版本等操作通常是错误的。

批量提交和延迟权衡。

每个请求都单独 fsync，语义清楚但吞吐受 flush 延迟限制。批量提交把多个请求放进同一次 journal commit 或 group commit，摊薄 flush 成本，但会增加单个请求等待批次的延迟。

```
group_commit_loop:
    wait_until(batch_not_empty or timeout)
    batch = collect_pending_requests(max_batch)
    append_records(batch)
    fsync(log)
    for req in batch:
        mark_committed(req)
        wake_waiter(req)
```

批量提交的正确性条件是：只有 fsync 成功后才能向这些请求返回 committed；fsync 失败时，整个批次都必须失败或进入恢复状态；批次中的每条记录要有长度、checksum 和 request id，恢复时能跳过不完整尾部。

网络服务的端到端状态机。

把网络和持久化合起来，一个请求至少经过这些状态：RECEIVING、PARSED、VALIDATED、EXECUTING、LOGGED、COMMITTED、REPLYING、DONE。错误和重试必须能从每个状态解释。

状态	持有资源	失败处理
RECEIVING	socket buffer、解析缓冲区	连接断开则丢弃不完整请求
VALIDATED	请求对象、权限结果	拒绝时不改变持久状态
EXECUTING	内存中业务变更或临时文件	失败则回滚临时资源
LOGGED	WAL/manifest 候选记录	未 commit 崩溃后忽略或 undo
COMMITTED	持久 commit record	重试返回同一结果
REPLYING	待发送响应 buffer	发送失败不撤销已提交业务

最后一行尤其重要。业务已经 committed 后，响应发送失败不能回滚业务，因为客户端可能稍后重试并期望得到同一结果。应用协议必须把“业务提交”和“响应送达”分开。

网络与持久化测试矩阵。

测试	注入故障	期望
TCP 分片	每次只发送 1 字节	parser 仍能组帧
短写	send 只接受部分响应	状态机保存剩余字节

连接 reset	commit 前/后分别 reset	前者不提交,后者可 幂等查询
重复 request id	响应丢失后重试	返回同一 committed 结果
fsync 失败	日志提交阶段返回 EIO	请求失败,不能发布 commit
崩溃恢复	commit 前/后断电	前者忽略,后者 redo/可查询
背压	慢客户端和慢磁盘同时出现	队列有上限,系统不 OOM

这些测试能证明服务不是只在理想网络和理想磁盘下正确,而是在短读短写、重复、断线、断电和过载下仍然有清晰语义。

尚未解决的问题。地址空间、fd 与 socket 已经隔离了对象命名,却没有回答某个主体是否应当获得某项操作权限,也不能限制一组进程耗尽全机资源。下一章从一次主体访问一个对象的判定开始建立最小权限。

第五部分

安全、恢复与证据

本部分导读

地址隔离只能阻止部分越权访问，接口返回成功也不保证系统能够从崩溃后的介质重新建立可信状态。最后一部分依次推导主体与对象之间的最小权限、跨层持久化协议，以及从用户症状回溯内核原因所需的证据。到这里，每一种机制都应能用同一组问题复核：对象由谁拥有，状态在何时改变，结果在何处可见，故障在何处收束，观察证据能够证明什么。

第 16 章 为什么已经隔离仍不等于最小权限

继承的机器。每个进程有自己的地址空间、对象引用和网络端点，但“看不见另一个地址空间”并不等于“只能做完成工作所需的动作”。同一用户的两个程序、容器内外的主体以及继承来的高权限描述符仍可能越过预期边界。

从受控进程到最小权限

设一个图片处理服务接收用户上传文件。解析库出现越界写后，攻击者已经能够以这个服务进程的身份执行任意代码。此时无法再依靠应用自身判断输入是否安全；操作系统必须回答一个更具体的问题：这个进程接下来能够读取哪些文件、访问哪些设备、调用哪些入口，以及消耗多少系统资源。

如果答案是“整台机器上的所有对象”，一个局部解析错误就可能扩展成整机失陷。若不同资源都有明确边界，攻击者即使控制了服务进程，也只能在该进程原本获得授权的范围内活动。由此得到操作系统安全的基本任务：它不是保证程序永不出错，而是在程序出错或被控制之后，仍然限制它能观察、修改和消耗的对象集合。

这种限制不可能由一个开关完成。内存访问、文件访问、进程可见性、设备入口、网络配置和资源消耗分别由不同对象管理，需要在各自的受控入口上作出决定。所谓隔离，就是这些约束共同形成的边界；所谓最小权限，就是只向主体保留完成既定任务所必需的能力。

错误设计通常长这样：

```
run_service():
    start as root
    mount host filesystem
    allow all syscalls
    no memory or process limit
    parse untrusted image
```

这个设计把“程序正常运行需要的能力”和“程序被攻破后拥有的能力”混成了一件事。正确思路要反过来：先列主体、对象和操作，再逐层收紧。

层次	限制什么	失败时挡住什么
CPU 特权级和页表	用户代码不能执行特权指令或访问内核页	任意代码直接改内核内存
权限模型层	哪个主体能访问哪个文件、socket、设备	越权读写对象
namespace	进程能看见哪些名字和对象集合	通过全局路径或 PID 找到宿主资源
cgroup/rlimit	能消耗多少 CPU、内存、进程和 I/O	fork bomb、内存耗尽、I/O 拖垮整机
沙箱收紧层	未来还能调用哪些入口、访问哪些路径	漏洞后扩大攻击面

最基础隔离来自 CPU 特权级和页表权限。用户态不能执行特权指令，不能访问内核页，不能修改页表。系统调用是受控入口，内核在入口处检查参数和权限。文件权限、用户 ID、组 ID、capability、namespace、cgroup、seccomp 和 LSM 则在更高层限制对象访问和资源消耗。

这些机制不是互相替代。页表保护内核地址，但不决定某个用户能不能写 `/etc/passwd`；文件权限能拒绝写文件，但不能限制进程创建十万个子进程；namespace 改变进程看到的文件树和 PID 视图，但共享宿主内核；seccomp 能减少系统调用入口，但不能把原本没有的文件权限授予进程。安全设计要叠加多层边界，每层解决不同失败模式。

本章第一次使用“主体”和“对象”两个词。主体是发起动作的一方，例如进程、线程、用户身份、容器或服务账号。对象是被访问的一方，例如虚拟页、inode、socket、设备、namespace、cgroup 文件。权限检查不是抽象口号，它必须落到“主体 S 对对象 O 做操作 A 是否允许”这个三元关系上。

继续展开图片处理服务的事故。攻击者上传一张畸形图片，触发解析库越界写，拿到了服务进程内的任意代码执行。此时 CPU 和页表已经无法阻止它在本进程地址空间里做事，因为它就是这个进程本身；OS 隔离要从“它接下来想越界访问别的对象”开始拦截。

攻击动作	若没有边界	哪层机制应当挡住
读取 <code>/etc/shadow</code>	泄露宿主口令哈希	DAC、capability、mount namespace、只读根文件系统
打开容器引擎 socket	创建特权容器控制宿主	mount namespace 不暴露 socket，文件权限和 LSM 拒绝
调用 <code>ptrace</code> 注入同机服务	横向控制其他进程	uid 边界、Yama/LSM、缺少 <code>CAP_SYS_PTRACE</code> 、seccomp
无限 <code>fork</code>	占满 pid 和调度资源	pids cgroup、rlimit
分配大量内存	触发宿主回收和 OOM	memory cgroup、oom 策略
创建设备节点或挂载文件系统	访问宿主设备或改写视图	缺少 <code>CAP_SYS_ADMIN</code> ，seccomp/LSM 拒绝

按这张表看，安全配置不是“开一个沙箱”就结束。假如服务以普通 uid 运行，但把宿主 Docker socket 挂进容器，攻击者不需要 root 也可能通过 socket 请求宿主创建新容器；假如没有 pids 限制，攻击者即使读不了敏感文件，也能 fork bomb 拖垮机器；假如 seccomp 允许 `mount`、`bpf`、`ptrace`、危险 `ioctl`，一次普通解析漏洞就有机会变成内核攻击面。每一层都只负责一类失败，叠加起来才叫最小权限。

把收紧后的设计写成状态变化，而不是口号：

```
before accepting untrusted images:
create service uid without shell login
mount only /app and /tmp/work, /app read-only
map container uid 0 to unprivileged host uid if using user namespace
drop all capabilities except the exact required set
set pids.max, memory.max, cpu.max, io.max
set no_new_privs
install seccomp profile for the real hot path
apply LSM/Landlock rules for file access
exec worker
```

这个顺序也有意义。先准备 namespace 和 cgroup，是为了让后续进程在受限视图和资源账本里出生；drop capability 和 no_new_privs 要发生在执行不可信工作前；seccomp 一旦安装，未来系统调用入口被缩小；最后通过 exec 进入 worker，可以顺便清理地址空间、环境变量和不该继承的 fd。若顺序反了，例如先开始解析图片再安装 seccomp，漏洞就可能发生在边界生效之前。

主体、对象、操作与审计

安全分析先写主体、对象、操作、权限：

主体	对象	操作	检查
进程	虚拟页	read/write/exec	页表权限、VMA
进程	文件	open/read/write	mode、owner、capability、mount
进程	系统调用	exec/fork/ioctl	seccomp、LSM、资源限制
容器	CPU/内存	消耗资源	cgroup quota/limit

然后写威胁模型：攻击者控制什么输入，目标对象是什么，期望阻止什么。没有威胁模型的“安全”讨论会变成口号。

DAC、能力、沙箱与资源限制

访问控制检查

访问控制算法接收主体、对象和操作，返回允许或拒绝。Unix mode 位是最小模型：如果主体是 owner，看 owner bits；否则如果主体属于 group，看 group bits；否则看 other bits。

```
may_access(subject, inode, op):
    if subject.uid == inode.uid:
        bits = inode.mode.owner
    else if subject.groups contains inode.gid:
        bits = inode.mode.group
    else:
        bits = inode.mode.other
    return bits.allow(op)
```

capability 是对 root 权限的拆分。与其让 uid 0 拥有全部能力，不如把网络配置、挂载、修改时间、加载模块等能力拆开。这样服务可以只拿最小需要的能力。

namespace 和 cgroup

namespace 改变“看见什么”，cgroup 限制“能用多少”。PID namespace 让进程看到自己的进程树；mount namespace 改变文件系统视图；network namespace 隔离网卡、路由和端口。cgroup 限制 CPU、内存、I/O 和进程数量。

```
container_start():
    create_namespaces(pid, mount, net, ipc, uts)
    configure_cgroup(cpu_quota, memory_limit, pids_max)
    drop_capabilities()
    apply_seccomp_filter()
    exec_init_process()
```

容器不是虚拟机。它共享宿主内核，所以内核漏洞仍可能突破容器边界。容器安全依赖 namespace、cgroup、capability、seccomp、LSM 和镜像文件系统共同工作。

seccomp 过滤

seccomp 允许进程限制自己未来能调用哪些系统调用。典型服务启动后，只保留 read、write、epoll、futex、clock 等必要调用，拒绝 mount、ptrace、模块加载等危险入口。

```
seccomp_check(syscall_nr, args):
```

```
action = bpf_program.evaluate(syscall_nr, args)
if action == ALLOW: continue_syscall()
if action == ERRNO: return -EPERM
if action == KILL: terminate_process()
```

seccomp 的价值是缩小攻击面。即使攻击者控制了进程内存，也很难调用被过滤的内核入口。但过滤规则写错也会让程序在正常路径崩溃，所以必须用测试覆盖。

namespace、容器与逃逸边界

- 用户态访问内核地址必须失败。
- 只读映射写入必须触发权限错误。
- 文件权限拒绝时，不创建、不截断、不写入目标文件。
- drop capability 后，相关特权操作必须失败。
- namespace 中不可见的对象不能通过旧路径访问。
- cgroup 内存限制触发时，系统有明确 OOM 或失败策略。
- seccomp 规则允许正常路径，拒绝非必要危险系统调用。

边界失效后的故障控制与证据

安全隔离不是最后一章唯一主题。一个真实系统还必须回答：错误怎样返回，panic 后哪些状态不可信，恢复依据是什么，观测证据从哪里来。把 panic、错误路径和可观测性放在安全章收束，是因为它们都在检查同一件事：边界被突破或状态不一致时，系统能不能停止扩散、留下证据、支持恢复。

场景	系统应做什么	验收证据
普通错误	返回明确 errno，回滚未提交对象	负面测试证明没有副作用和泄漏
可恢复资源压力	限流、回收、失败或局部 OOM	cgroup 事件、PSI、日志和请求错误率
内核不变量破坏	oops/panic，停止继续写不可信状态	panic 日志、kdump、最后 trace 事件
设备或 I/O 错误	停新请求、完成 in-flight、上报 delayed error	dmesg、block stats、fsync 错误、驱动计数器
安全拒绝	拒绝入口并记录主体、对象、操作	audit、LSM 日志、seccomp 统计和负面测试

观测工具也要按问题选择。tracepoint 和 ftrace 适合看内核路径是否经过；perf 适合看 CPU 时间、cache miss 和热点；eBPF 适合在线聚合延迟、队列长度和错误码；日志适合记录低频状态转换；crash dump 适合 panic 后离线检查。观测不是越多越好，证据必须能回答具体假设：线程是在等 CPU、等锁、等 I/O、等内存回收，还是被安全策略拒绝。

```
incident_report:
  symptom: request p99 rises from 20ms to 2s
  hypothesis: blocked on page cache writeback
  evidence:
    off-cpu stack shows balance_dirty_pages
    PSI io full rises during incident
    block queue latency p99 rises
    application CPU remains low
```

```
conclusion:
    not an algorithm CPU hotspot; storage backpressure
```

整册的最终验收可以压成七项。任一机制都要写出状态对象、入口、权限、等待、提交、失败路径和证据。调度有 task/runqueue/wait queue；内存有 mm/VMA/PTE/page；文件有 fd/file/inode/page cache/request；网络有 socket/skb/队列/错误状态；安全有主体/对象/策略/审计。若某个解释只剩术语，没有这七项，就还不能指导实现和排障。

验收标准

本册收束标准：拿到一次系统调用、缺页、I/O、网络请求或安全拒绝，能说清它改变了哪些对象，在哪个点可能睡眠，哪个点对用户可见，哪个点对持久化或远端可见，失败如何回滚，哪条观测证据能证明判断。做到这一点，章节压缩后内容仍然是厚的。

LSM、seccomp 与系统加固

访问控制模型

DAC 由对象拥有者控制访问，典型依据是 UNIX mode、uid 和 gid；MAC 由系统策略决定；RBAC 按角色授权；ABAC 按属性决策。Linux 的一次实际访问通常叠加 DAC、capability、LSM、挂载标志和 namespace。分析时必须跟随同一个对象，从路径解析一直追踪到最终安全决策，不能把其中任何一层单独视为全部权限规则。

```
open(path, O_WRONLY):
    resolve path to inode and mount
    check mount is writable
    check DAC mode or CAP_DAC_OVERRIDE
    call security_inode_permission()
    check file type and open flags
    create struct file
```

安全检查必须绑定实际对象。只检查字符串路径，再使用路径，是 TOCTOU 风险；解析后持有 inode、dentry 或 file 引用，再检查权限，才有稳定对象。权限失败时还要保证没有副作用：不能先 truncate 再发现权限不足。

capability 集合

进程有 permitted、effective、inheritable、bounding、ambient 等 capability 集合。bounding set 给进程树设置上界；ambient set 影响 exec 后保留的能力。容器安全经常要求清空不必要 capability，尤其避免 CAP_SYS_ADMIN、CAP_SYS_PTRACE、CAP_NET_RAW。

```
drop_privileges():
    clear_ambient_capabilities()
    drop_bounding_set_except(required_caps)
    set_no_new_privs()
    install_seccomp_filter()
```

能力检查要放在对象所属 user namespace 下解释。ns_capable(user_ns, CAP_NET_ADMIN) 和全局 capable 不是同一个边界。容器内 root 若只在容器 user namespace 有能力，不应自动获得宿主网络或宿主挂载的管理权。

namespace 与 cgroup v2

namespace	隔离内容
PID	进程编号和父子视图

Mount	挂载树和路径视图
Network	网络设备、路由、端口空间
IPC	System V/POSIX IPC 对象
UTS	hostname/domainname
User	uid/gid 映射和 namespace 内 capability
CGroup/Time	cgroup 视图、部分时间视图

cgroup v2 由父层声明可用控制器，并向子树委派控制能力，再分别表达 memory、cpu、io、pids 等限制。资源限制也是安全边界：没有进程数限制，fork bomb 可能扩散到宿主；没有内存限制，单个服务可能触发全局回收和 OOM；没有 I/O 限制，后台写入会拖慢其他租户。

Landlock、seccomp user notification 和 BPF LSM

Landlock 允许非特权进程给自己加文件访问规则，是基于 LSM 的沙箱。seccomp user notification 可把某些系统调用交给监督进程决策。BPF LSM 允许加载受验证的 BPF 程序到安全 hook。这些机制的共同边界是：它们可以进一步限制自己或被管理进程，但不能凭空授予原本没有的权限。

```
sandbox_process:
  create Landlock ruleset
  add allowed read-only paths
  restrict_self()
  install seccomp allowlist
  exec workload
```

这类沙箱必须有负面测试：禁止路径打开失败、禁止 syscall 被拒绝、规则安装后不能再获得新权限。只验证正常路径能运行，不足以证明沙箱有效。

TPM、measured boot 和 sealed storage

TPM 可记录启动链测量值，remote attestation 让远端验证机器启动了预期固件、bootloader、内核和策略；sealed storage 让密钥只在特定测量状态下解封。它解决的是“这台机器是不是以预期状态启动”，不是运行时所有访问控制。内核被测量启动后，仍然要依赖权限、LSM、seccomp 和更新机制维护运行时边界。

ASLR、stack canary、CFI 与 KASLR

ASLR 随机化栈、堆、mmap 和 PIE 基址；stack canary 检测栈溢出覆盖返回地址；CFI 限制间接控制流；KASLR 随机化内核基址。它们是缓解措施，不是权限模型替代品。

机制	防什么	不能防什么
ASLR/KASLR	降低地址可预测性	信息泄漏后效果下降
stack canary	简单栈返回地址覆盖	任意写、逻辑漏洞
CFI	非法间接跳转	合法路径上的权限绕过
WX/NX	数据页执行	JIT 配置错误、ROP

安全测试要同时证明正常路径可用、禁止路径失败、失败有日志、敏感数据不泄露。只打开硬化选项但没有负面测试，不能说明边界有效。

open 权限检查不能只看路径字符串

路径是输入，inode 和 file 才是稳定对象。安全代码若先检查字符串路径，再在另一步打开文件，中间目录项可能被替换，形成 TOCTOU。更可靠的做法是让内核

路径解析在同一次操作里完成权限、mount、LSM 和 open flag 检查，或者使用目录 fd、`openat2` 约束和已经打开的 fd 作为稳定能力。

```
unsafe pattern:
    if path starts with "/safe":
        open(path)

safer pattern:
    dirfd = open("/safe", O_PATH | O_DIRECTORY)
    openat2(dirfd, relative_path,
            RESOLVE_BENEATH | RESOLVE_NO_SYMLINKS)
```

这里的关键不是某个 API 名字，而是对象绑定：检查和使用必须落在同一个解析结果上。符号链接、bind mount、rename、并发 unlink、mount namespace 都会改变“同一个字符串”指向的对象。

exec 时 capability 和 no_new_privs 改变边界

权限不是进程启动后固定不变。exec 会重新计算凭据、环境、dumpable 状态、secureexec、文件 capability 和 setuid 影响。若进程设置 `no_new_privs`，后续 exec 不能获得比当前更多的权限；seccomp 过滤器通常要求先设置这个标志，防止低权限进程安装过滤器后再通过 exec 提权。

```
exec credential sketch:
    read current cred
    inspect executable mode and file capabilities
    if no_new_privs:
        suppress privilege gain
    compute new permitted/effective/ambient sets
    apply secureexec rules if privilege boundary changed
    commit new cred at exec commit point
```

ambient capability 容易被误用。它能跨 exec 传递给非特权可执行文件，所以启动服务前应清理不需要的 ambient set，并收紧 bounding set。否则子进程可能在没预期的路径上保留网络、ptrace 或管理能力。

seccomp 规则要按参数和返回动作设计

只按 syscall 号 allow/deny 往往太粗。例如 `ioctl`、`prctl`、`clone`、`socket` 的危险性高度依赖参数。高质量 seccomp profile 至少要区分常用参数组合，并为拒绝路径选择明确动作：返回 `errno`、kill 进程、记录日志或交给 user notification。

```
seccomp policy sketch:
    allow read/write/close/futex/clock_gettime
    allow mmap only without PROT_EXEC|PROT_WRITE together
    allow socket only AF_UNIX if service does not need network
    reject mount/ptrace/bpf/module loading
    return EPERM for expected probes
    kill process for impossible dangerous calls
```

规则还要随库和运行时变化测试。动态链接器、线程库、DNS、日志库都可能调用你没想到的系统调用。部署前应在真实启动路径、热路径、错误路径和 reload 路径下跑 profile，而不是只跑主函数。

Landlock 的边界：只能收紧，不能授权

Landlock 适合让普通进程给自己加文件系统沙箱。它的 ruleset 声明要处理哪些访问类型，再把路径 fd 加入 allow list，最后 `restrict_self`。一旦限制生效，进程不能用 Landlock 获得原来没有的权限，只能在原权限基础上进一步收紧。

```
landlock flow:
ruleset = create_ruleset(handled_access_fs)
dirfd = open(allowed_dir, O_PATH | O_DIRECTORY)
add_path_beneath_rule(ruleset, dirfd, allowed_access)
prctl(PR_SET_NO_NEW_PRIVS, 1)
landlock_restrict_self(ruleset)
exec or run workload
```

测试要覆盖允许目录、禁止目录、符号链接、rename、临时文件和继承。若只测试“程序还能启动”，无法说明沙箱真的挡住了写入、删除、执行或路径逃逸。

BPF LSM 和传统 LSM 的关系

LSM hook 把安全检查插入到 inode、file、task、socket、bpf 等对象操作中。传统 LSM 如 SELinux、AppArmor 使用预先加载的策略；BPF LSM 允许受验证的 BPF 程序挂到安全 hook 上，适合做可观测、渐进式或环境相关的策略。

```
security_inode_permission(inode, mask):
for each registered LSM:
ret = lsm->inode_permission(inode, mask)
if ret != 0:
return ret
return 0
```

BPF LSM 仍受 verifier、权限和 hook 上下文限制。程序不能任意睡眠，不能随便访问内核内存，返回值语义也要遵守 hook 约定。它是内核安全框架的一部分，不是绕过 DAC/capability 的后门。

容器逃逸常见入口

容器共享宿主内核，所以边界经常坏在“多给了一个能力或设备”。常见风险包括挂载宿主敏感路径、保留 CAP_SYS_ADMIN、开放容器引擎 socket、允许特权设备、未限制 /proc 和 /sys、seccomp profile 过宽、cgroup pids/memory 未限。

风险	为什么危险	收紧方向
CAP_SYS_ADMIN	覆盖大量管理操作	默认删除，只按具体需要添加
宿主容器引擎 socket	等价于控制宿主容器引擎	不挂载，改用受限代理
特权设备节点	可能直接访问内核或硬件接口	devices cgroup 和 最小设备集
宽松 /proc	泄露进程、内核和 namespace 信息	hidepid、只读挂载、裁剪视图
无 pids/memory 限制	fork bomb 或内存压力拖垮宿主	cgroup v2 限额

安全隔离要按“最小必要对象”配置，而不是按“程序跑起来需要什么就全开”。每放开一个 capability、设备、挂载或 syscall，都要写清楚它对应的业务需求和负面测试。

策略命中以后还需要可核对证据

安全与观测深讲：没有证据就没有边界

安全和可观测性必须一起讲。权限检查失败时，要能证明没有改变目标对象；进程被 seccomp 拒绝时，要能看到拒绝的系统调用；cgroup 限制触发时，要能解释

是内存、CPU 还是进程数限制；panic 发生时，要保留足够证据。没有观测能力，安全边界只能靠想象。

威胁模型写法。

安全分析应先写威胁模型，而不是先列工具名。

```
Threat model:
  attacker controls:
    HTTP request body and headers
    uploaded file content
  attacker does not control:
    service binary
    kernel code
    host root credentials
  assets:
    user database
    service token
    host filesystem
  required boundaries:
    process cannot read host secrets
    process cannot mount or ptrace
    process cannot consume unlimited memory
```

这样才能判断 namespace、cgroup、capability、seccomp 和文件权限各自解决哪一部分问题。否则“用了容器”不等于安全，“用了 seccomp”也不等于安全。

最小权限的阶段化。

服务启动阶段和运行阶段需要的权限不同。启动可能需要读取配置、绑定端口、打开日志；运行时只需要处理已有 fd 和少量系统调用。权限应随阶段下降。

```
startup:
  read config
  bind port 80
  open log file

drop:
  chroot or mount namespace
  drop capabilities
  set no_new_privs
  install seccomp allowlist
  enter cgroup limits

serve:
  accept, read, write, epoll, futex, clock
```

权限下降要可测试。测试不应只证明正常请求成功，还要证明 mount、ptrace、打开宿主敏感路径、超额 fork 和超额内存分配都会失败。

错误路径的证据。

错误路径也要写日志和计数器，但不能泄露敏感信息。比如权限拒绝应记录主体、对象类型、操作和拒绝原因，但不应把密钥内容写入日志。panic 记录寄存器、栈、锁和任务状态，但不能依赖复杂文件系统路径。

事件	必要证据	避免
权限拒绝	uid、capability、对象类型、操作	打印敏感文件内容
OOM	cgroup、进程、内存统计、victim	只写 killed without reason

seccomp 拒绝	syscall number、action、pid	静默杀进程无诊断
panic	原因、CPU、寄存器、栈、锁	panic 路径做复杂阻塞 I/O

从现象到证据链。

一个好的 OS 调试结论要形成证据链。

Symptom:
p99 latency increased from 50ms to 2s

Evidence chain:
application trace: requests stuck in write phase
syscall trace: fsync takes 1.8s
kernel metrics: dirty pages high
block trace: queue depth high, flush waits long
conclusion: tail latency dominated by storage flush

这种报告比“怀疑磁盘慢”有用，因为它说明了每一层的证据和排除方向。若没有 syscall trace，也可能是应用锁；若没有 block trace，也可能是文件系统 journal；证据链让下一步实验明确。

安全与观测练习。

- 1. 写一个网络服务的威胁模型，列出攻击者能控制和不能控制的对象。
- 2. 设计 seccomp allowlist，并说明正常路径需要哪些系统调用。
- 3. 构造权限拒绝测试，证明失败时目标文件没有被创建或截断。
- 4. 写一份 OOM 诊断报告，包含 cgroup、进程和内存统计。
- 5. 给一次 panic 设计最小 crash dump 字段。

安全边界的测试矩阵。

安全机制必须能被测试。下面是一组最小矩阵：

边界	测试	期望
页表隔离	用户态读内核地址	page fault 或进程终止
文件权限	无权限 open/write	返回拒绝且目标不变
capability	drop 后执行特权操作	返回 EPERM
namespace	访问不可见路径或进程	不可见或返回不存在
cgroup	超过内存或进程限制	OOM/失败 限制在 cgroup 内
seccomp	调用禁止 syscall	按策略返回 errno 或杀进程
观测	触发拒绝和 OOM	日志/计数器能解释原因

测试安全时要检查“没有发生什么”。权限拒绝后文件不能被截断；seccomp 拒绝后进程不能完成危险操作；cgroup OOM 不能扩大成宿主全局 OOM。负面保证是安全测试的核心。

可观测性常见误区。

常见误区

- CPU 低不代表系统没压力，可能都在 off-CPU 等 I/O 或锁。
- RSS 增长不一定是泄漏，可能是 page cache、arena 或 mmap。
- p99 变差不一定是平均路径慢，可能是少数 fsync、reclaim 或锁等待。
- 日志多不等于可观测，结构化字段和请求身份更重要。
- 没有反证的结论只是猜测。

本章小结。

- 安全需要威胁模型，不能只列机制名。
- 最小权限要随阶段下降，并用测试证明危险操作失败。
- 错误路径必须留下足够证据，但不能泄露敏感数据。
- 调试报告要形成现象、证据、反证、结论和边界的链条。
- 安全和可观测性互相支撑：看不见的边界很难被信任。

访问控制：主体、客体、操作和上下文。

权限判断可以抽象成四元组：主体是谁，客体是什么，操作是什么，上下文是什么。主体可能是 uid、gid、capability、进程安全标签或容器身份；客体可能是 inode、socket、进程、设备或命名空间对象；操作是 read、write、execute、mount、ptrace 等；上下文包括路径、挂载选项、是否 setuid、是否处于用户命名空间等。

```
may_access(subject, object, operation, context):
    if object.type == inode:
        if dac_denies(subject.uid, object.mode, operation):
            if !has_capability(subject, override_cap(operation)):
                return DENY
    if lsm_policy_denies(subject.label, object.label, operation):
        return DENY
    if mount_flags_deny(context.mount, operation):
        return DENY
    return ALLOW
```

真实系统通常叠加多层策略：传统 UNIX DAC、capability、ACL、LSM、namespace 边界和挂载选项。安全 bug 常来自只检查其中一层，或在路径解析和最终对象使用之间发生 TOCTOU。正确做法是尽量在持有对象引用后检查权限，并让检查结果绑定到实际对象而不是字符串路径。

capability：把 root 权限拆小。

传统 root 权限过大。capability 把特权拆成多个位，例如绑定低端口、修改网络配置、加载模块、改变文件所有者、绕过 DAC 等。服务启动时可能短暂需要某些 capability，运行时应丢弃。

能力类型	能做什么	风险
CAP_NET_BIND_SERVICE	绑定低端口	可在启动后丢弃
CAP_SYS_ADMIN	大量系统管理操作	过大，接近半个 root
CAP_SYS_PTRACE	调试或观察其他进程	可读敏感内存
CAP_DAC_OVERRIDE	绕过文件权限检查	可读写本不该访问的文件

CAP_NET_ADMIN	改网络接口和路由	可影响宿主网络
---------------	----------	---------

最小权限不是“非 root 运行”一句话。一个非 root 进程若仍有危险 capability，仍可能扩大影响面。反过来，一个 root 启动的进程若完成初始化后清空 capability、设置 no_new_privs 并进入受限 namespace，风险会显著降低。

namespace：隔离名字，不隔离内核。

namespace 改变进程看到的名字空间：mount namespace 改变路径树，pid namespace 改变进程编号视图，net namespace 改变网络设备和路由视图，uts namespace 改变 hostname，ipc namespace 改变 System V/POSIX IPC 视图，user namespace 改变 uid/gid 映射。

namespace	隔离内容	仍然共享
mount	文件系统挂载视图	同一内核 VFS 和底层设备风险
pid	进程编号和父子视图	调度器、内核内存、系统调用实现
net	网卡、路由、端口空间	协议栈代码和内核漏洞面
user	uid/gid 映射和部分能力	内核仍必须正确限制能力转换
ipc	IPC 对象名字空间	内核 IPC 实现

容器不是虚拟机，因为容器共享宿主内核。namespace 隔离的是名字和视图，不是内核代码本身。若内核某系统调用存在提权漏洞，容器内进程可能利用同一个内核漏洞逃逸。因此容器安全通常要叠加 seccomp、capability、只读挂载、cgroup 和及时更新内核。

cgroup：资源边界也是安全边界。

没有资源限制，攻击者可以通过 fork bomb、内存耗尽、磁盘写满或 CPU 忙等影响整机。cgroup 给进程集合设置资源边界，例如 CPU 权重/配额、内存上限、进程数、I/O 权重和设备访问。

资源	没有限制的风险	期望证据
CPU	忙等拖慢其他服务	throttled time、quota hit 计数
memory	OOM 扩散到宿主	cgroup OOM 事件、victim、统计
pids	fork bomb 填满进程表	pids.current/max、fork 失败
I/O	大量写回拖慢共享磁盘	io latency、队列深度、权重
devices	访问宿主敏感设备	设备拒绝日志和 errno

资源限制要和观测一起配置。只设置 memory.max 但不记录 OOM 事件，故障时只能看到进程消失；只设置 CPU quota 但不看 throttled time，容易误判成应用锁或网络慢。

seccomp：收缩系统调用面。

seccomp 允许进程限制自己可调用的系统调用。最小化系统调用面能减少内核攻击入口。网络服务正常处理请求可能只需要 `read/write/recv/send/epoll/futex/clock/mmap/brk/close` 等有限集合，不应该在运行阶段调用 `mount`、`ptrace`、`init_module`。

```
seccomp_policy:
  allow read, write, close
  allow epoll_wait, epoll_ctl
  allow recvfrom, sendto, accept4
  allow futex, clock_gettime
  allow mmap, munmap, brk
  deny mount, ptrace, bpf, init_module
```

`allowlist` 要从实际 `trace` 推导，而不是凭感觉写。测试时先在日志模式收集正常路径系统调用，再切到拒绝模式；同时要测试危险调用确实失败。`seccomp` 失败动作也要明确：返回 `errno`、kill 线程、kill 进程或记录审计。

安全日志：可审计但不泄密。

安全事件日志需要足够结构化，便于按主体、对象、操作和原因查询。日志字段应包含 `request id`、`pid`、`uid`、容器/`cgroup`、操作、对象标识、策略名称和结果。敏感数据如 `token`、密码、密钥、完整请求体不应写入普通日志。

```
security_event:
  request_id=...
  pid=...
  uid=...
  cgroup=...
  operation=open_write
  object=/data/report
  policy=readonly_mount
  result=denied
  errno=EROFS
```

日志也要有完整性和留存策略。攻击者若能删除或篡改日志，事后审计会失效；日志若无限增长，又会变成磁盘耗尽风险。高安全系统通常把安全事件发送到独立收集端，并给本地日志设置限额和轮转。

性能观测：USE 与 RED 方法。

系统性能观测可以用两套简单框架。USE 关注资源：Utilization、Saturation、Errors，即资源使用率、排队饱和度和错误。RED 关注服务请求：Rate、Errors、Duration，即请求速率、错误率和耗时分布。

框架	适用对象	指标例子
USE	CPU、内存、磁盘、网卡、锁、队列	CPU busy、runqueue、EIO、drop、reclaim
RED	HTTP/RPC/数据库请求	QPS、5xx、p50/p99、超时

这两套方法要结合。用户看到的是 RED：请求慢或失败；工程师定位要落到 USE：哪个资源饱和或错误。比如 `p99` 上升可能来自 CPU `runnable wait`、锁等待、`page fault`、`dirty writeback`、网络重传或 `cgroup throttling`。

tracepoint、采样和事件日志。

观测手段有不同成本。计数器成本低，但细节少；事件 `trace` 细节多，但可能扰动系统；采样 `profiler` 能看到热点，但对短暂事件和 `off-CPU` 等待不敏感；分布式 `trace` 能串请求路径，但需要上下文传播。

手段	适合回答	局限
----	------	----

计数器	是否发生、发生多少次	难以解释单个请求
直方图	延迟分布和尾部	需要合理 bucket
事件 trace	谁在何时做了什么	数据量大，需过滤
CPU sampling	on-CPU 热点在哪里	看不到 sleeping 时间
off-CPU trace	线程等待在哪里	栈采集和符号成本

一个成熟诊断通常先看低成本指标定位方向，再打开更细粒度 trace 验证假设。直接全量抓所有事件，往往会制造过多噪声和额外开销。

panic、oops 和可恢复错误。

内核错误并非都要 panic。用户传坏指针应返回 `EFAULT`；设备 I/O 失败应向上返回 `EIO`；某个驱动检测到可重置设备错误，可以隔离设备并恢复。panic 适用于内核不变量被破坏且继续运行可能造成更大损坏的情况，例如内核栈破坏、关键锁结构损坏、无法恢复的内存一致性错误。

错误类型	处理	证据
用户错误	返回 <code>errno</code> 或发信号	<code>syscall</code> 、 <code>pid</code> 、地址、 <code>errno</code>
设备错误	重试、 <code>reset</code> 、上报 <code>EIO</code>	设备状态、请求 <code>id</code> 、超时
资源不足	<code>ENOMEM</code> / <code>OOM</code> /限流	<code>cgroup</code> 、分配大小、 <code>victim</code>
内核 bug	<code>oops</code> 或 <code>panic</code>	寄存器、栈、锁、CPU、 <code>taint</code>

panic 路径要少依赖。此时文件系统、调度器或锁可能已经不可信，所以 crash dump 和 console 输出应尽量使用简单、预留或早期初始化的路径。复杂日志系统在 panic 中可能再次阻塞或破坏证据。

故障注入：证明错误路径真的可用。

错误路径如果不测试，基本等于没实现。故障注入让内核或应用在可控点返回错误：分配失败、copy 用户失败、磁盘第 N 次写失败、网络短写、时钟跳变、锁超时、设备 completion 丢失。

```
fault_injection_plan:
  fail allocation at callsite X once
  force copy_from_user to fail after 128 bytes
  return EIO on block write number N
  drop network response after commit
  delay wakeup by 500ms
```

每个注入点都要有预期：资源是否释放，引用计数是否回到原值，目标对象是否保持不变，错误是否传给调用者，日志是否足够诊断。故障注入把“理论上能处理错误”变成可验证事实。

安全与观测的最终验收。

验收标准

一个边界合格的系统必须同时满足三件事：正常路径能完成；越权路径明确失败且没有副作用；失败路径留下足够证据，能说明主体、客体、操作、原因和影响范围。

尚未解决的问题。权限检查可以决定某次操作是否允许，却不能让已经返回成功的写入在断电后自动恢复。下一章沿缓存、设备重排、撕裂写和多对象更新逐层寻找真正的持久化提交点。

第 17 章 为什么写成功仍不等于可恢复

继承的机器。文件系统能够恢复自身结构，安全策略能够限制写入主体；应用仍可能在多个文件或记录之间留下半完成版本。内存可见、页缓存变脏、设备报告完成、介质稳定以及恢复程序选择新版本是五个不同边界。

易失状态与持久状态

持久化问题的核心是：系统可能在任意两步之间崩溃。程序调用 `write`，内核可能只把数据放进 `page cache`；设备可能只完成了一部分请求；目录项和文件内容可能以不同顺序持久化；应用日志可能写了意图但没有应用数据；`manifest` 可能发布了一个并不完整的输出。崩溃一致性讨论的不是“正常运行时看起来对不对”，而是“重启后能否解释磁盘状态”。

`write`、`fsync`、`rename` 和 `manifest` 分别处在不同层次。`write` 把字节交给内核；`fsync` 推进文件内容和部分元数据的持久化；`rename` 可以提供目录项替换的原子可见性，但目录本身是否持久也需要考虑；`manifest` 是业务层承认某批输出有效的提交点。只有把这些层次分开，才能写可靠输出。

文件系统常用日志或 `copy-on-write` 技术维护自己的元数据一致性，但这不自动等于应用数据一致。文件系统保证的是它自己的结构能恢复，不保证你的两个文件之间满足业务事务。应用如果需要“数据文件和索引文件同时生效”，就必须设计应用层提交协议。

最小可靠写出协议通常是：写临时文件，写完后 `fsync` 临时文件，校验大小和 `checksum`，`rename` 到最终路径，`fsync` 父目录，最后写或更新 `manifest`。这个协议不是唯一答案，但它展示了 `prepared` 和 `committed` 的区别。

日志恢复要区分 `redo` 和 `undo`。`redo` 日志记录“应该完成什么”，崩溃后可以重放；`undo` 日志记录“如何撤销未完成修改”。`write-ahead logging` 的关键规则是：描述修改的日志必须先于实际修改持久化，否则崩溃后无法判断磁盘上的半成品是否应该保留。

从一次写入逐步得到持久化协议

先不使用同步、日志和事务这些名称。设程序要把文件中的整数从 41 改为 42。最直接的实现是把新字节交给 `write`，看到成功返回后就向用户报告完成。这个结论只有在系统一直运行时才可能成立；如果返回之后立即掉电，重启后仍可能读到 41。

原因在于一次写入会经过多个保存位置。用户缓冲区中的 42 首先被复制或引用，随后进入内核缓存；内核可以稍后才产生设备请求；设备也可能先把请求放进自己的易失缓存，最后才使介质上的内容具备跨掉电保存的能力。每一层都可能说“我已经接收”，但“接收”与“以后仍然存在”不是同一个承诺。

```
user buffer
-> kernel page cache
-> filesystem writeback request
-> device queue or volatile cache
-> non-volatile media
```

因此，讨论“写成功”时至少要回答四个不同问题：

问题	所要求的事实	不能据此推断的结论
----	--------	-----------

可见性	当前或其他执行流已经能读到新值	掉电后仍能读到新值
顺序性	更新 A 在更新 B 之前达到指定边界	A 与 B 作为整体原子生效
持久性	已确认的更新能在规定故障模型下保留	多个更新之间保持业务一致
原子性	观察者只看到操作前或操作后的完整状态	新状态一定已经持久化

这四项不能互相替代。`rename` 可以使路径替换具有原子可见性，却不自动证明目录修改已经跨过持久化边界；`fsync` 可以推动一个文件的内容和相关元数据持久化，却不能自动把两个独立文件合成一项业务事务；日志可以使文件系统结构恢复到一致状态，却不知道应用所说的“版本 42”还包含哪些文件。

为什么需要显式的持久化边界

如果所有写都立即等待介质完成，语义会简单一些，但每一次很小的修改都要承担设备延迟，顺序写合并和缓存复用也难以发挥作用。操作系统因此允许普通写先在内存中完成，把昂贵的设备提交推迟或合并。性能由此提高，同时也产生一个必须显式表达的边界：应用何时确实需要等待数据跨过易失层。

对普通文件来说，这个边界通常由 `fsync` 或更具体的同步接口表达。它不是“再写一次”，而是要求文件系统找出与该文件有关的脏数据和必要元数据，发出写回，等待错误可判定地返回，并在设备支持范围内传播刷新或强制单元访问要求。任意一层忽略这项要求，上层看到的成功就会失去原有含义。

这里还要区分文件内容和名称关系。文件同步主要围绕 `inode`、数据块和必要元数据；新建、删除或改名改变的是父目录保存的名字映射。若程序希望“重启后这个名字一定存在并指向新文件”，只同步文件内容可能不够，还要把目录项变化推进到持久化边界。这就是临时文件发布协议在 `rename` 之后继续同步父目录的原因。

为什么写入顺序也需要证据

设应用先写数据块 `D`，再写一个指向 `D` 的元数据指针 `M`。程序发出 `D` 后再发出 `M`，并不能保证介质按同样顺序完成：内核、块层、控制器和设备都可能为了吞吐重新排序。如果 `M` 先持久化，随后掉电，重启后元数据会指向尚未写好的 `D`。

正确协议必须使“`D` 已达到要求的边界”成为发布 `M` 的前提。可以等待 `D` 完成并刷新后再写 `M`；也可以先把这次修改的描述写入日志并提交；还可以用写时复制把新数据和新索引写到未发布位置，最后切换可恢复的根引用。三种方法的结构不同，但都在增加同一种东西：恢复程序能够依赖的顺序证据。

```
unsafe:
    issue(D)
    issue(M points to D)
    // issue order alone is not durable order

ordered publication:
    write(D)
    wait_until_D_is_durable()
    write(M points to D)
    persist(M)
```

为什么多对象更新需要恢复协议

单个文件创建已经会改变 `inode` 分配状态、`inode` 内容、目录项和可能的数据块分配状态。这些状态不可能依靠一次普通内存赋值同时写入介质。若系统在第二项和第三项之间崩溃，介质就会保存一个正常运行时不应长期出现的中间状态。

系统有两种基本选择。第一种是在重启后扫描全局结构，根据不变量判断并修复中间状态；第二种是在正常更新时额外保存意图与提交证据，使恢复只处理最近尚未归并的修改。前者得到全盘一致性检查，恢复工作量通常与文件系统规模有关；后者逐步得到日志、事务与检查点，恢复工作量主要与尚未归并的记录有关。

这时才有必要引入事务一词。这里的事务不是数据库功能的简单借用，而是一组必须以同一个恢复结论处理的更新。提交记录表示这组更新已经取得发布资格；没有提交记录的尾部不能被恢复程序当作已完成事实。若采用 redo，已提交但尚未全部写回的更新要能够重复执行；若采用 undo，未提交却已经泄漏到目标位置的修改要能够撤销。

故障模型决定保证范围

“绝不丢数据”不是可以脱离条件成立的保证。系统必须先规定考虑哪些故障：进程退出、内核崩溃、整机掉电、控制器丢失缓存、介质返回错误、单盘永久损坏，还是多个副本同时损坏。日志可以处理更新中断，却不能从唯一一块已经永久损坏的介质中恢复原数据；镜像可以承受一个副本丢失，却不能自动判断两个副本中哪个包含更新后的正确字节；校验和能够识别内容与期望不符，却必须配合冗余才有机会修复。

因此，后文所有恢复算法都要同时写明三件事：它把哪些记录视为权威事实，允许介质停留在哪些中间状态，以及超出故障模型后选择拒绝挂载、只读降级、丢弃事务还是请求人工处理。

文件发布、目录同步与 manifest

先把一次发布过程写成提交状态机：

```
NoOutput
-> TempCreated
-> DataWritten
-> DataSynced
-> RenamedVisible
-> DirectorySynced
-> ManifestCommitted
```

每个状态都要写出崩溃后恢复规则：

崩溃点	恢复策略
TempCreated	删除临时文件或忽略
DataWritten	校验失败则删除，不能发布
DataSynced	仍未提交，等待 rename 或清理
RenamedVisible	若 manifest 未提交，按业务规则决定是否采用
DirectorySynced	文件名持久，但业务仍以 manifest 为准
ManifestCommitted	读取者可以信任并校验文件

实践中要避免把“文件存在”当成业务成功。目录中有最终文件名，只说明某次 rename 可能发生过；manifest 提交并通过 checksum，才说明业务层承认它属于某个版本或批次。

再写一份日志协议：

```
append log record: transaction T intends to write block B with checksum C
fsync log
write data block
fsync data
append commit record for T
fsync log
```


恢复时扫描日志：有 commit 的事务必须 redo；没有 commit 的事务按规则忽略或 undo；checksum 不匹配的记录停止或进入人工恢复。这里的重点不是实现数据库，而是理解日志先行、提交记录和幂等恢复。

WAL、检查点与幂等恢复

原子 rename 发布

很多应用用临时文件加 rename 发布结果。算法分为写入、同步、替换、同步目录四步：

```
write_atomic(path, bytes):
    tmp = path + ".tmp." + unique_id()
    fd = open(tmp, O_CREAT | O_EXCL | O_WRONLY)
    write_all(fd, bytes)
    fsync(fd)
    close(fd)
    rename(tmp, path)
    dirfd = open(parent_dir(path))
    fsync(dirfd)
    close(dirfd)
```

这个算法保证读者要么看到旧文件，要么看到新文件，不应看到半个最终文件。但它仍然只是单文件发布。若业务需要多个文件同时生效，还需要 manifest 或事务日志。

manifest 提交

manifest 把多个输出文件绑定成一个版本。数据文件可以先各自写好并 fsync；manifest 最后写入，列出版本号、文件名、大小和 checksum。读取者只读取 manifest 指向的文件。

```
commit_version(version, files):
    for file in files:
        write_atomic(file.tmp_path, file.bytes)
        verify_checksum(file.tmp_path)
        rename(file.tmp_path, file.final_path)
    manifest = build_manifest(version, files)
    write_atomic("MANIFEST.new", manifest)
    rename("MANIFEST.new", "MANIFEST")
```

manifest 的优点是恢复简单：重启后读取 MANIFEST，把它当作权威事实；不在 manifest 里的临时文件或孤儿文件可以清理。缺点是提交粒度较粗，manifest 本身也需要可靠写出。

redo 日志

redo 日志适合“提交后必须能重放”的场景。日志记录足够的信息，使恢复程序可以重新执行已经提交但未完全应用的修改。日志记录必须先持久化，数据页才能写出。

```
transaction_write(T, block, data):
    log_append(INTENT, T, block, checksum(data))
    fsync(log)
    write(block, data)
    log_append(COMMIT, T)
    fsync(log)
```

恢复算法扫描日志：看到 INTENT 但没有 COMMIT 的事务不应用；看到 COMMIT 的事务检查数据是否已经应用，未应用则 redo。redo 操作必须幂等，即重复执行不会改变最终正确结果。

检查点

日志无限增长不可接受，所以系统要做 checkpoint。checkpoint 把当前稳定状态写成快照，并记录“恢复只需从这个日志位置之后开始”。算法难点是 checkpoint 期间仍可能有新事务进入，因此要记录 epoch 或 LSN。

```
checkpoint():
    begin_lsn = log.current_lsn()
    flush_dirty_pages_up_to(begin_lsn)
    write_checkpoint_record(begin_lsn)
    fsync(log)
    truncate_log_before(begin_lsn)
```

checkpoint 的正确性依赖顺序：只有当某个 LSN 之前的脏数据都已经安全写出，才能截断之前的日志。否则崩溃后会丢失 redo 所需信息。

撕裂写、校验和与介质故障

持久化测试必须做故障注入。只在正常路径写文件再读取，不足以证明崩溃一致性。

- 在每个提交状态后模拟崩溃，恢复程序能给出确定结果。
- 临时文件残留不会被读取者当成有效输出。
- manifest 指向缺失文件时，读取者拒绝该版本。
- checksum 错误时，恢复程序能定位坏文件并拒绝提交。
- 重复执行恢复程序是幂等的，不会一次恢复成功、二次恢复破坏状态。
- write 成功但 fsync 失败时，业务提交必须失败。
- rename 后但 manifest 前崩溃，恢复策略必须明确。

测试报告不要只写“断电恢复成功”。必须列出崩溃点、磁盘上允许出现的文件集合、恢复后权威事实、被清理对象和未覆盖边界。可靠性来自状态枚举，不来自运气。

COW、RAID 与多层存储映射

崩溃模型分层

崩溃一致性至少有三类故障：transaction crash，系统在两步之间断电；system crash，内核或整机停止但介质仍可靠；media failure，介质本身丢失或返回坏数据。日志能处理前两类的一部分，但不能让坏盘自动恢复数据，后者需要冗余、校验和副本。

故障	例子	需要的机制
transaction crash	commit 前断电	WAL/journal/manifest 状态机
system crash	内核 panic、掉电	fsync、flush、replay、fsck
media failure	坏块、整盘损坏	RAID、复制、checksum、备份

LSN 与 checkpoint

WAL 系统通常给每条日志记录 LSN。数据页记录 pageLSN，表示该页包含到哪个日志位置的修改。恢复时若 pageLSN 小于 committed LSN，就 redo；checkpoint 记录某个 LSN 之前的脏页已经安全传播。

```
redo_recovery():
    start = last_checkpoint_lsn
    for record in log from start:
        if record.committed and page.page_lsn < record.lsn:
            apply(record)
            page.page_lsn = record.lsn
```

LSN 的价值是让恢复可判定，而不是靠文件时间戳或猜测。数据库、文件系统 journal 和 manifest 版本号都在解决同一问题：给恢复程序一个可比较的事实。

COW 文件系统

btrfs、ZFS 等 COW 文件系统不在原地覆盖元数据，而是写新块，再用新的根指针发布版本。snapshot、clone、send/receive 都建立在“旧版本块仍被引用，新版本写到新位置”的基础上。

```
cow_update(tree, key, value):
    new_leaf = copy_and_modify(old_leaf)
    new_parent = copy_and_point_to(new_leaf)
    ...
    write_all_new_blocks()
    atomically_update_super_root(new_root)
```

COW 避免部分原地覆盖造成的结构破坏，但带来写放大、碎片和空间回收复杂度。snapshot 多时，一个旧块可能被许多版本引用，不能简单释放。

RAID 与 write hole

RAID 0 条带化提高吞吐但无冗余；RAID 1 镜像；RAID 5/6 用奇偶校验节省空间；RAID 10 结合镜像和条带。RAID 5 写入数据和 parity 时若断电，可能出现 write hole：数据和校验不一致。

级别	优点	风险
RAID0	高吞吐、高容量利用	任一盘坏全丢
RAID1	简单冗余，读可并行	容量减半
RAID5/6	容量效率高，有校验	小写放大、write hole、重建压力
RAID10	性能和可靠性较好	成本高

write hole 可用日志、非易失缓存、bitmap 或 copy-on-write parity 缓解。这里再次说明：持久化正确性来自顺序证据和冗余，不来自“写了两次应该差不多”。

LVM 和 device mapper

Linux device mapper 把上层块设备映射到底层设备区域，LVM 的 linear、snapshot、thin provisioning 都建立在此之上。dm-snapshot 用 COW 保存旧块，dm-thin 动态分配块，带来空间耗尽时的错误传播问题。

```
bio to /dev/mapper/vg-lv
-> dm target maps logical sector
-> lower device sector(s)
-> submit cloned bio
-> complete original bio after all lower bios complete
```

设备映射层需要处理 bio 克隆、错误合并、flush 传播和快照元数据一致性。上层的持久化请求只有被所有相关下层正确传播后，才能维持原有保证。

从版本 41 到版本 42：先确定恢复程序必须回答什么

前文已经建立了写回、日志、检查点、写时复制和冗余的基本作用。现在把这些机制放进同一项可复算任务：程序要发布模型版本 42。一个完整版本由三个持久对象组成：`weights.bin` 保存 12MiB 参数，`index.bin` 保存 64KiB 索引，`MANIFEST` 指明当前有效版本及两个数据文件的校验和。版本 41 仍在为读者服务，版本 42 必须满足下列业务不变量：

1. 读取者只能选择版本 41 或完整的版本 42，不能把两个版本的文件混合使用；
2. `MANIFEST` 一旦承认版本 42，两个数据文件就必须存在、长度正确且校验和匹配；
3. 发布程序重复执行、崩溃后重新执行或恢复程序重复扫描，都不能产生第三种结果；
4. 任一同步操作返回错误时，发布者不能把版本 42 报告为已提交。

这四条不是接口名称，而是恢复程序最终必须判断的事实。为了作出判断，系统需要保留“内容是什么”“内容保存在哪里”“哪个版本获得发布资格”和“哪一项证据已经跨过持久化边界”四类状态。把这些状态全部塞进文件名会造成歧义：文件名可以原子替换，却不能同时保存长度、校验和、生成号与提交状态。

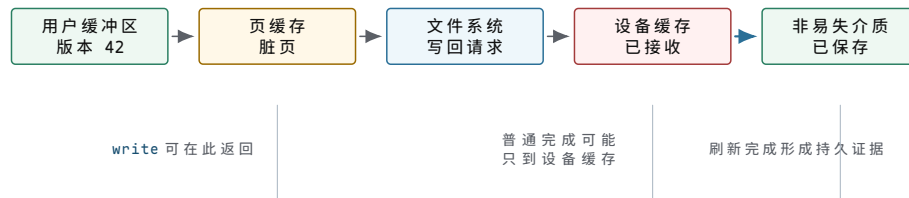
对象	代表字段	所有者与生命周期	提交意义
版本文件	version、length、content checksum	发布者创建；被 manifest 引用期间不得删除	只证明候选内容已经形成
manifest 记录	generation、文件集合、长度、checksum、record checksum	发布者生成；读取者与回收器共同引用	决定读取者承认哪个版本
目录项	name、inode identity、parent directory	文件系统维护；随创建、改名和删除变化	使名称关系可见，不等于业务提交
同步请求	object、requested range、result、error sequence	内核与设备完成路径持有，调用返回后结束	给调用者一项成功或失败证据

五个完成边界不能合并成一个“写好了”。

设 `weights.bin.tmp` 的第一个 4KiB 页面已经从 41 更新为 42。下面五个位置可以同时保存不同版本：

1. 用户缓冲区已经保存新字节；
2. 页缓存已经保存新字节并标为脏页；
3. 文件系统已经把脏页转换为块请求；
4. 设备已经接收请求，但数据仍可能位于易失写缓存；
5. 非易失介质已经保存新字节。

调用者观察到 `write` 返回，只能证明内核接受了指定数量的字节；观察到设备普通写完成，也未必能排除设备易失缓存。只有同步协议把数据、必要元数据和设备刷新要求推进到规定边界，并把下层错误传回调用者，程序才获得相应故障模型下的持久性证据。



图示：图中从左向右是数据传播，不是五次独立复制的强制实现。接口返回点取决于所请求的语义；“被下一层接收”不能替代“在故障后仍可恢复”。

这幅图建立的第一条判断规则是：完成必须带边界。书中后续出现“完成”时，应能回答完成的是复制、排队、设备执行、介质持久化还是业务发布。没有边界限定的“写成功”不能作为恢复协议的前提。

单文件发布：原子可见性如何与持久性配合

先只发布一个配置文件 `config.json`。朴素做法直接截断旧文件再写入新内容。若进程在截断后退出，读者会看到空文件；若系统在写入一半后掉电，读者可能看到短文件。问题不在于 JSON，而在于旧版本的名称已经指向正在被破坏的对象。

临时文件发布把“形成候选内容”和“让最终名称指向候选对象”分成两步。旧对象在候选内容完成前保持不变，名称替换成为单独的发布动作：

```

publish_one(parent_dir, final_name, bytes):
    // 1. 独占创建候选文件；随机后缀避免误用旧临时文件。
    tmp = create_temp_exclusive(parent_dir, final_name)

    // 2. 循环处理短写和可重试中断；只在全部字节被接受后继续。
    write_all_or_fail(tmp, bytes)
    set_required_metadata(tmp)

    // 3. 同步候选对象；任何错误都使本次发布失败。
    if fsync(tmp) != SUCCESS:
        close_and_record_cleanup(tmp)
        return NOT_COMMITTED

    // 4. 在同一文件系统内原子替换名称；读者只见旧对象或新对象。
    if rename(tmp.name, final_name) != SUCCESS:
        close_and_record_cleanup(tmp)
        return NOT_COMMITTED

    // 5. 同步父目录，推进新名称关系的持久化。
    if fsync(parent_dir) != SUCCESS:
        return COMMIT_UNCERTAIN

    return COMMITTED
  
```

算法故意把最后一种结果写成 `COMMIT_UNCERTAIN`。目录同步失败并不证明改名没有发生；当前运行中的读者可能已经看见新名称，但调用者缺少“重启后仍能据此恢复”的证据。可靠程序不能把不确定结果简单重试为一次全新的非幂等操作，而应重新读取名称、对象身份和内容校验和，再决定承认、补做同步或回滚到另一条版本记录。

每个崩溃切点对应一个允许状态集合。

切点	介质上允许存在的对象	重启后权威状态	恢复动作
临时文件创建前	只有旧最终文件	旧版本	无动作

候选同步前	旧文件和不完整临时文件	旧版本	删除或隔离临时文件
候选同步后、改名前	旧文件和完整临时文件	旧版本	校验后重试发布或清理
改名后、目录同步前	当前运行时名称可能指向新对象	证据不足，按协议判定	核对对象身份与校验和，不盲目覆盖
目录同步成功后	最终名称稳定指向新对象	新版本	回收旧对象或旧备份

表中的“允许状态集合”是测试预言，而不是对某次运行结果的猜测。故障注入可以在每个切点停止系统，重启后只要观察结果属于允许集合且恢复动作收敛，协议就满足已声明的故障模型。

关闭文件描述符不是持久化接口。

`close` 结束的是进程对打开文件对象的引用。内核可能在关闭时发现延迟错误，但普通关闭并不普遍承担“把所有脏数据推进到非易失介质”的承诺。进程退出同样不能替代显式同步。如果业务提交依赖掉电后的内容，程序必须在仍能处理错误时请求并检查相应同步结果。

`fdatasync` 通常只推进恢复文件数据所需的元数据，`fsync` 的目标还包括与文件一致性有关的更多元数据；具体范围由系统接口契约决定。二者都不能自动同步另一个文件或把父目录的名称变化合成同一业务事务。接口名字可以缩短调用代码，却不能缩短协议中的对象集合。

多文件发布：manifest 为什么必须成为独立提交对象

回到版本 42。若程序依次改名 `weights.bin` 和 `index.bin`，两个改名之间发生崩溃时，最终目录会混合版本 42 的参数与版本 41 的索引。两个单文件原子操作不能组合成一个多文件原子操作。解决办法不是寻找“更强的文件名”，而是让读者只通过一个独立的小对象选择整组数据。

教学化 manifest 记录可以具有以下字段：

字段	示例	恢复作用
magic 与格式版本	MV1, format=3	拒绝把随机字节或不兼容布局解释为记录
generation	42	比较两个有效候选记录的新旧，不依赖文件时间戳
entry count	2	限定后续变长区域，防止越界解析
文件条目	名称、长度、内容 checksum	证明所选版本的组成与期望字节
previous generation	41	支持回退并检查版本链是否断裂
record checksum	覆盖整个记录	检测 torn write 和静默损坏

manifest 保存业务选择，目录项保存名称映射，inode 保存文件身份。三个对象必须分开，因为它们的修改频率、所有者和恢复意义不同。读者不应扫描目录后“挑看起来最新的文件”，而应先验证 manifest，再严格读取它列出的对象。

```
commit_version_42(files):
    // 1. 每个数据文件使用不可复用名称形成候选，例如 weights.v42。
    for file in files:
```



```

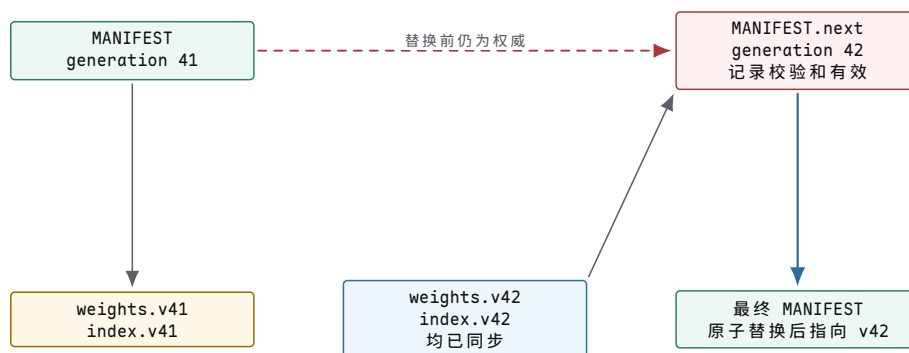
create_exclusive(file.versioned_name)
write_all_or_fail(file.bytes)
fsync_or_fail(file.fd)
verify_length_and_checksum(file.versioned_name)

// 2. 同步目录，使候选名称可在重启后找到；此时尚未业务提交。
fsync_or_fail(files_directory)

// 3. 构造只引用已验证候选文件的新 manifest。
record = make_manifest(
    generation = 42,
    previous = 41,
    entries = files.lengths_and_checksums)
write_atomic_and_sync("MANIFEST.next", record)

// 4. 最后替换权威入口，并同步其父目录。
rename_or_fail("MANIFEST.next", "MANIFEST")
fsync_or_fail(manifest_directory)
return COMMITTED

```



图示：候选数据文件先形成，manifest 最后发布。虚线表示替换前旧 manifest 仍是唯一权威入口；目录中出现 v42 文件并不等于读者已经选择版本 42。

读取路径也必须验证提交证据。

```

open_current_version():
// 1. 只从权威名称取得 manifest，并先验证记录自身。
m = read_exact("MANIFEST")
require(m.magic == "MV1")
require(checksum(m.without_record_checksum) == m.record_checksum)

// 2. 按 manifest 列表逐一验证，不扫描目录猜测版本。
opened = []
for entry in m.entries:
    fd = open_no_follow(entry.name)
    require(file_identity_is_stable(fd))
    require(length(fd) == entry.length)
    require(checksum(fd) == entry.content_checksum)
    opened.append(fd)

// 3. 全部验证以后才向上层发布同一代对象集合。
return VersionHandle(m.generation, opened)

```

读取者在验证期间持有文件引用，避免名称在“检查以后、使用以前”被替换。稳定引用解决运行期竞态，checksum 解决内容证据，generation 解决版本选择；任何一个都不能替代另外两个。

WAL：把恢复决定写在目标页之前

manifest 适合整组不可变文件的粗粒度发布。若程序每秒修改许多 4 KiB 页面，每次重写整组文件代价过高。此时目标页会被反复更新，恢复程序需要知道哪些更新已经获得提交资格。写前日志（write-ahead log, WAL）用顺序追加记录保存这项决定。

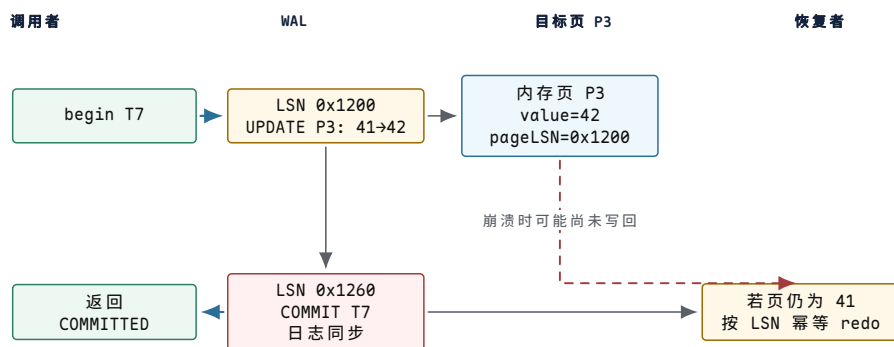
设事务 T7 把页 P3 的计数从 41 改为 42。日志记录不能只保存一句“更新成功”，至少要能识别事务、顺序位置、目标对象、重做内容和记录完整性：

字段	示例	必要性
LSN	0x1200	为记录建立全局可比较顺序
transaction id	T7	把多条页面修改归入同一提交决定
record type	UPDATE/COMMIT	区分数据描述、提交证据与恢复补偿
page id 与 offset	P3, 128	指定重做或撤销作用位置
before/after image	41/42	支持所选恢复策略所需的 undo 或 redo
previous LSN	0x1180	沿单个事务反向查找记录
length 与 checksum	96 B, C7	识别日志尾部 torn record，不越界解析

两条写前规则。

若系统允许脏目标页在事务提交前写回，恢复可能需要撤销未提交值，因此必须在目标页写回前先持久化足够的 undo 信息。若系统宣布 T7 已提交，恢复可能需要重做尚未写回的目标页，因此必须在向调用者返回提交成功前先持久化 T7 的 redo 信息和提交记录。写成不变量就是：

1. durableLSN $\geq$ pageLSN 后，包含该 pageLSN 的目标页才有资格写到原位置；
2. T7 的 COMMIT 记录跨过日志持久化边界后，系统才可以向事务调用者返回提交成功。



图示：提交资格来自已持久化的 COMMIT 记录，不来自目标页是否碰巧先写回。恢复者比较日志 LSN 与 pageLSN：需要时重复执行 redo，已经应用的更新则跳过。

带 pageLSN 的幂等重做。

```

redo_update(record, page):
    // 日志记录先通过长度、类型和 checksum 校验。
    require(record.type == UPDATE)
    require(record.is_committed)

    // pageLSN 已达到或超过本记录，说明效果已经包含在页中。
    if page.page_lsn >= record.lsn:
        return ALREADY_APPLIED
  
```

```
// 在固定偏移写入 after image，并同时推进页面证据。
page.bytes[record.offset : record.offset + record.length] =
    record.after_image
page.page_lsn = record.lsn
mark_dirty(page)
return APPLIED
```

幂等不是“多执行几次大概没关系”，而是存在可检查的前置条件和单调证据。恢复可能在重做一半时再次崩溃；下次启动仍从检查点扫描，只要 pageLSN 与日志记录遵循单调规则，重复扫描就会收敛到同一个结果。

日志尾部不完整时如何停止。

掉电可能只写入一条记录的前 37 字节。恢复程序必须先读取固定头部，验证长度上限，再读取完整记录并校验 checksum。首条不完整或校验失败的尾部记录之后没有可信边界，恢复程序不能在随机字节中搜索一个看似合法的 COMMIT 标记继续执行。

```
scan_log(from_lsn):
    // valid_end 只推进到连续、完整且校验成功的记录之后。
    valid_end = from_lsn
    while header_is_complete(valid_end):
        header = read_header(valid_end)
        if header.length < MIN_RECORD or header.length > MAX_RECORD:
            break
        record = read_exact_or_stop(valid_end, header.length)
        if checksum(record) != header.checksum:
            break
        index_record(record)
        valid_end += header.length
    return valid_end
```

检查点与三阶段恢复：缩短扫描不能删除必要证据

WAL 只追加不截断会无限增长。检查点的目标是证明某个日志位置以前的状态已经由另一组稳定对象承接，而不是机械删除旧文件。若检查点期间事务仍在运行，系统需要记录活跃事务与脏页状态；否则恢复无法判断从哪里开始 redo、哪些事务还需要 undo。

检查点字段	代表内容	恢复用途
checkpoint LSN	0x2000	找到最近一份完整检查点记录
transaction table	T7=COMMITTED, T8=ACTIVE, lastLSN=...	判断崩溃时哪些事务已经提交或仍需收束
dirty page table	P3=recLSN 0x1200, P9=0x1F20	找到每页可能首次尚未写回的日志位置
checkpoint checksum	覆盖记录与变长表	排除 torn checkpoint
previous checkpoint	0x1800	当前检查点损坏时回退到前一有效入口

恢复可分为分析、重做和撤销三阶段。分析重建“崩溃时有哪些事务和脏页”；重做从最早必要位置重复历史，使已提交结果与恢复所需的中间状态重新出现；撤销沿未提交事务的日志链反向执行，并写补偿日志记录（compensation log record, CLR），使撤销本身也可以在再次崩溃后继续。

```
recover(last_checkpoint):
    // 1. 分析：恢复事务表、脏页表以及连续有效日志尾。
```

```

state = analysis_pass(last_checkpoint)

// 2. 重做：按 LSN 正序检查；pageLSN 已覆盖的记录直接跳过。
for record in log_from(state.smallest_rec_lsn):
    if record.needs_redo and page_lsn(record.page) < record.lsn:
        redo_update(record, load_page(record.page))

// 3. 撤销：只处理崩溃时未提交的事务，按事务链逆序进行。
for transaction in state.loser_transactions:
    undo_with_compensation_records(transaction)

// 4. 所有恢复记录同步后，才发布“存储已可服务”。
fsync_or_fail(log)
return RECOVERY_COMPLETE

```

为什么恢复期间仍要写日志。

若恢复撤销 T8 的两个修改，在第一个修改撤销后再次掉电，下一次启动必须知道第一项撤销已经完成。CLR 保存“哪条更新已经被补偿”和“下一条待撤销记录在哪里”。它通常只需 redo，不再被 undo，因而撤销进度单调前进。没有这项证据，恢复程序只能从头猜测，甚至可能反复翻转同一字节。

检查点截断的安全条件。

日志前缀只有在以下信息都不再依赖它时才能回收：

- 任何未提交事务的 undo 链都不再指向该前缀；
- 任何脏页的最早未传播更新都不再位于该前缀；
- 至少一份完整、可定位且校验有效的检查点已经持久化；
- 旧检查点入口的替换本身具有可恢复规则。

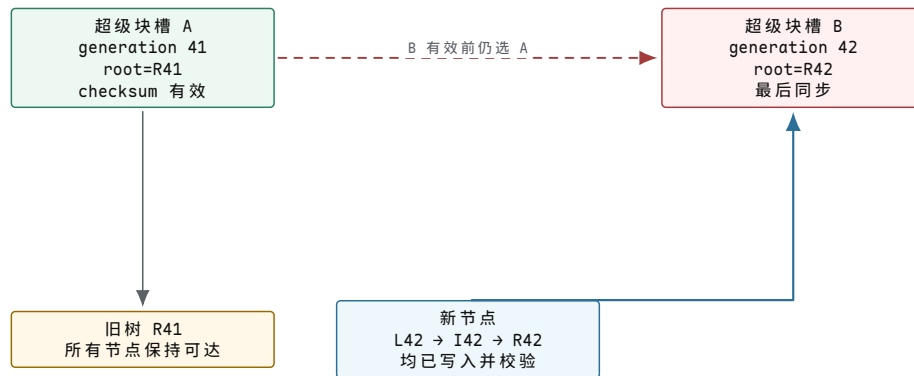
“脏页已经发出写请求”不足以满足第二项；系统必须获得目标边界的完成证据。日志截断发生过早会把性能优化变成数据丢失，发生过晚只会占用更多空间，因此正确性条件必须先于回收策略。

写时复制：根引用是提交记录，不是普通指针

写时复制 (copy-on-write, COW) 把新页面写到未发布位置，沿索引路径逐层生成新节点，最后切换根引用。旧根在切换前始终描述完整旧版本。这个结构减少原地覆盖产生的中间状态，却没有消除顺序要求：所有新节点必须先于新根获得持久化证据。

设根 A 的 generation 为 41，新建叶 L42、内部节点 I42 和根 R42。安全发布顺序为：

1. 写入 L42，并在节点头保存 generation、owner、长度和 checksum；
2. 写入引用 L42 的 I42，再写入引用 I42 的 R42；
3. 等待所有新节点跨过所需持久边界；
4. 把超级块槽 B 写成 generation 42，保存 R42 地址和 checksum；
5. 同步槽 B；恢复时在所有校验有效的槽中选择最高 generation。



图示：恢复程序不根据“哪个槽最后写过”猜测新旧，而是先验证槽与根节点，再比较 generation。若槽 B 是 torn write，槽 A 仍提供完整旧版本。

根切换以后仍有回收债务。

根 B 发布后，旧树不再是当前版本，却可能仍被快照、打开引用或后台发送任务使用。空间回收必须根据引用计数、延迟引用或可达性证据判断，不能在根切换时立即释放整棵旧树。否则一次逻辑上正确的提交会在并发读取或快照恢复中制造悬空引用。

COW 还会产生碎片和写放大：修改一个叶中的几个字节可能重写整条树路径。只有当基本根切换协议成立后，系统才可以用节点打包、延迟引用、日志树或批量提交减少成本；这些优化不能改变“子节点先稳定、根最后发布”的恢复不变量。

设备顺序契约：请求完成、刷新与 FUA 各证明什么

文件系统最终通过块层向设备提交请求。即使文件系统按 D 后 M 的顺序发出，设备也可能并行执行或把结果留在易失缓存。上层必须把所需顺序翻译成设备能够执行的命令，并把下层失败完整合并回原请求。

动作	能建立的证据	不能单独证明的事项
普通写完成	设备已按接口完成该命令	数据一定越过设备易失缓存
flush 完成	flush 之前受其覆盖的写被推进到规定边界	flush 之后才发出的写已经持久
FUA 写完成	该写按设备契约绕过或推进易失缓存	其他没有 FUA 的写也获得相同保证
队列 barrier/依赖	后续命令不能越过指定顺序点	顺序点之前的命令一定成功
设备错误完成	下层无法给出所请求的成功证据	目标介质完全没有发生任何部分修改

文件系统需要把一次高层同步拆成数据写、元数据写、flush 或 FUA 等下层动作，并记录它们之间的依赖。若映射层把一个上层请求克隆到两块下层设备，只有所有必要子请求都满足协议，上层才能报告成功。任一层未传播 flush、提前完成父请求或忽略迟到错误，都会破坏更高层的持久性含义。

同步失败后不能假装回到操作前。

`fsync` 返回错误表示系统没有取得请求的持久性证据，不表示所有写都没有落盘。部分页面可能已持久，另一些仍在缓存或已经失败。应用必须把该版本标为未提交或不确定，通过 manifest、日志或下一次恢复检查重新判定，不能假设简单重写最后一个系统调用就能恢复原子性。

错误也可能延迟到写回或同步时才出现。写入调用已经返回的字节仍可能在后台写回中失败，因此可靠程序要把同步结果纳入提交状态，并保存足够上下文说明失败对应哪个版本、对象和操作范围。

DAX 与持久内存只缩短路径，不取消顺序。

直接访问 (DAX) 可以让 CPU 通过地址映射访问持久介质，路径中没有普通页缓存与块请求。但 CPU store 可能仍停留在缓存层级或以不同顺序到达持久域。软件必须使用平台规定的写回、持久化屏障和原子写粒度，并明确平台把哪一层纳入掉电保护。少了一层队列不等于少了提交协议。

校验、冗余与修复：发现错误和恢复数据是两项能力

checksum 能回答“读出的字节是否符合记录时的摘要”，不能凭空生成正确副本。镜像能提供另一个候选副本，却必须用 generation、checksum 或更高层记录判断哪个副本可信。恢复系统因此需要把检测证据与修复来源配对。

机制	能发现或承受的故障	仍需的证据	失败后的动作
块 checksum	torn write、静默位翻转	checksum 本身受保护且绑定块身份	拒绝或寻找副本
双副本镜像	一个副本不可读或校验失败	哪个副本 generation 合法	从好副本读并修复坏副本
奇偶校验	规定数量的数据盘故障	条带成员身份与一致 generation	重建并校验
快照	逻辑误删后的旧版本可达	快照创建点本身已提交	选择旧根，不等同于异地备份
备份	主系统永久损坏	备份完整性、时间点和恢复演练	在独立介质重建服务

镜像读取的具体判定。

设块 B 的两个副本都能读取。副本 0 保存 generation 42 但 checksum 失败；副本 1 保存 generation 41 且 checksum 有效。系统不能因为 42 较新就返回损坏字节，也不能在不知道提交记录的情况下直接把 41 宣布为当前版本。它先查权威根或日志：若版本 42 已提交却两个有效副本均缺失，结果应是不可恢复错误；若版本 42 尚未提交，副本 1 才可能是正确旧状态。

后台 scrub 会主动读取数据、验证 checksum，并在冗余允许时修复坏副本。scrub 缩短潜伏错误存在时间，但不能替代在线写协议。若所有副本都以同样方式写入错误内容且重新计算了 checksum，块级校验仍会认为它们一致；端到端校验和业务不变量仍然必要。

RAID write hole 的状态推演。

一个条带由数据 D0、D1 和 parity P 组成。更新 D0 需要使 P 与新 D0、旧 D1 匹配。若先写新 D0，随后掉电而 P 仍是旧值，三者都可能单独可读，却不再满足 parity 方程。重建另一块盘时会生成错误内容。日志化条带更新、非易失写缓存、写意图位图或 COW parity 都是在为“D0 与 P 属于同一代”增加恢复证据。

故障注入：把协议变成可穷举的状态空间

正常运行十万次不能覆盖“恰好在提交记录的第 37 字节掉电”。崩溃一致性测试要控制持久化模型：记录每次写、刷新和根切换，在任意两个持久事件之间截断执行，然后从截断后的介质镜像启动真实恢复代码。

```

enumerate_crash_points(operation):
    // 基线运行只记录持久事件序列，不把内存可见误作介质可见。
    trace = run_with_persistence_recorder(operation)

    // 每个 cut 构造一次可能的介质状态，并从冷启动路径恢复。
    for cut in 0 .. trace.persistence_events.count:
        image = materialize_media_state(trace, cut)
        recovered = cold_boot_and_recover(image)

```



```
// 预言只允许旧版本或完整新版本；恢复再次运行也必须相同。
require(recovered in {VALID_V41, VALID_V42})
require(cold_boot_and_recover(image) == recovered)
```

测试模型必须与声明的故障范围一致。若设备允许扇区 torn write，模型要枚举部分扇区；若接口只保证某个原子写粒度，就不能假设更大的记录一次完成；若设备 flush 可能失败，测试要注入失败并检查上层没有发布成功。模型比真实设备更强或更弱，都会得到误导性结论。

测试维度	注入方式	必须核对的观察结果
进程崩溃	在系统调用之间终止发布者	内核继续运行，未提交候选不被读取者采用
内核崩溃	在写回、日志提交或根切换处停止	冷启动恢复只选择允许版本
短写与 EINTR	限制单次写入长度、注入可重试中断	循环写入不丢失偏移，失败不发布
同步错误	让文件或目录同步返回错误	业务状态保持未提交或不确定
torn record	只保留记录前缀或单个扇区	checksum 使扫描停在连续有效日志尾
迟到完成	超时后再交付旧设备完成	generation 阻止旧完成命中新请求
静默损坏	翻转数据但不更新 checksum	读取路径拒绝损坏并按冗余策略修复或报错

测试报告要保存因果证据。

一次失败报告至少包含：随机种子、初始介质镜像、持久事件序列、崩溃 cut、注入错误、恢复日志、最终权威 generation、孤儿对象集合和不变量检查结果。只有保留这些信息，开发者才能重放同一失败并判断错误来自发布协议、恢复算法还是测试模型。

完整复盘：版本 42 在哪里才算完成

现在沿对象和边界重新追踪开篇任务：

1. 发布者分别创建带 generation 的不可变数据文件。每个文件在写入后校验长度与内容 checksum，并检查文件同步结果。此时它们只是候选对象。
2. 发布者同步候选文件所在目录，使重启后的恢复程序能够按记录名称找到它们。目录中出现候选文件仍不是业务提交。
3. 发布者构造 generation 42 的 manifest，记录前一代、文件集合、长度、内容 checksum 和记录 checksum。manifest 只引用已经取得持久性证据的候选对象。
4. 临时 manifest 同步完成后，发布者原子替换权威 MANIFEST 名称，再同步父目录。目录同步成功是本协议向调用者返回 COMMITTED 的边界。
5. 读取者先稳定引用 manifest，再验证记录和全部文件。全部验证成功后才构造 VersionHandle(42)；在此之前，任何 v42 文件都不能泄漏给业务代码。
6. 崩溃恢复只在校验有效的 manifest 中选择 generation，忽略或回收未被权威版本引用的候选文件。恢复重复执行仍选择同一代。

7. 若同步、校验或设备层返回错误，发布者保存未提交或不确定状态。校验和负责检测，冗余负责在存在可信副本时修复；没有可信副本时必须报告不可恢复错误。

这条链路把四个容易混淆的结论分开：字节被接受、对象可见、介质持久和业务提交。WAL 用 COMMIT 记录承担细粒度更新的发布证据，COW 用校验有效的新根承担树形状态的发布证据，manifest 用一个小型权威记录承担多文件版本的发布证据。三种结构不同，却共享同一条恢复原则：先形成完整候选状态，再持久化足以重建决定的证据，最后才向观察者发布提交结果。

恢复结束以后，服务仍要经过开放门槛。

恢复程序返回并不自动表示所有接口都可以接收写请求。日志可能已经收束，但空间索引仍在核对；主副本可能可读，但后台仍在重建冗余；文件系统结构可能足以只读挂载，却不足以安全分配新块。系统应把“恢复算法完成”和“允许哪类服务”分成两个状态。

开放门槛	必须成立的事实	不满足时的服务状态
权威根可选	至少一个根或 manifest 的格式、generation 与 checksum 有效	拒绝装载，不能扫描目录猜测版本
日志已收束	所有 winner 已 redo，loser 已 undo 或隔离，恢复记录已同步	保持恢复模式，不接受普通读写
空间归属可判定	已分配、空闲、延迟回收和孤儿对象之间没有重复归属	只读开放或继续一致性检查
读取完整性可验证	被权威版本引用的数据有校验依据，损坏有明确报错路径	允许健康对象读取，损坏对象返回可定位错误
写入冗余满足策略	所需副本或 parity 已可更新，降级写规则已经明确	只读服务、受限写入或等待重建

开放状态必须可观察：启动日志记录所选 generation、检查点 LSN、未完成恢复项、降级原因与允许的操作集合。这样，下一章面对“服务启动很慢”“目录可读但写入失败”或“重建期间延迟升高”等症状时，才能把用户现象连接到一个具体恢复阶段，而不是把所有启动问题统称为存储故障。

尚未解决的问题。系统已经规定恢复协议，但真实故障首先表现为延迟、错误码、停顿或崩溃。单个症状不能直接指出原因；最后一章建立指标、事件、调用链、转储和故障注入之间的证据边界。

第 18 章 为什么症状不能直接说明原因

继承的机器。前十七章已经为执行、内存、I/O、文件、网络、安全和恢复建立对象、状态转换与提交边界。用户看到的一次超时可能来自等待 CPU、锁、页面、设备、远端确认或持久化；必须先知道证据在哪一层形成，才能把相关性收束为因果解释。

系统会出错、会 panic、会变慢。本章把错误路径收敛、崩溃恢复和可观测性工具放在一起：从 `errno` 到 `kdump`，从 `printk` 到 `eBPF`，从现象到证据到结论。

错误、oops 与 panic

操作系统必须区分普通错误、进程错误、内核 bug 和硬件不可恢复错误。普通错误应返回 `errno`，例如文件不存在、权限不足、非阻塞 I/O 暂时不可用；进程错误应向进程发送信号，例如非法地址访问；内核 bug 可能触发 `WARN`、`oops` 或 `panic`；硬件错误可能要求隔离设备、下线页面或重启系统。

常见误区

若把所有错误都 `panic`，系统不可用；若把所有错误都吞掉，数据会被悄悄破坏。错误分类的责任在于明确每个失败点：哪些返回 `errno`，哪些终止进程，哪些停止系统。

`panic` 的含义是内核认为继续运行比停止更危险。典型原因包括核心不变量破坏、调度器状态自相矛盾、文件系统元数据检测到不可恢复损坏、内核栈溢出、无法处理的机器检查。`panic` 不是调试打印的高级版本，而是系统级 `fail-stop` 策略：停止服务，尽量保留证据，避免进一步破坏状态。

核心思想

高质量 OS 实现不只写正常路径，还要把每个失败点的责任写清楚：返回错误、终止进程、重试、降级、隔离，还是 `panic`。

错误路径的工程难点在资源回收。系统调用可能已经分配页、获取锁、增加引用、插入表项、发起 I/O，然后第七步失败。正确实现必须让失败路径按照相反顺序撤销已完成步骤，并且不能撤销还没完成的步骤。C 语言内核常用 `goto out_...` 标签表达这种线性回滚；C++ 内核或教学模拟器可以用 `RAII` 表达资源所有权，但必须注意中断上下文、`panic` 路径和 `no-throw` 约束。

转向可观测性：操作系统问题很少只靠读代码解决。线程为什么不运行，可能是调度等待、锁等待、I/O 阻塞、`page fault`、信号、`cgroup` 限流或下游服务；内存为什么上涨，可能是真实泄漏、`allocator` 缓存、`page cache`、`mmap`、线程栈或内核 `buffer`；写文件为什么慢，可能是 `page cache` 脏页、设备队列、`fsync`、目录同步或日志提交。没有观测证据，结论只是猜测。

可观测性要把 OS 状态映射到证据。进程和线程状态可以从 `/proc`、`ps`、`top`、调度 `trace` 观察；系统调用可以用 `strace` 观察；CPU 热点可以用采样 `profiler`；`page fault`、`context switch`、I/O 等待和网络状态可以用系统计数器和 `tracing` 工具；应用层必须补充业务 `trace`，记录请求身份、队列等待、状态机阶段和错误码。

核心思想

可观测性的目标不是收集越多越好，而是把 OS 状态映射到证据：on-CPU 解释运行时消耗，off-CPU 解释等待时间，两者闭合才能定位问题。

调试时要区分 on-CPU 和 off-CPU。on-CPU 说明线程正在执行指令；off-CPU 说明线程没有运行，可能在等待。CPU profiler 只能解释运行时消耗在哪里，解释不了大部分时间都在睡眠的请求。尾延迟问题常常要从 off-CPU 等待开始看。

内存调试要区分 virtual size、resident set、anonymous memory、file-backed page、page cache、allocator arena 和 kernel memory。malloc 后不触页，可能只增加虚拟地址；首次写入才分配物理页；释放内存后 RSS 不马上下降，也可能是 allocator 保留 arena。

常见误区

把所有 RSS 增长都叫泄漏，是不合格诊断。虚拟地址、物理页、page cache、allocator arena 和内核内存各有不同来源，必须先分清再下结论。

I/O 调试要区分 buffered I/O、direct I/O、mmap、page cache hit/miss、dirty writeback、fsync 和设备吞吐。一次写很快，可能只是进入 page cache；一次读很快，可能只是热缓存；一次 fsync 很慢，可能是在偿还之前累计的脏页。实验必须控制冷缓存和热缓存，否则数字不可比较。

从症状建立证据链

错误分类可以写成下表：

类别	处理方式	示例
可预期用户错误	返回 <code>errno</code>	<code>ENOENT</code> 、 <code>EACCES</code> 、 <code>EAGAIN</code>
用户进程违规	信号或终止该进程	page fault 权限错误、非法指令
资源暂时不足	返回错误、等待或回收	<code>ENOMEM</code> 、队列满、端口耗尽
设备局部故障	重置设备、隔离队列、上报 I/O error	磁盘超时、网卡 reset
内核不变量破坏	<code>WARN/oops/panic</code>	双重释放、runqueue 计数错误
持久化一致性风险	<code>remount read-only</code> 、 <code>replay journal</code> 、 <code>panic</code>	元数据校验失败

一个系统调用实现应在设计阶段列出失败点：

```
open(path, flags):
    copied_path = copy_path_from_user(path)           may fail EFAULT/ENAMETOOLONG
    parent = resolve_parent(copied_path)               may fail ENOENT/EACCES
    inode = lookup_or_create(parent, name, flags)      may fail ENOENT/EXIST/EROFS
    file = alloc_file(inode, flags)                   may fail ENOMEM
    fd = alloc_fd(current.files)                       may fail EMFILE
    install_fd(fd, file)                               commit point
    return fd
```

提交点之前失败，要释放已拿到的引用；提交点之后失败，通常不能再返回“没有打开”，而要让 fd 表保持一致或执行明确关闭。把提交点写出来，可以避免半初始化 fd 留在表里。

本章的实践模板是“现象、假设、证据、反证、结论”。

字段	请求延迟升高示例
现象	p95 从 20ms 升到 500ms
假设	worker 等锁或等 I/O
证据	trace 中 queue_wait 低, blocked_io 高, 线程睡在 writeback
反证	on-CPU profiler 没有热函数占用 500ms
结论	延迟主要来自 I/O 完成等待, 不是 CPU 算法变慢

程序卡死时, 先列线程状态: 哪个线程持有锁, 哪个线程等待条件变量, 哪个线程在 join, 关闭标志是否设置, 是否有线程仍可能唤醒等待者。若所有消费者都睡在 not_empty, 而关闭路径只通知 not_full, 就能定位到等待谓词和通知对象不匹配。

实践中要保留原始证据: 命令、时间、输入规模、机器信息、内核版本、容器限制、采样频率、日志片段和无法观察的字段。没有这些, 报告不可复现。

指标、事件、调用链与转储

errno 返回与用户可见契约

errno 不是随便选的数字。调用者会根据错误码决定重试、降级、提示用户还是终止。

```
copy_from_user_checked(dst, user_ptr, len):
    if not user_range_valid(user_ptr, len, READ):
        return EFAULT
    if len > MAX_COPY:
        return EINVAL
    fault = copy_may_fault(dst, user_ptr, len)
    if fault:
        return EFAULT
    return OK
```

把非法用户地址返回 EFAULT, 而不是 panic, 是因为这是用户进程可触发的预期错误。内核必须把“不可信输入”当正常情况处理。只有当内核自己的页表、锁状态或对象引用被破坏时, 才进入更严重路径。

goto 回滚模板

C 风格内核常见模式:

```
sys_create(path):
    name = null
    inode = null
    file = null

    name = copy_name(path)
    if name == null:
        ret = ENOMEM
        goto out

    inode = create_inode(name)
    if is_error(inode):
        ret = error(inode)
        goto out_name

    file = alloc_file(inode)
```



```

if file == null:
    ret = ENOMEM
    goto out_inode

ret = install_fd(file)
if ret < 0:
    goto out_file
return ret

out_file:
    put_file(file)
out_inode:
    put_inode(inode)
out_name:
    free(name)
out:
    return ret

```

这个结构的优点是回滚顺序与获取顺序相反，且每个标签只释放已经成功获取的资源。缺点是标签命名和跳转边界容易腐烂，需要测试每个失败注入点。C++ RAII 可以减少样板，但内核中不是所有路径都适合异常；析构函数也不能在持有自旋锁或 panic 路径中做复杂阻塞操作。

WARN、BUG、oops 与 panic

可以把严重程度分成四级：

机制	含义	处理
WARN	发现异常但可能继续	打印栈、计数、继续或降级
BUG/oops	当前内核路径不可继续	杀死当前进程或停止当前 CPU，视上下文而定
panic	整个系统不可安全继续	停机、重启或进入 crash dump
watchdog	系统长时间无响应	触发诊断、中断栈采样或 panic

教学系统可以实现 `assert_kernel_invariant`：调试构建 panic，发布构建记录错误并尽量隔离。但对文件系统元数据损坏这类风险，继续写入可能扩大破坏，`remount read-only` 或 panic 是合理选择。

看门狗与故障转储

watchdog 检测“没有前进”。硬锁死可能表现为中断不再发生；软锁死可能表现为某 CPU 长时间不调度；RCU stall 表示读侧临界区过久导致回收无法完成。

```

watchdog_tick(cpu):
    now = time()
    if now - cpu.last_schedule_time > SOFTLOCKUP_LIMIT:
        dump_cpu_stack(cpu)
        warn("soft lockup")
    if now - cpu.last_interrupt_time > HARDLOCKUP_LIMIT:
        panic("hard lockup")

```

panic 后的目标不是恢复服务，而是保留证据：panic 原因、CPU 寄存器、栈、当前任务、锁持有、最近日志、脏页状态和设备错误。crash dump 的价值在于让下一次启动后能分析，而不是在 panic 路径中做复杂修复。

重试与幂等

不是所有失败都应该立即返回。临时内存不足可以触发回收后重试；设备超时可以 reset 后重试；网络发送可以等待窗口。但重试必须有边界，并且操作要么幂等，要么有明确的提交记录。

```
retry_io(req):
    while req.retries < MAX_RETRIES:
        submit(req)
        status = wait_completion(req, timeout)
        if status == OK:
            return OK
        if not is_transient(status):
            return status
        reset_path_if_needed(req.device)
        req.retries++
    return EIO
```

对写请求，若设备状态不明，简单重试可能造成重复写。对普通块覆盖写通常可接受；对“追加一条记录”这类上层操作，必须有序列号、校验或日志来识别重复提交。

事件日志

最简单的可观测性算法是事件日志。每个关键状态转换追加一条结构化记录：时间、线程、请求 ID、事件类型、对象 ID、旧状态、新状态和错误码。日志不能只写自然语言，否则无法聚合和验证。

```
record_event(req_id, object, old_state, new_state, reason):
    entry.ts = monotonic_time()
    entry.cpu = current_cpu()
    entry.thread = current_thread()
    entry.req_id = req_id
    entry.object = object
    entry.old_state = old_state
    entry.new_state = new_state
    entry.reason = reason
    ring_append(trace_buffer, entry)
```

事件日志本身也要有背压。无限日志会把故障变成磁盘或内存问题。常见做法是固定大小环形缓冲区、按级别采样、按请求 ID 打开详细追踪，或者把关键错误同步写入可靠日志。

延迟分解

请求延迟不能只记录总耗时。一个可行动的分解至少包括 queue wait、runnable wait、on CPU、blocked on lock、blocked on I/O、downstream wait 和 cleanup。实现上可以在状态转换处打点，然后按请求 ID 聚合。

```
transition(req, new_state):
    now = clock()
    req.time_in_state[req.state] += now - req.last_ts
    req.state = new_state
    req.last_ts = now
```

这个算法的复杂度是每次状态转换 $O(1)$ ，但要求状态枚举稳定。若所有等待都记成 blocked，后续无法判断是锁、I/O 还是 page fault。

off-CPU 分析

很多性能问题不在 CPU 火焰图里，因为线程大部分时间不在 CPU 上。off-CPU 分析记录线程从运行态切到睡眠态的原因，以及被谁唤醒。

```

on_schedule_out(task, reason, wait_object):
    task.offcpu_start = clock()
    task.wait_reason = reason
    task.wait_object = wait_object

on_wakeup(task, waker):
    delta = clock() - task.offcpu_start
    offcpu_histogram[task.wait_reason].add(delta)
    record_wakeup_edge(waker, task, task.wait_object)

```

这个数据能区分等锁、等 I/O、等页、等网络、等定时器和等子进程。若 p99 延迟主要来自 off-CPU 的 writeback 等待，优化业务 CPU 代码不会解决问题。

资源对账

资源泄漏调试需要对账算法。每次 open 增加 fd 计数，每次 close 减少；每次分配对象记录 owner，每次释放删除记录；系统退出或测试结束时检查剩余对象。

```

track_alloc(id, type, owner):
    live[id] = {type, owner, stack}

track_free(id):
    live.erase(id)

assert_no_leak():
    for each object in live:
        report(object)

```

真实系统不能给所有对象都保存完整 stack，因为成本太高；可以采样、按类型开启、只在测试构建开启。但思想不变：资源必须能被计数，生命周期必须能闭合。

最小可复现报告

高质量调试报告要让别人能复现判断。最小格式如下：

```

Problem:
    现象、影响范围、首次出现时间

Environment:
    内核版本、硬件/容器限制、文件系统、负载规模

Timeline:
    关键事件按时间排序

Evidence:
    系统调用、trace、线程状态、资源计数、日志、指标

Rejected hypotheses:
    已经排除的解释和证据

Conclusion:
    当前最能解释现象的机制

Boundary:
    仍未验证的假设和下一步实验

```

报告的价值不在于一次猜中，而在于缩小不确定性。写出反证能防止团队反复讨论已经排除的方向。

故障注入与因果验证

可观测性与错误路径都需要测试。一个系统若只能在成功时输出日志，失败时没有状态，就不可维护。

1. `errno` 矩阵：非法指针、权限不足、资源耗尽返回不同错误码。
2. 失败注入：在系统调用每个分配点失败，引用计数和锁计数最终归零。
3. 提交点测试：提交前失败无用户可见副作用，提交后状态自治。
4. `WARN` 测试：非致命不变量异常记录证据但不破坏后续测试。
5. `panic` 测试：核心不变量破坏后停止调度，输出原因和栈。
6. `watchdog` 测试：构造关中断死循环或不调度循环，能触发诊断。
7. 设备重试：临时错误可重试，永久错误返回 `EIO` 且唤醒等待者。
8. `crash dump`：`panic` 后保存当前任务、寄存器、锁和最近日志。
 - 每个长时间等待点都能输出等待原因或 `trace` 事件。
 - 每个请求都有稳定身份，能串起进入队列、开始执行、阻塞、完成和失败。
 - 错误码保留原始原因，不被统一改写成 `unknown error`。
 - 资源计数可对账：打开 `fd` 数、活跃线程数、队列长度、`in-flight I/O`、分配对象数。
 - 性能测试区分冷启动、热缓存、预热、输入规模和测量次数。
 - 故障注入能触发日志和恢复路径，而不是只测试成功路径。

本章要求读者能够区分 `errno`、信号、`oops` 与 `panic`，为系统调用写出失败回滚路径，并解释提交点、幂等重试和故障证据的关系。诊断结论还必须能够复现：若报告只写“怀疑是系统问题”，却没有线程状态、系统调用、等待点、资源计数或反证，就不能构成操作系统层面的分析。

调度、内存、I/O 与网络诊断

BUG、WARN、oops 和 panic 的代码路径

Linux 中 `WARN_ON` 记录调用栈但通常继续；`BUG_ON` 表示当前路径不可继续；`oops` 是内核异常报告，可能终止当前进程或进一步导致 `panic`；`panic` 停止系统或按配置重启。报告需要收集寄存器、调用栈、污染标志、已加载模块和当前任务，才能为离线分析保留上下文。

```
bad_kernel_access:
    fault in kernel mode
    -> do_error_trap or page fault path
    -> oops_begin()
    -> show registers, stack, code bytes
    -> decide die, panic, or kill current task
```

错误严重程度取决于上下文。用户进程传坏指针应返回 `EFAULT`；内核在持有自旋锁时 `page fault`，可能说明严重 bug；文件系统检测到元数据不一致继续写入可能扩大损坏，因此 `remount read-only` 或 `panic` 都是合理策略。

KASAN、KCSAN 和 KFENCE

工具	发现问题	取舍
----	------	----

KASAN	越界、use-after-free、部分栈/全局错误	高开销，高覆盖
KCSAN	数据竞争，非原子并发访问	采样式，可能需多跑
KFENCE	堆越界和 UAF	低开销，覆盖有限

这些工具不会替代锁设计。它们帮助你发现违反不变量的具体证据，例如哪个对象释放后被哪个路径继续访问，哪个字段被两个 CPU 无同步写入。测试报告应保留工具输出中的对象地址、分配栈、释放栈和访问栈。

fault injection

Linux 提供多种故障注入，例如 `fail_page_alloc`、`failslab`、`fail_make_request`。目标是在可控位置让分配、slab、块 I/O 失败，验证错误路径。

```
test plan:
fail next kmalloc in sys_open
expect no fd installed, inode ref dropped

fail block request number N during fsync
expect fsync returns EIO and writeback error recorded
```

没有故障注入，错误回滚代码很可能从未执行过。高质量 OS 代码要让每个 `goto out_*` 标签都能被测试触发。故障注入测试必须检查“没有发生什么”：没有泄漏引用、没有遗留锁、没有半初始化对象进入全局表。

kdump 与 /proc/vmcore

kdump 预留一段崩溃捕获内存。panic 后通过 kexec 启动捕获内核，导出内存转储，再结合符号信息分析。panic 路径越少依赖已经损坏的系统状态，转储成功率越高。

```
panic
-> stop other CPUs
-> save registers and notes
-> kexec crash kernel
-> expose /proc/vmcore
-> analyze task, locks, stacks, memory
```

生产环境中，panic 策略要结合服务目标：有的系统宁可快速重启，有的系统必须保留 dump，有的存储系统检测到元数据损坏要停止写入避免扩大破坏。

错误恢复的状态机写法

恢复不是“再试一次”。每个可恢复对象都应有明确状态：正常、降级、恢复中、不可用、已隔离。状态转换要有进入条件、退出条件和超时策略。

```
device state:
  ONLINE
    on timeout -> RECOVERING
  RECOVERING
    stop queues
    reset hardware
    replay configuration
    complete lost requests
    if success -> ONLINE
    if fail count exceeded -> FAILED
  FAILED
    reject new requests with EIO
    keep diagnostic evidence
```

没有状态机，恢复路径容易和正常 I/O 并发踩踏：reset 时仍提交新请求，失败请求没有唤醒等待者，或恢复成功后旧 completion 又回来释放同一资源。

错误预算和降级

并非所有错误都要立即 panic 或无限重试。系统可以设置错误预算：短时间少量校验失败触发重试，连续失败触发隔离，核心元数据错误触发只读或 panic。错误预算让系统避免因为瞬时故障过度反应，也避免因为反复失败无限拖延。

现象	初始处理	超过预算后
块 I/O 超时	重试、reset 队列	下线设备或返回 <code>EIO</code>
内存分配失败	回收、压缩、等待	OOM kill 或返回 <code>ENOMEM</code>
网络重传增多	降低发送速率	熔断连接或限流
文件系统校验失败	停止相关写入	remount read-only 或 panic

错误预算必须可观测：重试次数、最后错误、进入降级时间、当前状态和影响对象都要暴露。否则系统处在降级状态时，用户只会看到“偶尔很慢”。

cleanup attribute 与 RAII 的边界

用户态 C 可以用 `__attribute__((cleanup))` 模拟作用域清理，C++ 可以用 RAII。内核 C 常用 `goto out`，是因为错误路径需要精确控制上下文、锁和释放顺序。三种方法没有绝对高低，关键是所有权必须清楚。

```
// conceptual RAII style
FileRef file = open_checked(path)
PageRef page = allocate_page()
LockGuard guard(inode.lock)
commit_update(file, page)
```

RAII 适合普通进程上下文中的资源组合；但析构函数不应隐藏可能睡眠的复杂 I/O，也不应在 panic 路径中试图获取新锁。教学 OS 可以使用 C++20 RAII 管理引用和锁，但要为中断上下文、noexcept 和显式提交点写清规则。

panic 报告模板

panic 输出要稳定、短而有证据。最低字段如下：

```
panic report:
reason
cpu id and current task
instruction pointer and stack pointer
trap vector or syscall number
held locks
taint or loaded modules
last N kernel log records
affected device or inode if any
reboot/dump policy
```

报告不是越长越好。panic 时系统可能已经损坏，输出过多可能再次阻塞。核心原则是保留能定位第一现场的事实，而不是在崩溃路径中做复杂分析。

printk 和 dmesg

`printk` 有 `loglevel`、`rate limiting`、`console` 输出和 `ring buffer`。panic 前后的日志可能是唯一证据，但过量 `printk` 会扰动时序，甚至拖慢系统。驱动错误路径应记录设备、请求 id、状态寄存器、`errno`，而不是只写“failed”。

ftrace 和 tracepoint

ftrace 可以追踪函数入口、函数图和静态 tracepoint。tracepoint 的价值是稳定字段：调度切换、irq、block、net、syscalls 都有可解析事件。

```
trace-cmd record -e sched:sched_switch -e block:block_rq_issue
trace-cmd report
```

trace event 需要定义稳定字段，写入每 CPU 环形缓冲区，再由用户态读取。tracepoint 的作用是把一次状态变化转化为可采集证据；字段定义和丢失策略决定这些证据能回答哪些问题。

perf 与 PMU

perf stat 看计数，perf record/report 看采样调用栈，perf top 看实时热点。PMU 事件能观察 cycles、instructions、cache misses、branch misses、TLB misses。IPC 低可能来自 cache miss、分支错误、前端供给不足或后端等待，不能只说“CPU 慢”。

```
perf stat -e cycles,instructions,cache-misses,branches,branch-misses ./workload
perf record -g ./workload
perf report
```

eBPF 和 bpftrace

eBPF 程序可挂到 kprobe、uprobe、tracepoint、profile、LSM、cgroup、网络路径。BPF map 保存状态，helper 提供受控内核访问。bpftrace 适合快速一行观测。

```
bpftrace -e 'tracepoint:syscalls:sys_enter_write { @[comm] = count(); }'
bpftrace -e 'kprobe:vfs_read { @[kstack] = count(); }'
```

eBPF 的边界是 verifier、安全权限和观测开销。生产系统应限制采样频率和输出量，避免观测本身制造故障。

strace、ltrace 和 gdb

strace 基于 ptrace 观察系统调用 enter/exit，能回答“程序在请求内核做什么”；ltrace 观察动态库调用，能回答“用户态库在做什么”；gdb/kgdb 用符号和断点观察状态。strace 看不到内核内部等待原因，perf/ftrace/eBPF 可以补上。

/proc、/sys、sysctl 和 systemd journal

/proc/meminfo、/proc/interrupts、/proc/slabinfo、/proc/[pid]/status、/proc/[pid]/stack 是运行状态窗口；/sys 暴露 kobject、设备、cgroup、block queue 参数；/proc/sys 是 sysctl 控制面；journald 给系统日志结构化字段。

PSI: Pressure Stall Information

PSI 记录 CPU、memory、IO stall 时间，回答“任务因为资源压力停顿多久”。它比单纯利用率更接近用户感受到的延迟。内存 PSI 高说明任务因回收或内存不足停顿；I/O PSI 高说明任务等待 I/O；CPU PSI 高说明 runnable 但等不到 CPU。

```
cat /proc/pressure/cpu
cat /proc/pressure/memory
cat /proc/pressure/io
```

PSI 适合做过载保护信号。服务可以在 memory/io pressure 高时限流，避免把队列堆到超时。

工具选择决策表

可观测性工具很多，选择时先问问题类型。

问题	优先工具	证据
程序卡在哪个系统调用	<code>strace -tt -T</code>	syscall、耗时、errno
CPU 时间花在哪里	<code>perf record -g</code>	on-CPU 调用栈
线程为什么不运行	off-CPU trace、sched trace	睡眠原因、唤醒者
磁盘请求慢在哪里	block trace、iostat、perf sched	队列和 completion
内存压力是否存在	PSI、 <code>/proc/meminfo</code> 、vmstat	stall、reclaim、swap
内核路径偶发错误	tracepoint、kprobe、bpftrace	事件字段和栈
容器被限制	cgroup v2 文件、journal	throttled、OOM、pids limit

常见误区

不要一上来抓所有数据。先用低成本指标定位层次，再用高精度 trace 验证假设。观测也会扰动系统，尤其是高频 kprobe、过量 printk 和全量系统调用 trace——把被观察系统变成了另一个系统。

一次延迟诊断的完整演示

假设 HTTP 服务 p99 从 80ms 升到 2s。诊断不应直接改代码，而应建立证据链。

```
step 1: application trace
  parse: 1ms, business: 5ms, persist: 1900ms

step 2: strace
  fsync(fd) = 0 <1.87s>

step 3: /proc/pressure/io
  some avg10 increased

step 4: block trace
  many flush requests wait behind writeback

step 5: conclusion
  tail latency dominated by storage flush and dirty writeback
```

反证也要写：CPU profiler 没有 2s on-CPU 热点；网络 recv/send 没有长时间阻塞；锁等待直方图没有对应峰值。这样结论才不是“看起来像磁盘”。

指标设计：counter、gauge、histogram、event

系统指标至少分四类。

类型	含义	OS 示例
counter	单调增加事件数	syscall 次数、错误次数、重试次数
gauge	当前状态值	队列长度、活跃线程、脏页数量
histogram	分布	请求延迟、锁等待、I/O 完成时间

event	结构化状态转换	任务睡眠、唤醒、请求提交、panic
-------	---------	--------------------

错误率只用 counter 不够，还要有错误码和对象维度；延迟只报平均值不够，要有 p50/p95/p99 和最大值；队列只报当前值不够，最好有高水位和丢弃次数。

低开销 instrumentation 原则

观测点应贴近状态转换，而不是在循环里无节制打印。推荐做法：

- 热路径用 per-CPU counter，避免全局锁。
- 高频事件采样或按条件开启。
- 错误路径记录结构化字段和稳定 id。
- 日志消息带 request id、task id、object id。
- trace buffer 有固定容量和丢弃计数。
- 观测代码本身不能持有业务关键锁太久。

可观测性也是代码，必须有性能预算。一个 tracing patch 若把锁竞争扩大，就可能把被观察系统变成另一个系统。

验收：可复现诊断包

本卷要求最终报告能够打包复现。最小诊断包包括：

```
diagnostic bundle:
  exact command and workload
  kernel version and hardware/container limits
  application logs with request ids
  relevant /proc and /sys snapshots
  trace or perf data
  benchmark raw results
  analysis notes with rejected hypotheses
```

报告不是把命令输出堆在一起，而是把证据组织成因果链：现象是什么，在哪层出现，哪些解释被排除，当前结论的边界在哪里。能做到这一点，才算真正掌握 OS 调试。

调试工具与性能边界

可观测性、调试与性能边界

原理

操作系统问题很少只靠读代码解决。线程为什么不运行，可能是调度等待、锁等待、I/O 阻塞、page fault、信号、cgroup 限流或下游服务；内存为什么上涨，可能是真实泄漏、allocator 缓存、page cache、mmap、线程栈或内核 buffer；写文件为什么慢，可能是 page cache 脏页、设备队列、fsync、目录同步或日志提交。没有观测证据，结论只是猜测。

可观测性要把 OS 状态映射到证据。进程和线程状态可以从 `/proc`、`ps`、`top`、调度 trace 观察；系统调用可以用 `strace` 观察；CPU 热点可以用采样 profiler；page fault、context switch、I/O 等待和网络状态可以用系统计数器 and tracing 工具；应用层必须补充业务 trace，记录请求身份、队列等待、状态机阶段和错误码。

调试时要区分 on-CPU 和 off-CPU。on-CPU 说明线程正在执行指令；off-CPU 说明线程没有运行，可能在等待。CPU profiler 只能解释运行时消耗在哪里，解释不了大部分时间都在睡眠的请求。尾延迟问题常常要从 off-CPU 等待开始看。

内存调试要区分 virtual size、resident set、anonymous memory、file-backed page、page cache、allocator arena 和 kernel memory。malloc 后不触页，可能只增加虚拟地址；首次写入才分配物理页；释放内存后 RSS 不马上下降，也可能是 allocator 保留 arena。把所有 RSS 增长都叫泄漏，是不合格诊断。

I/O 调试要区分 buffered I/O、direct I/O、mmap、page cache hit/miss、dirty writeback、fsync 和设备吞吐。一次写很快，可能只是进入 page cache；一次读很快，可能只是热缓存；一次 fsync 很慢，可能是在偿还之前累计的脏页。实验必须控制冷缓存和热缓存，否则数字不可比较。

实践

本章的实践模板是“现象、假设、证据、反证、结论”。

字段	请求延迟升高示例
现象	p95 从 20ms 升到 500ms
假设	worker 等锁或等 I/O
证据	trace 中 queue_wait 低，blocked_io 高，线程睡在 writeback
反证	on-CPU profiler 没有热函数占用 500ms
结论	延迟主要来自 I/O 完成等待，不是 CPU 算法变慢

程序卡死时，先列线程状态：哪个线程持有锁，哪个线程等待条件变量，哪个线程在 join，关闭标志是否设置，是否有线程仍可能唤醒等待者。若所有消费者都睡在 not_empty，而关闭路径只通知 not_full，就能定位到等待谓词和通知对象不匹配。

实践中要保留原始证据：命令、时间、输入规模、机器信息、内核版本、容器限制、采样频率、日志片段和无法观察的字段。没有这些，报告不可复现。

算法与实现分析

事件日志。

最简单的可观测性算法是事件日志。每个关键状态转换追加一条结构化记录：时间、线程、请求 ID、事件类型、对象 ID、旧状态、新状态和错误码。日志不能只写自然语言，否则无法聚合和验证。

```
record_event(req_id, object, old_state, new_state, reason):
    entry.ts = monotonic_time()
    entry.cpu = current_cpu()
    entry.thread = current_thread()
    entry.req_id = req_id
    entry.object = object
    entry.old_state = old_state
    entry.new_state = new_state
    entry.reason = reason
    ring_append(trace_buffer, entry)
```

事件日志本身也要有背压。无限日志会把故障变成磁盘或内存问题。常见做法是固定大小环形缓冲区、按级别采样、按请求 ID 打开详细追踪，或者把关键错误同步写入可靠日志。

延迟分解。

请求延迟不能只记录总耗时。一个可行动的分解至少包括 queue wait、runnable wait、on CPU、blocked on lock、blocked on I/O、downstream wait 和 cleanup。实现上可以在状态转换处打点，然后按请求 ID 聚合。

```
transition(req, new_state):
    now = clock()
    req.time_in_state[req.state] += now - req.last_ts
    req.state = new_state
    req.last_ts = now
```

这个算法的复杂度是每次状态转换 $O(1)$ ，但要求状态枚举稳定。若所有等待都记成 blocked，后续无法判断是锁、I/O 还是 page fault。

资源对账。

资源泄漏调试需要对账算法。每次 open 增加 fd 计数，每次 close 减少；每次分配对象记录 owner，每次释放删除记录；系统退出或测试结束时检查剩余对象。

```
track_alloc(id, type, owner):
    live[id] = {type, owner, stack}

track_free(id):
    live.erase(id)

assert_no_leak():
    for each object in live:
        report(object)
```

真实系统不能给所有对象都保存完整 stack，因为成本太高；可以采样、按类型开启、只在测试构建开启。但思想不变：资源必须能被计数，生命周期必须能闭合。

off-CPU 分析。

很多性能问题不在 CPU 火焰图里，因为线程大部分时间不在 CPU 上。off-CPU 分析记录线程从运行态切到睡眠态的原因，以及被谁唤醒。

```
on_schedule_out(task, reason, wait_object):
    task.offcpu_start = clock()
    task.wait_reason = reason
    task.wait_object = wait_object

on_wakeup(task, waker):
    delta = clock() - task.offcpu_start
    offcpu_histogram[task.wait_reason].add(delta)
    record_wakeup_edge(waker, task, task.wait_object)
```

这个数据能区分等锁、等 I/O、等页、等网络、等定时器和等子进程。若 p99 延迟主要来自 off-CPU 的 writeback 等待，优化业务 CPU 代码不会解决问题。

最小可复现报告。

高质量调试报告要让别人能复现判断。最小格式如下：

```
Problem:
    现象、影响范围、首次出现时间

Environment:
    内核版本、硬件/容器限制、文件系统、负载规模

Timeline:
    关键事件按时间排序

Evidence:
    系统调用、trace、线程状态、资源计数、日志、指标
```

Rejected hypotheses:
已经排除的解释和证据

Conclusion:
当前最能解释现象的机制

Boundary:
仍未验证的假设和下一步实验

报告的价值不在于一次猜中，而在于缩小不确定性。写出反证能防止团队反复讨论已经排除的方向。

测试

可观测性也需要测试。一个系统若只能在成功时输出日志，失败时没有状态，就不可维护。

- 每个长时间等待点都能输出等待原因或 trace 事件。
- 每个请求都有稳定身份，能串起进入队列、开始执行、阻塞、完成和失败。
- 错误码保留原始原因，不被统一改写成 unknown error。
- 资源计数可对账：打开 fd 数、活跃线程数、队列长度、in-flight I/O、分配对象数。
- 性能测试区分冷启动、热缓存、预热、输入规模和测量次数。
- 故障注入能触发日志和恢复路径，而不是只测试成功路径。

调试报告的验收标准是别人能复现你的判断。若报告只写“怀疑是系统问题”，没有线程状态、系统调用、等待点、资源计数或反证，它不是 OS 分析。

源码级补充：printk、ftrace、perf、eBPF、strace、/proc、/sys 与 PSI

printk 和 dmesg。

printk 有 loglevel、rate limiting、console 输出和 ring buffer。panic 前后的日志可能是唯一证据，但过量 printk 会扰动时序，甚至拖慢系统。驱动错误路径应记录设备、请求 id、状态寄存器、errno，而不是只写“failed”。

ftrace 和 tracepoint。

ftrace 可以追踪函数入口、函数图和静态 tracepoint。tracepoint 的价值是稳定字段：调度切换、irq、block、net、syscalls 都有可解析事件。

```
trace-cmd record -e sched:sched_switch -e block:block_rq_issue
trace-cmd report
```

阅读 [kernel/trace/](#) 时，看 trace event 如何定义字段、如何写入 per-CPU ring buffer、如何被用户态读取。tracepoint 是把证据落成可采集事件。

perf 与 PMU。

perf stat 看计数，perf record/report 看采样调用栈，perf top 看实时热点。PMU 事件能观察 cycles、instructions、cache misses、branch misses、TLB misses。IPC 低可能来自 cache miss、分支错误、前端供给不足或后端等待，不能只说“CPU 慢”。

```
perf stat -e cycles,instructions,cache-misses,branches,branch-misses ./workload
perf record -g ./workload
perf report
```

eBPF 和 bpftrace。

eBPF 程序可挂到 kprobe、uprobe、tracepoint、profile、LSM、cgroup、网络路径。BPF map 保存状态，helper 提供受控内核访问。bpftrace 适合快速一行观测。

```
bpftrace -e 'tracepoint:syscalls:sys_enter_write { @[comm] = count(); }'
bpftrace -e 'kprobe:vfs_read { @[kstack] = count(); }'
```

eBPF 的边界是 verifier、安全权限和观测开销。生产系统应限制采样频率和输出量，避免观测本身制造故障。

strace、ltrace 和 gdb。

strace 基于 ptrace 观察系统调用 enter/exit，能回答“程序在请求内核做什么”；ltrace 观察动态库调用，能回答“用户态库在做什么”；gdb/kgdb 用符号和断点观察状态。strace 看不到内核内部等待原因，perf/ftrace/eBPF 可以补上。

/proc、/sys、sysctl 和 systemd journal。

/proc/meminfo、/proc/interrupts、/proc/slabinfo、/proc/[pid]/status、/proc/[pid]/stack 是运行状态窗口；/sys 暴露 kobject、设备、cgroup、block queue 参数；/proc/sys 是 sysctl 控制面；journald 给系统日志结构化字段。

PSI: Pressure Stall Information。

PSI 记录 CPU、memory、IO stall 时间，回答“任务因为资源压力停顿多久”。它比单纯利用率更接近用户感受到的延迟。内存 PSI 高说明任务因回收或内存不足停顿；I/O PSI 高说明任务等待 I/O；CPU PSI 高说明 runnable 但等不到 CPU。

```
cat /proc/pressure/cpu
cat /proc/pressure/memory
cat /proc/pressure/io
```

PSI 适合做过载保护信号。服务可以在 memory/io pressure 高时限流，避免把队列堆到超时。

工具选择决策表。

可观测性工具很多，选择时先问问题类型。

问题	优先工具	证据
程序卡在哪个系统调用	<code>strace -tt -T</code>	syscall、耗时、errno
CPU 时间花在哪里	<code>perf record -g</code>	on-CPU 调用栈
线程为什么不运行	off-CPU trace、sched trace	睡眠原因、唤醒者
磁盘请求慢在哪里	block trace、iostat、perf sched	队列和 completion
内存压力是否存在	PSI、/proc/meminfo、vmstat	stall、reclaim、swap
内核路径偶发错误	tracepoint、kprobe、bpftrace	事件字段和栈
容器被限制	cgroup v2 文件、journal	throttled、OOM、pids limit

不要一上来抓所有数据。先用低成本指标定位层次，再用高精度 trace 验证假设。观测也会扰动系统，尤其是高频 kprobe、过量 printk 和全量系统调用 trace。

一次延迟诊断的完整演示。

假设 HTTP 服务 p99 从 80ms 升到 2s。诊断不应直接改代码，而应建立证据链。


```

step 1: application trace
  parse: 1ms, business: 5ms, persist: 1900ms

step 2: strace
  fsync(fd) = 0 <1.87s>

step 3: /proc/pressure/io
  some avg10 increased

step 4: block trace
  many flush requests wait behind writeback

step 5: conclusion
  tail latency dominated by storage flush and dirty writeback

```

反证也要写：CPU profiler 没有 2s on-CPU 热点；网络 recv/send 没有长时间阻塞；锁等待直方图没有对应峰值。这样结论才不是“看起来像磁盘”。

指标设计：counter、gauge、histogram、event。

系统指标至少分四类。

类型	含义	OS 示例
counter	单调增加事件数	syscall 次数、错误次数、重试次数
gauge	当前状态值	队列长度、活跃线程、脏页数量
histogram	分布	请求延迟、锁等待、I/O 完成时间
event	结构化状态转换	任务睡眠、唤醒、请求提交、panic

错误率只用 counter 不够，还要有错误码和对象维度；延迟只报平均值不够，要有 p50/p95/p99 和最大值；队列只报当前值不够，最好有高水位和丢弃次数。

低开销 instrumentation 原则。

观测点应贴近状态转换，而不是在循环里无节制打印。推荐做法：

- 热路径用 per-CPU counter，避免全局锁。
- 高频事件采样或按条件开启。
- 错误路径记录结构化字段和稳定 id。
- 日志消息带 request id、task id、object id。
- trace buffer 有固定容量和丢弃计数。
- 观测代码本身不能持有业务关键锁太久。

可观测性也是代码，必须有性能预算。一个 tracing patch 若把锁竞争扩大，就可能把被观察系统变成另一个系统。

验收：可复现诊断包。

本卷要求最终报告能够打包复现。最小诊断包包括：

```

diagnostic bundle:
  exact command and workload
  kernel version and hardware/container limits
  application logs with request ids
  relevant /proc and /sys snapshots

```

```
trace or perf data  
benchmark raw results  
analysis notes with rejected hypotheses
```

报告不是把命令输出堆在一起，而是把证据组织成因果链：现象是什么，在哪层出现，哪些解释被排除，当前结论的边界在哪里。能做到这一点，才算真正掌握 OS 调试。

全书得到的系统。从无操作系统机器开始建立的每一项能力，如今都有明确对象、所有者、入口、等待关系、提交点、失败路径和观察证据。书末综合案例重新追踪一次文件保存，但此时其中每个对象都已在前文分别推导，因此完整链路只承担综合验证，不再承担术语开场。

全书综合：一次保存怎样穿过完整系统

前十八章已经分别建立 task、受控入口、地址空间、文件对象、页缓存、设备请求、文件系统事务、安全判定和恢复证据。现在可以把这些局部结构重新连接起来，用一次文件保存验证对象引用、缓冲区所有权、等待与唤醒、可见性、完成和持久化边界是否相互一致。

操作系统对象、路径与契约

从一次文件保存认识操作系统

编辑器里点“保存”，表面上只是把一段字节写进名为 `data` 的文件，再告诉用户“保存成功”。这一句话看起来很小，但它实际包含三个承诺：用户给出的内容被系统接收了；旧文件不会因为半路失败遭到非预期破坏；如果程序或机器随后出错，系统还能说明哪个版本可信。

如果机器上只有一个函数、一个设备和一个永远不会失败的写入动作，保存可以被想象成普通函数调用。真实机器同时存在更多条件：设备可能暂时不能接收数据；用户传入的地址可能无效；文件名和 `fd` 只是引用；写入可能先停在内存缓存里；断电后还要判断旧版本、新版本、临时文件和目录项哪个可信。每一项条件都需要明确的状态和完成承诺，无法由一行函数返回值概括。

保存按钮可以作为一张 OS 地图。简单写法一旦说不清“谁在等、等什么、失败在哪里、成功凭什么”，就说明状态还藏在函数调用里，没有进入内核的状态记录。能解释这些状态的对象，才是进程、内存、文件系统和驱动这些子系统真正要解决的问题。

核心思想

操作系统的核心工作，是把“不可信的用户请求”和“会延迟、会失败的硬件事件”翻译成可管理的对象、可恢复的状态转移和可检查的完成证据。

本书所说的“契约”指可检查的技术承诺。用户调用 `write` 时，OS 要说明怎样解释参数、成功和失败分别表示什么；OS 提交设备请求后，硬件要用状态位、中断或完成队列报告请求是否结束；文件系统宣布保存完成时，还要留下崩溃恢复能够识别的提交证据。任何一层缺少明确语义，“保存成功”都无法成为可信结论。

一句“保存成功”	必须进一步证明什么	如果不证明会怎样
用户传入了内容	这段地址属于当前进程，且内核有权读取	可能读到非法地址，甚至泄漏内核数据
系统接收了请求	当前执行流、文件引用和目标对象都被记录下来	设备慢时只能忙等，或完成后找不到发起者
写入推进了	数据进入了哪个对象：内存缓存、设备队列还是持久介质	<code>write</code> 返回后掉电，用户以为保存了，其实磁盘仍是旧内容
失败能报告	错误发生在参数、权限、地址、设备、持久化还是目录发布	只能返回模糊失败，无法修复或恢复

完成能证明	哪个提交点之后，崩溃恢复可以相信这个结果	重启后无法区分成功版本和半成品
-------	----------------------	-----------------

在继续引入 task、页表和文件对象之前，还要回答一个更基础的问题：这些工作为什么不能由每个应用程序各自完成？假设编辑器、备份程序和日志程序都需要写同一块设备。最初可以给三个程序各放一份设备访问库，让它们在写入之前调用 `acquire`，写完之后调用 `release`。只要三个程序都正确、都使用同一套库，这种协作可以维持。

困难在于，库只能为愿意调用它的代码提供帮助，不能约束绕过它的代码。一个程序可能因为缺陷忘记释放设备，另一个程序可能直接访问设备寄存器，第三个程序还可能把不属于自己的内存地址交给设备。此时，问题不在于库函数写得是否精巧，而在于它没有高于应用程序的控制权。

仅靠应用协作时的做法	无法保证的事情	产生的后果
约定每个程序主动让出 CPU	失控程序是否会遵守约定	一个死循环可以长期占用处理器
约定写设备前先取得软件锁	程序是否绕过库直接写寄存器	两个请求可能同时改动设备状态
约定只访问分配给自己的内存	任意机器指令是否遵守地址范围	一个错误指针可能破坏其他程序
每个程序维护自己的空闲块表	多份记录是否始终一致	同一磁盘块可能被重复分配
程序自行判断访问权限	被拒绝者是否愿意接受判断	权限规则无法对恶意或错误代码生效

要使约束成立，系统需要一个应用程序不能绕过的共同管理者。处理器必须能够区分普通程序和这个管理者的执行权限；敏感寄存器、页表和中断控制不能由普通程序直接改写；应用程序若要使用设备、创建地址映射或访问文件，只能通过受控入口提出请求。这个受保护的核心称为 内核 (*kernel*)。

内核不是因为代码规模更大而特殊，而是因为机器赋予了它不同的权限和入口。普通程序运行在受限制的执行级别，只能访问页表允许的地址；系统调用把请求交给内核；异常让内核处理当前指令无法自行解决的问题；中断让内核在设备完成或定时器到期时重新取得控制。这样，管理规则才从“请各程序自觉遵守”变成“机器和内核共同执行”。

所需能力	由谁提供	在保存请求中的作用
限制敏感操作	处理器特权级与内核入口	应用不能直接改写磁盘队列或页表
限制地址访问	页表检查与地址空间管理	编辑器的 <code>buffer</code> 不能指向其他进程的私有页面
周期性收回 CPU	硬件定时器与调度器	一个程序失控时，其他程序仍有运行机会
统一记录资源归属	内核对象与全局分配状态	<code>fd</code> 、页面和设备请求不会由各应用分别解释
接收异步结果	中断、完成队列与等待关系	设备结束工作后，结果能交还给正确请求

“操作系统”和“内核”也需要区分。内核是以特权方式运行、直接维护隔离和资源状态的核心；操作系统还包括系统库、后台服务、命令工具和管理配置。系统库可以把复杂接口整理得更易用，后台服务可以实现名称解析、日志收集或设备管理，但最终涉及地址隔离、处理器调度和设备授权时，仍要依赖内核提供的强制边界。

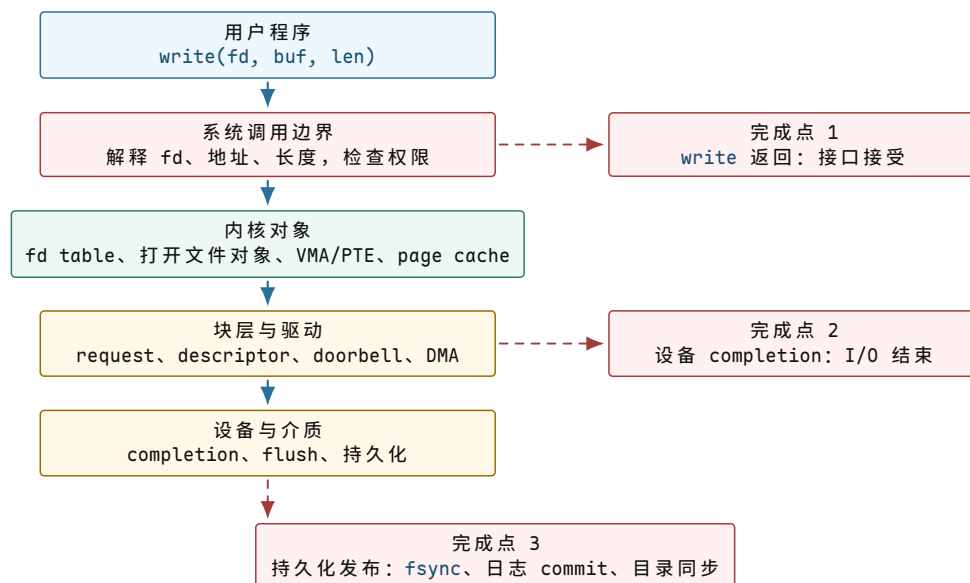
同一项管理还应区分机制 (*mechanism*) 与策略 (*policy*)。机制说明系统“怎样做到”，例如怎样保存执行现场、怎样把任务放入等待队列、怎样拒绝无权限的页访问；策略说明系统“选择什么”，例如先运行哪个任务、允许哪个主体打开文件、脏页积累到什么程度开始写回。若没有机制，策略无法执行；若策略被固定在每个底层动作里，系统又难以适应不同负载。后续章节会在每个子系统中分别说明两者。

至此，操作系统的起点已经可以从具体条件中得到：多个程序共享处理器、内存和设备，而程序可能出错、延迟或不合作；系统因而需要一个不能被普通程序绕过的内核，由它建立统一对象、执行隔离规则、安排等待关系，并把硬件事件转换成可检查的结果。

这些名字不是孤立术语。`task` 记录当前执行流，VMA/PTE 解释用户地址，`fd` 和打开文件对象解释整数句柄，page cache 解释写入为何能先停在内存，DMA/completion 解释设备异步完成，`fsync` 解释怎样把内存状态推进到持久化边界。每个名字都对应保存路径上的一个缺口。

判断一个 OS 机制是否讲清楚，要看四类可观察证据：用户程序提出了什么请求；进入内核后新增了哪些对象状态；硬件用什么事件、寄存器或权限检查支撑这些状态；失败时外部能看到什么结果。比如 `write(fd, buf, 4)` 这一行，在用户层只是一个函数调用，在内核层要解释 `fd`、buffer 和长度，在硬件层可能触发 page fault、DMA 和中断，在证据层则只返回字节数或错误码。

先把这张地图画出来。本章后面的每一步修正，都会回到这张图上的某一层或某一个完成点。



图示：保存请求自上而下穿过五层。三个“完成”在不同层生效：接口接受不等于设备完成，设备完成不等于崩溃后可恢复。掉电时能相信什么，取决于数据推进到了哪一层。

先考察最小环境：系统中只有一个用户程序，只有一个用来写数据的设备，暂不考虑掉电，也没有别的任务抢 CPU。在这个受限场景里，保存过程可能写成这样：

```

void save_request(const char* user_buf, size_t len) {
    while (!device_ready()) {
        // busy wait
    }

    for (size_t i = 0; i < len; ++i) {
        device_write_byte(user_buf[i]);
    }
}
  
```

```

}

print("saved\n");
}

```

这段代码没有交代任何 OS 坐标：CPU 等待设备时谁拥有执行权，设备完成时谁负责唤醒，`user_buf` 是否可信，`fd` 指向什么，以及 `print("saved")` 依据哪项完成证据。这些问题沿保存路径依次出现：CPU 先被无效占用，随后需要可靠睡眠和唤醒，接着要解释用户 `buffer`、`fd` 和文件对象，最后还要说明“保存成功”究竟越过了哪个边界。

先看等待设备的这一小段：

```

while (!device_ready()) {
    // CPU keeps spinning here.
}

```

它反复读取同一个事实：设备现在能不能接收下一个字节。这个会影响下一步行为、必须被记住的事实，就是状态。

核心思想

状态就是“会影响下一步行为的、必须被记住的事实”。在这里，`dev.ready = false` 表示设备还不能接收数据；`dev.ready == true` 表示保存路径可以继续往下走。

忙等会让 CPU 持续询问同一个状态。假设设备 5 毫秒后才准备好，这段时间原本可以运行另一个程序、处理网络包或响应输入；上面的 `while` 却把 CPU 固定在原地，使设备延迟直接转化为处理器时间消耗。

`device_ready()` 读取的是设备暴露给 CPU 的状态值。硬件设备常常会给 CPU 留出一些可读写的小格子，CPU 通过这些小格子知道设备是否空闲、是否出错、能不能接收下一个请求。这些小格子就是设备寄存器；在保存案例中，状态寄存器是 CPU 和设备之间交换状态的入口。

时刻	设备状态	CPU 正在做什么	问题
0 ms	not ready	读一次状态寄存器	发现不能写
1 ms	not ready	再读一次状态寄存器	没有任何新进展
2 ms	not ready	继续读状态寄存器	其他程序得不到 CPU
5 ms	ready	终于跳出循环	前面几毫秒被浪费

因此，等待设备不能写成原地自旋。设备没准备好时，当前执行流应当先停下，把 CPU 让出来；设备准备好之后，内核再把它恢复到可运行状态。

要让执行流可靠地停下并恢复，需要引入 `task`。`task` 是内核能暂停、恢复、选择运行的一条执行流。它不是用户写的一个普通函数；它必须保存运行位置、可运行状态以及当前等待条件。

这里第一次出现“对象”。对象不是面向对象编程里的类名，也不是把名词包装得更正式。对象是内核为了长期保存并共同访问某类状态而建立的记录。普通函数的局部变量只在当前调用栈里有效；一旦函数要睡眠、被中断、被调度走，或者等待另一个硬件事件回来，状态就不能只放在局部变量里。它必须放进一个能被调度器、中断处理路径、超时路径和关闭路径共同找到的地方。

普通函数视角	OS 对象视角	为什么必须升级
局部变量 <code>i</code>	当前复制进度、请求偏移、已完成字节数	函数可能睡眠，醒来后要知道从哪里继续

一次函数调用	<code>task</code> 和保存的执行现场	当前执行流可能被切走，稍后从同一位置恢复
一个布尔判断	谓词和受保护状态	条件可能被中断路径或另一个 CPU 修改
等待一个结果	wait queue 和 request/completion	设备完成时要知道该唤醒谁、哪个请求完成

```
if (!device_ready()) {
    current->state = BLOCKED;
    wait_queue_push(&dev.waiters, current);
    schedule();
}
```

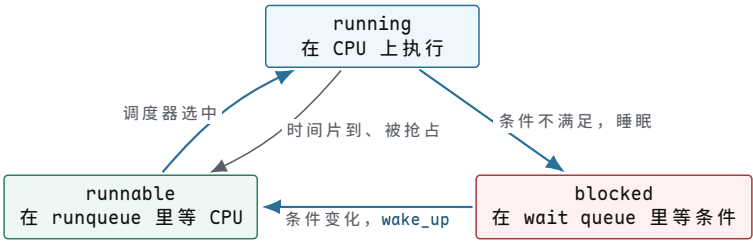
这段伪代码引入了三类内核状态，分别解决“能不能运行”“等在哪里”“谁来选择下一个执行流”三个问题。

词	在本例中的含义	反例
<code>BLOCKED</code>	当前 <code>task</code> 不能继续运行，因为它在等设备 <code>ready</code>	如果还把它放在可运行队列里，调度器可能又选中它，结果还是忙等
<code>wait queue</code>	等同一个条件的一组 <code>task</code> ，例如都在等 <code>dev.ready</code>	如果没有队列，设备完成时不知道该叫醒谁
<code>schedule()</code>	调度器入口：当前 <code>task</code> 让出 CPU，内核选择另一个能运行的 <code>task</code>	如果没有调度器，睡眠只会变成整台机器停住

`task` 至少有三种基本状态：

状态	含义	在保存例子里的位置
<code>running</code>	正在某个 CPU 上执行	保存函数正在跑 <code>device_ready()</code> 或复制数据
<code>runnable</code>	条件已经满足，只是暂时没轮到 CPU	设备已经 <code>ready</code> ， <code>task</code> 被唤醒，等调度器选中
<code>blocked</code>	现在不能推进，必须等某个条件变化	设备未 <code>ready</code> ， <code>task</code> 睡在 <code>dev.waiters</code> 上

注意 `runnable` 不等于 `running`。一个 `task` 可以已经具备运行条件，但 CPU 正在跑别的 `task`；它只是排队等调度器。



图示：调度器只在 `running` 和 `runnable` 之间做选择；`blocked` 的 `task` 不在候选集合里，必须先由唤醒路径把它挪回 `runqueue`。

这三个教学状态在 Linux 的任务对象中有对应表示。任务状态字段记录能否运行，队列成员关系再区分任务正在 CPU 上执行，还是已经进入就绪队列等待选择：

真实状态名	对应教学状态	含义
TASK_RUNNING	running 或 runnable	在 CPU 上，或已在 runqueue 排队；两者共用一个状态名，靠 on_rq 字段区分
TASK_INTERRUPTIBLE	blocked	睡眠等待条件，可被信号打断
TASK_UNINTERRUPTIBLE	blocked	睡眠等待条件（多为 I/O），信号打不断
__TASK_STOPPED	blocked 的一种	被信号或调试器停住
EXIT_ZOMBIE	已退出待回收	不再是执行候选，但保留退出状态等父进程读取

这个状态不需要相信教科书，可以直接在真实机器上看到。`/proc/<pid>/status` 的 `State` 字段就是 `__state` 的用户态投影，下面两行是本书实验机上的真实输出：

```
$ grep '^State' /proc/self/status
State:  R (running)
$ sleep 300 &
$ grep '^State' /proc/$!/status
State:  S (sleeping)
```

`R` 表示 `TASK_RUNNING`——注意它同时覆盖“正在跑”和“在排队”两种情况，正好对应上面的状态机图；`S` 表示可中断睡眠，即 `TASK_INTERRUPTIBLE`。后面章节的等待队列、唤醒和调度讨论，都会落到这几个真实状态名上。

仅把 `task` 放入等待队列还不够。前面的写法比忙等前进了一步，但它仍然不正确，问题出在“检查条件”和“登记等待者”之间：

```
if (!device_ready()) {
    current->state = BLOCKED;
    wait_queue_push(&dev.waiters, current);
    schedule();
}
```

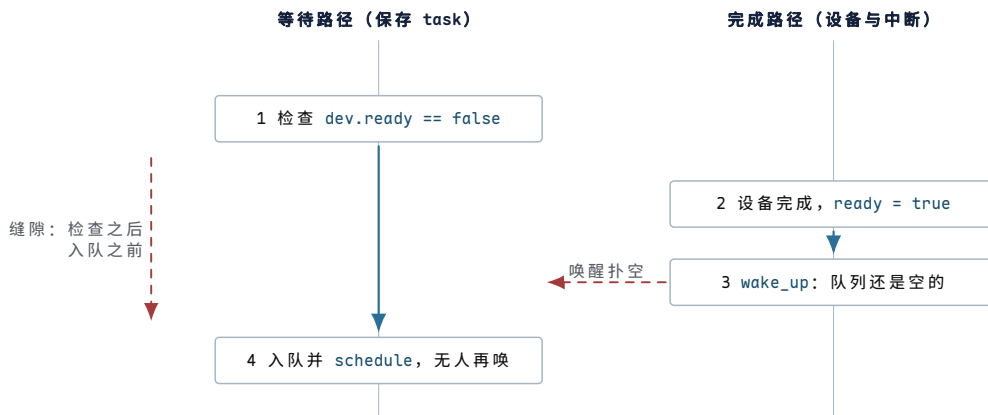
它把“检查条件”和“入等待队列”分成了几个互相可打断的动作。设备可能刚好在缝隙里完成：

顺序	保存 <code>task</code> 做了什么	设备完成路径做了什么
1	检查 <code>dev.ready == false</code>	还没发生
2	准备把自己睡下去	设备变成 <code>ready</code>
3	还没入队	中断处理调用 <code>wake_up(&dev.waiters)</code> ，但队列为空
4	<code>task</code> 入队并 <code>schedule()</code>	之后没有新的完成事件触发唤醒

这个错误叫 `lost wakeup`：唤醒确实发生过，但等待者还没登记，所以它错过了唤醒，之后可能永远睡下去。

这里的“唤醒”源于设备完成后交给 CPU 的事件，而非用户代码的一次普通函数调用。这类由设备主动触发的事件称为中断。设备完成和用户函数继续执行分属不同路径，内核负责把前者转换为“某个 `task` 已经具备继续运行的条件”。

lost wakeup 的根源是状态与等待者没有在同一规则下更新。等待者判断 `dev.ready == false` 时，完成路径可能马上把它改成 `true`；完成路径调用 `wake_up` 时，等待者又可能尚未进入队列。两个动作虽然都发生了，却没有建立可靠关系。因此，逐行检查代码还不够；不变量必须在所有可能的时序交错下成立。



图示：两条路径各自都“做了正确的事”，错误出在缝隙里：完成事件恰好落在等待者检查条件之后、登记入队之前。把检查和入队放进同一个受保护区间，缝隙才被封死。

错误理解	正确理解	后果
wakeup 是一条消息	wakeup 只是提示条件可能变化	醒来后必须重新检查谓词
入队和检查可以分开写	检查谓词和登记等待者必须受同一规则保护	否则完成事件会从缝隙里丢失
睡眠只是暂停函数	睡眠会让出 CPU，并改变 task 的调度状态	blocked task 不能继续留在 runqueue 上抢 CPU
锁只是防止变量同时写	锁保护的是“条件与队列的一致关系”	没有锁会出现状态正确但等待关系错误

解决 lost wakeup，需要同时约束两个对象：一个是判断能否继续的条件，一个是保护这个条件和等待队列的互斥规则。

词	在本例中是什么意思	为什么必须要它
谓词	一个能回答“现在能不能继续”的条件，这里是 <code>dev.ready</code>	wakeup 不是消息本身，task 醒来后还要重新检查条件
锁	保护共享状态的一段互斥区间，这里保护 <code>dev.ready</code> 和 <code>dev.waiters</code>	不能让完成路径插在“检查条件”和“入队”之间

可靠的等待路径要把“检查条件”和“入等待队列”放在同一个受保护区间里：

```
lock(dev.lock);
while (!dev.ready) {
    prepare_to_wait(&dev.waiters, current);
    unlock(dev.lock);

    schedule();

    lock(dev.lock);
}
finish_wait(&dev.waiters, current);
unlock(dev.lock);
```

设备完成时，中断处理路径做相反的事：

```
irq_handler() {
    lock(dev.lock);
    dev.ready = true;
    complete_current_request();
    wake_up(&dev.waiters);
    unlock(dev.lock);
}
```

这段代码有三个关键点。第一，用 `while`，不用 `if`，因为醒来只表示“条件可能变了”，不保证条件已经满足。第二，`prepare_to_wait` 发生在锁保护下，避免 lost wakeup。第三，`schedule()` 前要释放锁，否则设备完成路径拿不到锁，无法把 `dev.ready` 改成 `true`。

由此可以看到 OS 的核心形状：多条入口路径共同维护同一个谓词，正常执行只是这些路径形成的一种状态序列。

路径	它做什么	不能破坏的关系
等待路径	检查 <code>dev.ready</code> ，不满足就把当前 task 放入 wait queue	检查条件和入队不能被完成路径插到中间
完成路径	设备中断到来，更新请求状态，再唤醒等待者	必须先写完成状态，再唤醒；否则等待者醒来仍看不到结果
调度路径	<code>schedule</code> 选择另一个 runnable task 运行	blocked task 不能还留在 runqueue 里抢 CPU
返回路径	被唤醒后从 <code>schedule</code> 返回，重新检查条件	wakeup 不是消息投递，只表示“条件可能变了”

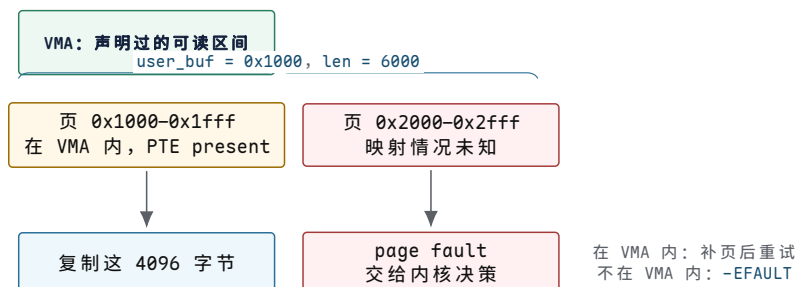
调度、锁、条件变量、futex 和 I/O completion 的共同骨架正是这组关系：先定义一个状态谓词，再定义谁能改它，最后用 wait queue 把“现在不能推进”的 task 挪开。

设备等待可靠之后，再处理 `user_buf`。在普通 C 函数里，`const char* user_buf` 看起来就是一个地址；但在 OS 里，它来自用户程序，内核不能直接相信。

具体地址如下：

```
char* buf = (char*)0x1000;
write(fd, buf, 6000);
```

假设一页是 4096 字节，那么这次写入跨过两页：`0x1000..0x1fff` 和 `0x2000..0x276f`。第一页可能合法，第二页可能根本没有映射。如果内核把它当普通指针直接读，轻则内核自己 fault，重则把不该读的内存泄漏出去。



图示：起点 `0x1000` 合法，只说明第一页可验证；buffer 是一个范围，检查必须按页推进到 `0x276f` 为止。第二页落入两种结局之一：合法但未装入（补页重试），或根本不属于任何 VMA（拒绝）。

这里先引入系统调用边界。`write` 不是普通函数调用到底层设备；它会通过受控入口进入内核。用户态只交出三个值：`fd`、`buffer` 地址、长度。内核必须把这些不可信值变成自己能负责的状态。

最容易写错的检查是只看起点：

```
if (is_user_pointer(user_buf)) {
    memcpy(kbuf, user_buf, len);
}
```

这段代码的问题和前面的忙等类似：它把一个连续过程压成了一个布尔判断。起点合法，只能说明第一个地址看起来属于用户空间；它不能证明后续 `len` 个字节都合法，也不能证明这些页现在都在内存里，更不能证明复制期间另一个线程不会改变映射。OS 需要的是按范围、按页、按权限推进，而不是对一个指针值投信任票。

初始判断	忽略的问题	完整追问
指针不为空	非空地址仍可能没有映射	它是否落在当前进程允许的地址区间内
起点在用户空间	后续字节可能跨到非法页	整个范围是否逐页可读
页表现在有翻译	复制中可能发生缺页、换入或权限变化	<code>fault</code> 能否修复，修复后是否需要重试
地址可读	目标对象未必允许写入	<code>fd</code> 、打开文件对象和权限还要单独检查

核心思想

系统调用进入内核后，首先要解释用户传入的整数、指针和长度：`fd` 是否存在，地址是否可读，长度计算是否越界，以及各类失败应返回什么错误。只有这些条件成立，实际操作才能开始。

`user_buf` 在 OS 里要拆成三层判断：

1. 这个地址是否落在当前进程声明过的某个地址区间里。
2. 这个区间是否允许当前操作，比如写文件时内核要从用户 `buffer` 读。
3. 当前页表里是否已经有可用翻译；没有时，缺页处理能不能补上。

这三层判断落到四个概念上：

词	含义	保存例子里的判断
虚拟地址	用户程序看到的地址编号，不直接等于物理内存位置	<code>0x1000</code> 是用户给出的虚拟地址
VMA	virtual memory area，一段地址区间的“使用意图”	这段地址是不是用户栈、堆或 <code>mmap</code> 文件，是否允许读
PTE	page table entry，一页地址的“当前翻译事实”	这一页现在是否在内存里，是否允许用户态读
page fault	CPU 发现地址翻译或权限不满足，把问题交给内核解释	第二页没有 PTE 时，内核决定是补页还是报错

因此，“读取用户 `buffer`”是一项受控复制过程，期间可能发生访问失败、缺页等待或部分完成。

这里的“可能睡眠”很关键。若第二页是合法文件映射，但内容还不在内存，内核可能需从磁盘把页读进来；读磁盘又会提交块 I/O，当前 `task` 可能睡在缺页等

待或 I/O completion 上。也就是说，一个看似单纯的 `copy_from_user`，可能临时走到调度器和块设备路径。系统调用不是一条不被打断的直线，它会在地址、权限、等待和设备完成之间来回穿行。

```
size_t copied = 0;
while (copied < len) {
    int rc = copy_one_page_from_user(kbuf + copied,
                                     user_buf + copied);

    if (rc == PAGE_FAULT_FIXED) {
        copied += bytes_copied_this_round;
        continue;
    }
    if (rc == BAD_USER_ADDRESS) {
        break;
    }
}
```

两页复制过程如下：

步骤	内核检查到什么	下一步
第一页	地址落在 VMA 内，VMA 允许读，PTE present	复制第一页的数据
第二页	地址落在 VMA 内，但 PTE not-present	触发 page fault，能补页就重试
第二页另一种情况	找不到覆盖它的 VMA	停止复制，返回部分写入或 <code>-EFAULT</code>
内核地址	PTE 标着“只允许内核访问”，用户无权访问	拒绝，不能泄漏内核内存

用户传入的情况	内核看到什么	合理结果
buffer 全部合法且页已在内存	VMA 允许读，PTE present	复制成功，继续写入路径
buffer 合法但页未装入	VMA 允许读，PTE not-present	触发 page fault，分配或读入页面后重试
buffer 前半段合法，后半段非法	前几页复制成功，后一页找不到 VMA	返回已写字节数或 <code>-EFAULT</code> ，不能越界读
buffer 指向内核地址	用户态无权访问内核专用页	拒绝，不能把内核内存泄漏给用户
另一个线程同时修改 buffer	指针仍合法，但内容在变化	内核复制到的是某一时刻的快照，不能假设之后不变

这个边界可以直接观察。下面的程序故意把 `buffer` 指向 `0x1`。这个地址通常不属于进程的有效 VMA；用户态把它作为普通整数传给内核并不违法，但内核不能相信它真的可读。

```
int fd = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
const char* bad = (const char*)0x1;
ssize_t n = write(fd, bad, 16);
printf("n=%zd errno=%d\n", n, errno);
```

系统调用记录中的具体 `fd` 号只服务当前例子，更重要的信息是 `write` 怎样把坏地址转换成用户可见错误：


```
$ strace -e trace=write ./badbuf
write(3, 0x1, 16) = -1 EFAULT (Bad address)
```

程序自己看到的返回值同样能证明这条受控路径。本书实验机上的真实输出：

```
$ ./badbuf
n=-1 errno=14 (Bad address)
```

这里没有出现“内核无条件崩溃”，也没有出现“从地址 `0x1` 读出无效数据”。发生的是一条受控错误路径：CPU 在内核复制用户数据时发现地址访问不成立，进入 page fault；内核发现这次 fault 发生在 `copy_from_user` 这类允许恢复的位置，于是把它翻译成 `-EFAULT` 返回给系统调用。内核里通常会为这类可恢复访问准备异常表。异常表记录特定访存指令及其修复入口：若这些指令发生 fault，内核不按普通缺陷处理，而是进入修复路径，设置错误结果并返回。

观察到的现象	背后的边界	说明
<code>0x1</code> 能作为参数传入	系统调用入口只接收整数值	入口接收参数不代表参数语义有效
<code>write</code> 返回 <code>EFAULT</code>	地址复制边界失败	错误发生在读取用户 buffer，而不是 fd 或磁盘
进程继续运行	fault 被内核修复路径接住	这类 fault 是可报告错误，不是必须 panic 的内核 bug
文件不应被当作完整写入	复制没有得到 16 个可信字节	没有可信输入，就不能推进保存语义

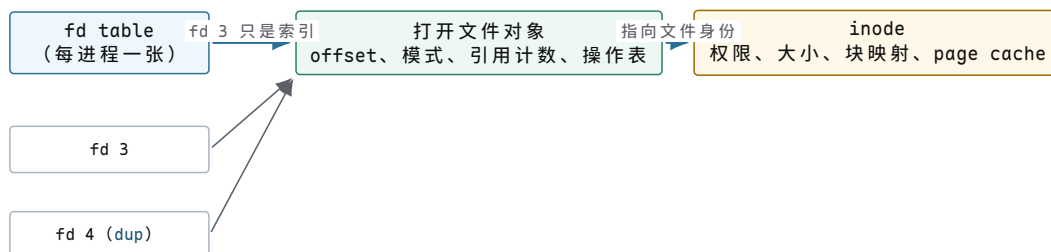
这里有两个边界需要同时成立。第一，系统调用边界会把不可信输入变成内核拥有的稳定状态；小数据通常复制，大 I/O 可能 pin page，也就是临时固定页面，保证设备 DMA 完成前页面不会被回收或移动。第二，page fault 不是单纯的“内存不够”，它是硬件把一个地址访问问题交给内核解释：能修就修，不能修就返回错误或发信号。

`user_buf` 解释完，还剩 `fd`。用户程序通常这样写：

```
int fd = open("data.tmp", O_WRONLY | O_CREAT | O_TRUNC, 0644);
write(fd, buf, len);
```

`fd` 看起来只是一个整数，例如 3。但它不是文件本体。它是当前进程 fd table 里的一个索引。fd table 可以先理解成“这个进程手里的资源号码表”。fd table 指向的 file object 可以理解为“打开文件对象”：它表示一次 `open` 或 `dup` 关系背后的内核对象，可对应 POSIX 语义里的 open file description。

层次	在本例中的含义	它回答的问题
fd 整数	用户态拿到的号码，例如 3	用户后续用哪个号码引用这次打开得到的对象
fd table	当前进程的号码表	3 号槽是否存在，指向哪个内核对象
打开文件对象 (file object)	一次打开文件形成的内核对象	当前 offset、读写模式、引用计数、具体操作函数
inode	文件系统里的文件身份和元数据	这个文件的权限、大小、块映射、page cache 归属



dup: 两个 fd 槽共享同一个打开文件对象和 offset

图示：整数 fd 只是进程 fd table 的索引；offset 和引用计数属于打开文件对象；inode 才是文件身份。dup 复制的是槽位，不是对象——这是后面所有共享 offset 现象的根源。

为什么要分这么多层？一个 dup 例子能直接暴露 fd 和文件本体之间的差别：

```
int a = open("data", O_WRONLY);
int b = dup(a);
write(a, "A", 1);
write(b, "B", 1);
```

dup(a) 会创建另一个 fd，但它们通常共享同一个打开文件对象，也就是共享这个对象里的 offset。因此第一次写入把 offset 从 0 推到 1，第二次写入接着从 1 写，文件内容会是 AB。如果把 fd 误认为“文件本体”，就解释不了为什么两个整数会共享 offset，也解释不了 close(a) 后 b 还能继续写。

这个共享 offset 可以通过一个小程序清楚地观察到：

```
int a = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
int b = dup(a);

write(a, "A", 1);      // offset: 0 -> 1
write(b, "B", 1);      // offset: 1 -> 2
lseek(a, 0, SEEK_SET); // shared offset: 2 -> 0
write(b, "C", 1);      // offset: 0 -> 1
```

如果 a 和 b 是两个互不相关的文件对象，lseek(a, 0, SEEK_SET) 不应该影响 b。但 dup 的结果会共享打开文件对象，所以 b 的下一次写入从 0 开始，覆盖第一个字节。最终文件内容不是 ABC，而是 CB。

```
$ ./dup-offset
$ od -An -c data
 C  B
```

动作	共享 offset	文件内容
write(a,"A",1)	0 变成 1	A
write(b,"B",1)	1 变成 2	AB
lseek(a,0)	2 变成 0	AB，只是位置变了
write(b,"C",1)	0 变成 1	CB，第一个字节被覆盖

这比口头说“fd 是引用”更有力。用户手里确实有两个整数，但内核中只有一个共享的打开文件对象；offset 属于这个对象，不属于整数变量本身。若代码需要两个互不影响的文件位置，就不能用 dup 得到第二个位置，而要重新 open，让内核创建另一个打开文件对象。

再看一个更容易在工程里踩到的例子：

```
int fd = open("data", O_WRONLY);
int copy = dup(fd);
close(fd);
write(copy, "X", 1);
```

`close(fd)` 关闭的是当前进程 `fd` table 里的一个槽位；只要 `copy` 还引用同一个打开文件对象，这个打开对象就不能释放。于是“文件是否还能写”不取决于整数 `fd` 这个变量是否还存在，而取决于打开文件对象的引用计数和权限状态。在 `fork`、`exec`、`socket` 和设备文件中也会反复遇到同一条规则：用户态整数只是引用入口，真正的生命周期在内核对象上。

现象	如果把 <code>fd</code> 当文件本体会怎样误解	用对象状态怎样解释
<code>dup</code> 后共享 <code>offset</code>	以为两个整数各写各的	两个 <code>fd</code> 槽指向同一个打开文件对象
<code>close(a)</code> 后 <code>b</code> 还能写	以为文件已经关闭	只减少一个引用，打开对象仍被 <code>b</code> 持有
<code>fork</code> 后子进程继承 <code>fd</code>	以为复制了整个文件	父子 <code>fd</code> table 槽可引用同一打开对象
<code>exec</code> 后部分 <code>fd</code> 消失	以为程序替换会关闭全部资源	<code>FD_CLOEXEC</code> 决定哪些引用在提交点被裁剪

这层关系在真实系统里可以直接看到。下面的程序打开 `data.tmp`，用 `dup` 复制一个 `fd`，再打开所在目录，然后睡眠一分钟；趁它睡眠时去读它的 `fd` 目录（输出删去了完整路径）：

```
$ ./fd_demo &
pid=26635 fd=3 copy=4 dirfd=5
$ ls -l /proc/26635/fd
3 -> .../data.tmp
4 -> .../data.tmp
5 -> .../
$ grep -E '^(State|Threads)' /proc/26635/status
State: S (sleeping)
Threads: 1
```

`fd 3` 和 `fd 4` 指向同一个文件，这正是 `dup` 共享打开文件对象的直接证据；`fd 5` 是一个目录——目录也能被打开并占有 `fd` 槽位，后面的 `fsync(dirfd)` 依赖的正是这个事实。睡眠中的进程 `State` 是 `S`，对应前面讲的 `TASK_INTERRUPTIBLE`。

这三层在 Linux 中分别由 `fd` 表、打开文件对象和 `inode` 对象表示。`fd` 表保存整数到对象引用的映射；打开文件对象保存当前 `offset`、引用计数和操作方法；`inode` 表示持久文件身份，`page cache` 则与该文件的映射对象关联。这里先确定一个结论：整数 `fd`、打开文件对象和 `inode` 是三层不同对象，`offset` 属于中间的打开文件对象。

为了避免把 `dup` 的结论误记成“同一个路径就共享 `offset`”，再看重新 `open` 的反例：

```
int a = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
int b = open("data", O_WRONLY);

write(a, "A", 1);      // a offset: 0 -> 1
write(b, "B", 1);      // b offset: 0 -> 1
```

这次最终内容通常是 `B`，不是 `AB`。原因不是路径名变了，而是两次 `open` 创建

了两个打开文件对象；它们都指向同一个 inode，但各自有独立 offset。第一次写入用 `a` 的 offset 0，把文件变成 `A`；第二次写入用 `b` 的 offset 0，又从文件开头覆盖成 `B`。

```
$ ./open-twice-offset
$ od -An -c data
B
```

写法	内核对象关系	offset 结果
<code>dup(a)</code>	两个 fd 槽指向同一个打开文件对象	<code>a</code> 写完后， <code>b</code> 接着往后写
重新 open	两个打开文件对象指向同一个 inode	<code>a</code> 的 offset 不影响 <code>b</code> 的 offset
<code>fork</code> 继承	父子进程的 fd 槽可指向同一个打开文件对象	子进程推进 offset，父进程随后也从新位置写

`fork` 的情况也可以直接观察。下面的程序让子进程先写，父进程等子进程退出后再写：

```
int fd = open("data", O_WRONLY | O_CREAT | O_TRUNC, 0644);
pid_t pid = fork();

if (pid == 0) {
    write(fd, "C", 1);
    _exit(0);
}

waitpid(pid, NULL, 0);
write(fd, "P", 1);
```

输出会显示父进程不是从 offset 0 覆盖子进程写入的字节，而是接在后面写：

```
$ ./fork-offset
$ od -An -c data
C P
```

`waitpid` 只是在这个例子里固定执行顺序，真正决定 `CP` 的是父子进程共享同一个打开文件对象。子进程把共享 offset 从 0 推到 1；父进程被唤醒后继续使用同一个对象，所以从 1 开始写。由此可以把 fd 生命周期说清楚：进程可以复制，fd 槽可以复制，路径名可以相同，但打开文件对象才是 offset、状态标志和引用计数的承载者。

有了 fd 和 buffer 两条账，`write(fd, buf, len)` 的路径可以写成：

```
write(fd, buf, len)
-> find fd in current process fd table
-> check the open file object allows write
-> copy data from user buffer
-> update file/page-cache state
-> return byte count or -errno
```

这一阶段会用到 `page cache`。它是内核用内存缓存文件内容的一组页面。写普通文件时，内核常常先把用户数据复制到 page cache，把相关页面标记为 `dirty`，然后 `write` 就可以返回。`dirty` 的意思是“内存里的文件页已经比磁盘上的版本更新，还需要写回”。

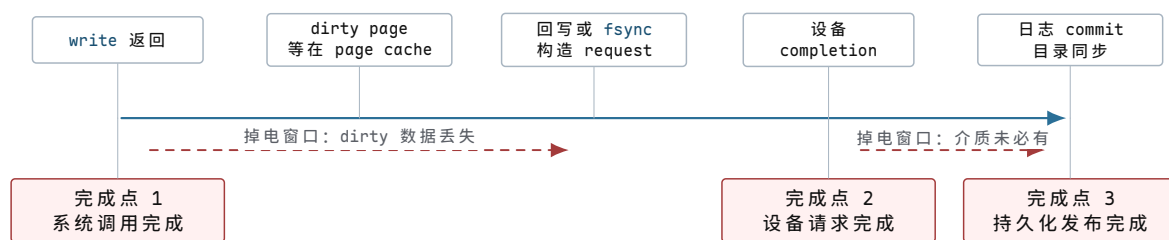
`page cache` 保存的是文件内容在内核内存中的副本。数据进入 `page cache` 后，

后续读取可以直接复用，多个小写也能够合并成更合适的块 I/O；与此同时，完成边界增加了。`write` 返回时，数据可能只到达内核内存；后台写回或 `fsync` 随后把它组织成块请求；`completion` 表示相应设备请求结束；设备缓存和文件系统日志还会继续影响崩溃后哪些状态可信。

边界	此时数据在哪里	还不能承诺什么
<code>copy_from_user</code> 结束	内核 buffer 或 page cache 中	目标文件名未必已经安全发布
<code>write</code> 返回	接口语义接受了若干字节	设备未必见过这批数据
块请求提交	request 和 DMA 映射交给驱动/设备	<code>completion</code> 未到，buffer 不能释放
设备 <code>completion</code>	某次设备请求完成或失败	介质持久性仍可能依赖 flush/FUA/日志
<code>fsync</code> 成功	文件相关数据和必要元数据推进到持久化边界	目录项更新仍可能需要目录 <code>fsync</code>

最后回到最初的 `print("saved")`。这行太早，因为保存请求至少有三个不同的完成点：

完成点	说明	反例
系统调用完成	<code>write</code> 返回，内核已经按接口语义接受了一部分数据	机器立刻掉电，dirty page 还没写到设备
设备请求完成	块层或驱动收到 <code>completion</code> ，某次 I/O 已结束	设备缓存还没 flush，断电后介质上未必有数据
持久化发布完成	<code>fsync</code> 、日志 commit、 <code>rename</code> 和目录同步形成可恢复状态	只同步文件内容，不同步目录项，崩溃后新名字可能丢失



图示：三个完成点在同一条真实时间线上，但彼此之间隔着可观的距离。两个红色区间是掉电会造成不同后果的窗口：第一段里数据只在内存，第二段里设备自己可能还攥着没 flush 的缓存。

这段距离不是修辞，可以直接实测。给 64KiB 的 `write` 和紧随其后的 `fsync` 分别计时，本书实验机（aarch64 容器、闪存介质，数字只用来说明量级）上的真实输出是：

```
write=83us fsync=3180us (wrote 65536 bytes)
write=43us fsync=195us (wrote 65536 bytes)
write=26us fsync=140us (wrote 65536 bytes)
```

`write` 几十微秒就返回，因为它只是把数据复制进 page cache 并标上 dirty；`fsync` 必须等设备真正确认，第一次甚至花了 3 毫秒多——完成点 1 和完成点 3 之间隔着一到两个数量级。中间各层仍在维护 page cache、块层队列、驱动请求和设备固件状态；本章后半部分的每一条路径，都是在解释这段距离里发生了什么。

按时间顺序展开后，三个完成点的差异更明显：


```

write(fd, buf, len)
-> copy from user
-> mark page cache dirty
-> return len

background writeback or fsync(fd)
-> build block request
-> map DMA buffer
-> device completion
-> record writeback error if any

rename("data.tmp", "data")
fsync(dirfd)
-> publish the name
-> make the directory update recoverable

```

三段边界各自承担不同含义。

`write` 返回，只说明内核按照 `write` 的接口语义接受了数据，或者接受了一部分数据。对普通文件来说，它经常停在 page cache 层：内存里有新内容，磁盘上未必有。

`fsync(fd)` 把语义往前推一层：内核要把这个文件相关的 dirty 数据和必要元数据推进到设备，并处理过程中发现的 I/O error。它比 `write` 强，但它通常只针对这个文件对象。

时间线里出现的 DMA 和 completion 是设备 I/O 的两个关键边界。DMA 是 direct memory access，意思是设备可以在内核授权后直接读写某段内存，不必让 CPU 一个字节一个字节搬。completion 是完成记录，表示某个提交给设备的请求结束了，内核可以根据它更新请求状态、释放 buffer、唤醒等待者。

`rename("data.tmp", "data")` 是发布新名字：让目录树里正式名字指向新文件。目录项本身也是元数据，所以很多崩溃安全写法还要打开目录并 `fsync(dirfd)`，让“名字更新”也跨过持久化边界。

如果崩溃发生在不同位置，恢复结果不同。

崩溃位置	可能看到的状态	为什么不能简单说“保存成功”
<code>write</code> 返回后、 <code>fsync</code> 前	内存里有 dirty page，磁盘旧内容还在	<code>write</code> 的契约不是持久化契约
<code>fsync(fd)</code> 中途	部分块已写，错误可能稍后报告	I/O error 需要被记录并返回给调用者
<code>rename</code> 后、目录未 <code>fsync</code>	文件内容可能在，但名字更新未必可恢复	目录项本身也是元数据，也需要提交边界
日志 commit 前	日志里有准备信息，但不能当成功事务	恢复时必须丢弃未 commit 的更新
日志 commit 后	恢复程序能重放或完成更新	成功来自可验证记录，不来自函数执行到最后

因此，“保存成功”不能只看一行返回值。它要说明数据进入了哪个对象、越过了哪个边界、失败时谁负责报告、崩溃后用什么证据恢复。page cache、块层、驱动、日志和观测工具，本质上都在追问这条时间线的不同边界。

前面的修正可以收束成一组操作系统契约：谁拥有状态，谁能修改状态，不能继续时睡在哪里，失败如何报告，以及外部依据什么证据判断操作已经完成。

继续沿保存请求走下去。最初的函数只有局部变量和设备寄存器；一旦要支持多个程序、多个请求和失败恢复，内核必须把隐含状态变成显式对象。

原来藏在哪里	内核对象	这个对象必须回答的问题
当前正在执行的函数	task	它能否运行，睡在哪里，怎样从同一位置恢复
用户传入的指针	VMA、PTE、page	这个地址是否合法，读写执行权限是什么，fault 能否修复
一个整数句柄	fd table、打开文件对象	这个 fd 指向谁，是否可写，关闭或 exec 时怎样处理
设备内部队列	request、descriptor、completion	哪个请求完成了，buffer 归谁，错误怎样返回
缓存中的文件页	page cache、inode	哪些字节已修改，何时写回，谁能看到
落盘顺序	journal、commit record	崩溃后哪些更新可信，哪些必须丢弃或重放

OS 对象把函数调用期间无法完整承载的状态保存到内核中。对象建立后，还要配套权限、生命周期、等待队列和完成证据；缺少这些规则，系统便无法在不可信程序、慢设备和故障路径中保持一致。

本章使用的 task、wait queue、VMA 等名称都对应具体的运行时对象。先给出对象与职责的对照，后续章节再展开字段和状态转换：

本章叫法	运行时对象与职责
task	任务对象保存执行现场、调度状态以及指向地址空间和文件表的引用
wait queue	等待队列头与等待节点保存“谁在等待哪个条件”
VMA	虚拟内存区域记录一段地址范围的用途与访问策略
PTE	页表项记录当前映射、权限和处理器可见状态
fd table	文件描述符表保存整数句柄到打开文件对象的映射
打开文件对象	保存当前偏移、打开标志、引用计数和操作方法
page cache	文件映射对象与缓存页共同保存文件内容的内存副本
request	块请求对象保存设备操作、范围、缓冲区和完成身份
completion	完成对象保存异步操作的结果并连接等待者
日志提交记录	事务对象与提交记录界定崩溃后可以重放的更新集合

这里列出对象不是为了记忆结构体名称，而是为了确认每一项状态由谁负责。后面任何一章提到“打开文件对象”时，都应当能够继续说明它保存什么状态、由谁引用以及何时释放。

保存请求经过的内核对象

最初版本的保存函数只看见一段顺序代码。真实保存请求从进入内核到对外宣布完成，中间每一步都要有对象承载状态、有边界说明语义、有证据报告失败。把这些对象按处理顺序排列，保存路径就不再是一串松散名词。

先把同一个保存请求放到真实 OS 语义里。用户程序看到的是几行 API：

```
int fd = open("data.tmp", O_WRONLY | O_CREAT | O_TRUNC, 0644);
write(fd, buf, len);
fsync(fd);
close(fd);
rename("data.tmp", "data");
```

```
int dirfd = open(".", O_RDONLY | O_DIRECTORY);
fsync(dirfd);
close(dirfd);
```

这几行代码不是只能靠想象理解。可以用系统调用跟踪工具观察它大致穿过了哪些内核入口。下面的输出是删去无关参数后的示意，真实机器上路径名、fd 号码和标志细节可能不同，但边界顺序是要看的重点：

```
$ strace -e trace=openat,write,fsync,close,renameat ./save-demo
openat(AT_FDCWD, "data.tmp", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
write(3, "hello\n", 6) = 6
fsync(3) = 0
close(3) = 0
renameat(AT_FDCWD, "data.tmp", AT_FDCWD, "data") = 0
openat(AT_FDCWD, ".", O_RDONLY|O_DIRECTORY) = 3
fsync(3) = 0
close(3) = 0
```

`strace` 只能看到用户态和系统调用边界，不能直接告诉你 page cache、DMA 或文件系统日志里发生了什么。但它给了第一层证据：保存不是一个内部过程完全不可见的函数，而是一串明确的内核入口。每一行返回 0 或正数，只说明对应系统调用自己的契约成立；它不能自动证明后续边界已经成立。

观察到的入口	这行真正越过的边界	还没有自动保证什么
<code>openat</code>	路径解析、权限检查、fd 分配成功	后续写入一定成功
<code>write</code>	内核接受了 6 个字节，并推进到目标对象的写路径	数据已经在断电后可恢复
<code>fsync(fd)</code>	文件数据和必要文件元数据被要求推进到持久化边界	目录项名字更新已经可恢复
<code>close</code>	fd 槽位释放，并可能暴露延迟错误	另一个 fd 不再引用同一个打开文件对象
<code>renameat</code>	目录项从临时名字发布到正式名字	崩溃后一定能看到新名字
<code>fsync(dirfd)</code>	目录对象的更新被要求推进到持久化边界	更上层业务事务已经提交

这类观察方式很重要：先看用户能写出的代码和能抓到的输出，再解释输出背后有哪些看不见的状态对象。没有这一步，page cache、fd table、journal 很容易变成互不相干的名词；有了这一步，它们都在回答同一个问题：这行返回值背后到底有什么证据。

这段代码只保留语义主线；真实工程还要检查返回值，循环处理 partial write、EINTR 和清理路径。即使只看这条主线，一个看似简单的“保存”也已经穿过了主要 OS 对象。

1. `open` 先走路径解析、权限检查和 fd 分配。成功后，用户拿到的只是当前进程 fd table 里的一个整数索引。
2. `write` 通过系统调用进入内核，复制或固定用户 buffer，把数据推进打开文件对象、page cache、socket buffer 或设备对象。返回值不等于持久化成功。
3. 如果用户页或文件页不在内存，路径可能触发 page fault、page cache miss、块 I/O 和调度睡眠。

4. 若块设备异步写入，内核要建立 request，映射 DMA buffer，提交描述符，等待 completion，再解除映射。
5. `fsync` 把“内核已经接受写入”推进到更强的持久化边界，但仍要处理设备错误、flush、日志提交和元数据顺序。
6. `rename` 把临时文件发布成正式名字；目录 `fsync` 让名字更新本身也有崩溃恢复语义。

把这些状态整理成一张路径表：

阶段	状态对象	关键边界	失败证据
入口	task、入口现场	用户态切到内核态，保存返回位置	错误码、审计记录
授权	fd、打开文件对象、inode	fd 是否存在，目标是否可写	-EBADF、-EACCES
复制	VMA、PTE、page	用户 buffer 是否可读	-EFAULT、部分写入
缓存	page cache、inode	dirty 页只是内存状态	dirty/writeback 计数、回写错误
提交 I/O	request、DMA map	buffer 在 completion 前不能释放	timeout、I/O error、completion status
持久化	journal、flush、directory entry	数据和名字是否崩溃后仍可解释	fsync 错误、replay、fsck 日志

这张表把保存请求从一个 API 调用拆成了六个阶段：入口、授权、复制、缓存、异步提交和持久化发布。任何一个阶段定义不清，程序都可能给出与实际状态不一致的成功结果。

常见误区

保存路径里最容易混淆的五个边界：

- `write` 返回成功，不等于数据已经落盘。
- 线程处于 `runnable`，不等于正在运行。
- `fd` 是整数句柄，不等于文件本体。
- 虚拟地址落在 VMA 内，不等于 PTE 已经存在。
- 唤醒函数被调用，不等于等待者已经继续执行。

再把这段保存代码当成一个完整案例看。很多程序会先写成这样：

```
bool save_bad(const char* path, const char* buf, size_t len) {
    int fd = open(path, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) return false;
    if (write(fd, buf, len) != len) return false;
    close(fd);
    return true;
}
```

它的表面意思很直接：打开目标文件，截断旧内容，写入新内容，关闭，返回成功。但按前面的状态路径逐项检查，会发现它存在三处失败窗口。

位置	发生了什么	失败后用户可能看到什么
----	-------	-------------

<code>O_TRUNC</code>	旧文件长度先变成 0	后续写失败时，旧内容已经丢失
一次 <code>write</code>	可能只复制部分字节，或停在 <code>page cache</code>	返回短写时，新文件是半成品
<code>close</code> 后返回	未检查 <code>close/fsync</code> 的延迟错误	后台写回失败，用户却看到保存成功

把它放到一个具体文件上看，会更容易发现危险。假设正式文件 `data` 原来保存的是 `"old\n"`，这次要保存的新内容是 `"new\n"`。用户想要的结果其实很严格：失败时最好还能读到旧版本；成功时读到完整新版本；最糟糕的状态是旧版本被破坏，而新版本又不完整。

执行到哪里	<code>data</code> 可能处于什么状态	为什么危险
<code>open(... O_TRUNC)</code> 之后	长度已经变成 0	还没写任何新数据，旧版本已经被破坏
<code>write</code> 短写之后	可能只有 <code>"ne"</code> 这样的前缀	系统调用允许部分推进，不能假设一次写完
<code>write</code> 返回、未 <code>fsync</code>	<code>page cache</code> 里可能是新内容，磁盘上可能仍是旧内容或部分更新	函数返回和持久化不是同一个边界
<code>close</code> 未检查错误	<code>fd</code> 被释放，但延迟 I/O 错误可能被忽略	调用者看到 <code>true</code> ，实际保存可能失败

改进方向不是机械记住“要用临时文件”，而是从失败路径推出临时文件的必要性：旧文件不能在新内容可靠之前被破坏。更稳的写法先把新内容写进同目录下的临时文件，确认它的数据和必要元数据已经推进，再用 `rename` 一次性替换名字。

先把这个推导拆开。错误版本从正式文件本身下手，所以第一步 `O_TRUNC` 就已经破坏旧版本；后面任何失败都会让用户处在尴尬状态：旧内容没了，新内容也没完整提交。临时文件版本把“准备新内容”和“发布新名字”分成两个阶段。准备阶段只影响临时名字，失败就删除临时文件；发布阶段用 `rename` 切换目录项，使正式名字尽量只在旧版本和新版本之间切换。

设计选择	它来自哪个失败	边界含义
不用 <code>O_TRUNC</code> 直接改正式文件	写到一半失败会破坏旧版本	旧文件在新版本准备好前保持可用
使用同目录临时文件	跨文件系统 <code>rename</code> 不能保证同样语义	临时文件和正式文件处在同一个目录事务范围内
循环 <code>write_all</code>	一次 <code>write</code> 可能短写或被信号打断	未写完整就不能进入发布阶段
先 <code>fsync(tmpfd)</code>	<code>page cache</code> 中的新内容不等于持久内容	新文件内容先越过文件持久化边界
再 <code>rename</code>	正式名字只能指向旧版本或新版本	发布动作集中到目录项切换
最后 <code>fsync(dirfd)</code>	目录项本身也是元数据	崩溃后正式名字更新有恢复证据

先展开 `write_all`。它不是一般性的防御写法，而是因为 `write` 的基本契约就是“返回本次实际推进了多少字节”。只要返回值小于请求长度，调用者就不能把完整数据视为已经进入保存路径。

```

bool write_all(int fd, const char* buf, size_t len) {
    size_t off = 0;

    while (off < len) {
        ssize_t n = write(fd, buf + off, len - off);

        if (n > 0) {
            off += (size_t)n;
            continue;
        }

        if (n == 0) {
            return false;          // no progress
        }

        if (errno == EINTR) {
            continue;              // interrupted before a final result
        }

        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            wait_until_writable(fd);
            continue;
        }

        return false;              // ENOSPC, EIO, EFAULT, ...
    }

    return true;
}

```

假设要写入 6 个字节 `hello\n`。第一次 `write` 只返回 2，表示 `he` 已经推进；第二次被信号打断，返回 `-1/EINTR`，表示没有得到一个最终完成结果；第三次返回 4，才把剩下的 `llo\n` 推进。此时 `off` 的变化是保存程序必须记住的状态。

调用	返回	<code>off</code> 变化	保存程序该做什么
第 1 次 <code>write</code>	2	0 变成 2	继续写剩余 4 字节
第 2 次 <code>write</code>	<code>-1/EINTR</code>	仍是 2	不能报成功，重试同一段剩余数据
第 3 次 <code>write</code>	4	2 变成 6	才能进入 <code>fsync(tmpfd)</code>

`EINTR`、`EAGAIN` 和短写看似只是接口细节，但它们对应三种不同边界。`EINTR` 表示调用被信号路径打断；`EAGAIN` 表示对象现在不能推进，调用者要等待可写条件；短写表示系统已经接受了一部分数据，下一次必须从新 `offset` 继续。若保存程序不记录 `off`，就会重复写前缀、漏写后缀，或者把半成品当完整版本发布。

```

bool save_better(const char* dir, const char* name,
                 const char* buf, size_t len) {
    int dirfd = open(dir, O_RDONLY | O_DIRECTORY);
    if (dirfd < 0) return false;

    char tmp_name[64];
    int tmpfd = open_temp_at(dirfd, tmp_name, sizeof(tmp_name));
    if (tmpfd < 0) goto fail_dir;

    if (!write_all(tmpfd, buf, len)) goto fail_tmp;
}

```

```

if (fsync(tmpfd) < 0) goto fail_tmp;
if (close(tmpfd) < 0) {
    tmpfd = -1;
    goto fail_unlink;
}
tmpfd = -1;

if (renameat(dirfd, tmp_name, dirfd, name) < 0) goto fail_unlink;
if (fsync(dirfd) < 0) goto fail_dir;

close(dirfd);
return true;

fail_tmp:
    close_if_open(tmpfd);
fail_unlink:
    unlinkat(dirfd, tmp_name, 0);
fail_dir:
    close(dirfd);
    return false;
}

```

这段代码不是完整库函数；真实工程还要处理 `EINTR`、权限、临时文件命名、保留文件模式、跨文件系统 `rename` 等细节。但它已经把关键边界摆清楚了。

逐行分析它的状态推进会更清楚。打开目录并非多余动作，因为最后要同步目录项；创建临时文件不是形式需要，而是为了让新内容有独立承载对象；`write_all` 不是普通循环，而是在处理“接口允许短推进”这个契约；`fsync(tmpfd)` 把临时文件内容向持久化推进；`close(tmpfd)` 还可能暴露延迟错误；`renameat` 才把正式名字切到新文件；`fsync(dirfd)` 则让这个名字变化本身在崩溃后可恢复。

处理阶段	此刻系统必须记住什么	如果失败该怎么解释
<code>open(dir)</code>	目录对象和权限	目录打不开，保存根本没有进入准备阶段
<code>open_temp_at</code>	临时 inode、临时目录项、fd 引用	只清理临时对象，不影响正式文件
<code>write_all</code>	已写入字节数、dirty page、可能的短写	未完整写入，不允许发布
<code>fsync(tmpfd)</code>	数据和必要元数据的提交结果	返回错误说明新版本不可信
<code>renameat</code>	目录项从旧 inode 切到新 inode	失败时正式名字仍应指向旧版本
<code>fsync(dirfd)</code>	目录项更新的持久化证据	失败时必须向上报告，不能宣称完全保存

动作	它保护什么	崩溃后恢复含义
同目录临时文件	保证 <code>rename</code> 不跨文件系统，旧名字暂时不变	旧文件仍可作为最后成功版本
<code>write_all</code>	处理短写、 <code>EINTR</code> 、 <code>EAGAIN</code> 等推进问题	临时文件要么完整写入，要么不发布
<code>fsync(tmpfd)</code>	推进临时文件数据和必要元数据	新内容有机会在崩溃后被读回

<code>rename</code>	原子切换目录项指向	名字层面看到旧版本或新版本，不应看到半个名字
<code>fsync(dirfd)</code>	推进目录项更新本身	崩溃后正式名字更新有恢复证据

再用同一个 `data = "old\n"`、新内容 `"new\n"` 的例子看崩溃点。下面的表不假设某一种具体文件系统实现细节，只抓通用语义边界：正式名字什么时候还能指向旧版本，临时文件什么时候可以被丢弃，什么时候才能对用户说“新版本已经发布”。

崩溃发生在	重启后合理看到的状态	保存程序应该如何解释
临时文件创建前	<code>data</code> 仍是旧版本	保存没有开始影响正式文件
<code>write_all</code> 中途	<code>data</code> 仍是旧版本，可能残留半个临时文件	临时文件不能发布，清理后重试
<code>fsync(tmpfd)</code> 失败	<code>data</code> 仍是旧版本，新内容不可信	必须返回失败，不能继续 <code>rename</code>
<code>renameat</code> 前	<code>data</code> 仍指向旧版本，临时文件可能完整	新版本还没有发布给正式名字
<code>renameat</code> 后、目录未同步	运行时能看到新名字，但崩溃恢复后名字更新可能没有证据	不能把目录项持久化当成已经完成
<code>fsync(dirfd)</code> 成功后	<code>data</code> 应当指向可恢复的新版本	可以向上层报告保存跨过发布边界

这张表比“用临时文件更安全”更重要。它说明安全不是来自某个函数名，而是来自一组顺序关系：先保护旧版本，再准备新版本，再持久化新版本，再发布名字，最后持久化名字。任何一步失败，都要停在还能解释的状态上。

这段保存代码把前面的对象串成了一条完整链路。`task` 解释了为什么等待 I/O 时不能忙等；VMA/PTE 解释了 `buf` 为什么要复制检查；`fd`、打开文件对象和 `inode` 解释了为什么整数句柄不是文件本体；`page cache` 解释了 `write` 为什么可以早于设备完成返回；`request/DMA/completion` 解释了内核如何跟踪异步设备结果；`fsync/rename/目录 fsync` 解释了“保存成功”如何从内存状态变成崩溃后可解释的证据。

再看“对象”这个词，它就不再是抽象名词。对象就是内核为了记住某类状态而建立的记录。前面的保存案例已经出现了多类对象：

对象	它记住什么	没有它会怎样
<code>task</code>	当前执行流是否 <code>running</code> 、 <code>runnable</code> 、 <code>blocked</code> ，睡在哪里	设备慢时只能忙等，或者睡了以后无法恢复
<code>wait queue</code>	哪些 <code>task</code> 在等同一个谓词	设备完成后不知道该叫醒谁，容易 <code>lost wakeup</code>
VMA/PTE	地址区间的意图和单页翻译事实	用户任意传入一个指针，内核不知道该复制、补页还是拒绝
<code>fd table/打开文件对象</code>	整数 <code>fd</code> 指向哪个打开对象， <code>offset</code> 和读写权限是什么	<code>dup</code> 、 <code>close</code> 、共享 <code>offset</code> 都解释不清
<code>page cache</code>	文件内容在内存中的页面，哪些是 <code>dirty</code>	<code>write</code> 返回后无法说明数据停在哪里

请求/完成记录	哪次设备 I/O 已提交，哪次已经完成	中断来了也不知道对应哪个保存请求
日志/提交记录	哪些更新崩溃后可信	写一半掉电后无法区分成功更新和半成品

这张表背后的判断规则是：只要某个状态会被多个路径共享，或者失败后需要恢复，就不能只藏在一个函数局部变量里。内核必须给它名字、所有者、修改入口、等待位置和失败证据。

同样的状态组织方式也会出现在别的对象上。socket 会把网络连接变成对象；地址空间会把整套页表和 VMA 组织起来；块层会把一次大写入拆成多个 request；安全子系统会把权限判断变成可审计的拒绝路径。对象一旦进入内核，就必须说明它记录什么、谁能改它、发生错误后如何给出证据。

保存案例还暴露出一个分层事实：同一个词在用户 API、内核对象和硬件事件三层里含义不同。用户说地址，可能只是一个指针值；内核说地址，要看 VMA、PTE 和权限；硬件说地址，要看页表翻译、TLB 和物理地址。用户说完成，可能只是函数返回；内核说完成，可能只是 page cache 或 request 状态更新；设备说完成，可能只是某次 I/O 结束。

因此，同一个动作要同时放进三层：用户 API 层、内核对象层、硬件事件层。

说法	用户 API 层	内核对象层	硬件层
地址	指针值，例如 0x1000	VMA/PTE 和权限	页表翻译、物理地址、设备寄存器地址
可读	read 不会立刻阻塞	buffer/队列中有数据或条件满足	中断、completion、DMA 写入
写完成	系统调用返回字节数	page cache 或 request 状态更新	设备接受请求，介质 flush 可能还没发生
上下文	用户调用链和线程身份	task、保存的返回位置、内核栈	程序计数器、栈指针、寄存器
错误	errno 或信号	对象状态拒绝、权限失败、生命周期结束	fault、设备错误、中断错误

以“写完成”为例。写普通文件时，用户看到 write 返回；内核看到 page cache 被改成 dirty；设备可能还什么都没看到。等后台回写或 fsync 提交 request 后，设备才开始处理这次 I/O。等 completion 到来，内核知道这次设备请求结束了。等 flush 或日志 commit 过了，崩溃恢复才有证据。这些都叫“完成”，但边界完全不同。

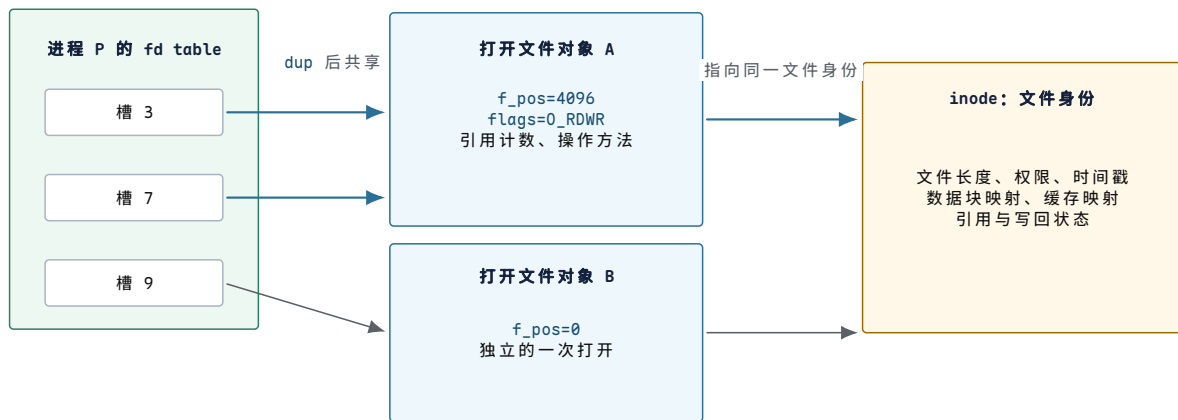
再看“可读”。普通文件几乎总是可读，因为内核可以从 page cache 或磁盘推进；socket 可读表示接收队列有数据、连接关闭或错误；设备 fd 可读可能表示驱动队列里有 completion。用户只看到 poll 返回，但内核必须把对象状态和硬件事件翻译成统一接口。

把一次写入展开为对象账本

前面已经沿调用顺序列出了 fd、打开文件对象、inode、page cache 和 request。仅知道这些名称仍然不足以解释一次写入：fd 为什么能够共享偏移，修改文件内容时为什么还要修改 inode，系统调用已经返回时又是谁继续记住尚未完成的设备请求。要回答这些问题，必须把对象之间的引用和每个对象保存的状态同时展开。

整数 fd 只负责找到一次打开实例。

设进程先后执行 `open("data", O_RDWR)` 和 `dup(fd)`。两个整数槽位指向同一个打开文件对象，因此它们共享当前偏移；另一个进程独立 `open` 同一路径时，会得到新的打开文件对象，偏移互不影响。三次打开最终可以指向同一个 `inode`，因为 `inode` 表示文件身份，而不是某一次打开行为。



图示：fd、打开文件对象与 `inode` 的引用关系。槽 3 和槽 7 共享打开文件对象 A，所以任一槽推进 `f_pos`，另一个槽随后观察到同一偏移；槽 9 拥有独立偏移，但三个槽最终可以到达同一个 `inode`。

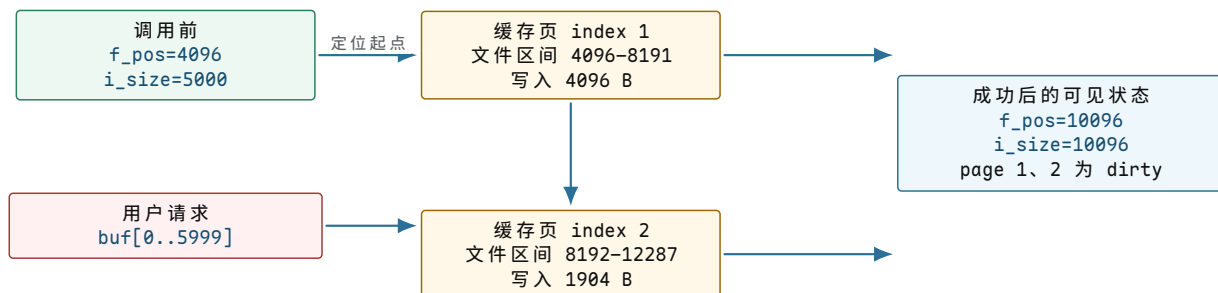
这三个层次不能合并为一个“文件结构”。`fd table` 是进程看到的句柄空间，它要处理分配、关闭和 `close-on-exec`；打开文件对象保存一次打开行为的可变状态；`inode` 保存不依赖某次打开的文件身份和元数据。若把偏移直接放进 `inode`，所有独立打开者都会相互改变读写位置；若把权限和数据块映射放进 `fd` 槽位，`dup`、进程间传递 `fd` 以及关闭其中一个槽位都会失去明确语义。

对象	关键状态	主要修改者	生命周期边界
fd 槽位	指向打开文件对象的引用、fd 级标志	<code>open</code> 、 <code>dup</code> 、 <code>close</code> 、 <code>exec</code>	槽位关闭后可被后续 <code>open</code> 复用
打开文件对象	当前偏移、打开标志、引用计数、操作方法	<code>read/write/lseek</code> 与引用管理路径	最后一个 fd、映射或内核引用消失后释放
<code>inode</code>	文件类型、长度、权限、时间戳、数据映射与缓存入口	文件系统元数据路径和写回路径	链接与打开引用都消失后才可回收身份
缓存页	页索引、内容、dirty、writeback 与 error 状态	读写路径、回写线程、I/O 完成路径	与映射脱离且无人引用、无人回写后才能回收

一次 `write` 同时推进三个位置。

假设打开文件对象的 `f_pos` 为 4096，用户请求写入 6000 字节。文件页大小为 4096 字节，因此这次请求会修改文件页索引 1 和 2：第一页从页内偏移 0 写到末尾，共 4096 字节；第二页从页首写入 1904 字节。若原文件只有 5000 字节，成功推进全部数据后，文件长度变为 10096。

这里至少有三个位置不能混为一个数值：用户缓冲区已经消费到哪里，打开文件对象的顺序偏移推进到哪里，文件内容的哪几个缓存页被修改。它们通常一起推进，但在缺页、短写或空间不足时可能停在不同边界。



图示：6000 字节写入的字段变化。写入不是“把一个 buffer 交给文件”这一项动作，而是把用户缓冲区进度映射到两个缓存页，并在成功范围确定后推进打开文件偏移与文件长度。dirty 表示内存副本已修改，不表示设备已经完成。

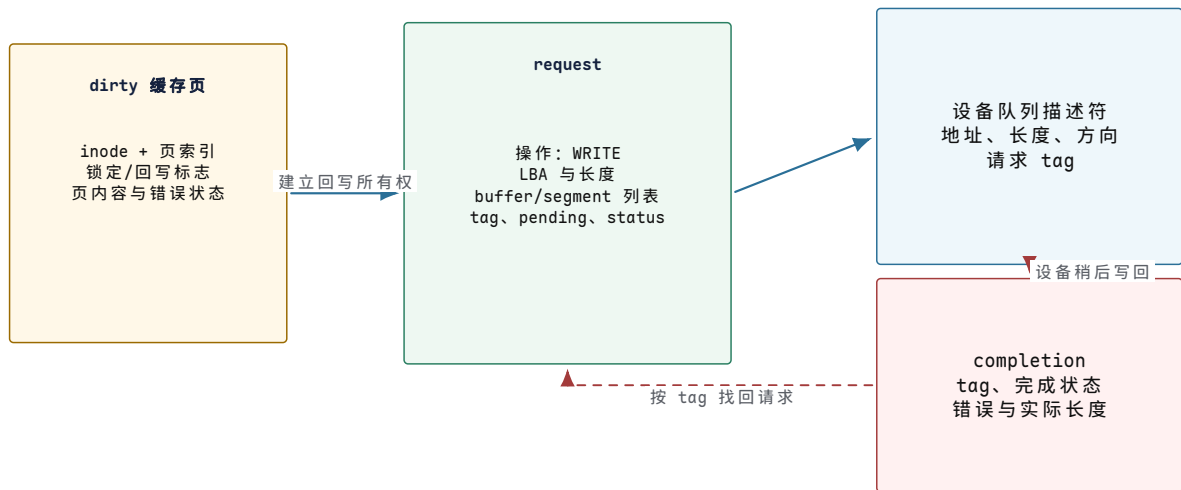
若第二个缓存页准备失败，系统调用可能只返回 4096。此时不能把整个操作回滚成“零字节发生”，因为第一页内容已经进入 page cache；也不能返回 6000，因为后 1904 字节没有被接受。返回值 4096 是本次调用的提交边界，调用者下一次应从 `buf+4096` 继续。顺序文件偏移也只能推进到 8192，不能提前写成 10096。

时刻	用户缓冲进度	打开文件偏移	inode 与缓存页	可返回结果
进入写路径	0/6000	4096	原长度 5000，页 1 含旧内容	尚未决定
页 1 复制完成	4096/6000	暂存为 8192	页 1 已修改并标 dirty，长度至少可到 8192	可形成短写 4096
页 2 准备失败	停在 4096	提交为 8192	页 2 未获得可提交的新内容	返回 4096
两页均完成	6000/6000	提交为 10096	页 1、2 dirty，长度为 10096	返回 6000

这个例子说明“保存状态”不是泛指把若干字段留在内存中。每个字段都回答一个独立问题：`f_pos` 决定下一次顺序访问从哪里开始，`i_size` 决定读者能够到达的文件末端，缓存页的 dirty 位决定回写路径是否必须处理它，系统调用返回值则把本次已经提交的前缀告诉调用者。字段之间需要按同一个成功范围更新，否则重试会重复、遗漏或越过尚未形成的数据。

系统调用返回后，异步请求接管未完成状态。

普通缓冲写可以在数据进入 page cache 后返回。稍后的写回路路径要把多个 dirty 页组织成块请求，并把完成责任交给设备。函数调用栈此时早已返回，未完成信息必须进入长期对象。一个最小请求记录至少需要保存操作类型、设备范围、缓冲区片段、请求身份、剩余子操作数和最终状态；只有请求对象持有这些信息，设备中断才能知道自己完成的是哪项工作。



图示：系统调用返回后的状态承接。request 把缓存页与设备描述符联系起来；completion 只携带足以定位原请求的身份和结果。中断路径按 tag 找回 request，合并子操作状态，最后解除缓存页的 writeback 状态并唤醒等待者。

请求字段	保存的事实	谁写入或更新	何时不再需要
operation、range	读或写、设备起点和长度	块层在提交前确定	所有子操作完成后
segment list	哪些内存区间归本次 DMA 使用	映射与拆分路径建立	设备不再访问且 DMA 解除映射后
tag	completion 应归属哪个在途请求	队列分配器在提交时分配	completion 已消费且 tag 归还后
pending	还有多少子操作尚未结束	提交时设定，完成路径递减	归零时形成请求级完成
status	第一项或聚合后的错误结果	设备完成路径更新	上层读取并传播结果后
通知目标	完成后通知谁或继续哪个阶段	请求建立者登记	通知已经交付、引用已经释放后

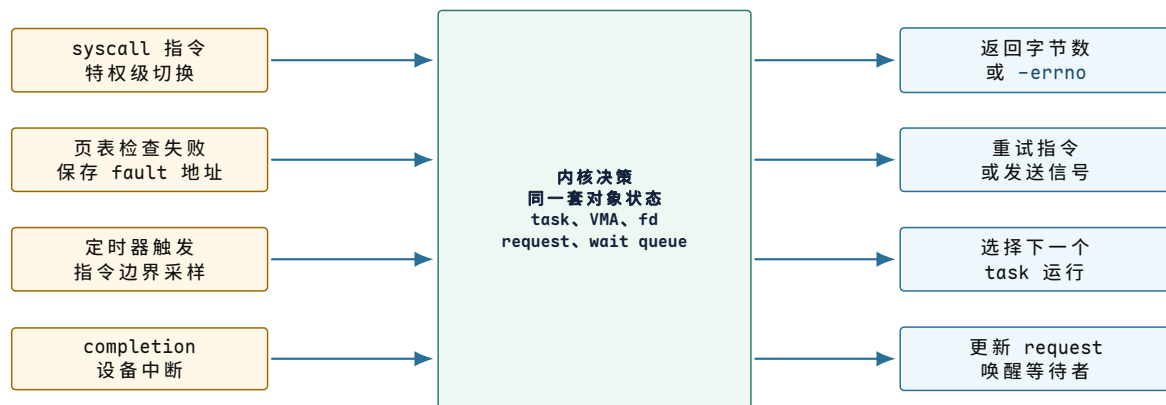
完成路径的提交点不是“中断已经发生”，而是所有属于该请求的完成项都已核对，设备不再拥有 buffer，最终状态已经发布。只有到这一边界，等待 `fsync` 的 task 才能被唤醒，缓存页才能从 writeback 转为 clean 或 error。若过早解除页面所有权，页面可能在 DMA 尚未结束时被改写或回收；若遗漏错误状态，调用者会把设备失败误解释为可靠保存。

核心理想

内核对象的必要性来自时间跨度和共享关系：当一次工作会越过当前函数调用、由另一条执行路径继续，或者需要在失败后说明已经完成的前缀时，相关状态就必须进入有身份、有所有者和有生命周期的对象。字段不是实现装饰，而是后续路径重新取得上下文的依据。

异步事件如何推动任务前进

一次保存请求不是一条顺着函数调用栈走到底的直线。它会被四类入口打断和续上：系统调用入口、缺页入口、定时器入口和设备完成入口。每条路径都从硬件事实进入内核决策，再回到用户能观察到的结果。



图示：四条路径入口不同、结果不同，但中间经过的是同一套内核对象状态。理解其中任何一条，都要能说清它读写了哪些共享状态——这正是后面每一章的展开方式。

第一条路径是系统调用入口。应用看见的是一个 C 库函数，内核看见的是一次受控入口和多个对象状态更新。这里的寄存器名字来自 x86-64 调用约定，用来表达四件事：系统调用号、fd、buffer 地址和长度被交给内核。

```

user:
    rax = SYS_write
    rdi = fd
    rsi = user_buf
    rdx = len
    syscall

kernel entry:
    save user return position and registers
    switch to kernel-owned stack
    locate current task and fd table
    validate fd refers to writable file, socket, or pipe
    validate user buffer range and copy or pin pages
    update target object state
    return byte count or -errno in rax
  
```

阶段	关键检查	失败表达
系统调用入口	syscall number 是否存在，当前上下文是否允许	-ENOSYS、拒绝或信号
fd 查找	fd 是否打开，是否有写权限	-EBADF、-EPERM
用户 buffer	地址格式是否有效、VMA 是否允许读、复制中是否 fault	-EFAULT 或部分写入
目标对象	文件、pipe 或 socket 是否可写、是否阻塞	-EAGAIN、睡眠、信号中断
提交语义	数据进入哪里：page cache、socket buffer、设备队列	返回值不一定表示落盘或发出网络包

这条路径说明了 OS 契约的层次。系统调用返回成功，只说明内核接受并完成了该接口定义下的一部分工作。写普通文件可能只是写入 page cache；写 socket 可能只是进入发送缓冲；写终端可能经过驱动队列和中断。若业务需要“崩溃后仍存在”，还要继续追 `fsync` 和存储 flush；若业务需要“对端已收到”，还要追协议确认。

第二条路径是 page fault。内核按用户地址复制 `user_buf` 时，CPU 可能发现页表里没有可用翻译，于是触发 page fault。硬件做的是检查和转交，内核做的是解释和修复。

CPU:

```
compute the address being loaded
check whether the address form is valid
lookup TLB or walk page table
if missing or permission denied:
    save faulting instruction position and error code
    record fault address
    enter page fault handler
```

kernel:

```
find VMA covering fault address
compare access type with VMA permissions
inspect PTE state
allocate a page, read a file page, copy a shared page, or reject
install PTE and clear stale address-cache entries when needed
return to the faulting instruction or deliver signal
```

硬件看到	内核进一步区分	结果
present=0	地址属于匿名 VMA	分配零页, minor fault
present=0	地址属于文件映射	page cache 命中或发起磁盘 I/O
write denied	PTE 标记 shared-copy	复制页并恢复写权限
execute denied	NX 或 VMA 不可执行	发送信号, 安全拒绝
user bit denied	用户访问内核专用页	发送信号或记录攻击/bug

缺页机制的关键不是“慢”，而是“可恢复边界”。如果 CPU 不能保存出错指令的位置，修好页表后就无法从原指令重试；如果内核不能判断 VMA 意图，present=0 就无法区分合法懒分配和非法访问；如果旧的地址翻译缓存没有清掉，PTE 更新后旧权限仍可能生效。

第三条路径是定时器中断。用户程序可能永远不主动让出 CPU。定时器中断给内核一个周期性入口，让它能统计时间、处理超时并决定是否切换任务。

hardware timer fires:

```
CPU takes interrupt at instruction boundary
entry saves interrupted return position and registers
timer handler increments scheduler clock
expired timers wake sleeping tasks
current task runtime is charged
if time slice or policy says preempt:
    set need_resched
return path calls scheduler before going back to user
scheduler saves current kernel context and restores next
```

这里有两个容易混淆的点。第一，中断处理函数通常不在任意深处直接把用户栈改成另一个任务；它设置调度标志，在安全返回点进入调度。第二，切换任务不是只改一个 current 指针，还可能涉及内核栈、需要保留的寄存器、地址空间和地址翻译缓存。

动作	需要保存什么	保存错误的现象
中断进入	用户返回位置、栈指针和寄存器	返回后用户程序寄存器随机坏
调度统计	当前任务运行时间和状态	CPU 时间不公平或饥饿

切换内核上下文	内核栈指针、必须保留的寄存器	任务从错误调用链恢复
切换地址空间	当前页表身份、必要的翻译缓存处理	任务访问到别人的内存

第四条路径是设备完成。设备 I/O 是异步的。提交请求后，任务可能睡眠；设备通过 DMA 和中断报告完成；内核再把 completion 变成任务可见结果。

```
submit I/O:
  allocate request object
  map buffer for DMA
  fill device-visible descriptor
  publish descriptor in the required order
  notify the device
  if synchronous API:
    put task on wait queue and schedule away

device:
  reads descriptor by DMA
  transfers data
  writes completion entry
  raises interrupt

interrupt/bottom half:
  read completion queue
  unmap DMA and mark request done
  store status and byte count
  wake tasks waiting on request
```

这条路径解释了为什么 I/O 子系统一定有 request identity，也就是“请求身份”。中断到来时，CPU 不知道“哪个用户函数在等它”，只看到某个设备队列有 completion。内核必须通过队列位置、tag、request 指针或 completion entry 找回原请求，撤销 DMA 授权，更新上层对象，再唤醒等待者。没有这层身份映射，异步 I/O 无法可靠对应到发起者。

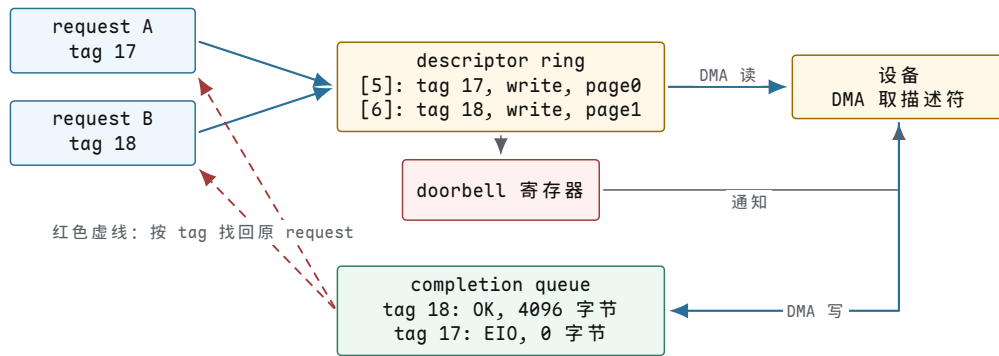
把分析范围集中到一个设备队列。驱动把请求写进一块设备也能读取的 descriptor ring；descriptor 是一条描述符，里面记录 buffer 地址、长度、方向和 tag。doorbell register 是设备暴露给 CPU 的一个通知寄存器；驱动把 descriptor 写好后，再写 doorbell，告诉设备“队列里有新请求”。completion queue 则是设备写回结果的队列。

```
submit A:
  descriptor[5] = { tag: 17, buffer: page0, len: 4096, op: write }
  doorbell = 5

submit B:
  descriptor[6] = { tag: 18, buffer: page1, len: 4096, op: write }
  doorbell = 6

completion queue:
  { tag: 18, status: OK, bytes: 4096 }
  { tag: 17, status: EIO, bytes: 0 }
```

完成顺序不一定等于提交顺序。设备可以先完成 B，再报告 A 失败。若内核只按“第一个 completion 对应第一个等待者”处理，就会把 B 的成功错误地归属于 A，再把 A 的失败归属于 B。tag 的作用就是把设备事实重新接回正确的内核请求对象。



图示：完成顺序不等于提交顺序：B 先成功、A 后失败是合法结果。completion entry 里的 tag 把设备事实重新接回正确的 request 对象，两条红色虚线表示按请求身份匹配结果。

阶段	硬件/驱动看到什么	内核必须维护什么
填 descriptor	buffer 地址、长度、操作类型、tag	request 对象仍然活着，buffer 不能释放
写 doorbell	设备被通知去取 descriptor	descriptor 字段必须已经完整可见
设备 DMA	设备直接读写内存 buffer	页面要被固定或映射，防止被回收复用
写 completion	completion entry 带回 tag 和 status	用 tag 找回 request，记录成功或错误
唤醒等待者	等待队列里的 task 重新变 runnable	醒来后读取 request status，不根据唤醒本身推断结果

这里最危险的错误不是“设备慢”，而是生命周期错位。若 completion 之前释放 buffer，设备可能把数据写进已经分配给别人的页面；若 completion 没有 status，调用者只能知道“有中断”，不知道哪次写失败；若唤醒早于 request status 更新，等待者醒来仍可能读到旧状态。异步设备因此要求 OS 明确三件事：请求身份、buffer 生命周期和完成证据。

还有一种更容易被忽略的情况：等待者已经不愿继续等待，设备却仍在工作。例如，线程等待一次写请求时收到信号，或者超时时间已经到达。系统调用可以先返回，但这只改变了“线程是否继续等待”，并不自动撤销已经交给设备的命令。

发生的动作	它实际改变的状态	不能据此推断什么
等待超时	等待者停止阻塞，准备取得一个超时结果	设备已经停止 DMA
提出取消	请求被标记为希望取消，并尝试通知下层	下层一定来得及接受取消
设备完成	completion 已经带回最终状态	原等待线程仍在原处等待
线程收到信号	系统调用可能提前返回或稍后重启	已提交 I/O 没有产生任何效果

这四个事实必须分别记录。若把“线程不等了”直接等同于“请求不存在了”，内核可能立即释放 request 和 buffer；迟到的设备完成随后仍会使用旧 tag 或旧地址，结果可能是重复完成、访问已释放内存，甚至把完成状态写给后来复用同一槽位的新请求。

一个最小异步请求可以按以下状态推进：

```
NEW
-> SUBMITTED
-> COMPLETED
-> REAPED

SUBMITTED
-> CANCEL_REQUESTED
-> COMPLETED           // completion may report cancelled or already done
-> REAPED
```

`CANCEL_REQUESTED` 不是终态。它表示取消意图已经登记，但设备仍可能报告“取消成功”、“请求早已完成”或普通 I/O 错误。只有收到确定的下层结果，或者驱动已经通过复位等协议证明设备不会再访问该请求，内核才可以进入 `COMPLETED`，随后在引用全部释放后进入 `REAPED`。

取消与完成还可能同时发生。正确处理需要一个共同的状态转换规则，保证只有一条路径取得“最终完成权”。完成路径负责写入最终状态和字节数，取消路径只能在成功取得相应状态转换后宣布取消；失败的一方读取已经存在的最终结果，而不能再次释放对象或再次唤醒同一完成事件。

竞争顺序	合法结果	必须保持的不变量
取消先被设备接受	completion 返回取消状态	buffer 在取消完成前仍保持映射
完成先到达	取消请求返回“已完成”	原 I/O 结果不能被取消状态覆盖
线程先超时返回	request 留在内核等待迟到 completion	调用者离开不导致 request 提前释放
队列槽位准备复用	旧请求已经完成并解除设备访问权	旧 completion 不能命中新请求

最后一行通常还需要代际编号或不会过早复用的 tag。只有队列位置而没有代际信息时，旧请求的迟到完成可能与新请求使用同一个编号；内核必须能识别“这是上一代请求的结果”，将其隔离或报告为设备协议错误。

保存操作也遵守这一规则。若等待 `fsync` 时线程被信号打断，应用程序不能只凭“系统调用提前返回”判断介质上一定没有新数据；写回可能继续并最终成功，也可能在稍后记录错误。应用需要依据接口定义决定重试、查询错误还是放弃发布，而内核必须继续维护请求，直到下层不再引用相关页面和对象。超时、取消和完成的分离，是所有异步 I/O、网络请求和设备驱动共同遵守的生命周期边界。

四条路径讲完，可以进一步比较它们进入内核时携带的证据和最终交给哪类机制处理：

路径	进入内核时的主要证据	后续处理
系统调用	调用号、参数寄存器、用户返回现场	分发、参数验证、权限检查与返回
缺页异常	故障地址、访问类型、当前页表状态	VMA 查找、权限判断、建立映射或报告错误
定时器中断	当前任务、运行时间记录和抢占状态	计时、调度决策与可能的任务切换
设备完成	完成队列项、请求标识和状态码	找回请求、转移缓冲区所有权并唤醒等待者

四条路径的共同点是：入口只提供作出决定所需的证据，真正的语义由内核对象和状态机完成。后续章节会分别展开这些决策如何保持权限、生命周期与失败边界。

操作系统抽象依赖的系统条件

内核对象必须建立在硬件提供的条件之上。CPU 能保存执行位置，因此 task 可以睡眠后恢复；页表能够检查地址权限，因此内核可以拒绝非法 buffer；设备以异步方式完成请求，因此内核需要维护 request 和 completion；存储可能在任意时刻失去供电，因此文件系统必须定义提交边界。

硬件条件	操作系统抽象	关键问题
程序计数器、寄存器和栈可保存	task 上下文	切走后如何从同一位置恢复
特权级限制敏感指令	用户态/内核态	用户如何安全请求内核服务
异常入口能保存出错位置	page fault、signal、debug	错误能否修复并返回原指令
定时器可强制中断	抢占式调度	不合作任务如何被收回 CPU
MMU 检查页表权限	地址空间	进程如何互相隔离又能共享
cache/TLB 需要维护	性能和一致性策略	为什么迁移、锁和取消映射有成本
设备寄存器控制设备	驱动模型	寄存器副作用和顺序如何管理
DMA 异步访问内存	高性能 I/O	buffer 何时归 CPU，何时归设备

看到“进程”，需要同时看到一组保存的 CPU 上下文、地址空间和内核对象，而不只是 pid。看到“文件”，需要同时看到 fd 表、打开文件对象、inode、page cache、块层和设备队列，而不只是路径。看到“权限”，需要同时看到 CPU 特权级、页表权限、文件权限和对象引用，而不只是口令或用户 ID。

用户接口词、内核对象词和硬件词经常指向同一条路径的不同层。用户接口会说 process、file descriptor、signal、pipe、socket；内核对象会说 task、file、inode、wait queue；硬件路径会说寄存器、页表、中断、DMA、设备寄存器。三套词不是互相替代，而是共同描述一次动作怎样穿过系统。

用户语义	内核对象	硬件支点	关键边界
process	task、地址空间、fd table	寄存器、页表、定时器	身份、地址空间、权限分离
thread	task、内核栈、共享地址空间	寄存器、栈指针	可调度上下文，不复制全部资源
fd	fd table、打开文件对象	设备队列、page cache、DMA	整数句柄不是文件本体
signal	待处理信号、用户返回现场	保存的异常/中断现场	异步交付但必须恢复用户现场
socket	socket 对象、wait queue	网卡队列、中断、DMA	协议状态和设备完成分离

timer	定时器对象、等待队列	硬件定时器中断	未来事件需要硬件唤醒入口
-------	------------	---------	--------------

例如 `fd` 是一个用户态整数，但内核要通过当前进程的 `fd table` 找到打开文件对象，再通过 `file operations` 调到 `pipe`、`socket`、`inode` 或 `device` 的具体实现。若底层是网卡，最终还会走网络缓冲区、驱动队列、DMA 和 `completion`。把 `fd` 当成“文件本体”，就解释不了 `dup` 后共享 `offset`、`socket readiness`、设备拔出后旧 `fd` 返回错误这些现象。

单片机和裸机程序能暴露 OS 的原始需求。一个裸机程序拥有全部内存和设备，所有代码同权，主循环靠自觉返回，中断直接修改全局状态。高级机制出现之前，失败通常已经在这种小系统里出现过。

裸机阶段的问题	局部处理	通用 OS 机制
函数忙等设备	改成状态机或中断	阻塞、等待队列、 <code>completion</code>
任务靠自觉返回	任务表和显式 <code>yield</code>	线程和协作式调度
任务可能不合作	加硬件定时器	抢占和时间片
全局变量被中断处理函数打断	关中断临界区	锁、原子操作、内存屏障
所有代码同权	约定模块边界	CPU 特权级和系统调用
所有内存同权	手工内存布局	MMU、地址空间、页表权限
CPU 搬每个字节	DMA 控制器	异步 I/O 和驱动队列

如果不知道硬件能否强制中断、能否区分用户/内核、能否检查页表权限，就无法判断一个 OS 设计到底是“规范约定”还是“硬件保证”。

用户可见、内核完成与持久化边界

保存请求走到这里，已经不是一段顺序代码，而是一串状态和边界。每个边界都回答一个具体问题：对象是谁，谁拥有它，哪个入口能修改它，不能继续时停在哪里，完成时用什么证据说明结果可信。

问题	保存路径中的内容
状态对象	进程、线程、页、 <code>fd</code> 、 <code>inode</code> 、 <code>socket</code> 、设备请求、日志记录
所有者	用户态对象、内核对象、硬件寄存器、设备控制器
入口	系统调用、异常、中断、内核线程、后台回收
权限	特权级、页表权限、文件权限、资源能力、命名空间、引用计数
等待	<code>runqueue</code> 、 <code>futex</code> 、 <code>page fault</code> 、I/O <code>completion</code> 、锁、条件变量
提交	状态更新、引用释放、写入 <code>page cache</code> 、落盘、目录项发布生效
证据	返回值、 <code>errno</code> 、 <code>trace</code> 、计数器、日志、测试断言

例如一次 `write(fd, buf, len)` 可以这样拆：用户态只有一个整数 `fd` 和一段用户 `buffer`；内核要先从进程 `fd` 表找到打开文件对象，再检查写权限，再复制用户 `buffer`，再把页标记为 `dirty`，最后返回写入字节数或错误。这个返回值通常不等于数据已经安全落盘。要证明持久化，还需要 `fsync`、目录同步或更高层提交记录。

再看一次 page fault。用户态执行 load/store，硬件发现页表项不存在或权限不符，CPU 进入异常入口，内核根据 VMA 判断这是合法缺页、权限错误还是非法地址。合法缺页可能分配物理页或读取文件页；权限错误可能发送信号；非法访问可能终止进程。这里的核心不是“缺页很慢”，而是虚拟地址访问被拆成硬件检查和内核决策。

只看正常路径仍然不够。OS 机制中许多错误，恰好在边界条件下暴露。

机制	正常路径	必须补的反例
系统调用	参数合法，返回字节数或结果	用户指针跨页且后一页不可读，返回 <code>-EFAULT</code> 或部分成功
调度	runnable 任务被选中运行	任务睡在 wait queue 中，不能因为优先级高就运行
锁	持锁者修改共享状态后释放	两把锁顺序反过来，形成 wait-for 环
缺页	VMA 合法，分配或读入页面	地址不属于任何 VMA，必须发信号而不是分配页
DMA	completion 到达后回收 buffer	completion 前释放 buffer，设备可能写坏别人的内存
持久化	日志提交后恢复可重放	写了数据但没写 commit，崩溃后不能当成功

一个机制是否成立，先看没有它时哪里坏，再看新增了什么状态对象，最后看正常路径和反例能不能守住边界。

层次	必须回答的问题	例子
原始失败	没有这个机制时，哪个具体动作会坏	死循环霸占 CPU；用户指针越界；写一半掉电
状态对象	内核新增哪本账，记录哪些字段	task state、runqueue、VMA、PTE、request、journal record
状态转移	谁能修改状态，入口是什么	timer interrupt 改调度状态；page fault 安装 PTE；completion 完成 request
不变量	哪些关系不能被破坏	sleeping 任务不能在 runqueue；DMA 完成前 buffer 不能释放
反例测试	怎样故意触发边界	lost wakeup、非法地址、共享页写入、I/O error、崩溃恢复

例如调度不能只列 FCFS、RR、CFS，而要回答：死循环为什么能霸占 CPU；定时器如何给内核抢占入口；task 在 runnable、running、blocked 之间怎样转移；runqueue 用队列、树或多队列时复杂度怎么变；lost wakeup 和饥饿怎样测出来。虚拟内存也一样：直接物理地址会互相破坏，VMA 表达地址区间意图，PTE 记录单页翻译事实，page fault 把硬件证据交给内核决策，非法地址、共享页写入、权限错误和 TLB 失效分别压在不同边界上。

保存路径最终会落到五个维度。

维度	追问	例子
身份	对象如何被引用	pid、tid、fd、inode number、socket 四元组、DMA tag
权限	谁能做什么	页表读写执行位、fd 标志、资源能力、命名空间
生命周期	何时创建和释放	fork/exit/wait、open/dup/close、page refcount、request completion
等待	无法立即推进时停在哪里	runqueue、wait queue、page fault、I/O queue、timer
提交	何时对外可见或持久	系统调用返回、PTE 安装、DMA 完成、fsync、提交记录

只说其中一个维度，很容易把机制讲偏。比如“socket 可读”只覆盖了等待维度，还要继续追：哪个 fd 引用它，谁有权限读，缓冲区里数据生命周期如何，读取后队列怎样变化，关闭时如何唤醒等待者。

这些维度还要放回层次关系中。操作系统经常呈现层次结构：硬件在底层，内核抽象在中间，用户 API 在上层。但真实问题会反向穿透层次。一个 `write` 系统调用看似属于文件 API，却可能等待内存回收、块层队列、驱动中断、设备 flush 和调度器唤醒。一个 `page fault` 看似属于内存管理，却可能读取文件页，触发块设备 I/O。一个安全拒绝看似属于权限系统，却依赖路径解析、命名空间和当前进程的身份信息。

```

user write()
-> fd table
-> open file object
-> page cache
-> writeback
-> block layer
-> driver DMA
-> interrupt completion
-> wakeup

```

真实系统不是互不相干的模块，而是交叉依赖的状态机网络。

保存请求不能只是一段普通函数代码，因为它至少要穿过五个关口。

关口	要解决的问题	保存案例中的落点
机器能保存状态	CPU、寄存器、栈、时钟和设备寄存器如何形成可恢复现场	task 能从睡眠处回来，设备 ready 位能被观察
机器能被打断	中断、异常和定时器怎样把外部事件交给内核	设备完成能唤醒等待者，死循环能被抢占
地址能被检查	MMU、页表、VMA 和 PTE 怎样区分合法缺页和非法访问	用户 buffer 跨页时，内核知道复制、补页还是拒绝
资源能被引用	fd、打开文件对象、inode、socket、request 怎样保存一次操作的状态	整数 fd 找到文件对象，I/O completion 找回请求

结果能被证明	page cache、writeback、journal、fsync 和 rename 怎样推进提交边界	“保存成功”从函数返回推进到崩溃后可恢复
--------	--	----------------------

这些关口也可以压成一张“失败、机制、代价”表：

失败	需要的机制	新增代价
固定电路不通用	存储程序 CPU	指令编码、取指译码、异常语义
单核状态无法被安全共享	多核一致性和原子操作	cache line 迁移、内存序、锁竞争
程序直接用物理地址	MMU 和虚拟地址	页表、TLB、缺页成本
CPU 搬运所有 I/O 数据	DMA 和描述符队列	buffer 生命周期、IOMMU、completion
轮询浪费 CPU	中断	异步共享状态
任务死循环霸占 CPU	定时器抢占	上下文保存和调度开销
任务互相覆盖内存	地址空间和权限	页表、缺页、TLB 成本
用户可直接改写设备	系统调用和特权级	入口检查和复制成本
I/O 等待拖住计算	阻塞队列、异步和 completion	请求身份和 buffer 生命周期
写一半崩溃	日志、fsync、提交记录	写放大和恢复复杂度

OS 测试不能只检查程序是否完成执行，还要验证契约是否得到维持。正常路径、错误路径和边界路径都应能够指出具体的不变量以及违反它的条件。

验收标准

- 系统调用能拆成用户态入口、内核对象、权限检查、状态更新和返回值。
- “函数返回成功”“内核接受请求”“设备完成”“业务提交生效”不能混成同一个完成点。
- 等待现象必须落到具体队列：CPU、锁、页面、I/O、子进程都不是同一种等待。
- 隔离边界必须落到硬件或内核对象：地址空间、文件权限、用户/内核态、命名空间各管不同问题。
- 任意机制都应有状态、入口、失败和证据四列。

反例同样重要。若仍然把 fd 当成文件本体，把 write 成功当成落盘，把线程 ready 当成 running，把 malloc 成功当成物理页已分配，把 notify_one 当成消息投递，就说明 OS 契约还没有建立。

沿保存路径串联全书

本章每个关键断言都可以在一台真实 Linux 机器上复查。下面这组实验不需要特殊环境；本章展示的输出就是在本书实验机（aarch64 容器）上跑出来的，你的数字可能不同，但边界关系应当一致。

1. 坏地址实验：把 `write` 的 `buffer` 指向 `0x1`，观察程序打印 `n=-1 errno=14 (Bad address)`；再用 `strace -e trace=write` 对照系统调用边界上的真实一行。
2. `offset` 归属实验：分别跑 `dup`、重新 `open`、`fork` 三个版本，用 `od -An -c data` 验证最终内容是 `CB`、`B`、`CP`，并解释差异来自打开文件对象的共享关系。
3. `fd` 观察：让 `fd_demo` 睡眠，读 `/proc/<pid>/fd` 和 `/proc/<pid>/status`，确认两个 `fd` 指向同一个文件、进程状态是 `S (sleeping)`。
4. 完成点距离实测：给 64KiB 的 `write` 和紧随其后的 `fsync` 分别计时，比较两者差几个数量级；换存储介质（`tmpfs`、SSD、机械盘）再测一次，观察差距怎样变化。
5. 崩溃点推演：在完成点时间线上任选一点假设“断电”，写出重启后用户、内核、设备各自能看到什么，并说明哪一层的持久记录足以支撑恢复结论。

完成这五项检查后，本章的契约就不再只是记忆结论，而是经过反例检验后重新建立的推理结果。

操作系统不是彼此孤立的子系统集合，而是在维护一组可验证契约。分析任何机制，都应先找出缺少它时会失败的具体动作，再问内核新增了哪些状态对象，谁能通过哪个入口修改这些状态，不能继续时等待在哪里，成功和失败分别留下什么证据。保存按钮只是一个入口；同样的方法也适用于调度、虚拟内存、文件系统、网络和设备驱动。

能力清单

验收标准

- 能从晶体管开关、逻辑门、寄存器和时钟解释状态机为什么能成为 CPU 的基础。
- 能手跑 X64S 教学子集的 `mov/add/jmp/syscall` 程序，说明 `RIP`、`RSP`、寄存器、`RFLAGS`、`CPL` 如何变化。
- 能从轮询设备推导出中断，从死循环推导出定时器抢占，从同权代码推导出特权级和系统调用。
- 能画出一条 x86-64 `load/store` 指令经过 `RIP`、寄存器、`ALU`、`MMU`、`TLB`、`cache` 和异常入口的路径。
- 能区分 `ISA`、微架构和 `ABI`，并说明 OS 哪些代码依赖 `ISA`，哪些性能现象来自微架构。
- 能解释总线、`MMIO`、`DMA`、`IOMMU`、中断控制器和设备描述符环怎样共同完成一次 `I/O`。
- 能说明特权级限制了哪些指令和状态，为什么系统调用必须是受控陷入而不是普通函数调用。
- 能把一次系统调用拆成用户态入口、内核对象、权限检查、状态更新和返回值。
- 能解释线程为何没有推进：等 CPU、等锁、等页面、等 `I/O`、等信号还是等子进程。
- 能解释从复位向量到 `kernel_main` 的启动阶段和每个阶段禁止事项。
- 能说明中断上半部、下半部、定时器 `tick`、抢占点和上下文保存。
- 能判断自旋锁、`mutex`、`semaphore`、`RCU` 的上下文限制，并用锁顺序图解释死锁。
- 能解释 `exec` 的 `ELF` 验证、段映射、用户栈构造、动态解释器和提交点。
- 能手算一次虚拟地址翻译，并区分合法缺页、权限错误和非法地址。
- 能把一次 x86-64 `#PF` 拆成 `CR2`、`error code`、`VMA` 查找、`PTE` 判定、`COW/page cache` 修复、`TLB` 失效和返回重试。
- 能画出 `VMA`、`PTE`、`page cache`、`inode` 的关系，并区分 `MAP_SHARED` 与 `MAP_PRIVATE`。
- 能比较 `buddy`、`slab`、`kmalloc size class` 的用途、复杂度和碎片成本。
- 能画出驱动描述符环、`DMA` 映射、`completion` 和 `buffer` 生命周期。
- 能说明 `bio`、`request`、`request queue`、`flush`、`FUA` 和 `barrier` 如何共同影响 `I/O` 顺序。
- 能画出 `fd`、`open file description`、`inode`、`page cache` 和设备请求之间的对象关系。
- 能解释 `inode`、目录项、空闲块 `bitmap`、`VFS` 和 `page cache` 如何协作。
- 能解释 `socket` 作为 `fd` 的生命周期，说明 `bind`、`listen`、`accept`、`connect`、`send`、`recv` 的等待语义。

- 能分析 `capability`、`namespace`、`cgroup`、`seccomp` 如何共同形成最小权限边界。
- 能区分 `errno`、信号、`WARN`、`oops`、`panic`，并为系统调用写出失败回滚路径。
- 能为可靠写出设计 `prepared`、`syncd`、`renamed`、`manifest committed` 的提交状态机。
- 能写出包含现象、假设、证据、反证和未验证边界的 OS 调试报告。

延伸阅读说明

本册正文由五部分、十八个编号章组成，主线限定为操作系统的对象、机制与边界。文件系统内容按照“格式形成、身份与索引、缓存与恢复、现代设计”拆成四章，使概念推导和故障边界不被压缩在一个超长章节中。计算机组成、数字电路和处理器实现已有独立卷册，因此不再作为本册的前置章节重复展开。正文遇到特权级、页表、中断、cache、DMA 或 IOMMU 时，只保留理解当前内核机制所必需的硬件条件。

延伸阅读可以按问题类型选择，不要求与正文连续阅读：

阅读方向	适合进入的时机	主要目的
计算机硬件基础	对指令执行、地址翻译、中断或设备互联的硬件过程仍有疑问时	查阅独立硬件卷，补足处理器、存储层次、总线和设备接口原理
裸机与内核启动	需要理解复位入口、链接布局、早期栈和内核初始化时	建立从固件交接到内核接管处理器的启动路径
Linux 实现阅读	已经理解本册给出的对象、不变量和失败边界之后	将教材中的 task、VMA、inode、request 等概念映射到真实内核实现
模型与实验	需要验证并发、内存、I/O 或恢复机制时	通过状态机、故障注入和性能测量检查机制是否满足预期契约

核心思想

延伸阅读用于回答正文中已经明确的问题，而不是替代操作系统主线。进入更深层实现之前，应先能说明相关机制的状态对象、权限边界、等待关系、提交条件和失败路径。

参考资料与版本基线

本册的机制解释以稳定概念为主；涉及 Linux 接口、调度策略、文件系统特性和持久化语义时，应回到带版本的主资料核对。下面列出本版技术校审使用的主要资料，核对日期为 2026 年 7 月 22 日。引用随版本变化的实现细节时，应同时注明适用的内核版本。

Linux 内核文档

Linux 6.16 文档总入口：<https://www.kernel.org/doc/html/v6.16/>。

公平调度

EEVDF 调度器说明：<https://www.kernel.org/doc/html/latest/scheduler/sched-eevdf.html>；CFS 历史设计：<https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html>。

用户指针访问

`copy_to_user/copy_from_user` 返回值和上下文约束：<https://docs.kernel.org/6.11/kernel-hacking/hacking.html>。

块设备粒度

Linux block queue 的 logical block size、physical block size 与 minimum I/O size：<https://www.kernel.org/doc/html/v6.16/core-api/kernel-api.html>。

持久化

Linux `fsync(2)`，包括文件与父目录持久化边界：<https://man7.org/linux/man-pages/man2/fsync.2.html>。

page cache 与映射接口

Linux 文件系统锁约定（含 folio 与失效路径）：<https://docs.kernel.org/next/filesystems/locking.html>；iomap 操作与端口迁移说明：<https://docs.kernel.org/filesystems/iomap/operations.html>、<https://docs.kernel.org/filesystems/iomap/porting.html>。

ext4 与 jbd2

ext4 journal 的磁盘格式、大小端、事务块与 revoke 记录：<https://docs.kernel.org/6.17/filesystems/ext4/journal.html>。

Btrfs

磁盘格式与设计资料：<https://btrfs.readthedocs.io/en/latest/dev/On-disk-format.html>、<https://btrfs.readthedocs.io/en/stable/dev/dev-btrfs-design.html>；RAID profile 与 RAID56 限制：<https://btrfs.readthedocs.io/en/latest/mkfs.btrfs.html>、<https://btrfs.readthedocs.io/en/stable/btrfs-man5.html>；在线碎片整理及其对 extent sharing 的影响：<https://btrfs.readthedocs.io/en/latest/Defragmentation.html>。

掉电测试

Linux device-mapper `log-writes` target，用于记录写入与在标记点重放：<https://docs.kernel.org/6.4/admin-guide/device-mapper/log-writes.html>。

F2FS

F2FS 设计、checkpoint、后台 GC 与前台 GC：<https://www.kernel.org/doc/html/v6.16/filesystems/f2fs.html>。

开放文件系统文档

OpenZFS 文档：<https://openzfs.github.io/openzfs-docs/>；Microsoft 文件系统协议资料：https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-fscc/efbfe127-73ad-4140-9967-ec6500e66d5e；Apple File System Reference：<https://developer.apple.com/support/downloads/Apple-File-System-Reference.pdf>。

验收标准

引用的最低标准不是“网页存在”，而是每个易变化断言都能回答三个问题：针对哪个版本，适用哪些配置，原始资料具体支持哪一部分结论。性能数字还必须同时给出硬件、工作负载、队列深度、缓存与持久化设置。